

Python: Pandas на практике

200 упражнений
по анализу данных
с решениями
и пояснениями

Реувен Лернер



MANNING



ОМК
издательство

Реувен Лернер

Python: Pandas на практике

Pandas Workout

200 exercises to make you
a stronger data analyst

Reuven M. Lerner



MANNING
SHELTER ISLAND

Python: Pandas на практике

200 упражнений по анализу данных
с решениями и пояснениями

Реувен Лернер



Москва, 2025

УДК 004.438Python
ББК 32.973.2
Л49

Л49 Реувен Лернер

Python: Pandas на практике. 200 упражнений по анализу данных с решениями и пояснениями / пер. с англ. А. Ю. Гинько. – М.: ДМК Пресс, 2025. – 552 с.: ил.

ISBN 978-5-93700-227-3

Сегодня трудно представить аналитика данных, не пользующегося библиотекой Pandas, но в тонкостях работы с ней немудрено запутаться. В этой книге собраны упражнения, основанные на многолетней преподавательской практике автора. Прочитав ее, вы будете чувствовать себя уверенно при встрече с недостатками реальных данных в виде пропущенных значений, смешанных форматов и отсутствия четкой структуры.

Книга предназначена начинающим аналитикам данных, изучающим Pandas, но будет полезна и опытным специалистам, стремящимся отточить свои навыки.

Copyright © 2024 Manning Publications. This translation is published and sold by permission of Manning Publications, the owner of all rights to publish and sell the same.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

Материал, изложенный в данной книге, многократно проверен. Но, поскольку вероятность технических ошибок все равно существует, издательство не может гарантировать абсолютную точность и правильность приводимых сведений. В связи с этим издательство не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-1-61729-972-8 (англ.)
ISBN 978-5-93700-227-3 (рус.)

Copyright © 2024 by Manning Publications Co.
© Оформление, перевод на русский язык,
издание, ДМК Пресс, 2025

В память об отце
Рабби Барри Дов Лернер (1942–2023)

Он научил меня:

- быть чрезвычайно любознательным;
- делиться всем, чему научился;
- верить в людей;
- делать все это с юмором.

Оглавление

Предисловие	9
Благодарности	11
Об этой книге	12
Об авторе	17
О переводчике	17
Об изображении на обложке	18
Глава 1. Объект Series	19
УПРАЖНЕНИЕ 1. Оценки за ежемесячные тесты	24
УПРАЖНЕНИЕ 2. Масштабирование оценок	37
УПРАЖНЕНИЕ 3. Считаем цифры разряда десятков	42
УПРАЖНЕНИЕ 4. Описательная статистика	52
УПРАЖНЕНИЕ 5. Температура по понедельникам	56
УПРАЖНЕНИЕ 6. Пассажиропоток в такси	60
УПРАЖНЕНИЕ 7. Длинные, средние и короткие поездки в такси	63
Заключение	67
Глава 2. Объект DataFrame	68
УПРАЖНЕНИЕ 8. Чистый доход	73
УПРАЖНЕНИЕ 9. Налоговое планирование	77
УПРАЖНЕНИЕ 10. Добавление новых товаров	88
УПРАЖНЕНИЕ 11. Лидеры продаж	94
УПРАЖНЕНИЕ 12. Поиск выбросов	97
УПРАЖНЕНИЕ 13. Интерполяция	104
УПРАЖНЕНИЕ 14. Выборочное обновление	108
Заключение	112
Глава 3. Импорт и экспорт	113
УПРАЖНЕНИЕ 15. Загадочные поездки на такси	117
УПРАЖНЕНИЕ 16. Такси и пандемия	125
УПРАЖНЕНИЕ 17. Установка типов данных для столбцов	134
УПРАЖНЕНИЕ 18. Файл passwd в датафрейм	138
УПРАЖНЕНИЕ 19. Курсы биткоина	142
УПРАЖНЕНИЕ 20. Большие города	148
Заключение	151
Глава 4. Индексы	152
УПРАЖНЕНИЕ 21. Парковочные талоны	154
УПРАЖНЕНИЕ 22. Оценки за вступительные тесты	167

УПРАЖНЕНИЕ 23. Олимпийские игры	172
УПРАЖНЕНИЕ 24. Олимпийские сводные таблицы	185
Заключение	192
Глава 5. Очистка данных.....	193
УПРАЖНЕНИЕ 25. Очистка данных о парковках	197
УПРАЖНЕНИЕ 26. Уход знаменитостей.....	207
УПРАЖНЕНИЕ 27. Титаник и интерполяция	214
УПРАЖНЕНИЕ 28. Несогласованные данные	220
Заключение	227
Глава 6. Группировка, объединение и сортировка	228
УПРАЖНЕНИЕ 29. Самые продолжительные поездки на такси	232
УПРАЖНЕНИЕ 30. Сравним поездки на такси	243
УПРАЖНЕНИЕ 31. Расходы туристов по странам	256
Заключение	266
Глава 7. Сложная группировка, объединение и сортировка	267
УПРАЖНЕНИЕ 32. Температура в разных городах	267
УПРАЖНЕНИЕ 33. Оценки за вступительные тесты, часть 2	279
УПРАЖНЕНИЕ 34. Снежные и дождливые города	293
УПРАЖНЕНИЕ 35. Вино и туризм... ..	302
Заключение	313
Глава 8. Промежуточный проект	314
Задача	315
Заключение	336
Глава 9. Строки	337
УПРАЖНЕНИЕ 36. Анализируем Алису	343
УПРАЖНЕНИЕ 37. Винные слова.....	350
УПРАЖНЕНИЕ 38. Зарплата программистов	360
Заключение	373
Глава 11. Даты и время	374
УПРАЖНЕНИЕ 39. Короткие, средние и длинные поездки на такси.....	381
УПРАЖНЕНИЕ 40. Пишем и читаем даты	388
УПРАЖНЕНИЕ 41. Цены на нефть.....	397
УПРАЖНЕНИЕ 42. Чаевые за поездки на такси.....	402
Заключение	412
Глава 11. Визуализация	413
УПРАЖНЕНИЕ 43. Города	416
УПРАЖНЕНИЕ 44. Погода в ящиках с усами	430
УПРАЖНЕНИЕ 45. Анализируем стоимость поездок на такси с помощью графиков	439
УПРАЖНЕНИЕ 46. Машины, нефть и мороженое	455

УПРАЖНЕНИЕ 47. Такси и визуализация в Seaborn.....	474
Заключение	483
Глава 12. Оптимизация	484
УПРАЖНЕНИЕ 48. Категории	490
УПРАЖНЕНИЕ 49. Быстрое чтение, быстрая запись.....	497
УПРАЖНЕНИЕ 50. query и eval	507
Заключение	515
Глава 13. Итоговый проект	516
Задача	516
Столбцы и их описание	519
Заключение.....	548
Предметный указатель	549

Предисловие

Когда я только начинал преподавать Python в компаниях по всему миру, я не был удивлен тем, как мои студенты используют этот язык программирования. Обычно они применяли его для написания скриптов взамен менее выразительных Bash-скриптов, создания серверных веб-приложений, разработки автоматизированных тестов и работы с реляционными базами данных.

Спустя какое-то время я с удивлением обнаружил, что они также используют Python для анализа данных. Да, это мощный и легкий в освоении язык, но с быстрым действием у него всегда были проблемы. Как же с его помощью можно анализировать данные?

Вскоре я узнал то, что многие уже знали: оказывается, библиотека NumPy способна объединить легкость использования Python с эффективностью C. Я быстро вскочил в этот вагон и уже совсем скоро начал активно применять эту связку в аналитике и обучать тех, кому только предстояло сделать для себя такое открытие. В то же время сам NumPy мне казался чересчур низкоуровневым инструментом для достижения моих целей.

Когда я познакомился с pandas, все встало на свои места. Эта библиотека объединила в себе скорость и эффективность NumPy с богатейшим API, позволяющим легко выполнять задачи, встающие перед аналитиком каждый день. Я люблю сравнивать pandas с машиной, оснащенной автоматической коробкой передач, значительно превосходящей в удобстве автомобиль с ручной коробкой, коим мне представляется NumPy. Pandas позволяет просто и быстро выполнять чтение и запись в самых разных форматах, очищать исходные данные, анализировать и визуализировать их. В общем, он дал мне все, что было нужно. Я был пленен его очарованием.

Спустя десятилетие после моего знакомства с pandas интерес к нему возрос до небес. Сегодня трудно представить себе аналитика данных, не пользующегося этой библиотекой. За это время где я только ни читал свои курсы по pandas – от команд стартапов до государственных учреждений и от небольших хеджевых фондов до компаний, входящих в первую сотню мирового рейтинга.

Pandas подходит к решению задач иначе по сравнению со стандартными библиотеками, входящими в состав Python. Синтаксис один, но структуры данных и принципы работы с ними совершенно разные. При этом библиотека pandas настолько обширна и разнообразна, что в ее хитросплетениях немудрено запутаться. В отличие от базового Python, незримо продвигающего догму «Должен быть только один способ решения задачи...», pandas не исключает множества вариантов и подходов к одной и той же проблеме. В то же время бывает непросто определить, какой из возможных способов окажется наиболее быстрым и простым в эксплуатации, даже (или особенно) если вы обладаете приличным опытом работы с Python.

Именно по этой причине я являюсь большим поклонником практического подхода к обучению. Только практика позволит вам проникнуть во все тайные

комнаты библиотеки `pandas` и научиться применять обнаруженные приемы для решения своих задач. Вам недостаточно тренироваться на вымышленных наборах данных – если вы хотите действительно хорошо освоить `pandas`, вам придется, вооружившись этой библиотекой и изрядной долей храбрости, подступаться к обработке и анализу данных из реального мира с характерными для них недостатками в виде пропущенных значений, смешанных форматов и отсутствия четкой структуры.

Упражнения, собранные в этой книге, проистекают из моих курсов и лекций, которые я читаю все последнее десятилетие. Многие из них за эти годы претерпели изменения, которые сделали их только лучше, поскольку в них были учтены все сложности, с которыми могут сталкиваться начинающие разработчики. Моя цель – дать вам возможность попрактиковаться с `pandas` и приобрести навыки, которые вы сможете успешно применить в своей работе. Подобно тому как каждый учебный полет пилота на авиасимуляторе приближает его к подъему в воздух настоящего самолета с пассажирами, каждое упражнение из этой книги позволит вам чувствовать себя при работе с `pandas` более уверенно, и вы не будете испытывать проблем при встрече с реальными задачами.

Благодарности

Написанием этой книги я обязан большому количеству людей.

Хотя на обложке книги красуется только мое имя, очень многие в издательстве Manning Publications оказывали мне бесконечную (и терпеливую) поддержку в процессе ее создания. В первую очередь я хотел бы поблагодарить Майка Стивенса (Mike Stephens), вдохновившего меня на написание второй книги (первая была посвящена Python), и Фрэнсис Лефковиц (Frances Lefkowitz), которая знает, где и как нужно надавить, чтобы процесс написания книги пошел легче. Также я благодарен ей за советы, связанные с редактурой. Кроме того, я получил немало дельных советов от технического рецензента Нинослава Черкеза (Ninoslav Cerkez).

Несколько десятков человек выразили желание помогать комментариями к книге в процессе ее написания и редактуры. Их советы помогли мне значительно улучшить книгу и сделать код в упражнениях и описания более выразительным. Я очень благодарен тем, кто купил книгу на стадии предварительного релиза (MEAP) и оставлял свои комментарии в системе liveBook от Manning.

Также хотелось бы сказать спасибо создателям сайта Pandas Tutor (<https://pandastutor.com>) за возможность красиво визуализировать запросы в pandas, подобно тому как это происходит на сайте Python Tutor (<https://pythontutor.com>). В конце большинства упражнений я буду давать ссылку на мое решение на этом сайте. Природа библиотеки pandas и сайта Pandas Tutor вынудила меня работать с ограниченными наборами данных, но визуализация решений от этого не пострадала.

Отдельные слова благодарности хотелось бы выразить в адрес всех рецензентов. Это Ален Куньот (Alain Couniot), Алекс Гарретт (Alex Garrett), Алекс Лукас (Alex Lucas), Александер Коглер (Alexander Kogler), Амилкар де Абро Нетто (Amilcar de Abreu Netto), Кейдж Слагел (Cage Slagel), Дин Лангсам (Dean Langsam), Джордж Маунт (George Mount), Хелен Мари Лабао Баррамеда (Helen Mary Labao Barrameda), Джефф Нойманн (Jeff Neumann), Джефф Смит (Jeff Smith), Хуан Дельгадо (Juan Delgado), Киран Ананта (Kiran Anantha), Микаэл Дотри (Mikael Dautrey), Мики Тебека (Miki Tebeka), Радучу Сергиу Попа (Răducu Sergiu Popa), Садхана Ганнапатираджу (Sadhana Ganapathiraju), Салил Аталайе (Salil Athalye), Сатедж Кумар Саху (Satej Kumar Sahu), Срути Шивакумар (Sruti Shivakumar), Стивен Херрера (Steven Herrera) и Сянгбо Мао (Xiangbo Mao) – ваши советы помогли сделать эту книгу лучше.

Наконец, мои самые глубочайшие благодарности семье за их терпение и понимание в отношении моих бесконечных «Минуточку, одну фразу подредактирую и иду...» на протяжении последних трех лет. Спасибо моей жене Шире и троим нашим детишкам: Атаре, Шикме и Амоцу.

Об этой книге

В былые времена сбор данных мог быть сопряжен с большими трудностями. Но сегодня, когда датчиками, сенсорами и чипами оборудованы все возможные устройства во всех областях жизнедеятельности человека, эти проблемы окончательно ушли в прошлое. Более того, в наши дни данных в мире собирается столько, что их просто не представляется возможным обработать. Отслеживается буквально все – от сделанных нами шагов на прогулке до эффективности рекламы и температуры в любой точке планеты, если не всей Солнечной системы.

Но вместе с такой активностью мы получили и новую проблему, связанную с обработкой и упорядочиванием всех получаемых данных. Как можно эффективно разобраться во всем многообразии полученных сведений и принимать на их основании решения?

На протяжении многих лет таким средством анализа был Microsoft Excel. И на то были свои причины. Excel представляет собой удобный пакет с графическим интерфейсом, который установлен едва ли не на всех компьютерах в мире. С помощью него вы можете достаточно быстро загрузить данные, очистить их, выполнить нужные вычисления и построить понятные отчеты и даже графики.

Но в последние годы у Excel появился юный и дерзкий конкурент в виде pandas. Изначально эта библиотека предстала перед нами в виде удобной обертки пакета NumPy, сочетающего в себе скорость и эффективность вычислений, присущие языку C, с дружелюбностью Python. Pandas дополнил NumPy новыми полезными методами в области обработки строк и дат со временем, а также позволил визуализировать данные. Кроме того, с помощью Pandas можно удобно читать и писать данные в самых разных форматах, включая онлайн-ресурсы и реляционные базы данных. Все это, помноженное на мощь языка Python, способность обрабатывать гораздо большие массивы данных, по сравнению с Excel, и возможность работать в консольном режиме, без графического интерфейса, уверенно склонило чашу весов в сторону pandas. Я преподавал Python и pandas во многих финансовых учреждениях, в которых аналитиков активно переучивали с Excel на pandas. Кроме того, во многих компаниях из самых разных секторов экономики использование библиотеки pandas утверждено на уровне стандарта.

Но, конечно, аналитики не ограничивались в своей работе одним лишь Excel. Сегодня на pandas переходят многие разработчики из R и Matlab – кто-то по экономическим соображениям, кто-то из-за быстродействия, а кто-то по причине очень развитого сообщества и экосистемы с модулями с открытым исходным кодом на Python, доступными в Python Package Index (PyPI).

Проблема с pandas заключается в том, что это огромная библиотека с тысячами методов, которые могут принимать сотни разных параметров. Кроме того, в pandas вы можете одну и ту же задачу решить самыми разными способами, значительно отличающимися в плане быстродействия.

Обучение эффективной работе с pandas – это долгий путь проб и ошибок. Сократить этот путь можно только при помощи интенсивных практических занятий

и решения задач, которые позволят вам лучше понять специфику этой библиотеки, подобно тому как постоянные тренировки в спортзале помогают укрепить мышцы спортсмена.

Именно в этом и состоит основная цель книги, которую вы держите в руках. Решив 50 основных и 150 дополнительных упражнений, которые здесь собраны, вы неожиданно обнаружите в себе способность бегло и уверенно говорить на новом для вас языке `pandas`. В каждом упражнении вы должны будете загрузить реалистичный набор данных и постараться ответить на поставленные вопросы. В процессе чтения книги вы научитесь применять все наиболее важные методы библиотеки `pandas` и, что более важно, начнете понимать, когда и какие из них являются наиболее приемлемыми.

Эта книга не учебник по `pandas`, хотя из нее вы в том числе почерпнете немало теоретических знаний. Вместо этого данная книга призвана помочь вам понять внутреннее устройство `pandas` и научиться применять эту библиотеку для решения задач в реальном мире.

Пожалуйста, не стоит читать эту книгу от корки до корки, как учебное пособие. Также ошибкой будет прочитать условие задачи, решить для себя, что никакой сложности она для вас не представляет, и проследовать дальше. Многие упражнения включают в себя вопросы, ответы на которые на самом деле более сложны, чем кажутся на первый взгляд. Кроме того, если вы будете просто читать мои решения задач, не пробуя решить их самостоятельно, вы никак не сможете погрузиться в глубины внутреннего устройства `pandas`. Так что, раз уж вы взялись за эту книгу, не стоит уклоняться от самостоятельного штудирования материала и попыток разобраться в поставленной проблеме собственными силами.

В наше время также сложно удержаться от советов не скармливать мои упражнения ChatGPT с дальнейшим просмотром предлагаемых решений. Мало того, что эти решения зачастую будут неправильными, они также не позволят вам самим пройти полноценный путь обучения, как известно, состоящий из ошибок и работы над ними.

Для кого предназначена эта книга

Если вы прошли курс по `pandas`, но по-прежнему часто обращаетесь к Stack Overflow или Google за решением той или иной задачи, эта книга для вас. Это не учебник в привычном понимании этого слова, а пособие по освоению внутреннего устройства `pandas` путем решения практических примеров.

Во многих курсах, посвященных `pandas`, не делается акцент на необходимости хорошо знать Python перед изучением этой библиотеки. Лично я глубоко убежден в том, что такие знания просто необходимы, и в процессе чтения этой книги вы не раз в этом удостоверитесь. В то же время вам не нужно быть настоящим экспертом в области Python. Мне кажется, вам будет достаточно глубокого понимания основных типов данных, циклов, функций и генераторов списков, а также навыка установки модулей с помощью инструкции `pip`. Кроме того, вам может пригодиться понимание анонимных функций в Python (`lambda`), но и это совсем не обязательно.

Структура книги

Эта книга насчитывает 13 глав, в каждой из которых мы сосредоточимся на отдельном аспекте библиотеки `pandas`. В упражнениях будут активно использоваться техники из предыдущих упражнений, а иногда и из следующих. К примеру, со строками (глава 9) и датами (глава 10) мы поработаем и в первых главах книги. Названия глав можно воспринимать лишь как обобщение того, с чем вам придется столкнуться при их чтении, а не как строгие правила.

Названия и описания глав книги приведены ниже.

- **Глава 1. Объект `Series`.** В этой главе вы узнаете, что из себя представляют объекты `Series` и как можно извлекать из них нужные вам данные.
- **Глава 2. Объект `DataFrame`.** В данной главе мы поговорим о создании датафреймов и извлечении из них требуемых значений.
- **Глава 3. Импорт и экспорт.** Эта глава будет посвящена чтению и записи данных в различные форматы, включая CSV и JSON.
- **Глава 4. Индексы.** В этой главе мы поговорим о техниках установки и извлечения обычных и множественных индексов в `pandas`.
- **Глава 5. Очистка данных.** В этой главе мы научимся приводить в порядок беспорядочные данные. В числе прочего мы узнаем, как определять наличие дубликатов, обрабатывать пропущенные значения в данных и удалять ненужные или некорректные данные.
- **Глава 6. Группировка, объединение и сортировка.** Здесь мы обсудим саму суть функционала `pandas`, заключающуюся в группировании данных, объединении нескольких датафреймов и их сортировке по индексам или значениям. Это настолько важные темы, что мы выделили для них сразу две главы.
- **Глава 7. Сложная группировка, объединение и сортировка.** В этой главе мы продолжим обсуждение ключевых методов библиотеки `pandas` и выведем их понимание на новый качественный уровень.
- **Глава 8. Промежуточный проект.** В этой главе мы реализуем большой проект на основе данных исследования о разработчиках Python.
- **Глава 9. Строки.** В этой главе мы поговорим о работе с текстовыми данными в библиотеке `pandas`.
- **Глава 10. Даты и время.** Эта глава будет посвящена взаимодействию со значениями, представляющими дату и время.
- **Глава 11. Визуализация.** Здесь мы будем визуализировать наши данные при помощи API `pandas` и модуля `Seaborn`.
- **Глава 12. Оптимизация.** В этой главе мы поговорим об оптимизации в отношении быстродействия и использования памяти при обработке данных.
- **Глава 13. Итоговый проект.** В заключительной главе книги мы реализуем итоговый большой проект на основе данных об американских колледжах и университетах.

Упражнения составляют основную часть глав этой книги. При этом каждое упражнение будет разбито на следующие секции:

- **Упражнение:** условие задачи для обдумывания способа ее решения;
- **Подробный разбор:** детальное описание задачи и способа ее решения;
- **Решение:** код решения и (в большинстве случаев) ссылка на код на сайте Pandas Tutor, чтобы вы могли его запустить. Код решения вместе с проверочными кодами доступны также в сопроводительных материалах на странице книги на сайте издательства и в репозитории на GitHub по адресу <https://github.com/reuven/pandas-workout>;
- **Дополнительные упражнения:** три вспомогательных упражнения на ту же тему, которые помогут вам лучше понять обсуждаемый предмет. Детального описания решений этих упражнений вы в книге не найдете, но сами решения¹ будут представлены.

Код решений в книге

Эта книга содержит большое количество кода на языке Python. В отличие от большинства книг код в этой книге стоит воспринимать как руководство к действию по написанию собственного кода, а не просто как полезное чтение. При наличии у вас достаточного опыта написания кода на Python с использованием библиотеки pandas вы вполне можете написать и более оптимальный код в сравнении с тем, который приведен в книге. В этом случае вы можете написать мне по адресу, приведенному в последнем абзаце книги, и мы вместе поучимся и порадуемся.

Помимо этой книги, код решений всех упражнений, включая дополнительные, можно найти в следующих местах:

- в сопроводительных материалах на странице книги на сайте издательства и в репозитории GitHub по адресу <https://github.com/reuven/pandas-workout>. Код организован по главам и номерам упражнений, чтобы вам удобно было загрузить нужное решение и запустить его на своем компьютере;
- на сайте Pandas Tutor по адресу <https://pandastutor.com>, представляющем великолепное место для изучения всех тонкостей библиотеки pandas. Работая с этим сайтом, вы можете ввести практически любой свой код и увидеть, как на самом деле он работает, с демонстрацией и анимацией всех выполняемых преобразований. В подавляющем большинстве упражнений из этой книги есть ссылки на сайт Pandas Tutor, чтобы вы могли легко и быстро перейти на нужную страницу и запустить пример. Обратите внимание, что в этих примерах обычно будут использоваться небольшие наборы данных.

Код в этой книге будет перемежаться пояснениями и комментариями, а для лучшей читаемости мы будем выделять код моноширинным шрифтом.

Внешне в книге фрагменты кода могут отличаться от того, как они представлены на сайте. Ограничения книжного формата вынудили нас вставлять переносы строк и другие элементы форматирования. Кроме того, если пояснения того или

¹ В переводном издании. – Прим. перев.

иного фрагмента кода даются в отдельном абзаце, в самом коде могут отсутствовать соответствующие комментарии. На сайте код представлен с полным набором комментариев.

Я надеюсь, что сочетание кода на странице книги, пояснений, ссылки на сайт Pandas Tutor и кода для скачивания поможет вам лучше понять все происходящее в упражнениях и применить полученные знания в своих рабочих сценариях.

Требования к программному/аппаратному обеспечению

Первое и главное, что вы должны установить для плодотворного чтения этой книги, – это, конечно, Python и библиотека pandas. Загрузить и установить Python легче всего по адресу <https://www.python.org>. Я рекомендую установить последнюю доступную версию. Также существуют и другие способы установки Python, включая Windows Store или Homebrew на Mac. Фрагменты кода из этой книги должны успешно работать с любой версией Python начиная с 3.9. На финальном прогоне кода я использовал версию 3.12.

Также вам понадобится библиотека pandas. Я использовал версию 2.1.4, но весь код должен нормально работать со всеми версиями 2.1.x. Загрузить и установить библиотеку можно, воспользовавшись командой `pip install pandas` в терминале.

Для решения упражнений из этой книги вам совсем не обязательно устанавливать графическую среду разработки (IDE) для Python, но с ней вам будет удобнее. Две наиболее популярные среды разработки – это PyCharm (от JetBrains) и Visual Studio Code (от Microsoft). Лично я большой поклонник Jupyter Notebook, который можно установить с помощью команды `pip install jupyter`.

Об авторе



Реувен Лернер (Reuven M. Lerner) – инструктор по Python и pandas, преподающий онлайн и офлайн как для сотрудников крупных компаний, так и в частном порядке. Реувен также выпускает еженедельную рассылку о Python под названием «Better Developers» и рассылку «Bamboo Weekly» с задачами по pandas. Реувен обладает степенью бакалавра Массачусетского технологического института (MIT) в области компьютерных наук и степенью доктора в области педагогических наук Северо-Западного университета (Northwestern). Автор книги «Python Workout», вышедшей в издательстве Manning в 2020 году.

О переводчике



Александр Гинько, обладающий богатым опытом работы в сфере ИТ и более десяти лет посвятивший переводам книг и статей на самые разные темы, в последние годы специализируется на переводе книг в области бизнес-аналитики и программирования для издательства «ДМК Пресс» по направлениям Python, SQL, Power BI, DAX, Excel, Power Query, Tableau, R... На данный момент в активе Александра уже порядка 25 книг, включая одну авторскую, и он продолжает плодотворно работать над переводом новых книг.

Возможно, вам также будут интересны книги «Сверхбыстрый Python» (<https://dmkpress.com/catalog/computer/programming/python/978-5-93700-226-6>) и «Введение в статистическое обучение с примерами на Python» (<https://dmkpress.com/catalog/computer/statistics/978-5-93700-217-4>) в переводе Александра.

Помимо перевода книг, Александр ведет свой канал в Telegram (https://t.me/alexanderginko_books), на котором вы можете из первых уст получить ответы на все интересующие вас вопросы об уже переведенных книгах, находящихся в работе и запланированных на будущее. Также на канале можно найти промокоды на все книги Александра для покупки книг на сайте издательства «ДМК Пресс» с большими скидками. Купить книги Александра и следить за переводом новых книг в режиме реального времени можно также с помощью его бота в Telegram по адресу https://t.me/alexanderginko_books_bot.

Об изображении на обложке

Картина на обложке книги носит название «Женщина с Тунгуски, Северная Сибирь» (*Femme Tongouse* или *Woman of Tunguska, Northern Siberia*) и принадлежит коллекции художника Жака Грассе де Сэйнт-Совера (Jacques Grasset de Saint-Sauveur). Впервые картина была показана в 1788 году. Все изображения тщательно прорисованы и раскрашены вручную.

В те времена очень легко было по одежде определить местожительство, род занятий и статус человека. Издательство Manning традиционно оформляет обложки книг по компьютерной тематике шедеврами мирового искусства, отдавая дань богатому разнообразию региональных культур прошлых веков.

Глава 1

Объект Series

Если у вас есть опыт работы с библиотекой *pandas*, вы знаете, что с ее помощью нам обычно приходится взаимодействовать с двумерными данными в виде таблиц со столбцами и строками, называемых *датафреймами* (data frame). В то же время каждый столбец представляет собой одномерную структуру, именуемую *Series*², что видно на рис. 1.1. Таким образом, вы можете представлять себе датафрейм как коллекцию объектов Series.

Индекс	Country	Area (sq km)	Population
0	United States	9,833,520	331,893,745
1	United Kingdom	93,628	67,326,569
2	Canada	9,984,670	38,654,738
3	France	248,573	67,897,000
4	Germany	357,022	84,079,811

Рис. 1.1. Каждый столбец в датафрейме представляет собой объект Series

Это бывает очень полезно, поскольку вскоре вы узнаете, что большинство методов, применимых к объектам Series, могут быть использованы и с датафреймами с той лишь разницей, что вместо единственного значения они будут возвращать значения для всех столбцов в датафрейме. К примеру, если применить к объекту Series метод `mean`, он вернет среднее значение по столбцу (см. рис. 1.2). Но если вызвать его применительно к датафрейму, *pandas* под капотом опросит все входящие в датафрейм столбцы на предмет среднего значения и вернет получен-

² Мы будем использовать оригинальное название объекта Series, поскольку общепринятого аналога в русском языке не существует. – Прим. перев.

ные результаты совокупно в виде нового объекта Series, к которому впоследствии также можно применить разные методы.

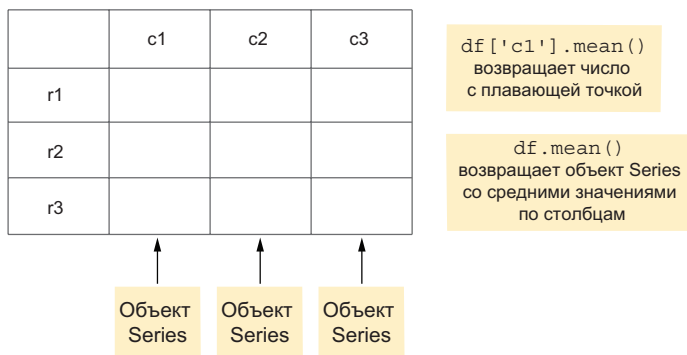


Рис. 1.2. Вызов методов, характерных для объектов Series, таких как `mean`, применительно к датафреймам обычно приводит к получению результата для всех столбцов

Глубокое понимание внутреннего устройства объектов Series поможет вам овладеть библиотекой pandas в полной мере. К примеру, с использованием *булевого индекса* (boolean index), также называемого *индексом-маской* (mask index), мы можем легко извлекать строки и столбцы из датафрейма (если вы не знакомы с булевыми индексами, см. врезку «Отбор при помощи булевых значений» далее в этой главе).

Соглашения об именовании, используемые в этой книге

На протяжении этой книги мы будем часто использовать одни и те же имена для переменных:

- переменной `s` мы будем обозначать объект Series;
- переменная `df` будет ссылаться на датафрейм;
- `pd` представляет собой алиас, или псевдоним, библиотеки pandas, загруженной следующим образом: `import pandas as pd`.

Хотя я являюсь горячим поклонником длинных описательных имен переменных в своих рабочих проектах, в процессе преподавания pandas я предпочитаю ограничиваться короткими именами `s` и `df`. Это бывает очень удобно, особенно с учетом того, что в основном мы будем одновременно использовать один датафрейм или объект Series. В тех редких случаях, когда в моих примерах будет присутствовать более одного датафрейма или Series, я буду добавлять к именам переменных `s` и `df` префиксы или порядковые номера.

Мне также нравится обращаться к классам Series и DataFrame без использования префикса `pd`. С этой целью я обычно импортирую эти классы из библиотеки pandas явно, как показано ниже:

```
from pandas import Series, DataFrame
```

Наиболее важным и мощным инструментом, который есть у нас, как у разработчиков на pandas, является *индекс* (index), который можно использовать для отбора значений как из объектов Series, так и из датафреймов. Более подробно мы будем говорить об индексах в следующих главах, но базовые знания о том, как устанавливать и модифицировать индексы, а также извлекать с их помощью значения, вы будете использовать при работе с библиотекой pandas практически постоянно. В этой главе мы научимся эффективно применять индексы.

В табл. 1.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения. Ссылки приведены как в коротком виде (на сайт Manning), так и в полном, на сайты документаций.

Таблица 1.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
Jupyter	Веб-среда разработки для программирования на Python	jupyter notebook	http://mng.bz/BmYq (https://jupyter.org)
f-строки	Строки с возможностью встраивания выражений	<code>f'It is currently {datetime.datetime.now()}'</code>	http://mng.bz/lWoz (https://peps.python.org/pep-0498/) и http://mng.bz/a1dJ (https://docs.python.org/3/reference/lexical_analysis.html#f-strings)
Типы данных (dtype)	Типы данных, допустимые для использования в объектах Series	<code>np.int64</code>	http://mng.bz/gBVR (https://pandas.pydata.org/pandas-docs/version/1.2.3/user_guide/basics.html#basics-dtypes)
<code>pd.Series.astype</code>	Возвращает новый объект Series с тем же содержимым, преобразованным в указанный тип данных	<code>s.astype(np.int32)</code>	http://mng.bz/xjVB (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.astype.html)
<code>pd.Series.mean</code>	Возвращает арифметическое среднее значений в объекте Series	<code>s.mean()</code>	http://mng.bz/e1DJ (https://pandas.pydata.org/docs/reference/api/pandas.Series.mean.html)
<code>pd.Series.max</code>	Возвращает максимальное значение в объекте Series	<code>s.max()</code>	http://mng.bz/A8pW (https://pandas.pydata.org/docs/reference/api/pandas.Series.max.html)
<code>pd.Series.idxmin</code>	Возвращает индекс минимального значения в объекте Series	<code>s.idxmin()</code>	http://mng.bz/ZR6Z (https://pandas.pydata.org/docs/reference/api/pandas.Series.idxmin.html)

Таблица 1.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.Series.idxmax</code>	Возвращает индекс максимального значения в объекте Series	<code>s.idxmax()</code>	http://mng.bz/RmrP (https://pandas.pydata.org/docs/reference/api/pandas.Series.idxmax.html)
<code>np.random.default_rng</code>	Возвращает генератор случайных чисел NumPy с необязательным начальным значением	<code>np.random.default_rng(0)</code>	http://mng.bz/27RX (https://numpy.org/doc/stable/reference/random/generator.html)
<code>g.integers</code>	Возвращает массив NumPy из случайно выбранных целочисленных значений при помощи генератора	<code>g.integers(0, 10, 100)</code>	http://mng.bz/1JZg (https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.integers.html)
<code>g.random</code>	Возвращает массив NumPy из случайно выбранных значений с плавающей точкой в интервале от 0 до 1 при помощи генератора	<code>np.random.rand(10)</code>	http://mng.bz/PRBP (https://numpy.org/doc/stable/reference/random/generated/numpy.random.Generator.random)
<code>s.std()</code>	Возвращает стандартное отклонение для значений в объекте Series	<code>s.std()</code>	http://mng.bz/Gy4N (https://pandas.pydata.org/docs/reference/api/pandas.Series.std.html)
<code>s.loc</code>	Позволяет осуществлять доступ к элементам объекта Series по меткам или с помощью массива булевых значений	<code>s.loc['a']</code>	http://mng.bz/zXlZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.loc.html)
<code>s.iloc</code>	Позволяет осуществлять доступ к элементам объекта Series по позиции	<code>s.iloc[0]</code>	http://mng.bz/0K7z (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.iloc.html)
<code>s.value_counts</code>	Возвращает отсортированный (в порядке убывающей частоты) объект Series с информацией о том, сколько раз каждое значение встречается в переменной <code>s</code>	<code>s.value_counts()</code>	http://mng.bz/WzOX (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.value_counts.html)

Таблица 1.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>s.round</code>	Возвращает новый объект Series на основе переменной <code>s</code> , в котором значения округлены до заданного количества знаков после запятой	<code>s.round(2)</code>	http://mng.bz/8rzg (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.round.html)
<code>s.diff</code>	Возвращает новый объект Series на основе переменной <code>s</code> , в элементах которого содержится разница между текущим значением и предыдущим	<code>s.diff(1)</code>	http://mng.bz/jP59 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.diff.html)
<code>s.describe</code>	Возвращает объект Series с перечислением основных объектов описательной статистики для переменной <code>s</code>	<code>s.describe()</code>	http://mng.bz/EQ1r (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.describe.html)
<code>pd.cut</code>	Возвращает объект Series с тем же индексом, что и у переменной <code>s</code> , но со значениями, разбитыми на категории в соответствии с заданными параметрами	<code>pd.cut(s, bins=[0, 10, 20], labels=['a', 'b', 'c'])</code>	http://mng.bz/N2eX (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.cut.html)
<code>pd.read_csv</code> и <code>s.squeeze</code>	Возвращает новый объект Series на основе значений из файла с одним столбцом	<code>s = pd.read_csv('filename.csv').squeeze()</code>	http://mng.bz/D4N0 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.squeeze.html)
<code>str.split</code>	Разбивает строку на составляющие по заданным правилам и возвращает список	<code>'abc def ghi'.split()</code> # возвращает ['abc', 'def', 'ghi']	http://mng.bz/aR4z (https://docs.python.org/3/library/stdtypes.html#str.split)
<code>str.get</code>	Извлекает символы из содержимого объекта Series	<code>s.str.get(0)</code>	http://mng.bz/JdWv (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.get.html)
<code>s.fillna</code>	Заменяет значения NaN заданными значениями	<code>s.fillna(5)</code>	http://mng.bz/wjrQ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.fillna.html)

УПРАЖНЕНИЕ 1. *Оценки за ежемесячные тесты*

Создайте объект Series, состоящий из десяти случайных целочисленных значений в диапазоне от 70 до 100, которые будут представлять оценки студента за ежемесячные тесты. Задайте в качестве индекса названия месяцев с сентября по июнь. Если эти месяцы не соответствуют началу и окончанию учебного года в вашем регионе, вы можете адаптировать их.

Напишите код, который будет отвечать на следующие вопросы:

- какова средняя оценка студента за тесты за весь год?
- какова средняя оценка студента за тесты за первое полугодие (т. е. за первые пять месяцев учебного года)?
- какова средняя оценка студента за тесты за второе полугодие (т. е. за последние пять месяцев учебного года)?
- повысилась ли успеваемость студента во втором полугодии в сравнении с первым? Если да, то насколько?

Подробный разбор

В первом упражнении я попросил вас создать объект Series, состоящий из десяти случайных целочисленных значений в интервале от 70 до 100. Здесь есть сразу несколько сопутствующих вопросов:

- как мы определяем новый объект Series?
- как заполнить Series десятью случайными числами в интервале от 70 до 100?
- как установить индекс в виде названий месяцев?

Для определения объекта Series мы можем воспользоваться классом Series, передав ему на вход итерируемый объект – обычно список Python или массив NumPy. Пример:

```
s = Series([10, 20, 30, 40, 50])
```

Но в задаче сказано, что нам нужно, чтобы объект Series содержал десять случайных элементов из заданного диапазона. Библиотека pandas во многом полагается на NumPy, включая область генерирования случайных чисел. Можно получить массив целочисленных значений NumPy путем создания генератора случайных чисел с помощью функции `np.default_rng` и последующего вызова метода `integers` созданного генератора.

Предсказуемые случайные числа

В стандартной библиотеке языка Python присутствует модуль `random`, в котором есть функция `randint`, возвращающая случайное целочисленное значение, как показано ниже:

```
random.randint(0, 100)
```

В результате этого вызова мы получим одно случайно выбранное целое число в заданном диапазоне от 0 до 100, включая число 100.

В мире NumPy мы это делаем несколько иначе. Сначала мы создаем объект генератора случайных чисел таким образом:

```
g = np.random.default_rng()
```

Этот метод отличается от функции Python с именем `random.randint` тем, что:

- возвращает массив NumPy, состоящий из случайных значений, а не одно случайное значение;
- принимает три аргумента: минимальное значение интервала, максимальное значение интервала и количество элементов в массиве;
- значение, переданное вторым аргументом, должно быть на единицу больше верхней границы чисел, которые мы генерируем.

Поскольку эта книга является практическим пособием, вам наверняка захочется сравнить свои результаты решения с моими. А как это сделать, если мы генерируем разные случайные величины? Для этого предусмотрена возможность передачи генератору начального значения, т. е. значения, с которым инициализируется генератор при вызове функции `np.random.default_rng`. Если мы с вами передадим одно и то же числовое значение на вход генератору, то получим одну и ту же последовательность сгенерированных впоследствии чисел. Ниже приведен пример создания массивов случайных чисел от 0 до 100, не включая число 100:

```
g = np.random.default_rng(0)  ❶
a = g.integers(0, 100, 10)  ❷
```

```
g = np.random.default_rng(0)  ❸
b = g.integers(0, 100, 10)  ❹
```

```
a == b  ❺
```

- ❶ Инициализируем генератор случайных чисел нулем.
- ❷ Получаем 10 случайных чисел от 0 до 100, не включая 100.
- ❸ Инициализируем генератор случайных чисел нулем.
- ❹ Получаем еще 10 случайных чисел от 0 до 100, не включая 100.
- ❺ Поскольку начальные значения генераторов были равны, равны будут и переменные `a` и `b`.

Если вы давно работаете с библиотекой NumPy, то наверняка знаете о функции `np.random.seed`, которая служит тем же целям, что и аргумент, передаваемый в функцию `default_rng`. Эта функция никуда не делась, но разработчики NumPy отдадут предпочтение способу с созданием объекта генератора.

Таким образом, мы можем создать массив из десяти случайных целых чисел в интервале от 70 до 100 следующим образом:

```
g = np.random.default_rng(0)
g.integers(70, 101, 10) ❶
```

❶ Верхняя граница 101 позволяет включить число 100 в диапазон возможных значений.

Объект Series можно создать так:

```
g = np.random.default_rng(0)
s = Series(g.integers(70, 101, 10))
```

Итак, у нас есть объект Series с десятью случайными числами от 70 до 100, символизирующими оценки студента за ежемесячные тесты. Но на данный момент индекс, созданный по умолчанию, состоит из чисел от 0 до 9, как в обычном массиве NumPy или списке Python. Нет ничего дурного в числовых индексах, но библиотека pandas предоставляет нам гораздо больше мощи и гибкости, позволяя использовать в качестве индексов все возможные типы данных, включая строки.

Изменить индекс можно, присвоив значение атрибуту index, как показано ниже:

```
g = np.random.default_rng(0)
s = Series(g.integers(70, 101, 10))
s.index = 'Sep Oct Nov Dec Jan Feb Mar Apr May Jun'.split()
```

Теперь вывод объекта Series будет содержать указанные нами индексы, что видно ниже:

```
Sep    96
Oct    89
Nov    85
Dec    78
Jan    79
Feb    71
Mar    72
Apr    70
May    75
Jun    95
dtype: int64
```

ПРИМЕЧАНИЕ. В качестве индекса вы можете указать массив NumPy или объект Series. При этом количество элементов в передаваемой вами структуре должно совпадать с количеством элементов в объекте Series. В противном случае вы получите исключение `ValueError`, и операция присваивания завершится неудачей.

Если вы знаете, какой индекс хотите присвоить объекту Series в момент его создания, то можете передать аргумент `index` прямо при инициализации объекта следующим образом:

```
g = np.random.default_rng(0)
months = 'Sep Oct Nov Dec Jan Feb Mar Apr May Jun'.split()
s = Series(g.integers(70, 101, 10),
            index=months)
```

Лично я предпочитаю именно такой способ и буду использовать его на протяжении большей части книги. При этом данный подход не лишает меня возможности изменить индекс объекта при необходимости с помощью переопределения атрибута `s.index`.

Теперь, когда мы создали наш объект `Series` с оценками студента, можно приступить к вычислениям из условия задачи. Для начала рассчитаем средний балл студента за весь учебный год. Это можно сделать при помощи метода `mean`, который применим ко всем числовым объектам `Series`. Стоит отметить, что даже при наличии в объекте `Series` только целочисленных значений метод `mean` всегда будет возвращать значение типа `float`. Причина в том, что в Python любая операция деления возвращает объект типа `float`:

```
print(f'Yearly average: {s.mean()}')
```

Обратите внимание, что я поместил вызов метода `s.mean()` в фигурные скобки, что характерно для *f-строк* (f-string) в Python. F-строки (официальное сокращение от *форматированные строки* (format strings), но я больше люблю называть их *причудливыми строками* (fancy string)) позволяют размещать выражения Python непосредственно внутри строк с использованием фигурных скобок. В результате мы получаем обычную строку, которую можно вывести на печать или передать в функцию или метод.

Теперь нам необходимо вычислить среднюю успеваемость студента за первое и второе полугодие. Для этого нужно извлечь первые и последние пять элементов из нашего объекта `Series`. Это, как и многое другое, можно сделать множеством способов.

Если бы мы использовали стандартные последовательности в Python, мы бы применили *срез* (slice) с квадратными скобками и указанием нужных нам границ. К примеру, для извлечения первых пяти элементов из списка или строки `s` мы бы воспользовались записью `s[:5]`. В результате нам бы вернулся список элементов с нулевого (начало списка) по пятый не включительно. Обычно в Python диапазоны работают именно в стиле «до такого-то элемента, не включая его самого».

Неудивительно, что мы можем воспользоваться тем же синтаксисом и для извлечения первых пяти элементов из объекта `Series`. Поскольку срез всегда возвращает объект того же типа, что и исходный элемент, мы получим на выходе новый объект `Series` с пятью элементами. А раз это `Series`, мы можем вызвать его метод `mean`, что позволит получить среднее значение успеваемости за первое полугодие, что нам и нужно:

```
s[:5].mean() ❶
```

❶ Средняя оценка за первое полугодие.

А как насчет второго полугодия? Мы можем получить перечисление этих оценок так же точно, воспользовавшись конструкцией `s[5:]`, что показано на рис. 1.3.

Здесь важно не передавать конечный индекс списка, поскольку, как мы знаем, реальная граница выборки всегда будет на единицу меньше указанного индекса. Таким образом, если мы воспользуемся конструкцией `s[5:9]` или `s[5:-1]`, то упустим последнее значение в последовательности. И да, мы можем написать `s[5:10]`. Несмотря на то что элемент индекса со значением 10 у нас отсутствует, Python пропустит это мимо ушей.

```
s[5:].mean()
```

❶ Средняя оценка за второе полугодие.

Индекс	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
Индекс по умолчанию (числовой) index	0	1	2	3	4	5	6	7	8	9
Значения	82	85	91	70	73	97	73	77	79	89



`s[:5]`



`s[5:]`

Рис. 1.3. Извлечение срезов из объекта Series

Но я бы предложил воспользоваться для извлечения нужных нам элементов атрибутами `loc` и `iloc`, также называемыми атрибутами доступа. Если атрибут `loc` служит для извлечения одного или нескольких элементов из объекта Series на основе индекса, то атрибут `iloc` опирается на позиции элементов, т. е. на индекс, устанавливаемый по умолчанию. Давайте начнем с атрибута `iloc`, поскольку его использование очень похоже на то, что мы писали до этого:

```
s.iloc[:5].mean()
```

— Погодите, — могли бы сказать вы, — а зачем мы продолжаем использовать числовой индекс, если при создании объекта Series установили индекс по месяцам? — И вы правы, мы можем воспользоваться и созданным ранее индексом.

Нам снова понадобятся срезы — благо, что pandas достаточно умен, чтобы работать с текстовыми срезами. Мы могли бы воспользоваться атрибутом `loc`, что является неплохой идеей при работе с объектами Series и обязательным правилом при работе с датафреймами.

Таким образом, для получения средней успеваемости за первые пять месяцев учебного года (с сентября по январь) мы можем построить следующий срез:

```
first_half_average = s.loc['Sep':'Jan'].mean()
```

Как вы уже знаете, обычно верхняя граница в срезах работает не включительно, но здесь январь был включен в нашу выборку. Почему?

Если говорить коротко, при использовании атрибута доступа `loc` верхняя граница диапазона включается в отбор. И это весьма логично, поскольку при использовании собственного индекса бывает трудно предположить, как сработает правило исключения последнего элемента из среза. Но многих разработчиков на Python, кто только начинают работать с библиотекой `pandas`, такое поведение сбивает с толку. Кроме того, оно отличается от того, что мы видели применительно к атрибуту `iloc` с использованием позиционного индекса.

loc против iloc против head

В большинстве случаев я предпочитаю пользоваться атрибутом доступа `loc`, что делает код легко читаемым и понятным. Кроме того, этот атрибут предоставляет дополнительную мощь и гибкость при работе с датафреймами. Но за все нужно платить. Когда я проводил тесты, я заметил, что `pandas` требуется вдвое больше времени на обработку текстовых срезов (с атрибутом `loc`) в сравнении с числовыми (с `iloc`).

На рис. 1.4 показана процедура извлечения значений при помощи атрибутов `loc` и `iloc`.

<code>.loc</code>	→ Индекс	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
<code>.iloc</code>	→ Индекс по умолчанию (числовой)	0	1	2	3	4	5	6	7	8	9
	Значения	82	85	91	70	73	97	73	77	79	89

Рис. 1.4. Получение значений при помощи атрибутов `loc` и `iloc`

Стоит сказать, что существует и другой способ извлечения первых и последних месяцев учебного года, предполагающий использование методов `head` и `tail`. Метод `head` принимает на вход числовой аргумент и возвращает соответствующее количество элементов из начала объекта `s`. Если аргумент не передать, по умолчанию будут возвращены первые пять элементов, что нам подходит. Таким образом, мы могли бы рассчитать среднюю успеваемость студента за первое полугодие следующим образом:

```
s.head().mean()
```

Если вы любите указывать все значения параметров явно, можете написать так:

```
s.head(5).mean()
```

Так же точно можно использовать и метод `tail`, позволяющий извлечь последние элементы из перечисления:

```
s.tail().mean()
```

И снова по умолчанию будет возвращено пять элементов, но вы всегда можете задать параметр явно, как показано ниже:

```
s.tail(5).mean()
```

Наконец, мы можем проверить, повысилась ли успеваемость студента во втором полугодии в сравнении с первым, путем простого вычитания среднего балла за первые пять месяцев учебного года из среднего балла за последние пять месяцев. Присвоим средние значения переменным с осмысленными именами и вычислим разницу внутри f-строки следующим образом:

```
first_half_average = s['Sep':'Jan'].mean()
second_half_average = s['Feb':'Jun'].mean()
print(f'First half average: {first_half_average}')
print(f'Second half average: {second_half_average}')
print(f'Improvement: {second_half_average - first_half_average}')
```

Решение

```
g = np.random.default_rng(0)
months = 'Sep Oct Nov Dec Jan Feb Mar Apr May Jun'.split()
s = Series(g.integers(70, 101, 10),
           index=months)

print(f'Yearly average: {s.mean()}')

first_half_average = s['Sep':'Jan'].mean()
second_half_average = s['Feb':'Jun'].mean()

print(f'First half average: {first_half_average}')
print(f'Second half average: {second_half_average}')

print(f'Improvement: {second_half_average - first_half_average}')
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/27ld>.

Дополнительные упражнения

Ниже приведены три дополнительных упражнения с решениями, которые помогут вам потренироваться применять атрибуты доступа `loc` и `iloc` для извлечения данных из нашего объекта Series с именем `s`.

1. В каком месяце наш студент получил наивысший балл за тест? Навскидку можно сказать, что существует как минимум три способа решить эту задачу: вы можете отсортировать значения и взять максимум, воспользоваться индексом-маской для нахождения значений, соответствующих `s.max()`, или применить метод `s.idxmax()`, который вернет индекс максимального значения.
2. Какие были пять его лучших оценок в году?
3. Округлите оценки студента до ближайшей десятки (оценка 82 должна быть округлена до 80, а 87 – до 90). Прочитайте документацию метода `round` (<http://mng.bz/8rz>, <https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.round.html>) на предмет возможных аргументов и того, как он ведет себя с числами 15 и 75.

Среднее значение и стандартное отклонение

Два наиболее важных вычисления, связанных с числовыми наборами данных, – это *среднее значение* и стандартное отклонение. Для расчета среднего в библиотеке pandas используется метод `mean`, а для расчета стандартного отклонения – метод `std`.

Что из себя представляют эти статистики и почему они столь важны?

Среднее значение – это обычное среднее арифметическое, или серединное значение (скоро вы узнаете, какие проблемы могут быть связаны с таким определением). Для его расчета мы складываем все значения в наборе данных и делим полученную сумму на количество элементов. В терминах pandas можно сказать, что вычисление `s.mean()` равнозначно `s.sum() / s.count()`, поскольку метод `s.sum()` позволяет сложить все значения в объекте Series, а метод `s.count()` подсчитывает количество значений, не равных NaN.

Является ли среднее значение истинным мерилем набора данных при определении обобщенного показателя метрики? Далеко не всегда. Иногда мы можем опираться на среднее значение, когда делаем выводы о показателе в целом. Например, если речь идет о среднем росте, весе, возрасте или уровне дохода группы людей. В результате мы получим конкретное число, при помощи которого можно описать данные в целом.

Но у среднего значения есть и недостаток в виде сильного искажения в присутствии в наборе данных одного очень большого значения. Старая шутка гласит о том, что при входе в бар Билла Гейтса все посетители бара в среднем становятся миллионерами. По этой причине среднее значение является не единственным показателем, позволяющим обобщить данные. Самой распространенной альтернативой среднему значению является *медиана*, т. е. значение, лежащее ровно посередине набора данных (если в наборе четное количество значений, берется среднее из двух центральных). В примере с Биллом Гейтсом медианное значение дохода посетителей бара сильно не изменилось бы.

На рис. 1.5 и 1.6 показаны различия при подсчете среднего и медианного значений в наборе данных.

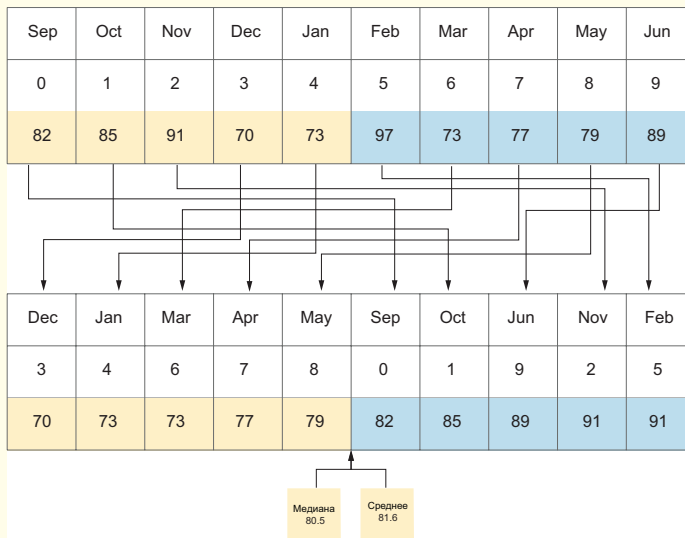


Рис. 1.5. Для вычисления медианы мы сортируем значения и берем среднее из них

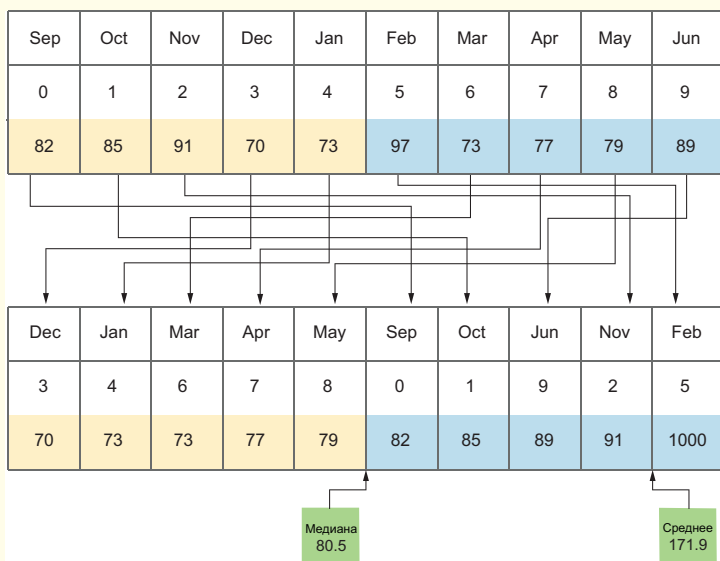


Рис. 1.6. Выбросы в данных сильнее влияют на среднее значение, чем на медиану

При вычислении срединного значения набора данных при помощи среднего или медианы нам почти наверняка потребуется узнать, насколько изменчивыми являются данные. Для этого существует статистика, называемая *стандартным отклонением* (standard deviation). В наборе с нулевым стандартным отклонением все значения будут одинаковыми. И наоборот, при высоком стандартном отклонении значения в наборе будут сильно варьироваться и отклоняться от среднего. Чем выше стандартное отклонение, тем больше значения отклоняются от среднего.

Для вычисления стандартного отклонения в объекте Series необходимо сделать следующее:

- вычислить разницу между каждым значением в наборе и средним значением;
- возвести в квадрат все полученные величины;
- сложить все полученные квадраты;
- разделить результат на количество элементов в наборе. Эта величина известна как *дисперсия* (variance);
- вычислить квадратный корень из дисперсии, что и даст нам стандартное отклонение.

В pandas вычислить стандартное отклонение можно следующим образом:

```
import math
math.sqrt(((s - s.mean()) ** 2).sum() / s.count())
```

Для нашего примера с оценками за тесты мы получим значение 8.380930735902785. Но если мы применим метод `s.std()`, то получим... Нет, не то же самое значение, а 8.83427667918797! В чем же дело?

По умолчанию в pandas заложена логика деления суммы квадратов отклонений значений от среднего не на `s.count()`, а на `s.count() - 1`. Полученный показатель называ-

ется *выборочным стандартным отклонением* (sample standard deviation), и он обычно используется применительно к выборке, а не ко всей совокупности данных. Разработчики библиотеки pandas решили выбрать по умолчанию именно это вычисление (std в NumPy работает иначе).

Если вы хотите получить такой же результат, как в NumPy, необходимо передать 0 в качестве значения параметра `ddof` (*дельта количества степеней свободы* – delta degrees of freedom), как показано ниже:

```
s.std(ddof=0)
```

Таким образом, мы сказали pandas вычесть 0 (а не 1) из `s.count()`, в результате чего получили тот же результат, что и при ручном вычислении. В этой книге я не передаю этот дополнительный параметр `ddof` методу `std`, так что для него используется значение по умолчанию в виде единицы.

При использовании *нормального распределения* (normal distribution), которое применяется в большинстве статистических гипотез, мы ожидаем, что 68 % значений в выборке будут лежать в пределах одного стандартного отклонения от среднего, 95 % значений – в пределах двух стандартных отклонений, а 99.7 % – в пределах трех. При вызове функций `np.random.randint` (для целых чисел) или `np.random.rand` (для чисел с плавающей точкой) мы получим равномерное (случайное) распределение. Если вам необходимо извлекать величины из нормального распределения, в котором значения координируются по указанным выше правилам вокруг среднего значения с заданным стандартным отклонением, вы можете воспользоваться методом `g.normal`. Этот метод принимает на вход три параметра: среднее значение, стандартное отклонение и количество значений для генерирования. В результате вы получите массив NumPy с атрибутом `dtype`, равным `np.float64`, который можно использовать для создания нового объекта `Series`.

В этом разделе мы воспользовались несколькими так называемыми *агрегирующими методами* (aggregation method), которые выполняются применительно к объекту `Series` и возвращают одно число. Примеры таких методов – `sum`, `mean`, `median` и `std`. Мы будем использовать их на протяжении всей книги, и вы можете активно применять их в своих проектах для сбора совокупной статистики на основе наборов данных.

Когда сумма – не сумма

Как вы понимаете, метод `sum` может оказаться очень полезным при работе с числовыми наборами данных. В то же время, если применить метод `s.sum()` к объекту `Series` со строковыми данными, мы получим результат конкатенации, как видно ниже:

```
s = Series('abcd efgh ijkl'.split())
s.sum()
```

❶

❶ Вернет 'abcdefghijkl'.

Неожиданный результат может ждать вас при попытке просуммировать строки, которые на самом деле хранят числовые значения, как в примере ниже:

```
s = Series('1234 5678 9012'.split())
s.mean()
```

❶ Вернет 41152263004.0.

Откуда взялось это число? А вот откуда. Строки, содержащиеся в переменной `s`, сначала объединились вместе, дав результат `'123456789012'`, а затем полученная строка была преобразована в число и разделена на 3 (исходное количество элементов в объекте `Series`) с помощью метода `s.mean()`.

Это один из тех случаев, когда все вроде выполнилось логично, но полученный результат не имеет ни малейшего смысла. Похоже, что в версии Python 3.12 этот недочет был исправлен, и теперь при попытке выполнить это вычисление мы получим ошибку `TypeError`.

Типы данных dtype

В языке Python мы все время используем встроенные типы данных, такие как `int`, `float`, `str`, `list`, `tuple` и `dict`. В `Pandas` мы применяем их в работе не так часто. Вместо них мы используем типы из библиотеки `NumPy`, которые представляют собой тонкий совместимый с Python слой, лежащий поверх значений, определенных в `C`.

У каждого объекта `Series` есть атрибут `dtype`, и вы всегда можете обратиться к нему, чтобы узнать тип содержимого. Каждое значение в объекте `Series` отвечает этому типу – в отличие от кортежей и списков в Python, в одном `Series` не могут содержаться значения разных типов. В то же время `pandas` позволяет определить `dtype` как `object`. В этом случае мы обычно можем предположить, что в объекте содержатся строки Python. Подробнее об этом мы поговорим в главе 9. Хранение нестроковых объектов – редкое явление, и его нужно всячески избегать, хотя иногда это может быть полезно. Также вы можете установить атрибуту `dtype` значение `object`, если собираетесь хранить в объекте `Series` значения разных типов.

Атрибут `dtype` может принимать типы данных, определенные в `NumPy` и использующиеся в `pandas`. Также существует несколько типов, специфичных для `pandas`, о многих из которых мы поговорим в этой книге. Основные значения атрибута `dtype` следующие:

- целочисленные разной длины: `np.int8`, `np.int16`, `np.int32` и `np.int64`;
- беззнаковые целочисленные разной длины: `np.uint8`, `np.uint16`, `np.uint32` и `np.uint64`;
- числа с плавающей точкой разной длины: `np.float16`, `np.float32` и `np.float64` (на некоторых компьютерах также можно использовать тип `np.float128`);
- объекты Python: `object`.

При создании объекта `Series` `pandas` обычно определяет значение атрибута `dtype` исходя из передаваемых значений следующим образом:

- если все значения в объекте `Series` являются целочисленными, атрибут `dtype` устанавливается в `np.int64`;

- если хотя бы одно из переданных значений имеет плавающую точку (включая NaN), атрибут `dtype` устанавливается в `np.float64`;
- в противном случае атрибуту `dtype` присваивается значение `object`.

Вы можете переопределить этот атрибут при создании объекта `Series`, как показано ниже:

```
s = Series([10, 20, 30], dtype=np.float16)
```

Если при инициализации объекта передать ему значения, несовместимые с выбранным значением для атрибута `dtype`, вы получите исключение `ValueError`.

Почему мы так подробно говорим об атрибуте `dtype`? Причина в том, что правильное определение типа значений, особенно при работе с большими данными, позволяет существенно сократить объем используемой памяти и повысить точность вычислений. В Python мы обычно не думаем об этом, но в `pandas` эти аспекты выходят на передний план.

Например, при использовании типа данных `np.int8` значения могут хранить 8-битные числа со знаком, т. е. числа из диапазона от -128 до 127 . А что будет, если перейти эти границы?

```
s = Series([127], dtype=np.int8)
s+1
```

❶

❶ Вернет объект `Series` из одного элемента со значением -128 .

Все правильно. В мире 8-битных чисел со знаком (т. е. допускающих как положительные, так и отрицательные значения) прибавление единицы к числу 127 даст в результате -128 . Это похоже на одометр в вашей машине, который циклически вернется на нулевое значение при превышении конструктивно заложенного в него пробига автомобиля.

Да, это проблема. И поэтому в том числе необходимо правильно выбирать значение атрибута `dtype`, чтобы ячейки были способны уместить все хранящиеся в них значения, включая результаты возможных вычислений. К примеру, если вы планируете умножать ваши данные на 10 , вы должны предусмотреть, чтобы выбранное значение `dtype` это допускало, даже если вы не собираетесь отображать результаты операции или использовать их напрямую.

Не стоит ли в связи с этим все время использовать 64-битные типы данных независимо от того, какие значения мы в действительности храним? В конце концов, они способны уместить в себе любые значения, которые мы только можем себе вообразить.

Однако в этом случае ваши структуры данных будут расходовать очень много памяти. Помните, что 64 бита – это 8 байт? Вроде не так много для компьютеров с современной архитектурой. Но если в вашем объекте `Series` миллиард значений, то объект `Series` с 64-битным типом данных будет занимать 8 Гб памяти без учета накладных расходов Python и вашей операционной системы, а также вспомогательных ресурсов, которые могут понадобиться `pandas`. Ну и, конечно, у вас в памяти должны находиться не только эти числа.

Как результат, вы должны всегда с умом подходить к выделению памяти для своих данных в `pandas`. Одного универсального решения здесь нет – все зависит от конкретного случая.

А что, если вам необходимо изменить значение атрибута `dtype` уже после создания объекта? Вы не сможете это сделать, поскольку этот атрибут работает только на чтение. Но вы можете создать новый объект `Series` на основе существующего, воспользовавшись методом `astype`, как показано ниже:

```
s = Series('10 20 30'.split())
s.dtype ❶

s = s.astype(np.int64)
s.dtype ❷
```

- ❶ Вернет "object".
- ❷ Вернет "np.int64".

Если вы попытаетесь вызвать метод `astype` с неподходящим для данных типом, то получите (как видели при создании объекта `Series`) исключение `ValueError`.

Ответы на дополнительные упражнения

Упражнение 1.1

```
# Вариант 1
s.sort_values(ascending=False).index[0]

# Вариант 2
s[s==s.max()].index[0]

# Вариант 3
s.idxmax()
```

Вывод:

```
'Sep'
```

Упражнение 1.2

```
s.sort_values(ascending=False).head(5)
```

Вывод:

```
Sep    95
Jun    94
Oct    89
Nov    85
Jan    79
dtype: int64
```

Упражнение 1.3

```
# Если передать методу round положительное число, он будет округлять значения после десятичной точки, а если отрицательное, то до десятичной точки, чем мы и воспользуемся
s.round(-1)
```

Вывод:

```
Sep    100
Oct     90
Nov     80
Dec     80
Jan     80
Feb     70
Mar     70
Apr     70
May     80
Jun     90
dtype: int64
```

УПРАЖНЕНИЕ 2. Масштабирование оценок

Когда я учился в старшей школе и колледже, преподаватели время от времени давали нам очень сложные тесты. А чтобы у всего класса не были низкие оценки, они масштабировали, или градуировали, их. К примеру, они полагали, что средняя оценка за тест в этом классе должна быть примерно равна 80, рассчитывали разницу между реальным средним значением оценок и 80 и добавляли ее к полученным нами оценкам.

В этом упражнении вам необходимо сгенерировать 10 оценок в диапазоне от 40 до 60 с таким же индексом по месяцам, что и в первом упражнении. Найдите среднее значение исходных оценок и добавьте к каждой оценке разницу между этим средним значением и 80.

Подробный разбор

Одной из важнейших концепций, применяемых в pandas (и NumPy), является *векторизация* (vectorizing) операций. При выполнении операций над двумя объектами Series с совпадающими индексами действия будут производиться в соответствии со значениями индексов, как показано на рис. 1.7.

Рассмотрим следующий пример:

```
s1 = Series([10, 20, 30, 40])
s2 = Series([100, 200, 300, 400])
```

```
s1 + s2
```

Результат сложения этих объектов Series будет таким:

```
0    110
1    220
2    330
3    440
dtype: int64
```

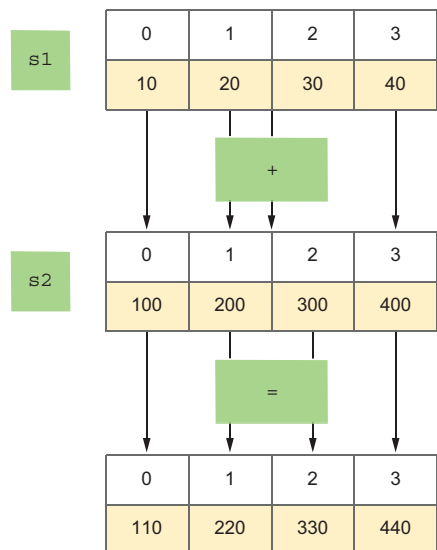


Рис. 1.7. При сложении двух объектов Series мы получим новый объект Series, элементы в котором будут представлять сумму соответствующих значений в исходных массивах с учетом индексов

А что произойдет, если мы зададим индекс для наших объектов явным образом, а не будем полагаться на индекс по умолчанию?

```
s1 = Series([10, 20, 30, 40],
            index=list('abcd'))
s2 = Series([100, 200, 300, 400],
            index=list('dcba'))

s1 + s2
```

Результат будет следующим:

```
a    410
b    320
c    230
d    140
dtype: int64
```

Как видите, pandas сложил два массива поэлементно с учетом актуальных значений в индексах. Обратите внимание на то, что операции сложения были выполнены перекрестно, поскольку в объекте `s1` индекс у нас идет в прямом порядке (`abcd`), а в объекте `s2` – в обратном (`dcba`). Важно, что именно индексы определяют соответствие между элементами, с которыми будет выполняться операция, а не их порядок следования в объекте `Series`, что видно на рис. 1.8.

А что будет, если мы попытаемся сложить не два объекта `Series`, а один объект `Series` и скалярное значение? В pandas для таких случаев используется концепция *транслирования*, или *бродкастинга* (*broadcasting*), предполагающая выполнение операции над каждым элементом объекта `Series` и скаляром. В результате мы получим новый объект `Series`. Пример:

```
s = Series([10, 20, 30, 40],
            index=list('abcd'))

s + 3
```

Результатом будет объект `Series` следующего содержания:

```
a    13
b    23
c    33
d    43
dtype: int64
```

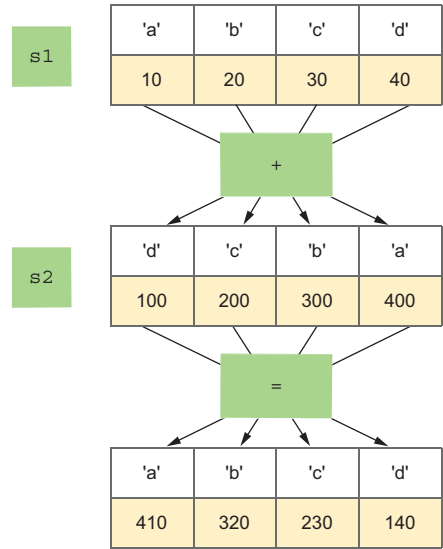


Рис. 1.8. Векторизованные операции выполняются в соответствии с индексами, а не порядком следования элементов

Обратите внимание, что в итоге, как видно на рис. 1.9, мы получили новый объект `Series`, индексы которого совпадают с индексами исходной переменной `s`, а значения являются результатом сложения каждого элемента последовательнос-

ти с целым числом 3 посредством концепции транслирования.

Подобные действия можно производить с участием любых операторов, включая операторы сравнения, такие как `==` и `<`. В случае с операторами сравнения результатом будет объект `Series`, заполненный булевыми значениями, который в дальнейшем может быть использован в качестве индекса-маски, чтобы оставить только те строки, которые нам нужно.

С учетом всего сказанного выше для создания десяти случайных чисел в диапазоне от 40 до 60 и прибавления к ним 10 можно выполнить следующие действия:

```
g = np.random.default_rng(0)
months = 'Sep Oct Nov Dec Jan Feb Mar
Apr May Jun'.split()
s = Series(g.integers(40, 60, 10),
           index=months)
```

```
s + 10
```

Результат будет следующим:

```
Sep    62
Oct    65
Nov    50
Dec    53
Jan    53
Feb    57
Mar    59
Apr    69
May    68
Jun    54
dtype: int64
```

Прекрасно, но не соответствует условию задачи, поскольку мы не знаем, сколько баллов нужно прибавлять к каждой оценке. Нам необходимо вычислить среднюю оценку в переменной `s` и узнать, насколько она отстоит от 80. Это можно легко сделать с помощью метода `s.mean()`. В результате мы получим величину масштабирования всех оценок:

```
s + (80-s.mean())
```

Результат будет следующим:

```
Sep    83.0
Oct    86.0
Nov    71.0
```

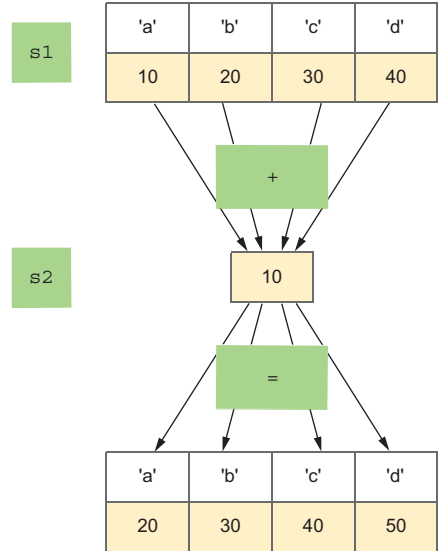


Рис. 1.9. Операция между объектом `Series` и скалярным значением. В результате применения концепции транслирования мы получили новый объект `Series`

```
Dec    74.0
Jan    74.0
Feb    78.0
Mar    80.0
Apr    90.0
May    89.0
Jun    75.0
dtype: float64
```

Обратите внимание, как мы в этом упражнении переходим от объектов Series к скалярным значениям и обратно, что характерно для работы с pandas. Вызов метода `s.mean()` вернул нам скалярное значение. Затем мы вычли результат из 80 (`80 - s.mean()`), также получив скаляр. Но после этого мы снова вернулись к Series в результате сложения скаляра с объектом Series. Так работает транслирование.

ПРИМЕЧАНИЕ. В результирующем объекте значение атрибута `dtype` равно `float64`, тогда как в переменной `s` тип был `int64`. Что послужило причиной такого изменения типа данных? Дело в том, что всякий раз, когда мы выполняем операции с целочисленными значениями и значениями с плавающей точкой, мы в результате получаем тип `float`, даже тогда, когда, казалось бы, в этом преобразовании нет необходимости, как в случае с операцией сложения. Кроме того, любая операция деления в Python 3 дает результат в виде числа с плавающей точкой. Таким образом, когда мы прибавляем к объекту Series (с числами с плавающей точкой) целочисленное скалярное значение посредством транслирования, мы получаем на выходе новый объект Series с дробными числами.

Решение

```
s + (80 - s.mean())
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/1JDV>.

Дополнительные упражнения

Выполняете ли вы операцию над двумя объектами Series или задействуете концепцию транслирования для объединения объекта Series со скаляром, с индексами нужно уметь работать. Именно индексы лежат в основе векторизованных операций и восстанавливаются из исходного объекта Series в результирующем. Попробуйте самостоятельно выполнить дополнительные упражнения на эту тему.

1. Существует еще как минимум один способ выполнить масштабирование оценок за тесты – путем анализа их средних значений и стандартных отклонений. К примеру, если студент уложился в одно стандартное отклонение от среднего значения, он получает оценку C (если она ниже среднего) или B (если выше среднего). Те из студентов, кто преодолел границу одного стандартного отклонения в сторону повышения, получают высший балл – A, а те, кто опустился больше чем на одно стандартное отклонение ниже средней оценки, получают худшую оценку D. С учетом этого правила определите суммарные баллы нашего студента по всем месяцам.

2. Встретились ли вам оценки выше или ниже двух стандартных отклонений от среднего балла? Если да, то в какие месяцы?
3. Насколько близки друг к другу оказались средняя арифметическая оценка и медиана? Что означает близость этих двух статистик? И что бы означала большая разница между ними?

Ответы на дополнительные упражнения

Упражнение 2.1

Оценку A получают студенты с оценками, превышающими среднюю оценку более
чем на одно стандартное отклонение

```
s[s > s.mean() + s.std()]
```

Вывод:

```
Sep    57
Jun     56
dtype: int64
```

Оценку B получают студенты с оценками, превышающими среднюю оценку менее
чем на одно стандартное отклонение

```
s[(s < s.mean() + s.std()) & (s > s.mean())]
```

Вывод:

```
Oct     52
Nov     50
dtype: int64
```

Оценку C получают студенты с оценками ниже средней оценки менее чем
на одно стандартное отклонение

```
s[(s > s.mean() - s.std()) & (s < s.mean())]
```

Вывод:

```
Dec     45
Jan     46
Mar     41
May     43
dtype: int64
```

Оценку D получают студенты с оценками ниже средней оценки более чем
на одно стандартное отклонение

```
s[s < s.mean() - s.std()]
```

Вывод:

```
Feb     40
Apr     40
dtype: int64
```

Упражнение 2.2

Встретились ли нам оценки больше или меньше средней оценки на два стандартных отклонения и более?

```
s[(s < s.mean()-2*s.std()) |  
   (s > s.mean()+2*s.std())]
```

Нет, таких оценок не оказалось

Упражнение 2.3

```
s.mean()
```

Вывод:

```
47.0
```

```
s.median()
```

Вывод:

```
45.5
```

Средняя оценка и медиана оказались достаточно близки, что означает отсутствие больших выбросов в данных, которые могли бы сильно сместить среднее значение в их сторону. Если бы среднее значение оказалось существенно выше медианы, можно было бы предположить, что хотя бы одна из оценок за тесты сильно отличается от остальных в лучшую сторону. То же самое, но с противоположным знаком мы могли бы сказать и про ситуацию, когда средняя оценка значительно ниже средней.

УПРАЖНЕНИЕ 3. Считаем цифры разряда десятков

В этом упражнении мы сгенерируем десять случайных чисел в интервале от 0 до 100 (помните, что функция `np.random.randint` захватывает нижнюю границу переданного диапазона, но не захватывает верхнюю). Создайте объект `Series`, содержащий только цифры разряда десятков из исходного массива случайных чисел. Таким образом, если в исходной последовательности будут находиться числа 10, 25 и 32, мы должны извлечь числа, занимающие второй разряд справа, т. е. 1, 2 и 3.

Подробный разбор

С учетом того что мы сгенерировали наш набор данных с помощью инструкции `np.random.randint(0, 100, 10)`, мы знаем, что наши десять случайных чисел будут находиться в диапазоне от 0 до 99 включительно, а значит, сгенерированные числа будут иметь в своем составе один или два знака.

Для получения цифр разряда десятков мы должны сделать следующее.

1. Разделить числа в нашем объекте `Series` на 10, тем самым изменив значение атрибута `dtype` на `float` и передвинув десятичную точку на один разряд влево.

2. Привести полученный массив к типу `np.int8`, чтобы отбросить не нужную нам дробную часть.

В результате если исходное число состояло из двух знаков, мы получим из него цифру разряда десятков, а если из одного, то получим ноль. Таким образом, наш новый объект `Series` будет выглядеть следующим образом:

```
0    4
1    4
2    6
3    6
4    6
5    0
6    8
7    2
8    3
9    8
dtype: int8
```

Обратите внимание на значение атрибута `dtype` в новом объекте (`int8`). На рис. 1.10 схематически показано выполнение этой операции.

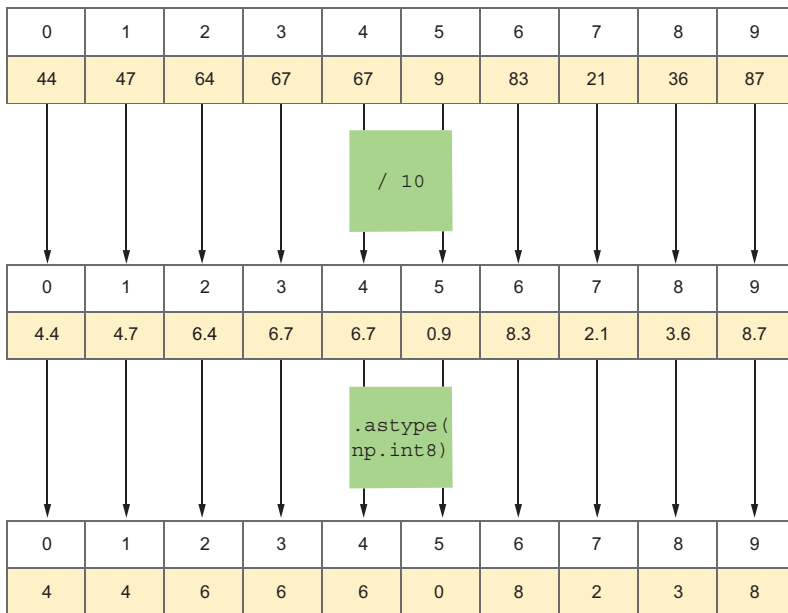


Рис. 1.10. Графическое отображение двух последовательных операций над данными: деления на 10 и приведения к целочисленному типу

Но можно придумать решение и проще. Оператор `//` в Python как раз предназначен для выполнения целочисленного деления. Если разделить объект `Series` на 10 с использованием этого оператора, мы сразу получим новую последовательность со значением атрибута `dtype`, равным `int8`. Именно этот способ мы и выберем, поскольку он требует выполнения меньшего количества действий.

Есть и другие варианты решения этой задачи, предполагающие большее количество операций преобразования. Мы можем конвертировать наш объект Series не в тип чисел с плавающей точкой, а в текстовый вид. Зачем? Чтобы затем воспользоваться строковыми методами для извлечения нужного нам разряда.

Давайте преобразуем наш целочисленный объект Series в новый объект с типом данных `str` с помощью метода `astype`, как показано ниже:

```
s.astype(str)
```

И что дальше? Подробно о методах работы со строками мы будем говорить в главе 8, а тут лишь скажем, что у объекта Series есть атрибут доступа `str`, позволяющий применять строковый метод ко всем элементам последовательности. Его метод `get` работает подобно квадратным скобкам применительно к обычным строкам в Python. Таким образом, если мы напишем `s.astype(str).str.get(0)`, то получим первую цифру каждого элемента целочисленного объекта Series, а инструкция `s.astype(str).str.get(-1)` позволит получить последнюю цифру (в Python отрицательные индексы отсчитывают символы с конца строки). Таким образом, мы можем получить нужные нам разряды десятков с помощью инструкции `s.astype(str).str.get(-2)`. Но, конечно, этого будет недостаточно. Что вернет метод `get(-2)`, если в исходном числе только один знак? Ошибку мы не получим, а получим значение `NaN`. Впоследствии мы можем воспользоваться методом `fillna` для замены значений `NaN` на любое другое значение, например на `'0'`, как показано на рис. 1.11. В результате мы получим объект Series, содержащий односимвольные строки с нужными нам разрядами. На данный момент наш код выглядит так:

```
s.astype(str).str.get(-2).fillna('0')
```

Результат приведен ниже:

```
0    4
1    4
2    6
3    6
4    6
5    0
6    8
7    2
8    3
9    8
dtype: object
```

Как видите, мы получили объект Series типа `object`, который обычно соответствует строкам в Python. Можно ли теперь преобразовать наш массив данных в целые числа? Да, с помощью того же метода `astype`. Мы воспользуемся типом `np.int8`, поскольку имеем дело с небольшими числами:

```
s.astype(str).str.get(-2).fillna('0').astype(np.int8)
```

Результат будет следующим:

```
0    4
1    4
```

```

2    6
3    6
4    6
5    0
6    8
7    2
8    3
9    8
dtype: int8

```

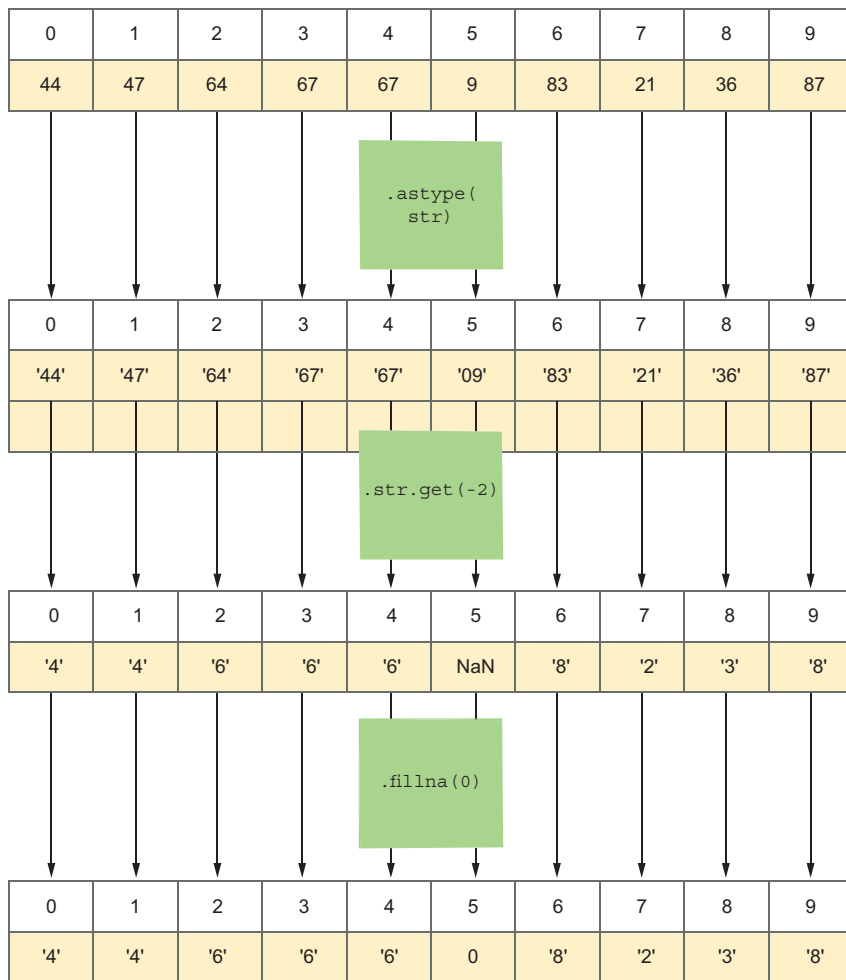


Рис. 1.11. Графическое отображение преобразования массива в строки, извлечения нужного символа и замены значений NaN нулями

Мне этот способ кажется более подходящим в сравнении с техникой преобразования значений из целочисленных в числа с плавающей точкой. Он более выразительный, но вместе с тем и чуть более сложный. Кроме того, если вы знаете, что все ваши значения имеют два знака, он может быть избыточным.

Если вам кажется, что в предыдущей инструкции слишком уж много всего написано в одной строке, вы можете воспользоваться приемом, который популяризировал мой друг Мэтт Харрисон (Matt Harrison, <https://www.metasnake.com>). Этот трюк заключается в переносе операций по строкам, что Python позволяет делать при наличии круглых скобок, обрамляющих выражение. Таким образом, мы можем удобно разместить каждое действие на отдельной строке и даже сочетать некоторые из них по своему усмотрению. Это позволит сделать код более легким для чтения и разместить комментарии для каждой операции, как показано ниже:

```
(
    s
    .astype(str)      # преобразуем наш массив данных в тип str
    .str.get(-2)      # извлекаем второй символ с конца
    .fillna('0')      # заменяем NaN на '0'
    .astype(np.int8)  # преобразуем результат в тип данных int8
)
```

Это выражение даст такой же результат, что и раньше, но код не будет терять своей привлекательности даже при дальнейшем увеличении сложности выполняемых операций.

ПРИМЕЧАНИЕ. В библиотеке pandas традиционно используются строки Python, но на момент написания этой книги появился новый экспериментальный тип данных `pd.StringDtype`, который в будущем может заменить собой тип `str`. Это один из шагов на пути глобального пересмотра обращения с типами данных в pandas. Другим изменением будет то, что значение NaN не будет всегда иметь тип `float`, а сможет представлять отсутствующее значение любого типа. Я не удивлюсь, если в ближайшие годы тип данных `pd.StringDtype` станет новым стандартом и будет рекомендован для работы в pandas. Кроме того, в pandas растет поддержка платформы Apache Arrow, включая ее строковые типы. Но на данный момент наиболее широкое распространение в pandas имеют обычные строки Python, и именно поэтому в данной книге мы работаем именно с ними.

Решение

```
g = np.random.default_rng(0)
s = Series(g.integers(0, 100, 10))
s//10
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/PRY9>.

Дополнительные упражнения

1. А что, если мы изменим диапазон, и числа у нас будут иметь границы 0 и 10 000? Как это повлияет на ваше решение, если вообще повлияет?
2. С учетом нового диапазона от 0 до 10 000 какой наименьший тип данных вы можете использовать для целочисленных значений?
3. Создайте новый объект Series с десятью значениями с плавающей точкой в интервале от 0 до 1000. Найдите числа, у которых целочисленная часть (без учета дробной) четная.

Выбор значений с помощью маски

В Python и других традиционных языках программирования элементы из списка обычно выбираются и фильтруются при помощи циклического прохода с использованием ключевых слов `for` и `if`. В `pandas`, конечно, тоже можно заниматься подобными вещами, но вы вряд ли захотите это делать. Вместо этого здесь принято осуществлять выборку нужных вам элементов при помощи *булева индекса* (boolean index), также называемого *индексом-маской* (mask index).

Индексы-маски бывают очень полезны при работе с данными, но к их синтаксису нужно привыкнуть. Для начала рассмотрим пример извлечения элемента из объекта `Series` посредством квадратных скобок и индекса:

```
s = Series([10, 20, 30, 40, 50])
s.loc[3]
```

❶ Вернет 40.

Но вместо передачи одного числового индекса вы можете передать список (или массив `NumPy`, или объект `Series`) из булевых значений (т. е. `True` и `False`), как показано ниже:

```
s = Series([10, 20, 30, 40, 50])
s.loc[[True, True, False, False, True]]
```

❶ Обратите внимание на двойные квадратные скобки! Внешняя пара скобок говорит о том, что мы собираемся извлекать значения из `s`, а внутренняя определяет список Python. В результате мы получим значения 10, 20 и 50, как показано на рис. 1.12.

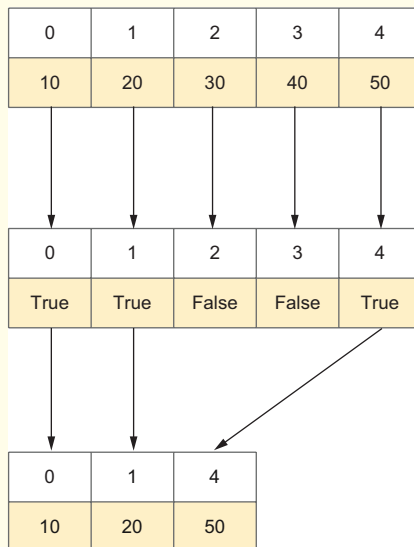


Рис. 1.12. Выбор элементов из объекта `Series` по маске

Для каждого значения `True` в переданной маске значение из исходного массива будет отправляться в вывод, а для каждого значения `False` будет игнорироваться. Эта операция

называется индексированием по маске, поскольку в данном случае мы используем список из булевых значений в качестве некоего решета, или маски, для отбора только нужных нам элементов. Индексирование по маске не изменяет исходные данные, а позволяет выбрать из них нужные нам элементы.

Но явно объявленный список из булевых значений редко бывает полезен в качестве маски. Вместо него мы можем воспользоваться объектом Series с булевыми значениями, который можно легко создать. Для этого достаточно воспользоваться оператором сравнения (к примеру, `==`), который возвращает `True` или `False`. С помощью концепции транслирования, о которой вы уже знаете, мы можем легко восстановить исходный объект Series, но на этот раз с булевыми значениями. Пример:

`s.loc[s < 30]` ❶

❶ Вернет объект Series со значениями 10 и 20.

На рис. 1.13 показан процесс получения объекта Series, наполненного булевыми значениями в соответствии с указанным фильтром, а на рис. 1.14 – процедура применения маски к исходному объекту Series.

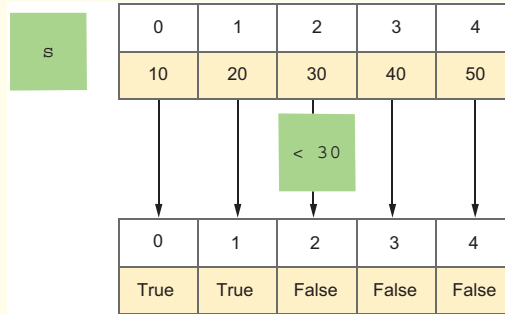


Рис. 1.13. Получение маски, т. е. объекта Series с булевыми значениями

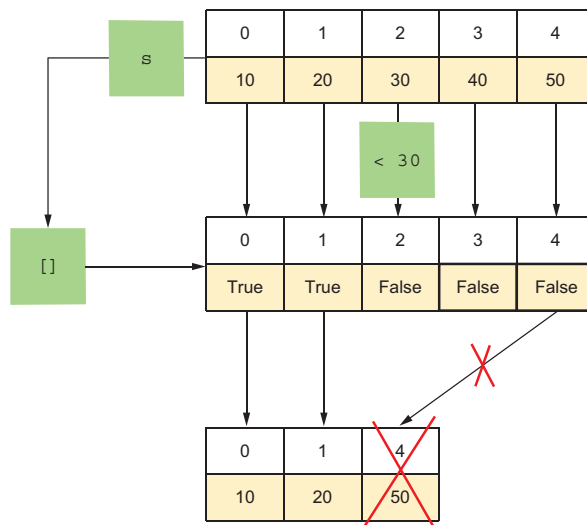


Рис. 1.14. Применение маски к исходному набору данных

Обратите внимание, что элемент со значением 50, который не прошел фильтр, не попал в итоговый объект Series.

С непривычки вам может показаться, что эта инструкция выглядит довольно странно. Даже опытных разработчиков она может поставить в тупик в первую очередь потому, что переменная `s` присутствует как вне квадратных скобок, так и внутри. Помните, что сначала вычисляется выражение внутри скобок. Наше выражение `s < 30` даст на выходе объект Series, состоящий из булевых значений, где `True` будет означать выполнение условия для элемента, а `False` – его невыполнение. Иными словами, этот объект будет выглядеть так: `Series([True, True, False, False, False])`.

После этого данный объект применяется в качестве маски к исходному объекту `s`. Это позволяет оставить в новом объекте Series только те элементы из массива, для которых в маске стоит значение `True`. Таким образом, мы получим только значения 10 и 20.

Но можно и усложнить фильтрующее выражение, например как показано ниже:

```
s.loc[s <= s.mean()] ❶
```

❶ Вернет объект Series с элементами 10, 20 и 30.

Теперь переменная `s` упоминается в выражении *трижды*: один раз при расчете среднего значения исходного перечисления (`s.mean()`), второй – при поэлементном сравнении исходного объекта Series с вычисленным средним значением (`s <= s.mean()`), а третий – при обращении к атрибуту `loc` для извлечения нужных нам элементов по маске. В результате мы получим только те элементы из исходного набора, которые меньше или равны среднему значению по выборке.

Наконец, мы можем использовать индекс-маску как для извлечения значений, так и для присваивания, как показано ниже:

```
s.loc[s <= s.mean()] = 999
```

Результат выполнения этой операции присваивания будет таким:

```
0    999
1    999
2    999
3     40
4     50
dtype: int64
```

Таким образом, мы заменили значения всех элементов в исходном объекте Series, не превышающих среднее значение по выборке, на 999.

Эту мощную и эффективную технику присваивания обязательно нужно иметь в своем арсенале. Она применима как к отдельным объектам Series, что мы увидели выше, так и к целым датафреймам, о чем мы будем говорить в следующей главе.

И последнее замечание, касающееся нашего набора данных `s`. Вы имеете возможность извлекать сразу несколько элементов с разными индексами с помощью техники, получившей название *причудливая индексация* (fancy indexing), передавая в квадратных скобках список, объект Series или другой итерируемый объект. Пример:

```
s.loc[[2,4]]
```

Это выражение вернет объект Series, содержащий два элемента: `s.loc[2]` и `s.loc[4]`.

Внешние квадратные скобки говорят о нашем намерении извлечь элементы из объекта Series с помощью атрибута `loc`, а внутренние – о желании получить больше одного элемента. В результате мы получим новый объект Series с нужными нам значениями и сохраненной индексацией.

Не путайте причудливую индексацию с применением индекса-маски. В первом случае внутренние квадратные скобки содержат список значений из индекса, а во втором – булевы значения (`True` и `False`).

Ответы на дополнительные упражнения

Упражнение 3.1

Наша стратегия работы со строками здесь сработает отлично! В отсутствие
чисел меньше десяти мы можем даже удалить вызов метода `fillna`,
но мне кажется, что лучше его оставить

```
s.astype(str).str.get(-2).fillna('0').astype(np.int8)
```

Вывод:

```
0    0
1    6
2    1
3    9
4    7
5    0
6    5
7    6
8    5
9    3
dtype: int8
```

Упражнение 3.2

Давайте выведем минимальное и максимальные значения для нашего объекта Series

```
print(s.min(), s.max())
```

Вывод:

```
165 8506
```

Что будет, если использовать тип данных `int8`?
`s.astype(np.int8)`

Вывод:

```
0    58
1   -31
2   -9
```

```
3    -119
4         6
5    -103
6     -16
7     -91
8     -40
9     -60
dtype: int8
```

```
# Что будет, если использовать тип данных uint8?
s.astype(np.int8)
```

Вывод:

```
0      58
1     225
2     247
3     137
4         6
5     153
6     240
7     165
8     216
9     196
dtype: uint8
```

```
# Похоже, во избежание проблем нам придется использовать тип np.int16
# или np.uint16!
s.astype(np.int16)
```

Вывод:

```
0      8506
1     6369
2     5111
3     2697
4     3078
5      409
6      752
7      165
8     1752
9     8132
dtype: int16
```

Упражнение 3.3

```
# Сначала создадим набор данных
s = Series(np.random.rand(10) * 1000)
s
```

Вывод:

```
0      209.429210
1      950.023993
```



```

2    565.990291
3    125.967625
4    857.917191
5    966.625315
6    176.835746
7    951.227401
8    143.765381
9    747.060886
dtype: float64

```

```

# Оставим в наборе данных только значения, у которых остаток от деления
# целочисленной части на 2 даст в результате 0. Воспользуемся для этого
# индексом-маской

```

```
s[s.astype(np.int64) % 2 == 0]
```

Вывод:

```

1    950.023993
5    966.625315
6    176.835746
dtype: float64

```

УПРАЖНЕНИЕ 4. *Описательная статистика*

Мы уже научились извлекать среднее значение, медиану и стандартное отклонение наборов данных для получения более полного представления о них. Но этими показателями *описательная статистика* (descriptive statistics) не ограничивается, включая в себя, помимо перечисленных метрик, минимальное и максимальное значения, а также 25-й и 75-й процентиля. Хорошее понимание показателей описательной статистики – ключ к полноценному анализу данных, и в этом упражнении мы потренируемся работать с ними. Выполните следующие действия:

- сгенерируйте объект Series, содержащий 100 000 чисел с плавающей точкой, принадлежащих нормальному распределению со средним значением, равным нулю, и стандартным отклонением, равным 100;
- получите для этого набора данных показатели описательной статистики. Насколько близки друг к другу оказались среднее значение и медиана? Вам не нужно вычислять разницу между ними, просто прикиньте глазами, почему они не равны;
- замените минимальное значение в наборе данных числом, в пять раз превышающим максимальное;
- снова извлеките показатели описательной статистики. Изменились ли среднее значение, медиана и стандартное отклонение? Опять же, никаких расчетов проводить не нужно, достаточно зрительного анализа. Если изменили, то почему?

Подробный разбор

В этом упражнении мы будем работать с данными, распределенными несколько иначе, чем раньше. Вместо функции `np.random.randint` мы воспользуемся методом `g.normal`, который я упоминал на врезке «Среднее значение и стандартное отклонение» ранее в этой главе. При вызове метода `g.normal` мы также получим набор из случайных значений, но на этот раз они будут выбраны из нормального распределения с заданными средним значением и стандартным отклонением.

Итак, создадим наш объект `Series` следующим образом:

```
g = np.random.default_rng(0)
s = Series(g.normal(0, 100, 100_000)) ❶
```

- ❶ При написании целочисленных значений допустимо разделять разряды символом подчеркивания для удобства чтения.

Для получения показателей описательной статистики объект `Series` располагает целым набором методов, но, к счастью, `pandas` предлагает нам обобщенный метод `describe`, позволяющий извлечь все следующие основные метрики разом:

- `count` – количество значений в наборе данных, не равных `NaN`;
- `mean` – среднее значение, которое можно получить с помощью метода `s.mean()`;
- `std` – стандартное отклонение, которое можно получить с помощью метода `s.std()`;
- `min` – минимальное значение, которое можно получить с помощью метода `s.min()`;
- `25%` – 25-й перцентиль в наборе данных. Это значение, лежащее на правой границе первой четверти всех значений в наборе, упорядоченном по возрастанию. Можно получить с помощью метода `s.quantile(0.25)`;
- `50%` – медиана, которую можно получить с помощью методов `s.median()` или `s.quantile(0.5)`;
- `75%` – 75-й перцентиль в наборе данных. Это значение, лежащее на правой границе третьей четверти всех значений в наборе, упорядоченном по возрастанию. Можно получить с помощью метода `s.quantile(0.75)`;
- `max` – максимальное значение, которое можно получить с помощью метода `s.max()`.

Все эти показатели можно извлечь по отдельности, но часто бывает удобно рассматривать их в совокупности. Ниже показан результат вызова метода `s.describe()`:

```
count    100000.000000
mean         0.157670
std        99.734467
min       -485.211765
25%       -66.864170
50%         0.172022
75%        67.343870
```

```
max      424.177191
dtype: float64
```

Среднее значение в нашем наборе составляет 0.157670. Не ноль, конечно, как мы запрашивали, но не забывайте, что мы имеем дело со случайными значениями, выбранными из нормального распределения, так что какие-то колебания непременно будут. Медианное значение, также именуемое 50-м перцентилем, у нас равно 0.172022, что довольно близко к среднему. Это вполне объяснимо, поскольку в нормальном распределении у нас половина значений располагаются выше среднего, а половина – ниже. Стандартное отклонение получилось примерно равным 100, а значит, 68 % значений в переменной `s` будут находиться в интервале от -100 до $+100$.

А что произойдет, если заменить минимальное значение в наборе данных числом, в пять раз превышающим максимальное?

Для начала нам необходимо найти все индексы, соответствующие минимальному значению. Мы могли бы воспользоваться методом `idxmin`, но он возвращает только первый найденный индекс, а нам бы хотелось заменить все минимальные значения в наборе. Чтобы сделать это, сначала получим объект Series с булевыми значениями, в котором значения `True` будут указывать на позиции элементов, равных минимальному значению:

```
s == s.min()
```

После этого мы можем применить полученную маску к нашему набору следующим образом:

```
s.loc[s == s.min()]
```

В результате получим объект Series, содержащий минимальные значения в наборе. В нем может быть минимум один элемент, а могут быть и повторы. Теперь изменим эти значения на число, в пять раз превышающее максимальное, как сказано в условии задачи:

```
s.loc[s == s.min()] = 5*s.max()
```

Поскольку мы изменили наш исходный набор данных, можно повторно воспользоваться методом `s.describe()` и посмотреть, как изменились показатели описательной статистики. Что же мы видим?

```
count    100000.000000
mean      0.183731
std       99.947900
min      -465.995297
25%      -66.862839
50%       0.174214
75%       67.345174
max      2120.885956
dtype: float64
```

Первое, что стоит отметить, – среднее значение немного подросло, что вполне объяснимо, поскольку мы увеличили минимальные значения, сделав их очень большими. Таким образом, мы можем отметить чувствительность среднего к выбросам, которые мы искусственно внесли в наши данные.

Что касается стандартного отклонения, то оно также возросло, на что также повлияли устроенные нами всплески в исходных данных. Да, этот показатель не сильно увеличился, но достаточно, чтобы сказать, что теперь данные в нашем наборе распределены несколько иначе, чем раньше.

Наконец, медиана практически не изменилась. Причина в том, что этот показатель является наиболее стабильным и меньше всего реагирует на выбросы в данных. Это не значит, что вам всегда нужно использовать при анализе данных только медиану, но иногда эта метрика бывает очень полезна. Например, если государство пытается определить пороговые значения для материальной поддержки населения, то небольшая часть богатых людей существенно сместит средний показатель, что не позволит действительно нуждающимся получить финансовую поддержку. Если ориентироваться на медиану, можно сделать так, чтобы, к примеру, помощь от государства получили граждане, входящие в нижние 20 % в отношении получаемого дохода.

Решение

```
import numpy as np
import pandas as pd
from pandas import Series, DataFrame

g = np.random.default_rng(0)
s = Series(g.normal(0, 100, 100_000))
print(s.describe())

s.loc[s == s.min()] = 5*s.max()
print(s.describe())
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/JdM0>.

Дополнительные упражнения

1. Продемонстрируйте, что 68, 95 и 99.7 % значений в переменной `s` действительно находятся в границах одного, двух и трех стандартных отклонений от среднего.
2. Рассчитайте среднее значение для элементов, превышающих `s.mean()`. Затем рассчитайте среднее для элементов, которые меньше `s.mean()`. Совпадает ли среднее значение этих двух величин с `s.mean()`?
3. Каково среднее значение элементов за границей трех стандартных отклонений?

Ответы на дополнительные упражнения

Упражнение 4.1

```
# В рамках одного стандартного отклонения
s[(s > s.mean() - s.std()) &
 (s < s.mean() + s.std())].count() / s.count()
```

Вывод:

0.68396

```
# В рамках двух стандартных отклонений
s[(s > s.mean() - 2*s.std()) &
  (s < s.mean() + 2*s.std())].count() / s.count()
```

Вывод:

0.95461

```
# В рамках трех стандартных отклонений
s[(s > s.mean() - 3*s.std()) &
  (s < s.mean() + 3*s.std())].count() / s.count()
```

Вывод:

0.99708

Упражнение 4.2

```
(s[s < s.mean()].mean() + s[s > s.mean()].mean()) / 2
```

Вывод:

0.12941477214831565

```
# Достаточно близко!
s.mean()
```

Вывод:

-0.09082507731206121

Упражнение 4.3

```
# Весьма сложная комбинация индексов-масок, но результатом будет объект
Series, у которого мы можем посчитать среднее
s[(s < s.mean() - 3*s.std()) |
  (s > s.mean() + 3*s.std())].mean()
```

Вывод:

-11.606040282602287

УПРАЖНЕНИЕ 5. Температура по понедельникам

Новички в pandas часто предполагают, что индекс в объекте Series должен содержать уникальные значения. В конце концов, именно такой характеристикой обладают индексы в строках Python, списках и кортежах, да и ключи в словарях тоже не содержат повторений. Однако индексы в pandas могут содержать дубликаты, что облегчает извлечение элементов с одинаковыми значениями индекса. Если в индексе располагаются идентификаторы пользователей, коды стран или

адреса электронной почты, мы можем использовать их для извлечения элементов, связанных с конкретным значением индекса, что иначе потребовало бы выполнения более сложных операций с использованием индекса-маски.

В этом упражнении мы создадим объект `Series` с 28 измерениями температуры воздуха в градусах Цельсия за четыре недели, выбрав значения из нормального распределения со средним значением 20 и стандартным отклонением 5 с округлением до ближайшего целого. Индекс у нас будет состоять из повторяющихся названий дней недели с `Sun` (воскресенье) по `Sat` (суббота). А вопрос будет такой: какова была средняя температура по понедельникам за исследованный период времени?

Подробный разбор

Это упражнение можно условно разделить на две части. Во-первых, нам необходимо создать объект `Series`, содержащий 28 значений с циклически повторяющимися значениями в индексе. Начнем с создания массива `NumPy` из случайных значений, выбранных из нормального распределения со средним значением 20 и стандартным отклонением 5. Как мы уже знаем, в этом случае 95 % значений будут лежать в интервале двух стандартных отклонений от среднего, т. е. между отметками 10 и 30 °C. Довольно сильный перепад температур для одного месяца, вам не кажется? Ну что ж, будем считать, что это ранняя весна или поздняя осень. Воспользуемся уже знакомым нам методом `g.normal`, как показано ниже:

```
g = np.random.default_rng(0)
g.normal(20, 5, 28)
```

Как можно создать индекс с днями недели для этих 28 значений? Можно, конечно, перечислить все дни недели вручную, но вряд ли мы собрались здесь именно за этим. Давайте начнем со списка с семью днями недели:

```
days = 'Sun Mon Tue Wed Thu Fri Sat'.split()
```

Если бы наши наблюдения распространялись только на одну неделю, мы могли бы присвоить нашим данным индекс, передав при создании объекта `Series` атрибут `index=days`. Но у нас целых 28 наблюдений за четыре недели, а значит, нам необходимо, чтобы наш список с индексами циклически повторялся. Этого можно добиться, просто умножив наш список на 4 следующим образом: `days * 4`. Это поведение кардинально отличается от операции транслирования в `pandas`!

Таким образом, мы можем создать объект `Series` так:

```
s = Series(g.normal(20, 5, 28),
           index=days*4) ❶
```

- ❶ В результате умножения списка в Python вернется новый список с повторяющимися элементами из исходного списка.

Но метод `g.normal` возвращает числа с плавающей точкой (а именно `np.float64`). Как можно преобразовать их в целочисленные значения?

Один из способов состоит в применении метода `astype(np.int8)` (этого типа данных вполне достаточно, ведь если бы температура могла упасть ниже -100° или подняться выше $+100^{\circ}\text{C}$, вы бы сейчас не читали эту книгу). Этот подход работает, но при этом дробные части чисел будут просто отброшены, без округления. Если мы хотим округлять температуру до ближайшего целого, как сказано

в условиях задачи, необходимо предварительно воспользоваться методом `round`, как показано ниже. После этого мы можем вызвать метод `astype(np.int8)`, чтобы привести значения к целым числам:

```
g = np.random.default_rng(0)
s = Series(g.normal(20, 5, 28),
           index=days*4).round().astype(np.int8)
```

Теперь поговорим о повторяющихся значениях в индексе. Да, они действительно могут повторяться, и речь идет не только о числах, но и о строках (как в нашем примере), и о других типах данных вроде даты и времени, как мы увидим в главе 9. Обычно при извлечении значений из объекта `Series` посредством атрибута доступа `loc` мы ожидаем получить единственный элемент. Но если значения в индексе повторяются, мы получим сразу несколько элементов, которые в `pandas` традиционно возвращаются в виде объекта `Series`.

ПРИМЕЧАНИЕ. При использовании инструкции `s.loc[i]` вы не можете заранее знать, вернется ли вам единственное значение, скаляр (если запрошенное значение индекса уникально) или объект `Series` (если в индексе есть повторения). Это один из примеров того, когда вам нужно знать свои данные, чтобы понимать, какой тип значения вернется.

В данном случае мы знаем, что нужное нам значение (`Mon`) встречается в индексе четыре раза. Таким образом, если запросим `s.loc['Mon']`, то на выходе получим объект `Series` с четырьмя значениями, соответствующими понедельникам в нашем наборе данных:

```
s.loc['Mon']
```

Вывод:

```
Mon    22
Mon    19
Mon    22
Mon    24
dtype: int8
```

Поскольку здесь мы имеем дело с `Series`, мы можем применять к нему любые методы, характерные для этого объекта. Нам необходимо узнать среднюю температуру по понедельникам, и мы это можем сделать так: `s.loc['Mon'].mean()`. Ответ: 21.75. Это и есть наше решение.

Решение

```
days = 'Sun Mon Tue Wed Thu Fri Sat'.split()

g = np.random.default_rng(0)
s = Series(g.normal(20, 5, 28),
           index=days*4).round().astype(np.int8)

s.loc['Mon'].mean()
```

Вы можете посмотреть это решение на сайте `Pandas Tutor` по адресу <http://mng.bz/wjeq>.

Дополнительные упражнения

1. А какая была средняя температура на выходных (суббота и воскресенье)?
2. Сколько раз температура отличалась более чем на два градуса по сравнению с предыдущим днем?
3. Какая температура встречалась в нашем наборе данных чаще остальных? Сколько раз она появлялась?

Ответы на дополнительные упражнения

Упражнение 5.1

```
s[['Sun', 'Sat']].mean()
```

Вывод:

```
20.875
```

Упражнение 5.2

```
# По умолчанию метод diff выполняет сравнение как раз с предыдущим элементом
s[s.diff() > 2]
```

Вывод:

```
Tue    23
Fri     22
Sat     27
Wed     17
Thu     20
Sat     19
Thu     22
Fri     25
Sun     27
Tue     22
Wed     25
dtype: int8
```

Упражнение 5.3

```
# Метод value_counts возвращает объект Series, в котором значения из s являются
# индексами, а количество их вхождений – значениями, при этом данные
# упорядочены в порядке убывания количества вхождений. После этого можно
# применить метод head для получения двух наиболее часто встречающихся значений
s.value_counts().head(2)
```

Вывод:

```
17     4
19     3
Name: count, dtype: int64
```


УПРАЖНЕНИЕ 6. *Пассажиропоток в такси*

В этом упражнении мы обратимся к реальному набору данных, полученному из одноколоночного файла CSV. Подробнее о чтении и записи в файлы мы будем говорить в главе 3, а здесь лишь воспользуемся функцией `pd.read_csv` и методом `squeeze` для преобразования датафрейма, состоящего из одного столбца, в объект `Series`.

Данные, которые мы будем использовать в этом упражнении, располагаются в файле `taxi-passenger-count.csv` – его вы можете найти в архиве на странице этой книги на сайте издательства. Источником для них послужил открытый городской сайт Нью-Йорка, на котором можно почерпнуть массу полезной информации за последние годы. Мы воспользуемся данными за 2015 год о поездках пассажиров в желтых такси, коими славится этот город. В файле содержится информация о 9999 поездках.

Наша задача будет состоять в том, чтобы выяснить, в каком проценте заказов пассажир в такси был только один, а в каком – шесть (максимальная вместимость).

Подробный разбор

Начнем с чтения данных и преобразования их в объект `Series`. Функция `pd.read_csv` является одной из наиболее распространенных в `pandas`, и предназначена она для чтения файлов CSV (или любых файлов, напоминающих этот формат). Как я уже упоминал ранее, функция `read_csv` возвращает датафрейм, и, даже если в исходном файле была всего одна колонка, нам вернется датафрейм с единственным столбцом. Чтобы преобразовать его в объект `Series`, можно воспользоваться удобным методом `squeeze`. Поскольку в нашем наборе данных присутствуют целочисленные значения, `pandas` присвоит нашему объекту `Series` тип данных `np.int64`.

Также при чтении файла мы передадим параметр `header=None`, чтобы первая строка в файле воспринималась не как заголовок, а как часть данных:

```
s = pd.read_csv('../data/taxi-passenger-count.csv',
               header=None).squeeze()
```

У полученного объекта `Series` будет присутствовать атрибут `name` со значением 0, но его можно игнорировать.

ПРИМЕЧАНИЕ. Хотя в большинстве случаев мы работаем в `pandas` с методами конкретных объектов (`Series`, датафрейм и т. д.), `read_csv` представляет собой функцию верхнего уровня в пространстве имен `pd`. Причина в том, что она не выполняет действия с определенным объектом, а создает новый объект на основе указанного файла.

После получения объекта `Series` нам остается выяснить, как часто в нем встречаются нужные нам значения. Это можно сделать при помощи индекса-маски и метода `count`, как показано ниже:

```
s.loc[s==1].count()      ❶
s.loc[s==6].count()      ❷
```

❶ Вернет 7207.

❷ Вернет 369.

Но нам необходимо получить процент поездок с одним и шестью пассажирами от общего числа заказов. Это можно сделать, разделив полученные результаты на `s.count()`:

```
s.loc[s==1].count() / s.count() ❶
s.loc[s==6].count() / s.count() ❷
```

❶ Вернет примерно .720772.

❷ Вернет примерно .036904.

Можно эту задачу решить и так, но в нашем распоряжении есть более мощная техника, предполагающая использование метода `value_counts` – одного из моих любимых. Если применить его к объекту `Series` с именем `s`, мы получим новый объект `Series`, в котором в качестве индексов будут выступать уникальные значения из исходного набора данных, а в качестве значений – их количество в выборке.

Обратившись к методу `s.value_counts()`, мы получим следующий вывод:

```
1    7207
2    1313
5     520
3     406
6     369
4     182
0         2
Name: 0, dtype: int64
```

Обратите внимание, что строки в выводе автоматически сортируются в порядке убывания частоты появления значений в наборе данных.

Поскольку мы снова получили объект `Series`, мы можем применять к нему любые доступные нам методы. К примеру, можно было бы воспользоваться методом `head` для получения пяти наиболее часто встречающихся значений. Также мы можем применить к нашему объекту причудливую индексацию, о которой говорили ранее, для извлечения нужных значений. Поскольку нас интересуют поездки с одним и шестью пассажирами, мы можем написать следующее выражение:

```
s.value_counts()[[1,6]]
```

В результате получим такой вывод:

```
1    7207
6     369
Name: 0, dtype: int64
```

Но нам нужны не абсолютные показатели, а относительные. На этот случай метод `value_counts` располагает удобным параметром `normalize` и выводит информацию в процентах, если ему передано значение `True`.

Таким образом, мы можем оформить вызов следующим образом:

```
s.value_counts(normalize=True)[[1,6]]
```

Результат будет следующим:

```
1    0.720772
6    0.036904
Name: 0, dtype: float64
```

Решение

```
import pandas as pd
from pandas import Series, DataFrame

s = pd.read_csv('../data/taxi-passenger-count.csv', header=None).squeeze()
s.value_counts(normalize=True)[[1,6]]
```

Дополнительные упражнения

1. Определите 25-й, 50-й (медиана) и 75-й проценти в нашем наборе данных. Можете ли вы заранее предположить, каким будем результат?
2. В каком проценте случаев такси перевозят троих, четверых, пятерых или шестерых пассажиров? Необходимо подсчитать общий процент.
3. Представьте, что вы отвечаете за выдачу лицензий на такси в Нью-Йорке. Как вы считаете, с учетом полученных данных какие машины необходимо чаще лицензировать: маленькие, вмещающие одного-двух пассажиров, или большие, способные перевозить пять или шесть пассажиров?

Ответы на дополнительные упражнения

Упражнение 6.1

```
# Поскольку поездки с одним пассажиром занимают 72% набора данных,
# можно предположить, что 25-й и 50-й проценти будут попадать на единицу,
# тогда как 75-й может попадать на двойку или тройку в зависимости
# от их популярности
```

```
s.quantile([.25, .50, .75])
```

Вывод:

```
0.25    1.0
0.50    1.0
0.75    2.0
Name: 0, dtype: float64
```

Упражнение 6.2

```
s.value_counts(normalize=True)[[3,4,5,6]].sum()
```

Вывод:

```
0.1477147714771477
```

Упражнение 6.3

С учетом большой популярности поездок с одним и двумя пассажирами логично предположить, что необходимо развивать парк небольших автомобилей.

УПРАЖНЕНИЕ 7. Длинные, средние и короткие поездки в такси

В этом упражнении мы вновь обратимся к набору данных о поездках в нью-йоркском такси за 2015 год. Но на этот раз нас будет интересовать не количество перевозимых пассажиров, а продолжительность поездок в милях. Создайте объект `Series` на основе данных из файла `taxi-distance.csv` из сопроводительных материалов. Затем создайте новый объект `Series` (или модифицируйте существующий), чтобы он содержал имена категорий поездок, а не их дистанцию в милях, согласно следующим критериям:

- `short` (короткая), если дистанция меньше или равна 2 милям;
- `medium` (средняя), если дистанция больше 2 миль, но меньше или равна 10 милям;
- `long` (долгая), если дистанция больше 10 миль.

Вычислите количество поездок в каждой из категорий.

Подробный разбор

Преобразование числовых значений в текстовые с группировкой по определенному критерию – довольно часто встречающаяся задача на практике. В этом упражнении мы хотим подразделить поездки на такси на короткие, средние и длинные. Как это можно сделать?

Один из подходов состоит в использовании комбинации операций сравнения и присваивания, как показано ниже:

```
categories = s.astype(str) ❶
categories.loc[:] = 'medium' ❷
categories.loc[s<=2] = 'short' ❸
categories.loc[s>10] = 'long' ❹
categories.value_counts()
```

- ❶ Создаем новый объект `Series` такой же длины.
- ❷ Присваиваем всем поездкам категорию `medium`.
- ❸ Небольшие значения меняем на `short`.
- ❹ Большие значения меняем на `long`.

Теперь при вызове метода `value_counts` мы получим следующий результат:

```
short      5890
medium     3402
long        707
Name: 0, dtype: int64
```

Этот подход работает, но он, согласитесь, не отличается изяществом, в отличие от функции `pd.cut`, позволяющей задать границы и разбить значения массива

данных на категории (известные как *корзины* (bin)). Более того, эта функция присваивает созданным корзинам метки.

Обратите внимание, что `pd.cut` представляет собой не метод, а функцию верхнего уровня в пространстве имен `pd`. Мы передадим ей следующие значения в виде аргументов:

- наш объект Series с именем `s`;
- список из четырех целых значений, представляющих границы для будущих категорий поездок (параметр `bins`);
- список из трех текстовых меток для наших корзин (параметр `labels`).

Заметим, что границы корзин задаются включительно с правой стороны и не включительно – с левой. Иными словами, определяя границы для корзины со средними по продолжительности поездками в 2 и 10 миль, мы будем относить к этой категории поездки с продолжительностью, строго превышающей 2 мили и равной или меньшей чем 10 миль. Из этого следует, что первую границу стоит определять как число, меньшее минимального значения в нашем наборе данных. Изменить это поведение, заданное по умолчанию, можно, передав параметру `include_lowest` функции `pd.cut` значение `True`. Это приведет к включению нижней границы в корзину.

В результате вызова функции `pd.cut` мы получим новый объект Series той же длины, что и `s`, но с выбранными метками в виде значений, как показано ниже:

```
pd.cut(s,
       bins=[0, 2, 10, s.max()],
       include_lowest=True,
       labels=['short', 'medium', 'long'])
```

Результат будет следующим (показан фрагмент):

```
0      short
1      short
2      short
3      medium
4      short
...
9994   medium
9995   medium
9996   medium
9997   short
9998   medium
Name: 0, Length: 9999, dtype: category ❶
Categories (3, object): ['short' < 'medium' < 'long'] ❷
```

- ❶ Обратите внимание на тип данных `dtype: category`. Подробнее мы поговорим об этом типе позже.
- ❷ Показывает относительный порядок следования категорий в их описании.

Но наша задача состоит не в том, чтобы преобразовать числовые данные в текстовые категории поездок, а в том, чтобы определить количество поездок в каждой из категорий. Для этого обратимся к нашему старому другу – методу `value_counts`:

```
pd.cut(s,
      bins=[0, 2, 10, s.max()],
      include_lowest=True,
      labels=['short', 'medium', 'long']).value_counts()
```

Вполне ожидаемо этот метод даст нам ответы на все интересующие нас вопросы:

```
short      5890
medium     3402
long        707
Name: 0, dtype: int64
```

Решение

```
import pandas as pd
from pandas import Series, DataFrame

s = pd.read_csv('../data/taxi-distance.csv', header=None).squeeze()

pd.cut(s,
      bins=[0, 2, 10, s.max()],
      include_lowest=True,
      labels=['short', 'medium', 'long']).value_counts()
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/7vx9>.

Дополнительные упражнения

1. Сравните среднюю и медианную продолжительность поездок на такси. Что это скажет вам о распределении данных в наборе?
2. Каково было количество коротких, средних и длинных поездок с одним пассажиром? Обратите внимание, что данные о перевозке пассажиров и продолжительности поездок взяты из одного источника, а значит, их индексы совпадают.
3. Что будет, если не передавать в функцию `pd.cut` интервалы явно, а задать только количество корзин (`bins=3`)?

Ответы на дополнительные упражнения

Упражнение 7.1

```
s.describe()
```

Вывод:

```
count      9999.000000
mean         3.158511
std          4.037516
min          0.000000
25%          1.000000
50%          1.700000
```

```

75%          3.300000
max          64.600000
Name: 0, dtype: float64

```

Поскольку среднее значение продолжительности поездки существенно превышает медиану, можно сделать вывод о том, что в наборе данных присутствует несколько очень дальних поездок, которые сильно смещают среднее значение в сторону увеличения. Это можно понять и по тому, что при стандартном отклонении в 4 мили у нас в наборе есть как минимум одна поездка более чем на 64 мили.

Упражнение 7.2

```

passenger_count = pd.read_csv('../data/taxi-passenger-count.csv',
header=None).squeeze()

```

```

pd.cut(s[passenger_count == 1],
      bins=[s.min(), 2, 10, s.max()],
      include_lowest=True,
      labels=['short', 'medium', 'long']).value_counts()

```

Вывод:

```

0
short      4333
medium     2387
long        487
Name: count, dtype: int64

```

Упражнение 7.3

```

passenger_count = pd.read_csv('../data/taxi-passenger-count.csv',
header=None).squeeze()

```

```

pd.cut(s[passenger_count == 1],
      bins=3,
      labels=['short', 'medium', 'long'], retbins=True)[-1]

```

Вывод:

```

array([-0.0646      , 21.53333333, 43.06666667, 64.6      ])

```

```

pd.cut(s[passenger_count == 1],
      bins=3,
      labels=['short', 'medium', 'long']).value_counts()

```

Вывод:

```

0
short      7179
medium       26
long         2
Name: count, dtype: int64

```

Функция `pd.cut` принимает интервал от `s.min()` до `s.max()` и делит его на три равные части, относя их к категориям `short`, `medium` и `long`. Как видим, к категории долгих поездок у нас были отнесены поездки на расстояние от 43 до 64.6 миль. По расстоянию это ровно треть, но эта корзина включает в себя всего пару значений.

Заклучение

В этой главе мы увидели, как можно эффективно работать с объектом `Series` в `pandas` при анализе данных. Мы научились работать с простыми индексами, читать данные из файлов, вычислять показатели описательной статистики, получать значения с помощью причудливой индексации и даже разбивать имеющиеся значения на корзины по заданным критериям. В следующей главе мы перейдем к работе с датафреймами, представляющими собой двумерные данные в табличной форме, ведь именно с ними нам в большинстве случаев приходится взаимодействовать при работе с библиотекой `pandas`.

Глава 2

Объект DataFrame

С незапамятных времен люди привыкли анализировать информацию в табличном виде, а когда появились первые компьютеры, именно так в большинстве случаев они и стали представлять нужную им информацию. Каждая строка в таблице представляет одну запись, или наблюдение, а каждый столбец – конкретный атрибут, связанный с этим наблюдением. К примеру, в табл. 2.1 показана некоторая информация по странам из «Википедии» по состоянию на 2022 год.

Таблица 2.1. Статистические данные по странам

Страна	Площадь (кв. км.)	Население
США	9 833 520	331 893 745
Великобритания	93 628	67 326 569
Канада	9 984 670	38 654 738
Франция	248 573	67 897 000
Германия	357 022	84 079 811

Для нас кажется вполне естественным именно такое представление информации, поскольку за долгие годы мы к этому привыкли. Лишь за последние несколько дней я могу назвать десятки примеров табличных данных, с которыми сталкивался. Несколько примеров:

- *обновления биржевых данных:* в строках – акции и популярные индексы, а в столбцах – текущая цена и изменение цены в сравнении с предыдущим днем в абсолютном и относительном выражении;
- *правила перевозки багажа на международных рейсах:* в строках – различные категории билетов, а в столбцах – объем и вес багажа, допустимые для перевозки;
- *информация о пищевой ценности на фасованном товаре:* в строках – интересующие нас показатели, такие как калории, жиры и углеводы, а в столбцах – количество на 100 г готового продукта.

Поскольку в столбце обычно хранится информация об одном атрибуте, или категории, вполне естественно, что данные по столбцам чаще всего характеризуются одним типом. В то же время строка может содержать совершенно разнородные

данные, поскольку, по сути, она является неким срезом по столбцам. Добавление нового столбца означает добавление измерения, или аспекта, в каждую запись (строку). А добавление записи характеризуется появлением строки со значениями по всем столбцам.

Компьютеры уже много десятилетий обрабатывают табличные данные, и наибольших высот в этом добились так называемые табличные процессоры, такие как Excel. Pandas предстал продолжателем этой традиции, представив новый тип данных для таблиц, называемый *датафреймом* (data frame). Каждый столбец в датафрейме представляет собой объект Series. В датафрейме присутствует один индекс, распространяющийся на все столбцы. В некотором смысле датафрейм можно назвать коллекцией объектов Series, объединенной общим индексом.

Поскольку столбцы в датафрейме представлены отдельными объектами Series, для каждого из них можно задать свой атрибут dtype. К примеру, в датафрейме могут успешно сосуществовать столбцы с целочисленным типом, типом с плавающей точкой и строковым типом, как показано на рис. 2.1.

Индекс	Country	Area (sq km)	Population
0	United States	9,833,520	331,893,745
1	United Kingdom	93,628	67,326,569
2	Canada	9,984,670	38,654,738
3	France	248,573	67,897,000
4	Germany	357,022	84,079,811

Строки

Числовые столбцы

Столбец с текстом

Имена столбцов

Рис. 2.1. Данные, представленные в табл. 2.1, в виде датафрейма pandas

В датафрейме зачастую хранится больше информации, чем нам требуется для анализа. Перед тем как работать с данными и анализировать их, нужно выделить поднабор данных, состоящий из нужных нам строк и столбцов. В этой главе мы научимся делать это – оставлять для анализа только нужные строки и столбцы на основании различных критериев. Мы узнаем, как применять атрибут доступа loc, булевы индексы и различные методы pandas для сложной фильтрации данных на основе запроса. В главе 3 мы поговорим о способах импорта данных из внешних источников, а в главе 5 – о тонкостях очистки данных и приведении их в устойчивое надежное состояние.

Мы попрактикуемся в создании, модифицировании и обновлении датафреймов. Иногда обновлять датафреймы требуется при поступлении новых данных, чтобы они отражали эти изменения, а иногда – когда нам нужно очистить данные, удалив или исправив некорректные значения.

После прочтения этой главы вы будете себя комфортно чувствовать при работе с датафреймами. В следующих главах мы будем опираться на приобретенные здесь знания, которые понадобятся вам для организации данных в удобном для вас виде.

В табл. 2.2 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 2.2. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
DataFrame	Возвращает новый датафрейм на основе двумерных данных	DataFrame([[10, 20], [30, 40], [50, 60]])	http://mng.bz/d1xz (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html#pandas.DataFrame)
s.loc	Позволяет осуществлять доступ к элементам объекта Series по меткам или с помощью массива булевых значений	s.loc['a']	http://mng.bz/zXlZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.loc.html)
df.loc	Позволяет осуществлять доступ к одной или нескольким строкам датафрейма посредством индекса	df.loc[5]	http://mng.bz/V1Pr (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html)
s.iloc	Позволяет осуществлять доступ к элементам объекта Series по позиции	s.iloc[0]	http://mng.bz/x4lq (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.iloc.html)
df.iloc	Позволяет осуществлять доступ к одной или нескольким строкам датафрейма по позиции	df.iloc[5]	http://mng.bz/AoNE (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.iloc.html)
[]	Позволяет осуществлять доступ к одному или нескольким столбцам датафрейма	df['a']	http://mng.bz/ZqeJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.html)
df.assign	Позволяет добавить один или несколько столбцов в датафрейм	df.assign(a=df['x']*3)	http://mng.bz/OPln (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.assign.html)

Таблица 2.2. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>str.format</code>	Этот метод работает подобно f-строкам	<code>'ab{0}'.format(5)</code>	http://mng.bz/YR5N (https://docs.python.org/3/library/stdtypes.html#str.format)
<code>s.quantile</code>	Позволяет получить элемент, соответствующий определенному процентилю	<code>s.quantile(0.25)</code>	http://mng.bz/RxPn (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.quantile.html)
<code>pd.concat</code>	Позволяет объединить вместе два датафрейма	<code>df = pd.concat([df, new_products])</code>	http://mng.bz/2DJN (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html)
<code>df.query</code>	Позволяет писать запросы к датафреймам в стиле, похожем на SQL	<code>df.query('v > 300')</code>	http://mng.bz/1qwZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.query.html)
<code>pd.read_csv</code>	Позволяет прочитать содержимое файла CSV в виде датафрейма	<code>df = pd.read_csv('filename.csv')</code>	http://mng.bz/PzO2 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
Interpolate	Возвращает новый датафрейм со значениями NaN, заполненными при помощи интерполяции	<code>df = df.interpolate()</code>	http://mng.bz/Jgzp (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.interpolate.html)
<code>df.dropna</code>	Возвращает новый датафрейм без значений NaN	<code>df.dropna()</code>	http://mng.bz/o1PN (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html)
<code>s.isin</code>	Возвращает объект Series с булевыми значениями, показывающими, присутствует ли элемент последовательности в переданном в качестве аргумента списке	<code>s.isin([10, 20, 30])</code>	http://mng.bz/9D08 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isin.html)

Скобки или точки?

При работе с объектами Series мы можем извлекать элементы несколькими способами, среди которых использование индекса (`loc`), позиции (`iloc`) и квадратных скобок, являющихся эквивалентом метода `loc` для простых случаев. Однако для извлечения строк из датафреймов можно применять только методы `loc` и `iloc`, поскольку квадратные скобки служат для указания выбираемых столбцов.

Давайте создадим следующий датафрейм:

```
df = DataFrame([[10, 20, 30, 40],
                [50, 60, 70, 80],
                [90, 100, 110, 120]],
               index=list('xyz'),
               columns=list('abcd'))
```

Обратиться к столбцу с именем `a` в этом датафрейме можно при помощи выражения `df['a']`, а для получения нового датафрейма на основе существующего, в котором будут содержаться только два столбца из четырех, необходимо перечислить эти столбцы в виде списка. Тем самым мы получим список, вложенный в квадратные скобки, что приведет к их задвоению: `df[['a', 'b']]`. Если мы попытаемся выполнить выражение `df['x']`, то pandas будет пытаться извлечь столбец из датафрейма с именем `x` и, не обнаружив его, вернет исключение `KeyError`. Для извлечения строки с индексом `x` из датафрейма нужно написать `df.loc['x']` или, если вы хотите получить доступ к строке по позиции, `df.iloc[0]`.

Но из правила обращения к столбцам с помощью квадратных скобок есть одно исключение. Если указать в квадратных скобках срез, то pandas будет извлекать диапазон строк по индексу. Таким образом, мы можем получить строки из нашего датафрейма с индексами `x` по `y`, написав выражение `df['x': 'y']`. Срез указывает pandas на то, что мы хотим извлечь именно строки, а не столбцы. Более того, строки будут извлечены до индекса `y` включительно, что нетипично для Python, но вполне типично для метода `loc`. Все эти методы доступа показаны на рис. 2.2.

Еще одним способом работы со столбцами в датафрейме является использование *точечной нотации* (dot notation). То есть для обращения к столбцу с именем `colname` в датафрейме `df` можно написать `df.colname`.

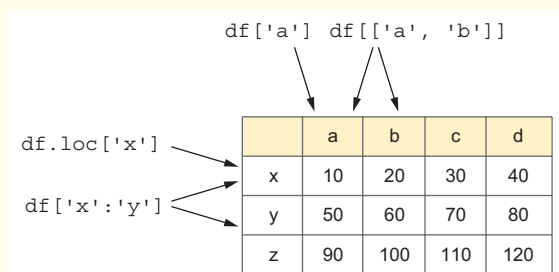


Рис. 2.2. Разные методы доступа к столбцам и строкам в датафрейме

Этому синтаксису многие разработчики отдают предпочтение, и на то есть свои причины: во-первых, он проще для написания, во-вторых, он лаконичнее, а значит, легче читается, да и выглядит более естественно для программистов.

Но есть и доводы против использования этого синтаксиса. В частности, он не допускает использования в именах колонок пробелов и иных специальных символов. Кроме того, при виде выражения `df.whatever` легко можно забыть, что имеется в виду под `whatever` – атрибут или имя столбца.

Лично я предпочитаю нотацию с квадратными скобками и именно ее буду использовать в этой книге. Если вы больше любите точки, знайте, что вы не одиноки, но не забывайте, что применить эту нотацию вы сможете далеко не всегда.

УПРАЖНЕНИЕ 8. Чистый доход

Разработчики на `pandas` редко создают датафреймы с нуля. Обычно они загружают их из файлов CSV или получают путем преобразования существующих датафреймов (одного или нескольких). Но время от времени приходится строить датафреймы с чистого листа – например, при извлечении данных из нестандартных источников или экспериментировании с новыми техниками `pandas`. Так что знать о способах создания датафреймов нужно.

В этом упражнении вы должны будете создать датафрейм с представлением складских запасов компании по пяти товарам. Каждый товар обладает своим уникальным идентификатором (двухзначного целого числа хватит), а также характеризуется наименованием, оптовой и розничной ценой и объемом продаж за последний месяц. Предметная область – на ваш выбор, так что, если вы всегда хотели стать продавцом современных звездолетов, пришел ваш час! После создания датафрейма рассчитайте общий доход по всем товарам в ассортименте.

Подробный разбор

Первая часть упражнения состоит в создании датафрейма. Для этого необходимо передать нужные параметры классу `DataFrame`. Сделать это можно четырьмя способами:

- передать список списков, как показано на рис. 2.3. Каждый вложенный список при этом соответствует отдельной строке в датафрейме. При этом все вложенные списки должны быть одной длины, а значения в них должны быть расположены в соответствии с позициями столбцов;
- передать список словарей, как показано на рис. 2.4. Каждый словарь в этом случае представляет отдельную строку, а имя ключа должно соответствовать имени столбца;

```
df = DataFrame([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]],
    index = list('xyz'),
    columns=list('abcd'))
```

	a	b	c	d
x	10	20	30	40
y	50	60	70	80
z	90	100	110	120

Рис. 2.3. Создание датафрейма на основе списка списков.

Каждый вложенный список – это строка. Имена столбцов располагаются по позициям

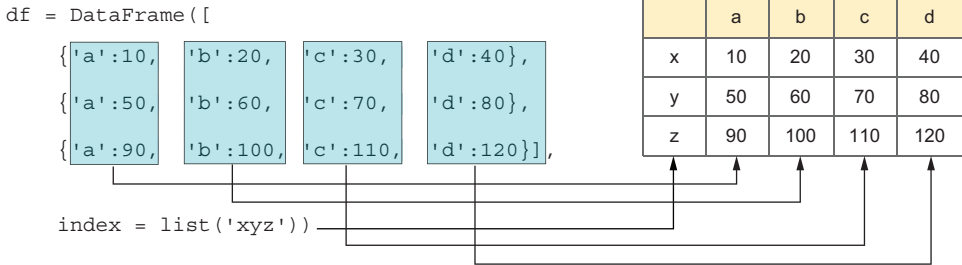


Рис. 2.4. Создание датафрейма на основе списка словарей.

Каждый словарь – это строка. Имена ключей соответствуют именам столбцов

- передать словарь со списками в виде значений, как показано на рис. 2.5. Каждый ключ в словаре представляет отдельный столбец, а значения ключа (список) соответствуют значениям в столбце;
- передать двумерный массив NumPy, как показано на рис. 2.6.

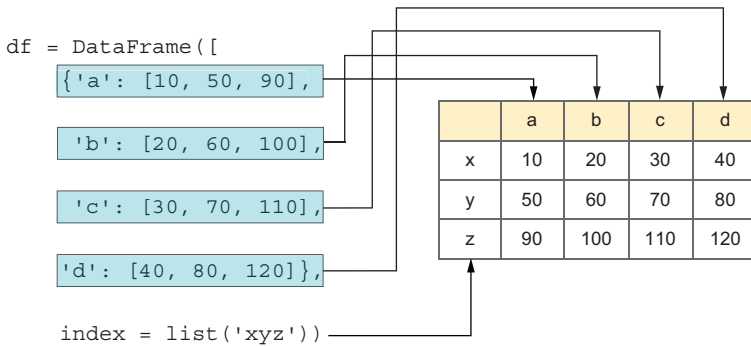


Рис. 2.5. Создание датафрейма на основе словаря со списками.

Каждый ключ – это имя столбца, а значения в виде списка соответствуют значениям этого столбца

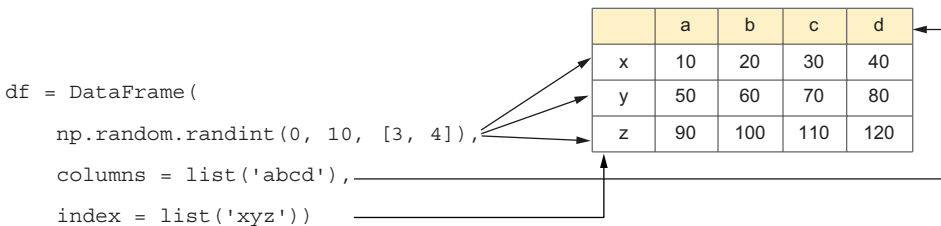


Рис. 2.6. Создание датафрейма на основе двумерного массива NumPy

Какую технику выбрать – зависит от конкретной задачи. В данном случае, поскольку мы хотим вносить информацию в датафрейм по конкретным товарам, мы воспользуемся способом с передачей списка словарей.

Преимуществом этого подхода является то, что нам нет необходимости передавать отдельно имена столбцов, pandas вычленил их из ключей в словарях. В качестве индекса будет использоваться позиционный индекс по умолчанию, так что мы не будем его трогать.

Как после создания датафрейма мы сможем рассчитать общий доход по нашим товарам? Для этого нужно для каждого товара вычесть оптовую цену из розничной, в результате чего мы получим сумму чистого дохода:

```
df['retail_price'] - df['wholesale_price']
```

Здесь мы извлекли объект Series, соответствующий `df['retail_price']`, и вычли его из объекта `df['wholesale_price']`. Поскольку эти объекты представляют собой столбцы в одном датафрейме, индексы у них будут совпадать, а значит, вычитание будет выполнено для каждой строки, и в результате мы получим новый объект Series с тем же индексом и значениями, содержащими разницу между двумя столбцами с ценами.

Осталось умножить значения из полученного объекта Series на объемы продаж, располагающиеся в столбце с именем `sales`:

```
(df['retail_price'] - df['wholesale_price']) * df['sales'] ❶
```

❶ Без использования круглых скобок оператор умножения сработал бы первым.

В результате мы получим новый объект Series с тем же индексом, что и у датафрейма `df`, и с общей суммой по каждому товару. Суммировать полученные значения можно при помощи метода `sum`, как показано ниже и на рис. 2.7.

```
((df['retail_price'] - df['wholesale_price']) * df['sales']).sum() ❶
```

❶ Внешние скобки говорят pandas о необходимости применения операции `sum` к результату умножения, а не к столбцу `df['sales']`.

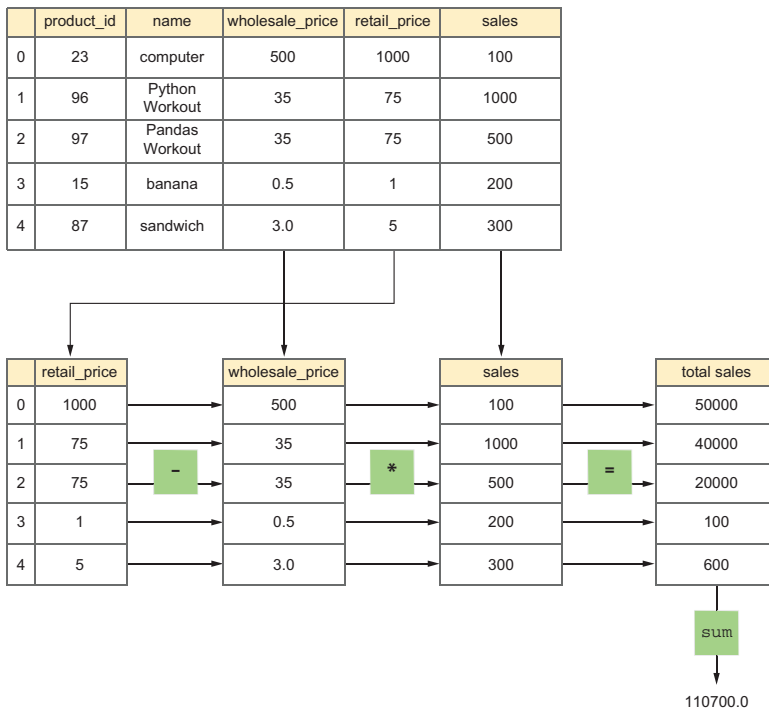


Рис. 2.7. Схематическое решение упражнения 8

Решение

```
df = DataFrame([{'product_id':23, 'name':'computer', 'wholesale_price': 500,
                'retail_price':1000, 'sales':100},
                {'product_id':96, 'name':'Python Workout', 'wholesale_price': 35,
                'retail_price':75, 'sales':1000},
                {'product_id':97, 'name':'Pandas Workout', 'wholesale_price': 35,
                'retail_price':75, 'sales':500},
                {'product_id':15, 'name':'banana', 'wholesale_price': 0.5,
                'retail_price':1, 'sales':200},
                {'product_id':87, 'name':'sandwich', 'wholesale_price': 3,
                'retail_price':5, 'sales':300},
                ])
((df['retail_price'] - df['wholesale_price']) * df['sales']).sum() ❶
```

❶ Вернет 110 700.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/0lAx>.

Дополнительные упражнения

1. Для каких товаров розничная цена более чем вдвое превосходит оптовую?
2. Сравните доход компании от продажи продуктов, компьютеров и книг. Вы можете ориентироваться просто на индексы, не нужно выдумывать ничего сложного.
3. Ваша компания неплохо зарабатывает, так что вы можете себе позволить дать 30-процентную скидку на все оптовые цены. Рассчитайте новый чистый доход после этого изменения.

Ответы на дополнительные упражнения

Упражнение 8.1

```
# Глядите-ка, как выгодно торговать книгами!
df['name'][df['retail_price'] * 0.5 > df['wholesale_price']]
```

Вывод:

```
1    Python Workout
2    Pandas Workout
Name: name, dtype: object
```

Упражнение 8.2

```
# Компьютеры
((df['retail_price'] - df['wholesale_price']) * df['sales'])[0].sum()
```

Вывод:

```
50000.0
```

```
# Книги
((df['retail_price'] - df['wholesale_price']) * df['sales'])[1,2].sum()
```

Вывод:

60000.0

Продукты

```
((df['retail_price'] - df['wholesale_price']) * df['sales'])[3,4].sum()
```

Вывод:

700.0

Упражнение 8.3

```
((df['retail_price'] - df['wholesale_price']*0.7) * df['sales']).sum()
```

Вывод:

141750.0

УПРАЖНЕНИЕ 9. *Налоговое планирование*

В предыдущем упражнении мы создали датафрейм, представляющий данные о продажах различных товаров. А сейчас мы расширим этот пример, причем в буквальном смысле. Добавление новых столбцов в существующие датафреймы – весьма распространенная практика. Это может потребоваться как для хранения новой информации, так и для построения вычислений на основе существующих в наборе данных столбцов, чем мы сейчас и займемся. Одной из причин добавления столбцов в датафрейм является необходимость хранить промежуточные данные для расчетов.

Итак, перейдем к упражнению. Руководство региона подумывает о введении налога на продажу и рассматривает три возможных варианта: 15, 20 и 25 %. Наглядно покажите, насколько снизится ваш чистый доход в каждом из случаев путем добавления столбцов в датафрейм, отражающих ваш текущий доход и потенциальный доход для каждого из перечисленных вариантов ставок.

Подробный разбор

Если два объекта Series располагают одинаковым индексом, мы можем спокойно выполнять над ними любые арифметические операции, не опасаясь за целостность данных. Результатом будет новый объект Series с таким же индексом, как у исходных объектов. Часто, как в упражнении 8, нам приходится выполнять вычисления на основе двух столбцов в датафрейме (в конце концов, это просто объекты Series) и отображать результат.

Но иногда необходимо сохранять полученные в результате данные в датафрейме для дальнейшего их использования, что мы и продемонстрируем в этом упражнении.

Как же можно добавить столбец в уже существующий датафрейм? На удивление просто. Для этого достаточно присвоить столбцу с вымышленным названием значение, обычно являющееся объектом Series, но можно также использовать массив NumPy или список, если длина этих объектов такая же, как у исходных

столбцов в датафрейме. Имена столбцов в датафрейме уникальны, так что, как и в случае со словарями, присваивание новых значений существующему столбцу просто перезапишет его.

В предыдущем упражнении мы рассчитывали общую сумму продажи для каждого товара. Сейчас мы возьмем это выражение и присвоим результат новому столбцу в датафрейме, что приведет к его созданию:

```
df['current_net'] = ((df['retail_price'] - df['wholesale_price']) * df['sales'])
```

Добавление столбцов путем использования метода assign

Еще одним способом добавить новый столбец в датафрейм в pandas является использование метода `assign`, который возвращает новый датафрейм, а не изменяет существующий. К примеру, запись

```
df['current_net'] = ((df['retail_price'] - df['wholesale_price']) * df['sales'])
```

можно было бы заменить на

```
df.assign(current_net = (df['retail_price'] - df['wholesale_price']) * df['sales'])
```

Ключевые аргументы, переданные методу `assign`, превращаются в новые столбцы с именами, соответствующими именам аргументов.

Использование метода `assign` бывает полезно, когда нам нужно произвести достаточно сложные расчеты. В этом случае можно открыть скобки и прописать все требуемые операции на отдельных строках. Некоторым разработчикам такой подход приходится по душе, поскольку они находят его более простым для чтения и более воспроизводимым по сравнению с обычным присваиванием. Лично мне кажется, что при необходимости реализовывать достаточно сложную логику запросов, состоящую из нескольких шагов, этот метод действительно подходит лучше. Многие примеры в этой книге будут написаны с использованием такого многострочного синтаксиса. Попробуйте писать свои запросы именно в таком стиле, ведь многие считают его более наглядным и простым для отладки.

Итак, что будет, если мы применим налоговую ставку на уровне 15 %? Это приведет к снижению нашего чистого дохода на 15 %, что мы можем зафиксировать в новом столбце с именем `after_15`, как показано ниже:

```
df['after_15'] = df['current_net'] * 0.85
```

Подобным образом мы можем создать еще два столбца для оставшихся вариантов налоговой ставки:

```
df['after_20'] = df['current_net'] * 0.80
```

```
df['after_25'] = df['current_net'] * 0.75
```

После выполнения этой операции наш датафрейм будет насчитывать девять столбцов: `product_id`, `name`, `wholesale_price`, `retail_price`, `sales`, `current_net`, `after_15`, `after_20` и `after_25`. Поскольку добавленные четыре столбца имеют числовой тип, мы можем выделить их в отдельный датафрейм с тем же индексом, что

и в исходном датафрейме, для дальнейших вычислений. Воспользуемся для этого описанной выше техникой причудливой индексации:

```
df[['current_net', 'after_15', 'after_20', 'after_25']]
```

Применив метод `sum` к полученному датафрейму, мы получим сумму по каждой колонке. При этом результат будет возвращен в виде объекта `Series`, в котором имена столбцов перекоچуют в индекс, как показано ниже:

```
current_net    110700.0
after_15       94095.0
after_20       88560.0
after_25       83025.0
dtype: float64
```

В результате мы видим, сколько мы заработали бы при каждой ставке налога. Мы также можем показать разницу в доходах для каждого из сценариев, применив технику транслирования к операции вычитания:

```
df['current_net'].sum() - df[['current_net',
                             'after_15', 'after_20', 'after_25']].sum()
```

Решение

```
df['current_net'] = ((df['retail_price'] - df['wholesale_price'])
                    * df['sales'])
df['after_15'] = df['current_net'] * 0.85
df['after_20'] = df['current_net'] * 0.80
df['after_25'] = df['current_net'] * 0.75
df[['current_net', 'after_15', 'after_20', 'after_25']].sum()
```

Вы можете посмотреть это решение на сайте *Pandas Tutor* по адресу <http://mng.bz/K98K>.

Дополнительные упражнения

1. Альтернативный план налогообложения состоит в применении налоговой ставки 25 %, но только для тех товаров, по которым наш чистый доход превышает 20 000. Если применить этот план, как изменится благосостояние нашей компании?
2. Еще один вариант плана налогообложения состоит в установке величины налога в 25 % на товары с розничной ценой, превышающей 80, 10 % – на товары с розничной ценой между 30 и 80 и освобождении от налога всех остальных товаров. Воспроизведите в нашем примере такой сценарий налогообложения.
3. Все эти длинные числа с плавающей точкой очень трудно читать. Установите в *pandas* опцию `float_format` таким образом, чтобы числа с плавающей точкой отображались с запятыми, разделяющими разряды (три знака), точкой и лишь двумя десятичными знаками после нее. Это непростое задание, требующее понимания вызываемых объектов в *Python* и метода `str.format`.

Извлечение и присваивание с помощью атрибута loc

Нет ничего проще, чем извлечь целую строку из датафрейма или даже заменить ее новыми значениями. К примеру, мы можем легко получить строку с индексом `abcd` с помощью следующей записи: `df.loc['abcd']`. Если вы предпочитаете использовать позиционные индексы, добиться аналогичного результата можно так: `df.iloc[5]`. В обоих случаях мы получим объект `Series`, созданный на лету из значений в указанной строке. Напротив, при извлечении столбца нам ничего нового создавать не придется, поскольку каждый столбец датафрейма хранится в памяти в виде объекта `Series`.

А что, если нам необходимо извлечь лишь часть строки? Или, что более важно, как нам установить значения лишь для части строки?

Мы можем сделать это несколькими способами, но лично я предпочитаю подход с использованием атрибута `loc` с двумя аргументами в квадратных скобках. Первый из них описывает строки, которые мы хотим извлечь (селектор строк), а второй – столбцы (селектор столбцов).

Предположим, у нас есть датафрейм размером 5 на 5 с индексами `a–e`, столбцами с именами `v–z` и поступательно увеличивающимися значениями от 10 до 250, как показано на рис. 2.8.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.8. Разные методы доступа к столбцам и строкам в датафрейме

Чтобы извлечь строку с индексом `a`, можно воспользоваться записью `df.loc['a']`. А чтобы получить в этой строке только значение из столбца `x`, нужно написать `df.loc['a', 'x']`.

В случае с длинными и сложными аргументами вы можете разместить их на разных строках, как показано ниже:

```
df.loc['a',    ❶
      'x']    ❷
```

❶ Селектор строк.

❷ Селектор столбцов.

Результат показан на рис. 2.9.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.9. Схематическое изображение выражения `df.loc['a', 'x']`

Поняв этот синтаксис, вы сможете использовать его в более сложных сценариях. К примеру, так можно извлечь строки с индексами a и c из столбца x. Схема доступа показана на рис. 2.10.

```
df.loc[['a', 'c'],  
       'x']
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.10. Схематическое изображение выражения `df.loc[['a', 'c'], 'x']`

Обратите внимание, что можно использовать причудливую индексацию для описания строк, которые необходимо получить, вместе с обычным индексом (второе значение в квадратных скобках) для указания требуемого столбца. Таким образом, мы можем заодно извлечь и несколько столбцов. В примере ниже мы получаем значения из строки с индексом a в столбцах с именами v и y, как видно на рис. 2.11.

```
df.loc['a',  
       ['v', 'y']]
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.11. Схематическое изображение выражения `df.loc['a', ['v', 'y']]`

А что, если скомбинировать эти требования и извлечь элементы на пересечении строк с индексами a и c и столбцов с именами v и y? Результат показан на рис. 2.12:

```
df.loc[['a', 'c'],  
       ['v', 'y']]
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.12. Схематическое изображение выражения `df.loc[['a', 'c'], ['v', 'y']]`

Но можно продвинуться еще дальше и использовать в качестве селектора строк булев индекс. Объект Series с булевыми значениями можно создать с помощью условных операторов, таких как `<` или `==`, и применить его в виде маски к строкам и/или столбцам. К примеру, так мы можем извлечь все строки, в которых значение в столбце x превышает 200 (схема показана на рис. 2.13):

```
df.loc[df['x'] > 200]
```

- ❶ Селектор строк, а селектора столбцов нет.

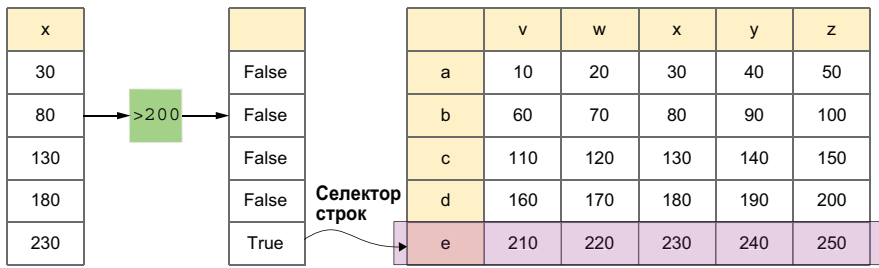


Рис. 2.13. Схематическое изображение выражения `df.loc[df['x'] > 200]`

Теперь мы можем добавить в выражение еще один индекс с булевыми значениями после запятой, показывающий, какие столбцы нам нужны (результат продемонстрирован на рис. 2.14):

```
df.loc[df['x'] > 200, ❶  
       df.loc['c'] > 135] ❷
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

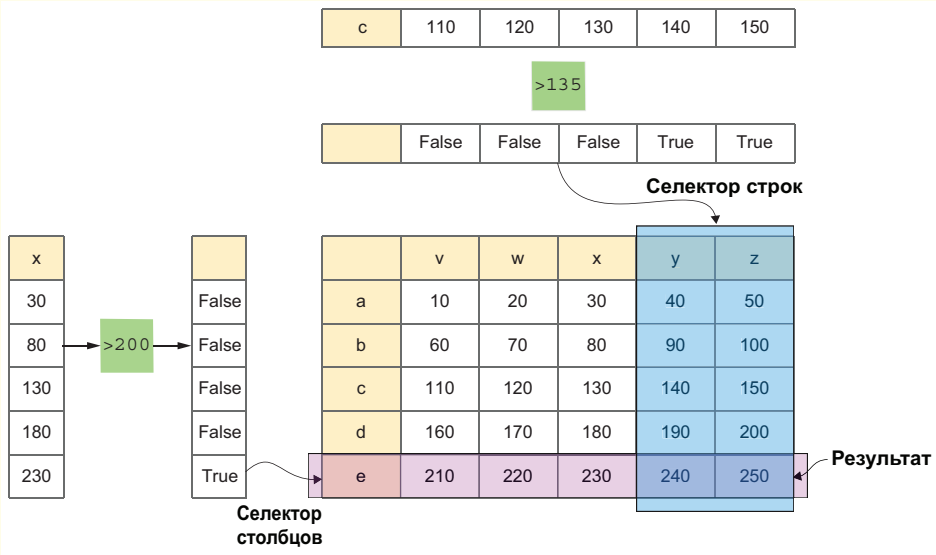


Рис. 2.14. Схематическое изображение выражения `df.loc[df['x'] > 200, df.loc['c'] > 135]`

Здесь мы вернули все строки из датафрейма `df`, в которых значение в столбце `x` превышает 200, и столбцы, в которых значение в строке с индексом `c` превышает 135.

Можно немного откатиться назад и написать выражение для извлечения значений из строки с индексом `b`, но только для тех столбцов, в которых значение в строке с индексом `c` превышает 135 (рис. 2.15):


```
df.loc['b',           ❶
      df.loc['c']>135] ❷
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

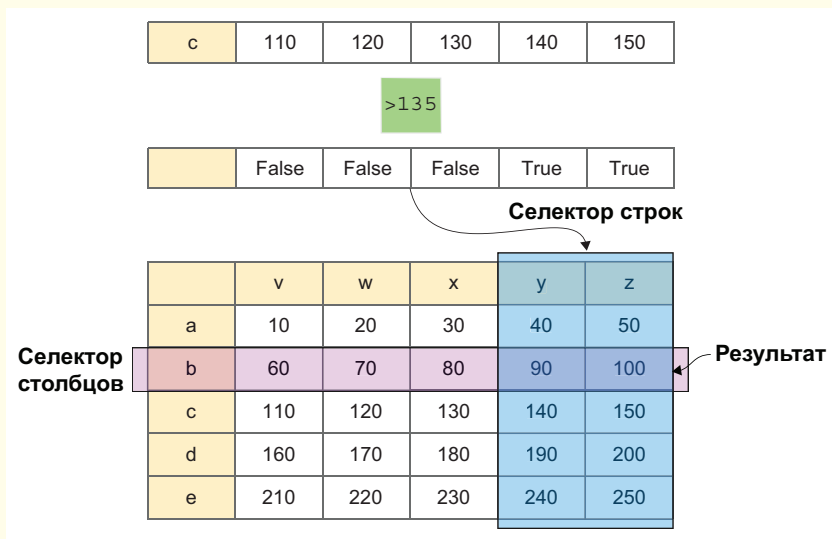


Рис. 2.15. Схематическое изображение выражения `df.loc['b', df.loc['c'] > 135]`

Разумеется, мы можем задавать и более сложные условия. Главное – помнить, что первым параметром в квадратных скобках задается критерий отбора строк, а вторым – столбцов, и все будет в порядке.

Во всех показанных примерах мы извлекали значения из датафрейма. А что, если нам необходимо *модифицировать* эти значения? Это можно сделать, поместив запрос для извлечения слева от оператора присваивания. Единственным нюансом здесь является то, что присваиваемое значение должно либо быть скаляром, чтобы при помощи транслирования его можно было распространить на все ячейки слева, либо иметь сопоставимую форму в отношении количества строк и столбцов.

Допустим, нам нужно в строке с индексом `b` заменить значение в столбце с именем `y` на 123. Это можно сделать следующим образом (рис. 2.16):

```
df.loc['b',           ❶
      'y'             ❷
      ] = 123
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.16. Схематическое изображение выражения `df.loc['b', 'y'] = 123`

А если нам нужно установить новые значения элементов в строке с индексом `b`, для которых в строке с индексом `c` значения превышают 125? В этом случае мы можем присвоить нашему диапазону список (или массив NumPy, или объект Series) из трех элементов, что соответствует форме вывода нашего запроса (рис. 2.17):

```
df.loc['b',  
      df.loc['c'] > 125  
      ] = [123, 456, 789]
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

	v	w	x	y	z
a	10	20	30	40	50
b	60	70	80	90	100
c	110	120	130	140	150
d	160	170	180	190	200
e	210	220	230	240	250

Рис. 2.17. Схематическое изображение выражения `df.loc['b', df.loc['c'] > 125] = [123, 456, 789]`

Конечно, для успешного выполнения этой операции нам нужно точно знать, сколько значений нам понадобится. Но зачастую вы можете не знать этого заранее, а присваивать значения на основе другого столбца или даже значений из этой же выборки! К примеру, в следующем примере мы удвоим значения в строке с индексом `b`, для которых соответствующие значения в строке с индексом `c` делятся на 3 без остатка (рис. 2.18):

```
df.loc['b',  
      df.loc['c'] % 3 == 0  
      ] *= 2
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.
- ❸ Умножение значения на 2 с помощью оператора `*=`.

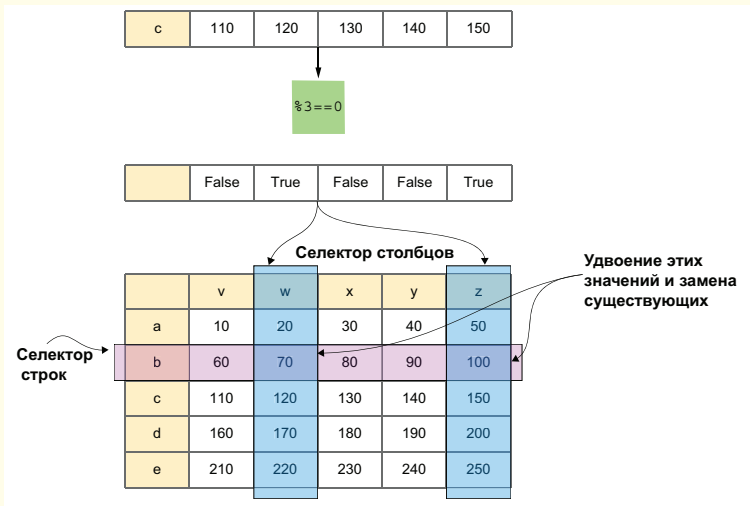


Рис. 2.18. Схематическое изображение выражения `df.loc['b', df.loc['c'] % 3 == 0] *= 2`

Также мы можем присвоить скалярное значение элементам, описанным с помощью атрибута `loc`, как показано ниже (рис. 2.19):

```
df.loc[df['v'] > 100, ❶
      df.loc['d'] > 180 ❷
      ] = 987
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

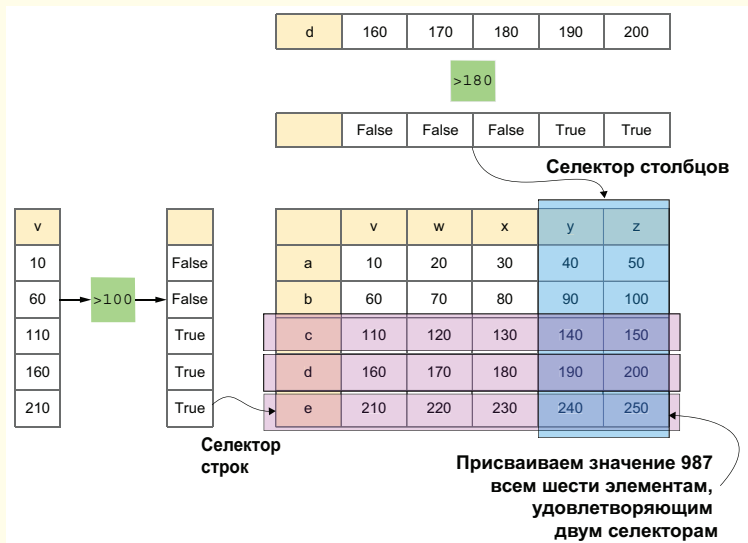


Рис. 2.19. Схематическое изображение выражения `df.loc[df['v'] > 100, df.loc['d'] > 150] = 987`

К этому синтаксису нужно привыкнуть. Но как только вы его освоите и впитаете, он покажется вам абсолютно понятным и гибким. Кроме того, этот подход позволяет писать очень эффективные выражения и не опасаться возникновения ошибок, которые характерны при применении квадратных скобок к выражениям, полученным ранее также с помощью квадратных скобок.

Ответы на дополнительные упражнения

Упражнение 9.1

```
# Проще всего реализовать это можно с помощью лямбды и встроенного if-else
df['current_net'].apply(lambda c: c*0.75 if c > 20000 else c).sum()
```

Вывод:

88200.0

```
# Более длинный подход состоит в написании отдельной функции с обычным if-else
def calculate_tax(c):
    if c > 20000:
        return c * 0.75
    return c
```

```
df['current_net'].apply(calculate_tax).sum()
```

Вывод:

88200.0

Упражнение 9.2

```
# Воспользуемся функцией pd.cut и переведем категории в числа с плавающей точкой
df['after_tax'] = pd.cut(df['retail_price'],
                        bins=[0, 30, 80, df['retail_price'].max()],
                        labels=[1, 0.9, 0.75]).astype(np.float64)
```

```
df['final_net'] = df['current_net'] * df['after_tax']
df
```

Вывод:

	product_id	name	wholesale_price	retail_price	sales	current_net	after_tax	final_net
0	23	computer	500.0	1000	100	50000.0	0.75	37500.0
1	96	Python Workout	35.0	75	1000	40000.0	0.90	36000.0
2	97	Pandas Workout	35.0	75	500	20000.0	0.90	18000.0
3	15	banana	0.5	1	200	100.0	1.00	100.0
4	87	sandwich	3.0	5	300	600.0	1.00	600.0

Упражнение 9.3

```
pd.options.display.float_format = '{:,.2f}'.format
df
```

Вывод:

	product_id	name	wholesale_price	retail_price	sales	current_net	after_tax	final_net
0	23	computer	500.00	1000	100	50,000.00	0.75	37,500.00
1	96	Python Workout	35.00	75	1000	40,000.00	0.90	36,000.00
2	97	Pandas Workout	35.00	75	500	20,000.00	0.90	18,000.00
3	15	banana	0.50	1	200	100.00	1.00	100.00
4	87	sandwich	3.00	5	300	600.00	1.00	600.00

УПРАЖНЕНИЕ 10. Добавление новых товаров

Отличные новости! Дела у вашего магазина пошли в гору, и вы решили расширить ассортимент товаров. При этом вам бы хотелось новые позиции собрать в отдельном датафрейме, а затем добавить его к существующему. В этом новом датафрейме должны содержаться три показанных ниже товара с идентификаторами, наименованиями, а также оптовой и розничной ценами:

- phone: ID = 24, оптовая цена = 200, розничная цена = 500;
- apple: ID = 16, оптовая цена = 0.5, розничная цена = 1;
- pear: ID = 17, оптовая цена = 0.6, розничная цена = 1.2.

Поскольку это новые товары в нашем ассортименте, мы не включаем для них столбец с продажами `sales`. Во избежание конфликтов необходимо обеспечить, чтобы значения индекса у новых товаров не совпадали с существующими (в главе 4 мы поговорим об индексах подробнее и научимся элегантно решать подобного рода проблемы).

После добавления новых товаров необходимо присвоить для них некие значения в столбце `sales`. В итоге нужно рассчитать наш чистый доход с учетом добавленных товаров.

Подробный разбор

Мы часто думаем о датафреймах как о таблицах с данными, которые мы уже ввели или загрузили из файла. Но датафреймы обладают намного большей гибкостью, позволяя нам представлять данные в любом виде и формате. Кроме того, данные в датафреймах могут меняться с течением времени либо в результате добавления новых сведений, либо в процессе их анализа и корректировки.

В этом упражнении вам для начала нужно создать новый датафрейм с тремя появившимися товарами. В нем должны быть все те же столбцы, что и в существующем, за исключением столбца `sales`.

Таким образом, первый шаг будет достаточно простым, ведь он напоминает процедуру создания датафрейма из начала этой главы. Единственное отличие будет состоять в том, что значения индекса мы будем устанавливать самостоятельно, с помощью функции `range` из базовой библиотеки Python, чтобы избежать коллизий с существующими товарами. Pandas все равно, есть ли у нас в индексе повторяющиеся значения, так что разработчикам приходится самим следить за этим.

Итак, создадим новый датафрейм следующим образом:

```
new_products = DataFrame([
    {'product_id':24, 'name':'phone', 'wholesale_price': 200, 'retail_price':500},
    {'product_id':16, 'name':'apple', 'wholesale_price': 0.5, 'retail_price':1},
    {'product_id':17, 'name':'pear', 'wholesale_price': 0.6, 'retail_price':1.2}
], index=range(5,8))
```

Теперь нам необходимо добавить созданный датафрейм к существующему. Для этого можно воспользоваться функцией `pd.concat`. Это верхнеуровневая функция библиотеки `pandas`, принимающая на вход список датафреймов для объединения.

В результате мы получим новый датафрейм, показанный на рис. 2.20, который снова присвоим переменной `df`:

```
df = pd.concat([df, new_products])
```

	product_id	name	wholesale_price	retail_price	sales
0	23	computer	500	1000.0	100.0
1	96	Python Workout	35	75.0	1000.0
2	97	Pandas Workout	35	75.0	500.0
3	15	banana	0.5	1.5	200.0
4	87	sandwich	3.0	5.0	300.0
5	24	phone	200.0	500.0	NaN
6	16	apple	0.5	1.0	NaN
7	17	pear	0.6	1.2	NaN

Рис. 2.20. Схематическое изображение выражения `pd.concat([df, new_products])`

Теперь у нас есть объединенный датафрейм со всеми товарами: старыми и новыми. Но поскольку мы не включили в новый датафрейм столбец `sales`, после объединения он оказался для новых товаров заполнен значениями `NaN`, что видно ниже:

	product_id	name	wholesale_price	retail_price	sales
0	23	computer	500.0	1000.0	100.0
1	96	Python Workout	35.0	75.0	1000.0
2	97	Pandas Workout	35.0	75.0	500.0
3	15	banana	0.5	1.0	200.0
4	87	sandwich	3.0	5.0	300.0
5	24	phone	200.0	500.0	NaN
6	16	apple	0.5	1.0	NaN
7	17	pear	0.6	1.2	NaN

Нам нужно заполнить эти пропущенные значения. Сделать это можно разными способами. К примеру, вы можете выполнить операцию присваивания с использованием атрибута `loc`, которому можно передать список индексов в качест-

ве селектора строк и имя нужной колонки в качестве селектора столбцов, как показано ниже (пока без присваивания):

```
df.loc[[5,6,7], 'sales']
```

Вывод будет таким:

```
5    NaN
6    NaN
7    NaN
Name: sales, dtype: float64
```

Как и ожидалось, мы получили три значения NaN, присутствующие в нашем наборе данных. Обратите внимание, что тип данных для столбца изменился на float64. Причина в том, что значение NaN обладает типом float. И каждый раз, когда pandas необходимо обработать значение NaN, он принудительно устанавливает для столбца тип с плавающей точкой.

ПРИМЕЧАНИЕ. В NumPy присваивание чисел с плавающей точкой массиву с целочисленным dtype приводит к безмолвному обрезанию дробной части у чисел. А попытка присваивания значения NaN (float, но довольно странный float) массиву с целочисленным dtype завершится ошибкой, говорящей о том, что NumPy не знает целочисленного значения, соответствующего значению NaN. Pandas в этом смысле более сговорчив, но он молча, без всяких предупреждений, присвоит атрибуту dtype значение float64 для поддержки ваших NaN. Нет, вы не потеряете данные, но можете быть удивлены тому, что тип ваших данных изменился, хотя вы не давали такого распоряжения.

Как можно присвоить этим значениям NaN целые числа? Один из способов – воспользоваться атрибутом loc для установки новых значений, как показано ниже и на рис. 2.21:

```
df.loc[[5,6,7], 'sales'] = [100, 200, 75]
```

Всего в одной строке кода заложено довольно много действий. Давайте посмотрим, что здесь происходит.

1. С помощью атрибута df.loc мы можем получить доступ к одной или нескольким строкам в датафрейме. В нашем случае мы воспользовались причудливой индексацией для извлечения трех строк на основе индекса.
2. Если оставить запись так, то мы получим все колонки для выбранных строк, т. е. новый датафрейм. Но нам нужен лишь один столбец из набора, имя которого мы и передали вторым аргументом ('sales').
3. Поскольку мы затребовали только один столбец, результат нам вернулся в виде объекта Series с тремя значениями NaN.
4. В заключение мы присвоили выборке, полученной с помощью атрибута df.loc, новые значения из списка, заменив ими предыдущие значения NaN.

Обратите внимание, что атрибут dtype не был автоматически изменен на np.int64.

Селектор столбцов

	product_id	name	wholesale_price	retail_price	sales
0	23	computer	500	1000.0	100.0
1	96	Python Workout	35	75.0	1000.0
2	97	Pandas Workout	35	75.0	500.0
3	15	banana	0.5	1.5	200.0
4	87	sandwich	3.0	5.0	300.0
5	24	phone	200.0	500.0	NaN
6	16	apple	0.5	1.0	NaN
7	17	pear	0.6	1.2	NaN

Селектор строк

Присваиваем [100, 200, 75] этим трем ячейкам

Рис. 2.21. Схематическое изображение выражения `df.loc[[5,6,7], 'sales'] = [100, 200, 75]`

Если вам не по душе такие массовые операции присваивания, вы можете сделать это построчно с помощью следующего синтаксиса:

```
df.loc[5, 'sales'] = 100
df.loc[6, 'sales'] = 200
df.loc[7, 'sales'] = 75
```

В результате столбец `sales` окажется заполненным числовыми значениями для всех товаров в датафрейме. После этого мы можем выполнить финальные расчеты, как уже делали раньше:

```
(df['retail_price'] - df['wholesale_price']) * df['sales'].sum()
```

Решение

```
new_products = DataFrame([
    {'product_id':24, 'name':'phone', 'wholesale_price': 200, 'retail_price':500},
    {'product_id':16, 'name':'apple', 'wholesale_price': 0.5, 'retail_price':1},
    {'product_id':17, 'name':'pear', 'wholesale_price': 0.6, 'retail_price':1.2}
],
index=range(5,8))
df = pd.concat([df, new_products])
```

❶
❷


```
df.loc[[5,6,7], 'sales'] = [100, 200, 75]
(df['retail_price'] - df['wholesale_price']) * df['sales'].sum()
```

- ❶ Создаем датафрейм с новыми товарами.
- ❷ Объединяем вместе старые и новые товары.
- ❸ Присваиваем значения с продажами трем новым товарам.
- ❹ Рассчитываем чистый доход по всем товарам.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/9Q4L>.

Дополнительные упражнения

- Добавьте еще один товар в датафрейм без использования функции `pd.concat`. Каковы преимущества функции `pd.concat` и когда стоит ее использовать?
- Добавьте в датафрейм новый столбец `department` (отдел). Задайте отделы для всех товаров в датафрейме. К примеру, в нашем наборе данных отделы получились следующие: `electronics`, `books` и `food`. Добавьте столбец `current_net` в датафрейм и выведите описательную статистику для этого столбца по товарам из отдела `electronics`.
- Воспользуйтесь методом `query` (см. следующую врезку) для получения описательной статистики по товарам из отдела `food`.

Извлечение данных с помощью метода `query`

Как мы уже видели, традиционно строки из датафрейма выбираются с помощью булева индекса. Но есть и другой способ, заключающийся в использовании метода `query`. Этот метод может показаться вам знакомым, если вы ранее работали с SQL и реляционными базами данных.

Основная идея, лежащая в основе этого метода, абсолютно проста: мы предоставляем pandas строку, которую он преобразует в полноценный запрос. В результате применения этого метода мы получаем отфильтрованный набор данных на основе исходного датафрейма. Скажем, нам нужно получить все строки, в которых значение в столбце `v` превышает 300. С помощью традиционного индекса-маски мы бы сделали это так:

```
df[df['v'] > 300]
```

С использованием метода `query` мы можем переписать эту инструкцию следующим образом:

```
df.query('v > 300')
```

Эти два выражения вернут одинаковый результат. Разница в том, что при использовании метода `query` можно обращаться к столбцам без квадратных скобок и точечной нотации, что бывает очень удобно.

А что, если необходимо написать более сложный запрос? К примеру, нам нужно получить строки, в которых значение в столбце `v` превышает 300, а в столбце `w` находится нечетное число. Это можно сделать так:

```
df.query('v > 300 & w % 2 == 1') ❶
```

❶ В запросах оператор `&` используется в качестве логического И.

В данном случае это не обязательно, но я предпочитаю все время использовать круглые скобки для явного указания последовательности выполнения операций, как показано ниже:

```
df.query('(v > 300) & (w % 2 == 1)')
```

Обратите внимание, что метод `query` не может стоять в левой части операции присваивания.

Применительно к небольшим наборам данных использование метода `query` может оказаться неэффективным. Но при работе с датафреймами объемом от 10 000 строк этот способ доступа к данным может быть оптимальным. Кроме того, он задействует гораздо меньше памяти. Более подробно о методе `query` мы поговорим в главе 12.

Ответы на дополнительные упражнения

Упражнение 10.1

```
# Если вам нужно добавить всего одну строку в датафрейм, вы можете просто
# присвоить новое значение атрибуту df.loc[индекс]. Если такой индекс
# отсутствует, будет добавлена новая строка в датафрейм. Вы также можете
# воспользоваться атрибутом df.iloc[индекс], указав следующий номер индекса
# за максимальным
```

```
# Функцию pd.concat уместно использовать при необходимости объединить два
# набора данных в один датафрейм
df.loc[8] = [99, 'persimmon', 2, 4.5, 1]
```

Упражнение 10.2

```
df['department'] = ['electronics', 'books', 'books', 'food', 'food',
                   'electronics', 'food', 'food', 'food']
df
```

Вывод:

	product_id	name	wholesale_price	retail_price	sales	department
0	23	computer	500.0	1000.0	100.0	electronics
1	96	Python Workout	35.0	75.0	1000.0	books
2	97	Pandas Workout	35.0	75.0	500.0	books
3	15	banana	0.5	1.0	200.0	food
4	87	sandwich	3.0	5.0	300.0	food
5	24	phone	200.0	500.0	100.0	electronics
6	16	apple	0.5	1.0	200.0	food
7	17	pear	0.6	1.2	75.0	food
8	99	persimmon	2.0	4.5	1.0	food

```
df['current_net'] = (df['retail_price'] - df['wholesale_price']) * df['sales'].sum()
```

```
# Воспользуемся индексом-маской для столбца current_net
df['current_net'][df['department'] == 'electronics'].describe()
```

Вывод:

```
count      2.000000e+00
mean       9.904000e+05
std        3.501593e+05
min        7.428000e+05
25%        8.666000e+05
50%        9.904000e+05
75%        1.114200e+06
max        1.238000e+06
Name: current_net, dtype: float64
```

Упражнение 10.3

```
df.query('department == "food"')['current_net'].describe()
```

Вывод:

```
count      5.000000
mean     3020.720000
std      2371.020496
min      1238.000000
25%      1238.000000
50%      1485.600000
75%      4952.000000
max       6190.000000
Name: current_net, dtype: float64
```

УПРАЖНЕНИЕ 11. Лидеры продаж

Мы снова будем работать с нашим набором данных по магазину. На этот раз вам необходимо будет найти идентификаторы и наименования товаров, которых в сумме было продано больше, чем среднее значение по продажам (столбец sales).

Подробный разбор

Pandas – это про анализ данных. А по большей части вопросы в аналитике формулируются так: «Если верно *это*, покажи мне *то*». И вариантов здесь масса:

- выбери акции в моем портфеле, которые за этот год показали себя плохо;
- покажи мне сотрудников команды, которые исправили больше всех багов;
- покажи мне три самые забивающие команды в лиге...

В этом упражнении я попрошу вас показать мне идентификаторы и наименования товаров, объем продаж по которым (столбец sales) превышает средний объем по всем товарам. Как обычно, в pandas одну и ту же задачу можно решить множеством способов. Но я обычно отдаю предпочтение методам, связанным с

использованием атрибута доступа `loc` (см. врезку «Извлечение и присваивание с помощью атрибута `loc`» ранее в этой главе).

При работе с атрибутом `loc` мы по определению начинаем со строк. В данном случае нас интересуют строки, в которых значение в столбце `sales` превышает среднее значение по этому столбцу. Мы можем создать объект `Series` с булевыми значениями с помощью следующего запроса:

```
df['sales'] > df['sales'].mean()
```

После этого созданный объект можно применить в качестве маски к исходному датафрейму, чтобы получить товары с хорошими продажами:

```
df.loc[df['sales'] > df['sales'].mean()] ❶
```

❶ Используем объект `Series` с булевыми значениями в качестве селектора строк.

Но нам не нужны все столбцы из набора данных. Нам достаточно столбцов `product_id` и `name`. Перечислим их во втором аргументе атрибута `loc`, как показано ниже:

```
df.loc[
    df['sales'] > df['sales'].mean(), ❶
    ['product_id', 'name']           ❷
]
```

❶ Используем объект `Series` с булевыми значениями в качестве селектора строк.

❷ Используем список имен столбцов в качестве селектора столбцов.

В результате мы получим именно то, что нужно, как видно на рис. 2.22.

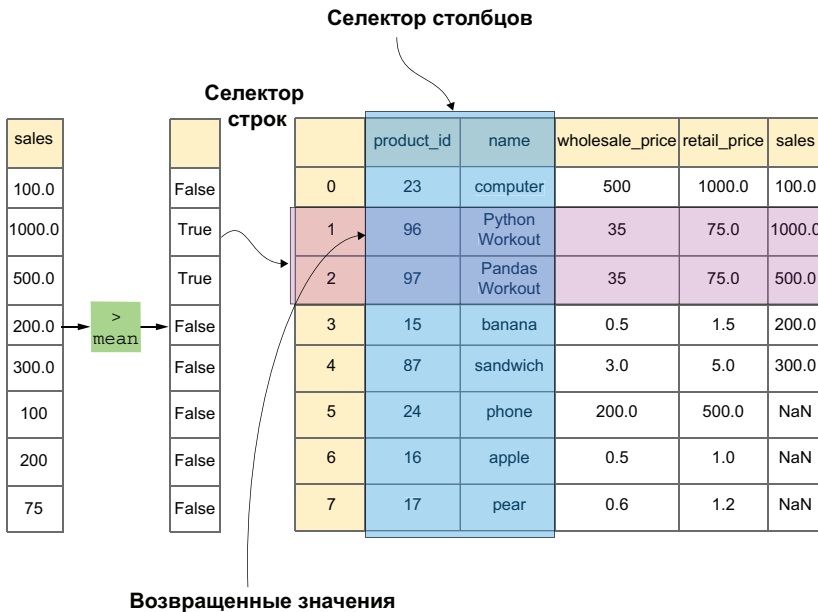


Рис. 2.22. Схематическое изображение выражения

```
df.loc[df['sales'] > df['sales'].mean(), ['product_id', 'name']]
```

Также можно решить эту задачу при помощи метода `query`. Нужные строки мы получим так:

```
df.query('sales > sales.mean()')
```

Для ограничения вывода требуемыми столбцами необходимо применить квадратные скобки к результату метода `df.query`:

```
df.query('sales > sales.mean()')[['product_id', 'name']]
```

Решение

```
df.loc[
    df['sales'] > df['sales'].mean(),
    ['product_id', 'name']
]
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/j1zx>.

Дополнительные упражнения

При выполнении этих упражнений попробуйте использовать оба способа: с помощью атрибута `loc` и метода `query`.

1. Покажите идентификаторы и наименования товаров, чистый доход по которым входит в первый квартиль.
2. Покажите идентификаторы и наименования товаров с объемами продаж ниже среднего и оптовой ценой выше среднего.
3. Покажите наименования, а также оптовую и розничную цены товаров с идентификаторами, входящими в диапазон от 80 до 100, и объемом продаж менее 400 единиц.

Ответы на дополнительные упражнения

Упражнение 11.1

```
df['net'] = df['retail_price'] - df['wholesale_price']
df.loc[
    df['net'] > df['net'].quantile(0.75),
    ['product_id', 'name']
]
```

Вывод:

	product_id	name
0	23	computer

С использованием метода `query`

```
df.query('net > net.quantile(0.75)')[['product_id', 'name']]
```

Вывод:

	product_id	name
0	23	computer

Упражнение 11.2

```
df.loc[
    (df['sales'] < df['sales'].mean()) &
    (df['wholesale_price'] > df['wholesale_price'].mean()),
    ['product_id', 'name']]
```

Вывод:

	product_id	name
0	23	computer

```
# В данном случае выражение с использованием метода query выглядит более
# читаемым
df.query('sales < sales.mean() & wholesale_price > wholesale_price.mean()')
[['product_id', 'name']]
```

Вывод:

	product_id	name
0	23	computer

Упражнение 11.3

```
df.loc[
    (df['product_id'] > 80) & (df['product_id'] < 100) & (df['sales'] < 400),
    ['name', 'wholesale_price', 'retail_price']]
```

Вывод:

	name	wholesale_price	retail_price
4	sandwich	3.0	5

```
# С использованием метода query
df.query('product_id > 80 & product_id < 100 & sales < 400')[['name',
    'wholesale_price', 'retail_price']]
```

Вывод:

	name	wholesale_price	retail_price
4	sandwich	3.0	5

УПРАЖНЕНИЕ 12. Поиск выбросов

Мы уже говорили о том, как среднее значение, стандартное отклонение и медиана могут помочь в понимании характера данных. Эти метрики описывают большую часть данных, пытаясь выявить тенденции расположения большинства

значений в наборе. Но иногда бывает полезно взглянуть на частицы данных, не входящие в основную группу. Примеры таких запросов:

- у каких пользователей было зафиксировано необычно высокое количество неудачных входов в систему?
- какие товары обладают наибольшим спросом?
- в какие дни и часы продажи в нашем магазине наименьшие?

Подобные вопросы отнюдь не редки в процессе анализа данных. К примеру, во многих заведениях вводятся так называемые счастливые часы с большими скидками, и выпадают они ровно на то время, когда по статистике в заведении бывает меньше всего посетителей. Наука о данных позволяет задавать такие вопросы, получать на них точные ответы, а затем проверять, привели ли наши действия к желаемому результату.

ПРИМЕЧАНИЕ. Термин *выброс* (outlier) не имеет четкого определения. Многие определяют выбросы с использованием *межквартильного размаха* (interquartile range – IQR), вычисляемого как разница между 75-м процентилем (`quantile(0.75)`) и 25-м (`quantile(0.25)`). В этом случае выбросами считаются точки данных, располагающиеся ниже, чем 25-й процентиль, из которого вычли полтора межквартильного размаха ($25\% - 1.5 * IQR$), и выше, чем 75-й процентиль, к которому прибавили полтора межквартильного размаха ($75\% + 1.5 * IQR$). В этой книге мы будем использовать это распространенное определение, но вы можете встретить и другие формальные описания статистических выбросов, которые могут лучше подойти вашим данным. К примеру, выбросами могут считаться точки, лежащие ниже и выше среднего значения более чем на два стандартных отклонения.

В этом упражнении мы создадим датафрейм, состоящий из двух столбцов, на основе набора данных по поездкам на такси, с которым встречались в упражнении 6. В первом столбце будет находиться количество пассажиров в поездке, а во втором – продолжительность поездки в милях. После создания датафрейма вам нужно будет ответить на два следующих вопроса:

- сколько поездок можно отнести к выбросам в отношении их продолжительности?
- какое среднее количество пассажиров приходится на поездки, отнесенные к категории выбросов? Отличается ли оно от среднего количества пассажиров для всех поездок в наборе?

Подробный разбор

Перед нами стоят четыре следующие задачи.

1. Создать датафрейм на основе отдельных объектов Series.
2. Рассчитать межквартильный размах (IQR).
3. Найти выбросы.
4. Использовать найденные выбросы для ответа на поставленные вопросы.

Для начала нам необходимо собрать общий датафрейм из двух отдельных объектов Series. Мы уже видели, как можно создать эти объекты Series на основе разных файлов. Давайте сохраним их в разные переменные:

```
trip_distance = pd.read_csv('../data/taxi-distance.csv', header=None).squeeze()
passenger_count = pd.read_csv('../data/taxi-passenger-count.csv',
                               header=None).squeeze()
```

Как можно собрать эти разрозненные столбцы в один датафрейм? Простейшим способом является создание датафрейма на основе словаря, в котором ключи будут относиться к именам будущих столбцов, а значения – к их содержимому (объекты Series), что схематично показано на рис. 2.23.

	trip_distance	passenger_count
0	1.63	1
1	0.46	1
2	0.87	1
3	2.13	1
4	1.40	1
5	1.40	1
6	1.80	1
7	11.90	4

Рис. 2.23. Схематическое изображение создания датафрейма на основе словаря

Код создания датафрейма:

```
df = DataFrame({'trip_distance': trip_distance,
                'passenger_count': passenger_count})
```

Теперь нам необходимо рассчитать межквартильный размах (IQR), который поможет обнаружить выбросы в данных. Помните, что IQR вычисляется как разница между 75-м процентилем (`quantile(0.75)`) и 25-м (`quantile(0.25)`). Это означает, что, если бы мы отсортировали наши значения по возрастанию, мы должны были бы найти значения, лежащие на правой границе первой и третьей четвертей.

Эти значения можно вычислить с помощью метода `quantile`, на вход которому передается нужное значение процентиля (в нашем случае 0.25 или 0.75). Но не делайте ошибку – не вызывайте этот метод применительно ко всему датафрейму. В этом случае вы получите процентиля для всех столбцов, а нас интересует только столбец `trip_distance`. Таким образом, мы можем вычислить IQR следующим образом:

```
iqr = (
    df['trip_distance'].quantile(0.75) -
    df['trip_distance'].quantile(0.25)
)
```

Разумеется, мы не обязаны были сохранять значение межквартильного размаха в отдельной переменной, но так нам будет удобнее проводить дальнейшие вычисления. Теперь, когда у нас есть переменная `iqr`, мы можем легко найти наши

выбросы. Начнем с нижних выбросов, т. е. значений, располагающихся ниже, чем 25-й процентиль, из которого вычли полтора межквартильного размаха ($25\% - 1.5 * IQR$). Вот как можно формализовать это выражение в pandas:

```
df[df['trip_distance'] < df['trip_distance'].quantile(0.25) - 1.5*iqr]
```

И какой результат? Выбросов в нижней части распределения у нас нет. Вероятно, дело в том, что слишком много поездок на такси являются довольно непродолжительными, а нижняя часть диапазона у нас ограничена нулем.

Но в верхней части диапазона выбросы у нас обнаружили:

```
df[df['trip_distance'] > df['trip_distance'].quantile(0.75) + 1.5*iqr]
```

Из 10000 поездок сразу 1889 поездок были признаны выбросами! Это означает, что почти 19 % всех поездок на такси значительно превосходят среднюю продолжительность.

Обратите внимание, что мы выполнили эти вычисления путем создания объектов Series с булевыми значениями и применения их к датафрейму в качестве индекса. Но мы не обязаны применять их ко всему датафрейму, достаточно применить к нужной нам колонке. К примеру, если мы применим индекс к столбцу `passenger_count`, то получим информацию о том, сколько пассажиров перевозили водители такси на сверхдальние расстояния:

```
df['passenger_count'][df['trip_distance'] >
    df['trip_distance'].quantile(0.75) + 1.5*iqr]
```

А как получить среднее из этих значений? На выходе мы получили объект Series, так что нам старый добрый метод `mean` вполне подойдет:

```
df['passenger_count'][df['trip_distance'] > df['trip_distance'].quantile(
    0.75) + 1.5*iqr].mean()
```

Мы получили значение 1.70, что очень близко к среднему значению по столбцу `passenger_count`.

Решение

```
trip_distance = pd.read_csv('../data/taxi-distance.csv',
    header=None).squeeze()
passenger_count = pd.read_csv('../data/taxi-passenger-count.csv',
    header=None).squeeze()
```

```
df = DataFrame({'trip_distance': trip_distance,
    'passenger_count': passenger_count}) ❶
```

```
iqr = (df['trip_distance'].quantile(0.75)
    - df['trip_distance'].quantile(0.25))
```

```
df[df['trip_distance']
    < df['trip_distance'].quantile(0.25) - 1.5*iqr] ❶
df[df['trip_distance']
    > df['trip_distance'].quantile(0.75) + 1.5*iqr] ❷
```

```
df['passenger_count'][df['trip_distance']
    > df['trip_distance'].quantile(0.75) + 1.5*iqr].mean() ❸
```

- ❶ В нижней части набора выбросов нет.
- ❷ В верхней части набора есть 1889 выбросов.
- ❸ Среднее количество пассажиров в поездках, являющихся выбросами.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/W1R0>.

Дополнительные упражнения

Как мы уже говорили ранее, существуют разные способы для определения выбросов в данных. Давайте опробуем другие техники.

1. Если определить выбросы как нижние и верхние 10 % упорядоченных значений, то сколько их будет в нашем наборе? Чем этот способ определения выбросов хорош/плох?
2. Избавьтесь от выбросов в столбце `trip_distance`, принимая за выбросы 10 % значений с каждой стороны. Для удаления выбросов из данных можно воспользоваться удобной функцией `scipy.stats.trimboth` применительно к объекту `Series`. Вторым аргументом она принимает долю значений, которые нужно отсечь с обоих концов массива.
3. Функция `scipy.stats.zscore` позволяет изменить масштаб и центрировать (т. е. нормализовать) набор данных. После ее применения среднее значение в наборе устанавливается в ноль, и все значения располагаются выше или ниже этой отметки. Найдите все длительности поездок, для которых *z-оценка* (*z-score*) превышает 3.

Значение NaN и отсутствующие данные

Пока что все, что мы делали в `pandas`, не было сопряжено с большими сложностями. Достаточно было правильно сформулировать вопрос, и ответ на него тут же находился. Это могло создать у вас ложное ощущение того, что работа аналитика – это сплошной мед с сахаром.

Но не спешите радоваться. Для большинства данных характерен один общий недостаток, заключающийся в их неполноте. Компьютер, отвечающий за сбор важных сведений на прошлой неделе, мог выйти из строя. Или какие-то датчики могли дать сбой. Ну или при опросе населения многие из них просто отказались дать ответ.

Какая бы ни была причина, аналитику придется – и всегда приходится – мириться и работать с неполными данными. Зачастую от них можно услышать, что 70–80 % их работы состоит в очистке, скалировании и прочих манипуляциях с исходными данными в попытках привести их в сколько-нибудь божеский вид. Иногда хочется все бросить и просто игнорировать все пропущенные значения в данных. Но так делать нельзя. Если мы будем исключать из набора записи с пропущенными значениями, мы в результате можем остаться без данных вовсе. И что тогда анализировать?

Как принято представлять пропущенные значения в pandas? Если подставлять вместо них нули, это сильно скажется на описательной статистике, в частности на средних значениях. Так что в этой библиотеке для отсутствующих значений было введено особое значение, именуемое NaN (сокращенно от *Not a Number* – не является числом). При этом можно использовать обе записи: `np.nan` и `np.NaN`, но в pandas принято отдавать предпочтение последней.³ Вне зависимости от способа записи, все пропущенные значения будут представлены как `np.nan`. Это необычное значение представлено типом числа с плавающей точкой, не может быть преобразовано в целочисленный тип и не равно само себе.

Стоит отметить, что на момент написания этой книги разработчики библиотеки pandas подумывали о замене значения NaN на их собственное – `pd.NA` – в рамках обширной миграции на новые типы данных pandas, которые должны обладать большей гибкостью в сравнении с традиционными типами NumPy. Но в этой книге мы будем использовать обычные значения NaN.

В NumPy мы обычно ищем значения NaN с помощью функции `isnan`. В pandas принято использовать несколько иной подход. Мы можем заменить пропущенные значения в объекте Series или в датафрейме с помощью метода `fillna`, а отбросить строки с пропущенными значениями можно при помощи метода `dropna`.

Эти методы возвращают новый объект Series или датафрейм, а не модифицируют исходный. При этом в новом объекте может не оказаться скопированных данных, что впоследствии может приводить к появлению злосчастного предупреждения `SettingWithCopyWarning`. Если вы планируете модифицировать Series или датафрейм, полученный в результате вызова метода `df.dropna`, вам может понадобиться дополнительный метод `copy`, чтобы обезопасить следующие операции:

```
df = df.dropna().copy()
```

Это позволит в дальнейшем изменять объект `df` без опасений получить показанное выше предупреждение.

Вполне очевидно, что удаление всех строк, в которых содержится хотя бы одно значение NaN, может быть нежелательным. По этой причине в методе `dropna` предусмотрен параметр `thresh`, принимающий целое число. С помощью этого параметра можно задать минимальное количество непропущенных значений в строке, достаточное для ее сохранения в наборе. Таким образом, у вас есть способ контролировать прореживание неполных данных.

Подробнее операции, связанные с очисткой данных, мы будем рассматривать в главе 5. Сейчас же вам достаточно будет знать о необходимости проверки данных на наличие пропущенных значений и принятии определенных мер. Иногда от пропущенных значений нужно избавляться путем удаления строк, а иногда, как в упражнении 13, более приемлемым вариантом может быть заполнение таких значений на основе соседствующих с ними элементов.

ПРИМЕЧАНИЕ. Метод `count`, примененный к объекту Series, возвращает количество непропущенных значений. Если в объекте нет значений NaN, будет возвращено число, соответствующее общему количеству элементов в объекте. В то же время метод `count`, примененный к датафрейму, возвращает новый объект Series с именами столбцов в качестве индексов. Если количество строк в каких-то столбцах будет меньше, чем в остальных, это значит, что в них есть пропущенные значения.

³ В NumPy версии 2.0 запись `np.NaN` стала невозможна. – Прим. перев.

Ответы на дополнительные упражнения

Упражнение 12.1

```
df[(df['trip_distance'] < df['trip_distance'].quantile(0.1)) |
    (df['trip_distance'] > df['trip_distance'].quantile(0.9)) ]
```

Вывод:

	trip_distance	passenger_count
1	0.46	1
7	11.90	4
9	0.60	1
10	0.01	3
13	0.50	2
...	...	...
9976	12.60	1
9978	0.38	1
9979	11.30	1
9980	9.13	1
9982	9.30	1

[1984 rows x 2 columns]

Преимущество этой меры – в ее простоте. Ни для кого не составит труда понять, что здесь имеется в виду. Недостаток же его состоит в том, что при наличии большого количества коротких поездок (как в нашем случае) нам придется все их называть выбросами, даже если они очень близки к значениям, не вошедшим в категорию выбросов.

Упражнение 12.2

```
from scipy.stats import trimboth
trimboth(df['trip_distance'], 0.1)
```

Вывод:

```
array([0.63, 0.63, 0.63, ..., 8.2 , 8.2 , 8.2 ])
```

Упражнение 12.3

```
from scipy.stats import zscore
df['trip_distance'][abs(zscore(df['trip_distance'])) > 3]
```

Вывод:

88	23.76
238	18.32
379	16.38
509	16.82
641	19.72

```

...
9897    16.11
9899    17.48
9906    17.70
9955    15.49
9964    18.55
Name: trip_distance, Length: 306, dtype: float64

```

УПРАЖНЕНИЕ 13. Интерполяция

При содержании в данных пропущенных значений мы можем применить экстремально строгий подход и избавиться от всех строк, в которых есть хотя бы одно такое значение. Но это может приводить к удалению слишком большого количества строк с полезной для анализа информацией. Альтернативный подход состоит в применении операции *интерполяции* (interpolation), при котором пропущенные значения заменяются на наиболее правдоподобные с применением определенных критериев. Да, мы можем не попасть с этими значениями точно в цель, но приблизительные значения иногда восстановить очень даже можно.

В этом упражнении мы будем работать с набором данных о температурах в Нью-Йорке, зафиксированных в конце 2018 года и начале 2019-го. При этом мы намеренно испортим данные, полученные в 3 и 6 ч утра, тем самым симитировав собой сенсоров. Узнаем, поможет ли нам здесь интерполяция и насколько далеко от реальных показателей (которые мы припрятали) окажутся интерполированные значения в отношении среднего и медианы.

Выполните следующие действия.

1. Загрузите данные о температуре в Нью-Йорке из файла `nyc-temps.txt` в объект Series. Измерения приведены в градусах Цельсия.
2. Создайте датафрейм с двумя столбцами: `temp` со значением температуры и `hour` со значением часа, в который было произведено измерение. Измерения мы проводили через каждые три часа, а значит, в столбце `hour` должны циклически располагаться значения 0, 3, 6, 9, 12, 15, 18 и 21, повторяясь для всех 728 измерений.
3. Рассчитайте среднее значение температуры и медиану. Это истинные значения, которые мы хотим воссоздать при помощи интерполяции.
4. Замените значения температуры для измерений в 3 и 6 ч на NaN.
5. Восстановите потерянные значения при помощи метода `interpolate`.
6. Снова рассчитайте среднее значение температуры и медиану. Насколько близкими оказались эти значения к реальным, которые мы получили в пункте 3, и почему?

Подробный разбор

Первое, что нам необходимо сделать, – это загрузить данные в объект Series. Мы уже делали это раньше, но лишний раз повторить изученное будет полезно:

```
s = pd.read_csv('../data/nyc-temps.txt').squeeze()
```

Итак, мы прочитали данные из файла `nyc-temps.txt` и вернули их в виде объекта `Series` с помощью метода `squeeze`. Теперь мы можем использовать все доступные методы для работы с `Series` применительно к этим данным.

Во второй колонке с именем `hour` в нашем датафрейме должны присутствовать числа 0, 3, 6, 9, 12, 15, 18 и 21 с циклическими повторениями. Поскольку в наших данных содержится 728 строк, а в сутки мы производили восемь измерений, мы можем воспользоваться встроенным функционалом Python и умножить список из восьми элементов на 91, в результате чего получим расширенный список из 728 циклически повторяющихся элементов.

После создания датафрейма мы должны удалить значения температур, соответствующие измерениям в 3 и 6 ч утра. Для этого мы можем воспользоваться атрибутом `loc`, выбрав столбец `temp`, и заменить содержимое этих ячеек на значение `NaN` с помощью присваивания:

```
df.loc[
    df['hour'].isin([3,6]), ❶
    'temp'                  ❷
] = np.nan
```

❶ Селектор строк для 3 и 6 ч.

❷ Селектор столбцов.

Этот запрос можно условно разделить на несколько частей:

- поиск строк в датафрейме, в которых значение в столбце `hour` равно 3 или 6, с помощью метода `isin`. В результате мы получили объект `Series` с булевыми значениями;
- вторым параметром мы передаем в атрибут `loc` имя колонки `temp`;
- к полученной выборке мы применяем операцию присваивания, заменяя значения в ней на `NaN` (`np.nan`).

Наконец, мы вызываем метод `df.interpolate`, возвращающий новый датафрейм, как показано на рис. 2.24. В теории все столбцы в датафрейме будут интерполированы, но на деле эта операция применяется только к пропущенным значениям, которые у нас присутствуют в столбце `temp`. Новый датафрейм мы присвоим той же переменной `df`.

По умолчанию метод `interpolate` заполняет все значения `NaN` усредненными значениями из соседних ячеек, так что если в строке с индексом 4 у нас стоит значение -1 , а в строке с индексом 6 – значение 3, то пропущенное значение в строке с индексом 5 будет заменено на значение $(-1 + 3) / 2 = 1$, что мы и видим на рис. 2.24. Если несколько пропущенных значений в столбце следуют друг за другом, они будут заменены таким образом, чтобы все новые значения были равномерно распределены между обрамляющими эту последовательность из `NaN` непустыми значениями (см. заполнение ячеек в строках с индексами 1 и 2 на рис. 2.24).

ПРИМЕЧАНИЕ. С помощью параметра `method` можно задать для метода `interpolate` альтернативный способ интерполяции. К примеру, если передать `method='nearest'`, значения `NaN` будут заменены на ближайшие к ним непустые значения. Другие варианты интерпо-

ляции можно найти в документации по адресу <http://mng.bz/MBo7> (<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.interpolate.html>).

Поскольку температура воздуха обычно не сильно меняется от часа к часу, а изменяется плавно, мы воспользовались линейным способом интерполяции, принятым по умолчанию. В то же время температуру в духовке вряд ли будет уместно интерполировать этим способом. Таким образом, перед использованием метода `interpolate` стоит задуматься о том, как именно интерполировать значения.

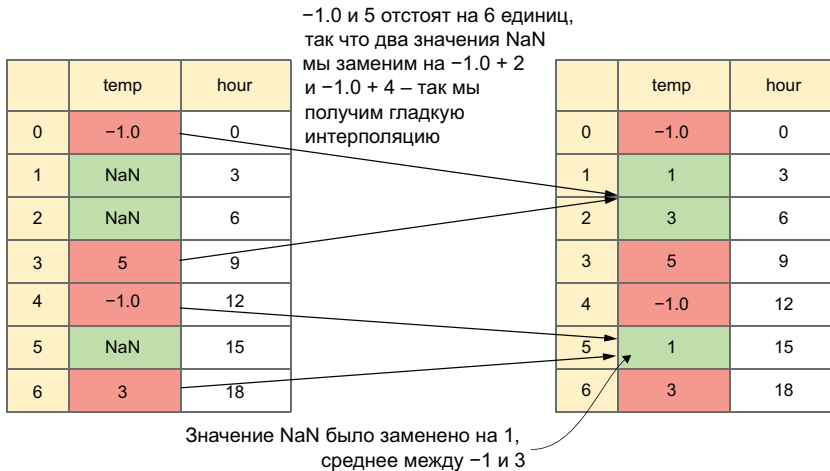


Рис. 2.24. Схематическое изображение результатов применения метода `interpolate`

Решение

```
s = pd.read_csv('../data/nyc-temps.txt').squeeze() ❶
df = DataFrame(
    {'temp': s,
     'hour': [0,3,6,9,12,15,18,21] * 91}) ❷

df.loc[
    df['hour'].isin([3,6]),
    'temp'
] = NaN ❸

df = df.interpolate() ❹

df['temp'].describe() ❺
```

- ❶ Читаем с диска значения и сохраняем их в виде `Series`.
- ❷ Создаем датафрейм из полученного объекта `Series` и списка с часами измерений.
- ❸ Сбрасываем в `NaN` все измерения для 3 и 6 ч утра.
- ❹ Применяем метод `df.interpolate` и присваиваем результат обратно переменной `df`.
- ❺ Извлекаем показатели описательной статистики для получения данных о среднем значении и медиане.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/84vP>.

Дополнительные упражнения

1. Как изменится поведение метода `interpolate` в нашем примере при передаче ему параметра `method='nearest'`?
2. Предположим, что с датчиками, считывающими температуру по часам, все в порядке, но при хранении возникла ошибка и все данные об отрицательных температурах потерялись. Будут ли в этом случае значения, полученные с использованием интерполяции, близки к реальным значениям и почему?
3. Наиболее простым способом интерполяции является заполнение пропущенных значений средними значениями по столбцу. Выполните такой вид интерполяции (с отсутствующими данными о минусовых температурах) и сравните среднее значение и медиану. Поясните, почему был получен такой результат?

Ответы на дополнительные упражнения

Упражнение 13.1

Похоже, ничего особо не изменилось – возможно, потому что температура не
сильно изменяется с течением времени

```
df.interpolate(method='nearest').describe()
```

Вывод:

	temp	hour
count	728.000000	728.000000
mean	-1.050824	10.500000
std	5.026357	6.878589
min	-14.000000	0.000000
25%	-4.000000	5.250000
50%	0.000000	10.500000
75%	2.000000	15.750000
max	12.000000	21.000000

Упражнение 13.2

```
# Восстанавливаем данные
df = DataFrame({'temp': s,
                'hour': [0,3,6,9,12,15,18,21] * 91})
```

```
# Меняем отрицательные значения на NaN
df.loc[df['temp'] <= -1, 'temp'] = np.nan
```

```
# Интерполируем
df = df.interpolate()
```

```
# Ух ты, среднее значение теперь равно 2, а медиана – 1, что значительно выше
# И это не удивительно, поскольку мы удалили низкие температуры
df['temp'].describe()
```


Вывод:

```
count    721.000000
mean      2.022191
std       2.345483
min       0.000000
25%      0.209524
50%      1.000000
75%      3.000000
max      12.000000
Name: temp, dtype: float64
```

Упражнение 13.3

```
# Восстанавливаем данные
df = DataFrame({'temp': s,
               'hour': [0,3,6,9,12,15,18,21] * 91})

# Меняем отрицательные значения на NaN
df.loc[df['temp'] <= -1, 'temp'] = np.nan

df = df.fillna(df.mean())

# Результат оказался еще хуже, чем после интерполяции!
# Очевидно, лучше использовать метод interpolate, чем mean - в частности,
# потому что в этом случае производится расчет локального среднего,
# а не глобального по всем данным в наборе

df.describe()
```

Вывод:

	temp	hour
count	728.000000	728.000000
mean	2.763926	10.500000
std	1.935689	6.878589
min	0.000000	0.000000
25%	2.000000	5.250000
50%	2.763926	10.500000
75%	2.763926	15.750000
max	12.000000	21.000000

УПРАЖНЕНИЕ 14. Выборочное обновление

В этом упражнении мы будем работать с тем же датафреймом с двумя столбцами, что и в предыдущем. Задача будет заключаться в том, чтобы избавиться от отрицательных значений температур, присвоив им нулевые значения.

Подробный разбор

Если вы немного работали с pandas, то могли бы предложить следующий алгоритм действий.

1. Получаем булев индекс на основе условия `df['temp'] < 0`.
2. Применяем полученный индекс-маску к датафрейму.
3. Извлекаем данные из столбца с именем `temp`.
4. Присваиваем выборке новое значение.
5. Код мог бы получиться таким:

```
df[df['temp'] < 0]['temp'] = 0
```

Вроде все логично. Но здесь есть одна проблема, состоящая в том, что вы не можете знать заранее, сработает ли этот подход. При построении запросов `pandas` выполняет тщательный анализ и оптимизацию исходных компонентов. Это приводит к тому, что вы никогда не можете знать заранее, приведет ли ваша операция присваивания к изменению значений в столбце `temp` датафрейма `df`, или – и это очень важно! – `pandas` решит кешировать результаты вашего первого запроса и применит инструкцию `['temp']` к этому кешированному внутреннему значению, а не к исходному.

В результате вы можете получить одно из самых раздражающих предупреждений от `pandas` – `SettingWithCopyWarning`. Выглядеть оно может так:

```
<ipython-input-2-acedf13a3438>:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

(Обнаружена попытка установки значения для копии среза из датафрейма. Попробуйте воспользоваться конструкцией `.loc[row_indexer,col_indexer] = value`)

Этим предупреждением `pandas` пытается сказать вам о том, что ваша операция присваивания может завершиться неудачей. Здесь важно понимать, что она может выполняться, а может и нет. Все зависит от того, с каким объемом данных вы работаете и как именно `pandas` решит оптимизировать ваш запрос.

Предвестником этого назойливого предупреждения может служить использование в выражении двойных квадратных скобок – не вложенных одни в другие, а идущих подряд, как в нашем примере. Как только вы видите сочетание `][` в запросах `pandas`, вы должны насторожиться, поскольку следующая за ним операция присваивания может завершиться неудачей. Кроме того, извлечение данных с помощью такого синтаксиса может оказаться менее эффективным в сравнении с использованием атрибута доступа `loc` с селекторами строк и столбцов. Итак, как нам следовало бы поступить в данной ситуации?

1. Начните выражение с обращения к атрибуту `df.loc`.
2. Первым аргументом в квадратных скобках (селектор строк) укажите наш индекс-маску `df['temp'] < 0`.
3. Вторым аргументом (селектор столбцов) передайте имя требуемого столбца в тех же квадратных скобках: `'temp'`.
4. Выполните операцию присваивания.

Вот такое выражение должно получиться в итоге:

```
df.loc[
    df['temp'] < 0,      ❶
    'temp'               ❷
] = 0
```

❶ Селектор строк – булев индекс.

❷ Селектор столбцов – имя колонки.

Если вы будете всегда использовать подобный синтаксис при выполнении присваивания, вы никогда не будете сталкиваться со злосчастным предупреждением `SettingWithCopyWarning`. Можете применять такие конструкции как для извлечения данных, так и для присваивания. В качестве бонуса вы можете быть уверены, что ваши инструкции выполняются оптимально.

Решение

```
df.loc[df['temp'] < 0, 'temp'] = 0
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/E9zJ>.

Дополнительные упражнения

1. Замените все нечетные значения температуры на среднее значение по всему набору данных.
2. Замените все четные значения температуры для 9- и 18-часовых измерений на число 3.
3. Установите температуру, равную 5 °C, для всех измерений, произведенных в нечетные часы.

Ответы на дополнительные упражнения

Упражнение 14.1

```
df = DataFrame({'temp': s*1.0,
                'hour': [0,3,6,9,12,15,18,21] * 91})
```

```
df.loc[
    df['temp']%2 == 1,
    'temp'
] = df['temp'].mean()
```

```
df
```

Вывод:

	temp	hour
0	-1.050824	0
1	-1.050824	3
2	-1.050824	6
3	-1.050824	9
4	-1.050824	12

```

..      ...      ...
723  2.000000      9
724  2.000000     12
725  2.000000     15
726  2.000000     18
727  2.000000     21

```

[728 rows x 2 columns]

Упражнение 14.2

```

df = DataFrame({'temp': s,
                'hour': [0,3,6,9,12,15,18,21] * 91})
df.loc[
    df['hour'].isin([9, 18]),
    'temp'] = 3

```

df

Вывод:

	temp	hour
0	-1	0
1	-1	3
2	-1	6
3	3	9
4	-1	12
..	...	...
723	3	9
724	2	12
725	2	15
726	3	18
727	2	21

[728 rows x 2 columns]

Упражнение 14.3

```

df = DataFrame({'temp': s,
                'hour': [0,3,6,9,12,15,18,21] * 91})
df.loc[
    df['hour']%2 == 1,
    'temp'
] = 5

```

df

Вывод:

	temp	hour
0	-1	0
1	5	3
2	-1	6

```
3      5      9
4     -1     12
...    ...    ...
723     5      9
724     2     12
725     5     15
726     2     18
727     5     21
```

```
[728 rows x 2 columns]
```

Заключение

В этой главе мы начали работать с датафреймами. Мы научились создавать их, добавлять в них столбцы и строки, извлекать из них данные, анализировать их и даже расправляться с пропущенными значениями. Изложенные в первых двух главах темы и представленные техники являются строительными блоками, на которых держится вся обработка данных в `pandas`. В следующей главе мы углубимся в более реалистичные сценарии из обычной жизни.

Глава 3

Импорт и экспорт

До сих пор мы в основном создавали датафреймы вручную или с использованием случайным образом сгенерированных значений. В реальном мире данные в датафреймах в основном берутся из файловых и иных источников – зачастую из CSV, Excel или реляционных баз данных. Кроме того, в результате проведенного анализа нам часто требуется сохранить свои изыскания, для чего мы также используем текстовый и иные форматы данных.

В этой главе мы научимся импортировать данные в pandas из различных форматов, но упор сделаем на файлы CSV, поскольку они пользуются наибольшей популярностью. Мы рассмотрим разные методики при загрузке данных, которые помогут улучшить качество загружаемых данных и оптимизировать процесс в целом.

В табл. 3.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 3.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.read_csv</code>	Возвращает новый датафрейм на основе данных в файле CSV	<code>df = pd.read_csv('myfile.csv')</code>	http://mng.bz/wvl7 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
<code>df.to_csv</code>	Позволяет записать данные из датафрейма в файл или строку в формате CSV	<code>df.to_csv('myfile.csv')</code>	http://mng.bz/7Dzx (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html)
<code>pd.read_json</code>	Возвращает новый датафрейм на основе данных в формате JSON	<code>df = pd.read_json('myfile.json')</code>	http://mng.bz/mV4n (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_json.html)
<code>df.corr</code>	Позволяет увидеть корреляцию между столбцами в датафрейме	<code>df.corr()</code>	http://mng.bz/6DQG (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.corr.html)

Таблица 3.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>df.dropna</code>	Возвращает новый датафрейм с удаленными пропущенными значениями	<code>df.dropna()</code>	http://mng.bz/o1PN (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html)
<code>df.loc</code>	Позволяет извлекать из датафрейма определенные строки и столбцы	<code>df.loc[df['trip_distance'] > 10, 'passenger_count']</code>	http://mng.bz/nWPv (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html)
<code>pd.read_html</code>	Возвращает список датафреймов на основе HTML	<code>df = df.read_html('https://a-site.com')</code>	http://mng.bz/vnxx (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_html.html)
<code>s.value_counts</code>	Возвращает в отсортированном по убыванию значений виде объект Series с информацией о количестве вхождений в s каждого значения	<code>s.value_counts()</code>	http://mng.bz/yQyJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.value_counts.html)
<code>s.round</code>	Возвращает новый объект Series на основе s, в котором значения округлены до указанного количества знаков	<code>s.round(2)</code>	http://mng.bz/QPyM (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.round.html)
<code>df.memory_usage</code>	Показывает, сколько места в памяти (в байтах) занимает датафрейм и связанные с ним данные	<code>df.memory_usage()</code>	http://mng.bz/XNPY (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.memory_usage.html)
<code>pd.Series.idxmin</code>	Возвращает индекс минимального значения в объекте Series	<code>s.idxmin()</code>	http://mng.bz/ZR6Z (https://pandas.pydata.org/docs/reference/api/pandas.Series.idxmin.html)
<code>pd.Series.idxmax</code>	Возвращает индекс максимального значения в объекте Series	<code>s.idxmax()</code>	http://mng.bz/RmrP (https://pandas.pydata.org/docs/reference/api/pandas.Series.idxmax.html)

Таблица 3.1. Предметы изучения (*продолжение*)

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.DataFrame.agg</code>	Позволяет вызывать один или несколько методов агрегации применительно к датафрейму	<code>df.agg(['min', 'max'])</code>	http://mng.bz/27QX (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.agg.html)

CSV, нестандартный стандарт

Программист Эндрю С. Таненбаум (Andrew S. Tanenbaum) как-то сказал: «Прелесть стандартов заключается в возможности выбрать любой из них». Во многом это применимо и к файлам в формате CSV, с большим отрывом преобладающим в сфере обработки данных. Конечно, многие используют в своей работе реляционные базы данных и Excel, но в большинстве источников данных в интернете файлы хранятся именно в формате CSV.

В своей основе формат CSV предполагает, что данные могут быть представлены в двумерном виде, т. е. в виде таблицы. В качестве строк в файле выступают строки, а для разделения данных на столбцы используются символы-разделители – по умолчанию запятые. Прелесть файлов CSV состоит в том, что по своей сути они представляют обычные текстовые файлы, которые можно прочитать с помощью любого текстового редактора.

Несмотря на свою широкую популярность, формат CSV до сих пор не имеет общей спецификации формата. Существует одно рабочее предложение за номером 4110, доступное по адресу <https://datatracker.ietf.org/doc/html/rfc4180>, но оно носит исключительно информационный характер и относится к далекому 2005 году. И хотя в общих чертах всем понятно, что собой представляет этот формат, существует множество серых зон, которые могут затруднять процесс разбора файлов и делать его неоднозначным.

Вместо того чтобы полагаться на строгие правила о том, как должны выглядеть файлы CSV, pandas пытается быть в этом отношении открытым и гибким. При чтении из файла CSV (с помощью функции `pd.read_csv`) или записи датафрейма в файл в формате CSV (метод `df.to_csv`) мы можем задать огромное количество параметров, управляющих правилами ввода и вывода информации. Среди самых популярных параметров можно отметить следующие:

- `sep` – символ-разделитель, по умолчанию представляющий запятую, но часто может быть представлен и символом табуляции (`'\t'`);
- `header` – отвечает за то, присутствуют ли в файле данные о названиях столбцов, и в какой именно строке;
- `index_col` – отвечает за то, какой столбец в датафрейме станет индексом;
- `usecols` – в этом параметре можно перечислить столбцы из файла CSV, которые будут включены в датафрейм.

К примеру, мы можем вызвать функцию чтения из CSV следующим образом. Графическое изображение работы этой функции показано на рис. 3.1.


```
pd.read_csv('mydata.csv',      ❶
            sep='\t',          ❷
            index_col='w',      ❸
            usecols=['w', 'x', 'z'], ❹
            header=0)          ❺
```

- ❶ Имя файла CSV для чтения.
- ❷ Символы табуляции в качестве разделителей.
- ❸ В качестве индекса выступает столбец w.
- ❹ Читаем из файла только столбцы w, x и z.
- ❺ Первая строка файла содержит имена столбцов.

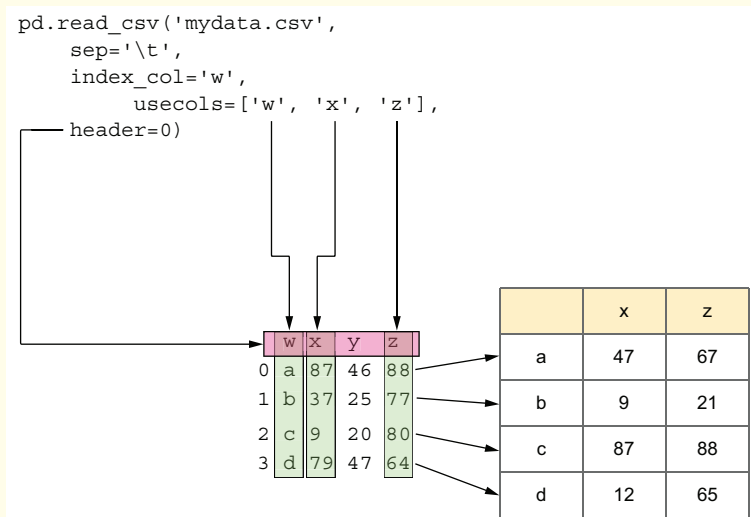


Рис. 3.1. Чтение из файла CSV с использованием нескольких популярных аргументов

Вам стоит ознакомиться с документацией по функции `pd.read_csv`. Вас может удивить, насколько много параметров она может принимать и как по-разному можно читать файлы с ее помощью. Мы в этой книге будем пользоваться лишь основными параметрами, но вам необходимо изучить и остальные.

ПРИМЕЧАНИЕ. На своих курсах я часто повторяю фразу «Изучайте свои данные». Вам действительно необходимо узнать о данных, с которыми вы работаете, как можно больше, прежде чем считывать их в память. Возможно, для работы вам достаточно будет всего нескольких столбцов. А может, для каких-то столбцов лучше будет указать типы данных вручную, а не позволять `pandas` принимать решение по этому поводу самостоятельно. Многие наборы данных поставляются вместе со словарем, в котором описываются колонки, их типы, смысл и область допустимых значений. Если такой словарь есть, вам просто необходимо тщательно его изучить. Это позволит вам более эффективно прочитать данные из набора и обработать их.

УПРАЖНЕНИЕ 15. Загадочные поездки на такси

Во времена моей юности такси в Нью-Йорке поймать было очень просто – ты просто поднимал руку, садился и говорил водителю, куда тебе нужно. В конце поездки платил по счетчику, давал чаевые и получал чек. Конечно, оплата была наличными.

Сейчас же все иначе. Во всех такси есть мониторы, на которых бесконечно крутится реклама и что-то развлекательное. Но эти мониторы там не только для того, чтобы вас раздражать, они выполняют еще и функцию терминала, с помощью которого вы можете оплатить поездку с карты, да еще и там же оставить чаевые. Вся информация о вашей поездке сохраняется и отправляется в так называемую *Комиссию по такси и лимузинам* (Taxi and Limousine Commission – TLC) – городской департамент контроля за пассажирскими перевозками. Далее информация обо всех поездках анализируется, и на ее основании принимаются решения, касающиеся развития транспорта в городе.

К счастью, в этой комиссии работают нежадные люди, и они с радостью предоставляют всем желающим доступ к информации о поездках. Таким образом, вы можете просмотреть все поездки такси за прошедшее десятилетие или около того и изучить, куда люди ездили, сколько они платили, как платили и оставляли ли чаевые. Признаться, это один из моих любимых наборов данных, так что я буду довольно часто обращаться к нему в этой книге. В частности, мы будем исследовать содержимое следующих столбцов:

- `passenger_count` – количество пассажиров в поездке;
- `trip_distance` – дистанция поездки в милях;
- `total_amount` – общая сумма оплаты за поездку, включая налоги, пошлины и чаевые;
- `payment_type` – целое число, описывающее тип оплаты за поездку. Наиболее популярные варианты – 1 (банковская карта) и 2 (наличные).

В этом упражнении мы поработаем с данными о поездках за январь 2019 года.

1. Загрузите информацию из перечисленных выше столбцов файла CSV в датафрейм.
2. Сколько поездок включали в себя больше восьми пассажиров?
3. Сколько поездок включали в себя ноль пассажиров?
4. Сколько раз пассажиры платили в такси наличными, когда сумма составляла больше 1000 долл.?
5. В скольких случаях сумма оплаты за поездку оказывалась отрицательной?
6. Сколько раз за дистанцию меньше средней пассажиры вынуждены были заплатить сумму, превышающую среднюю сумму поездок?

ПРИМЕЧАНИЕ. Почему при чтении данных из файлов CSV мы используем функцию `pd.read_csv`, принадлежащую модулю `pandas`, а не метод одного из существующих датафреймов? Смысл этой функции в том, чтобы создать (и вернуть) новый датафрейм на основе содержимого файла CSV, а не обновить информацию в уже имеющемся датафрейме.

Подробный разбор

Для решения этой задачи нам сперва необходимо прочитать данные из файла. К счастью, информация в нужном нам файле с поездками представлена в виде, пригодном для чтения функцией `pd.read_csv` без дополнительных параметров. Достаточно только указать нужные нам столбцы. Ограничивать количество загружаемых столбцов при чтении данных очень важно. Наш файл содержит данные о 7 667 792 поездках, и, ограничив загрузку четырьмя столбцами, мы сможем снизить объем используемой памяти с 2.4 Гб до 234 Мб. В главе 10 мы поговорим о способах измерения расходуемой памяти и оптимизации в этой области.

Параметр `usecols`, отвечающий за выбор конкретных столбцов, принимает список, который может содержать либо целые числа, указывающие на порядковый номер столбцов, либо строки, соответствующие именам столбцов. Я обычно предпочитаю использовать строки, поскольку это более наглядно.

В результате мы получим датафрейм, состоящий из четырех столбцов и более 7.6 млн строк, в котором собраны все поездки на такси в Нью-Йорке за январь 2019 года. Теперь мы можем ответить на вопросы упражнения.

Сначала нам необходимо узнать, сколько поездок включали в себя больше восьми пассажиров. Стандартный способ решения подобных задач состоит в создании объекта `Series` из булевых значений и применении его к исходному объекту в виде маски. Получить саму маску в виде объекта `Series` для первого задания мы можем так:

```
df['passenger_count'] > 8
```

Теперь воспользуемся атрибутом доступа `loc` для применения к датафрейму нашего индекса-маски:

```
df.loc[df['passenger_count'] > 8]
```

Мы можем подсчитать количество элементов в каждом столбце следующим образом:

```
df.loc[df['passenger_count'] > 8].count()
```

Количество элементов и значение NaN

При применении метода `count` к объекту `Series` мы получим целое число, показывающее, сколько в нем содержится непустых значений. Для датафрейма этот метод возвращает объект `Series`, в котором в качестве индексов выступают названия столбцов, а в качестве значений – количество непустых значений в каждом из них. Рассмотрим простой пример:

```
s = Series([10, 20, np.nan, 40, 50])
s.count()
```

Здесь ответом будет число 4. А следующий запрос вернет объект `Series`:

```
df = DataFrame([[10, 20, np.nan, 40],
                [50, np.nan, np.nan, np.nan],
                [np.nan, 60, 70, 80]],
```

```
index=list('abc'),
columns=list('wxyz'))
df.count()
```

В возвращенном результате показано количество элементов в каждом столбце, не равных NaN:

```
w    2
x    2
y    1
z    2
dtype: int64
```

Но нас интересует только столбец с именем `passenger_count` и количество непустых элементов именно в нем. Дополнительно ограничить количество столбцов в анализе можно в том же атрибуте `loc`, как показано ниже:

```
df.loc[df['passenger_count'] > 8, ❶
      'passenger_count'          ❷
      ].count()                  ❸
```

❶ Селектор строк: только строки с девятью пассажирами и более.

❷ Селектор столбцов: только столбец `passenger_count`.

❸ Сколько здесь непустых значений?

Как видим, всего в девяти случаях нью-йоркские такси за этот период перевозили более восьми пассажиров (надеюсь, в достаточно просторных машинах). На рис. 3.2 схематично показано это вычисление.

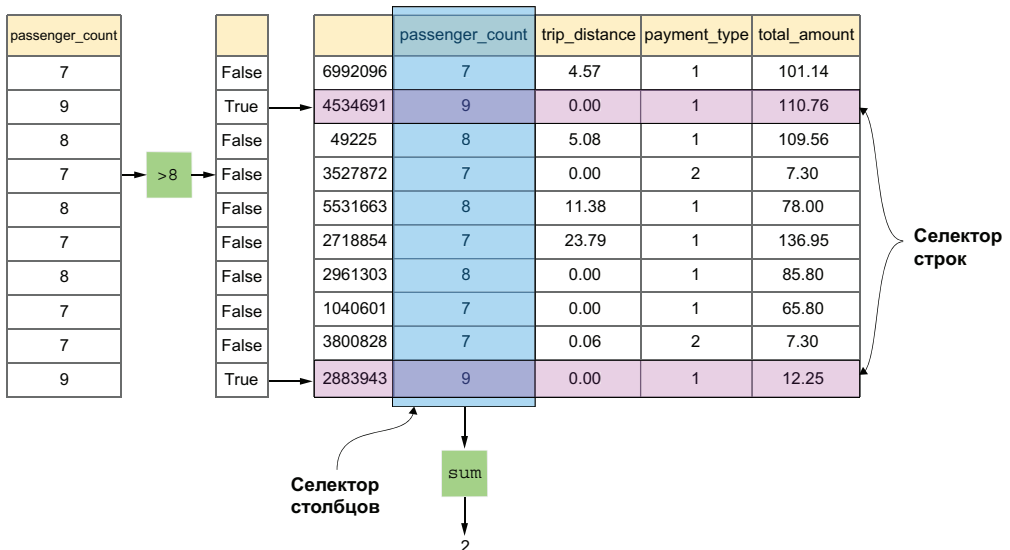


Рис. 3.2. Графическое изображение процедуры выбора строк с `passenger_count > 8` и применение к полученному объекту Series метода `count`

Следующий вопрос звучит так: сколько поездок включали в себя ноль пассажиров? Судя по всему, имеются в виду случаи использования такси для перевозки грузов. Либо водитель просто забыл ввести количество пассажиров. В инструкции к набору данных указано, что водитель вручную вводит эту информацию, так что возможны всякие ошибки.

Решим мы эту задачу так же, как и первую:

```
df.loc[df['passenger_count'] == 0, ❶
       'passenger_count'          ❷
       ].count()                  ❸
```

- ❶ Селектор строк: ищем поездки без пассажиров.
- ❷ Берем только столбец `passenger_count`.
- ❸ Сколько здесь непустых значений?

Ответ вас может удивить: 117 381. Кажется, что довольно много, но это всего 1.5 % от всех поездок за этот месяц.

Хотя в наше время почти все оплачивают такси банковской картой, по-прежнему встречаются те, кто предпочитает платить наличными. Наш третий вопрос звучит так: сколько раз пассажиры платили в такси наличными, когда сумма составляла больше 1000 долл.?

Здесь нам необходимо объединить два условия, а значит, и два объекта `Series` с булевыми значениями. В первом будут собраны фильтрующие значения по типу оплаты (столбец `payment_type`), а во втором – по сумме (столбец `total_amount`). Объединить два условия можно с помощью оператора `&`, как показано ниже:

```
(df['payment_type'] == 2) & (df['total_amount'] > 1000)
```

В результате мы получим объект `Series` с булевыми значениями, в котором значение `True` будет появляться только в случаях, когда в обеих объединяемых масках стоит значение `True`. В противном случае мы увидим значение `False`.

Применить эту объединенную маску к датафрейму можно с помощью атрибута `loc`, ограничив выбор только столбцом `passenger_count` и вызвав метод `count`. На рис. 3.3 схематично показано это вычисление:

```
df.loc[(df['payment_type'] == 2) & (df['total_amount'] > 1000), ❶
       'passenger_count'          ❷
       ].count()                  ❸
```

- ❶ Селектор строк: ищем поездки с оплатой наличными дороже 1000 долл.
- ❷ Берем только столбец `passenger_count`.
- ❸ Сколько здесь непустых значений?

В результате мы получили всего пять поездок. Возможно, это мои личные тараканы, но я был шокирован тем, что кто-то может заплатить больше тысячи долларов наличными в такси. Конечно, таких случаев оказалось предельно мало, но вы можете себе представить, чтобы кто-то в такси вытащил бумажник и отсчитал 1000 долл.?

Но мы отвлеклись.

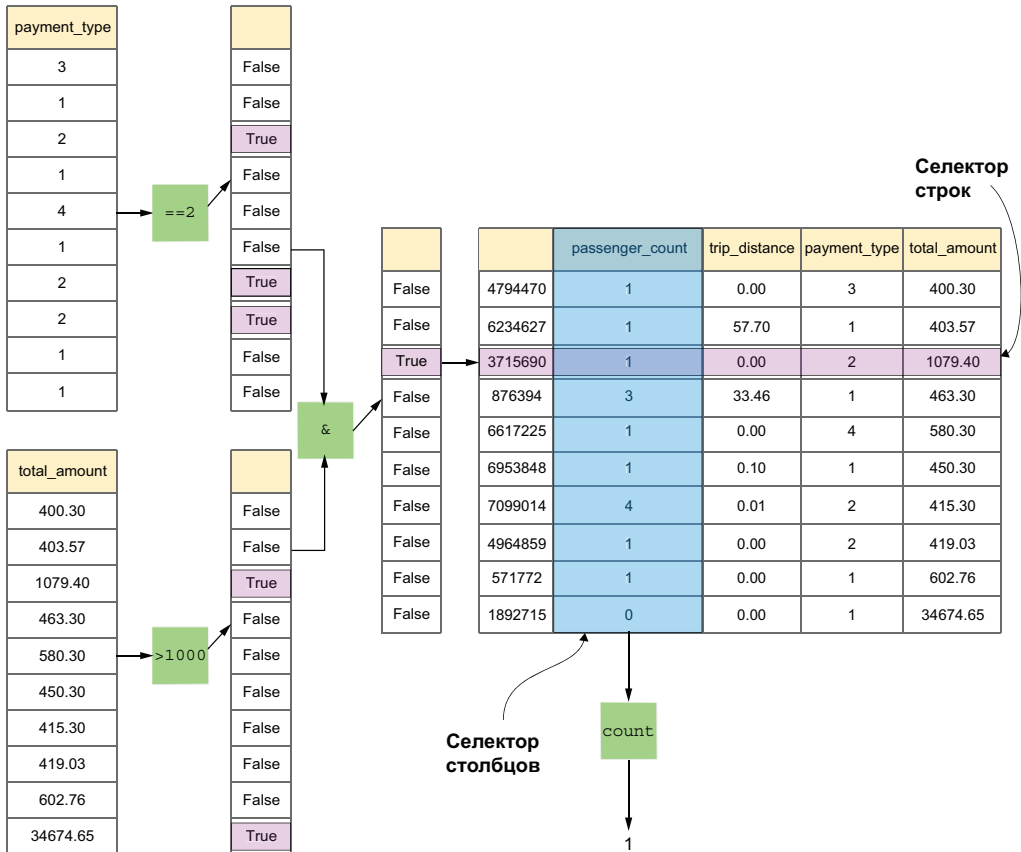


Рис. 3.3. Графическое изображение процедуры выбора строк с `payment_type == 2` и `total_amount > 1000` и применение к полученному объекту Series метода `count`

Далее нас просят найти поездки в наборе данных с отрицательными суммами. Возможно, здесь мы имеем дело с какими-то возвратами, а может, у пассажира возникла переплата за предыдущую его поездку. Как бы то ни было, пойдем по старой схеме и создадим объект Series с нашим условием:

```
df['total_amount'] < 0
```

Далее применим этот объект к колонке `total_amount` в качестве маски и рассчитаем количество элементов методом `count`:

```
df.loc[df['total_amount'] < 0, 'total_amount'].count()
```

В результате мы получили 7131 поездку с отрицательной суммой, что составляет порядка 0.01 % от общего количества поездок.

Наконец, мы спросили, сколько раз за дистанцию меньше средней пассажиры вынуждены были заплатить сумму, превышающую среднюю сумму поездок.

Для решения этой задачи нам сначала надо выбрать поездки с дистанцией меньше средней. Сделать это можно так:

```
df['trip_distance'] < df['trip_distance'].mean()
```

Затем сформулируем условие для отбора поездок с суммой, превышающей среднюю сумму поездок:

```
df['total_amount'] > df['total_amount'].mean()
```

Объединим наши условия для получения единого объекта Series с нужной нам маской:

```
(df['trip_distance'] < df['trip_distance'].mean()) &
(df['total_amount'] > df['total_amount'].mean())
```

Наконец, воспользуемся атрибутом `loc`, ограничив выбор столбцом `trip_distance`, и применим к результату метод `count`. На рис. 3.4 схематично показано это вычисление:

```
df.loc[(df['trip_distance'] < df['trip_distance'].mean()) & ❶
(df['total_amount'] > df['total_amount'].mean()), ❷
'trip_distance' ❸
].count() ❹
```

- ❶ Первая часть селектора строк: ищем поездки с дистанцией меньше средней.
- ❷ Вторая часть селектора строк: ищем поездки с суммой, превышающей среднюю сумму поездок.
- ❸ Берем только столбец `trip_distance`.
- ❹ Сколько здесь непустых значений?

В сумме мы получили 411 255 поездок, удовлетворяющих этим условиям, или 5 % от всего набора данных.

Решение

```
df = pd.read_csv('../data/nyc_taxi_2019-01.csv',
                 usecols=['passenger_count', 'trip_distance',
                          'total_amount', 'payment_type'])
df.loc[df['passenger_count'] > 8, 'passenger_count'].count()
df.loc[df['passenger_count'] == 0, 'passenger_count'].count()
df.loc[(df['payment_type'] == 2) & (df['total_amount'] > 1000),
        'passenger_count'].count()
df.loc[df['total_amount'] < 0, 'total_amount'].count()
df.loc[(df['trip_distance'] < df['trip_distance'].mean()) &
(df['total_amount'] > df['total_amount'].mean()), 'trip_distance'].count()
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/84vP>.

Дополнительные упражнения

- Повторите это упражнение с применением метода `query` вместо булева индекса и атрибута `loc`.
- Сколько поездок с отрицательной суммой были помечены в базе как спорные (`payment_type = 4`) или аннулированные (`payment_type = 6`)?
- Как мы уже говорили ранее, большинство людей сегодня платят за такси банковскими картами. Выведите доли оплаты наличными и банковскими картами.

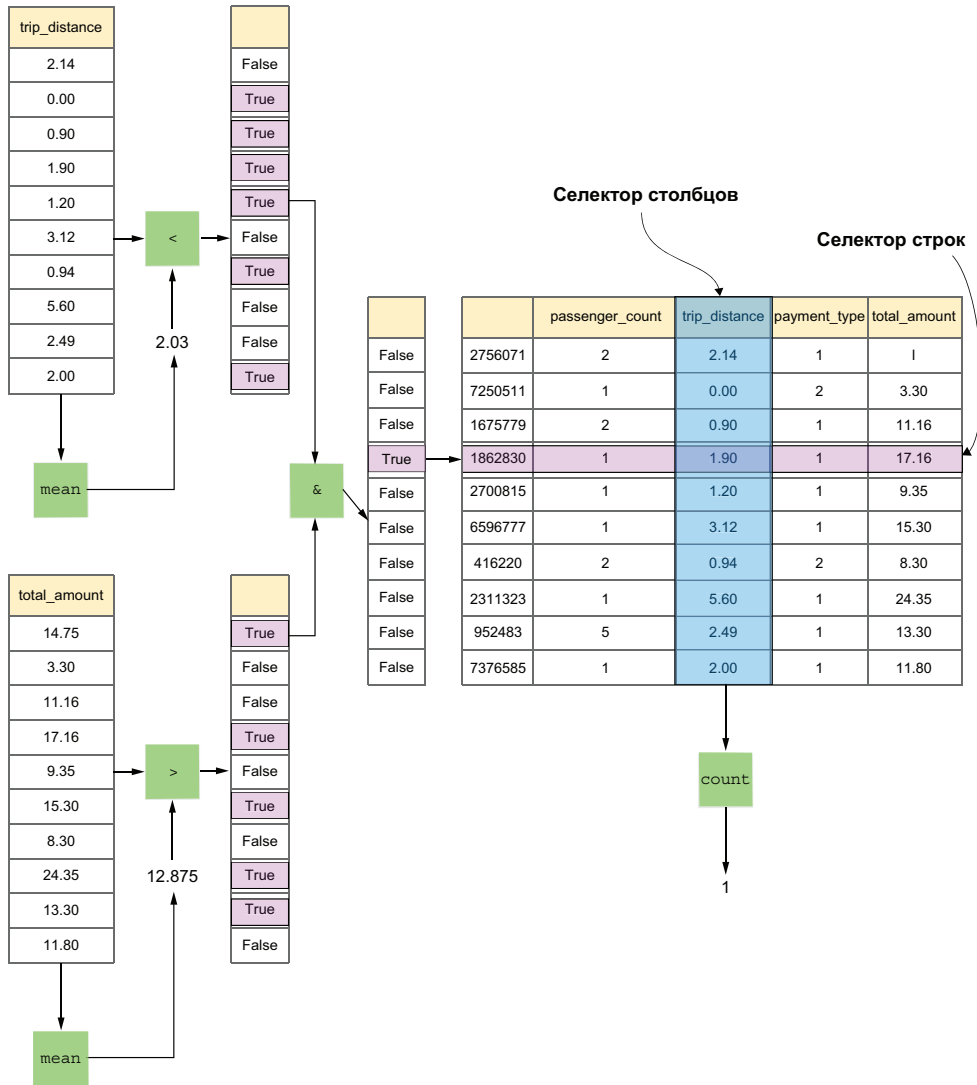


Рис. 3.4. Графическое изображение процедуры выбора строк с `trip_distance` меньше среднего и `total_amount` больше среднего и применение к полученному объекту Series метода `count`

Ответы на дополнительные упражнения

Упражнение 15.1

```
# Сколько поездок включали в себя больше восьми пассажиров? (версия с query)
df.query('passenger_count > 8')['passenger_count'].count()
```

Вывод:

9

```
# Сколько поездок включали в себя ноль пассажиров? (версия с query)
```



```
df.query('passenger_count == 0')['passenger_count'].count()
```

Вывод:

```
117381
```

```
# Сколько раз пассажиры платили в такси наличными, когда сумма
# составляла больше 1000 долл.? (версия с query)
```

```
df.query('payment_type == 2 & total_amount > 1000')['payment_type'].count()
```

Вывод:

```
5
```

```
# В скольких случаях сумма оплаты за поездку оказалась
# отрицательной? (версия с query)
```

```
df.query('total_amount < 0')['total_amount'].count()
```

Вывод:

```
7131
```

```
# Сколько раз за дистанцию меньше средней пассажиры вынуждены были
# заплатить сумму, превышающую среднюю сумму поездок? (версия с query)
```

```
df.query('trip_distance < trip_distance.mean() &
total_amount > total_amount.mean()')['trip_distance'].count()
```

Вывод:

```
411255
```

Упражнение 15.2

```
df.loc[(df['total_amount'] < 0) &
       ((df['payment_type'] == 4) |
        (df['payment_type'] == 6)), 'total_amount'].count()
```

Вывод:

```
2666
```

Упражнение 15.3

```
# 1 == банковская карта
```

```
# 2 == наличные
```

```
df['payment_type'].value_counts(normalize=True)[[1,2]]
```

Вывод:

```
payment_type
1    0.715464
2    0.278752
Name: proportion, dtype: float64
```

УПРАЖНЕНИЕ 16. *Такси и пандемия*

Неудивительно, что пандемия коронавируса, разразившаяся в начале 2020 года и принесящая много бед и несчастий, не могла не сказаться и на поездках в нью-йоркских такси. В этом упражнении мы сосредоточимся на объединении разных наборов данных и сравнении показателей до и во время эпидемии.

Я хочу, чтобы вы собрали датафрейм на основе двух файлов CSV: один относится к июлю 2019 года (до коронавируса), а второй – к июлю 2020 года (во время пика болезни, по крайней мере в Нью-Йорке). В датафрейме должны присутствовать три столбца из исходных файлов: `passenger_count`, `total_amount` и `payment_type`. Также будет удобно иметь столбец с годом, который будет заполнен в зависимости от файла-источника.

Вооружившись этими данными, ответьте на следующие вопросы:

- сколько поездок было сделано в июле в 2019 и 2020 годах? Какова разница между этими показателями?
- сколько денег в сумме заплатили пассажиры за такси в июле в 2019 и 2020 годах? Какова разница между этими показателями?
- существенно ли изменилась доля поездок с более чем одним пассажиром в июле 2020 года по сравнению с июлем 2019-го?
- стали ли люди меньше пользоваться наличными деньгами (`payment_type = 2`) в июле 2020 года по сравнению с июлем 2019-го?

ПРИМЕЧАНИЕ. В `pandas` есть множество техник для выполнения группировки данных и работы с датой и временем, о которых мы будем подробно говорить в главах 6, 7 и 10. Пока же мы будем решать подобные задачи без их использования.

Подробный разбор

Существует бесчисленное число способов оценить влияние пандемии коронавируса на жизнедеятельность людей. В этом упражнении мы коснемся вопросов, связанных с изменениями пассажиропотока в результате распространения заболевания.

Для начала соберем датафрейм на основе двух наборов данных за разные годы. В главе 1 мы видели, как можно применить функцию `pd.concat` для объединения двух объектов `Series`. Оказывается, эту функцию столь же эффективно можно использовать и для объединения целых датафреймов, что мы и попробуем сделать. Загрузим информацию из двух файлов CSV в разные датафреймы, после чего объединим их:

```
df_2019_jul = pd.read_csv('../data/nyc_taxi_2019-07.csv',
                          usecols=['passenger_count',
                                   'total_amount', 'payment_type'])

df_2020_jul = pd.read_csv('../data/nyc_taxi_2020-07.csv',
                          usecols=['passenger_count',
                                   'total_amount', 'payment_type'])

df = pd.concat([df_2019_jul, df_2020_jul])
```

Если бы нам было достаточно сквозных агрегаций в полученном наборе данных, мы бы на этом остановились. Но нам необходимо выполнять сравнение двух годов, так что без столбца `year` нам здесь просто не обойтись. Мы можем добавить этот столбец перед объединением данных, как показано ниже и на рис. 3.5:

```
df_2019_jul = pd.read_csv('../data/nyc_taxi_2019-07.csv',
                           usecols=['passenger_count',
                                    'total_amount', 'payment_type'])
df_2019_jul['year'] = 2019 ❶

df_2020_jul = pd.read_csv('../data/nyc_taxi_2020-07.csv',
                           usecols=['passenger_count',
                                    'total_amount', 'payment_type'])
df_2020_jul['year'] = 2020 ❷

df = pd.concat([df_2019_jul, df_2020_jul]) ❸
```

- ❶ Добавление столбца с годом для набора данных 2019 года.
- ❷ Добавление столбца с годом для набора данных 2020 года.
- ❸ Создание нового датафрейма на основе двух отдельных.

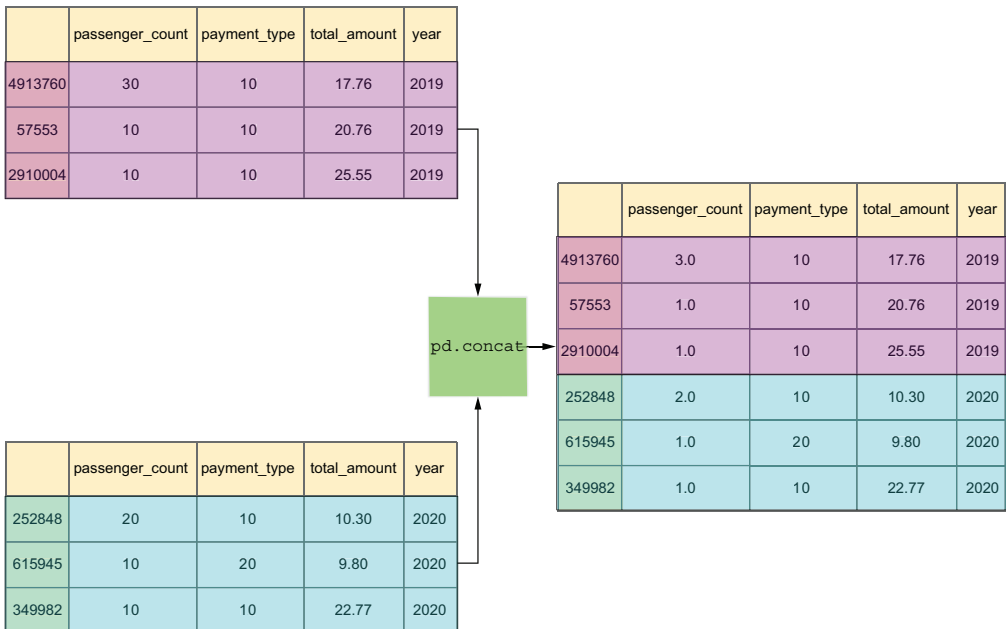


Рис. 3.5. Объединение двух датафреймов с дополнительным столбцом для года в один

Итак, теперь в нашем распоряжении есть единый датафрейм `df`, который мы можем использовать для ответов на поставленные ранее вопросы. Для начала узнаем, сколько поездок было в июле 2019 года и 2020-го. Это можно сделать, вызвав метод `count` для любого из столбцов и применив фильтр на год, после чего вычесть один показатель из другого (см. рис. 3.6):

```
(
    df.loc[df['year'] == 2019, 'total_amount'].count() - ❶
    df.loc[df['year'] == 2020, 'total_amount'].count() ❷
)
```

- ❶ Количество поездок в 2019 году.
- ❷ Количество поездок в 2020 году.

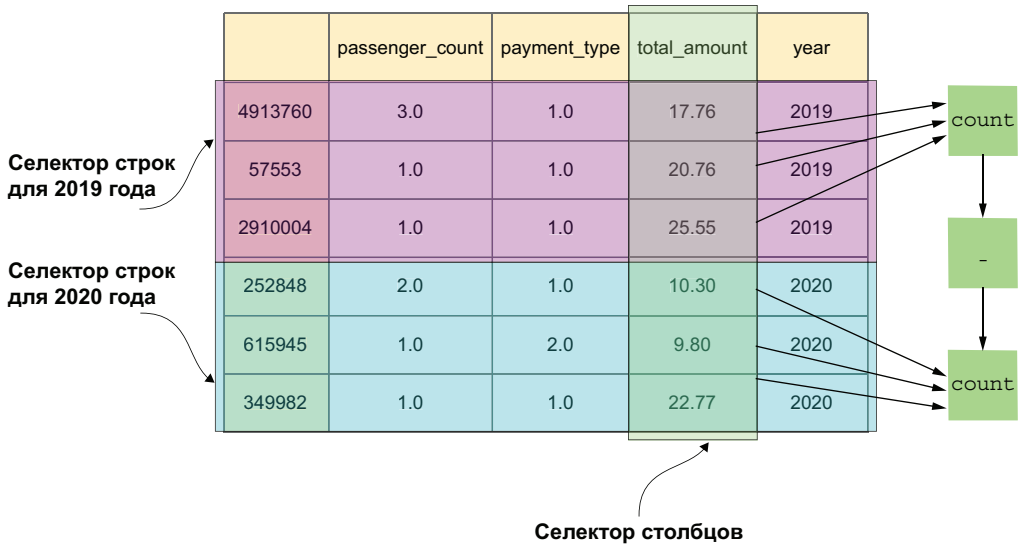


Рис. 3.6. Сравнение количества поездок в двух годах

Результат оказался равен 5 510 007. Таким образом, в июле 2020 года жители Нью-Йорка совершили на 5.5 млн поездок на такси меньше, чем годом ранее. Если в 2019-м общее количество поездок составило 6 310 419, то всего спустя год оно снизилось до 800 412, т. е. почти в 8 раз!

Теперь давайте подсчитаем убытки таксопарка. Здесь вместо метода `count` мы воспользуемся методом `sum` применительно к столбцу `total_amount`:

```
(
    df.loc[df['year'] == 2019, 'total_amount'].sum() - ❶
    df.loc[df['year'] == 2020, 'total_amount'].sum() ❷
)
```

- ❶ Суммарная прибыль в июле 2019 года.
- ❷ Суммарная прибыль в июле 2020 года.

Ответ, который я получил, – 108848979.24000001, или более 108 млн долл., – 123 млн долл. в 2019 году против 15 млн долл. в 2020-м (см. рис. 3.7). Не знаю, как вы, а я просто в шоке от таких цифр.

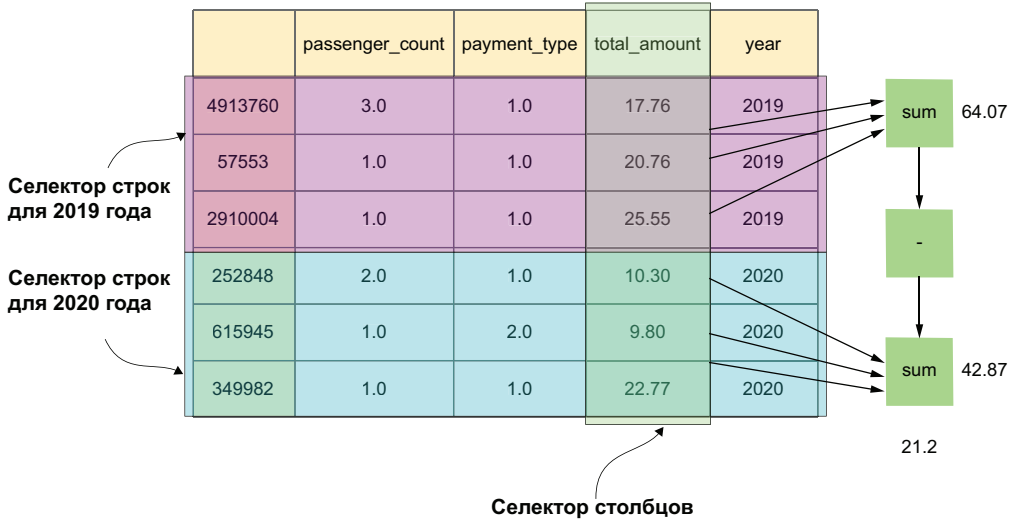


Рис. 3.7. Сравнение суммарной прибыли от такси в двух годах

Округление чисел с плавающей точкой

Если вас беспокоит большое количество десятичных знаков в показателях, вы можете воспользоваться методом `round`, применив его к результатам метода `sum`, как показано ниже:

```
df.loc[df['year'] == 2019, 'total_amount'].sum().round(2) -
df.loc[df['year'] == 2020, 'total_amount'].sum().round(2)
```

Вполне объяснимо, что общее число поездок на такси существенно снизилось во время пандемии. Но мы могли бы также задаться вопросом о том, как изменилось поведение людей. К примеру, с учетом того, что в июле 2020 года наблюдался пик заболевания, а вакцины на тот момент еще не было, люди старались больше дистанцироваться друг от друга. В результате могло снизиться количество поездок в такси в компании с кем-то. И в следующем нашем вопросе затрагивается именно этот аспект, а именно: существенно ли изменилась доля поездок с более чем одним пассажиром в июле 2020 года по сравнению с июлем 2019-го? Для ответа на него мы разделим количество поездок с более чем одним пассажиром на общее количество поездок:

```
df.loc[
    (df['year'] == 2019) &
    (df['passenger_count'] > 1), 'passenger_count'].count() / ❶
    df.loc[df['year'] == 2019, 'payment_type'].count() ❷

df.loc[
    (df['year'] == 2020) &
```

```
(df['passenger_count'] > 1), 'passenger_count'].count() / ❸
df.loc[df['year'] == 2020, 'payment_type'].count() ❹
```

- ❶ Количество поездок с более чем одним пассажиром в июле 2019 года.
- ❷ Общее количество поездок в июле 2019 года.
- ❸ Количество поездок с более чем одним пассажиром в июле 2020 года.
- ❹ Общее количество поездок в июле 2020 года.

В итоге доля поездок с более чем одним пассажиром в июле 2019 года составила 28 %, а в июле 2020-го – 21 %, что говорит о том, что во время пандемии люди начали больше ездить в такси по одному. Но дело может быть еще и в том, что в пик заболеваемости развлекательных событий в Нью-Йорке почти не осталось, и люди больше стали использовать такси для поездок на работу и домой.

Наконец, мы хотели бы узнать, как изменилась доля оплаты такси наличными с приходом коронавируса, поскольку многие старались отказаться от купюр и физических контактов. Вот как можно это рассчитать (см. рис. 3.8):

```
df.loc[
    (df['year'] == 2019) &
    (df['payment_type'] == 2), 'payment_type'].count() / ❶
df.loc[df['year'] == 2019, 'payment_type'].count() ❷
```

```
df.loc[
    (df['year'] == 2020) &
    (df['payment_type'] == 2), 'payment_type'].count() / ❸
df.loc[df['year'] == 2020, 'payment_type'].count() ❹
```

- ❶ Общее количество поездок в июле 2019 года.
- ❷ Количество поездок с наличной оплатой в июле 2019 года.
- ❸ Общее количество поездок в июле 2020 года.
- ❹ Количество поездок с наличной оплатой в июле 2020 года.

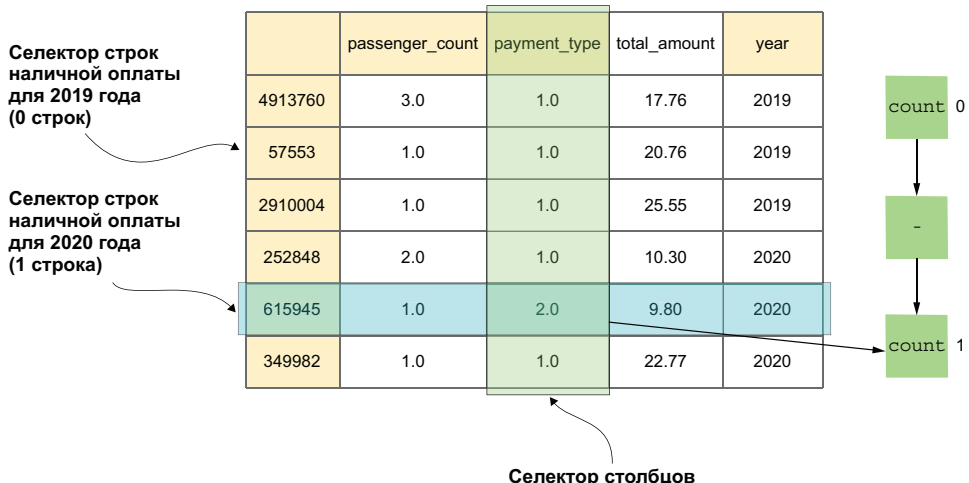


Рис. 3.8. Сравнение долей наличной оплаты такси в двух годах

В данном случае результат меня несколько удивил. В июле 2019 года доля наличных оплат составляла 29 %, а через год повысилась до 32 %, хотя я ожидал обратной тенденции с учетом того, что многие стали склоняться к бесконтактным способам оплаты услуг. Посмею выдвинуть теорию о том, что во время пандемии многие перешли на удаленку, а личное присутствие на рабочем месте требовалось только от работников жизненно важных сфер услуг. Но, как мы знаем, их труд оплачивается не так высоко, и они зачастую по-прежнему предпочитают расплачиваться наличными. Так или иначе, доля наличных оплат в такси выросла, это факт.

Решение

```
df_2019_jul = pd.read_csv('../data/nyc_taxi_2019-07.csv',
                          usecols=['passenger_count',
                                   'total_amount', 'payment_type'])
df_2019_jul['year'] = 2019

df_2020_jul = pd.read_csv('../data/nyc_taxi_2020-07.csv',
                          usecols=['passenger_count',
                                   'total_amount', 'payment_type'])
df_2020_jul['year'] = 2020

df = pd.concat([df_2019_jul, df_2020_jul])

df.loc[df['year'] == 2019, 'total_amount'].count() - df.loc[df['year'] ==
2020, 'total_amount'].count()
df.loc[df['year'] == 2019, 'total_amount'].sum() - df.loc[df['year'] ==
2020, 'total_amount'].sum()

df.loc[(df['year'] == 2019) &
       (df['passenger_count'] > 1), 'passenger_count'].count() /
df.loc[df['year'] == 2019, 'payment_type'].count()
df.loc[(df['year'] == 2020) &
       (df['passenger_count'] > 1), 'passenger_count'].count() /
df.loc[df['year'] == 2020, 'payment_type'].count()

df.loc[(df['year'] == 2019) &
       (df['payment_type'] == 2), 'payment_type'].count() /
df.loc[df['year'] == 2019, 'payment_type'].count()
df.loc[(df['year'] == 2020) &
       (df['payment_type'] == 2), 'payment_type'].count() /
df.loc[df['year'] == 2020, 'payment_type'].count()
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/g7JE>.

Дополнительные упражнения

- Воспользуйтесь методом corr датафрейма df для обнаружения корреляций между столбцами. Как бы вы объяснили полученные результаты?
- Продемонстрируйте с помощью одной команды разницу в описательной статистике для столбца total_amount между 2019 и 2020 годами. Округлите значения не более чем до двух знаков после запятой.

- Если предположить, что поездки без пассажиров могут использоваться для перевозки грузов, как во время пандемии изменилась ситуация с этим видом активности? Рассчитайте доли таких поездок в июле 2019 года и 2020-го.

Датафреймы и dtype

В главе 1 мы уже говорили, что каждый объект `Series` имеет атрибут `dtype` с типом содержащихся в нем данных. При этом тип данных можно извлечь, обратившись к атрибуту `dtype`, а можно и установить его, задав значение для этого атрибута при создании объекта `Series`.

Как вам уже известно, в датафрейме каждый столбец представляет собой отдельный объект `Series`, а значит, обладает своим атрибутом `dtype`. С помощью атрибута `dtypes` (обратите внимание на множественное число) мы можем определить типы данных столбцов в датафрейме. Эту, а также другую вспомогательную информацию вы можете извлечь и при помощи метода `info`.

При чтении данных из файла `CSV` `pandas` по умолчанию пытается самостоятельно вывести типы данных для всех загружаемых столбцов. Если помните, файлы `CSV` по своей сути являются обычными текстовыми файлами, так что `pandas` приходится анализировать содержимое столбцов, чтобы правильно определить их тип. В своем выборе он ограничен тремя следующими вариантами:

- если все значения в столбце могут быть представлены как целые числа, выбор падает на тип `int64`;
- если все значения могут быть преобразованы в числа с плавающей точкой, что включает значения `NaN`, тип определяется как `float64`;
- иначе тип становится `object`, что характерно для всех объектов в `Python`.

Но с автоматическим выбором типов данных есть определенные проблемы. Первая из них состоит в том, что `pandas` может и обычно определяет для данных избыточный тип. Поверьте, вам в ваших данных редко понадобятся 64-битные числа, так зачем хранить их с типом `int64` или `float64`?

Вторая проблема более тонкая и состоит в том, что движку `pandas` для определения типа данных в столбце сперва необходимо проанализировать его содержимое. И если таблица содержит миллионы строк, для этого ему понадобится приличное количество памяти. По этой причине функция `read_csv` читает файл в память блоками, анализируя каждый из них и создавая датафрейм в фоновом режиме. Обычно вы даже не знаете, как именно происходит этот процесс, но его целью является экономия памяти.

Потенциально это может приводить к проблемам, если `pandas` обнаружит, к примеру, в верхней части файла значения, похожие на целые числа, а в нижней – значения, напоминающие строки. В этом случае атрибут `dtype` будет установлен в `object`, а значения будут разных типов. Это почти наверняка будет плохо для анализа, и `pandas` непременно проинформирует вас об этом при помощи предупреждения `DtypeWarning`. Если загрузить данные о нью-йоркских такси за январь 2020 года без выбора столбцов с помощью параметра `usecols`, вы можете получить такое предупреждение. На моем компьютере оно появляется часто.

Один из способов избежать проблемы со смешанными типами данных состоит в том, чтобы позволить pandas не экономить память и проанализировать все входные данные. Это можно сделать, передав значение `False` параметру `low_memory` при вызове функции `read_csv`. По умолчанию этот параметр установлен в `True`, что приводит к описанному здесь поведению. Но помните, что этот способ может быть сопряжен с большим расходом памяти при открытии объемных наборов данных.

Лучше будет избавить pandas от необходимости определять типы данных в столбцах автоматически, а задать их вручную. Это можно сделать при помощи параметра `dtype` при вызове функции `read_csv`, передав ему словарь. В качестве ключей словаря указываются имена читаемых столбцов, а в качестве значений – устанавливаемые для них типы данных. Обычно принято использовать типы данных pandas и NumPy, но, если вы укажете просто тип `int` или `float`, pandas будет использовать тип `np.int64` или `np.float64` соответственно. Если же указать тип `str`, данные будут сохранены в виде строк Python, а атрибут `dtype` примет значение `object`.

Пример:

```
df_2019_jul = pd.read_csv('../data/nyc_taxi_2019-07.csv',
                          usecols=['passenger_count',
                                   'total_amount', 'payment_type'],
                          dtype={'passenger_count': np.int8,
                                 'total_amount': np.float32,
                                 'payment_type': np.int8})
```

Наконец, стоит отметить, что часто можно поддаться соблазну и присвоить атрибуту `dtype` целочисленный тип. Но не забывайте, что при наличии пропущенных значений (`NaN`) столбец не может быть определен как целочисленный. В этом случае вам придется прочитать столбец в виде чисел с плавающей точкой, затем избавиться от пропущенных значений путем удаления строк или интерполяции и только после этого преобразовать данные в целочисленный тип при помощи метода `astype`.

Ответы на дополнительные упражнения

Упражнение 16.1

```
df.corr()
```

Вывод:

	passenger_count	payment_type	total_amount	year
passenger_count	1.000000	0.016410	0.014943	-0.049558
payment_type	0.016410	1.000000	-0.138561	0.029277
total_amount	0.014943	-0.138561	1.000000	-0.019706
year	-0.049558	0.029277	-0.019706	1.000000

Как видим, между столбцами в нашем датафрейме отсутствует явная корреляция. Самая тесная связь наблюдается между столбцами `payment_type` и `total_amount`, но и она довольно слабая.

На самом деле поле с типом оплаты является категориальным, а значит, мы не можем говорить здесь о какой-то числовой корреляции в явном виде. Но с учетом того, что значение 1 соответствует банковской карте, а 2 – наличными деньгами, положительная корреляция может означать, что при увеличении стоимости поездки увеличивается желание пассажиров оплачивать услуги наличными. Отрицательная корреляция, наоборот, может говорить об увеличении доли оплаты банковской картой при увеличении стоимости поездки, что мы видим в данном случае и что соответствует здравому смыслу. Но опять же корреляция в абсолютном выражении здесь крайне мала.

Упражнение 16.2

```
(
    df.loc[df['year'] == 2020, 'total_amount'].describe().round(2) -
    df.loc[df['year'] == 2019, 'total_amount'].describe().round(2)
)
```

Вывод:

```
count    -5510007.00
mean         -0.98
std          -0.75
min          53.20
25%         -0.50
50%         -0.60
75%         -0.75
max        -4672.45
Name: total_amount, dtype: float64
```

Упражнение 16.3

```
df.loc[(df['year'] == 2019) &
       (df['passenger_count'] == 0), 'passenger_count'].count() /
df['passenger_count'].count()
```

Вывод:

```
0.01666432611802781
```

```
df.loc[(df['year'] == 2020) &
       (df['passenger_count'] == 0), 'passenger_count'].count() /
df['passenger_count'].count()
```

Вывод:

```
0.0027809994974354953
```

Получается, что с началом пандемии доля использования такси в качестве средства для перевозки грузов также снизилась – с 1.6 % в 2019 году до 0.2 % в 2020-м. Честно признаться, я ожидал, что этот показатель вырастет.

УПРАЖНЕНИЕ 17. Установка типов данных для столбцов

В этом упражнении вам нужно будет снова создать датафрейм на основе данных о поездках в нью-йоркском такси за январь 2020 года. Но в этот раз мы будем хранить их в максимально компактном виде, насколько это возможно, чтобы не расходовать лишнюю память. Итак, в этом упражнении вы должны сделать следующее:

- установить типы столбцов при чтении из файла;
- найти строки со значениями NaN. В каких столбцах стоят пропущенные значения и почему?
- удалить все строки, содержащие пропущенные значения;
- задать атрибуту `dtype` наименьшие допустимые значения.

Подробный разбор

На первый взгляд может показаться, что в этом упражнении мы научимся только устанавливать типы данных при их чтении, но на самом деле мы затронем и тему очистки данных, и научимся задавать атрибут `dtype` после загрузки.

Начнем с чтения данных за январь 2020 года с помощью функции `read_csv`, но на этот раз прямо при чтении установим атрибуты `dtype` для колонок. В теории наиболее приемлемым типом данных для столбцов `passenger_count` и `payment_type` является `int8`, поскольку значения в этих столбцах не могут превышать 128.

Но если попытаться задать этот тип при чтении, мы столкнемся с проблемой, связанной с обнаружением пустых значений в целочисленных столбцах. Поскольку значение NaN с точки зрения `pandas` представляет собой число с плавающей точкой и не может быть преобразовано в целое число, нам нужно изначально задать для наших столбцов тип с плавающей точкой. Мы можем выбрать для этой цели тип `float32`, а затем, когда избавимся от значений NaN, преобразовать столбец в `int8`.

Кажется странным, что мы устанавливаем для столбцов при их загрузке заведомо неверный тип данных. Может, лучше дать `pandas` самому определить тип и не вмешиваться в этот процесс, а затем просто переопределить его? Дело в том, что в больших наборах данных может оказаться сразу несколько `dtype` для одной колонки. Причина в том, что `pandas` читает данные блоками и определяет тип данных для каждого из них. Если атрибуты `dtype` для всех блоков данных совпадут, этот тип будет определен для всего столбца. В противном случае будет выбран базовый тип `object`.

ПРИМЕЧАНИЕ. Разбиение на блоки, о котором мы здесь говорим, производится в `pandas` автоматически при чтении данных из файла. Существуют и отдельные функции для чтения данных по блокам, но о них мы поговорим в главе 12.

Почему в столбцах `passenger_count` и `payment_type` могут содержаться пропуски? Одна из причин состоит в том, что эти поля заполняются водителем такси вручную. Но при этом стоит отметить, что пропущенных значений в наших данных не так много: на 6.4 млн поездок всего 65 441 строка со значениями NaN, что

составляет чуть более 1 %. Нет ничего страшного в том, что в одной поездке из ста водитель может забыть установить какое-то значение.

Как бы то ни было, чтобы назначить этим столбцам целочисленный тип, необходимо сначала избавиться от пустых значений в них. Это можно сделать с помощью метода `df.dropna()`, возвращающего новый датафрейм с удаленными строками с пустыми значениями (см. рис. 3.9). Мы можем присвоить результат этого метода обратно переменной `df`:

```
df = df.dropna()
```



	passenger_count	payment_type	total_amount
1989781	1.0	1.0	10.296875
6355241	NaN	NaN	25.546875
6234861	1.0	1.0	75.812500
4320340	1.0	1.0	16.562500
1847070	3.0	1.0	9.296875
211378	2.0	1.0	25.703125
3581544	1.0	1.0	15.359375
3568409	1.0	1.0	15.953125
1057067	1.0	2.0	5.300781
5894087	2.0	1.0	23.156250

	passenger_count	payment_type	total_amount
1989781	1.0	1.0	10.296875
6234861	1.0	1.0	75.812500
4320340	1.0	1.0	16.562500
1847070	3.0	1.0	9.296875
211378	2.0	1.0	25.703125
3581544	1.0	1.0	15.359375
3568409	1.0	1.0	15.953125
1057067	1.0	2.0	5.300781
5894087	2.0	1.0	23.156250

Рис. 3.9. Удаление строк из датафрейма, содержащих пропущенные значения

Несмотря на то что метод `df.dropna()` возвращает новый датафрейм, данные в нем могут совместно использоваться разными датафреймами для большей эффективности. Как следствие, попытка модифицировать полученный датафрейм может привести к появлению предупреждения `SettingWithCopyWarning`, о котором мы уже рассказывали ранее. Во избежание этого можно воспользоваться методом `copy()` для нашего датафрейма, чтобы гарантировать разделение данных за сценой:

```
df = df.dropna().copy()
```

Без использования метода `copy` вы в дальнейшем можете столкнуться с предупреждением и невозможностью внести в данные какие-либо изменения.

Теперь, когда мы избавились от значений NaN, можно присвоить столбцам желаемые типы данных, как показано ниже:

```
df['passenger_count'] = df['passenger_count'].astype(np.int8)
df['payment_type'] = df['payment_type'].astype(np.int8)
```

Решение

```
df = pd.read_csv('../data/nyc_taxi_2020-01.csv',
                 usecols=['passenger_count',
                          'total_amount', 'payment_type'],
                 dtype={'passenger_count':np.float32,
                       'total_amount':np.float32,
                       'payment_type':np.float32}) ❶

df.count() ❷
df = df.dropna().copy() ❸

df['passenger_count'] = df['passenger_count'].astype(np.int8) ❹
df['payment_type'] = df['payment_type'].astype(np.int8)
```

- ❶ Используем тип `np.float32` для всех столбцов, поскольку в двух из них есть значения NaN.
- ❷ Применяем метод `df.count` для определения того, в каких столбцах есть значения NaN.
- ❸ Удаляем все строки хотя бы с одним значением NaN, копируем данные в новый датафрейм и присваиваем его обратно переменной `df`.
- ❹ Присваиваем столбцам новые типы данных.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/eEKv>.

Дополнительные упражнения

- Создайте датафрейм из других четырех столбцов: `VendorID`, `trip_distance`, `tip_amount` и `total_amount`, указав для каждого из них атрибут `dtype`. Какие типы данных лучше всего подходят каждому из этих столбцов? Можно ли установить их напрямую, или сначала необходимо очистить данные?
- Вместо удаления строк из датафрейма с пропущенными значениями в столбце `VendorID` поменяйте эти значения на число 3. Как это скажется на процессе указания типов и очистки данных?
- Подробное о методе `memory_usage` мы будем говорить в главе 11, а сейчас вам достаточно знать, что он позволяет определить, сколько места в памяти (в байтах) занимает датафрейм и связанные с ним данные. Результат этот метод возвращает в виде объекта `Series`, в котором индексами выступают имена столбцов, а в качестве значений представлены целочисленные показатели использования памяти каждым столбцом. Сравните объем памяти, занимаемый датафреймом с типами данных `np.float32` для всех столбцов (как при его загрузке), с объемом памяти для датафрейма с расширенными типами данных `np.float64` для последних трех столбцов.

Ответы на дополнительные упражнения

Упражнение 17.1

```
df = pd.read_csv('../data/nyc_taxi_2020-01.csv',
                 usecols=['VendorID', 'trip_distance', 'tip_amount', 'total_amount'],
                 dtype={'VendorID':np.float32,
                        'trip_distance':np.float32,
                        'tip_amount':np.float32,
                        'total_amount':np.float32})

df = df.dropna().copy()
df.loc['VendorID'] = df['VendorID'].astype(np.int8)
```

Вывод:

Для столбца VendorID мы выбрали тип данных np.int8.

Упражнение 17.2

```
df = pd.read_csv('../data/nyc_taxi_2020-01.csv',
                 usecols=['VendorID', 'trip_distance', 'tip_amount', 'total_amount'],
                 dtype={'VendorID':np.float32,
                        'trip_distance':np.float32,
                        'tip_amount':np.float32,
                        'total_amount':np.float32})

df['VendorID'] = df['VendorID'].fillna(3)
df['VendorID'] = df['VendorID'].astype(np.int8)
```

Упражнение 17.3

```
# Использование памяти с типами float32
df.memory_usage().sum()
```

Вывод:

```
memory usage: 145.1+ MB
152149632
```

Изменим типы данных для последних трех столбцов на np.float64 и снова посмотрим на используемую память:

```
df['trip_distance'] = df['trip_distance'].astype(np.float64)
df['tip_amount'] = df['tip_amount'].astype(np.float64)
df['total_amount'] = df['total_amount'].astype(np.float64)
```

```
# Использование памяти с типами float64 для последних трех столбцов
df.memory_usage().sum()
```

Вывод:

```
memory usage: 217.7+ MB
228224448
```

Как видите, расход памяти вырос со 145 до 217 Мб.

УПРАЖНЕНИЕ 18. Файл *passwd* в датафрейм

Как вы уже успели заметить, формат CSV является достаточно гибким. Многие файлы, о которых вы даже не могли подумать как о CSV, могут быть с легкостью загружены в *pandas* именно в этом формате с помощью функции `read_csv` благодаря множеству доступных параметров.

В этом упражнении мы загрузим в виде CSV файл, который никак не ассоциируется с этим форматом, а именно файл *passwd*, присутствующий в Unix-системах и содержащий в текстовом формате список пользовательских учетных записей. С годами в стандарт этого файла были внесены изменения, и теперь он не хранит пароли в открытом виде.

Хотя операционная система MacOS основана на Unix, в ней не используется файл *passwd* для большей части аккаунтов.

Итак, что вам нужно сделать в этом упражнении.

1. Создать датафрейм на основе содержимого файла *linux-etc-passwd.txt*, присутствующего в сопроводительных материалах к книге. Обратите внимание, что в файле могут содержаться комментарии, начинающиеся с символа `#`, а также пустые строки, которые необходимо игнорировать при загрузке данных. Символом, разделяющим поля, является двоеточие (`:`).
2. Добавить следующие имена столбцов, которые в файле по понятным причинам отсутствуют: *username*, *password*, *userid*, *groupid*, *name*, *homedir* и *shell*.
3. Определить в качестве индекса в датафрейме столбец *username*.

Не волнуйтесь, если вы ничего не знаете о Unix или файле *passwd*, – в этом упражнении нашей целью будет исследование дополнительных параметров функции `read_csv`.

Подробный разбор

Для решения поставленной задачи нам нужно будет воспользоваться дополнительными параметрами функции `read_csv`, которые мы ранее не применяли. Это позволит корректно прочесть файл *passwd* и преобразовать его в датафрейм. Со временем вы поймете, что функция `read_csv` в большинстве случаев применяется с дополнительными параметрами, поскольку исходные файлы далеко не всегда пригодны для выполнения чтения из них с параметрами по умолчанию. В результате самые распространенные аргументы вы запомните довольно быстро.

Давайте внимательнее присмотримся к основным параметрам функции `read_csv` и узнаем, что они делают и как ими пользоваться. Для начала разберемся с символом-разделителем, который по умолчанию представляет запятую. Но вы можете изменить его, передав функции параметр *sep*. В нашем случае поля в файле разделены двоеточиями, а значит, нам необходимо снабдить вызов функции параметром `sep=':'`.

Далее нужно учесть, что в нашем файле с учетными записями есть комментарии, начинающиеся с символа решетки (`#`). Немногие в файле *passwd* пишут комментарии, но раз это возможно, нужно это учесть. Таким образом, все содержимое, начинающееся с этого символа и завершающееся окончанием строки, нам нужно игнорировать. В функции `read_csv` для обработки комментариев

предусмотрено элегантное решение в виде параметра `comment`. Мы можем передать ему символ, и при разборе файла строки, начинающиеся с этого символа, будут игнорироваться.

Следующим аргументом, который нам понадобится, является `header`. По умолчанию функция `read_csv` считает, что в первой строке в файле содержатся заголовки столбцов. Также первая строка используется для определения того, сколько полей будет в каждой строке файла. Если в вашем файле присутствуют заголовки столбцов, но располагаются они не в первой строке, вы можете задать параметру `header` целочисленное значение, соответствующее номеру строки, в которой их следует искать. Но файл `/etc/passwd` не является файлом CSV в чистом виде, и в нем нет никаких заголовков. Для таких случаев вы можете передать при вызове функции `read_csv` параметр `header=None`.

А как насчет пустых строк? Здесь нам ничего делать не надо, поскольку функция `read_csv` по умолчанию игнорирует все пустые строки в файле. Если же вы хотите, чтобы пустые строки воспринимались как значения `NaN`, передайте функции параметр `skip_blank_lines=False` вместо установки значения `True` по умолчанию.

Имена столбцам можно дать при помощи аргумента `names`. Если этого не сделать, столбцы будут помечены целыми числами начиная с нуля. В этом нет ничего плохого, но работать с такими данными может быть непросто. Передать имена столбцов можно при помощи списка. На рис. 3.10 видно, что мы воспользовались именами столбцов, указанными в условии задачи.

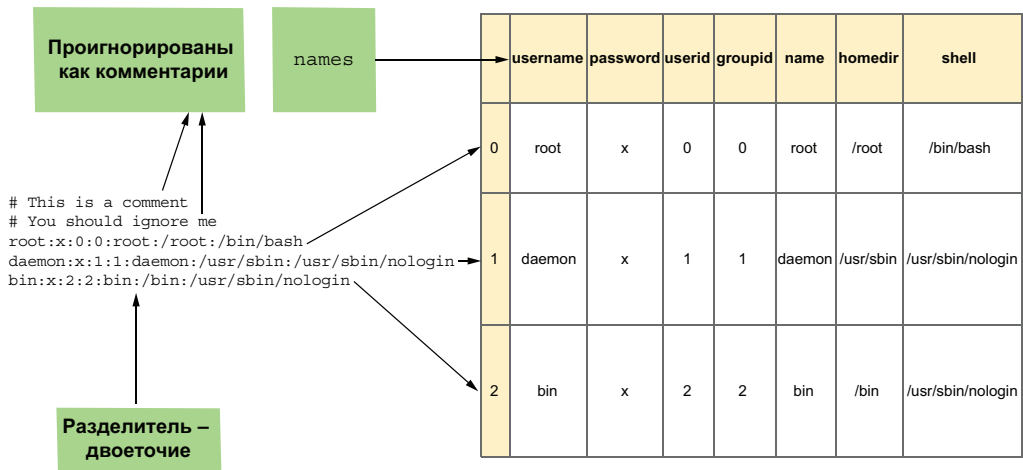


Рис. 3.10. Превращение файла `passwd` в датафрейм

Передав все необходимые параметры функции `read_csv`, можно легко преобразовать наш файл `passwd` в датафрейм. Надеюсь, вы поняли, насколько гибким на самом деле является формат CSV.

Символы-разделители и регулярные выражения

Меня часто спрашивают, можно ли в качестве разделителей при чтении файла использовать сразу несколько символов. К примеру, поля в файле могут быть разделены как при помощи двоеточия, так и посредством запятой. Что делать в этом случае?

У pandas на этот счет предусмотрено очень элегантное решение. Если в параметре `sep` содержится более одного символа, его значение рассматривается как регулярное выражение. Так что, если вы хотите использовать в качестве разделителя как двоеточие, так и запятую, можете передать параметру `sep` значение `[:, ]`. Это очень удобно, если вы знаете регулярные выражения. В противном случае я очень рекомендую вам их изучить. Регулярные выражения – просто незаменимый инструмент для тех, кто работает с текстом, а значит, для всех программистов. В интернете есть масса учебных пособий по регулярным выражениям.

Обычно pandas осуществляет разбор файлов CSV с использованием библиотеки, написанной на языке C. Но если в качестве параметра с разделителями вы передали регулярное выражение, разбор будет выполняться силами Python, что может замедлить процесс и увеличить расход памяти. Так что стоит лишний раз подумать, стоит ли вам использовать такой расширенный функционал.

Последним требованием мы указали использование в качестве индекса в датафрейме столбца `username`. Это можно легко сделать, передав при вызове функции `read_csv` параметр `index_col` и присвоив ему имя нужного столбца.

Решение

```
df = pd.read_csv('../data/linux-etc-passwd.txt',
                 sep=':', comment='#', header=None, index_col='username',
                 names=['username', 'password', 'userid', 'groupid',
                       'name', 'homedir', 'shell'])
df
```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/G9lv>.

Дополнительные упражнения

Теперь, когда вы знаете, насколько сильно могут преобразиться внешне непохожие на CSV файлы за счет использования дополнительных параметров функции `read_csv`, можно еще немного поупражняться, чтобы набить руку.

1. При чтении данных проигнорируйте столбцы с именами `password` и `groupid`, чтобы они не загружались в итоговый датафрейм.
2. В Unix-системах идентификаторы ниже 1000 обычно резервируются для служебных учетных записей. Оставьте в нашем датафрейме только строки с идентификаторами не ниже 1000.
3. Сразу после входа в Unix-систему запускается интерпретатор команд, также известный как *shell*. Сколько разных интерпретаторов команд есть в нашем файле?

Ответы на дополнительные упражнения

Упражнение 18.1

```
# Обратите внимание, что мы дали имена всем столбцам, а выбрали только часть из них
df = pd.read_csv('../data/linux-etc-passwd.txt',
                 sep=':', comment='#', header=None,
                 usecols=['username', 'userid', 'name', 'homedir', 'shell'],
                 names=['username', 'password', 'userid', 'groupid', 'name',
                       'homedir', 'shell'])
df
```

Вывод:

	username	userid	name	homedir	shell
0	root	0	root	/root	/bin/bash
1	daemon	1	daemon	/usr/sbin	/usr/sbin/nologin
2	bin	2	bin	/bin	/usr/sbin/nologin
3	sys	3	sys	/dev	/usr/sbin/nologin
4	sync	4	sync	/bin	/bin/sync
...					

Упражнение 18.2

```
df = pd.read_csv('../data/linux-etc-passwd.txt',
                 sep=':', comment='#', header=None,
                 names=['username', 'password', 'userid', 'groupid', 'name',
                       'homedir', 'shell'])

df[df['userid'] >= 1000]
```

Вывод:

	username	password	userid	groupid	name	homedir	shell
17	nobody	x	65534	65534	nobody	/nonexistent	/usr/sbin/nologin
23	user	x	1000	1000	user,,	/home/user	/bin/bash
24	reuven	x	1001	1001	Reuven M. Lerner,,	/home/reuven	/bin/bash
33	genadi	x	1002	1003	Genadi Reznichenko,,	/home/genadi	/bin/bash
34	shira	x	1003	1004	Shira Friedman,,	/home/shira	/bin/bash
...							

Упражнение 18.3

Можно воспользоваться методом unique объекта Series, который вернет массив
 # NumPy, но в pandas есть свой метод drop_duplicates, возвращающий объект Series

```
df['shell'].drop_duplicates()
```

Вывод:

```
0      /bin/bash
1    /usr/sbin/nologin
4      /bin/sync
18     /bin/false
```

```

31         /bin/sh
42         /bin/nologin
Name: shell, dtype: object

```

УПРАЖНЕНИЕ 19. Курсы биткоина

Обычно когда мы говорим о файлах CSV, то подразумеваем данные, которые однажды были загружены и теперь их нужно проанализировать. Но в наше время существует огромное количество автоматизированных систем, публикующих данные с определенной периодичностью, и зачастую также в удобном формате CSV. Неудивительно, что первым аргументом в функцию `read_csv` можно передать не только путь к файлу на диске, но и другие объекты, такие как:

- файловые объекты, открытые для чтения, обычно путем вызова функции `open`, но это могут быть также и объекты `StringIO`;
- объекты `Path`, представляющие собой экземпляры класса `pathlib.Path`;
- строки с адресами в интернете.

Последний пункт представляет наибольший интерес, и в этом упражнении мы поговорим именно о нем. Итак, функция `read_csv` может принимать на вход ссылку в интернете, и в случае, если эта ссылка возвращает файл CSV, `pandas` создаст на его основе новый датафрейм. Все остальные параметры функции `read_csv` остаются прежними, отличие состоит лишь в том, что мы используем вместо пути к файлу на диске ссылку на внешний ресурс.

Почему такой вариант использования функции `read_csv` может быть столь удобен? Дело в том, что существует масса служб, ежечасно, ежеминутно или с любой иной периодичностью публикующих полезные сведения по определенным адресам, которые не меняются. При обращении по одному из таких адресов мы можем получить доступ к некой актуальной и очень важной информации. Таким образом, благодаря возможности непосредственно из `pandas` обращаться к внешним ресурсам за ценной информацией мы можем заложить в свои приложения механизм опроса нужных нам служб и анализа полученных данных, меняющихся с определенной периодичностью.

Использование пакета `requests`

Зачастую файлы CSV публикуются на страницах, доступ к которым можно получить только при помощи логина и пароля. Некоторые службы допускают отправку учетных данных для подключения непосредственно в ссылке. К службам, не допускающим таких неосмотрительных вольностей, подключиться с помощью функции `read_csv`, увы, не удастся. Вместо этого вам придется специально подключаться к ресурсу при помощи, например, пакета `requests`, а затем создавать объект `StringIO` с содержимым нужного нам файла.

Сделать это вы можете, к примеру, так:

```

import requests
from io import StringIO

```

```

r = requests.get('https://data_for_you.com/data.csv') ❶
s = StringIO(r.content.decode()) ❷
df = pd.read_csv(s) ❸

```

- ❶ Ссылка на файл.
- ❷ Преобразуем содержимое файла в строку и используем ее для создания объекта StringIO.
- ❸ Подаем объект StringIO на вход функции read_csv для получения датафрейма.

В этом упражнении вам нужно будет извлечь даты и курсы биткоина за последний год. Поскольку эти данные постоянно меняются, ваши результаты не будут совпадать с моими, даже если мы будем использовать один и тот же код. После извлечения данных вы должны будете ответить на следующие вопросы:

- какова была цена закрытия биткоина в последний торговый день?
- в какую дату наблюдался минимальный курс биткоина за весь исследуемый период и каким был этот курс?
- в какую дату наблюдался максимальный курс биткоина и каким он был?

На момент написания книги файл с историческими данными о курсах биткоина располагается по ссылке <https://api.blockchain.info/charts/market-price?format=csv>.

ПРИМЕЧАНИЕ. Многие сайты, хранящие биржевые и финансовые сведения, требуют авторизации доступа, но ресурс api.blockchain.info на момент написания книги не предусматривал процедуру авторизации.

Подробный разбор

Что меня всегда впечатляло в функции `pd.read_csv` – так это то, с какой легкостью она позволяет извлекать файлы CSV из интернета. За исключением того, что исходный файл хранится не локально, а формируется удаленно, никакой разницы между двумя вызовами этой функции не существует. В частности, мы так же, как и в случае с файлом на диске, можем попросить извлечь только нужные нам столбцы с помощью параметра `usecols`.

Итак, давайте попробуем загрузить удаленный файл CSV. Мы знаем, что в нем есть два столбца и отсутствуют заголовки, так что воспользуемся параметром `names` и опцией `header=None`, как показано ниже:

```

df = pd.read_csv('https://api.blockchain.info/charts/market-
price?format=csv',
                 header=None,
                 names=['date', 'value'])

```

После создания датафрейма нам необходимо ответить на первый вопрос упражнения, касающийся цены закрытия биткоина в последний торговый день. С учетом того что эти файлы формируются ежедневно, нам необходимо выработать единую стратегию извлечения актуальной информации из них. Беглый анализ данных с помощью методов `head` и `tail` позволил понять, что сведения в

файле располагаются строго в хронологическом порядке по возрастанию даты. А значит, мы можем легко извлечь данные за последний день с помощью конструкции `df.tail(1)`. Если запускать наш скрипт каждый день, он будет все время возвращать актуальные данные.

Также нам нужно получить из последней строки в данных только значение столбца `value`. Как можно это сделать? Достаточно вспомнить, что вызов метода `df.tail(1)` вернет нам новый датафрейм, в котором мы можем запросто обратиться к конкретному столбцу.

Вам нужно только значение?

Вызов `df.tail(1)` возвращает последнюю строку из датафрейма `df` в виде нового датафрейма, который, так же как и исходный, будет содержать столбцы `date` и `value`. А что, если нам нужен только столбец `value`?

Можно думать о возвращенном объекте как о датафрейме, состоящем из одной строки. Каждый столбец в датафрейме, как мы помним, представляет собой объект `Series`, и мы можем получить нужный нам столбец так:

```
df.tail(1)['value']
```

В результате мы получим объект `Series` с единственным значением. Но помните, что нам может понадобиться извлечь более одного столбца из датафрейма путем передачи списка имен столбцов, задействовав при этом двойные квадратные скобки. А что будет, если использовать двойные скобки с одним столбцом, как показано ниже?

```
df.tail(1)[['value']]
```

Результатом будет датафрейм, содержащий одну строку (ту же, что и при вызове `df.tail(1)`) и один столбец `value`.

Выбор синтаксиса зависит от того, что вы собираетесь делать с полученными данными. В нашем случае это не имеет значения.

Далее нас попросили узнать, каким был минимальный и максимальный курс биткойна за исследуемый период и на какие даты выпадали эти экстремумы. Мы можем воспользоваться булевым индексом для поиска строк – или, что более вероятно, одной строки – с минимальной ценой закрытия. Вторым аргументом атрибута `loc` позволяет выбрать нужные столбцы из найденной строки. Обратите внимание, что мы сначала должны найти минимальное значение по столбцу `value`, а затем отобрать все строки с таким значением. В результате мы найдем строку с минимальным курсом. В теории мы можем получить и более одной строки. В этом случае мы выведем их все. То же самое мы сделаем и с поиском максимума. На рис. 3.11 показана схема выполнения этой процедуры.

```
df.loc[df['value'] == df['value'].min(), ['date', 'value']]
df.loc[df['value'] == df['value'].max(), ['date', 'value']]
```

Если бы нам было достаточно получить только дату, мы могли бы применить более элегантный подход, сделав столбец `date` индексом и вызвав метод `idxmin` или `idxmax` применительно к новому датафрейму, как показано ниже:

```
df.set_index('date').idxmin()
df.set_index('date').idxmax()
```

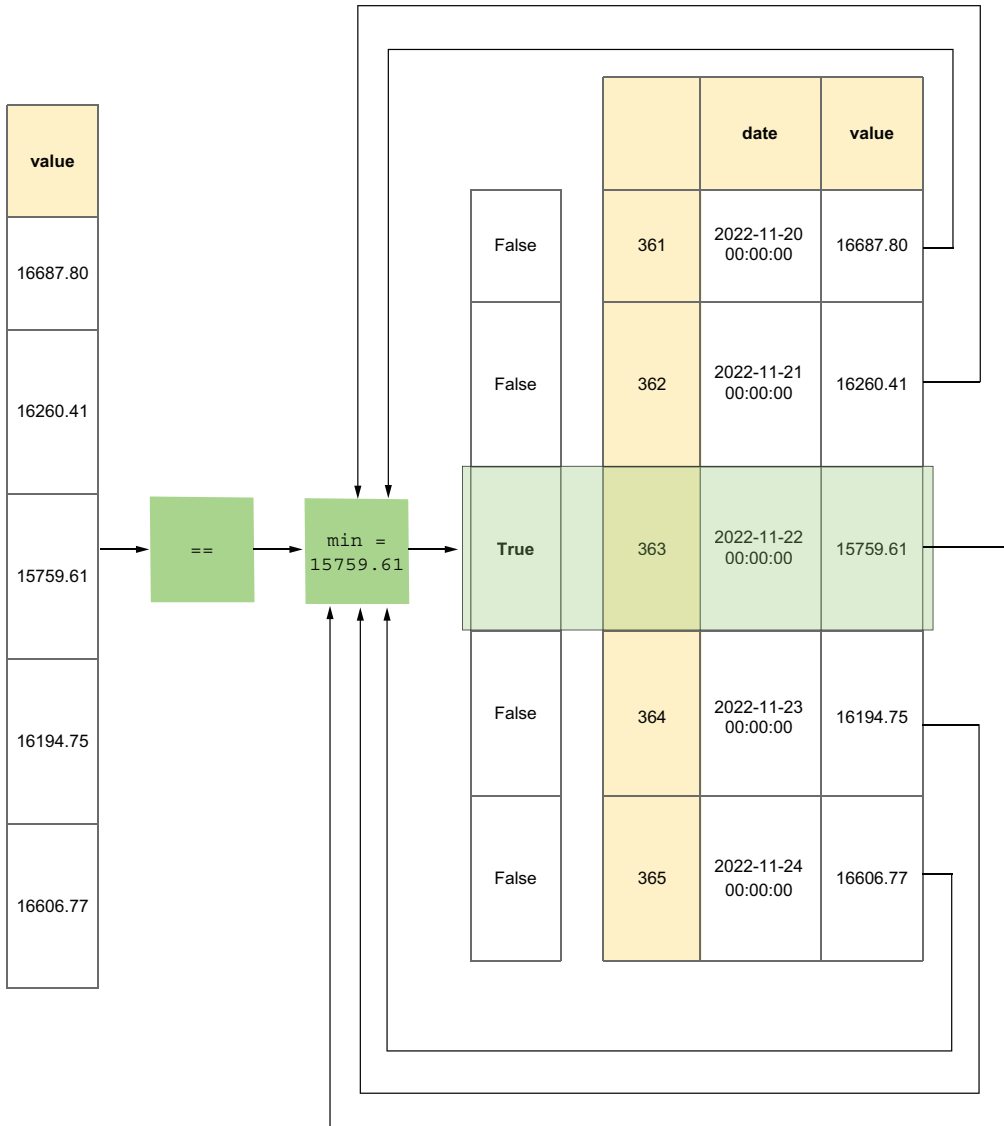


Рис. 3.11. Выбор минимального курса биткоина с помощью индекса-маски

Также мы можем применить метод `agg` к датафрейму для вычисления более одной агрегации, передав методы агрегирования как список строк. Так мы сможем одновременно получить дату с минимальным курсом и максимальным, что показано ниже:

```
df.set_index('date').agg(['idxmin', 'idxmax'])
```

Решение

```
import pandas as pd
from pandas import Series, DataFrame

df = pd.read_csv('https://api.blockchain.info/charts/market-
price?format=csv',
                 header=None,
                 names=['date', 'value']) ❶

df.tail(1)[['value']] ❷

df.loc[df['value'] == df['value'].min(), ['date', 'value']]
df.loc[df['value'] == df['value'].max(), ['date', 'value']]

df.set_index('date').idxmin()
df.set_index('date').idxmax()

df.set_index('date').agg(['idxmin', 'idxmax']) ❸
```

- ❶ Именуем столбцы как date и value.
- ❷ Извлекаем значение колонки value из последней строки датафрейма.
- ❸ Устанавливаем столбец date в качестве индекса и находим строки с минимальным и максимальным значением в столбце value.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/YRXB>.

Дополнительные упражнения

- В этом упражнении вы сначала загрузили данные в датафрейм, а затем произвели над ними вычисления. А сможете вернуть актуальное значение курса биткоина без сохранения данных в промежуточную переменную? Ваше решение должно уместиться в одну строку кода, в которой вы загрузите данные, осуществите выборку и выполните вычисления.
- Функция `pd.read_html` во многом похожа на функцию `pd.read_csv` в том отношении, что на вход она принимает путь на диске или ссылку на ресурс. Эта функция предполагает, что переданный путь будет содержать текст, отформатированный как HTML, как минимум с одной таблицей. Она преобразовывает каждую таблицу в датафрейм и возвращает их список. Извлеките исторические данные за один год с Yahoo Finance по индексу S&P 500 (<https://finance.yahoo.com/quote/%5EGSPC/history/?p=%5EGSPC>), только столбцы Date, Close и Volume. Выведите даты (Date) и объемы (Volume) для дней с минимальным и максимальным значением в столбце Close. Обратите внимание на то, что Yahoo, судя по всему, считывает заголовок User-Agent из запроса HTTP, и он не должен быть установлен в значение `read_html`. Так что вам придется воспользоваться библиотекой `requests` для извлечения данных, установив для заголовка User-Agent строковое значение 'Mozilla 5.0'. Преобразуйте полученный результат в объект `StringIO`, передайте его функции `read_html` и извлеките данные.

- Создайте датафрейм, состоящий из двух строк для максимальной и минимальной цены закрытия индекса S&P 500. Воспользуйтесь функцией `to_csv` для записи данных в новый файл CSV.

Ответы на дополнительные упражнения

Упражнение 19.1

```
pd.read_csv('https://api.blockchain.info/charts/market-price?format=csv',
            header=None,
            names=['date', 'value']).tail(1)['value']
```

Вывод:

```
365      36497.35
Name: value, dtype: float64
```

Упражнение 19.2

```
import requests
from io import StringIO
```

```
r = requests.get('https://finance.yahoo.com/quote/%5EGSPC/
history?p=%5EGSPC',
                headers={'User-Agent': 'Mozilla/5.0'})
```

```
df = pd.read_html(StringIO(r.content.decode()))[0].set_index('Date').
iloc[:-1]
df = df.rename(columns={'Close Close price adjusted for splits.': 'Close*',
'Adj Close Adjusted close price adjusted for splits and dividend and/or
capital gain distributions.': 'Adj**', })
df['Close*'] = df['Close*'].astype(np.float64)
```

df

Вывод:

	Open	High	Low	Close*	Adj Close**	Volume
Date						
Sep 20, 2024	5709.64	5715.14	5674.49	5702.55	5702.55	7867260000
Sep 19, 2024	5702.63	5733.57	5686.42	5713.64	5713.64	4024530000
Sep 18, 2024	5641.68	5689.75	5615.08	5618.26	5618.26	3691390000
Sep 17, 2024	5655.51	5670.81	5614.05	5634.58	5634.58	3443600000
Sep 16, 2024	5615.21	5636.05	5604.53	5633.09	5633.09	3437070000
...	...	...	...	...	...	...
Sep 28, 2023	4269.65	4317.27	4264.38	4299.70	4299.70	3846230000
Sep 27, 2023	4282.63	4292.07	4238.63	4274.51	4274.51	3875880000
Sep 26, 2023	4312.88	4313.01	4265.98	4273.53	4273.53	3472340000
Sep 25, 2023	4310.62	4338.51	4302.70	4337.44	4337.44	3195650000
Sep 22, 2023	4341.74	4357.40	4316.49	4320.06	4320.06	3349570000

[251 rows x 6 columns]

Упражнение 19.3

```
print(df.loc[df['Close*']].agg(['idxmin', 'idxmax']), 'Close*'].to_csv())
```

Вывод:

```
Date,Close*
"Oct 27, 2023", 4117.37
"Jul 31, 2023", 4588.96
```

УПРАЖНЕНИЕ 20. Большие города

Формат CSV, без сомнений, является одним из наиболее популярным форматах обмена данными. Но ему на пятки уверенно наступает формат *JSON* (сокращенно от JavaScript Object Notation). Этот формат позволяет хранить числа, текст, списки и словари в текстовом формате, и сегодня с ним работает большинство приложений для анализа данных. Легкость работы, меньший размер в сравнении с XML и лучшая выразительность, чем у CSV, сделали JSON одним из основных форматов для хранения и передачи данных. Кроме того, JSON фактически стал стандартом обмена данными с API, что позволяет использовать его для доступа ко всему разнообразию сведений, располагающихся в интернете, вне зависимости от платформ.

Подобно тому как мы можем извлекать содержимое файлов в формате CSV с помощью функции `pd.read_csv`, мы можем обращаться к данным в формате JSON посредством функции `pd.read_json`. В этом упражнении мы будем извлекать данные о 1000 самых крупных городов в США из файла с именем `cities.json` (источнику данных уже больше десяти лет, так что на актуальность он не претендует). После создания датафрейма на основе этих данных вам нужно будет ответить на следующие вопросы:

- какова средняя и медианная численность населения в 1000 крупнейших городов США? О чем вам это говорит?
- что произойдет со средней и медианной численностью населения при удалении из списка городов для анализа 50 наиболее густонаселенных?
- какой город в этом списке является самым северным и какое место по численности населения он занимает?
- какой штат представлен в этом списке наиболее широко?
- в каком штате меньше всего крупных городов?

Подробный разбор

Чтение данных из файла JSON в датафрейм не представляет труда. Частично это объясняется тем, что формат JSON являет собой список объектов, а на языке Python – список словарей. При чтении файла функция `read_json` представляет каждый словарь в виде отдельной записи, используя ключ словаря в качестве имени столбца. Фактически этот процесс сильно напоминает процедуру создания датафрейма из списка словарей, которую мы видели в главе 2. После создания датафрейма мы можем работать с ним как с любым другим.

В первом задании этого упражнения нам просто нужно вычислить среднюю и медианную численность населения в представленных городах. Это можно сделать при помощи метода `describe` в столбце `population`, который вернет нам объект `Series`. Поскольку нас интересует только два показателя, мы можем ограничить вывод только ими, как показано ниже:

```
df['population'].describe()[['mean', '50%']]
```

Средняя численность населения составила 131 132, а медианная – 68 207. Это означает, что несколько густонаселенных городов перетянули среднее значение вправо, не повлияв при этом на медиану. И действительно, в США есть лишь несколько действительно крупных городов, а мелких и средних – огромное множество. Как видим, ровно половина из анализируемой тысячи городов имеет население менее 68 207 человек.

Ответ на следующий вопрос требует исключения 50 наиболее крупных городов из списка анализируемых. Для этого мы применим срез совместно с атрибутом `loc` и снова вычислим среднее значение и медиану численности населения:

```
df.loc[50:, 'population'].describe()[['mean', '50%']]
```

Помните, что, когда мы передаем атрибуту `loc` два параметра, мы тем самым ограничиваем выборку сначала по строкам, а затем по столбцам. В данном случае мы сказали, что нам нужны все строки, начиная с индекса 50, и только один столбец `population`. Далее мы обращаемся к методу `describe` и извлекаем из результата только среднее значение и медиану. Как видим, среднее значение сильно уменьшилось – до 87 027, тогда как медианное почти осталось на месте и составило 65 796. Это демонстрирует устойчивость медианы, которая гораздо меньше подвержена изменениям при появлении очень больших или очень маленьких значений в наборе данных.

Теперь нам необходимо найти самый северный город в списке. Это значит, что нам нужно определить город с наибольшим значением широты (столбец `latitude`). Для этого нужно найти максимальное значение в этом столбце и определить, каким строкам оно соответствует. Мы снова воспользуемся атрибутом `loc`, ограничив вывод только нужными нам столбцами:

```
df.loc[df['latitude'] == df['latitude'].max(), ['city', 'state', 'rank']]
```

Результат меня не удивил – самым северным городом в этом списке является Анкоридж (штат Аляска). Он занимает 63-е место в списке самых населенных городов США, и вот это стало для меня сюрпризом!

Наконец, нам необходимо найти штаты с наибольшим и наименьшим количеством городов в представленном списке. Для этого можно воспользоваться методом `value_counts` применительно к столбцу `state`. Калифорния выиграла в этой гонке с большим отрывом – в этом штате представлено сразу 212 городов:

```
df['state'].value_counts().head(1)
```

Помните, что по умолчанию метод `value_counts` сортирует результаты по убыванию, что позволяет нам говорить о том, что обращение к первой строке (`head(1)`) позволит получить штат с самым большим количеством густонаселенных городов

или один из таких штатов, если количество городов в нескольких штатах окажется одинаковым.

А как насчет штата с наименьшим количеством городов из списка? Мы взяли последние десять штатов в списке с помощью метода `tail(10)` и увидели, что в пяти штатах располагается всего по одному городу из нашего перечня, включая Вашингтон:

```
df['state'].value_counts().tail(5)
```

Решение

```
filename = '../data/cities.json'
df = pd.read_json(filename)
```

```
df['population'].describe()[['mean', '50%']] ❶
df.loc[50:, 'population'].describe()[['mean', '50%']] ❷
df.loc[df['latitude'] == df['latitude'].max(), ['city', 'state', 'rank']] ❸
df['state'].value_counts().head(1) ❹
df['state'].value_counts().tail(5) ❺
```

- ❶ Забираем только данные о среднем значении и медиане из описательной статистики.
- ❷ Забираем данные о среднем значении и медиане для строк с индексом 50 и выше.
- ❸ Находим город с максимальной широтой, т.е. самый северный.
- ❹ Один штат с самым большим количеством городов в списке.
- ❺ Сразу пять штатов содержат лишь один город из списка.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/z0oB>.

Дополнительные упражнения

1. Приведите столбец `growth_from_2000_to_2013` к типу с плавающей точкой. Найдите среднее и медианное значения изменения численности населения в городах с 2000 по 2013 год. Если численность населения в городе не изменилась, установите ее в ноль.
2. В скольких городах из нашего набора данных численность населения в период с 2000 по 2013 год выросла, а в скольких снизилась? Попробуйте решить это задание с помощью метода `pd.cut`.
3. Найдите город или города, значение широты которых отстоит от среднего показателя широты более чем на два стандартных отклонения.

Ответы на дополнительные упражнения

Упражнение 20.1

```
# Удаляем замыкающий символ %
df['growth_from_2000_to_2013'] = df['growth_from_2000_to_2013'].str.rstrip('%')

# Находим пустые строки и ставим в них ноль
df.loc[df['growth_from_2000_to_2013'] == '', 'growth_from_2000_to_2013'] = '0'
```

```
# Приводим к типу float и возвращаем среднее значение и медиану
df['growth_from_2000_to_2013'].astype(float).describe()[['mean', '50%']]
```

Вывод:

```
mean      22.936
50%       9.650
Name: growth_from_2000_to_2013, dtype: float64
```

Упражнение 20.2

```
pd.cut(df['growth_from_2000_to_2013'],
       bins=[df['growth_from_2000_to_2013'].min(), 0, df['growth_
from_2000_to_2013'].max()],
       include_lowest=True,
       labels=['-', '+']).value_counts()
```

Вывод:

```
growth_from_2000_to_2013
+      852
-     148
Name: count, dtype: int64
```

Упражнение 20.3

```
df[(df['latitude'] > df['latitude'].mean() + 2*df['latitude'].std()) |
   (df['latitude'] < df['latitude'].mean() - 2*df['latitude'].std()) ]
```

Вывод:

	city	growth_from_2000_to_2013	latitude	longitude	population	rank	state
43	Miami	14.9%	25.761680	-80.191790	417650	44	Florida
53	Honolulu	-6.2%	21.306944	-157.858333	347884	54	Hawaii
62	Anchorage	15.4%	61.218056	-149.900278	300950	63	Alaska
..	...	...	...	...	...	...	...
956	Hallandale Beach	12.4%	25.981202	-80.148379	38632	957	Florida
990	Aventura	47.2%	25.956481	-80.139212	37199	991	Florida
995	Weslaco	28.8%	26.159519	-97.990837	37093	996	Texas

[52 rows x 7 columns]

Заключение

В этой главе мы начали работать с реальными данными. Мы извлекли нужную нам информацию из файлов CSV и JSON, а также из таблиц HTML и познакомились с самыми полезными параметрами в pandas, с помощью которых можно управлять процедурой загрузки и представления данных. С учетом того что большинство данных из интернета мы получаем именно в таком виде, очень важно уметь искусно преобразовывать их на лету и представлять в нужном виде, устанавливать типы данных для столбцов и выбирать только те столбцы, которые вам нужны.

Глава 4

Индексы

Со всеми прелестями публичной библиотеки я познакомился в очень раннем возрасте – меня туда привели родители. Там было невероятное количество книг по всем темам, которые только можно себе вообразить.

Но в том же и проблема таких огромных хранилищ книг. Как можно найти в них именно ту книгу, которая тебе нужна, или даже просто узнать, какие книги есть в наличии? Ответ прост – по (алфавитному) *индексу*. В те времена библиотеки, как правило, обладали тремя разными индексами в виде библиотечной картотеки, представляющей собой шкафчики с сотнями карточек, соответствующих книгам. Вы могли найти книгу по (а) автору, (b) названию или (с) теме. Помимо этого, сами книги были расставлены на полках в соответствии тематикой – либо с использованием десятичной классификации Дьюи, либо по системе Библиотеки Конгресса. Если вы были знакомы с этими системами, то могли легко найти книгу, которая вас интересует, – будь то конкретная книга, упомянутая в газете, сочинения вашего любимого автора или книги по интересующему вас школьному предмету. Разумеется, сегодня все эти индексы собраны в компьютерах, что позволяет выполнять более гибкий поиск и легче и быстрее находить нужные вам книги.

Возможно ли наличие библиотек без индексов? Да, но их польза будет сведена практически к нулю. Вам было бы намного сложнее найти нужную вам книгу, и поиск мог бы продолжаться бесконечно долго. Принципы создания каталогов настолько важны, что их изучение было выделено в отдельную дисциплину, именуемую библиотечным делом, или библиотековедением.

Подобно тому как каталоги помогают нам искать книги в библиотеке, индексы позволяют находить нужные данные в *pandas*. Мы уже видели, что объекты *Series* обладают одним индексом, идентифицирующим значения, а датафреймы – двумя (один для строк и один для столбцов). Также мы успели ощутить всю мощь атрибута *loc*, используемого совместно с селекторами строк и столбцов.

Но индексы в *pandas* представляют собой гораздо более гибкий механизм в сравнении с тем, что мы уже видели. Вы можете превратить существующий столбец в индекс, а индекс – обратно в столбец. Также вы можете комбинировать разные столбцы при построении иерархических *множественных индексов* (*multi-index*) и затем осуществлять поиск по определенным составляющим иерархии. В действительности в умении полноценно работать с множественными индексами кроется секрет гибкого использования библиотеки *pandas* при анализе данных. Кроме того, с помощью индексов вы также можете создавать *сводные таблицы* (*pivot table*), в которых в качестве строк или столбцов могут быть представлены не исходные данные, а какие-то агрегированные сведения на их основе.

В этой главе мы научимся использовать все эти техники, и в результате вы сможете создавать, изменять и использовать разные типы индексов. Прочитав эту главу и выполнив все упражнения, включая дополнительные, вы сможете без труда создавать индексы в pandas и очень гибко извлекать нужные вам данные с их помощью.

В табл. 4.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 4.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
pd.set_index	Возвращает новый датафрейм с новым индексом	<code>df = df.set_index('name')</code>	http://mng.bz/MBd2 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.set_index.html)
pd.reset_index	Возвращает новый датафрейм с индексом по умолчанию (числовым, порядковым)	<code>df = df.reset_index()</code>	http://mng.bz/a1RJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.reset_index.html)
df.loc	Извлекает выбранные строки и столбцы	<code>df.loc[:, 'passenger_count'] = df['passenger_count']</code>	http://mng.bz/e1QJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html)
s.value_counts	Возвращает отсортированный (в порядке убывающей частоты) объект Series с информацией о том, сколько раз каждое значение встречается в переменной s	<code>s.value_counts()</code>	http://mng.bz/Y1r7 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.value_counts.html)
s.isin	Возвращает объект Series с булевыми значениями, показывающими, присутствует ли элемент последовательности в переданном в качестве аргумента списке	<code>s.isin(['A', 'B', 'C'])</code>	http://mng.bz/9D08 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isin.html)
df.pivot	Создает сводную таблицу на основе датафрейма <i>без агрегации</i>	<code>df.pivot(index='month', columns='year', values='A')</code>	http://mng.bz/zXjZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pivot.html)
df.pivot_table	Создает сводную таблицу на основе датафрейма <i>с агрегацией</i> , если это нужно	<code>df.pivot_table(index='month', columns='year', values='A')</code>	http://mng.bz/0K4z (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pivot_table.html)

Таблица 4.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>s.is_monotonic_increasing</code>	Содержит True, если значения в объекте Series отсортированы по возрастанию	<code>s.is_monotonic_increasing</code>	http://mng.bz/Ke2n (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.is_monotonic_increasing.html)
<code>Slice</code>	Встроенный объект Python для создания срезов	<code>slice(10, 20, 2)</code>	http://mng.bz/278g (https://docs.python.org/3/library/functions.html#slice)
<code>df.xs</code>	Возвращает поперечный срез на основе датафрейма	<code>df.xs(2016, level='Year')</code>	http://mng.bz/jPg9 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.xs.html)
<code>df.dropna</code>	Возвращает новый датафрейм с удаленными строками, содержащими значения NaN	<code>df.dropna()</code>	http://mng.bz/o1PN (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html)
<code>IndexSlice</code>	Производит объект, облегчающий запросы к датафреймам с использованием срезов	<code>IndexSlice[:, 2016]</code>	http://mng.bz/WzPX (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.IndexSlice.html)

УПРАЖНЕНИЕ 21. Парковочные талоны

Мы уже рассмотрели множество способов извлечения одной или нескольких строк из датафреймов с помощью атрибута `loc`. Нам совершенно не обязательно использовать индекс для выбора строк из датафрейма, но это делает процесс более понятным, а код – легко читаемым. По этой причине мы часто стремимся использовать один из существующих столбцов датафрейма в качестве индекса. При этом иногда мы оставляем это изменение на постоянной основе, а иногда вносим его для ответа на какой-то интересующий нас вопрос.

В этом упражнении я попрошу вас поработать с еще одним набором данных из социальной жизни Нью-Йорка, в котором представлена информация обо всех штрафных парковочных талонах за 2020 год, коих было выдано более 12 млн. В теории вы могли бы ответить на поставленные вопросы и без внесения изменений в существующий индекс. Но я хочу, чтобы вы потренировались в создании и изменении индексов. В этой главе мы будем делать это достаточно часто, и вскоре вы поймете, как можно эффективно применять индексы в реальных примерах.

Итак, это задание будет состоять из следующих пунктов.

1. Создайте датафрейм на основе файла `nyc-parking-violations-2020.csv`. Нам будут интересны лишь следующие столбцы:
 - Date First Observed;
 - Plate ID;

- Registration State;
 - Issue Date (строка в формате MM/DD/YYYY, за которой всегда следует время 12:00:00 AM);
 - Vehicle Make;
 - Street Name;
 - Vehicle Color.
2. Установите в качестве индекса столбец Issue Date (дата выдачи талона).
 3. Узнайте, какие три марки машины (Vehicle Make) чаще других получали талоны 2 января 2020 года.
 4. Определите пять улиц (Street Name), на которых водители чаще всего получали парковочные талоны 1 июня 2020 года.
 5. Установите в качестве индекса столбец Vehicle Color (цвет машины).
 6. Определите самую популярную марку машины, если брать в расчет только красный и синий цвет.

Подробный разбор

Мы уже видели, что для извлечения из датафрейма строк, удовлетворяющих определенным условиям, можно воспользоваться булевым индексом, или индексом-маской. Но часто, особенно если мы ищем конкретные значения в столбце, бывает удобнее и правильно преобразовать этот столбец в индекс, что позволит сделать код более понятным и лаконичным. В pandas вы легко можете сделать это с помощью метода `set_index`. В этом упражнении вам необходимо будет ответить сразу на несколько вопросов, для чего нужно будет устанавливать в качестве индекса в датафрейме разные столбцы.

Для начала прочитаем данные из файла CSV, ограничив датафрейм только нужными нам столбцами:

```
filename = '../data/nyc-parking-violations-2020.csv'

df = pd.read_csv(filename,
                  usecols=[
                      'Date First Observed',
                      'Registration State', 'Plate ID',
                      'Issue Date', 'Vehicle Make',
                      'Street Name', 'Vehicle Color'])
```

После создания датафрейма нам нужно будет ответить на несколько вопросов, связанных с выдачей штрафных парковочных талонов в Нью-Йорке, на основе дат их выдачи. Для этого удобно будет установить в качестве индекса столбец Issue Date, как показано на рис. 4.1:

```
df = df.set_index('Issue Date')
```

Обратите внимание, что метод `set_index` возвращает новый датафрейм на основе исходного, который мы присвоили обратно переменной `df`. Теперь если мы будем запрашивать данные из нашего датафрейма с помощью атрибута `loc`, то наши запросы будут обращаться напрямую к полю Issue Date. По сути, при пре-

образовании в индекс этот столбец прекращает свое существование в виде обычного именованного столбца в датафрейме. Некоторые методы в pandas, такие как `groupby`, могут находить индекс и работать с ним, обращаясь к нему по исходному имени столбца, но другие делать этого не могут.

	Plate ID	Registration State	Issue Date	Vehicle Make	Street Name	Date First Observed	Vehicle Color
725518	JFG4137	NY	07/16/2019 12:00:00 AM	DODGE	PACIFIC STREET	0	WHT
247136	DPH1199	NY	07/01/2019 12:00:00 AM	NISSA	160th St	0	BK
1628916	8D45B	NY	08/06/2019 12:00:00 AM	FORD	NB BAYCHESTER AVE @	0	YW
6757299	67974JV	NY	12/11/2019 12:00:00 AM	ISUZU	95th St	0	WHITE
4482906	JBN3055	NY	10/13/2019 12:00:00 AM	DODGE	SWINTON AVE	0	GRY
12331922	CKS1861	GA	06/17/2020 12:00:00 AM	Jeep	NB OCEAN PKWY @ AVE	0	GRAY
1723597	58388MG	NY	08/08/2019 12:00:00 AM	CHEVR	E 38th St	20190808	WH
2474539	AP628T	NJ	08/26/2019 12:00:00 AM	INTER	1st Ave	0	WHITE

```
set_index('Issue date')
```

	Plate ID	Registration State	Vehicle Make	Street Name	Date First Observed	Vehicle Color
Issue date						
07/16/2019 12:00:00 AM	JFG4137	NY	DODGE	PACIFIC STREET	0	WHT
07/01/2019 12:00:00 AM	DPH1199	NY	NISSA	160th St	0	BK
08/06/2019 12:00:00 AM	8D45B	NY	FORD	NB BAYCHESTER AVE @	0	YW
12/11/2019 12:00:00 AM	67974JV	NY	ISUZU	95th St	0	WHITE
10/13/2019 12:00:00 AM	JBN3055	NY	DODGE	SWINTON AVE	0	GRY
06/17/2020 12:00:00 AM	CKS1861	GA	Jeep	NB OCEAN PKWY @ AVE	0	GRAY
08/08/2019 12:00:00 AM	58388MG	NY	CHEVR	E 38th St	20190808	WH
08/26/2019 12:00:00 AM	AP628T	NJ	INTER	1st Ave	0	WHITE

Рис. 4.1. Схематическое изображение преобразования столбца Issue Date в индекс

ПРИМЕЧАНИЕ. На момент написания книги метод `set_index`, как и многие другие методы `pandas`, поддерживал параметр `inplace`. Если при вызове метода `set_index` передать параметр `inplace=True`, он вернет значение `None`, а все изменения будут внесены в исходный датафрейм. Разработчики библиотеки `pandas` уже не раз заявляли, что в этом параметре нет ничего полезного, а его использование связано с ложными умозаключениями по поводу использования памяти. Они отметили, что нет никаких преимуществ в использовании параметра `inplace=True`. Кроме того, получение нового датафрейма позволяет использовать цепочки методов, что значительно облегчает чтение длинных запросов. Таким образом, велика вероятность, что в будущих версиях библиотеки этот параметр будет упразднен. И даже если вам кажется расточительным процесс установки в датафрейме нового индекса с последующим возвращением нового датафрейма из метода `set_index`, вы должны использовать именно этот подход.

Теперь после установки нового индекса в датафрейме найти парковочные талоны, выданные 2 января, можно очень просто:

```
df.loc['01/02/2020 12:00:00 AM']
```

Но вызов атрибута `loc` в таком виде вернет нам все столбцы в датафрейме. А первый вопрос, на который мы должны ответить, предполагает нахождение трех марок машины (`Vehicle Make`), водители которых чаще других получали талоны 2 января 2020 года. Следовательно, мы можем ограничить вывод лишь одним этим столбцом следующим образом:

```
df.loc['01/02/2020 12:00:00 AM', 'Vehicle Make']
```

Как вы помните, обращение к аргументу `loc` с использованием двух аргументов предполагает передачу первым параметром селектора строк, а вторым – селектора столбцов. В этом случае нас интересует только один столбец `Vehicle Make`.

Но это еще не все, ведь нам нужно получить три наиболее часто встречающиеся марки в наборе данных за указанную дату. Воспользуемся для этого методом `value_counts`, как показано ниже:

```
df.loc['01/02/2020 12:00:00 AM', 'Vehicle Make'].value_counts()
```

В результате мы получим объект `Series`, в котором в качестве индекса будут выступать марки машин, а в качестве значений – их количество в итоговом датафрейме с сортировкой по убыванию. В заключение можно ограничить набор тремя первыми записями, вызвав метод `head(3)`:

```
df.loc['01/02/2020 12:00:00 AM', 'Vehicle Make'].value_counts().head(3)
```

Подобным образом мы можем извлечь информацию и из других столбцов. К примеру, чтобы ответить на второй вопрос, касающийся пяти улиц (`Street Name`), на которых водители чаще всего получали парковочные талоны 1 июня 2020 года, можно написать следующее выражение:

```
df.loc['06/01/2020 12:00:00 AM', 'Street Name'].value_counts().head(5)
```

Мы снова выбираем строки посредством индекса, а затем извлекаем нужный нам столбец. Далее вызываем метод `value_counts` и берем первые пять строк.

Далее нам необходимо ответить на вопрос, касающийся цвета машин (столбец `Vehicle Color`). Таким образом, нужно снять индекс со столбца `Issue Date` и установить в качестве индекса столбец `Vehicle Color`. Можно сделать это в две строки кода, как показано ниже:

```
df = df.reset_index()
df = df.set_index('Vehicle Color')
```

Но благодаря возможности выстраивания цепочек методов можно это реализовать и в одной строке:

```
df = df.reset_index().set_index('Vehicle Color')
```

Если вам важно выполнять шаги построчно или комментировать каждую строку кода, можете выбрать такой стиль:

```
df = (
    df
    .reset_index()
    .set_index('Vehicle Color')
)
```

Последовательность этих действий схематически показана на рис. 4.2.

Как видите, информация в нашем датафрейме осталась прежней, а индекс изменился, обеспечив удобный доступ к данным. Это поможет нам ответить на следующий вопрос, в котором спрашивается, какая марка машин получила больше всех парковочных талонов, если брать в расчет только красный и синий цвет.

Сначала нам нужно найти только синие и красные машины. Это можно сделать, передав атрибуту `loc` список:

```
df.loc[['BLUE', 'RED']]
```

После этого можно применить селектор столбцов, как показано ниже:

```
df.loc[['BLUE', 'RED'], 'Vehicle Make']
```

Это позволит выбрать только синие и красные машины из набора данных и при этом оставить только столбец с маркой автомобиля. Теперь можно воспользоваться методом `value_counts` для нахождения самых популярных марок машин и оставить только одну:

```
(
    df
    .loc[['BLUE', 'RED'], 'Vehicle Make']
    .value_counts()
    .head(1)
)
```

	Plate ID	Registration State	Vehicle Make	Street Name	Date First Observed	Vehicle Color
Issue Date						
07/16/2019 12:00:00 AM	JFG4137	NY	DODGE	PACIFIC STREET	0	WHT
07/01/2019 12:00:00 AM	DPH1199	NY	NISSA	160th St	0	BK
08/06/2019 12:00:00 AM	8D45B	NY	FORD	NB BAYCHESTER AVE @	0	YW
12/11/2019 12:00:00 AM	67974JV	NY	ISUZU	95th St	0	WHITE
10/13/2019 12:00:00 AM	JBN3055	NY	DODGE	SWINTON AVE	0	GRY
06/17/2020 12:00:00 AM	CKS1861	GA	Jeep	NB OCEAN PKWY @ AVE	0	GRAY
08/08/2019 12:00:00 AM	58388MG	NY	CHEVR	E 38th St	20190808	WH
08/26/2019 12:00:00 AM	AP628T	NJ	INTER	1st Ave	0	WHITE

```
reset_index()
```

	Plate ID	Registration State	Issue Date	Vehicle Make	Street Name	Date First Observed	Vehicle Color
725518	JFG4137	NY	07/16/2019 12:00:00 AM	DODGE	PACIFIC STREET	0	WHT
247136	DPH1199	NY	07/01/2019 12:00:00 AM	NISSA	160th St	0	BK
1628916	8D45B	NY	08/06/2019 12:00:00 AM	FORD	NB BAYCHESTER AVE @	0	YW
6757299	67974JV	NY	12/11/2019 12:00:00 AM	ISUZU	95th St	0	WHITE
4482906	JBN3055	NY	10/13/2019 12:00:00 AM	DODGE	SWINTON AVE	0	GRY
12331922	CKS1861	GA	06/17/2020 12:00:00 AM	Jeep	NB OCEAN PKWY @ AVE	0	GRAY
1723597	58388MG	NY	08/08/2019 12:00:00 AM	CHEVR	E 38th St	20190808	WH
2474539	AP628T	NJ	08/26/2019 12:00:00 AM	INTER	1st Ave	0	WHITE

```
set_index  
( 'Vehicle  
Color' )
```

	Plate ID	Registration State	Vehicle Make	Issue Date	Street Name	Date First Observed
Vehicle Color						
WHT	JFG4137	NY	DODGE	07/16/2019 12:00:00 AM	PACIFIC STREET	0
BK	DPH1199	NY	NISSA	07/01/2019 12:00:00 AM	160th St	0
YW	8D45B	NY	FORD	08/06/2019 12:00:00 AM	NB BAYCHESTER AVE @	0
WHITE	67974JV	NY	ISUZU	12/11/2019 12:00:00 AM	95th St	0
GRY	JBN3055	NY	DODGE	10/13/2019 12:00:00 AM	SWINTON AVE	0
GRAY	CKS1861	GA	Jeep	06/17/2020 12:00:00 AM	NB OCEAN PKWY @ AVE	0
WH	58388MG	NY	CHEVR	08/08/2019 12:00:00 AM	E 38th St	20190808
WHITE	AP628T	NJ	INTER	08/26/2019 12:00:00 AM	1st Ave	0

Рис. 4.2. Схематическое изображение переноса индекса со столбца Issue Date на столбец Vehicle Color

Решение

```

filename = '../data/nyc-parking-violations-2020.csv'
df = pd.read_csv(filename,
    usecols=['Date First Observed', 'Registration State', 'Plate ID',
            'Issue Date', 'Vehicle Make', 'Street Name', 'Vehicle Color'])
df = df.set_index('Issue Date') ❶
df.loc['01/02/2020 12:00:00 AM',
    'Vehicle Make'].value_counts().head(3) ❷
df.loc['06/01/2020 12:00:00 AM',
    'Street Name'].value_counts().head(5) ❸
df = df.reset_index().set_index('Vehicle Color') ❹
df.loc[['BLUE', 'RED'],
    'Vehicle Make'].value_counts().head(1) ❺

```

- ❶ Устанавливаем в качестве индекса столбец Issue Date.
- ❷ Находим данные за 2 января и оставляем только столбец Vehicle Make, после чего берем три самые популярные марки машин.
- ❸ Находим данные за 1 июня и оставляем только столбец Street Name, после чего берем пять самых популярных названий улиц.
- ❹ Убираем индекс со столбца Vehicle Make и устанавливаем его на столбец Vehicle Color.
- ❺ Находим все строки для красных и синих автомобилей и оставляем столбец Vehicle Make, после чего извлекаем одну наиболее популярную марку.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/1JWX>.

Дополнительные упражнения

Подобно тому как смена угла зрения часто помогает в решении той или иной задачи, изменение индекса в датафрейме позволяет значительно упростить ваш код. Выполните перечисленные ниже дополнительные упражнения, чтобы набить руку.

1. Водители каких марок автомобилей чаще остальных получали штрафные парковочные талоны в период новогодних праздников со 2 по 10 января?
2. На машину с каким регистрационным номером (столбец Plate ID) было выписано больше всех парковочных талонов? Почему она располагается на втором месте по популярности и почему нас не интересует лидер в этом списке? Из какого штата эта машина и были ли все штрафы выписаны в одном и том же месте?
3. Можно ли установить индекс на столбец Date First Observed и будет ли это полезно?

Множественные индексы

Все датафреймы обладают индексом в виде меток строк. Мы уже много раз выбирали строки по индексу с помощью атрибута `loc`. К примеру, мы могли написать `df.loc['a']` для извлечения всех строк со значением индекса `a`. Помните, что в индексе не обязаны находиться уникальные значения. Так что выражение `loc['a']` может вернуть как объект `Series`, представляющий одну строку, так и датафрейм, в котором будут собраны все строки с указанным значением индекса.

Такие одиночные индексы очень часто помогают нам строить лаконичные запросы к данным. Но иногда их бывает недостаточно. Причина в том, что наш мир полон иерархических данных, или данных, которые гораздо легче анализировать после приведения их к иерархическому виду.

К примеру, бизнес всегда интересует цифры по продажам. Но получение одной цифры не поможет нам проанализировать всю информацию в целом. Вместо этого нам бы хотелось видеть аналитику с разбивкой по товарам, чтобы понимать, какие из них вносят наибольший вклад в общее дело (в упражнении 8 мы уже касались этого вопроса). Но даже этого нам будет недостаточно. В идеале мы бы хотели знать, как те или иные товары продаются в разрезе месяцев. А если нашему магазину уже много лет, то аналитику придется строить и по годам. И в этом нам может помочь *множественный индекс* (*multi-index*).

Давайте для примера симулируем продажи трех разных товаров (A, B и C) за 36 месяцев: с января 2018 года по декабрь 2020-го:

```
# 3 товара * 3 года * 12 месяцев = 108 элементов

g = np.random.default_rng(0)
df = DataFrame(g.integers(0, 100, [36,3]),
               columns=list('ABC'))
df['year'] = [2018] * 12 + [2019] * 12 + [2020] * 12
df['month'] = ""Jan Feb Mar Apr May Jun
              Jul Aug Sep Oct Nov Dec"".split() * 3 ❶
df
```

❶ Тройные кавычки позволяют писать многострочный текст.

Вывод:

	A	B	C	year	month
0	85	63	51	2018	Jan
1	26	30	4	2018	Feb
2	7	1	17	2018	Mar
3	81	64	91	2018	Apr
..	..	..	..	...	...
32	4	8	37	2020	Sep
33	83	40	78	2020	Oct
34	31	23	79	2020	Nov
35	87	7	5	2020	Dec

[36 rows x 5 columns]

Мы бы могли определить в качестве индекса столбец `year`, как показано ниже:

```
df = df.set_index('year')
```

Но это не позволит нам провести анализ по месяцам с использованием индекса. Так что мы можем построить множественный индекс, передав список имен столбцов на вход методу `set_index` следующим образом (см. рис. 4.3):

```
df = df.set_index(['year', 'month'])
```

	A	B	C	year	month
0	44	47	64	2018	Jan
1	67	67	9	2018	Feb
2	83	21	36	2018	Mar
3	87	70	88	2018	Apr
4	88	12	58	2018	May
5	65	39	87	2018	Jun
6	46	88	81	2018	Jul
7	37	25	77	2018	Aug
8	72	9	20	2018	Sep

```
df.set_index(
    ['year',
     'month'])
```

		A	B	C
year	month			
2018	Jan	44	47	64
2018	Feb	67	67	9
2018	Mar	83	21	36
2018	Apr	87	70	88
2018	May	88	12	58
2018	Jun	65	39	87
2018	Jul	46	88	81
2018	Aug	37	25	77
2018	Sep	72	9	20

Рис. 4.3. Схематическое изображение процесса создания множественного индекса по столбцам с годом и месяцем

Помните, что при создании множественного индекса необходимо учитывать порядок образования иерархии. В нашем случае мы хотим, чтобы месяцы были включены в годы, в связи с чем указываем сначала столбец `year` и только затем `month`. Если бы вы создавали множественный индекс на основе данных компании о продажах, вы могли бы при его создании первым указать столбец со страной, вторым – с регионом внутри страны, а третьим – с городом. Обычно (но не всегда) множественный индекс отражает иерархию объектов.

Построив такой индекс, мы можем извлекать данные из датафрейма в различных разрезах. К примеру, мы можем получить суммарную информацию о продажах за весь 2018 год следующим образом:

```
df.loc[2018]
```

Также мы можем ограничить этот запрос только двумя товарами А и С, как показано ниже:

```
df.loc[2018, ['A', 'C']]
```

Обратите внимание, что здесь мы применили обычное правило обращения к датафрейму с помощью атрибута `loc`, указав сначала селектор строк, а затем – селектор столбцов. Без передачи второго аргумента мы бы получили данные из всех столбцов.

Но мы помним, что создали в нашем датафрейме множественный индекс, а значит, можно при помощи одного только нашего индекса извлечь информацию о продажах как по годам, так и по месяцам. К примеру, мы можем посмотреть, как все три товара продавались в июне 2018 года, следующим образом:

```
df.loc[(2018, 'Jun')]
```

Мы по-прежнему вызываем атрибут `loc` с квадратными скобками, но на этот раз передаем ему единственный аргумент в виде *кортежа* (tuple) в круглых скобках. Кортежи традиционно используются для обращения к множественному индексу, когда нам необходимо указать конкретную комбинацию уровней и значений в индексе. В данном случае мы хотим извлечь данные за июнь (вложенный уровень) 2018 года (внешний уровень), так что используем кортеж `(2018, 'Jun')`. Конечно, как и раньше, мы можем ограничить вывод только двумя столбцами с помощью второго аргумента, как показано ниже (см. рис. 4.4):

```
df.loc[(2018, 'Jun'), ['A', 'C']]
```

А что, если нам нужно посмотреть данные за два года? Все просто:

```
df.loc[[2018, 2020]]
```

Наложить дополнительный фильтр по столбцам в этом случае также не составит труда:

```
df.loc[[2018, 2020], ['B', 'C']]
```

А если мы захотим получить объединенную информацию за июнь в двух разных годах: 2018-м и 2020-м? Тут все несколько сложнее. В этом случае выстраивать мысли нужно примерно так:

- как обычно, используем квадратные скобки с атрибутом `loc`;
- первым аргументом мы должны передать селектор строк;
- нам нужны все столбцы, так что второго аргумента не будет;
- нам необходимо выбрать разные комбинации из множественного индекса, так что понадобится список;
- нам нужно несколько комбинаций из года и месяца, а значит, это должен быть список кортежей.

Результатом таких размышлений должно стать следующее выражение:

```
df.loc([(2018, 'Jun'), (2020, 'Jun')])
```

Селектор столбцов:
['A', 'C']

		A	B	C
year	month			
2018	Jan	44	47	64
2018	Feb	67	67	9
2018	Mar	83	21	36
2018	Apr	87	70	88
2018	May	88	12	58
2018	Jun	65	39	87
2018	Jul	46	88	81
2018	Aug	37	25	77
2018	Sep	72	9	20

Селектор строк:
(2018, 'Jun')

Результат:
[65, 87]

Рис. 4.4. Схематическое изображение процесса извлечения данных о продажах товаров A и C в июне 2018 года

А что, если нам понадобится получить информацию по трем летним месяцам за все годы? Конечно, можно собрать выражение по предыдущей схеме. Получится такой монстр:

```
df.loc([(2018, 'Jun'), (2018, 'Jul'), (2018, 'Aug'),
        (2019, 'Jun'), (2019, 'Jul'), (2019, 'Aug'),
        (2020, 'Jun'), (2020, 'Jul'), (2020, 'Aug')])
```

Работает, но выглядит не очень приятно. Есть ли более короткий способ? Вы могли бы предположить, что можно просто передать на вход атрибуту кортеж списков с нужными годами и месяцами, как показано ниже:

```
df.loc([(2018, 2019, 2020], ['Jun', 'Jul', 'Aug'])]
```

Но, к сожалению, это не сработает. Как ни странно, в этом случае pandas требует явного указания столбцов, которыми мы хотим ограничить свой выбор:

```
df.loc[[2018, 2019, 2020], ['Jun', 'Jul', 'Aug']],
      ['A', 'B', 'C']
```

Хотя второй аргумент в атрибуте loc обычно является необязательным и может быть опущен, если нам нужны все столбцы без исключения, в данной ситуации такой подход не работает. Но обычно вы не будете извлекать все колонки из датафрейма, так что можно и указать их явно.

Это можно сделать с помощью списка, как было показано выше, а можно передать и срез:

```
df.loc[[2018, 2019, 2020], ['Jun', 'Jul', 'Aug']], 'A':'C']
```

Для выбора всех столбцов есть очень короткий синтаксис с одним символом двоеточия, показанный ниже:

```
df.loc[[2018, 2019, 2020], ['Jun', 'Jul', 'Aug']], :]
```

С учетом того, что индекс отсортирован, мы могли бы выбрать все годы с помощью среза следующим образом:

```
df.loc[:, ['Jun', 'Jul', 'Aug']], 'A':'B'] ❶
```

❶ Это не сработает!

Но подождите, так делать нельзя, поскольку в Python двоеточие можно ставить только внутри квадратных скобок, а мы попытались использовать его в кортеже, т.е. в круглых скобках. Это ограничение можно обойти с помощью встроенной функции slice, передав ей аргумент None, как показано ниже:

```
df.loc[slice(None), ['Jun', 'Jul', 'Aug']], 'A':'B']
```

Да, теперь работает! Посредством вызова slice(None) вы можете сказать pandas, что хотите выбрать все значения.

Как видите, атрибут loc является довольно гибким и позволяет использовать множественные индексы на всю мощь.

Ответы на дополнительные упражнения

Упражнение 21.1

```
# Можно воспользоваться срезом, но только после сортировки индекса по возрастанию
df = df.set_index('Issue Date')
df = df.sort_index()
df.loc['01/02/2020 12:00:00 AM':'01/10/2020 23:59:59 PM', 'Vehicle Make'].
value_counts().head(3)
```

Вывод:

Vehicle Make

```
FORD      38958
TOYOTA    37096
HONDA     35962
Name: count, dtype: int64
```

```
df = df.reset_index() # Отменим предыдущую установку индекса
```

```
# Выполним это задание с помощью цепочки методов
```

```
(
    df
    .set_index('Issue Date')
    .sort_index()
    .loc['01/02/2020 12:00:00 AM':'01/10/2020 23:59:59 PM',
         'Vehicle Make']
    .value_counts()
    .head(3)
)
```

Вывод:

```
Vehicle Make
FORD      38958
TOYOTA    37096
HONDA     35962
Name: count, dtype: int64
```

Упражнение 21.2

```
# Самый распространенный регистрационный номер – пустой (BLANKPLATE)!
```

```
# Второй по популярности номер – 2704819
```

```
df = df.reset_index()
df['Plate ID'].value_counts().head(2)
```

Вывод:

```
Plate ID
BLANKPLATE    8882
2704819       1535
Name: count, dtype: int64
```

```
# Это номер из Индианы
```

```
df = df.set_index('Plate ID')
df.loc['2704819', 'Registration State']
```

Вывод:

```
Plate ID
2704819    IN
2704819    IN
...
2704819    IN
2704819    IN
Name: Registration State, Length: 1535, dtype: object
```

```
# Были ли все штрафы выписаны в одном и том же месте?
# Нет, но многие были где-то рядом
df.loc['2704819', 'Street Name'].value_counts()
```

Вывод:

```
Street Name
8th Ave          395
Penn Plz         230
7th Ave          92
9th Ave          63
Broadway         57
...
6TH AVE          1
W 54TH ST        1
E 39th St        1
N/S NW C/O W 30TH 1
E 49th St        1
Name: count, Length: 113, dtype: int64
```

Упражнение 21.3

Не очень полезно – в 99 % случаев значение в поле равно нулю

```
df = df.reset_index()
df['Date First Observed'].value_counts()
```

Вывод:

```
Date First Observed
0          12371344
20200311      887
20200205      795
20200212      793
20200310      770
...
20220412        1
20191131        1
20200813        1
20160614        1
20201230        1
Name: count, Length: 465, dtype: int64
```

УПРАЖНЕНИЕ 22. Оценки за вступительные тесты

Мы увидели, что установка правильных индексов способна существенно облегчить написание запросов к данным. Но иногда данные являются иерархическими по своей природе. И здесь в игру вступает концепция множественных индексов в pandas. С помощью них можно проиндексировать сразу несколько столбцов в датафрейме. К примеру, вам может понадобиться проанализировать продажи компании по годам, а затем по регионам. Когда в речи появляется это

«а затем», это почти всегда говорит о необходимости применить множественный индекс, о котором мы подробно писали в предыдущей врезке.

В этом упражнении мы поработаем с данными об оценках за стандартизированные вступительные тесты (SAT) в университет, широко распространенные в США. В файле-источнике (`sat-scores.csv`) находится 99 столбцов и 577 строк, в которых описываются результаты прохождения тестов с 2005 по 2015 годы в 50 штатах США и трех неинкорпорированных организованных территориях США (Пуэрто-Рико, Виргинские острова и Вашингтон, округ Колумбия).

Вот что вам необходимо сделать в этом упражнении.

1. Прочитать файл с оценками и сохранить его в виде датафрейма. Вам понадобятся только столбцы `Year`, `State.Code`, `Total.Math`, `Total.Test-takers` и `Total.Verbal`.
2. Создать множественный индекс на основе года и двухбуквенного обозначения штата.
3. Узнать, сколько человек проходили тесты в 2005 году (`Total.Test-takers`).
4. Рассчитать среднюю оценку по математике (`Total.Math`) за 2010 год для абитуриентов из штатов Нью-Йорк (NY), Нью-Джерси (NJ), Массачусетс (MA) и Иллинойс (IL).
5. Рассчитать среднюю оценку по чтению (`Total.Verbal`) за годы с 2012-го по 2015-й для абитуриентов из штатов Аризона (AZ), Калифорния (CA) и Техас (TX).

Подробный разбор

В этом упражнении вы по достоинству оцените всю мощь и гибкость множественных индексов в `pandas`. Я попросил вас загрузить файл CSV в датафрейм и создать множественный индекс по столбцам `Year` и `State.Code`. Это можно сделать в два этапа – сначала прочитав содержимое файла в датафрейм, включив в него нужные нам столбцы, а затем выбрав в качестве индекса указанные колонки:

```
filename = '../data/sat-scores.csv'

df = pd.read_csv(filename,
                  usecols=['Year', 'State.Code',
                           'Total.Math', 'Total.Test-takers',
                           'Total.Verbal'])
df = df.set_index(['Year', 'State.Code'])
```

Результатом вызова метода `set_index` будет новый датафрейм, который мы обратно присвоим переменной `df`. Но вы помните, что у метода `read_csv` есть параметр `index_col`, с помощью которого можно на этапе создания датафрейма указать, какие столбцы должны выступать в качестве индекса. Это позволит сократить число шагов до одного, как показано ниже:

```
filename = '../data/sat-scores.csv'
df = pd.read_csv(filename,
                  usecols=['Year', 'State.Code',
                           'Total.Math', 'Total.Test-takers',
                           'Total.Verbal'],
                  index_col=['Year', 'State.Code'])
```

Теперь, когда у нас есть датафрейм, мы можем исследовать его и ответить на поставленные вопросы.

Сначала узнаем, сколько человек проходили тесты в 2005 году. Для этого нам необходимо найти все строки за 2005 год (часть множественного индекса), извлечь данные из столбца `Total.Test-takers` и просуммировать их, как показано ниже:

```
df.loc[2005,          ❶
       'Total.Test-takers'  ❷
       ].sum()
```

- ❶ Селектор строк.
- ❷ Селектор столбцов.

Далее нас попросили рассчитать среднюю оценку по математике (`Total.Math`) за 2010 год для абитуриентов из штатов Нью-Йорк (NY), Нью-Джерси (NJ), Массачусетс (MA) и Иллинойс (IL). Как обычно, воспользуемся атрибутом доступа `loc`, чтобы получить доступ ко всей интересующей нас информации. Для создания правильного запроса нам понадобятся три составляющие:

- из первой части множественного индекса (`Year`) нужно извлечь 2010 год;
- из второй части множественного индекса (`State.Code`) нужно извлечь штаты NY, NJ, MA и IL;
- из столбцов нам понадобится только столбец с именем `Total.Math`.

При извлечении данных из множественного индекса мы должны объединять составляющие части запроса в кортежи. А списки помогают выбрать не один, а сразу несколько элементов из индекса. В результате мы получим следующее выражение:

```
df.loc[(2010, ['NY', 'NJ', 'MA', 'IL']), ❶
       'Total.Math'].mean()           ❷
```

- ❶ Селектор строк, объединяющий 2010 год и четыре выбранных штата.
- ❷ Селектор столбцов, указывающий на столбец `Total.Math`.

В итоге мы получим все строки за 2010 год по всем четырём штатам и оставим для вычисления среднего значения только столбец `Total.Math`, как показано на рис. 4.5.

Следующий вопрос похож на предыдущий, но выбор необходимо сделать на основе нескольких годов и штатов. Опять же, в этом нет никаких проблем, если понимать, как строятся запросы:

- из первой части множественного индекса (`Year`) нам нужно извлечь следующие годы: 2012, 2013, 2014 и 2015;
- из второй части множественного индекса (`State.Code`) нам нужно оставить штаты AZ, CA и TX;
- из столбцов нас снова интересует только столбец `Total.Math`.

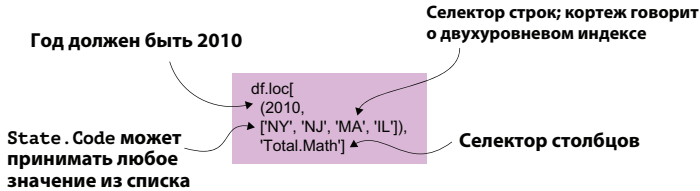


Рис. 4.5. Схематическое изображение применения атрибута `loc` с множественным индексом

В результате получим следующий запрос:

```
df.loc([(2012, 2013, 2014, 2015), ['AZ', 'CA', 'TX']], ❶
       'Total.Math'].mean() ❷
```

- ❶ Селектор строк, объединяющий 2012–2015 годы и три выбранных штата.
- ❷ Селектор столбцов, указывающий на столбец `Total.Math`.

Обратите внимание, что `pandas` сам знает, как комбинировать части нашего множественного индекса, так что мы получим только строки, соответствующие нашему запросу.

Решение

```
filename = '../data/sat-scores.csv'
df = pd.read_csv(filename,
                  usecols=['Year',
                           'State.Code',
                           'Total.Math',
                           'Total.Test-takers',
                           'Total.Verbal'])

df = df.set_index(['Year', 'State.Code']) ❶
df.loc[2005, 'Total.Test-takers'].sum() ❷

df.loc[(2010, ['NY', 'NJ', 'MA', 'IL']), ❸
       'Total.Math'].mean()
df.loc([(2012, 2013, 2014, 2015), ❹
       ['AZ', 'CA', 'TX']],
       'Total.Math'].mean()
```

- ❶ Устанавливаем индекс в виде комбинации столбцов `Year` и `State.Code`.
- ❷ Извлекаем строки за 2005 год (столбец `Total.Test-takers`) и суммируем значения.
- ❸ Извлекаем строки за 2010 год по четырем указанным штатам (столбец `Total.Math`) и вычисляем среднее значение.
- ❹ Извлекаем строки за 2012–2015 годы по трем указанным штатам (столбец `Total.Math`) и вычисляем среднее значение.

Вы можете посмотреть это решение на сайте `Pandas Tutor` по адресу <http://mng.bz/PRpw>.

Дополнительные упражнения

1. Какие наблюдались средние оценки по математике и чтению за все время по штатам Флорида (FL), Индиана (IN) и Айдахо (ID)? По штатам аналитику разбивать не нужно.
2. В каком штате был получен высший балл за тест по чтению и в каком это было году?
3. Был ли средний балл за тест по математике в 2005 году выше или ниже аналогичного показателя в 2015 году?

Сортировка по индексу

Говоря о сортировке в pandas, мы обычно подразумеваем сортировку данных в датафреймах. К примеру, мы можем упорядочить строки в датафрейме по цене или любому другому столбцу. Подробнее о сортировке данных мы будем говорить в главах 6 и 7.

Но в pandas мы также можем сортировать датафреймы на основе индекса. Для этого существует отдельный метод `sort_index`, который, как и многие другие методы, возвращает новый датафрейм с тем же содержимым, упорядоченным по значениям в индексе. Таким образом, вы можете просто написать:

```
df = df.sort_index()
```

Если в датафрейме присутствует множественный индекс, сортировка будет выполняться по уровням индекса последовательно – сначала по первому, затем по второму и т. д.

В дополнение к эстетическим превосходствам сортировка по индексу может значительно облегчить некоторые операции, а какие-то без нее выполнить невозможно. К примеру, если вы попытаетесь извлечь срез наподобие `df.loc['a': 'c']`, pandas потребует, чтобы индекс был отсортирован.

К тому же, если ваш датафрейм не отсортирован и в нем есть множественный индекс, при выполнении некоторых операций вы будете получать предупреждение следующего вида:

```
PerformanceWarning: indexing past lexsort depth may impact performance
```

Тем самым pandas пытается сказать вам о том, что при наличии объемных данных и неотсортированного множественного индекса у вас вполне могут возникнуть проблемы с быстродействием. Избежать подобных предупреждений поможет сортировка датафрейма по индексу.

Для проверки того, отсортирован ли ваш датафрейм, вы можете воспользоваться атрибутом `is_monotonic_increasing`:

```
df.index.is_monotonic_increasing
```

Когда мы говорим, что индекс монотонно увеличивается, мы просто имеем в виду, что значения в нем постоянно растут. Так же точно при постоянном снижении значений мы можем сказать, что индекс монотонно уменьшается (`is_monotonic_decreasing`). Обратите внимание, что это не методы, а атрибуты с булевыми значениями. Они присутствуют во всех объектах Series, а не только в индексах. В некоторых более ранних документациях упоминается также метод `is_lexsorted`, который был упразднен в поздних версиях pandas.

Ответы на дополнительные упражнения

Упражнение 22.1

```
df.loc[(slice(None), ['FL', 'IN', 'ID']), ['Total.Math', 'Total.Verbal']].mean()
```

Вывод:

```
Total.Math      507.090909
Total.Verbal     504.606061
dtype: float64
```

Упражнение 22.2

```
# Мы можем вычислить это вручную
df.loc[df['Total.Verbal'] == df['Total.Verbal'].max()]
```

Вывод:

```
                Total.Math  Total.Test-takers  Total.Verbal
Year State.Code
2013 ND                613                174            612
```

```
# ... а можем воспользоваться методом idxmax для получения индекса наивысшей оценки
df['Total.Verbal'].idxmax()
```

Вывод:

```
(np.int64(2013), 'ND')
```

Упражнение 22.3

```
# При наличии множественного индекса мы можем игнорировать второй уровень
df.loc[2005, 'Total.Math'].mean() - df.loc[2015, 'Total.Math'].mean()
```

Вывод:

```
2.559506531204647
```

УПРАЖНЕНИЕ 23. Олимпийские игры

Современные Олимпийские игры проводятся уже более столетия, и даже такие далекие от спорта люди, как я, временами с упоением наблюдают по телевизору за состязаниями лучших в мире атлетов. К счастью, Олимпиада производит на свет не только чемпионов и чемпионки, но и огромное количество данных, которые можно проанализировать при помощи *pandas*.

В предыдущем упражнении мы узнали на практике, как строить множественные индексы, состоящие из двух уровней. Но вы не обязаны ограничиваться таким их количеством. Фактически уровней в индексах может быть сколько угодно. Вы можете легко представить себе большую корпорацию, в которой отчеты по продажам разбиваются на регионы, страны и отделы. Множественные индексы позволяют вам проанализировать данные в самых разных разрезах – будь то просмотр верхнеуровневых данных или детализация до нижних уровней.

В этом упражнении мы потренируемся создавать глубокие множественные индексы, с помощью которых можно погружаться в данные и извлекать из них нужные нам сведения на любом уровне. Задание будет состоять из следующих пунктов.

1. Прочитайте файл с данными (`olympic_athlete_events.csv`) в датафрейм. Нам понадобятся только столбцы `Age`, `Height`, `Team`, `Year`, `Season`, `City`, `Sport`, `Event` и `Medal`. Множественный индекс должен базироваться на следующих четырех полях: `Year`, `Season`, `Sport` и `Event`.
2. Ответьте на поставленные вопросы:
 - каков был средний возраст олимпийских чемпионов летних игр за период с 1936 по 2000 годы?
 - какая страна завоевала больше всех медалей в стрельбе из лука ('Archery')?
 - рассчитайте средний рост спортсменов, участвовавших в соревнованиях по настольному теннису начиная с 1980 года ('Table Tennis Women's Team');
 - рассчитайте средний рост спортсменов и спортсенок, участвовавших в соревнованиях по настольному теннису начиная с 1980 года ('Table Tennis Women's Team' и 'Table Tennis Men's Team');
 - какой рост был у самого высокого теннисиста ('Tennis'), принимавшего участие в Олимпийских играх в период с 1980 по 2016 годы?

Подробный разбор

В этом упражнении мы создадим множественный индекс, состоящий из четырех уровней, и затем воспользуемся им для ответа на поставленные вопросы. В процессе решения упражнения вы прочувствуете всю глубину и мощь множественных индексов в `pandas`.

Но для начала загрузим данные в датафрейм, перечислив нужные нам столбцы и одновременно создав индекс:

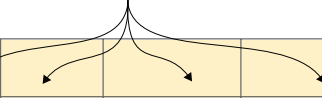
```
filename = '../data/olympic_athlete_events.csv'

df = pd.read_csv(filename,
                  index_col=['Year', 'Season',
                             'Sport', 'Event'],          ❶
                  usecols=['Age', 'Height', 'Team',
                           'Year', 'Season', 'City',
                           'Sport', 'Event', 'Medal'])    ❷
df = df.sort_index()                                     ❸
```

- ❶ Указываем столбцы и их порядок следования в индексе.
- ❷ Читаем файл CSV в датафрейм с девятью столбцами, четыре из которых будут использованы в качестве индекса.
- ❸ Сортируем датафрейм по индексу.

Передав параметр `index_col` в функцию `read_csv`, мы тем самым определили множественный индекс в процессе создания датафрейма, что видно на рис. 4.6.

Множественный индекс



Year	Season	Sport	Event	Age	Height	Team	City	Medal
1996	Summer	Athletics	Athletics Men's 10,000 meters	27.0	178.0	United States	Atlanta	NaN
1992	Winter	Biathlon	Biathlon Women's 15 kilometers	22.0	NaN	China	Albertville	NaN
2012	Summer	Fencing	Fencing Men's Foil, Team	29.0	180.0	China	London	NaN
1988	Winter	Cross-Country Skiing	Cross-Country Skiing Men's 50 kilometers	24.0	174.0	Sweden	Calgary	NaN
1900	Summer	Rowing	Rowing Men's Coxed Eights	21.0	NaN	Germania Ruder Club, Hamburg	Paris	NaN
2006	Winter	Biathlon	Biathlon Men's 4 x 7.5 kilometers Relay	28.0	180.0	Czech Republic	Torino	NaN
2004	Summer	Cycling	Cycling Men's Mountainbike, Cross-Country	22.0	178.0	Spain	Athina	NaN
1912	Summer	Gymnastics	Gymnastics Men's Team All-Around	20.0	NaN	Germany	Stockholm	NaN
1952	Summer	Rowing	Rowing Men's Coxless Fours	26.0	186.0	Norway	Helsinki	NaN
1994	Winter	Ski Jumping	Ski Jumping Men's Large Hill, Team	23.0	175.0	Italy	Lillehammer	NaN

Рис. 4.6. Датафрейм с четырьмя столбцами, выступающими в роли индекса

После создания датафрейма мы воспользовались методом `sort_index`, который вернул нам новый датафрейм с упорядоченными строками в соответствии с определенным индексом. При применении метода `sort_index` к датафрейму с множественным индексом данные сначала упорядочиваются по первому уровню индекса (в нашем случае `Year`), далее по второму (`Season`) и т. д.

ПРИМЕЧАНИЕ. При вызове метода `set_index` вы можете передать параметр `inplace=True`. В этом случае будет модифицирован исходный датафрейм, а сам метод вернет значение `None`. Но, как мы уже говорили, разработчики библиотеки `pandas` настоятельно не рекомендуют пользоваться этой техникой. Вместо этого лучше оставить параметр `inplace` по умолчанию (`False`) и присвоить возвращенный методом новый датафрейм той же переменной, в которой находился исходный датафрейм.

Хотя нам совсем не обязательно сортировать данные по индексу, некоторые операции в `pandas` в этом случае будут работать эффективнее. Кроме того, при взаимодействии с неотсортированным датафреймом вы можете получить предупреждение `PerformanceWarning`, которое мы упоминали ранее. Так что, даже когда

мы имеем дело с множественным индексом, всегда лучше выполнить сортировку по нему сразу после создания датафрейма.

Теперь, когда мы получили нужный нам датафрейм, можно начать отвечать на вопросы из упражнения. Сначала нас спросили, каков был средний возраст олимпийских чемпионов летних игр за период с 1936 по 2000 годы. Для ответа на этот вопрос нам необходимо выбрать подмножество лет (первый уровень нашего индекса) и подмножество сезонов (т. е. только те строки, для которых в столбце Season, представляющем второй уровень в нашем индексе, стоит значение Summer). По третьему и четвертому уровням множественного индекса нам ограничивать данные не нужно, а значит, мы можем просто проигнорировать их. В этом случае из этих колонок будут выбраны все значения. Иными словами, нам нужно выбрать следующее (см. рис. 4.7):

- все годы с 1936-го по 2000-й, что можно выразить следующим образом: `slice(1936, 2000)`;
- все игры, для которых в столбце Season стоит значение Summer;
- столбец Age для вычисления агрегации.

Селектор столбцов: Age

Селектор строк: сезон 'Summer' и годы с 1936 по 2000

				Age	Height	Team	City	Medal
Year	Season	Sport	Event					
1996	Summer	Athletics	Athletics Men's 10,000 meters	27.0	178.0	United States	Atlanta	NaN
1992	Winter	Biathlon	Biathlon Women's 15 kilometers	22.0	NaN	China	Albertville	NaN
2012	Summer	Fencing	Fencing Men's Foil, Team	29.0	180.0	China	London	NaN
1988	Winter	Cross-Country Skiing	Cross-Country Skiing Men's 50 kilometers	24.0	174.0	Sweden	Calgary	NaN
1900	Summer	Rowing	Rowing Men's Coxed Eights	21.0	NaN	Germania Ruder Club, Hamburg	Paris	NaN
2006	Winter	Biathlon	Biathlon Men's 4 x 7.5 kilometers Relay	28.0	180.0	Czech Republic	Torino	NaN
2004	Summer	Cycling	Cycling Men's Mountainbike, Cross-Country	22.0	178.0	Spain	Athina	NaN
1912	Summer	Gymnastics	Gymnastics Men's Team All-Around	20.0	NaN	Germany	Stockholm	NaN
1952	Summer	Rowing	Rowing Men's Coxless Fours	26.0	186.0	Norway	Helsinki	NaN
1994	Winter	Ski Jumping	Ski Jumping Men's Large Hill, Team	23.0	175.0	Italy	Lillehammer	NaN

Рис. 4.7. Схематическое изображение применения селектора строк к множественному индексу

Среднее значение по этим фильтрам можно вычислить следующим образом:

```
df.f.loc[(slice(1936,2000), 'Summer'),  
         'Age']  
         .mean()
```

- ❶ Селектор строк: годы с 1936-го по 2000-й и летний сезон (данные из первых двух уровней множественного индекса).
- ❷ Селектор столбцов: нам понадобится только столбец Age.
- ❸ Применяем агрегацию mean к полученному объекту Series.

Ответом будет число с плавающей точкой 25.026883940421765.

Далее нас попросили узнать, какая страна завоевала больше всех медалей в стрельбе из лука ('Archery'). Как мы построим наш запрос? Нам нужно продумать фильтры по всем уровням нашего множественного индекса:

- нужно оставить все годы, так что первый уровень индекса мы отфильтруем с помощью выражения slice(None);
- стрельба из лука входит в состав летних Олимпиад, так что мы можем либо указать для столбца Season (второй уровень индекса) значение Summer, либо воспользоваться выражением slice(None);
- на третьем уровне индекса мы должны оставить только интересующий нас вид спорта – Archery;
- четвертый уровень индекса мы просто проигнорируем, что позволит оставить в нем все имеющиеся данные.

Нам необходимо узнать, у какой страны окажется больше всех медалей при действии указанных выше фильтров. Таким образом, нас интересует столбец Team. Для определения того, какая команда оказалась первой в списке, воспользуемся методом value_counts. Соответственно, наш запрос будет выглядеть так:

```
df.loc[(slice(None), 'Summer', 'Archery'),  
       'Team']  
       .value_counts()
```

- ❶ Селектор строк: все годы, летние игры, все соревнования по стрельбе из лука.
- ❷ Селектор столбцов: нам понадобится только столбец Team.
- ❸ Применяем метод value_counts к итоговому объекту Series.

Но постойте, этот запрос выведет *всех* участников соревнований по стрельбе из лука, а нам ведь нужны только медалисты! Для реализации этого требования можно на ранней стадии запроса избавиться от строк, в которых в столбце Medal стоит значение NaN, воспользовавшись методом dropna с параметром subset='Medal'. В формате цепочки методов этот запрос выглядит очень лаконично и понятно:

```
(  
    df  
    .dropna(subset='Medal')  
    .loc[(slice(None), 'Summer', 'Archery'), 'Team']  
    .value_counts()  
)
```

Вот так выглядят первые пять строк результата:

Team	
South Korea	69
Belgium	52
France	48
United States	41
China	19

Поскольку метод `value_counts` сам упорядочивает данные по убыванию значения, мы можем сделать вывод, что представители Южной Кореи завоевали в означенный период больше всех медалей в стрельбе из лука. Следом за ними идут бельгийцы, французы, американцы и китайцы.

Далее нам нужно ответить на вопрос о том, каков средний рост спортсменов, участвовавших в соревнованиях по настольному теннису начиная с 1980 года ('Table Tennis Women's Team'). Давайте снова разберем наш индекс на составляющие и мысленно отфильтруем его:

- нам нужны все результаты начиная с 1980 года (первый уровень индекса);
- настольный теннис входит в состав летних Олимпиад, так что мы снова можем либо указать для столбца `Season` (второй уровень индекса) значение `Summer`, либо воспользоваться выражением `slice(None)`;
- нас интересует только вид спорта настольный теннис ('Table tennis'), так что мы могли бы указать его на третьем уровне индекса, но, учитывая, что все события с названием 'Table Tennis Women's Team' входят в этот вид, можно для третьего уровня использовать выражение `slice(None)`;
- определим для четвертого уровня индекса (Event) значение 'Table Tennis Women's Team'.

Нас интересует только столбец `Height`, так что мы укажем его в селекторе столбцов. Запрос получится следующий:

```
df.loc[(slice(1980, None),           ❶
        'Summer',                   ❷
        slice(None),                 ❸
        "Table Tennis Women's Team"), ❹
        'Height'                     ❺
        ].mean()                     ❻
```

- ❶ Селектор строк, часть 1: начиная с 1980 года.
- ❷ Селектор строк, часть 2: летние Олимпийские игры.
- ❸ Селектор строк, часть 3: все виды спорта.
- ❹ Селектор строк, часть 4: только события "Table Tennis Women's Team".
- ❺ Селектор столбцов: только столбец `Height`.
- ❻ Применяем метод `mean` к итоговому объекту `Series`.

В результате мы получим значение 165.04827586206898, что соответствует росту 165 см.

Для следующего запроса мы расширим требования к запрашиваемым данным, добавив к ним мужские соревнования по настольному теннису. Как следст-

вие, первые три части селектора строк у нас останутся прежними, а в четвертой мы укажем список из двух значений вместо строки, как показано ниже:

```
df.loc[(slice(1980, None),           ❶
        'Summer',                     ❷
        slice(None),                  ❸
        ["Table Tennis Men's Team",
         "Table Tennis Women's Team"]), ❹
        'Height'                      ❺
        ].mean()                      ❻
```

- ❶ Селектор строк, часть 1: осталась прежней.
- ❷ Селектор строк, часть 2: осталась прежней.
- ❸ Селектор строк, часть 3: осталась прежней.
- ❹ Селектор строк, часть 4: только события "Table Tennis Women's Team" и "Table Tennis Men's Team".
- ❺ Селектор столбцов: остался прежним.
- ❻ Снова применяем метод `mean` к итоговому объекту `Series`.

С учетом того что мужчины в среднем выше женщин, неудивительно, что добавление в запрос мужских соревнований существенно повлияло на средний рост спортсменов, который стал равен 171.26643598615917, или 171 см.

Наконец, нас попросили узнать, какой рост был у самого высокого теннисиста ('Tennis'), принимавшего участие в Олимпийских играх в период с 1980 по 2016 годы. И снова пройдемся по структуре индекса:

- нам нужно оставить только соревнования, проводившиеся в период с 1980-го по 2016-й, что можно легче всего сделать с помощью среза `slice(1980, 2016)`;
- теннис входит в состав летних Олимпиад, так что мы опять можем либо указать для столбца `Season` (второй уровень индекса) значение `Summer`, либо воспользоваться выражением `slice(None)`;
- нас интересует только теннис ('Tennis'), так что мы укажем его на третьем уровне индекса;
- нам нужны все события в рамках теннисных турниров, а значит, четвертый элемент в кортеже можно пропустить.

Нас снова интересует только столбец `Height`, так что укажем его в селекторе столбцов. На этот раз воспользуемся методом агрегации `max`, поскольку нас попросили найти самого высокого теннисиста. Итоговый запрос будет выглядеть так:

```
df.loc[(slice(1980, 2016),           ❶
        'Summer',                     ❷
        'Tennis'),                   ❸
        'Height'                      ❹
        ].max()                      ❺
```

- ❶ Селектор строк, часть 1: годы с 1980-го по 2016-й.
- ❷ Селектор строк, часть 2: только летние Олимпийские игры.
- ❸ Селектор строк, часть 3: только теннис.
- ❹ Селектор столбцов: только столбец `Height`.
- ❺ Применяем метод `max` к итоговому объекту `Series`.

Рост самого высокого теннисиста составил 208 см. Очень высокий парень!

Решение

```

filename = '../data/olympic_athlete_events.csv'

df = pd.read_csv(filename,
                  index_col=['Year', 'Season',
                             'Sport', 'Event'],
                  usecols=['Age', 'Height', 'Team',
                           'Year', 'Season', 'City',
                           'Sport', 'Event', 'Medal']) ❶

df = df.sort_index() ❷
df.loc[(slice(1936, 2000), 'Summer'), 'Age'].mean() ❸
df.dropna(subset='Medal').loc[
    (slice(None), 'Summer', 'Archery'),
    'Team'].value_counts() ❹
df.loc[(slice(1980, None), 'Summer', slice(None),
          "Table Tennis Women's Team"),
        'Height'].mean() ❺
df.loc[(slice(1980, None),
          'Summer', slice(None),
          ["Table Tennis Men's Team",
           "Table Tennis Women's Team"]),
        'Height'].mean() ❻
df.loc[(slice(1980, 2016),
          'Summer',
          'Tennis'), 'Height'].max() ❼

```

- ❶ Читаем файл CSV в датафрейм с девятью столбцами, четыре из которых будут использованы в качестве индекса.
- ❷ Сортируем датафрейм по индексу.
- ❸ Получаем средний возраст атлетов, участвовавших в летних играх в период с 1936-го по 2000-й.
- ❹ Какие страны завоевали больше всех медалей в стрельбе из лука?
- ❺ Каким был средний рост спортсменов, участвовавших в соревнованиях по настольному теннису начиная с 1980 года?
- ❻ Каким был суммарный средний рост спортсменов и спортсенок, участвовавших в соревнованиях по настольному теннису начиная с 1980 года?
- ❼ Какой рост был у самого высокого теннисиста, принимавшего участие в Олимпийских играх в период с 1980 по 2016 годы?

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mnng.bz/JdXo>.

Углубляемся...

Как мы уже видели, с помощью атрибута доступа `loc` можно легко и просто извлечь нужные вам данные, воспользовавшись множественным индексом. Но иногда индексы такого типа мы используем иначе. Для этого в библиотеке `pandas` присутствуют два следующих метода: `xs` и `IndexSlice`.

Поскольку множественные индексы получили большое распространение, в pandas предусмотрели разные способы извлечения данных с их помощью. Начнем с метода `xs`, который можно применить для решения задач, представленных в этом упражнении, а именно для извлечения данных на разных уровнях множественного индекса. Один из вопросов в упражнении звучал так: рассчитайте средний рост спортсменов, участвовавших в соревнованиях по настольному теннису (годы участия мы опустим). С помощью атрибута `loc` мы можем сообщить pandas о необходимости использовать все значения на уровнях индекса `Year`, `Season` и `Sport`, ограничив выбор только четвертым уровнем индекса (`Event`). Запрос с `loc` выглядел бы так:

```
df.loc[(slice(None),           ❶
        'Summer',             ❷
        slice(None),          ❸
        "Table Tennis Women's Team"), ❹
        'Height']              ❺
        .mean()                ❻
```

- ❶ Селектор строк, часть 1: все годы.
- ❷ Селектор строк, часть 2: летние Олимпийские игры.
- ❸ Селектор строк, часть 3: все виды спорта.
- ❹ Селектор строк, часть 4: только события "Table Tennis Women's Team".
- ❺ Селектор столбцов: только столбец Height.
- ❻ Применяем метод `mean` к итоговому объекту `Series`.

С использованием метода `xs` мы можем значительно сократить запись:

```
df.xs("Table Tennis Women's Team", ❶
      level='Event'                 ❷
      ).mean()                       ❸
```

- ❶ Находим строки со значением "Table Tennis Women's Team"...
- ❷ ...на уровне множественного индекса с именем `Event`.
- ❸ Применяем метод `mean` к итоговому объекту `Series`.

Но вы могли заметить, что в выражении с использованием атрибута `loc` мы также применили фильтр по сезону. К счастью, в методе `xs` мы можем передать список уровней в параметре `level` и кортеж – в качестве первого аргумента со значениями для поиска, как показано ниже:

```
df.xs(('Summer', "Table Tennis Women's Team"), ❶
      level=['Season', 'Event']).mean() ❷
```

- ❶ Передаем кортеж из двух элементов для фильтрации по двум уровням индекса.
- ❷ Список в аргументе `level` говорит о необходимости выполнять поиск по нескольким уровням индекса.

Обратите внимание, что `xs` представляет собой метод, в связи с чем вызывается с круглыми скобками. Напротив, `loc` – это атрибут доступа, что объясняет обращение к нему посредством квадратных скобок. Да, иногда с этим путаешься...

Кстати, параметр `level` может принимать целочисленные значения с указанием порядковых номеров уровней индекса вместо их названий. Мне кажется, что доступ по именам уровней выглядит более наглядно, так что я рекомендую вам пользоваться именно этим способом.

Более обобщенный способ извлечения данных из множественного индекса предоставляет класс `IndexSlice`. Помните, я упоминал, что мы не можем ставить двоеточия в круглых скобках, в связи с чем вынуждены использовать более многословный синтаксис `slice(None)`? Класс `IndexSlice` позволяет решить эту проблему. В нем применяются квадратные скобки, и синтаксис, характерный для срезов, может быть использован для любого набора значений. К примеру, мы можем написать:

```
from pandas import IndexSlice as idx
df.loc[idx[1980:2016, :, 'Swimming':'Table tennis'], :] ❶
```

- ❶ **Годы 1980–2016, все сезоны, все виды спорта от плавания (Swimming) до настольного тенниса (Table tennis).**

Такой синтаксис позволяет выбрать диапазон значений для каждого уровня во множественном индексе. Нам больше нет нужды обращаться к функции `slice`, достаточно обычного синтаксиса срезов в Python с использованием двоеточия. Результатом вызова `IndexSlice` (или `idx`, принятого у нас в качестве алиаса) будет кортеж, состоящий из объектов `slice`:

```
(slice(1980, 2016, None),
 slice(None, None, None),
 slice('Swimming', 'Table tennis', None))
```

Иными словами, класс `IndexSlice` по своей сути является синтаксическим сахаром, позволяющим обращаться к структурам данных в `pandas`, как к обычным объектам в Python, даже при наличии очень сложных индексов.

И последнее замечание: датафреймы могут располагать множественными индексами как по строкам или столбцам, так и по обоим измерениям сразу. По умолчанию метод `xs` предполагает, что множественный индекс установлен на строки. При использовании индекса по столбцам вам необходимо передать ему дополнительный аргумент `axis='columns'`.

Дополнительные упражнения

1. Соревнования на Олимпийских играх могут проводиться или летом, или зимой, но не одновременно. Как следствие, уровень `Season` в нашем множественном индексе зачастую будет не нужен. Избавьтесь от этого уровня индекса и снова ответьте на вопрос о самом высоком теннисисте, выступавшем в период с 1980 по 2016 годы.
2. В каком городе было вручено максимальное количество золотых медалей начиная с 1980 года?
3. Сколько золотых медалей получили спортсмены и спортсменки из США с 1980 года? Воспользуйтесь индексом для выбора значений.

Сводные таблицы

До сих пор мы использовали индексы для реорганизации наших данных с целью облегчения доступа к ним в любых нужных нам разрезах и ответа на интересующие нас вопросы аналитического характера. Но все заданные вопросы предполагали наличие одного скалярного числового ответа. Зачастую же нам требуется применить определенного рода агрегацию к разным комбинациям строк и столбцов. И одним из самых удобных инструментов для реализации этого являются *сводные таблицы* (pivot table).

Сводная таблица позволяет создать новую таблицу (датафрейм) на основе поднабора данных из существующего датафрейма. Основная идея такая:

- в наших данных есть два столбца, содержащих категориальные, повторяющиеся, неиерархические значения. Это могут быть годы, названия стран, цвета или подразделения компании;
- есть третий столбец с числовыми данными;
- мы хотим создать новый датафрейм из существующего с соблюдением следующих правил:
 - все уникальные значения из первой колонки с категориальными данными должны стать индексами, или метками строк;
 - все уникальные значения из второй колонки с категориальными данными должны стать метками столбцов;
 - на пересечениях строк и столбцов должны располагаться либо одиночные значения, соответствующие совпадению категорий в исходных данных, либо средние (при наличии нескольких значений).

Некоторым требуется время, чтобы понять, как на самом деле работают сводные таблицы. Но как только вы проникнетесь этим инструментом, вы начнете применять его практически везде – где надо и не надо.

Рассмотрим следующий набор исходных данных:

```
g = np.random.default_rng(0)
df = DataFrame(g.integers(0, 100, [8,3]),
               columns=list('ABC'))
df['year'] = [2018]*4+[2019] * 4
df['quarter'] = 'Q1 Q2 Q3 Q4'.split() * 2
```

В этом датафрейме представлена случайным образом сгенерированная информация о продажах трех разных товаров (A, B и C) в разрезе годов (2018 и 2019) и кварталов (Q1, Q2, Q3 и Q4). При взгляде на эти данные легко понять, что они из себя представляют:

	A	B	C	year	quarter
0	85	63	51	2018	Q1
1	26	30	4	2018	Q2
2	7	1	17	2018	Q3
3	81	64	91	2018	Q4
4	50	60	97	2019	Q1
5	72	63	54	2019	Q2
6	55	93	27	2019	Q3
7	81	67	0	2019	Q4

Но что, если нас интересует сводная информация по продажам товара А? Нам было бы легче анализировать эти данные, если бы кварталы (категориальные, повторяющиеся значения) располагались в строках, годы (тоже категориальные и повторяющиеся) – в столбцах, а на пересечениях присутствовали цифры продаж товара А за соответствующий период. Такую сводную таблицу можно очень легко создать с помощью следующего выражения:

```
df.pivot_table(index='quarter', ❶
               columns='year',    ❷
               values='A')        ❸
```

- ❶ Строки (индекс) – уникальные значения из столбца с кварталами.
- ❷ Столбцы – уникальные значения из столбца с годами.
- ❸ Средние значения по пересечению года и квартала.

Результат может выглядеть так, как показано ниже (см. рис. 4.8):

```
year    2018  2019
quarter
Q1      85.0  50.0
Q2      26.0  72.0
Q3       7.0  55.0
Q4      81.0  81.0
```

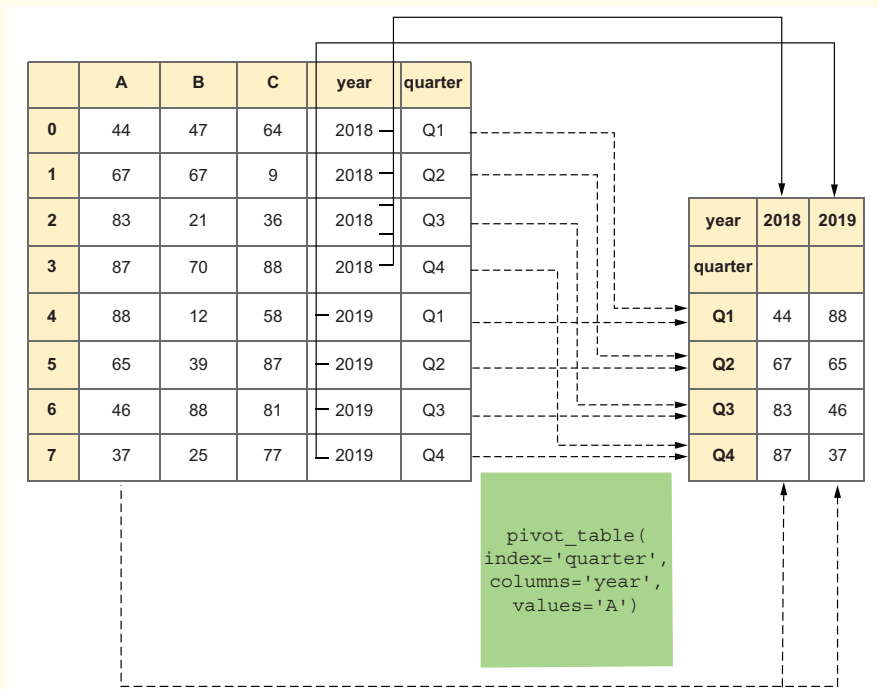


Рис. 4.8. Схематическое изображение процесса создания сводной таблицы с индексом в виде кварталов, столбцами в виде годов и значениями продаж товара А на пересечениях

Кварталы отсортированы в алфавитном порядке, что в данном случае нам подходит. В других ситуациях, например при использовании названий месяцев, вы можете передать методу параметр `sort=False`.

А что будет, если сразу несколько значений окажутся на пересечении года и квартала? По умолчанию метод `pivot_table` использует агрегацию `mean` (среднее значение). (В `pandas` также присутствует метод `pivot`, который не агрегирует значения и не поддерживает их дублирование. Лично я им не пользуюсь.) Для использования других агрегатных функций вы можете передать параметр `aggfunc` при вызове метода `pivot_table`. К примеру, вы могли бы подсчитать количество значений на каждом из пересечений с помощью функции `size`, как показано ниже:

```
df.pivot_table(index='quarter', ❶
               columns='year',   ❷
               values='A',       ❸
               sort=False,       ❹
               aggfunc='size')   ❺
```

- ❶ Строки (индекс) – уникальные значения из столбца с кварталами.
- ❷ Столбцы – уникальные значения из столбца с годами.
- ❸ Значения: данные из столбца A.
- ❹ Не сортировать значения.
- ❺ Применить к значениям функцию `size`.

Обратите внимание, что между функциями `size` и `count` есть разница, поскольку они по-разному обрабатывают значения `NaN`: если функция `size` принимает их в расчет, что функция `count` их не учитывает.

Результат этого выражения будет не так интересен, поскольку у нас в данных отсутствуют повторы:

year	2018	2019
quarter		
Q1	1	1
Q2	1	1
Q3	1	1
Q4	1	1

Помните, что в вашей сводной таблице будет присутствовать ровно одна строка для каждого уникального значения в первом выбранном вами столбце и одна колонка для каждого уникального значения из второго столбца. Если в каком-то из этих столбцов (или, что еще хуже, в обоих) присутствуют сотни уникальных значений, вы в результате можете получить огромную сводную таблицу. Понять и проанализировать ее будет невероятно сложно, не говоря о том, что она займет много места в памяти.

Ответы на дополнительные упражнения

Упражнение 23.1

```
df = df.reset_index('Season')
df.loc[(slice(1980,2020), 'Tennis'), 'Height'].max()
```

Вывод:

208.0

Упражнение 23.2

```
df.loc[1980:].loc[lambda df_: df_['Medal'] == 'Gold', 'City'].value_counts()
```

Вывод:

City	
Beijing	671
Rio de Janeiro	665
Athina	664
Sydney	663
London	632
Atlanta	608
Barcelona	559
Seoul	520
Los Angeles	497
Moskva	457
Sochi	202
Torino	176
Vancouver	174
Salt Lake City	162
Nagano	145
Lillehammer	110
Albertville	104
Calgary	87
Sarajevo	74
Lake Placid	72

Name: count, dtype: int64

Упражнение 23.3

```
df.loc[1980:].loc[lambda df_: (df_['Team'] == 'United States')
& (df_['Medal'] == 'Gold'), 'City'].count()
```

Вывод:

1257

УПРАЖНЕНИЕ 24. Олимпийские сводные таблицы

В этом упражнении мы снова коснемся темы Олимпийских игр, но на этот раз воспользуемся сводными таблицами, чтобы иметь возможность не только отвечать на односложные вопросы, но и сравнивать разнородную информацию в табличном виде. Сводные таблицы идеально подходят для случаев, когда вам нужно сопоставить какие-то сложные данные в двумерном формате.

Что вы должны сделать в этом упражнении.

1. Снова прочитать данные из предыдущего упражнения в датафрейм с учетом следующих требований:

- нам понадобятся только столбцы Age, Height, Team, Year, Season, Sport и Medal;
 - нас будут интересовать только игры начиная с 1980 года;
 - включим в набор данных только информацию по атлетам из следующих стран: Великобритания (Great Britain), Франция (France), США (United States), Россия (Russia), Китай (China) и Индия (India).
2. Ответить на следующие вопросы:
- каков средний возраст атлетов по странам и годам участия? Участники из какой страны в среднем были моложе соперников?
 - какой максимальный рост спортсменов был зафиксирован в каждом виде спорта на каждой Олимпиаде?
 - сколько медалей завоевали все страны в разрезе Олимпиад по годам?

Подробный разбор

Сперва необходимо создать датафрейм, на основе которого мы в дальнейшем будем строить сводные таблицы. Мы загрузим тот же файл CSV, что и в предыдущем упражнении, но с некоторыми ограничениями по строкам и столбцам.

Для начала отберем нужные нам колонки и построим датафрейм:

```
df = pd.read_csv(filename,
                  usecols=['Age', 'Height',
                           'Team', 'Year',
                           'Season', 'Sport',
                           'Medal'])
```

Обратите внимание, что мы не установили индекс в датафрейме. Причина в том, что в этом упражнении мы будем опираться в своем анализе на сводные таблицы, а не индексы. Поскольку сводные таблицы базируются на обычных столбцах датафрейма, а не на индексах, нам достаточно будет наличия индекса по умолчанию, который создается для каждого датафрейма.

Теперь необходимо избавиться от лишних строк по странам, которые нас в этом упражнении не интересуют. Мы уже знаем, как удалить строки с определенными значениями, но что делать, если нам нужно избавиться от строк, в столбце Team у которых значится одна из перечисленных стран? Мы могли бы воспользоваться в запросе оператором | (логическое ИЛИ), но в этом случае он будет довольно громоздким.

Вместо этого можно прибегнуть к помощи метода isin, который может принять список возможных значений и вернуть значение True в случае, если в столбце Team обнаружилось хотя бы одно соответствие. По моему опыту, метод isin может казаться вполне очевидным, когда только начинаешь его использовать, но нужно уметь его правильно применять.

Оставить только нужные нам страны можно следующим образом:

```
df = df.loc[df['Team'].isin(['Great Britain', 'France',
                             'United States', 'Russia',
                             'China', 'India'])]
```

Теперь пришло время избавиться от строк, предшествующих 1980 году. Это стандартная операция, которую мы выполняли уже много раз:

```
df = df.loc[df['Year'] >= 1980]
```

Далее мы можем строить сводные таблицы на основе нашего датафрейма, которые помогут ответить на поставленные вопросы. Сначала нас попросили вычислить средний возраст спортсменов по странам и годам участия. Как всегда, при создании сводной таблицы необходимо четко определиться с тем, что должно располагаться в строках, что в столбцах, а что в значениях:

- в строках (индексе) нашей сводной таблицы должны быть перечислены годы, т. е. уникальные значения из столбца Year;
- в столбцах будут страны, т. е. уникальные значения из столбца Team;
- на пересечениях строк и столбцов мы выведем средний возраст спортсменов из столбца Age.

Таким образом, мы можем построить нашу сводную таблицу так:

```
df.pivot_table(index='Year',      ❶
                columns='Team',    ❷
                values='Age')      ❸
```

- ❶ Индекс: уникальные значения из столбца Year.
- ❷ Столбцы: уникальные значения из столбца Team.
- ❸ Значения: средние значения из столбца Age.

Эти данные собраны по всем видам спорта, хотя не все страны заявляют своих участников во все соревнования. По представленным ниже данным видно, что спортсмены из Китая в среднем оказываются значительно моложе своих соперников:

Team	China	France	Great Britain	India	Russia	United States
Year						
1980	21.868421	23.524590	22.882507	25.506667	NaN	22.770992
1984	22.076336	24.369830	24.445423	24.905660	NaN	24.437118
1988	22.358447	24.520076	25.439560	24.000000	NaN	24.904977
1992	21.955752	25.140187	25.584055	24.184615	NaN	25.474866
1994	20.627907	24.601307	25.282051	NaN	24.042553	24.976744
1996	22.021531	25.296629	26.746032	24.629630	24.268116	26.273277
1998	21.784091	25.462069	27.243902	16.000000	25.435028	25.146154
2000	22.515306	25.982833	26.406948	25.400000	25.229236	26.576203
2002	23.127451	25.737805	26.833333	20.000000	25.518692	25.726316
2004	23.006122	26.139073	26.303977	24.728395	26.053356	26.439093
2006	23.457143	26.303226	26.851852	25.200000	25.745098	25.637288
2008	23.903955	26.285714	25.200969	25.402985	25.432432	26.225806
2010	23.239669	25.911458	26.147059	25.666667	25.506173	25.841584
2012	23.894168	26.606635	25.922619	25.637363	25.454713	26.461883
2014	23.400000	25.708995	25.628571	25.000000	25.842271	26.189189
2016	23.873706	27.095238	26.653191	26.100000	25.366834	26.217454

Подтвердить это можно с помощью простого вызова метода `mean` следующим образом:

```
df.mean()

Team
China          22.694375
France          25.542854
Great Britain   25.848253
India           24.157465
Russia          25.324542
United States   25.581184
dtype: float64
```

Далее нам необходимо определить, какой максимальный рост спортсменов был зафиксирован в каждом виде спорта на каждой Олимпиаде. Поскольку у нас в данных присутствует довольно много видов спорта, а самих игр – не так много, в этом случае будет уместно вынести годы в столбцы:

- в строках (индексе) нашей сводной таблицы должны быть перечислены виды спорта, т. е. уникальные значения из столбца `Sport`;
- в столбцах будут представлены годы, т. е. уникальные значения из столбца `Year`;
- на пересечениях строк и столбцов мы выведем рост спортсменов из столбца `Height`. Поскольку нас интересует максимальный рост атлетов, мы также передадим в метод параметр `aggfunc` со значением `max`.

ПРИМЕЧАНИЕ. В предыдущих версиях `pandas` было принято передавать способ агрегации значений с помощью методов `NumPy`, таких как `np.max` или `np.size`. Но сейчас предпочтительно передавать строковые значения вроде `'max'` или `'size'`, которые автоматически транслируются во внутренние функции или ссылки.

Выражение для создания этой сводной таблицы может выглядеть так:

```
df.pivot_table(index='Sport',      ❶
                columns='Year',     ❷
                values='Height',     ❸
                aggfunc='max')       ❹
```

- ❶ Индекс: уникальные значения из столбца `Sport`.
- ❷ Столбцы: уникальные значения из столбца `Year`.
- ❸ Значения: максимальные значения из столбца `Height`.
- ❹ Используем `max` в качестве функции агрегирования.

Year	1980	1984	1988	1992	...	2010	2012	2014	2016
Sport					...				
Alpine Skiing	180.0	182.0	185.0	185.0	...	193.0	NaN	193.0	NaN
Archery	183.0	188.0	188.0	191.0	...	NaN	193.0	NaN	188.0
Athletics	197.0	203.0	203.0	198.0	...	NaN	208.0	NaN	203.0
Badminton	NaN	NaN	NaN	186.0	...	NaN	201.0	NaN	201.0
Baseball	NaN	NaN	NaN	198.0	...	NaN	NaN	NaN	NaN

...	...	...	...	...	...	...	...	...	...
Triathlon	NaN	NaN	NaN	NaN	...	NaN	191.0	NaN	191.0
Volleyball	NaN	203.0	203.0	202.0	...	NaN	219.0	NaN	210.0
Water Polo	NaN	198.0	205.0	205.0	...	NaN	203.0	NaN	206.0
Weightlifting	180.0	188.0	190.0	190.0	...	NaN	192.0	NaN	187.0
Wrestling	205.0	190.0	193.0	193.0	...	NaN	203.0	NaN	200.0

[51 rows x 16 columns]

По большому количеству значений NaN мы можем понять, что информация о росте спортсменов на Олимпиадах в этом наборе данных представлена куда более скудно в сравнении с другими показателями. Это довольно распространенная ситуация для реальной жизни. Зачастую вам приходится анализировать разреженные данные, далекие от идеала.

Наконец, нас попросили узнать, сколько медалей завоевали все страны в разрезе Олимпиад по годам. Давайте по уже сложившейся традиции спланируем нашу будущую сводную таблицу:

- в строках (индексе) нашей сводной таблицы должны быть перечислены годы проведения Олимпиад, т. е. уникальные значения из столбца Year;
- в столбцах будут представлены страны, т. е. уникальные значения из столбца Team;
- нам нужно подсчитать количество медалей, а не получить среднее значение по ним (как будто это вообще возможно). Это значит, что нам необходимо воспользоваться параметром `aggfunc` со значением `size`, но перед этим мы избавимся от строк, в которых в столбце Medal стоят значения NaN, при помощи метода `dropna` с параметром `subset`.

Наш код создания сводной таблицы может выглядеть так:

```
pd.pivot_table(df.dropna(subset='Medal'), ❶
               index='Year',              ❷
               columns='Team',             ❸
               values='Medal',             ❹
               aggfunc='size')             ❺
```

- ❶ Индекс: уникальные значения из столбца Sport.
- ❷ Используем подмножество данных, где в столбце Medal – не NaN.
- ❸ Столбцы: уникальные значения из столбца Team.
- ❹ Значения: количество значений в столбце Medal.
- ❺ Используем `size` в качестве функции агрегирования.

Решение

```
filename = '../data/olympic_athlete_events.csv'
```

```
df = pd.read_csv(filename,
                  usecols=['Age', 'Height', 'Team',
                           'Year', 'Season',
                           'Sport', 'Medal']) ❶
```

```
df = df.loc[df['Team'].isin(['Great Britain', 'France',
```

```

        'United States', 'Russia',
        'China', 'India']])
df = df.loc[df['Year'] >= 1980]

df.pivot_table(index='Year', columns='Team',
               values='Age')
df.pivot_table(index='Sport',
               columns='Year', values='Height',
               aggfunc='max')
pd.pivot_table(df.dropna(subset='Medal'),
               index='Year',
               columns='Team',
               values='Medal',
               aggfunc='size')
```

- ❶ Загружаем всего семь столбцов без индекса.
- ❷ Удаляем строки по странам, которые нас не интересуют.
- ❸ Избавляемся от данных до 1980 года.
- ❹ Сводная таблица с полями Year (индекс), Team (столбцы) и средними значениями по полю Age.
- ❺ Сводная таблица с полями Sport (индекс), Year (столбцы) и максимальными значениями по полю Height.
- ❻ Сводная таблица с полями Year (индекс), Team (столбцы) и количеством значений по полю Medal.

Дополнительные упражнения

1. Постройте сводную таблицу, показывающую, сколько медалей завоевывали команды по годам, в которой в индексе будут располагаться год проведения Олимпиады и сезон.
2. Постройте сводную таблицу, показывающую максимальные возраст и рост участников Олимпиад по годам и командам.
3. Постройте сводную таблицу, показывающую максимальные возраст и рост участников Олимпиад по годам и командам с дополнительной разбивкой лет на сезоны – летний и зимний.

Ответы на дополнительные упражнения

Упражнение 24.1

```

pd.pivot_table(df.dropna(subset='Medal'),
               index=['Year', 'Season'],
               columns='Team',
               values='Medal',
               aggfunc='size')
```

Вывод:

Team	China	France	Great Britain	India	Russia	United States
Year Season						
1980 Summer	NaN	29.0	47.0	16.0	NaN	NaN
Winter	NaN	1.0	1.0	NaN	NaN	30.0

```

1984 Summer 74.0 67.0 71.0 NaN NaN 352.0
      Winter NaN 3.0 NaN NaN NaN 7.0
1988 Summer 50.0 29.0 54.0 NaN NaN 207.0
...
2008 Summer 170.0 77.0 81.0 3.0 142.0 309.0
2010 Winter 15.0 14.0 1.0 NaN 21.0 89.0
2012 Summer 117.0 78.0 122.0 6.0 138.0 238.0
2014 Winter 12.0 18.0 10.0 NaN 56.0 52.0
2016 Summer 109.0 96.0 145.0 2.0 113.0 256.0

```

[20 rows x 6 columns]

Упражнение 24.2

```

pd.pivot_table(df,
                index='Year',
                columns='Team',
                values=['Age', 'Height'],
                aggfunc='max')

```

Вывод:

	Age						Height				
Team	China	France	Great Britain	India	...	...	Great Britain	India	Russia	United States	
Year					...						
1980	32.0	34.0		37.0	56.0	...	205.0	196.0	NaN	193.0	
1984	30.0	50.0		47.0	45.0	...	203.0	188.0	NaN	213.0	
1988	34.0	43.0		55.0	36.0	...	205.0	193.0	NaN	216.0	
1992	35.0	47.0		48.0	38.0	...	205.0	188.0	NaN	216.0	
1994	29.0	33.0		36.0	NaN	...	185.0	NaN	195.0	193.0	
...	...	...		...	...	...	...	...	...	...	
2008	45.0	51.0		53.0	42.0	...	207.0	192.0	215.0	211.0	
2010	34.0	38.0		45.0	28.0	...	193.0	183.0	199.0	193.0	
2012	41.0	49.0		56.0	39.0	...	211.0	196.0	219.0	216.0	
2014	32.0	37.0		41.0	30.0	...	193.0	173.0	197.0	196.0	
2016	38.0	53.0		60.0	43.0	...	207.0	200.0	210.0	211.0	

[16 rows x 12 columns]

Упражнение 24.3

```

pd.pivot_table(df,
                index=['Year', 'Season'],
                columns='Team',
                values=['Age', 'Height'],
                aggfunc='max')

```

Вывод:

		Age						Height				
Team		China	France	Great Britain	India	...	...	Great Britain	India	Russia	United States	
Year	Season					...						
1980	Summer	NaN	34.0		37.0	56.0	...	205.0	196.0	NaN	NaN	

	Winter	32.0	29.0		36.0	NaN	...		183.0	NaN	NaN	193.0
1984	Summer	30.0	50.0		47.0	45.0	...		203.0	188.0	NaN	213.0
	Winter	28.0	36.0		34.0	NaN	...		189.0	NaN	NaN	193.0
1988	Summer	34.0	43.0		55.0	36.0	...		205.0	193.0	NaN	216.0
...	...	...	...		...	...	...		...	...	...	...
2008	Summer	45.0	51.0		53.0	42.0	...		207.0	192.0	215.0	211.0
2010	Winter	34.0	38.0		45.0	28.0	...		193.0	183.0	199.0	193.0
2012	Summer	41.0	49.0		56.0	39.0	...		211.0	196.0	219.0	216.0
2014	Winter	32.0	37.0		41.0	30.0	...		193.0	173.0	197.0	196.0
2016	Summer	38.0	53.0		60.0	43.0	...		207.0	200.0	210.0	211.0

[20 rows x 12 columns]

Заключение

В этой главе мы увидели, что индексы в датафреймах могут быть использованы не только для упорядочивания строк, но и для реорганизации данных и облегчения извлечения нужной информации. Также мы рассмотрели технику создания сводных таблиц, позволяющих легко и быстро сравнивать целые массивы данных внутри датафреймов.

Глава 5

Очистка данных

Я помню, в конце 1980-х одному моему работодателю потребовалась информация о количестве осадков в разных районах. И что он сделал? Дал мне список городов и телефонный справочник и попросил обзвонить все интересующие его города и собрать в Excel данные об осадках в предшествующий день. Сегодня получить подобного рода информацию проще простого, и для этого не нужно никуда звонить. Многие государственные учреждения предоставляют данные такого рода бесплатно, а коммерческие компании публикуют много полезной информации за деньги. Какую бы информацию вы ни запросили, она наверняка уже кем-то собрана. Остается найти ее и узнать, сколько она стоит и в каком формате предоставляется.

Также вас должно интересовать, насколько точными являются используемые вами данные. Можно легко предположить, что данные в файлах CSV, предоставляемые неким официальным сайтом, содержат достоверные сведения. Но часто в таких данных встречаются неточности, которые могут быть связаны как с человеческим фактором (люди могут ошибаться), так и с ошибками хранения данных. Кто-то мог случайно дать файлу не то имя или ввести данные не в то поле. Автоматические датчики, призванные собирать информацию, нередко выходят из строя или просто отключаются от сети. Да и серверы не вечны – на них может заканчиваться место и возникать другие неожиданные проблемы.

Все это предполагает, что у нас есть какие-то исходные данные для анализа. Но иногда этих данных просто нет.

Именно поэтому специалисты по работе с данными часто говорят, что 80 % их работы заключается в очистке этих самых данных. Что подразумевается под очисткой данных? Вот лишь несколько составляющих этого процесса:

- переименование столбцов;
- переименование индекса;
- удаление лишних столбцов;
- разделение одного столбца на два;
- объединение нескольких столбцов в один;
- удаление строк с отсутствием данных;
- удаление повторяющихся строк;
- удаление строк с пропущенными значениями (NaN);
- замена NaN конкретными значениями;
- замена NaN с применением интерполяции;
- стандартизация строк;

- исправление ошибок в строках;
- удаление пробельных символов в строках;
- корректировка типов данных в столбцах;
- определение и устранение выбросов.

Некоторые из этих задач мы решали в предыдущих главах. Но важность задачи очистки данных, позволяющей обеспечить корректность и точность выполняемого анализа, трудно переоценить.

В этой главе мы подробно рассмотрим техники очистки данных, используемые в pandas. Мы обсудим несколько способов обработки пропущенных значений, научимся сохранять как можно больше данных, даже если они изначально «зашумлены», и узнаем, как лучше понимать данные и характерные для них ограничения. Кроме того, мы рассмотрим несколько продвинутых приемов, позволяющих придать данным удобную для анализа форму.

В табл. 5.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 5.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>df.shape</code>	Возвращает кортеж из двух элементов, показывающий размерность датафрейма, т. е. количество строк и столбцов в нем	<code>df.shape</code>	http://mng.bz/8rpg (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.shape.html)
<code>len(df)</code> или <code>len(df.index)</code>	Возвращает количество строк в датафрейме	<code>len(df)</code> или <code>len(df.index)</code>	http://mng.bz/EQdr (https://stackoverflow.com/questions/15943769/how-do-i-get-the-row-count-of-a-pandas-dataframe)
<code>s.isnull</code>	Возвращает объект Series с булевыми значениями, показывающий расположение пропущенных (обычно NaN) значений в объекте <code>s</code>	<code>s.isnull()</code>	http://mng.bz/N2KX (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isnull.html#pandas.Series.isnull)
<code>s.notnull</code>	Возвращает объект Series с булевыми значениями, показывающий расположение непропущенных значений в объекте <code>s</code>	<code>s.notnull()</code>	http://mng.bz/D420 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.notnull.html#pandas.Series.notnull)
<code>df.isnull</code>	Возвращает датафрейм с булевыми значениями, показывающий расположение пропущенных (обычно NaN) значений в объекте <code>df</code>	<code>df.isnull()</code>	http://mng.bz/IWGz (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.isnull.html#pandas.DataFrame.isnull)

Таблица 5.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>df.replace</code>	Заменяет значения в одном или более столбцах	<code>df.replace('a':{'b':'c'}, 'd')</code>	http://mng.bz/Bm2q (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.replace.html)
<code>s.map</code>	Применяет функцию к каждому элементу объекта <code>s</code> , возвращая результат в виде объекта той же размерности	<code>s.map(lambda x: x**2)</code>	http://mng.bz/d1yz (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.map.html)
<code>df.fillna</code>	Заменяет значения NaN другими значениями	<code>df.fillna(10)</code>	http://mng.bz/rWRE (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.fillna.html)
<code>df.dropna</code>	Удаляет строки, в которых отсутствуют значения NaN	<code>df = df.dropna()</code>	http://mng.bz/V1gr (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html)
<code>s.str</code>	Позволяет работать с данными как со строкой	<code>df['colname'].str</code>	http://mng.bz/x4Wq (https://pandas.pydata.org/pandas-docs/stable/user_guide/text.html#string-methods)
<code>str.isdigit</code>	Возвращает объект <code>Series</code> с булевыми значениями, показывающими, какие строки содержат только числа от 0 до 9	<code>df['colname'].str.isdigit()</code>	http://mng.bz/AoAE (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.isdigit.html)
<code>pd.to_numeric</code>	Возвращает объект <code>Series</code> с целочисленными значениями или числами с плавающей точкой на основе последовательности строк	<code>pd.to_numeric(df['colname'])</code>	http://mng.bz/Zq2j (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.to_numeric.html)
<code>df.sort_index</code>	Упорядочивает строки в датафрейме на основе значений в индексе по возрастанию	<code>df = df.sort_index()</code>	http://mng.bz/RxAn (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.sort_index.html)
<code>pd.read_excel</code>	Создает датафрейм на основе файла Excel	<code>df = pd.read_excel('myfile.xlsx')</code>	http://mng.bz/wvl7 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_excel.html)

Таблица 5.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.read_csv</code>	Создает датафрейм на основе файла CSV	<code>df = pd.read_csv('myfile.csv')</code>	http://mng.bz/wvl7 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
<code>s.value_counts</code>	Возвращает отсортированный (в порядке убывающей частоты) объект Series с информацией о том, сколько раз каждое значение встречается в переменной <code>s</code>	<code>s.value_counts()</code>	http://mng.bz/1qzZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.value_counts.html)
<code>s.unique</code>	Возвращает объект Series с уникальными (т. е. не повторяющимися) значениями в объекте <code>s</code> , включая значение <code>NaN</code> (если оно присутствует в данных)	<code>s.unique()</code>	http://mng.bz/PzA2 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.unique.html)
<code>s.mode</code>	Возвращает объект Series с наиболее часто встречающимися значениями в объекте <code>s</code>	<code>s.mode()</code>	http://mng.bz/7vBm (https://pandas.pydata.org/docs/reference/api/pandas.Series.mode.html)

Много пропусков – это сколько?

Мы уже много раз видели, что датафреймы и объекты Series могут содержать пропущенные значения, или `NaN`. При анализе мы часто задаемся вопросом о том, сколько пропущенных значений находится в том или ином столбце.

Одним из способов является ручной подсчет. У объекта Series есть метод `count`, который возвращает количество непустых элементов в объекте. В совокупности с атрибутом `shape` он говорит о том, сколько именно у вас значений `NaN`, как показано ниже:

```
s.shape[0] - s.count() ❶
```

❶ Возвращает целое число, соответствующее количеству пропущенных значений.

Это довольно утомительный способ. Неужели в `pandas` нет лучшего способа для определения количества значения `NaN`? Он есть и реализуется методом `isnull`. Если вызвать его применительно к столбцу, мы получим объект Series, в котором значения `True` будут соответствовать значениям `NaN`, а `False` – остальным значениям. Далее вы можете применить к этому объекту метод `sum`, который позволит получить сумму значений `True` благодаря тому, что в Python булевы значения происходят от целочисленных и могут быть при необходимости представлены как 1 (`True`) и 0 (`False`):

```
s.isnull().sum() ❶
```

❶ Вычисляет количество значений `NaN` в объекте `s`.

Если вызвать метод `isnull` применительно к датафрейму, мы получим новый датафрейм, в котором значения `True` будут соответствовать пропущенным значениям, а значения `False` – всем остальным. Разумеется, вы, так же как и в случае с объектом `Series`, можете подсчитать общее количество значений `NaN` в датафрейме с помощью метода `sum`, как показано ниже:

```
df.isnull().sum() ❶
```

❶ Вычисляет количество значений `NaN` в датафрейме.

Наконец, метод `df.info` возвращает массу информации о датафрейме, включая имена и типы столбцов, а также сводку по количеству столбцов каждого типа и оценку расхода памяти (подробнее о расходовании памяти мы поговорим в главе 12). Если датафрейм небольшой, этот метод также предоставит вам информацию о количестве пропущенных значений в каждом столбце. Однако эти вычисления могут занять некоторое время, так что метод `df.info` будет заниматься этими расчетами только до заданного порога. Если этот порог, определяемый опцией `pd.options.display.max_info_columns`, превышен, вы можете явным образом указать `pandas` на необходимость вывода количества элементов путем передачи аргумента `show_counts=True`, как показано ниже:

```
df.info(show_counts=True) ❶
```

❶ Выводит полную информацию о датафрейме, включая количество пропущенных значений в каждом столбце.

ПРИМЕЧАНИЕ. В `pandas` определены оба метода – `isna` и `isnull` – для датафреймов и объектов `Series`. Какая между ними разница? А ее нет! Если вы взгляните в документацию по `pandas`, то обнаружите, что эти методы отличаются только именами. В этой книге я буду пользоваться методом `isnull`, но, если вы предпочитаете использовать метод `isna`, можете не изменять своим привычкам. Обратите внимание, что оба метода отличаются от функции `np.isnan`, определенной в модуле `NumPy`, лежащем в основе `pandas`. Лично я отдаю предпочтение методам, определенным в `pandas`, поскольку это обеспечивает лучшую интеграцию с другими системами. Вместо использования оператора `~` (тильда), который в `pandas` применяется для инвертирования булевых датафреймов и объектов `Series`, вы зачастую можете прибегать к помощи методов `notnull`, определенных как для `Series`, так и для датафреймов.

УПРАЖНЕНИЕ 25. Очистка данных о парковках

В главе 4 мы обращались к набору данных, в котором хранится информация о выданных парковочных талонах в Нью-Йорке в 2020 году. Но представим, что эти данные изначально заполняются полицейскими, инспекторами в местах парковки и другими ответственными людьми, которые все же могут ошибаться по своей человеческой природе. Кажется, что это незначительный момент, но он может приводить к неправильно выданным штрафным талонам, некорректной информации в базе данных и другим неточностям. Кстати, в Израиле при выписывании штрафных талонов инспектор фотографирует машину и ее регистрационный номер. Это делает появление ошибок менее вероятным, но не спасает на 100 %.

В этом упражнении мы поработаем с пропущенными значениями – главным бичом анализа данных. Мы узнаем, сколько таких значений присутствует в наборе данных и на что это может повлиять. Мы условимся, что выписанные штрафы, в которых есть пропущенные значения, могут быть аннулированы. Если вам выпишут штраф за неправильную парковку в Нью-Йорке, не используйте этот аргумент для апелляции.

Итак, что вам нужно сделать.

1. Создайте датафрейм на основе файла `nyc-parking-violations-2020.csv` с использованием только следующих столбцов:
 - Plate ID;
 - Registration State;
 - Vehicle Make;
 - Vehicle Color;
 - Violation Time;
 - Street Name.

Сколько строк окажется в датафрейме после его загрузки в память?

2. Удалите все строки, в которых присутствуют пропущенные значения. Сколько строк останется в датафрейме? Если представить, что каждый парковочный талон добавляет в среднем 100 долл. в городской бюджет, а талоны с неполными данными могут быть успешно оспорены в суде, сколько потеряет бюджет Нью-Йорка из-за этих ошибок?
3. Теперь давайте представим, что талон может быть аннулирован только в случае, если при его записи была пропущена информация о регистрационном номере (Plate ID), штате (Registration State), марке машины (Vehicle Make) и/или названии улицы (Street Name). Удалите строки из датафрейма, в которых присутствуют пропущенные значения в одном или более столбцов из перечисленных выше. Сколько строк останется? Сколько городской бюджет потеряет в этом случае?
4. Пересчитайте потери бюджета для случая, когда пропущенные значения в столбце с маркой машины (Vehicle Make) не могут являться основанием для обжалования штрафа, а все остальные столбцы, перечисленные в п. 3, могут.

Подробный разбор

Когда вы только начинаете свой путь в аналитике данных, то кажется, что вы можете безжалостно избавляться от всех строк, в которых есть пропущенные данные. В конце концов, данные ведь неполные, так зачем они нам нужны? В этом упражнении я бы хотел, чтобы вы не только сосредоточились на удалении строк из набора данных, но и прочувствовали возможные проблемы, которые могут быть с этим связаны.

Для начала загрузим информацию из файла CSV в датафрейм. Нас будут интересовать только несколько столбцов:

```
filename = '../data/nyc-parking-violations-2020.csv'

df = pd.read_csv(filename,
```

```
usecols=['Plate ID',
        'Registration State',
        'Vehicle Make',
        'Vehicle Color',
        'Violation Time',
        'Street Name']])
```

Определить количество строк в датафрейме можно с помощью первого элемента (с индексом 0) его атрибута `shape`, как показано ниже:

```
df.shape[0]
```

Но есть и более простой способ, предполагающий использование встроенной функции Python под названием `len`:

```
len(df)
```

Этот вариант не только более короткий, но и, согласно моим замерам, работает примерно вдвое быстрее первого способа. Но и это не предел. Еще быстрее получить количество строк в датафрейме можно, обратившись к его индексу следующим образом:

```
len(df.index)
```

Мои подсчеты показали, что `len(df.index)` выигрывает в скорости у `len(df)` в районе 45 %, а у `df.shape[0]` – около 65 %.

Подсчет значений

Может показаться, что метод `count` естественным образом может подходить для подсчета количества строк в датафрейме. Но с ним связана пара проблем:

- он игнорирует значения `NaN`;
- на объемных датафреймах он выполняется дольше.

Если вы хотите узнать количество всех значений, включая значения `NaN`, то можете воспользоваться атрибутом `size` (не методом), который есть как у объектов `Series`, так и у датафреймов. Также вы можете вызвать функцию `np.size` и передать ей объект `Series` так: `np.size(s)`.

Лично я для подсчета значений предпочитаю использовать вызов `len(df.index)`, который возвращает полную длину датафрейма и выполняется максимально быстро.

Теперь, когда у нас есть датафрейм, мы можем начать отвечать на поставленные вопросы относительно последствий исключения талонов, по которым заполнена не вся информация. Сначала избавимся от всех строк, содержащих по крайней мере одно пропущенное значение, с помощью метода `df.dropna`. Он возвращает новый датафрейм с той же структурой, но без значений `NaN`, что видно на рис. 5.1:

```
all_good_df = df.dropna()
```

	Plate ID	Registration State	Vehicle Make	Violation Time	Street Name	Vehicle Color
2752511	LHLP99	FL	HYUN	0230P	JACOB RIIS PARK	RED
964568	JXJ1561	PA	TOYOT	0119P	E 58th St	BLUE
5049760	S82HUN	NJ	HONDA	0846A	SB UNIVERSITY AVE @	BK
4248515	HYK8920	NY	FORD	1151A	NB PARK AVE @ E 83RD	GY
353397	KMF8349	PA	NaN	0850P	S/S SEAVIEW AVE	WHITE
2703401	XHXE40	NJ	NaN	1039A	W 43 ST	WH
1434853	TRD7943	OH	NaN	0937A	BASSETT AVE	WH
9585754	76654MK	NY	INTER	NaN	6TH AVE	RED
8915985	HJD9647	NY	ME/BE	NaN	29TH ST	WH
2868914	JHM3686	99	NaN	NaN	NaN	NaN

Рис. 5.1. Пример датафрейма с пропущенными значениями

Таким образом, если датафрейм будет состоять из одних только пропущенных значений, применение к нему метода `dropna` вернет пустой датафрейм с такой же структурой, как показано на рис. 5.2.

А сколько строк мы удалили из датафрейма? Это можно узнать следующим образом:

```
len(df.index) - len(all_good_df.index)
```

Мы получили довольно большое число: 447 359. Это порядка 3.5 % исходных данных, что кажется не так много, пока мы не ответим на следующий вопрос, касающийся потерь в бюджете Нью-Йорка при аннулировании некорректно заполненных талонов. Если предположить, что каждый такой талон обходится городу в 100 долл., получим следующую формулу:

```
(len(df.index) - len(all_good_df.index))*100
```

В итоге 44.7 млн долл. потерь. Можно воспользоваться `f`-строками, чтобы вывести результат с разделением разрядов. Для этого можно после двоеточия поставить запятую, как показано ниже:

```
f'${(len(df.index) - len(all_good_df.index) ) * 100:.,}'
```

	Plate ID	Registration State	Vehicle Make	Violation Time	Street Name	Vehicle Color
2752511	LHLP99	FL	HYUN	0230P	JACOB RIIS PARK	RED
964568	JXJ1561	PA	TOYOT	0119P	E 58th St	BLUE
5049760	S82HUN	NJ	HONDA	0846A	SB UNIVERSITY AVE @	BK
4248515	HYK8920	NY	FORD	1151A	NB PARK AVE @ E 83RD	GY
353397	KMF8349	PA	NaN	0850P	S/S SEAVIEW AVE	WHITE
2703401	XHXE40	NJ	NaN	1039A	W 43 ST	WH
1434853	TRD7943	OH	NaN	0937A	BASSETT AVE	WH
9585754	76654MK	NY	INTER	NaN	6TH AVE	RED
8915985	HJD9647	NY	ME/BE	NaN	29TH ST	WH
2868914	JHM3686	99	NaN	NaN	NaN	NaN

↓

```
dropna()
```

↓

	Plate ID	Registration State	Vehicle Make	Violation Time	Street Name	Vehicle Color
2752511	LHLP99	FL	HYUN	0230P	JACOB RIIS PARK	RED
964568	JXJ1561	PA	TOYOT	0119P	E 58th St	BLUE
5049760	S82HUN	NJ	HONDA	0846A	SB UNIVERSITY AVE @	BK
4248515	HYK8920	NY	FORD	1151A	NB PARK AVE @ E 83RD	GY

Рис. 5.2. Применение метода dropna к датафрейму

Как видите, удаление данных с пропусками из набора может дать вам ощущение уверенности в том, что оставшиеся строки заполнены правильно, но в то же время потери от этой операции могут накапливаться достаточно быстро.

Давайте снизим наши требования и избавимся только от тех строк, в которых пропущенные значения присутствуют хотя бы в одном из следующих столбцов: Plate ID, Registration State, Vehicle Make или Street Name.

Для этого можно было бы применить метод `notnull` ко всем перечисленным столбцам, вспомнив о том, что каждый из них представляет собой объект `Series`. В результате мы бы получили довольно громоздкое выражение, показанное ниже:

```
semi_good_df = df[df['Plate ID'].notnull() &
                  df['Registration State'].notnull() &
                  df['Vehicle Make'].notnull() &
                  df['Street Name'].notnull()]
```

Это сработает, но так лучше не делать. Вместо этого можно воспользоваться методом `dropna`, передав ему параметр `subset`, как мы уже делали ранее:

```
semi_good_df = df.dropna(subset=['Plate ID',
                                'Registration State',
                                'Vehicle Make',
                                'Street Name'])
```

Результат выполнения обеих операций показан на рис. 5.3.

Использование параметра `thresh` с методом `dropna`

При передаче параметра `thresh` методу `dropna` совместно с параметром `subset` мы можем указать pandas, сколько значений в переданных столбцах должны быть непущенными, чтобы строка осталась в наборе. К примеру, если мы хотим оставить строки, в которых любые три из четырех указанных столбцов содержат значимые величины, можно написать следующее выражение:

```
semi_good_df = df.dropna(subset=['Plate ID',
                                'Registration State',
                                'Vehicle Make',
                                'Street Name'],
                        thresh=3)
```

Конечно, это также означает, что в нашем результирующем наборе данных останутся пропущенные значения, но зачастую такой компромисс бывает приемлемым.

Давайте посмотрим, сколько на этот раз денег недосчитается бюджет Нью-Йорка:

```
f'${(len(df.index) - len(semi_good_df.index) ) * 100:,}
```

Мы получили сумму 6 378 500 долл. Большие деньги, но гораздо меньшие, чем в предыдущем примере.

	Plate ID	Registration State	Vehicle Make	Violation Time	Street Name	Vehicle Color
2752511	LHLP99	FL	HYUN	0230P	JACOB RIIS PARK	RED
964568	JXJ1561	PA	TOYOT	0119P	E 58th St	BLUE
5049760	S82HUN	NJ	HONDA	0846A	SB UNIVERSITY AVE @	BK
4248515	HYK8920	NY	FORD	1151A	NB PARK AVE @ E 83RD	GY
353397	KMF8349	PA	NaN	0850P	S/S SEAVIEW AVE	WHITE
2703401	XHXE40	NJ	NaN	1039A	W 43 ST	WH
1434853	TRD7943	OH	NaN	0937A	BASSETT AVE	WH
9585754	76654MK	NY	INTER	NaN	6TH AVE	RED
8915985	HJD9647	NY	ME/BE	NaN	29TH ST	WH
2868914	JHM3686	99	NaN	NaN	NaN	NaN

```
df[df['Plate ID'].notnull()
    &
    df['Registration
      State'].notnull()
    &
    df['Vehicle Make'].notnull()
    &
    df['Street Name'].notnull()]
```

	Plate ID	Registration State	Vehicle Make	Violation Time	Street Name	Vehicle Color
2752511	LHLP99	FL	HYUN	0230P	JACOB RIIS PARK	RED
964568	JXJ1561	PA	TOYOT	0119P	E 58th St	BLUE
5049760	S82HUN	NJ	HONDA	0846A	SB UNIVERSITY AVE @	BK
4248515	HYK8920	NY	FORD	1151A	NB PARK AVE @ E 83RD	GY
353397	KMF8349	PA	NaN	0850P	S/S SEAVIEW AVE	WHITE
2703401	XHXE40	NJ	NaN	1039A	W 43 ST	WH

Рис. 5.3. Применение метода `dropna` с параметром `subset` к датафрейму

Давайте еще больше снизим требования к неточностям в исходных данных и будем искать пропущенные значения только в столбцах Plate ID, Registration State и Street Name. Мы снова воспользуемся методом `df.dropna` с параметром `subset`, чтобы оставить в наборе только те строки, в которых все три эти столбца содержат значения:

```
loosest_df = df.dropna(subset=['Plate ID',
                               'Registration State',
                               'Street Name'])
```

В результате мы удалили всего 1618 строк из датафрейма. А сколько денег при этом потеряет бюджет?

```
f'${(len(df.index) - len(loosest_df.index) ) * 100:},'
```

Получилось всего 161 800 долл., что на несколько порядков меньше, чем было изначально.

Решение

```
filename = '../data/nyc-parking-violations-2020.csv'
```

```
df = pd.read_csv(filename,
                  usecols=['Plate ID',
                           'Registration State',
                           'Vehicle Make',
                           'Vehicle Color',
                           'Violation Time',
                           'Street Name'])
```

```
all_good_df = df.dropna()
len(df.index) - len(all_good_df.index)
f'${(len(df.index) - len(all_good_df.index) ) * 100:},'
```

```
semi_good_df = df.dropna(subset=['Plate ID',
                                  'Registration State',
                                  'Vehicle Make',
                                  'Street Name'])
len(df.index) - len(semi_good_df.index)
f'${(len(df.index) - len(semi_good_df.index) ) * 100:},'
```

```
loosest_df = df.dropna(subset=['Plate ID',
                               'Registration State',
                               'Street Name'])
len(df.index) - len(loosest_df.index)
f'${(len(df.index) - len(loosest_df.index) ) * 100:},'
```

- ❶ Читаем содержимое нескольких колонок из файла CSV в датафрейм.
- ❷ Удаляем все строки, содержащие хотя бы одно значение NaN.
- ❸ Сколько строк мы удалили?
- ❹ Используем f-строки для вывода суммы потерь в бюджете.
- ❺ Удаляем строки, содержащие значение NaN хотя бы в одном из четырех столбцов.

- ⑥ Сколько строк мы удалили теперь?
- ⑦ Снова выведем на экран сумму потерь.
- ⑧ Удаляем строки, содержащие значение NaN хотя бы в одном из трех столбцов.
- ⑨ Сколько строк мы удалили на этот раз?
- ⑩ Финальный вывод суммы потерь в бюджете.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/6nlo>.

Дополнительные упражнения

1. До сих пор мы указывали, какие столбцы нужно проверять на пропущенные значения. Но иногда бывает допустимо, чтобы значения NaN оставались в наборе при условии, что их будет не так много. Сколько строк вы исключите из нашего датафрейма, если от вас потребуется, чтобы по крайней мере три значения в столбцах Plate ID, Registration State, Vehicle Make и Street Name были непущенными?
2. В каком из загруженных в датафрейм столбцов присутствует наибольшее количество пропущенных значений? Есть в этом какая-то проблема?
3. Пропущенные значения – это плохо, но есть и не очень показательные значения. Пример такого значения – BLANKPLATE в столбце с регистрационными номерами. Преобразуйте эти значения в NaN и снова запустите запрос из пункта 1.

Объединение и разделение столбцов

При очистке данных нам зачастую требуется создавать один столбец на основании нескольких и, наоборот, разделять один столбец на несколько. К примеру, в упражнении 8 мы видели, как можно создать столбец с именем `current_net` путем расчета чистой цены товаров и умножения полученной величины на количество проданных единиц:

```
df['current_net'] = ((df['retail_price'] -
                    df['wholesale_price']) * df['sales'])
```

Это не совсем из области очистки данных, но подобные операции позволяют сделать данные в наборе более понятными и очевидными. Кроме того, так мы можем создать себе задел на будущее в плане обнаружения ошибок в данных.

Я всегда говорил своим детям, когда они изучали математику в школе, что одним из важнейших качеств в этой дисциплине является умение находить способы преобразования задач для облегчения их понимания и решения. То же касается и структур данных в программировании, и анализа данных, где всегда можно создать новый столбец, облегчающий процесс понимания данных в целом.

Но, пожалуй, еще чаще мы сталкиваемся с необходимостью разделить какой-то сложный столбец на два и более простых столбцов. К примеру, иногда может понадобиться разделить столбец с типом `float64` на два целочисленных столбца, в одном из которых будут храниться целые части чисел, а в другом – дробные.

То же касается и сложных структур данных, о которых мы будем подробно говорить в главах 9 и 10. Давайте рассмотрим распространенный пример, когда у нас есть некие текстовые данные, из которых нам необходимо по определенным правилам извлечь подстроки. В обычной программе на Python мы обычно для этого используем срезы, как показано ниже:

```
s = '00:11:22'
print(s[3:5]) # напечатает '11'
```

Помните, что в Python срезы записываются в формате [начало:конец+1] с индекса, начинающегося с нуля. Так что для извлечения символов, стоящих на позициях 3 и 4, нам потребовалось написать срез 3:5, что означает «начиная читать с четвертого символа и до шестого, не включая его».

А что, если `s` – это не одна строка, а объект `Series`, содержащий строки? В этом случае для извлечения срезов 3:5 из каждого значения нам придется воспользоваться атрибутом доступа `str` объекта `Series` и вызвать его метод `slice`. Синтаксис получился несколько далеким от стандартного Python, но все же он должен быть вам понятен:

```
s.str.slice(3,5)
```

Результатом этого выражения будет новый объект `Series` той же длины, что и `s`, состоящий из двухбуквенных строк, представляющих собой срезы значений из исходной последовательности.

При разборе и анализе данных вам зачастую придется подобным образом извлекать разные составляющие столбцов и сохранять их в датафрейме отдельно. Это позволяет облегчить понимание данных, избавиться от сложных столбцов, сэкономить память и повысить эффективность вычислений.

Ответы на дополнительные упражнения

Упражнение 25.1

```
at_least_three_df = df.dropna(subset=['Plate ID', 'Registration State',
                                       'Vehicle Make', 'Street Name'],
                              thresh=3)
df.shape[0] - at_least_three_df.shape[0]
```

Вывод:

253

Упражнение 25.2

```
# Чаще всего пропущенные значения встречаются в поле с цветом машины
# Вряд ли в этом есть проблема, поскольку у нас почти всегда есть
# регистрационный номер
df.isnull().sum()
```

Вывод:

Plate ID

202

```

Registration State      0
Vehicle Make           62420
Violation Time          278
Street Name            1417
Vehicle Color          391982
dtype: int64

```

Упражнение 25.3

```

# Используем метод df.replace для замены BLANKPLATE на NaN, затем удаляем
# строки, в которых в трех из четырех столбцов не стоят значимые величины
no_blankplate_df = df.replace({'Plate ID': 'BLANKPLATE'}, np.nan).
dropna(subset=['Plate ID',
               'Registration State', 'Vehicle Make', 'Street Name'],
       thresh=3)
df.shape[0] - no_blankplate_df.shape[0]

```

Вывод:

944

УПРАЖНЕНИЕ 26. Уход знаменитостей

Зачастую, как в предыдущем упражнении, лишь небольшая часть данных оказывается нечитаемой, отсутствующей или поврежденной. Но иногда приходится сталкиваться с весьма проблемными наборами данных. И чтобы эффективно их обработать, нужно не только удалить из них некачественные данные, но и как-то сохранить, или даже спасти, отрывки ценной информации.

В этом упражнении мы поработаем с довольно мрачным набором данных, в котором хранится список знаменитостей, ушедших из жизни в 2016 году и дата смерти которых была отражена в «Википедии» вместе с краткой биографией и причиной смерти. Проблемой является то, что в этом наборе данных достаточно много пропущенных и ошибочных данных, которые могут помешать нам при работе с ним.

В этом упражнении мы постараемся определить средний возраст знаменитостей, ушедших из жизни с февраля по июль 2016 года. Для этого нам понадобится сделать следующее.

1. Создать датафрейм на основе файла `celebrity_deaths_2016.csv`. Для этого упражнения нам понадобятся всего два столбца:
 - `dateofdeath`;
 - `age`.
2. Создать новый столбец с именем `month`, в котором будет содержаться номер месяца из даты смерти.
3. Установить индекс на столбце `month`.
4. Отсортировать датафрейм по индексу.
5. Убрать нечисловые значения из столбца `age`.

6. Привести столбец `age` к целочисленному типу.
7. Найти средний возраст знаменитостей, ушедших из жизни в этот период.

Поиск чисел в строках

Обычно строковые колонки переводятся в числовые следующим образом:

```
df['colname'] = df['colname'].astype(np.int64)
```

Однако в этом случае вы получите ошибку, если какое-то из значений в столбце `df['colname']` не удастся преобразовать в целое число. Это может произойти при наличии пустых или нечисловых значений в столбце.

Но мы можем узнать, какие из значений в столбце гарантированно могут быть представлены в виде числа, с помощью метода `isdigit` атрибута доступа `str`, как показано ниже:

```
df['colname'].str.isdigit()
```

Этот метод возвращает объект `Series` со значениями `True`, соответствующими строковым значениям в оригинале, которые могут быть преобразованы в число, а `False` – тем, которые не могут. Полученный объект в дальнейшем можно применить к исходному столбцу в качестве маски. Эта техника бывает очень полезна при работе с «грязными» данными, как в нашем случае.

Подробный разбор

В этом упражнении мы создадим датафрейм из двух столбцов и произведем его очистку. При этом каждый из двух столбцов нужно будет очищать по-своему, чтобы ответить на поставленный вопрос о среднем возрасте знаменитостей, ушедших из жизни с февраля по июль 2016 года.

Начнем, как и всегда, с загрузки данных из файла CSV в датафрейм. Нам понадобятся всего два столбца, так что код загрузки может выглядеть так:

```
filename = '../data/celebrity_deaths_2016.csv'
```

```
df = pd.read_csv(filename,
                  usecols=['dateofdeath', 'age'])
```

Теперь мы можем приступить к задаче очистки датафрейма.

Поскольку нас интересуют только случаи смерти в означенный период времени, а именно в заданные месяцы, будет удобно выделить из столбца `dateofdeath`, содержащего полную дату, только номер месяца для дальнейшей фильтрации по нему. Для решения подобной задачи есть и другие способы, которые мы обсудим в главе 9. Столбец `dateofdeath` является строковым, а значит, мы можем воспользоваться методом `slice` его атрибута доступа `str` для получения подстроки. Давайте извлечем нужные нам цифры месяца следующим образом:

```
df['dateofdeath'].str.slice(5,7)
```

Мы можем выделить полученные значения в столбец с именем `month`, как показано на рис. 5.4:

```
df['month'] = df['dateofdeath'].str.slice(5,7)
```

	dateofdeath	age	month		dateofdeath
1277	2016-03-03	82	03		2016-03-03
5555	2016-11-02	61	11		2016-11-02
1022	2016-02-19	80	02		2016-02-19
3302	2016-06-21	87	06	← <code>str.slice(5,7)</code> ←	2016-06-21
2214	2016-04-19	87	04		2016-04-19
4890	2016-09-23	96	09		2016-09-23
48	2016-01-03	83	01		2016-01-03
751	2016-02-04	94	02		2016-02-04
1106	2016-02-24	86	02		2016-02-24
3915	2016-07-26	85	07		2016-07-26

Рис. 5.4. Добавление столбца month в датафрейм на основе среза из столбца dateofdeath

Обратите внимание, что мы не преобразовывали столбец в целочисленный. Мы могли бы это сделать, но ведущий ноль в номерах месяцев может нам помешать. К тому же нам нет необходимости делать это, поскольку данных у нас немного, и нам не нужно беспокоиться об экономии памяти.

Теперь, когда у нас есть столбец с месяцами, преобразуем его в индекс:

```
df = df.set_index('month')
```

Далее необходимо отсортировать набор данных по индексу, что означает расположение строк в порядке возрастания значений в столбце индекса. Это нам нужно для дальнейшего извлечения среза. А когда значения в индексе повторяются, его стоит упорядочить. Сделаем это следующим образом:

```
df = df.sort_index()
```

Итак, мы готовы извлекать данные за конкретные месяцы или интервалы месяцев. Но мы еще не все сделали, ведь нас интересует средний возраст знамени-

тостей, умерших в 2016 году. Для этого нам понадобится привести столбец `age` к числовому типу, скорее всего, к типу `int`. Это можно было бы сделать так:

```
df['age'] = df['age'].astype(np.int64)
```

Но это нам не удастся по двум причинам. Во-первых, в нашем столбце с возрастом могут присутствовать строковые составляющие, а во-вторых, в столбце есть пропущенные значения (`NaN`), которые, как мы знаем, обладают типом `float` и не могут быть преобразованы в целые числа. Но сначала давайте узнаем, сколько пропущенных значений у нас есть. Для этого мы можем воспользоваться последовательностью методов `isnull().sum()` и разделить результат на количество строк в датафрейме, чтобы узнать долю пропущенных значений:

```
df['age'].isnull().sum() / len(df['age'])
```

Мы получили результат 0.004, или 0.4 % значений `NaN`. Таким количеством строк мы можем пожертвовать без сожалений. Давайте избавимся от пропущенных значений в столбце `age`:

```
df = df.dropna(subset=['age'])
```

Заметьте, что мы снова воспользовались параметром `subset`. Не то чтобы в нашем индексе присутствуют пропущенные значения, но, как вы знаете, явное всегда лучше, чем неявное.

Как можно избавиться от других загрязнений в наших данных, которые характеризуются нечисловыми значениями? Один из способов состоит в том, чтобы воспользоваться методом `str.isdigit`, возвращающим `True`, если значение непустое и содержит только цифры. Стоит помнить, что этот метод вернет `False` в случае наличия в строке знака – (минус) или десятичной точки, так что он подойдет не всегда, но для данных о возрасте – вполне. Применим этот метод к столбцу `df['age']` следующим образом:

```
df['age'].str.isdigit()
```

Полученный объект `Series` мы используем в качестве маски для удаления из датафрейма строк, в которых значения в столбце с возрастом не могут быть преобразованы в число:

```
df = df[df['age'].str.isdigit()]
```

Однако, как и всегда, в `pandas` на этот случай припасено более элегантное решение, предполагающее использование функции `pd.to_numeric`. Эта функция верхнего уровня библиотеки `pandas` служит для создания нового объекта `Series` числового типа. При этом если ей не удастся перевести значения в целочисленные, она пытается перевести их в значения с плавающей точкой:

```
df['age'] = pd.to_numeric(df['age'])
```

Получается, что мы можем вовсе не использовать метод `str.isdigit`, а удовлетвориться одной лишь функцией `pd.to_numeric`. Дело в том, что при невозможности преобразовать значение в целое или дробное число эта функция по умол-

чанию выбрасывает исключение. Но если передать ей дополнительный параметр `errors='coerce'`, исключение возбуждаться не будет, а вместо этого все «грязные» значения будут преобразовываться в `NaN`. Таким образом, мы можем написать следующее выражение:

```
df['age'] = pd.to_numeric(df['age'], errors='coerce')
```

Прежде чем продолжать, давайте посмотрим, что у нас получилось, с помощью метода `describe`:

```
df['age'].describe()
```

Ниже приведен результат:

```
count    6505.000000
mean      100.960338
std       413.994127
min        7.000000
25%       69.000000
50%       81.000000
75%       89.000000
max      9394.000000
Name: age, dtype: float64
```

Не знаю, как вам, а мне лично средний возраст 100 лет кажется немного подозрительным. Да и максимальный возраст 9394 года для человека как-то многовато, даже если он регулярно занимается спортом. Дело в том, что в одной из строк датафрейма оказалось строковое значение 9394, которое функция `pd.to_numeric` послушно перевела в число, ее ничто не смутило.

Давайте установим искусственный порог для возраста на уровне 120 лет, как показано ниже:

```
df = df.loc[df['age'] < 120]
```

Теперь мы готовы к ответу на поставленный вопрос о среднем возрасте знаменитостей, ушедших из жизни с февраля по июль:

```
df.loc['02':'07', 'age'].mean()
```

Поскольку значения в индексе мы не стали приводить к целым числам, мы воспользовались строковым срезом от '02' до '07'. В результате мы получили средний возраст 77.1788, что уже больше похоже на правду.

Решение

```
filename = '../data/celebrity_deaths_2016.csv'

df = pd.read_csv(filename,
                  usecols=['dateofdeath', 'age']) ❶

df['month'] = df['dateofdeath'].str.slice(5,7)    ❷
df = df.set_index('month')                       ❸

df = df.sort_index()                             ❹
```



```
df = df.dropna(subset=['age']) 5
df['age'] = pd.to_numeric(df['age'], errors='coerce') 6
df.loc['02':'07', 'age'].mean() 7
```

- ❶ Загружаем данные из файла CSV в датафрейм с двумя столбцами.
- ❷ Создаем новый столбец с месяцем.
- ❸ Преобразуем созданный столбец в индекс.
- ❹ Сортируем датафрейм по индексу.
- ❺ Избавляемся от значений NaN в возрасте.
- ❻ Переводим столбец с возрастом в числовой тип.
- ❼ Получаем средний возраст с февраля по июль.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/or7d>.

Дополнительные упражнения

- Добавьте в датафрейм новый столбец с именем `day`, содержащий дни месяца. Далее создайте множественный индекс по столбцам `month` и `day`. Рассчитайте средний возраст знаменитостей, ушедших из жизни в период с 15 февраля по 15 июля.
- В нашем файле CSV содержится еще один столбец с именем `causeofdeath` (причина смерти). Загрузите его в датафрейм и определите пять наиболее частых причин смерти знаменитостей. Теперь замените все значения NaN в этом столбце на строку `'unknown'` и снова выведите пять самых частых причин ухода из жизни.
- Если вам скажут, что в десять самых распространенных причин смерти входит рак (`cancer`), что вы на это скажете? Можете дать более детальный ответ, основываясь на имеющихся данных?

Ответы на дополнительные упражнения

Упражнение 26.1

```
# Получаем номер месяца с помощью среза [5:7]
df['month'] = df['dateofdeath'].str.slice(5,7)

# Получаем день месяца с помощью среза [8:]
df['day'] = df['dateofdeath'].str.slice(8,None)

# Устанавливаем множественный индекс
df = df.set_index(['month', 'day'])

# Сортируем датафрейм по индексу
df = df.sort_index()

# Получаем строки с 15 февраля по 15 июля, берем возраст и усредняем его
df.loc[('02', '15'):( '07', '15'), 'age'].mean()
```

Вывод:

```
77.05183037332367
```

Упражнение 26.2

```
filename = '../data/celebrity_deaths_2016.csv'

df = pd.read_csv(filename,
                  usecols=['dateofdeath', 'age', 'causeofdeath'])
```

```
# Получаем пять частых причин смерти
df['causeofdeath'].value_counts().head()
```

Вывод:

```
causeofdeath
cancer                248
heart attack          125
traffic collision      56
lung cancer           51
pneumonia             50
Name: count, dtype: int64
```

```
# Заменяем значения NaN на 'unknown'... и получим более 5000 таких строк
# Что ж, этот набор не особо надежен в плане анализа причин смерти знаменитостей
df['causeofdeath'] = df['causeofdeath'].fillna('unknown')
df['causeofdeath'].value_counts().head()
```

Вывод:

```
causeofdeath
unknown              5008
cancer               248
heart attack         125
traffic collision     56
lung cancer          51
Name: count, dtype: int64
```

Упражнение 26.3

```
# Как мы видим, среди причин смерти есть как рак, так и отдельно
# рак легких (lung cancer) и рак поджелудочной железы (pancreatic cancer).
```

```
# Невозможно сказать, что означает запись просто о раке, - это другие типы
# рака или сюда могли войти прочие заболевания.
```

```
# В целом этот набор данных можно назвать довольно поучительным, поскольку
# он не слишком информативен, по крайней мере в плане определения причин
# смерти звезд. Чтобы делать какие-то серьезные выводы, нам потребуются
# более надежные сведения.
df['causeofdeath'].value_counts().head(10)
```

Вывод:

```
causeofdeath
unknown          5008
cancer           248
heart attack     125
traffic collision  56
lung cancer      51
pneumonia        50
heart failure    49
shot             42
stroke           36
pancreatic cancer 35
Name: count, dtype: int64
```

УПРАЖНЕНИЕ 27. «Титаник» и интерполяция

При встрече с пропущенными значениями у нас есть три основных варианта:

- удалить их;
- оставить их в наборе данных;
- заменить их на другие значения.

Как выбрать правильный вариант? Конечно, это зависит от ситуации. Если, к примеру, вы готовите исходные данные для модели машинного обучения, вам необходимо будет избавиться от значений NaN либо путем их удаления, либо с помощью замены их альтернативными значениями. С другой стороны, если вы составляете отчет о продажах, то пропущенные значения могут вас и не волновать, поскольку они не скажутся на итоговых цифрах. И конечно, эти подходы подразумевают массу вариантов реализации.

При выборе третьего варианта (с заменой пропущенных значений) перед вами непременно встанет новый вопрос: а на что их менять? На некое заранее выбранное значение? Или на расчетную величину на основе данных в датафрейме? Или нужно делать какие-то вычисления в самом столбце? Каждый из этих вариантов предусматривает какой-то свой сценарий применения.

В этом упражнении мы потренируемся замещать пропущенные значения в знаменитом – и снова не самом оптимистичном – наборе данных, посвященном событиям на пароходе «Титаник». Некоторые столбцы в этом наборе данных заполнены полностью, другие имеют пропуски. Вы сами можете решить, как именно обойтись с пропущенными значениями. В упражнении 13 мы уже применяли метод `interpolate`, позволяющий выполнять заполнение автоматически.

В этом упражнении я хочу, чтобы вы сделали следующее.

1. Загрузите данные из файла `titanic3.xls` в датафрейм. Обратите внимание, что это файл Excel, так что вместо функции `read_csv` вам следует воспользоваться функцией `read_excel`.
2. Определите, в каких столбцах присутствуют пропущенные значения.
3. Применительно к каждому столбцу, содержащему пропущенные значения, решите, как вы будете его заполнять: некими фиксированными значениями или расчетными.

В отличие от большинства упражнений в этой книге данное упражнение не предполагает наличия единственно правильного ответа. Конечно, есть разные техники заполнения пустот, включая среднее значение, медиану и моду, но мне важно, чтобы вы не просто сделали это, а разобрались в данных и аргументировали, почему выбрали тот или иной способ.

Подробный разбор

Это не только практическое упражнение, но и в какой-то степени философское. Причина в том, что зачастую нет единственно правильного ответа на вопрос о том, что делать с пропущенными значениями. Я часто люблю повторять своим корпоративным студентам, что вы должны знать свои данные, что подразумевает понимание того, как эти данные будут анализироваться и использоваться. Это путь проб и ошибок – для одного случая ваш метод заполнения может оказаться приемлемым, а для другого – ошибочным.

По этой причине бывает полезно выполнять стоящие перед вами задачи в Jupyter Notebook или другой схожей системе, позволяющей при необходимости восстановить исходные данные и действия.

Давайте вместе пройдем по всем шагам в этом упражнении и посмотрим, какие решения здесь могут быть приемлемыми, а заодно я предложу вам свой вариант. Для начала создадим датафрейм на основе файла Excel с именем `titanic3.xls`, воспользовавшись функцией `read_excel`:

```
filename = '../data/titanic3.xls'
df = pd.read_excel(filename)
```

ПРИМЕЧАНИЕ. Подобно функции `read_csv`, `read_excel` также представляет собой функцию библиотеки `pandas` высшего уровня. И причина та же, состоящая в том, что мы не модифицируем уже имеющийся датафрейм, а создаем новый. И так же, как у функции `read_csv`, у функции `read_excel` есть знакомые вам параметры `index_col`, `usecols` и `names`, позволяющие выбрать только нужные вам столбцы, определить их имена и задать индекс.

Теперь, когда мы создали датафрейм, можно проверить его на присутствие пропущенных значений. Мы сделаем это двумя разными способами. Сначала воспользуемся последовательностью методов `isnull().sum()`, чтобы узнать, какие столбцы сколько значений NaN насчитывают:

```
df.columns[df.isnull().sum() > 0]
```

В результате получим следующие столбцы:

```
Index(['age', 'fare', 'cabin', 'embarked',
       'boat', 'body', 'home.dest'],
      dtype='object')
```

Обратите внимание, что имена столбцов здесь хранятся в объекте типа `Index`, который работает похоже на объект `Series`. Мы также можем воспользоваться последовательностью методов `isnull().sum()` применительно ко всему датафрейму, чтобы узнать, в каких столбцах сколько значений NaN присутствует:

```
df.isnull().sum()
```

Результат показан ниже, а процесс отображен графически на рис. 5.5:

```
pclass      0
survived    0
name        0
sex         0
age        263
sibsp       0
parch       0
ticket      0
fare        1
cabin     1014
embarked     2
boat        823
body       1188
home.dest   564
dtype: int64
```

	name	age
206	Minahan, Dr. William Edward	44.0
945	Lam, Mr. Ali	NaN
1156	Rosblom, Miss. Salli Helena	2.0
1183	Salonen, Mr. Johan Werner	39.0
98	Douglas, Mrs. Walter Donald (Mahala Dutton)	48.0

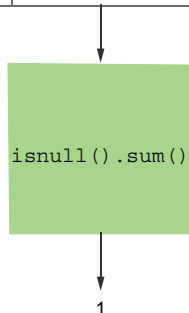


Рис. 5.5. Нахождение количества пропущенных значений в столбцах путем суммирования результатов метода `isnull()`

Решение о том, что делать с конкретными колонками, содержащими пропущенные значения, зависит от множества факторов, включая тип данных столбца. Еще одним значимым фактором является количество пропущенных значений в столбце. К примеру, если в столбце всего одно или два пропущенных значения, как в случае со столбцами `fare` и `embarked`, мы обычно можем без сожалений расстаться с этими строками. Сделать это можно следующим образом (см. рис. 5.6):

```
df = df.dropna(subset=['fare', 'embarked'])
```

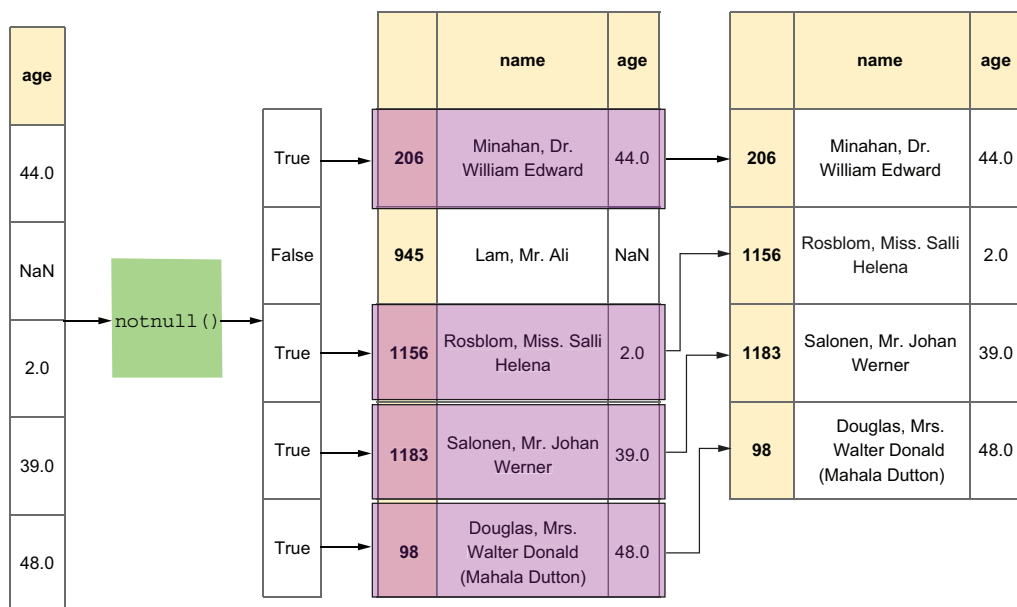


Рис. 5.6. Удаление строк, содержащих в определенном столбце значение NaN

Если же мы говорим о столбце `age`, то здесь такой радикальный подход не годится. В данном случае можно воспользоваться для заполнения средним значением, а можно и модой. Также можно применять и более сложные алгоритмы, например считать среднее значение по какому-то определенному типу каюты. Можно даже выбрать возраст из равномерного распределения, построенного на основании всех возрастов пассажиров «Титаника».

Использование среднего значения в отношении возраста выглядит вполне разумно. В этом случае среднее не изменится, хотя несколько снизится стандартное отклонение. Конечно, это не идеальное решение, но и не худшее. В случае с наборами данных по продажам товаров средние значения могут сработать лучше, особенно при наличии однородных позиций со схожей историей продаж.

Итак, с пропущенными значениями в столбце `age` мы определились, заменим их так (см. рис. 5.7):

```
df['age'] = df['age'].fillna(df['age'].mean())
```

Давайте разберем это выражение справа налево.

1. Вычисляем среднее значение по столбцу: `df['age'].mean()`. По умолчанию pandas игнорирует значения NaN, а значит, расчет среднего будет основываться только на непустых значениях в этом столбце. В результате мы получим число 29.8811345124283.
2. Применяем метод `fillna` к столбцу `df['age']`, заменяя пропущенные значения на вычисленное ранее среднее значение возраста. Немного сбивает с толку двойное использование выражения `df['age']`. Результатом применения метода `fillna` будет новый объект Series, аналогичный `df['age']`, но с заполненными значениями NaN числами 29.8811345124283, полученными на предыдущем шаге.
3. Присваиваем новый объект Series столбцу `df['age']`, тем самым заменяя его.

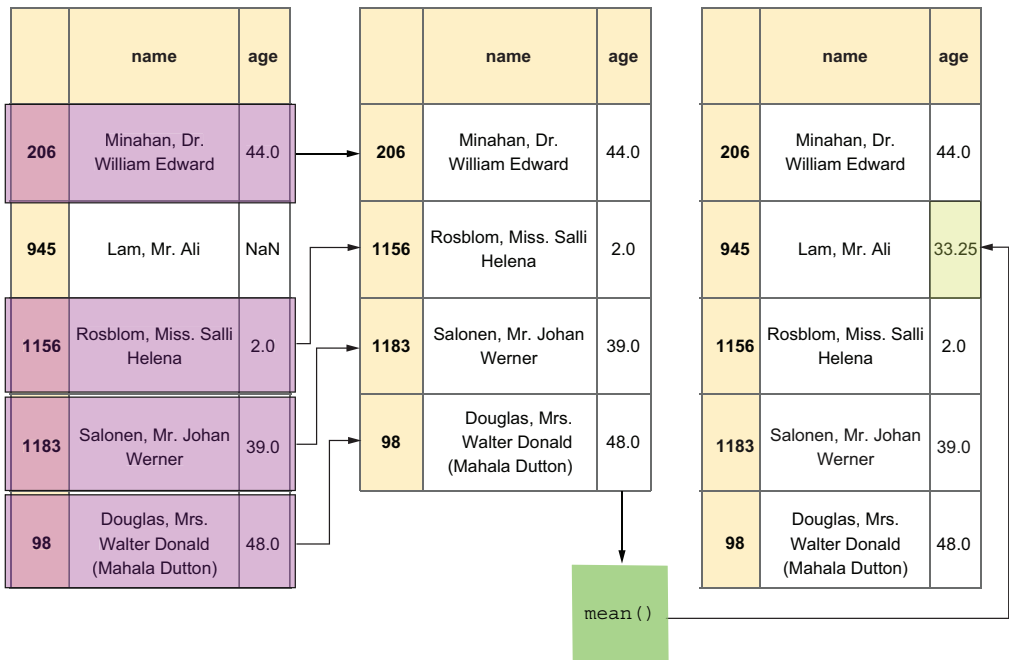


Рис. 5.7. Замена значений NaN в столбце age средним значением по этому столбцу

Со столбцом `home.dest` мы проделаем то же самое, что и со столбцом `age`, но на этот раз вместо среднего значениями воспользуемся модой, т. е. наиболее часто встречающимся значением в столбце. Сделаем мы это по двум причинам. Во-первых, среднее значение может быть вычислено только на основе числового столбца, а столбец `home.dest` носит категориальный характер. Также логично будет предположить, что, если мы не знаем, куда направлялся пассажир, можно сделать вывод, что он плыл туда же, куда и большинство из них. Конечно, мы можем ошибиться, но это лучший вариант в данном случае. Если вы хотите, вы можете пойти дальше и выбрать моду только среди пассажиров, севших на пароход в том же городе, но мы не будем так все усложнять.

В результате наш код для этой колонки будет выглядеть весьма похоже на преобразование столбца `age`, за исключением примененного метода. К тому же, поскольку метод `mode` всегда возвращает объект `Series`, мы должны извлечь из результата первый элемент, воспользовавшись нотацией `[0]`, а не просто передать его методу `fillna`:

```
df['home.dest'] = df['home.dest'].fillna(df['home.dest'].mode()[0])
```

Давайте разберемся, что здесь происходит.

1. Рассчитываем моду по столбцу `home.dest`: `df['home.dest'].mode`. В результате мы получим наиболее популярное значение из указанного столбца. Еще один способ получить это значение – `df['home.dest'].value_counts().index[0]`. Здесь мы смотрим, сколько раз встречается каждое значение, берем индекс (список уникальных значений) и извлекаем из него первое значение, т. е. самый популярный элемент.
2. Получив самое популярное место назначения, мы передаем его в качестве аргумента в метод `fillna`, вызванный у объекта `df['home.dest']`, тем самым заменяя пропущенные значения.
3. Метод `fillna` возвращает объект `Series`, который мы присваиваем столбцу `df['home.dest']`, заменяя его.

Решение

```
filename = '../data/titanic3.xls'
```

```
df = pd.read_excel(filename) ❶
```

```
df.columns[df.isnull().sum()>0] ❷
```

```
df.isnull().sum() ❸
```

```
df['age'] = df['age'].fillna(df['age'].mean()) ❹
```

```
df = df.dropna(subset=['fare', 'embarked']) ❺
```

```
df['home.dest'] = df['home.dest'].fillna(df['home.dest'].mode()[0]) ❻
```

- ❶ Загружаем все колонки из Excel.
- ❷ Определяем столбцы со значениями NaN.
- ❸ Показываем, сколько пропущенных значений есть в каждом столбце.
- ❹ Заменяем значения NaN в столбце `age` на средний возраст.
- ❺ Удаляем строки, содержащие пропущенные значения в столбцах `fare` или `embarked`.
- ❻ Заменяем значения NaN в столбце `home.dest` на моду.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/n17a>.

Дополнительные упражнения

В этих дополнительных упражнениях мы сделаем кое-что, о чем я упоминал раньше, а именно заменим пропущенные значения в столбце `home.dest` на наиболее часто встречающееся значение среди пассажиров с таким значением в столбце `embarked` (место посадки).

1. Создайте объект Series с именем `most_common_destinations`, в котором индексом будут служить уникальные значения из столбца `embarked`, а значения будут содержать самое популярное место назначения для каждого значения из столбца `embarked`.
2. Замените значения `NaN` в столбце `home.dest` на значения из столбца `embarked` (поскольку значения в столбцах `embarked` и `home.dest` отличаются, это нормальный промежуточный шаг).
3. Воспользуйтесь объектом `most_common_destinations` для замены пропущенных значений в столбце `home.dest` на наиболее популярные значения для каждой точки посадки.

Ответы на дополнительные упражнения

Упражнение 27.1

```
most_common_destinations = Series([], dtype=object)

for embarked_value in df['embarked'].dropna().unique():
    most_common_destinations.loc[embarked_value] =
df.loc[df['embarked']==embarked_value, 'home.dest'].value_counts().index[0]

most_common_destinations
```

Вывод:

```
S      New York, NY
C      New York, NY
Q  Ireland Chicago, IL
dtype: object
```

Упражнение 27.2

```
df['home.dest'] = df['home.dest'].fillna(df['embarked'])
```

Упражнение 27.3

```
df['home.dest'] = df['home.dest'].replace(most_common_destinations)
```

УПРАЖНЕНИЕ 28. Несогласованные данные

Пропущенные значения – это одна из главных, но не единственная проблема, присущая наборам данных, с которыми приходится работать. Также очень часто мы сталкиваемся с несогласованными, или неконсистентными, данными, когда одно и то же значение может быть представлено по-разному.

Такое бывает повсеместно. К примеру, я помню, что при работе над университетским проектом фандрайзинга мне пришлось считывать информацию из их базы данных, которой было черт знает сколько лет и в которой была полная неразбериха. В частности, в столбце со странами США могли быть представлены следующими значениями:

- ☐ United States of America;
- ☐ USA;
- ☐ U.S.A.;
- ☐ U.S.A;
- ☐ United States;
- ☐ US;
- ☐ U.S.

Человеку не составит никакого труда понять, что все эти значения указывают на одну и ту же страну, но компьютер этого знать не может. При несогласованности данных их бывает очень трудно обрабатывать и анализировать. Но зачастую приходится работать именно с такими источниками, и от их очистки, или нормализации, бывает просто не уйти.

В этом упражнении мы снова обратимся к набору данных со штрафными парковочными талонами и попробуем сделать его более согласованным, а значит, и более легким для анализа (но я уверен, что и после выполнения этого задания в этом наборе останется масса неточностей). Вот что вы должны сделать.

1. Создать датафрейм на основе файла `nyc-parking-violations-2020.csv`. Нам потребуется всего несколько столбцов:
 - ☐ Plate ID;
 - ☐ Registration State;
 - ☐ Vehicle Make;
 - ☐ Vehicle Color;
 - ☐ Street Name.
2. Определить, сколько разных цветов машин (столбец `Vehicle Color`) присутствует в наборе данных.
3. Вывести 30 самых распространенных цветов и попытаться выявить одинаковые цвета, написанные по-разному. К примеру, цвет `WHITE` может быть также записан как `WT`, `wt` и `WNT`.
4. Подготовить словарь в Python, в котором в качестве ключей будут выступать неправильно записанные цвета, а в качестве значений – цвета, которые вы хотите подставить вместо них. Я рекомендую использовать в качестве целевых названий цветов более длинные варианты, такие как `WHITE`.
5. Заменить существующие цвета новыми с использованием вашего словаря. Сколько уникальных цветов осталось в наборе данных?
6. Просмотрите 50 самых распространенных цветов в наборе данных после исправления. Осталось что-то, что необходимо еще поправить? Есть ли непонятные для вас варианты? Видите ли вы явные опечатки и ошибки при написании цветов?

Подробный разбор

Опечатки – это наша вина. Но если вы опечатаетесь в письме другу, вряд ли он сильно на вас обидится. В случае с наукой о данных опечатки и ошибки могут приносить гораздо больше вреда, даже если их немного и внешне они могут ка-

заться незначительными. При анализе данных нам чаще всего приходится сталкиваться с разного рода ошибками, связанными с человеческим фактором, но и системы автоматизации тоже могут доставлять определенные проблемы.

В этом упражнении я попрошу вас поработать с цветами автомобилей, фигурирующих в списке выдачи штрафных парковочных талонов в Нью-Йорке за 2020 год. Оказывается, при выдаче парковочных талонов можно наделать невероятное количество ошибок, которые потенциально могут сказаться на дальнейшем анализе данных. Хотя вряд ли мы будем строить сколько-нибудь серьезный анализ, основываясь на данных о цвете машин.

Перед тем как исправлять опечатки и ошибки, мы должны понять, с чем имеем дело. В конце концов, может, у нас вообще нет проблем. После загрузки данных в датафрейм можно быстро проверить, сколько уникальных цветов есть в нашем наборе, следующим образом:

```
len(df['Vehicle Color'].value_counts().index)
```

Как вы знаете, метод `value_counts` часто помогает нам в случае необходимости понять, сколько неповторяющихся значений присутствует в объекте `Series`, и к тому же сортирует значения от наиболее популярных к наименее популярным. Поскольку метод `value_counts` возвращает объект `Series`, мы можем применить к нему функцию `len`.

Итак, у нас есть 1896 различных цветов. Конечно, эксперт в области цветопередачи может сказать, что это крайне мало в сравнении с тем, какое количество цветов способен различать человеческий глаз. Но для цветов машин, согласитесь, как-то многовато.

Давайте посмотрим на 30 самых распространенных цветов в нашем наборе данных:

```
df['Vehicle Color'].value_counts().head(30)
```

Мы видим, что в плане определения цвета машины инспекторы не пользуются никакими стандартами, а записывают цвет в буквальном смысле на глаз. И это речь только о 30 самых популярных цветах. А всего их порядка 1900.

Для очистки данных создадим специальный словарь в Python. Мы могли бы воспользоваться для подстановок и объектом `Series`, но со словарем, как кажется, все будет проще и понятнее:

```
colormap = {'WH': 'WHITE', 'GY': 'GRAY', 'BK': 'BLACK',
            'BL': 'BLUE', 'RD': 'RED', 'SILV': 'SILVER',
            'GR': 'GRAY', 'TN': 'TAN', 'BR': 'BROWN',
            'YW': 'YELLOW', 'BLK': 'BLACK', 'GRY': 'GRAY',
            'WHT': 'WHITE', 'WHI': 'WHITE', 'OR': 'ORANGE',
            'BK.': 'BLACK', 'WT': 'WHITE', 'WT.': 'WHITE'}
```

Мы определили 18 пар для стандартизации 18 названий цветов.

В качестве ключей в нашем словаре присутствуют названия цветов в том виде, в каком мы встречаем их в фактических записях, а в качестве значений – желаемые цвета. Подобные таблицы соответствий бывают крайне полезны при очистке данных, поскольку позволяют легко добавлять новые пары в процессе выявления очередных творческих попыток инспекторов как-то хитро сократить название цвета.

Применив метод `replace` к нашему объекту `Series`, представляющему столбец `Vehicle Color`, мы получим новый `Series`. Далее можно присвоить полученный объект обратно столбцу `df['Vehicle Color']`, что позволит заменить в нем старые значения, как показано ниже:

```
df['Vehicle Color'] = df['Vehicle Color'].replace(colormap)
```

ПРИМЕЧАНИЕ. Значения, отсутствующие в ключах словаря `colormap`, останутся неизменными. Соответствие со словарем должно быть точным, включая пробелы, регистр и знаки пунктуации.

Давайте снова проверим количество уникальных цветов в наборе данных:

```
len(df['Vehicle Color'].value_counts().index)
```

Получилось 1880 цветов, что на 16 меньше, чем было. В нашем словаре присутствует 18 пар, а значит, два цвета в нашем наборе не изменили свои значения. Как такое может быть? Это может значить, что *мы* где-то допустили две ошибки.

Во-первых, мы попросили заменить короткое название цвета `SILVE` на полное – `SILVER`. Но проблема в том, что учетная система, в которую попадают парковочные талоны, ограничивает цвет пятью символами, а значит, в исходном наборе данных машин с цветом `SILVER` просто нет. Следовательно, мы можем удалить пару с `SILVER` из нашего словаря, поскольку это слишком длинное название.

А что с парой `'OR': 'ORANGE'`? Мы ошибочно использовали название, состоящее из шести символов. Изменив эту пару на `'OR': 'ORANG'`, мы уменьшим количество уникальных цветов в наборе данных на единицу и объединим все оранжевые цвета под одной общей крышей.

Наш окончательный словарь будет выглядеть так:

```
colormap = {'WH': 'WHITE', 'GY': 'GRAY',
            'BK': 'BLACK', 'BL': 'BLUE',
            'RD': 'RED', 'GR': 'GRAY',
            'TN': 'TAN', 'BR': 'BROWN',
            'YW': 'YELLOW', 'BLK': 'BLACK',
            'GRY': 'GRAY', 'WHT': 'WHITE',
            'WHI': 'WHITE', 'OR': 'ORANG',
            'BK.': 'BLACK', 'WT': 'WHITE',
            'WT.': 'WHITE'}
```

Снова выполним замену в столбце с помощью словаря и посмотрим на количество уникальных цветов:

```
df['Vehicle Color'] = df['Vehicle Color'].replace(colormap)
```

Теперь их 1879. Разумеется, это самая верхушка айсберга под названием *очистка данных*. Мы обработали только малую часть ошибок, и речь шла всего об одном столбце. Теперь вы понимаете, почему процесс очистки данных так важен и может занимать так много времени.

Решение

```

filename = '../data/nyc-parking-violations-2020.csv'

df = pd.read_csv(filename,
                  usecols=['Plate ID',
                           'Registration State',
                           'Vehicle Make',
                           'Vehicle Color',
                           'Street Name'])

len(df['Vehicle Color'].value_counts().index) ❶
df['Vehicle Color'].value_counts().head(30)    ❷

colormap = {'WH': 'WHITE', 'GY': 'GRAY',
            'BK': 'BLACK', 'BL': 'BLUE',
            'RD': 'RED', 'GR': 'GRAY',
            'TN': 'TAN', 'BR': 'BROWN',
            'YW': 'YELLOW', 'BLK': 'BLACK',
            'GRY': 'GRAY', 'WHT': 'WHITE',
            'WHI': 'WHITE', 'OR': 'ORANG',
            'BK.': 'BLACK', 'WT': 'WHITE',
            'WT.': 'WHITE'} ❸

df['Vehicle Color'] = df[
    'Vehicle Color'].replace(colormap) ❹
len(df['Vehicle Color'].value_counts().index) ❺
df['Vehicle Color'].value_counts().head(50) ❻

```

- ❶ Сколько различных цветов в наборе данных?
- ❷ Какие 30 уникальных цветов встречаются в наборе данных чаще остальных?
- ❸ Словарь для преобразования неправильных имен цветов в правильные.
- ❹ Воспользуемся методом `replace` совместно с нашим словарем и присвоим результат обратно столбцу.
- ❺ Количество уникальных цветов действительно уменьшилось.
- ❻ Взглянем на 50 самых популярных цветов в наборе данных и поищем новые цели для очистки.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/M9EW>.

Дополнительные упражнения

1. Примените метод `value_counts` к столбцу `Vehicle Make` и рассмотрите названия производителей автомобилей. Всего в наборе данных насчитывается более 5200 различных марок, что явно говорит о большой несогласованности в данных. Какие проблемы вы видите? Напишите функцию для первичной очистки данных. На вход она должна принимать марку автомобиля, очищать ее от лишних знаков пунктуации, приводить к верхнему регистру и возвращать результат. Затем примените метод `apply` к столбцу с использованием этой функции. Сколько уникальных марок машин осталось в наборе данных после выполнения этой операции?

2. Как стандартизованы названия улиц в нашем наборе данных? Какие изменения вы бы внесли для улучшения ситуации?
3. Нужно ли подвергать очистке столбец Registration State и почему?

Ответы на дополнительные упражнения

Упражнение 28.1

Я бы воспользовался регулярными выражениями, но здесь приведен более простой вариант
import string

```
def clean_name(one_string):

    if not isinstance(one_string, str):
        return one_string

    output = ''

    for one_character in one_string.strip().upper():
        if one_character in string.ascii_uppercase:
            output += one_character

    return output

print(len(df['Vehicle Make'].value_counts()))
df['Vehicle Make'] = df['Vehicle Make'].apply(clean_name)
print(len(df['Vehicle Make'].value_counts()))
```

Вывод:

5210
4915

Упражнение 28.2

Давайте проведем ряд экспериментов и посмотрим, как стандартизованы данные

К примеру, в столбце часто встречаются строки E 110th St и E 110 ST
s = df['Street Name'].dropna()
s[s.str.contains('110')].value_counts()

Вывод:

W 110th St	2970
110th St	2388
E 110th St	2048
WB 110TH AVE/BRINKER	922
110th Ave	704
...	
O/F 77 EAST 110 ST	1
C/O 110 RD	1

```
S/E C/O E 110 ST      1
E/B 110 W 48 ST      1
E 110 ST              1
Name: Street Name, Length: 73, dtype: int64
```

Также иногда встречается название BWAY, а иногда BROADWAY ...

Для приведения данных в нормальный вид необходимо определиться с тем,
как использовать постфиксы st/nd/rd/th и как сокращать названия улиц.
Кроме того, для пересекающих улиц есть отдельный столбец, так что нет
смысла указывать их здесь. Ужасные данные! (или новые возможности?..)

```
s[s.str.contains('BWAY') | s.str.contains('BROADWAY')].value_counts()
```

Вывод:

```
SB BROADWAY @ 252ND      21939
NB BROADWAY @ W 228T     13367
BROADWAY                 10771
SB BROADWAY @ W 196T     6623
NB BROADWAY @ W 120T     5691
...
S/B BWAY                  1
BROADWAY PL              1
S/S BWAY                 1
S/O 1350 BROADWAY        1
N/E 220 BROADWAY         1
Name: Street Name, Length: 181, dtype: int64
```

Упражнение 28.3

У нас есть 68 штатов, включая канадские провинции и другие страны
Похоже, в целом здесь все в порядке, хотя какая-то дополнительная очистка
может понадобиться

```
df['Registration State'].value_counts()
```

Вывод:

```
NY      9753643
NJ      1096110
PA      338779
FL      174056
CT      165205
...
PE       18
SK        8
MX        7
NT        3
YT        2
Name: Registration State, Length: 68, dtype: int64
```

Заключение

Очистка данных представляет собой одну из важнейших составляющих процесса анализа данных, хотя выполнять ее бывает очень трудно и нудно. В этой главе мы постарались показать вам, что для эффективной очистки данных необходимо не только владеть техническим арсеналом всех доступных средств, но и использовать здравый смысл для определения того, когда стоит оставить в наборе данных пропущенные значения или дубликаты, а когда нужно избавляться от них (и каким именно способом). В библиотеке `pandas` представлено множество инструментов для очистки данных, включая операции удаления значений `NaN`, их замены различными методами и применения пользовательских функций для преобразования значений нужным вам способом построчно. Техники, которые мы рассмотрели в этой главе, а также метод `interpolate`, показанный в упражнении 13, составляют важный инструментарий для очистки данных, который пригодится вам в ваших собственных проектах.

Глава 6

Группировка, объединение и сортировка

В предыдущих главах мы научились создавать датафреймы в pandas, загружать в них данные, очищать и анализировать с применением разных техник. Но анализ зачастую требует более глубокого погружения в исходные данные. В частности, вам может потребоваться разложить данные на удобные для анализа части, углубиться в один из аспектов данных, объединить наборы из разных источников, привести данные к нужной форме и упорядочить их по определенным сложным критериям. В pandas эти операции собраны под общим названием *разделить-применить-объединить* (split-apply-combine), и именно им будет посвящена эта глава. Если у вас есть определенный опыт работы с языком запросов SQL и реляционными базами данных, вы обнаружите много общего как в наименовании применяемых операций, так и в образе их действий и функционале.

К примеру, в компании могут заинтересоваться цифрами продаж за последний квартал. Помимо этого, им будет интересно узнать, в каких странах продажи идут лучше, а в каких – хуже. Кроме того, начальнику отдела продаж может понадобиться информация о продажах в разрезе менеджеров, чтобы понять, кто из них работает лучше или хуже остальных, или в разрезе товаров, чтобы узнать, какие позиции пользуются наивысшим спросом.

На эти вопросы можно ответить с помощью техники *группировки данных* (grouping). Подобно тому как мы используем инструкцию GROUP BY в запросах SQL, мы можем использовать похожую технику в pandas для анализа наборов данных под разными углами.

Еще одна распространенная операция в языке SQL – это *объединение данных* (joining). С ее помощью мы можем ограничивать размер данных и сочетать разные наборы при необходимости. Например, в одном наборе могут содержаться данные о регионах продажи и менеджерах, а в другом – о самих продажах. И чтобы посмотреть результаты продаж по месяцам в разрезе регионов и менеджеров, нам необходимо объединить эти наборы.

Третья техника, которую вы, скорее всего, видели в других языках программирования и фреймворках, – это *сортировка данных* (sorting). В главе 5 мы увидели, как можно с помощью метода sort_index упорядочивать данные в датафрейме по значениям индекса. В этой главе мы обратимся к методу sort_values, позволяющему сортировать данные по одному или нескольким столбцам.

При работе с наборами данных в pandas эти три техники должны буквально отскакивать от ваших пальцев. В этой главе мы покажем, как решать распространенные типы задач в pandas с помощью сочетания этих приемов. Но эта тема слишком объемна для одной главы, так что в следующей главе мы рассмотрим более сложные случаи с применением описанных здесь операций.

В табл. 6.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 6.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>s.isnull</code>	Возвращает объект Series с булевыми значениями, показывающий расположение пропущенных (обычно NaN) значений в объекте <code>s</code>	<code>s.isnull()</code>	http://mng.bz/N2KX (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isnull.html#pandas.Series.isnull)
<code>df.sort_index</code>	Упорядочивает строки в датафрейме на основе значений в индексе по возрастанию	<code>df = df.sort_index()</code>	http://mng.bz/RxAn (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.sort_index.html)
<code>df.sort_values</code>	Упорядочивает строки в датафрейме на основе значений в одном или нескольких столбцах	<code>df = df.sort_values('distance')</code>	http://mng.bz/qRMK (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.sort_values.html)
<code>df.transpose()</code> или <code>df.T</code>	Возвращает новый датафрейм с теми же значениями, что и в исходном, но со столбцами и индексом, поменянными местами	<code>df.transpose()</code> или <code>df.T</code>	http://mng.bz/7DXx (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.transpose.html)
<code>df.expanding</code>	Позволяет выполнять оконные функции на расширенном наборе строк	<code>df.expanding().sum()</code>	http://mng.bz/mVBn (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.expanding.html)
<code>df.rolling</code>	Позволяет выполнять оконные функции в фиксированном окне, скользящем по датафрейму	<code>df.rolling(3).mean()</code>	http://mng.bz/5wp4 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.rolling.html)
<code>df.pct_change</code>	Для всего датафрейма показывает процентные различия между соседними ячейками в столбцах	<code>df.pct_change()</code>	http://mng.bz/4DBB (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.pct_change.html)

Таблица 6.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>df.diff</code>	Для заданного датафрейма показывает абсолютные различия между соседними ячейками в столбцах	<code>df.diff()</code>	http://mng.bz/OPDE (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.diff.html)
<code>df.groupby</code>	Позволяет применить один или несколько методов агрегации для каждого значения в конкретном столбце	<code>df.groupby('year')</code>	http://mng.bz/vn9x (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.groupby.html)
<code>df.loc</code>	Извлекает заданные строки и столбцы	<code>df.loc[:, 'passenger_count'] = df['passenger_count']</code>	http://mng.bz/nWzv (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html)
<code>s.iloc</code>	Позволяет осуществлять доступ к элементам объекта Series по позиции	<code>s.iloc[0]</code>	http://mng.bz/QPxM (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.iloc.html)
<code>df.dropna</code>	Удаляет строки со значениями NaN	<code>df = df.dropna()</code>	http://mng.bz/XN0Y (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.dropna.html)
<code>s.unique</code>	Позволяет получить уникальные значения в объекте Series (лучше использовать <code>drop_duplicates</code>)	<code>s.unique()</code>	http://mng.bz/yQrJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.unique.html)
<code>df.join</code>	Объединяет два датафрейма на основе индексов	<code>df.join(other_df)</code>	http://mng.bz/MB02 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.join.html)
<code>df.merge</code>	Объединяет два датафрейма на основе любых столбцов	<code>df.merge(other_df)</code>	http://mng.bz/a1wJ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.merge.html)
<code>df.corr</code>	Показывает корреляцию между числовыми столбцами в датафрейме	<code>df.corr()</code>	http://mng.bz/gBgR (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.corr.html)
<code>s.to_frame</code>	Преобразовывает объект Series в датафрейм с одним столбцом	<code>s.to_frame()</code>	http://mng.bz/5wp1 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.to_frame.html)
<code>s.removesuffix</code>	Возвращает строку с тем же содержанием, что и в строке <code>s</code> , но без указанного суффикса (если он есть)	<code>s.removesuffix('.csv')</code>	http://mng.bz/6DAD (https://docs.python.org/3/library/stdtypes.html#str.removesuffix)

Таблица 6.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>s.removeprefix</code>	Возвращает строку с тем же содержанием, что и в строке <code>s</code> , но без указанного префикса (если он есть)	<code>s.removeprefix('abcd')</code>	http://mng.bz/o1Rr (https://docs.python.org/3/library/stdtypes.html#str.removeprefix)
<code>s.title</code>	Возвращает новую строку на основе строки <code>s</code> , в которой каждое слово начинается с заглавной буквы	<code>s.title('hello out there')</code>	http://mng.bz/nWzg (https://docs.python.org/3/library/stdtypes.html#str.title)
<code>pd.concat</code>	Позволяет объединить вместе два датафрейма	<code>pd.concat([df1, df2, df3])</code>	http://mng.bz/vn9J (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html)
<code>df.assign</code>	Позволяет добавить один или несколько столбцов в датафрейм	<code>df.assign(a=df['x']*3)</code>	http://mng.bz/YR2A (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.assign.html)
<code>DataFrameGroupBy.agg</code>	Применяет множество методов агрегации к объекту <code>groupby</code>	<code>df.groupby('a')['b'].agg(['mean', 'std'])</code>	http://mng.bz/G900 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.core.groupby.DataFrameGroupBy.agg.html)
<code>DataFrameGroupBy.filter</code>	Сохраняет строки, для которых результат сторонней функции равен <code>True</code>	<code>df.groupby('a').filter(filter_func)</code>	http://mng.bz/zOBQ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.core.groupby.DataFrameGroupBy.filter.html)
<code>DataFrameGroupBy.transform</code>	Преобразует строки на основе результата сторонней функции	<code>df.groupby('a').transform(transform_func)</code>	http://mng.bz/0l26 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.core.groupby.DataFrameGroupBy.transform.html)
<code>df.rename</code>	Переименовывает столбцы в датафрейме	<code>df.rename(columns={'a':'b', 'c':'d'})</code>	http://mng.bz/K9W0 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.rename.html)
<code>df.drop_duplicates</code>	Возвращает датафрейм со строками, содержащими уникальные значения	<code>df.drop_duplicates()</code>	http://mng.bz/9Qv1 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.drop_duplicates.html)
<code>df.drop</code>	Удаляет строки или столбцы в датафрейме и возвращает новый датафрейм	<code>df.drop('a', axis='columns')</code>	http://mng.bz/j1eP (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.drop.html)

УПРАЖНЕНИЕ 29. Самые продолжительные поездки на такси

Когда я только начал работать с реляционными (SQL) базами данных, я был удивлен тому, что данные физически не хранятся в определенном порядке. Позже я узнал, что на то есть сразу несколько причин:

- порядок хранения строк обычно не оказывает влияния на запросы;
- с точки зрения базы данных гораздо более эффективно определять порядок хранения строк самостоятельно;
- нам может быть необходимо упорядочивать строки в запросах самым разным образом, и база данных просто не может определить, в каком порядке нам понадобятся данные. Так что лучше было переложить ответственность за сортировку строк на пользователя, а база данных пусть просто предоставляет информацию.

В *pandas*, в отличие от баз данных, строки в датафреймах хранятся в строго упорядоченном виде. При этом зачастую при выполнении анализа данных вас не интересует, как именно хранятся исходные данные. В конце концов, если нам нужно вычислить среднее значение по столбцу, какая разница, как физически располагаются данные?

В то же время при извлечении данных, будь то сведения о продажах, статистика работы сети или прогнозные показатели инфляции, нам обычно требуется как-то упорядочивать получаемую информацию. При этом способ упорядочивания будет зависеть от контекста. К примеру, данные о продажах будет логично отсортировать по отделам, сетевую статистику – по подсетям, а прогнозные показатели инфляции – по датам.

Еще один повод для сортировки данных состоит в получении максимального и минимального значений из конкретного столбца в датафрейме. И в этом упражнении мы именно этим и займемся. В частности, вы напишете несколько запросов к данным, касающимся поездок на такси в Нью-Йорке в январе 2019 года.

1. Загрузите информацию из файла `nyc_taxi_2019-01.csv` в датафрейм с использованием колонок `passenger_count`, `trip_distance` и `total_amount`.
2. Применив сортировку по убыванию (*descending*), рассчитайте среднюю стоимость 20 самых продолжительных (по дистанции) поездок на такси в январе 2019 года.
3. Применив сортировку по возрастанию (*ascending*), рассчитайте среднюю стоимость 20 самых продолжительных (по дистанции) поездок на такси в январе 2019 года. Отличаются ли полученные результаты?
4. Отсортируйте датафрейм по количеству пассажиров (по возрастанию) и продолжительности поездки (по убыванию). Таким образом, в первой строке должна оказаться поездка с минимальным количеством пассажиров и максимальной дистанцией. Какова средняя цена 50 первых поездок на такси в полученном датафрейме?

Подробный разбор

Когда мы намереваемся сортировать датафрейм в `pandas`, то сначала должны определиться с тем, будем ли мы это делать по индексу или по значениям в столбцах. Мы уже видели в предыдущих главах, что вызов метода `sort_index` возвращает датафрейм, аналогичный исходному, но с упорядоченными строками по значениям в индексе по возрастанию.

В этом упражнении мы снова будем сортировать датафрейм, но на этот раз на основании значений в столбцах, а не в индексе. Вы могли бы сказать, что разница не так велика, ведь мы можем взять столбец, временно перевести его в индекс, упорядочить по нему и восстановить столбец. Но разница между методами `sort_index` и `sort_values` не ограничивается одним лишь техническим аспектом. При использовании этих методов мы представляем наши данные и осуществляем доступ к ним по-разному.

Также метод `sort_values` отличается от `sort_index` тем, что мы можем выполнять упорядочивание сразу по нескольким столбцам. Опять же, представим данные о продажах. Мы можем сортировать их по цене, региону или менеджеру, а можем и по всем трем столбцам одновременно. При выполнении сортировки по индексу мы, по сути, упорядочиваем данные по одному столбцу.

В первой части упражнения я попросил вас создать датафрейм со столбцами `passenger_count`, `trip_distance` и `total_amount`:

```
filename = '../data/nyc_taxi_2019-01.csv'

df = pd.read_csv(filename,
                  usecols=['passenger_count',
                           'trip_distance',
                           'total_amount'])
```

Теперь можно начать анализировать наши данные. Сначала мы найдем 20 самых продолжительных поездок на такси, после чего рассчитаем по ним среднюю стоимость. Для этого необходимо упорядочить данные по столбцу `trip_distance` в порядке убывания. Попробуем сделать это так:

```
df.sort_values('trip_distance')
```

В результате мы получим новый датафрейм, аналогичный исходному, но отсортированный по столбцу `trip_distance` по возрастанию. Мы могли бы извлечь нужную информацию и из такого датафрейма, но будет правильнее установить направление сортировки по убыванию. Это делается с помощью параметра `ascending=False`, как показано ниже и проиллюстрировано на рис. 6.1:

```
df.sort_values('trip_distance',
               ascending=False)
```

Нам необходимо извлечь только столбец `total_amount`. Это можно сделать посредством квадратных скобок, как показано ниже и на рис. 6.2:

```
df.sort_values('trip_distance',
               ascending=False)
    ['total_amount']
```

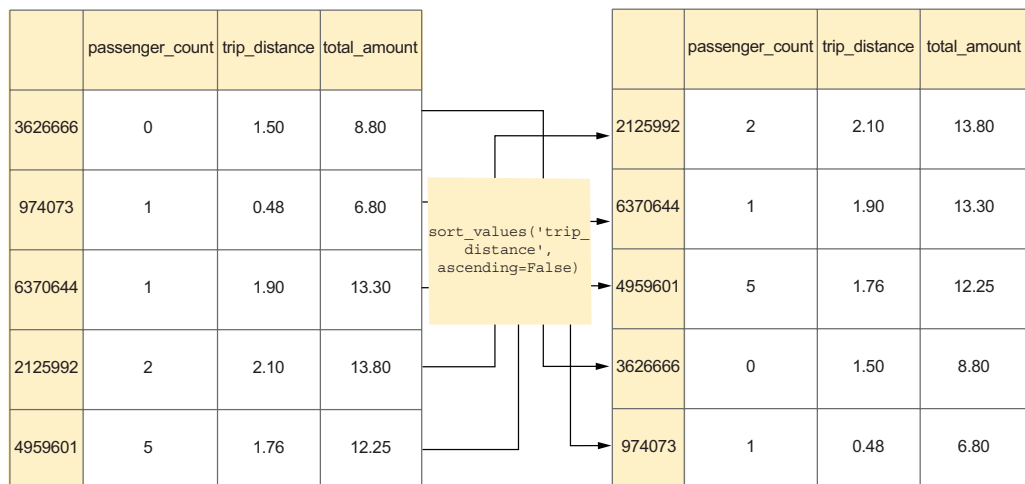


Рис. 6.1. Метод `sort_values` возвращает новый датафрейм с упорядоченными строками

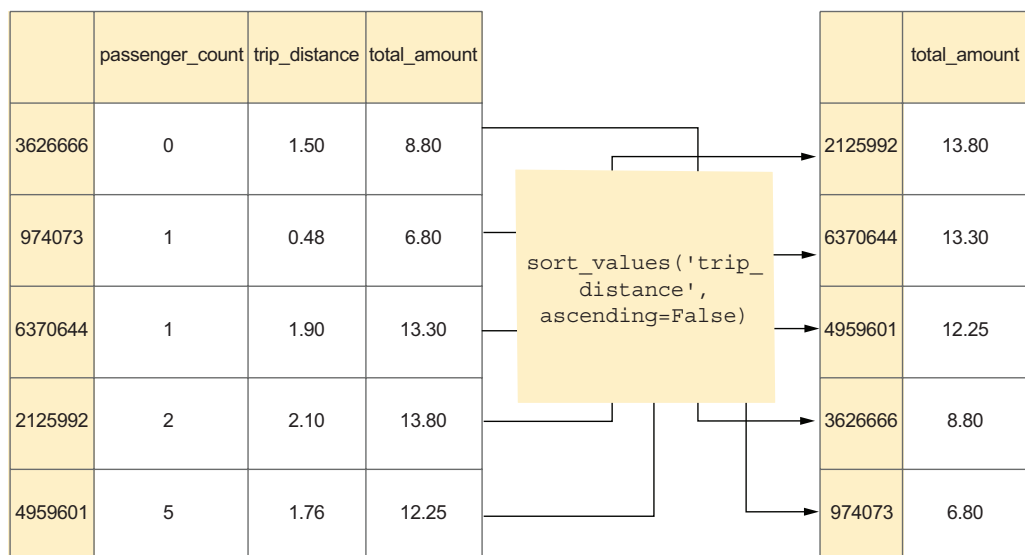


Рис. 6.2. Метод `sort_values` с извлечением одного столбца

Столбец есть, теперь нужно рассчитать среднюю стоимость по 20 самым продолжительным поездкам. Как извлечь первые 20 строк? Можно сделать это с помощью конструкции `head(20)`. Еще один способ – воспользоваться атрибутом доступа `iloc` следующим образом (см. рис. 6.3):

```
df.sort_values('trip_distance',
               ascending=False
               )['total_amount'].iloc[:20]
```

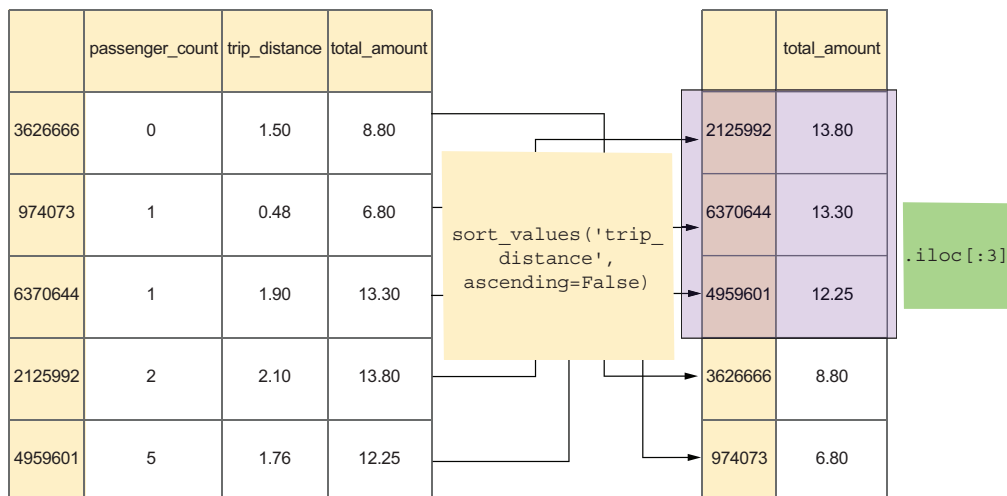


Рис. 6.3. Метод `sort_values` с извлечением одного столбца и получением первых 20 строк

Обратите внимание, что мы здесь использовали атрибут `iloc`, а не `loc`. Дело в том, что атрибут `loc` осуществляет доступ к строкам по текущему индексу, который в результате применения сортировки к столбцу `trip_distance` оказался разбросанным. Как следствие, использование выражения `loc[:20]` вернет нам намного больше 20 строк.

Получив стоимости всех 20 самых продолжительных поездок, мы можем легко рассчитать среднее значение по ним следующим образом:

```
df.sort_values('trip_distance',
               ascending=False)
    ['total_amount'].iloc[:20].mean()
```

В итоге мы получили значение 290.00999999999993, которое можно округлить до средней стоимости 290 долл. Мы можем выполнить округление прямо в запросе. Давайте применим формат цепочки методов, чтобы итоговый запрос выглядел опрятно:

```
(
    df
    .sort_values('trip_distance',
                 ascending=False)
    ['total_amount']
    .iloc[:20]
    .mean()
    .round(2)
)
```

Далее мы должны выполнить то же вычисление, но на этот раз с сортировкой по возрастанию. Для начала упорядочим датафрейм по значениям:

```
df.sort_values('trip_distance')
```


Помните, что по умолчанию метод `sort_values` упорядочивает значения по возрастанию, так что нам нет нужды указывать дополнительные параметры. Теперь оставим только столбец `total_amount`:

```
df.sort_values('trip_distance')['total_amount']
```

Нас интересуют только 20 самых продолжительных поездок на такси. Мы отсортировали данные по возрастанию, а значит, самые длинные поездки располагаются в конце списка, а не в начале.

Как и раньше, мы можем извлечь самые продолжительные поездки двумя основными способами. Первый состоит в использовании метода `tail(20)`, но мы будем использовать атрибут доступа `iloc` для извлечения 20 последних строк из датафрейма:

```
df.sort_values('trip_distance')[
    'total_amount'].iloc[-20:]
```

Помните, что в Python отрицательные индексы означают отсчет данных с конца структуры данных, а не с начала. Таким образом, индекс `-1` означает последний элемент в списке, `-2` – второй с конца и т. д. Более того, наш срез может быть пустым с одной стороны – это значит, что с этой стороны мы доходим до конца списка. Так что выражение `iloc[-20:]` в нашем случае позволяет получить 20 последних элементов в объекте `Series`.

ПРИМЕЧАНИЕ. Какой способ работает быстрее: `tail` или `iloc`? Проведя ряд исследований, я пришел к выводу, что по скорости они существенно не отличаются.

В заключение воспользуемся методом `mean` для усреднения стоимости поездок:

```
df.sort_values('trip_distance')[
    'total_amount'].iloc[-20:].mean()
```

Результат получился... 290.01000000000005, примерно такой же, как в предыдущем случае. Мы можем снова округлить его и написать выражение в виде цепочки вызовов следующим образом:

```
(
    df
    .sort_values('trip_distance')
    ['total_amount']
    .iloc[-20:]
    .mean()
    .round(2)
)
```

Округленный результат – 290.01. Но что у нас с неокругленными цифрами? Мы получили в разных расчетах 290.00999999999993 и 290.01000000000001. Разница небольшая, но она есть. В чем причина?

Ответ кроется в нюансах математики чисел с плавающей точкой. Есть хороший сайт <https://0.30000000000000004.com>, на котором вы можете подробно почитать про проблемы вычислений с плавающей точкой. А что-нибудь можно сделать, чтобы их избежать?

Ответ – отчасти. При использовании больших типов данных с плавающей точкой (задействующих больше памяти) эти проблемы будут возникать реже. К примеру, мы можем прочесть столбец `total_amount` с применением 128-битного типа данных, а не 64-битного, использующегося по умолчанию:

```
df = pd.read_csv(filename,
                  usecols=['passenger_count',
                           'trip_distance',
                           'total_amount'],
                  dtype={'total_amount': np.float128})
```

В этом случае оба способа вычисления дадут нам одинаковый результат – 290.01000000000000076. Но и памяти столбец потребует вдвое больше.

ПРИМЕЧАНИЕ. Если 128-битные типы данных обеспечивают наибольшую точность расчетов, почему бы всегда их не использовать? Во-первых, они очень дорого обходятся с точки зрения памяти – одно число занимает целых 16 (!) байт. При наличии миллиона строк в датафрейме на один этот столбец потребуется 16 Мб памяти. И далеко не все задачи требуют применения такой хирургической точности расчетов. К тому же 128-битные числа с плавающей точкой могут стать источником проблем. На моем Mac некоторые методы `pandas` не работают при использовании в столбцах типа данных `np.float128`. А на компьютерах под управлением Windows, похоже, тип `np.float128` отсутствует вовсе. Если вам просто необходима такая точность и платформа позволяет, можете использовать тип данных `np.float128`. Но учтите, что в этом случае ваше решение трудно будет перенести на другую платформу.

Далее нас попросили отсортировать данные в датафрейме по двум столбцам. Это то, что мы делаем постоянно, даже не задумываясь об этом. К примеру, в телефонных книгах контакты упорядочены сначала по фамилии, а затем – по имени. Это позволяет искать нужного человека сначала по фамилии, а в рамках этой фамилии – по имени.

Первым столбцом сортировки в нашем задании будет столбец `passenger_count`. Мы должны упорядочить строки по количеству перевозимых пассажиров по возрастанию – от меньшего к большему. В то же время внутри окна с одинаковой численностью пассажиров строки должны быть упорядочены по убыванию преодоленной дистанции, т. е. по столбцу `trip_distance`.

`Pandas` позволяет выполнять сортировку датафрейма по разным столбцам в разном порядке. Для этого необходимо передать методу `sort_values` первым аргументом список имен столбцов, по которым должна выполняться сортировка. В качестве дополнительного аргумента `ascending` можно указать список булевых значений, указывающих порядок сортировки каждого из перечисленных столбцов в том же порядке, что показано ниже и на рис. 6.4:

```
df.sort_values(['passenger_count', 'trip_distance'],
               ascending=[True, False])
```

В результате выполнения этого выражения мы получим новый датафрейм с тремя столбцами, в котором исходные строки будут упорядочены сначала по столбцу `passenger_count` (в возрастающем порядке), а затем по столбцу

trip_distance (в убывающем). Таким образом, в первой строке итогового датафрейма будет присутствовать запись о самой длинной поездке с минимальным количеством пассажиров, а в последней – о самой короткой поездке с максимальной заполненностью автомобиля.

	passenger_count	trip_distance	total_amount
3626666	0	1.50	8.80
974073	1	0.48	6.80
6370644	1	1.90	13.30
2125992	2	2.10	13.80
4959601	5	1.76	12.25

	passenger_count	trip_distance	total_amount
3626666	0	1.50	8.80
6370644	1	1.90	13.30
974073	1	0.48	6.80
2125992	2	2.10	13.80
4959601	5	1.76	12.25

Рис. 6.4. Сортировка датафрейма по столбцам passenger_count (по возрастанию) и trip_distance (по убыванию)

После этого мы извлечем из набора данных столбец total_amount, возьмем из него первые 50 значений с помощью атрибута iloc (хотя могли бы применить и метод head(50)) и вычислим среднее, как показано ниже:

```
(
    df
    .sort_values(['passenger_count',
                  'trip_distance'],
                  ascending=[True, False])
    ['total_amount']
    .iloc[:50]
    .mean()
)
```

Полученный результат – 135.49740000000001.

Решение

```
filename = '../data/nyc_taxi_2019-01.csv'

df = pd.read_csv(filename,
                  usecols=['passenger_count',
                           'trip_distance',
                           'total_amount'],
                  dtype={'total_amount': np.float128})

df.sort_values('trip_distance',
               ascending=False)[
    'total_amount'].iloc[:20].mean()
df.sort_values('trip_distance')[
```

```

        'total_amount'].iloc[-20:].mean() ❷

(
    df
    .sort_values(['passenger_count',
                  'trip_distance'],
                  ascending=[True, False])
    ['total_amount']
    .iloc[:50]
    .mean()
) ❸

```

- ❶ Сортируем данные по столбцу `trip_distance` в порядке убывания, извлекаем только столбец `total_amount`, берем первые 20 строк и вычисляем среднее значение.
- ❷ Сортируем данные по столбцу `trip_distance` в порядке возрастания, извлекаем только столбец `total_amount`, берем последние 20 строк и вычисляем среднее значение.
- ❸ Сортируем данные по столбцу `passenger_count` в порядке возрастания, затем по столбцу `trip_distance` в порядке убывания, извлекаем только столбец `total_amount`, берем первые 50 строк и вычисляем среднее значение.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/W1Z1>.

Дополнительные упражнения

1. Выберите пять случаев, когда пассажиры платили больше всего в расчете на милю преодоленного пути. Какова была продолжительность этих поездок?
2. Предположим, что при поездке нескольких пассажиров они делят оплату поровну. Выберите десять поездок, в которых каждый пассажир заплатил максимальную сумму.
3. В упражнении я сказал, что для извлечения первых или последних записей из отсортированного датафрейма необходимо использовать атрибут `iloc` или методы `head/tail`, поскольку индекс после упорядочивания набора данных оказывается перемешанным. Но при вызове метода `sort_values` вы можете передать параметр `ignore_index=True`, что позволит сохранить в отсортированном датафрейме индекс, начинающийся с нуля. Воспользуйтесь этой опцией и примените атрибут `loc` для получения среднего значения по столбцу `total_amount` для 20 самых продолжительных поездок.

Группировка

Мы уже видели, что функции агрегации, такие как `mean` и `std`, позволяют нам лучше понять наши данные. Но иногда эти и другие функции нам необходимо применять не ко всему набору данных, а к его отдельным частям. К примеру, нам может потребоваться узнать суммы продаж в разрезе регионов, среднюю стоимость проживания по городам или стандартное отклонение некоего показателя для каждой возрастной группы населения. Конечно, мы могли бы делить данные на группы и применять к каждой из них отдельную функцию.

Но это было бы довольно утомительно. Да и зачем этим заниматься вручную, если pandas все может сделать за нас?

Вам должна быть знакома операция группировки данных, если вам доводилось работать с реляционными базами данных. В этом упражнении мы попробуем выяснить, влияет ли численность пассажиров такси (в среднем) на дистанцию поездок. Иными словами, представьте, что вы, как водитель такси, подрабатывающий аналитиком данных (или наоборот, если угодно), можете выбирать между заказами с одним пассажиром и группой пассажиров. Повлияет ли ваш выбор на вероятность совершения длинной поездки, а значит, и на ее стоимость?

Давайте вернемся к нашему набору данных с продажами товаров из главы 2:

```
df = DataFrame([{'product_id':23, 'name':'computer',
                 'wholesale_price': 500,
                 'retail_price':1000, 'sales':100,
                 'department':'electronics'},
                {'product_id':96, 'name':'Python Workout',
                 'wholesale_price': 35,
                 'retail_price':75, 'sales':1000,
                 'department':'books'},
                {'product_id':97, 'name':'Pandas Workout',
                 'wholesale_price': 35,
                 'retail_price':75, 'sales':500,
                 'department':'books'},
                {'product_id':15, 'name':'banana',
                 'wholesale_price': 0.5,
                 'retail_price':1, 'sales':200,
                 'department':'food'},
                {'product_id':87, 'name':'sandwich',
                 'wholesale_price': 3,
                 'retail_price':5, 'sales':300,
                 'department':'food'},
                ])
```

Как видите, мы немного изменили наши данные, добавив в них текстовое поле department. Сейчас мы им воспользуемся.

Для определения того, сколько товаров присутствует в нашем ассортименте, мы можем просто посчитать количество строк в датафрейме следующим образом:

```
df.count()
```

Это полезная информация, но нам бы хотелось посмотреть ее в разрезе отделов. Для этого можно воспользоваться методом groupby, как показано ниже:

```
df.groupby('department')
```

Обратите внимание, что в качестве аргумента метод groupby принимает именно имя колонки для группировки. А какой результат вернет этот метод?

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x13174f970>
```

Как видите, мы получили объект типа `DataFrameGroupBy`, к которому теперь можно применить нужную нам агрегацию. К примеру, мы можем использовать тот же метод агрегации `count`, который позволит нам определить, сколько товаров из нашего ассортимента относится к каждому из отделов:

```
df.groupby('department').count()
```

В результате мы получим новый датафрейм с теми же столбцами, что и в оригинале, и строками, отражающими уникальные значения из столбца `department`. Поскольку у нас в данных присутствует три уникальных отдела, мы увидим данные по трем строкам: `electronics`, `books` и `food`.

Иногда нам нет необходимости извлекать все столбцы из набора данных, достаточно лишь нескольких. В теории для этого мы могли бы ограничить количество столбцов в результате следующим образом:

```
df.groupby('department').count()['product_id']
```

В итоге мы получили бы объект `Series` с индексом, наполненным уникальными значениями из столбца `department`, и значениями в виде количества элементов в каждом из отделов. Ответ был бы правильный.

Однако в этом случае мы произвели бы избыточные действия. Что мы сделали? Мы применили функцию агрегации `count` к объекту `DataFrameGroupBy` и только затем исключили ненужные столбцы, оставив лишь `product_id`. Гораздо более эффективно, особенно при работе с большими наборами данных, было бы сначала извлечь нужный столбец из объекта `DataFrameGroupBy`, а затем применить агрегацию, как показано ниже:

```
df.groupby('department')['product_id'].count()
```

Увидеть этот процесс можно по ссылке <http://mng.bz/84nw>. Мы получим тот же результат, но сами операции будут выполнены более эффективно.

В данном примере мы воспользовались функцией `count`, но вы можете применять любые доступные функции агрегации, такие как `mean`, `std`, `min`, `max` или `sum`. Например, мы можем получить среднюю цену на товар в разрезе отделов следующим образом:

```
df.groupby('department')['retail_price'].mean()
```

А что, если нам необходимо узнать одновременно и среднее значение цены товаров, и стандартное отклонение этой величины в разрезе отделов? Это можно сделать, немного изменив синтаксис выражения. Вместо вызова функции агрегации напрямую вы можете воспользоваться методом `agg` объекта `DataFrameGroupBy`. Он принимает на вход список методов, каждый из которых применяется к объекту `GroupBy`:

```
df.groupby('department')['retail_price'].agg(['mean', 'std'])
```

Обратите внимание, что на вход методу `agg` мы передали список названий функций агрегации. Обычно в таких случаях передаются сами функции вроде `np.mean` или `np.std`, но в последние годы это перестало быть тенденцией. Теперь принято передавать методы в виде строк и давать возможность `pandas` определять, какие функции применять при вычислениях.

В результате вызова метода `agg` мы получим датафрейм с двумя столбцами (`mean` и `std`) и тремя строками – по одной на каждый отдел. Это вычисление позволило определить среднее значение и стандартное отклонение по розничным ценам в разрезе отделов. Визуально вы можете посмотреть процесс выполнения операций по адресу <http://mng.bz/E9G0>.

Ну, хорошо, а если вам понадобится применить разные функции агрегации к разным столбцам? В этом случае вам не придется фильтровать столбцы с помощью квадратных скобок. Вместо этого нужно вызвать метод `agg` у объекта `DataFrameGroupBy` и передать ему по одной паре ключ/значение для каждого столбца в новом датафрейме:

- в качестве ключей должны выступать имена будущих столбцов;
- в качестве значений – кортежи, состоящие из двух элементов:
 - первый элемент кортежа соответствует имени столбца в исходном датафрейме, который мы хотим анализировать;
 - второй элемент – это функция агрегации в виде строки.

К примеру, если нам нужно получить среднее значение и стандартное отклонение по столбцу `retail_price`, а также максимальное количество проданных товаров в разрезе отделов, достаточно написать следующее выражение:

```
df.groupby('department').agg(mean_price=('retail_price', 'mean'),
                             std_price=('retail_price', 'std'),
                             max_sales=('sales', 'max'))
```

Группировка без сортировки ключей

По умолчанию при вызове метода `groupby` ключи группировки сортируются по возрастанию. Если вам не нужен этот эффект или вы видите, что это негативно сказывается на эффективности выполнения операции, можете передать этому методу дополнительный параметр `sort=False`, как показано ниже:

```
df.groupby('department', sort=False)['retail_price'].agg(['mean', 'std'])
```

Ответы на дополнительные упражнения

Упражнение 29.1

```
# Для начала избавимся от поездок с нулевой дистанцией
df = df[df['trip_distance'] != 0]
```

```
# Создадим новый столбец, в котором будет храниться стоимость преодоления
# одной мили
df['cost_per_mile'] = df['total_amount'] / df['trip_distance']
```

```
# Отсортируем датафрейм по этому столбцу и получим пять наивысших значений
df.sort_values('cost_per_mile').tail(5)
```

```
# Очевидно, что в наших данных есть какие-то ошибки – это видно по поездкам
```

```
# на расстояние 0.01 мили и присутствию поездки за 623261.66 долл.
# на расстояние 2.40 мили
```

Вывод:

	passenger_count	trip_distance	total_amount	cost_per_mile
4136499	1	0.01	273.96	27396.000000
6403254	1	0.01	322.30	32230.000000
7099014	4	0.01	415.30	41530.000000
478791	1	0.10	6667.45	66674.500000
2499600	1	2.40	623261.66	259692.358333

Упражнение 29.2

```
# Избавимся от поездок меньше чем с двумя пассажирами
df = df[df['passenger_count'] >= 2]
```

```
# Создадим расчетный столбец со средней оплатой на пассажира
df['payment_per_person'] = df['total_amount'] / df['passenger_count']
```

```
# Найдем десять самых крупных сумм на пассажира
df.sort_values('payment_per_person').tail(10)
```

Вывод:

	passenger_count	trip_distance	total_amount	payment_per_person
1154626	2	0.00	400.80	200.400
4751745	2	100.78	403.50	201.750
6496403	2	0.00	410.95	205.475
6857368	2	0.00	411.36	205.680
5726185	2	65.05	416.82	208.410
149362	2	17.20	426.80	213.400
7593395	2	83.61	449.32	224.660
3842620	2	110.04	515.82	257.910
3014027	2	16.60	560.76	280.380
2972145	2	19.90	589.96	294.980

Упражнение 29.3

```
df.sort_values('trip_distance',
               ascending=False,
               ignore_index=True)['total_amount'].loc[:20].mean()
```

Вывод:

```
253.65904761904761955
```

УПРАЖНЕНИЕ 30. Сравним поездки на такси

Ранее мы уже рассматривали набор данных с поездками на такси в Нью-Йорке в январе 2019 года под разными углами. Но в основном мы либо оценивали весь набор целиком, либо делали небольшие ручные группировки. В этом упражнении мы воспользуемся операцией группировки для лучшего понимания сути данных. Вот что вам нужно сделать.

1. Загрузите в датафрейм данные из файла `nyc_taxi_2019-01.csv` с использованием только столбцов `passenger_count`, `trip_distance` и `total_amount`.
2. Для каждой численности пассажиров найдите среднее значение стоимости поездки. Отсортируйте результаты по возрастанию стоимости.
3. Теперь упорядочьте полученный датафрейм по количеству пассажиров по возрастанию.
4. Создайте новый столбец `trip_distance_group`, заполнив его значениями `short` (< 2 миль), `medium` (≥ 2 миль и ≤ 10 миль) и `long` (> 10 миль). Каково среднее количество перевозимых пассажиров в каждой из этих категорий? Отсортируйте результаты по убыванию количества пассажиров.

Подробный разбор

Группировка данных представляет собой очень простую операцию, но при этом позволяет выполнять довольно глубокий и разносторонний анализ. С помощью группировки мы можем вычислять различные показатели по разным группам в рамках одного запроса, создавать новые датафреймы и анализировать их. В этом упражнении мы снова начнем с загрузки данных из файла `nyc_taxi_2019-01.csv` в датафрейм:

```
filename = '../data/nyc_taxi_2019-01.csv'

df = pd.read_csv(filename,
                  usecols=['passenger_count',
                           'trip_distance',
                           'total_amount'])
```

Теперь нам необходимо найти среднее значение стоимости поездки для каждой численности пассажиров. При использовании метода `groupby` мы должны последовательно ответить на следующие вопросы:

- с каким датафреймом мы будем работать?
- по какому столбцу мы будем выполнять группировку? Чаще всего это будет столбец с категориальными данными по своей природе, в котором будет находиться ограниченное количество строковых или целочисленных (как в нашем случае) значений. Уникальные значения из этого столбца будут располагаться в строках на выходе метода группировки;
- какие столбцы мы хотим анализировать? То есть к каким столбцам нужно применить функции агрегации?
- какие функции агрегации мы будем использовать?

В нашем случае ответы на эти вопросы будут следующими:

- мы будем работать с датафреймом `df`;
- группы будут создаваться на основе значений в столбце `passenger_count`;
- анализируемый столбец – `total_amount`;
- используемая функция агрегации – `mean`.

Иными словами, нам необходимо выполнить следующее выражение:

```
df.groupby('passenger_count')['total_amount'].mean()
```

На выходе мы получим объект Series. В индексе этого объекта будут содержаться все уникальные значения из столбца `passenger_count`. В качестве значений объекта Series мы увидим результат применения метода агрегации `mean` к столбцу `df['total_amount']`. В виде цикла эту операцию можно представить следующим образом:

```
for i in range(df['passenger_count'].max() + 1):
    print(i,
          df.loc[df['passenger_count'] == i,
                  'total_amount']
          .mean())
```

- ❶ Выводим текущее значение `passenger_count`.
- ❶ В селекторе строк извлекаем строки, в которых значение в столбце `passenger_count` равно `i`.
- ❶ В селекторе столбцов извлекаем столбец `total_amount`.
- ❶ Рассчитываем среднее значение по столбцу `total_amount` с заданным значением `passenger_count`.

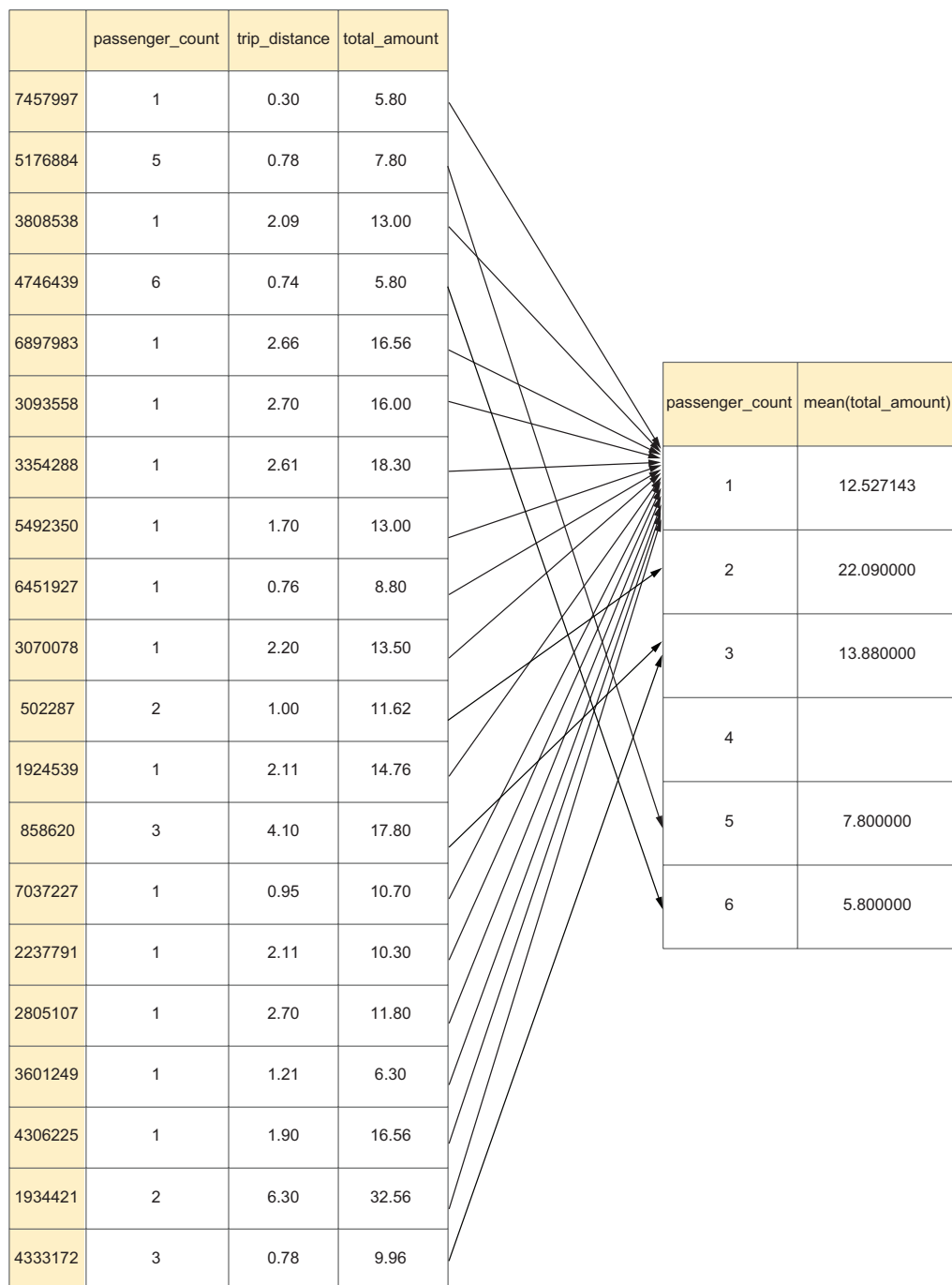
Здесь мы воспользовались циклом `for`, чтобы пройти по всем уникальным значениям в столбце `df['passenger_count']` и применить функцию агрегации `mean` к столбцу `total_amount` в каждом из полученных наборов. В итоге мы получим тот же результат, но по эффективности этот способ будет значительно уступать методу `groupby`. Кроме того, вычисленные результаты не будут объединены в удобную для дальнейшего использования структуру данных. По этим и другим причинам практически никогда не стоит использовать цикл `for` применительно к структурам данных `pandas`. Вместо этого вы всегда должны стремиться применять родные для `pandas` методы, такие как `groupby`. В то же время цикл позволил нам лучше разобраться в том, что происходит под капотом метода `groupby` (см. рис. 6.5).

Теперь, когда мы рассчитали среднюю стоимость поездки для каждой численности пассажиров в такси, нам нужно отсортировать полученные результаты по значению в порядке возрастания. Это можно сделать с помощью метода `sort_values`, как показано ниже и на рис. 6.6:

```
df.groupby('passenger_count')[
    'total_amount'].mean().sort_values()
```

Следующее задание состоит в том, чтобы выполнить то же вычисление, но результаты отсортировать по количеству пассажиров в порядке возрастания. Помните, что в результате вызова метода `mean` у сгруппированного объекта мы получаем на выходе объект Series. В индексе нашего объекта содержатся уникальные значения численности пассажиров. Таким образом, для выполнения задания нам достаточно отсортировать результаты по индексу, как показано ниже:

```
df.groupby('passenger_count')[
    'total_amount'].mean().sort_index()
```

Рис. 6.5. Схематическое изображение совместной работы методов `groupby` и `mean`

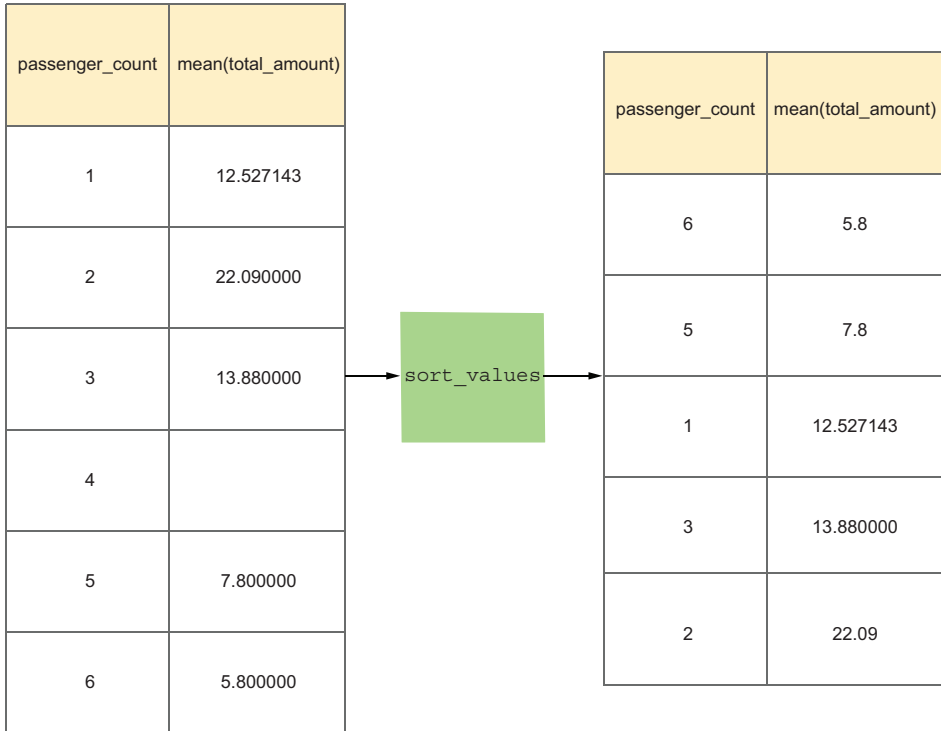


Рис. 6.6. Схематическое изображение сортировки результатов, полученных с помощью метода `groupby`

Далее нас попросили создать новый столбец с именем `trip_distance_group` и проставить в нем значения `short`, `medium` и `long` в зависимости от преодоленной дистанции. Это можно сделать с помощью функции `pd.cut`, как показано ниже:

```
df['trip_distance_group'] = pd.cut(
    df['trip_distance'],
    [df['trip_distance'].min(), 2, 10,
     df['trip_distance'].max()],
    labels=['short', 'medium', 'long'],
    include_lowest=True)
```

- ❶ Используем функцию `pd.cut` для преобразования числовых значений в столбце в категориальные.
- ❷ Категории будут строиться на основе столбца `trip_distance`.
- ❸ Наши точки разрывов – это минимум, 2, 10 и максимум.
- ❹ Помещаем значение в одну из следующих категорий: `short`, `medium` или `long`.
- ❺ Обеспечиваем включение левой границы для первой категории.

Теперь мы можем использовать созданный столбец в методе `groupby`. В нашем упражнении требуется найти среднее количество пассажиров для каждой категории из нового столбца. Это можно сделать так:

```
df.groupby('trip_distance_group')[
    'passenger_count'].mean().sort_values(ascending=False)
```

Таким образом, мы рассчитали среднее значение численности пассажиров для каждой уникальной категории из столбца `trip_distance_group`. В качестве результата мы получили объект `Series`, индекс которого содержит неповторяющиеся значения из столбца `trip_distance_group`, а в качестве значений присутствуют средние значения численности пассажиров для каждой категории.

После выполнения вычислений мы отсортировали результат по значениям в порядке убывания, отметив при этом, что для коротких, средних и длинных поездок среднее количество пассажиров отличается очень незначительно. Иными словами, нашему таксисту-аналитику нет особого смысла выбирать заказы в зависимости от количества пассажиров, поскольку в среднем продолжительность поездок от этого показателя сильно не зависит.

Решение

```
filename = '../data/nyc_taxi_2019-01.csv'

df = pd.read_csv(filename,
                  usecols=['passenger_count',
                           'trip_distance',
                           'total_amount'])

df.groupby('passenger_count')['total_amount']
   .mean().sort_values()
df.groupby('passenger_count')['total_amount']
   .mean().sort_index()

df['trip_distance_group'] = pd.cut(
    df['trip_distance'],
    [df['trip_distance'].min(), 2, 10,
     df['trip_distance'].max()],
    labels=['short', 'medium', 'long'])
df.groupby('trip_distance_group')['passenger_count']
   .mean().sort_values(ascending=False)
```

- ❶ Возвращает средние значения по столбцу `total_amount` для каждого уникального значения в столбце `passenger_count` и сортирует результаты по значению (т. е. по средней стоимости за поездку).
- ❶ Возвращает средние значения по столбцу `total_amount` для каждого уникального значения в столбце `passenger_count` и сортирует результаты по индексу (т. е. по численности пассажиров в поездке).
- ❶ Функция `pd.cut` используется для создания категорий на основе поля `trip_distance` в новом столбце `df['trip_distance_group']`.
- ❶ Для каждого значения в столбце `trip_distance_group` рассчитывается среднее значение по столбцу `passenger_count`, и результат сортируется по значениям в порядке их убывания.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/NVN1>.

Дополнительные упражнения

1. Создайте единый датафрейм, объединив данные о поездках на такси за январь 2019 и 2020 годов. Добавьте столбец `year` с номером года. Воспользуйтесь методом `groupby` для сравнения средней стоимости поездок в январе 2019 и 2020 годов.
2. Создайте двухуровневую группировку – сначала по году, затем по столбцу `passenger_count`.
3. Метод `corr` позволяет увидеть, насколько сильно коррелируют между собой данные в столбцах. Воспользуйтесь методом `corr`, после чего отсортируйте результаты методом `sort_values`, чтобы узнать, какие столбцы коррелируют больше остальных.

Объединение

Подобно группировке, *объединение данных* (joining) представляет собой концепцию, с которой вы наверняка сталкивались, если работали с реляционными базами данных. Объединение данных в `pandas` сильно напоминает аналогичную команду в `SQL`, но синтаксис немного отличается.

Рассмотрим датафрейм, с которым мы уже сталкивались в этой главе:

```
df = DataFrame([{'product_id':23, 'name':'computer',
                 'wholesale_price': 500,
                 'retail_price':1000, 'sales':100,
                 'department':'electronics'},
                {'product_id':96, 'name':'Python Workout',
                 'wholesale_price': 35,
                 'retail_price':75, 'sales':1000,
                 'department':'books'},
                {'product_id':97, 'name':'Pandas Workout',
                 'wholesale_price': 35,
                 'retail_price':75, 'sales':500,
                 'department':'books'},
                {'product_id':15, 'name':'banana',
                 'wholesale_price': 0.5,
                 'retail_price':1, 'sales':200,
                 'department':'food'},
                {'product_id':87, 'name':'sandwich',
                 'wholesale_price': 3,
                 'retail_price':5, 'sales':300,
                 'department':' food'},
                ])
```

Но на этот раз мы не будем создавать его в таком виде, а разделим на два датафрейма с товарами и продажами:

- в первом будет храниться исчерпывающая информация о товарах;
- во втором будут собраны данные о продажах.

Ниже показано, как можно осуществить это разделение:

```
products_df = DataFrame([{'product_id':23, 'name':'computer',
                          'wholesale_price': 500,
                          'retail_price':1000,
                          'department':'electronics'},
                        {'product_id':96, 'name':'Python Workout',
                          'wholesale_price': 35,
                          'retail_price':75, 'department':'books'},
                        {'product_id':97, 'name':'Pandas Workout',
                          'wholesale_price': 35,
                          'retail_price':75, 'department':'books'},
                        {'product_id':15, 'name':'banana',
                          'wholesale_price': 0.5,
                          'retail_price':1, 'department':'food'},
                        {'product_id':87, 'name':'sandwich',
                          'wholesale_price': 3,
                          'retail_price':5, 'department':'food'},
                        ])

sales_df = DataFrame([{'product_id': 23, 'date':'2021-August-10', 'quantity':1},
                      {'product_id': 96, 'date':'2021-August-10', 'quantity':5},
                      {'product_id': 15, 'date':'2021-August-10', 'quantity':3},
                      {'product_id': 87, 'date':'2021-August-10', 'quantity':2},
                      {'product_id': 15, 'date':'2021-August-11', 'quantity':1},
                      {'product_id': 96, 'date':'2021-August-11', 'quantity':1},
                      {'product_id': 23, 'date':'2021-August-11', 'quantity':2},
                      {'product_id': 87, 'date':'2021-August-12', 'quantity':2},
                      {'product_id': 97, 'date':'2021-August-12', 'quantity':6},
                      {'product_id': 97, 'date':'2021-August-12', 'quantity':1},
                      {'product_id': 87, 'date':'2021-August-13', 'quantity':2},
                      {'product_id': 23, 'date':'2021-August-13', 'quantity':1},
                      {'product_id': 15, 'date':'2021-August-14', 'quantity':2}
                      ])
```

Что мы сделали? Мы поместили всю информацию о товарах, которая редко будет меняться, в датафрейм `products_df`. При добавлении нового товара в ассортимент, а также удалении или изменении существующих мы будем вносить изменения в этот датафрейм. Но все данные о продажах мы будем хранить в отдельном датафрейме с именем `sales_df`. Здесь будет содержаться информация об идентификаторе проданного товара, количестве в чеке и дате транзакции. Созданные датафреймы вы можете увидеть на рис. 6.7 и 6.8 соответственно.

Это все, конечно хорошо, но как нам понять, какие именно товары были проданы и в каком количестве? И здесь нам поможет операция объединения данных. Мы можем легко объединить датафреймы `products_df` и `sales_df` в один общий датафрейм, в котором будет присутствовать информация из обеих таблиц.

А как `pandas` узнает, каким строкам из одной таблицы соответствуют строки из другой? По умолчанию для объединения данных используются индексы. При совпадении значений в индексах в двух датафреймах строки объединяются и становятся частью нового датафрейма, включая в себя все столбцы из обеих исходных таблиц.

	product_id	name	wholesale_price	retail_price	department
0	23	computer	500.0	1000	electronics
1	96	Python Workout	35.0	75	books
2	97	Pandas Workout	35.0	75	books
3	15	banana	0.5	1	food
4	87	sandwich	3.0	5	food

Рис. 6.7. Датафрейм products_df

	product_id	date	quantity
0	23	2021-August-10	1
1	96	2021-August-10	5
2	15	2021-August-10	3
3	87	2021-August-10	2
4	15	2021-August-11	1
5	96	2021-August-11	1
6	23	2021-August-11	2
7	87	2021-August-12	2
8	97	2021-August-12	6
9	97	2021-August-12	1
10	87	2021-August-13	2
11	23	2021-August-13	1
12	15	2021-August-14	2

Рис. 6.8. Датафрейм products_df

Таким образом, нам необходимо обеспечить, чтобы значения в индексах обоих датафреймов соответствовали друг другу. Самым очевидным кандидатом на превращение в индекс здесь выглядит столбец `product_id`, присутствующий в обоих датафреймах (см. рис. 6.9 и 6.10):

```
products_df = products_df.set_index('product_id')
sales_df = sales_df.set_index('product_id')
```

product_id	name	wholesale_price	retail_price	department
23	computer	500.0	1000	electronics
96	Python Workout	35.0	75	books
97	Pandas Workout	35.0	75	books
15	banana	0.5	1	food
87	sandwich	3.0	5	food

Рис. 6.9. Датафрейм `products_df` со столбцом `product_id` в качестве индекса

product_id	date	quantity
23	2021-August-10	1
96	2021-August-10	5
15	2021-August-10	3
87	2021-August-10	2
15	2021-August-11	1
96	2021-August-11	1
23	2021-August-11	2
87	2021-August-12	2
97	2021-August-12	6
97	2021-August-12	1
87	2021-August-13	2
23	2021-August-13	1
15	2021-August-14	2

Рис. 6.10. Датафрейм `sales_df` со столбцом `product_id` в качестве индекса

Теперь, когда оба наших датафрейма имеют единую точку опоры в виде индекса, мы можем создать на их основе новый единый датафрейм:

```
products_df.join(sales_df)
```

Результатом этого объединения будет датафрейм, состоящий из 13 строк и шести столбцов. В него будут включены сначала столбцы из датафрейма `products_df`, а затем – из датафрейма `sales_df`:

- `name`;
- `wholesale_price`;
- `retail_price`;
- `department`;
- `date`;
- `quantity`.

Каждая строка в новом датафрейме будет представлять собой результат сопоставления значений в индексах, как показано на рис. 6.11. Поскольку по одному товару у нас присутствует несколько продаж, мы получим больше строк в итоговой таблице по сравнению с исходными датафреймами.

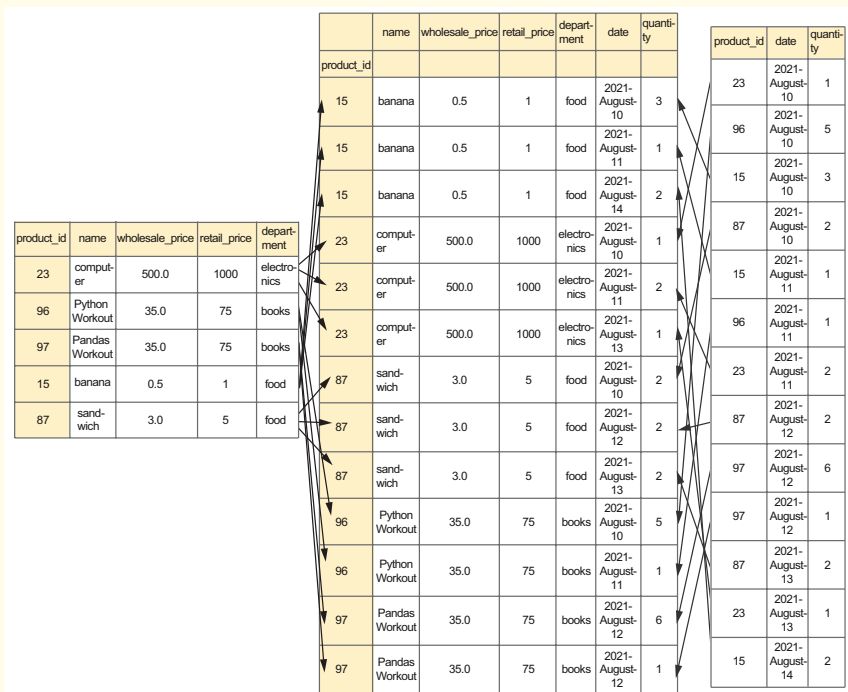


Рис. 6.11. Схематическое изображение объединения датафреймов `products_df` и `sales_df`

Теперь мы можем делать запросы к объединенному датафрейму. К примеру, мы можем узнать, сколько было продано каждого товара, следующим образом:

```
products_df.join(sales_df).groupby(
    'name')['quantity'].sum()
```



```
jan_2020_df['year'] = 2020
df = pd.concat([jan_2019_df, jan_2020_df])

df.groupby('year')['total_amount'].mean()
```

Вывод:

```
year
2019    15.682222
2020    18.663149
Name: total_amount, dtype: float64
```

Упражнение 30.2

```
# Группировка сначала по полю year, затем - по passenger_count
# В результате мы получим объект Series с множественным индексом
df.groupby(['year', 'passenger_count'])['total_amount'].mean()
```

Вывод:

```
year  passenger_count
2019  0.0              18.663658
      1.0              15.609601
      2.0              15.831294
      3.0              15.604015
      4.0              15.650307
      5.0              15.546940
      6.0              15.437892
      7.0              48.278421
      8.0              64.105517
      9.0              31.094444
2020  0.0              18.059724
      1.0              18.343110
      2.0              19.050504
      3.0              18.736862
      4.0              19.128092
      5.0              18.234443
      6.0              18.367962
      7.0              71.143103
      8.0              58.197059
      9.0              81.244211
Name: total_amount, dtype: float64
```

Упражнение 30.3

```
# Отсортировав данные по столбцу passenger_count, мы видим, что данные
# в нем не коррелируют с данными в других столбцах (кроме самого этого
# столбца, разумеется).
# Этим объясняется тот факт, что водителю такси не важно, сколько
# пассажиров везти
df.corr().sort_values('passenger_count')
```

Вывод:

	passenger_count	trip_distance	total_amount	year
year	-0.021602	0.001140	0.007657	1.000000
total_amount	-0.000136	0.004331	1.000000	0.007657
trip_distance	0.008974	1.000000	0.004331	0.001140
passenger_count	1.000000	0.008974	-0.000136	-0.021602

УПРАЖНЕНИЕ 31. Расходы туристов по странам

До начала пандемии я регулярно путешествовал по разным странам как по работе, так и для отдыха. Но с распространением заболевания многие страны ограничили въезд, что серьезно ударило по всей индустрии туризма. В этом упражнении мы поработаем с набором данных от OECD (ОЭСР – Организация экономического сотрудничества и развития), который в журнале «The Economist» озаглавили как «клуб самых богатых стран», и посмотрим, сколько разные государства зарабатывали на туристах. Как мы увидим, в этом наборе присутствует информация и о странах, отсутствующих в списке ОЭСР.

Вот что вам нужно будет сделать.

1. Загрузить данные из файла `oecd_tourism.csv` в датафрейм. Нам понадобятся следующие столбцы:
 - LOCATION – трехбуквенное обозначение страны;
 - SUBJECT – тип операции: INT_REC = доходы от туризма, INT-EXP = расходы на туризм;
 - TIME – год (целое число);
 - Value – сумма, выраженная в тысячах долларов.
2. Вывести пять стран, получивших в среднем больше всех денег от туризма за все годы.
3. Вывести пять стран, граждане которых потратили на туризм меньше остальных в среднем за все годы.
4. В отдельном CSV файле `oecd_locations.csv` собрана информация о некоторых странах в виде двух колонок: в первой указано трехбуквенное обозначение страны, как в файле `oecd_tourism.csv`, а во второй – полное название страны. Загрузите эти данные в отдельный датафрейм, установив в качестве индекса столбец с сокращенными названиями стран.
5. Объедините два датафрейма в один. В новом датафрейме не должно быть колонки LOCATION – вместо нее должна появиться колонка name с полными названиями стран.
6. Снова выполните запросы из шагов 2 и 3, осуществляющие поиск стран, больше остальных получающих от туризма и меньше всех тратящих на туризм. На этот раз названия стран должны быть полные, а не сокращенные.
7. Если не брать в расчет названия стран, отличаются ли результаты от полученных ранее и почему?

ПРИМЕЧАНИЕ. На примере имен столбцов и значений в этом наборе можно продемонстрировать некую несогласованность в данных. Как вы заметили, в столбце SUBJECT может присутствовать одно из двух возможных значений: INT_REC или INT-EXP. Почему в одном случае используется символ подчеркивания, а во втором – дефис? Хороший вопрос. Также вас может удивить, что имена всех столбцов, кроме столбца Value, написаны заглавными буквами. Такое случается в наборах данных сплошь и рядом. Старайтесь обращать внимание на подобные нюансы при первом знакомстве с данными. И если вы создаете наборы данных для кого-то другого, стремитесь к тому, чтобы информация в них была максимально согласована, насколько это возможно.

Подробный разбор

В этом упражнении мы потренируемся объединять разные датафреймы в один. Это позволит нам использовать в отчетах полные названия стран, а не их трехбуквенные обозначения. Итак, давайте приступим.

Для начала нам нужно загрузить основные данные в датафрейм. В исходном наборе данных есть множество столбцов, которые не помогут нам ответить на поставленные вопросы, так что мы выберем только нужные нам колонки:

```
tourism_filename = '../data/oecd_tourism.csv'

tourism_df = pd.read_csv(tourism_filename,
                        usecols=['LOCATION',
                                'SUBJECT',
                                'TIME',
                                'Value'])
```

Теперь в нашем датафрейме содержится вся информация о том, сколько денег страны тратили и получали от туризма в доковидную эпоху. К примеру, чтобы узнать, сколько французская экономика получила денег от туризма в 2016 году, когда у них проходил Чемпионат Европы по футболу, можно отфильтровать данные в столбцах SUBJECT, LOCATION и TIME значениями INT_REC, FRA и 2016 соответственно. В результате мы получим единственную строку из датафрейма, а столбец с именем Value покажет доходы страны от туризма.

А что, если нам нужно получить средний доход стран от туризма? Это можно сделать, применив цепочку методов следующим образом:

```
(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT_REC']
    ['Value']
    .mean()
)
```

Но это будет не слишком полезно, поскольку разные страны сильно отличаются в отношении потока туристов. Если разбить эту аналитику по странам, пользы будет гораздо больше.

Как получить средний доход от туризма по странам? Очень просто – сгруппировав отфильтрованные данные по полю LOCATION и применив операцию усреднения, как показано ниже:

```
(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT_REC']
    .groupby('LOCATION')['Value']
    .mean()
)
```

Вот что мы делаем в этом выражении.

1. Выбираем строки, в которых в столбце SUBJECT стоит значение INT_REC.
2. Группируем данные по столбцу LOCATION, оставляя по одному результату для каждой страны.
3. Запрашиваем только столбец Value.
4. Вызываем метод mean для усреднения результатов.

В итоге мы получим объект Series, в котором в качестве индекса будут присутствовать трехбуквенные сокращения стран, а в качестве результатов – средний доход от туризма по странам.

Нам необходимо вывести пять стран, получивших в среднем больше всех денег от туризма за все годы. Для этого мы отсортируем результаты в порядке убывания и воспользуемся методом head для получения первых строк:

```
(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT_REC']
    .groupby('LOCATION')['Value']
    .mean()
    .sort_values(ascending=False)
    .head()
)
```

Затем нас попросили написать запрос для вывода пяти стран, граждане которых потратили на туризм меньше остальных в среднем за все годы. Иными словами, теперь нас интересуют строки со значениями INT-EXP в столбце SUBJECT, и мы хотим получить пять стран с наименьшими средними расходами. Решение будет очень простым:

```
(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT-EXP']
    .groupby('LOCATION')['Value']
    .mean()
    .sort_values()
    .head()
)
```

Здесь мы использовали другой фильтр и иной порядок сортировки значений (по умолчанию значения сортируются по возрастанию). При таком раскладе метод head извлечет пять строк с наименьшими значениями.

Теперь нам необходимо выполнить объединение датафреймов для получения названий стран в привычном виде, а не в виде трехбуквенных сокращений. Для

этого воспользуемся созданным файлом CSV с именем `oecd_locations.csv`. Но сначала с ним нужно немного поработать. Дело в том, что в этом файле отсутствует строка заголовков, так что нам нужно вручную задать имена для столбцов.

Кроме того, мы планируем выполнять объединение наших датафреймов по трехбуквенным сокращениям стран, а значит, этот столбец необходимо сделать индексом при загрузке данных. В результате код загрузки данных в новый датафрейм будет выглядеть так:

```
locations_filename = '../data/oecd_locations.csv'

locations_df = pd.read_csv(locations_filename,
                           header=None,
                           names=['LOCATION', 'NAME'],
                           index_col='LOCATION')
```

Теперь объединим наши датафреймы `locations_df` и `tourism_df`. Проблема в том, что в датафрейме `tourism_df` столбец `LOCATION` не является индексом. Да, вы можете объединять датафреймы по обычным столбцам, но объединение по индексам выглядит более лаконично.

Мы сделаем следующее.

1. Создадим новый (анонимный) датафрейм на основе датафрейма `tourism_df`, в котором установим индекс на столбец `LOCATION`.
2. Выполним операцию объединения применительно к датафрейму `locations_df` и новому безымянному датафрейму с индексом на столбце `LOCATION`.
3. Присвоим результат новому датафрейму, который назовем `fullname_df`.

Процесс установки индекса в наших датафреймах показан на рис. 6.12 и 6.13:

```
fullname_df = locations_df.join(tourism_df.set_index('LOCATION'))
```

ПРИМЕЧАНИЕ. Датафрейм `fullname_df` насчитывает гораздо меньше строк, чем датафрейм `tourism_df` – 364 против 1234. Причина в том, что в датафрейм `locations_df` мы намеренно поместили информацию лишь о нескольких странах, а не о всех, и именно эта таблица указана при объединении слева, что делает ее значения в ключе объединения приоритетными.

В качестве индекса в датафрейме `fullname_df` выступает столбец с трехбуквенными сокращениями названий стран. Колонки в этом датафрейме будут следующие:

- NAME – полное имя страны, взятое из датафрейма `locations_df`;
- SUBJECT – расходы или доходы;
- TIME – год;
- Value – сумма.

Теперь, используя столбец `NAME` в качестве поля для группировки, мы можем получить в отчетах нормальные названия стран, а не сокращения. И в задании как раз говорится о том, что вам нужно выполнить ранее написанные запросы

применительно к объединенному датафрейму, процесс создания которого показан на рис. 6.14.

	LOCATION	SUBJECT	TIME	Value		LOCATION	SUBJECT	TIME	Value
0	AUS	INT_REC	2008	31159.8		AUS	INT_REC	2008	31159.8
1	AUS	INT_REC	2009	29980.7		AUS	INT_REC	2009	29980.7
2	AUS	INT_REC	2010	35165.5		AUS	INT_REC	2010	35165.5
3	AUS	INT_REC	2011	38710.1		AUS	INT_REC	2011	38710.1
4	AUS	INT_REC	2012	38003.7		AUS	INT_REC	2012	38003.7
1229	SRB	INT-EXP	2015	1253.644	→ <code>set_index('LOCATION')</code> →	SRB	INT-EXP	2015	1253.644
1230	SRB	INT-EXP	2016	1351.098		SRB	INT-EXP	2016	1351.098
1231	SRB	INT-EXP	2017	1549.183		SRB	INT-EXP	2017	1549.183
1232	SRB	INT-EXP	2018	1837.317		SRB	INT-EXP	2018	1837.317
1233	SRB	INT-EXP	2019	1999.313		SRB	INT-EXP	2019	1999.313

Рис. 6.12. Установка индекса на столбец LOCATION в датафрейме tourism_df

Так мы извлечем список из пяти стран, получивших в среднем больше всех денег от туризма за все годы, на основе объединенного датафрейма:

```
(
    fullname_df
    .loc[fullname_df['SUBJECT'] == 'INT_REC']
    .groupby('NAME')['Value']
    .mean()
    .sort_values(ascending=False)
    .head()
)
```

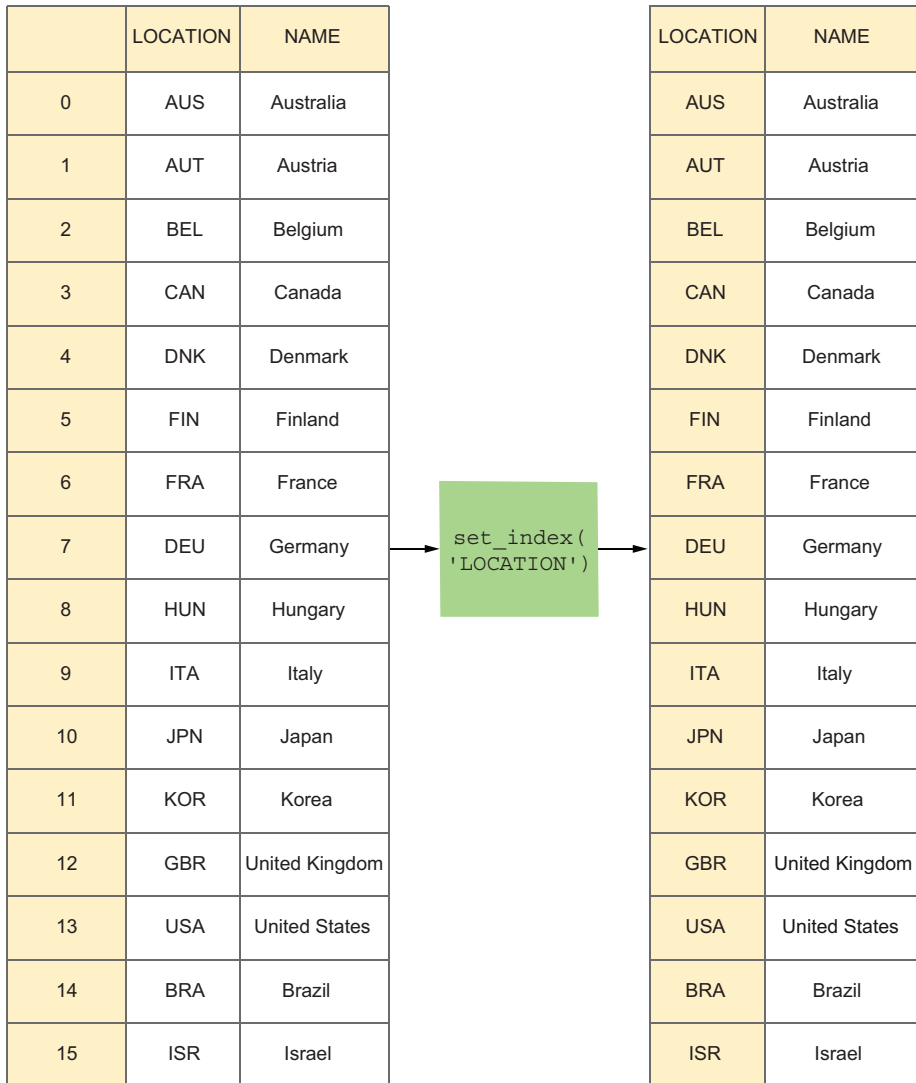


Рис. 6.13. Установка индекса на столбец `LOCATION` в датафрейме `locations_df`

А так – перечень из пяти стран, граждане которых потратили на туризм меньше остальных в среднем за все годы:

```
(
    fullname_df
    .loc[fullname_df['SUBJECT'] == 'INT-EXP']
    .groupby('NAME')['Value']
    .mean()
    .sort_values()
    .head()
)
```

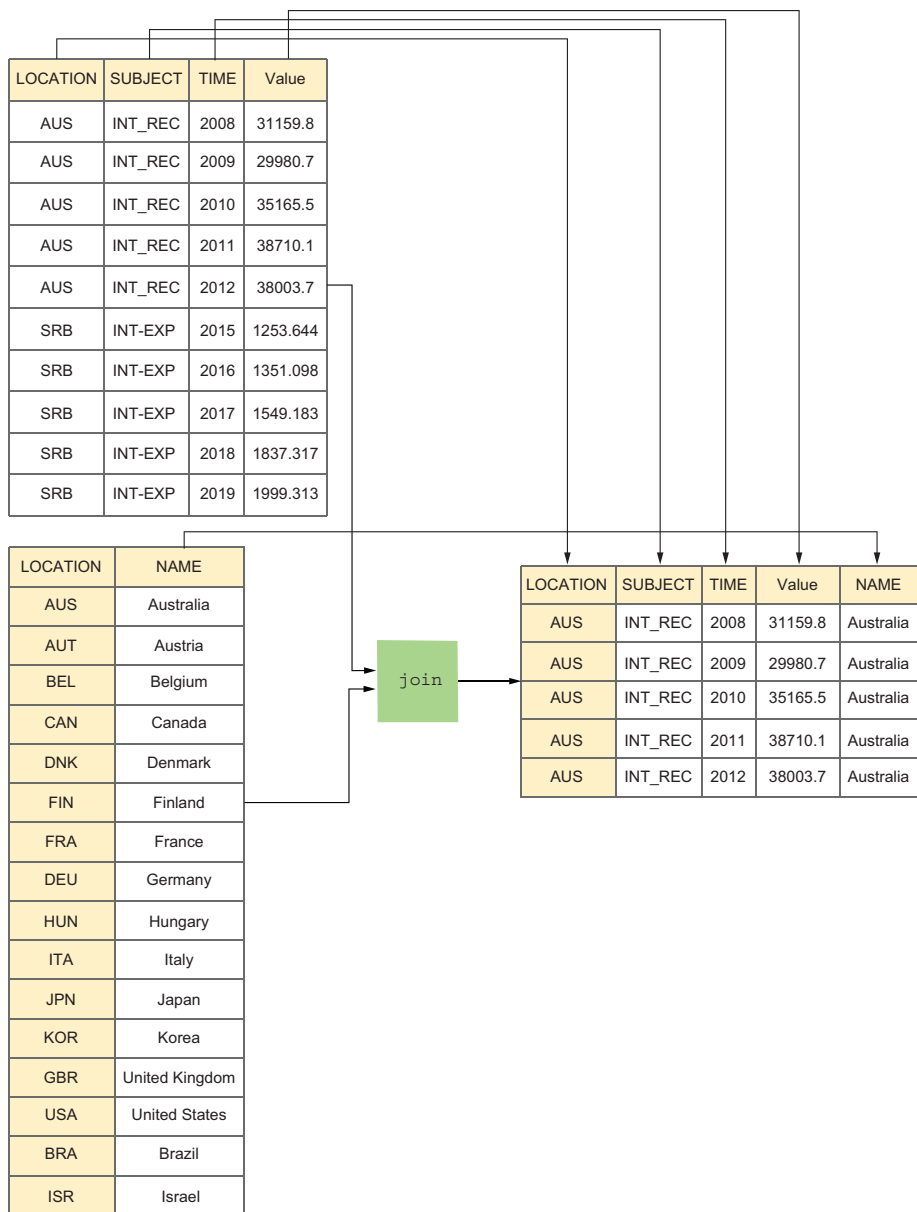


Рис. 6.14. Объединение датафреймов `locations_df` и `tourism_df`.

Обратите внимание, что строки со странами, отсутствующими в датафрейме `locations_df`, исключаются из результата

Наконец, нас попросили сравнить полученные результаты. Помимо отличающихся названий стран, мы видим, что в результатах, построенных на основе датафрейма `locations_df`, присутствуют данные далеко не по всем странам, которые есть в датафрейме `tourism_df`. Часть данных была потеряна в результате объединения датафреймов.

ПРИМЕЧАНИЕ. Во избежание таких потерь можно было воспользоваться другим типом объединения, но в этом случае полные названия стран подтянулись бы не для всех записей.

Решение

```

tourism_filename = '../data/oecd_tourism.csv'
tourism_df = pd.read_csv(tourism_filename,
                          usecols=['LOCATION',
                                   'SUBJECT', 'TIME', 'Value'])

(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT_REC']
    .groupby('LOCATION')['Value']
    .mean()
    .sort_values(ascending=False)
    .head()
)

(
    tourism_df
    .loc[tourism_df['SUBJECT'] == 'INT-EXP']
    .groupby('LOCATION')['Value']
    .mean()
    .sort_values()
    .head()
)

locations_filename = '../data/oecd_locations.csv'
locations_df = pd.read_csv(locations_filename,
                           header=None,
                           names=['LOCATION', 'NAME'],
                           index_col='LOCATION')

fullname_df = locations_df.join(
    tourism_df.set_index('LOCATION'))

(
    fullname_df
    .loc[fullname_df['SUBJECT'] == 'INT_REC']
    .groupby('NAME')['Value']
    .mean()
    .sort_values(ascending=False)
    .head()
)

(
    fullname_df
    .loc[fullname_df['SUBJECT'] == 'INT-EXP']
    .groupby('NAME')['Value']
    .mean()
)

```

```
.sort_values()
.head()
)
```

7

- ❶ Создаем датафрейм на основе данных из четырех столбцов файла CSV.
- ❷ Выбираем строки, в которых в столбце SUBJECT стоит значение INT_REC, для каждой страны извлекаем среднее значение по столбцу Value, сортируем результат в порядке убывания и берем первые пять значений.
- ❸ Выбираем строки, в которых в столбце SUBJECT стоит значение INT-EXP, для каждой страны извлекаем среднее значение по столбцу Value, сортируем результат в порядке возрастания и берем первые пять значений.
- ❹ Создаем датафрейм на основе другого файла CSV, установив в качестве имен столбцов значения LOCATION и NAME и сделав столбец LOCATION индексом.
- ❺ Создаем новый датафрейм как результат объединения датафреймов tourism_df и locations_df.
- ❻ В объединенном датафрейме выбираем строки со значением INT_REC в столбце SUBJECT, для каждой страны извлекаем среднее значение по столбцу Value, сортируем результат в порядке возрастания и берем первые пять значений.
- ❼ В объединенном датафрейме выбираем строки со значением INT-EXP в столбце SUBJECT, для каждой страны извлекаем среднее значение по столбцу Value, сортируем результат в порядке возрастания и берем первые пять значений.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/D9Yw>.

Дополнительные упражнения

1. Что произойдет, если при объединении таблиц поменять их местами? То есть вызвать метод join у датафрейма tourism_df, передав ему в качестве аргумента датафрейм locations_df? Вы получите тот же результат?
2. Рассчитайте средний доход от туризма не по странам, а по годам. Наблюдался ли спад активности туристов во время мирового финансового кризиса, начавшегося в 2008 году?
3. Переустановите индекс в датафрейме в состояние по умолчанию, чтобы он был числовым (в этом случае LOCATION и NAME станут обычными столбцами). Теперь вызовите метод join у датафрейма locations_df, указав, что вы хотите использовать для объединения столбец LOCATION, а не индекс (датафрейм, переданный в качестве аргумента, всегда будет участвовать в объединении по индексу).

Ответы на дополнительные упражнения

Упражнение 31.1

```
# Мы снова выполняем объединение слева, т. е. именно левый датафрейм
# (у которого вызван метод join) определяет, какие строки окажутся в
# результирующем датафрейме. Если совпадений в правом датафрейме не найдется,
# мы получим в столбце NAME значения NaN
```

```
tourism_df.set_index('LOCATION').join(locations_df)
```

Вывод:

	SUBJECT	TIME	Value	NAME
LOCATION				
AUS	INT_REC	2008	31159.800	Australia
AUS	INT_REC	2009	29980.700	Australia
AUS	INT_REC	2010	35165.500	Australia
AUS	INT_REC	2011	38710.100	Australia
AUS	INT_REC	2012	38003.700	Australia
...	...	...	...	...
SRB	INT-EXP	2015	1253.644	NaN
SRB	INT-EXP	2016	1351.098	NaN
SRB	INT-EXP	2017	1549.183	NaN
SRB	INT-EXP	2018	1837.317	NaN
SRB	INT-EXP	2019	1999.313	NaN

[1234 rows x 4 columns]

Упражнение 31.2

Мы действительно видим, что 2008, 2009 и 2010 годы замыкают список

```
fullname_df = locations_df.join(tourism_df.set_index('LOCATION'))
```

```
(
    fullname_df
    .loc[fullname_df['SUBJECT'] == 'INT_REC']
    .groupby('TIME')['Value']
    .mean()
    .sort_values(ascending=False)
)
```

Вывод:

TIME	
2019	62786.617333
2018	43185.853875
2017	40326.702250
2014	40043.334563
2016	39483.592062
2015	38912.695437
2013	37996.198750
2012	35628.632063
2011	34299.966375
2008	31757.065750
2010	30949.524125
2009	28505.886562

Name: Value, dtype: float64

Упражнение 31.3

```
locations_df = locations_df.reset_index()
tourism_df = tourism_df.set_index('LOCATION')
```

```
locations_df.join(tourism_df, on='LOCATION')
```

Вывод:

	LOCATION	NAME	SUBJECT	TIME	Value
0	AUS	Australia	INT_REC	2008	31159.8
0	AUS	Australia	INT_REC	2009	29980.7
0	AUS	Australia	INT_REC	2010	35165.5
0	AUS	Australia	INT_REC	2011	38710.1
0	AUS	Australia	INT_REC	2012	38003.7
..	...	...	...	...	...
15	ISR	Israel	INT-EXP	2015	7507.0
15	ISR	Israel	INT-EXP	2016	8210.3
15	ISR	Israel	INT-EXP	2017	8986.0
15	ISR	Israel	INT-EXP	2018	9974.7
15	ISR	Israel	INT-EXP	2019	10389.5

[364 rows x 5 columns]

Заключение

После загрузки данных в датафрейм вы можете обрабатывать и анализировать их самыми разными способами. В этой главе мы рассмотрели наиболее распространенные примеры: группировку данных для анализа и для включения/исключения строк, а также объединение датафреймов. Располагая этим инструментарием, можно выполнять достаточно сложный анализ данных. Упражнения из этой главы продемонстрировали, как и когда можно применять описанные приемы при реализации операций *разделить-применить-объединить*, составляющих львиную долю операций в pandas. В следующей главе мы воспользуемся этими наработками при решении более сложных задач.

Глава 7

Сложная группировка, объединение и сортировка

В предыдущей главе мы научились применять на практике три основных вида операций в `pandas`: группировку датафрейма по одному и более столбцам, объединение двух датафреймов, а также сортировку датафрейма по индексу или одному или нескольким столбцам. Как мы видели, эти операции могут быть крайне полезными при манипулировании данными и приведении их к виду, пригодному для анализа и интерпретации.

В этой главе мы углубимся в использование описанных ранее техник и приемов. Мы будем объединять данные из нескольких файлов CSV в один датафрейм, группировать и сортировать его по нескольким столбцам, а также воспользуемся методом `filter` для сохранения и исключения строк на основе групповых свойств. Эти упражнения помогут вам более эффективно применять уже знакомые вам методы при решении задач и правильно выбирать их в зависимости от стоящих перед вами вопросов.

УПРАЖНЕНИЕ 32. Температура в разных городах

Операция группировки – один из самых распространенных приемов при анализе данных. Причина в том, что почти всегда полезнее бывает анализировать не весь набор данных в целом, а отдельные его части, созданные по неким правилам, что позволяет также сравнивать их между собой. К примеру, нам может быть интересно, как люди проголосовали на недавних выборах. При этом, если вы хотите баллотироваться сами и убедить большинство людей проголосовать за вас, вам будет исключительно полезно знать предпочтения населения по группам возрастов, месту жительства и другим группирующим характеристикам.

В этом упражнении мы снова поработаем на практике с группировкой. Но до анализа данных вам придется довольно хитрым образом собрать их. В папке с данными представлены восемь файлов с информацией о погоде в восьми разных городах. Но сложность заключается в том, что эти восемь городов представляют четыре штата, и мы хотим, чтобы в нашем датафрейме были отдельные столбцы `city` и `state`, в которых будет храниться информация о городе и штате. При этом название города и штата содержится в названии файла.

Во всех интересующих нас файлах присутствуют столбцы с одинаковыми именами и в одном формате. Воспользуемся этим при объединении данных в один датафрейм.

Что вам нужно будет сделать?

1. Объединить данные из всех восьми файлов CSV в один датафрейм. Названия файлов следующие: `san+francisco,ca.csv`, `new+york,ny.csv`, `springfield,ma.csv`, `boston,ma.csv`, `springfield,il.csv`, `albany,ny.csv`, `los+angeles,ca.csv` и `chicago,il.csv`.
2. Нас интересуют только три первые колонки в файлах, в которых представлена дата и время измерения, а также максимальная и минимальная температура.
3. Добавить столбцы `city` и `state` в датафрейм, содержащие название города и штата из файла и позволяющие различать измерения.

Выполнив описанные выше действия, ответьте на следующие вопросы:

- совпадают ли начальная и конечная даты измерений для всех городов в наборе? Как вы можете это узнать?
- какая минимальная температура была зафиксирована в каждом городе?
- какая максимальная температура была зафиксирована в каждом *штате*?

Подробный разбор

Я часто говорю студентам, делающим первые шаги в программировании, что выбор структур данных и способы работы с ними критическим образом влияют на итоговый продукт. Работая с Python, вы должны очень скрупулезно относиться к выбору структур данных для решения той или иной задачи. Вам необходимо четко понимать, когда уместно использовать кортежи, когда – списки, когда – словари, а когда некое их сочетание.

Аналогичный совет для изучающих `pandas` будет звучать так: вы должны всегда стараться включать в датафрейм только ту информацию, которая необходима вам для обработки данных и их итогового представления. Зачастую это определяет действия, производимые с данными во время и после их загрузки из файлов. Но если вы добьетесь, чтобы в датафрейме были полностью очищенные данные, это поможет вам строить эффективные запросы на их основе.

В этом упражнении нам необходимо, чтобы в итоговом датафрейме присутствовали столбцы `city` и `state`, которых нет в исходных файлах, а также столбцы с датой и временем измерения, минимальной и максимальной температурой. Название города и штата мы можем извлечь из имен файлов. Мы загрузим данные из всех нужных нам файлов, после чего объединим их в один итоговый датафрейм.

Давайте подумаем, как правильно объединить информацию из нескольких файлов в один датафрейм. Мы умеем загружать данные из одного файла CSV с помощью функции `read_csv`, как показано ниже:

```
one_filename = 'new+york,ny.csv'
one_df = pd.read_csv(one_filename)
```

В качестве параметров мы используем следующие:

- `usecols` – для извлечения только нужных нам столбцов из файла CSV. В данном случае мы получим их по числовому индексу;
- `names` – для указания имен столбцов в датафрейме, чтобы во всех датафреймах столбцы назывались одинаково;
- `header` – для указания того, что в первой строке файла содержится информация о заголовках столбцов. Чтобы проигнорировать их и заменить своими именами.

В результате выражение для загрузки одного датафрейма будет выглядеть так:

```
one_df = (
    pd
    .read_csv(one_filename,
              usecols=[0, 1, 2],
              names=['date_time', 'max_temp',
                    'min_temp'],
              header=0)
)
```

Этот код позволит загрузить три нужных нам столбца из файла CSV. Теперь нам нужно добавить столбцы с городом и штатом и заполнить их значениями, взятыми из имени файла.

Для начала избавимся от расширения файла следующим образом:

```
base_filename = one_filename.removesuffix('.csv')
```

В моем случае нужные нам файлы собраны в директории `../data` (соседняя с нашей текущей директорией папка с именем `data`), так что полное имя файла будет выглядеть так: `../data/new+york,ny.csv`. Нам надо избавиться как от расширения, так и от пути к файлу, и мы это сделаем в одном выражении, как показано ниже:

```
one_filename.removeprefix('../data/').removesuffix('.csv')
```

В результате мы получим имя файла, которое сохраним в переменной. Далее нам необходимо извлечь из имени файла название города и штата, для чего мы воспользуемся методом `str.split`, возвращающим список строк, полученный в результате разделения исходной строки по указанному разделителю. В нашем случае разделителем является запятая, так что получить имя города и штата можно так:

```
one_filename.removeprefix('../data/').removesuffix('.csv').split(',')
```

С учетом специфики имен файлов мы можем утверждать, что это выражение вернет список из двух элементов, соответствующих названиям города и штата, в которых производилось измерение. Python позволяет очень элегантно извлекать элементы списка в переменные, как показано ниже:

```
city, state = (
    one_filename
    .removeprefix('../data/')
```

```

        .removesuffix('.csv')
        .split(',')
    )

```

В результате в переменной `city` у нас будет храниться название города, а в переменной `state` – название штата.

Как мы можем создать в нашем датафрейме столбцы и заполнить их значениями из наших переменных?

Один из способов состоит в присваивании скалярных значений новым столбцам, что аналогично вводу этого значения в каждую строку в столбце:

```

one_df['city'] = city
one_df['state'] = state

```

Но есть один нюанс. Если название города состоит больше чем из одного слова, то эти слова в нем будут разделены символом `+`, а не пробелом и будут написаны строчными буквами. И буквы, обозначающие штат, также будут строчные. Давайте это исправим:

```

one_df['city'] = city.replace('+', ' ').title()
one_df['state'] = state.upper()

```

Хотя этот код работает, мы должны воспользоваться цепочкой методов при импорте файлов. Это можно сделать, применив метод `assign`, который позволяет добавить один или несколько столбцов в датафрейм:

```

one_df = (
    pd
    .read_csv(one_filename,
              usecols=[0, 1, 2],
              names=['date_time', 'max_temp',
                    'min_temp'],
              header=0)
    .assign(city=city.replace('+', ' ').title(),
            state=state.upper())
)

```

Функция `read_csv` создает новый датафрейм на основе данных из файла CSV, перед возвращением которого мы добавляем в него столбцы `city` и `state`. Результатом будет датафрейм `one_df` с пятью желаемыми столбцами.

Как можно воспользоваться этим шаблоном для сбора данных из всех восьми файлов CSV в единый датафрейм? Мы можем воспользоваться функцией `pd.concat`, принимающей на вход список датафреймов и возвращающей объединенный датафрейм. Таким образом, осталось получить отдельные датафреймы, соответствующие каждому файлу CSV.

Мы будем проходить циклом по списку имен файлов, возвращенному функцией `glob.glob`, которая входит в стандартную библиотеку языка Python. На каждой итерации мы будем читать данные из файла, дополнять их колонками с городом и штатом и добавлять полученный датафрейм в общий список, как показано ниже. По завершении цикла мы можем воспользоваться функцией `pd.concat` для объединения всех датафреймов:

```

import glob

all_dfs = []

for one_filename in glob.glob('../data/*,*.csv'):
    print(f'Loading {one_filename}...')

    city, state = (
        one_filename
        .removeprefix('../data/')
        .removesuffix('.csv')
        .split(',')
    )

    one_df = (
        pd
        .read_csv(one_filename,
                  usecols=[0, 1, 2],
                  names=['date_time', 'max_temp',
                        'min_temp'],
                  header=0)
        .assign(city=city.replace('+', ' ').title(),
                state=state.upper())
    )

    all_dfs.append(one_df)

```

Здесь мы проходим в цикле по именам файлов с шаблоном `*.csv`. Далее создаем новый датафрейм на основе текущего файла CSV, добавляем (с помощью метода `assign`) столбец `city` с информацией о городе, извлеченной из имени файла, и столбец `state` с двухбуквенным сокращением названия штата из того же имени файла.

Все полученные датафреймы мы собираем в заранее инициализированный список `all_dfs`. Теперь можно воспользоваться функцией `pd.concat` для объединения данных в единый датафрейм (схематически этот процесс показан на рис. 7.1):

```
df = pd.concat(all_dfs)
```

Теперь, когда у нас есть большой датафрейм с пятью столбцами и всеми наблюдениями по всем восьми городам из четырех штатов, мы можем начать отвечать на поставленные вопросы. Для начала ответим на вопрос о том, совпадают ли начальная и конечная даты измерений для всех городов в наборе, и как вы можете это узнать. У нас есть столбец с именем `date_time`, в котором записана дата и время фиксации измерения. Если мы вычислим минимум и максимум по этому столбцу для каждого города, мы сможем произвести сравнение.

Именно для подобных операций и был придуман метод `groupby` – взять датафрейм и вычислить агрегацию (например, минимум или максимум) для каждого уникального значения из того или иного столбца.

Но здесь есть нюанс, состоящий в том, что нам недостаточно сгруппировать данные только по городу, поскольку в разных штатах могут встречаться города с

одинаковыми именами. В нашем наборе данных есть город Спрингфилд из штата Иллинойс и город с таким же названием из штата Массачусетс. Таким образом, мы будем группировать данные по штату и городу. Это также позволит сделать отчет более привлекательным. Запрос в этом случае будет выглядеть так:

```
(
    df.groupby(['state', 'city'])['date_time'].min()
    .sort_values()
)
```



Рис. 7.1. Использование функции `pd.concat` для объединения нескольких датафреймов

Вывод:

```
state  city
CA     Los Angeles      2018-12-11 00:00:00
       San Francisco    2018-12-11 00:00:00
IL     Chicago          2018-12-11 00:00:00
       Springfield      2018-12-11 00:00:00
MA     Boston           2018-12-11 00:00:00
       Springfield      2018-12-11 00:00:00
NY     Albany           2018-12-11 00:00:00
       New York         2018-12-11 00:00:00
Name: date_time, dtype: object
```

Здесь мы говорим pandas, что хотим получить минимальные значения из столбца `date_time` для каждой уникальной комбинации значений из столбцов `state` и `city`. Далее мы сортируем полученные значения, чтобы можно было легко определить, принадлежат ли они одному временному отрезку. Для получения максимальных значений можно воспользоваться функцией `max`:

```
(
    df.groupby(['state', 'city'])['date_time'].max()
    .sort_values()
)
```

Выполнив эти запросы и сравнив результаты, мы можем понять, что измерения во всех городах были начаты 22 декабря 2018 года, а завершены 11 марта 2019-го. Как мы увидим в главе 9, pandas позволяет работать непосредственно с датами и временем и выполнять над ними вычисления. В нашем случае в столбце `date_time` представлена текстовая информация, что позволяет нам применять к нему лишь базовые операции агрегации, но не такие изощренные, как в случае с объектами `timestamp`.

Далее нас спросили, какая минимальная температура была зафиксирована в каждом городе. С этим тоже позволит справиться метод `groupby`, но на этот раз нас интересуют сами скалярные значения, а не их сравнение. Минимальные температуры у нас содержатся в столбце `min_temp`. А значит, собрать все наименьшие минимальные температура в разрезе городов мы можем следующим образом:

```
df.groupby(['state', 'city'])['min_temp'].min()
```

Вывод:

```
state  city
CA     Los Angeles      4
       San Francisco    3
IL     Chicago        -28
       Springfield     -25
MA     Boston         -14
       Springfield     -20
NY     Albany         -19
       New York        -14
Name: min_temp, dtype: int64
```

В результате мы получим объект Series, в индексе которого будут собраны уникальные комбинации из городов и штатов, а в значениях представлены минимальные значения из столбца min_temp. Мы можем предположить, что измерения делались зимой, поскольку у нас много значений ниже нуля.

Наконец, нас попросили узнать, какая максимальная температура была зафиксирована в каждом штате, а не в городе. Это означает, что нам достаточно сгруппировать данные по штатам:

```
df.groupby('state')['max_temp'].max()
```

Вывод:

```
state
CA      23
IL      16
MA      17
NY      15
Name: max_temp, dtype: int64
```

Таким образом мы получим максимальную температуру, которая была зафиксирована в каждом из штатов. Обратите внимание, что в наших данных представлены всего четыре штата и восемь городов, так что в выводе окажется лишь четыре строки. Количество строк в сгруппированных данных всегда равно числу уникальных значений в столбце или столбцах, по которым выполняется группировка.

Конечно, мы могли бы в предыдущем запросе воспользоваться и методом agg для расчета сразу двух показателей:

```
(
    df.groupby(['state', 'city'])['date_time']
    .agg(['min', 'max'])
)
```

Решение

```
import glob
```

```
all_dfs = []
```

```
for one_filename in glob.glob('../data/*/*.csv'):
    print(f'Loading {one_filename}...')
```

```
    city, state = (
        one_filename
        .removeprefix('../data/')
        .removesuffix('.csv')
        .split(',')
    )
```

```
    one_df = (
        pd
        .read_csv(one_filename,
```

```

        usecols=[0, 1, 2],
        names=['date_time',
               'max_temp',
               'min_temp'],
        header=0)
    .assign(city=city.replace('+', ' ').title(),
            state=state.upper())
)
all_dfs.append(one_df)

df = pd.concat(all_dfs)

df.groupby(['state', 'city'])['date_time'].min().sort_values()

df.groupby(['state', 'city'])['date_time'].max().sort_values()

df.groupby(['state', 'city'])['min_temp'].min()
df.groupby('state')['max_temp'].max()

```

- ❶ Создаем пустой список.
- ❷ Используем функцию `glob.glob` для получения всех имен файлов по шаблону и запускаем цикл по ним.
- ❸ Метод `str.split` используется для извлечения частей из строки.
- ❹ Нам нужны только первые три колонки в каждом файле CSV.
- ❺ Задаем имена для загруженных столбцов.
- ❻ В первой строке файла (индекс 0) содержатся заголовки.
- ❼ Добавляем в датафрейм столбец `city`.
- ❽ Добавляем в датафрейм столбец `state`.
- ❾ Добавляем новый датафрейм в список `all_dfs`.
- ❿ Собираем один датафрейм из списка датафреймов.
- ⓫ Вычисляем самое раннее измерение для каждого города и штата.
- ⓫ Вычисляем самое позднее измерение для каждого города и штата.
- ⓫ Извлекаем минимальную зафиксированную температуру для каждого города.
- ⓫ Извлекаем максимальную зафиксированную температуру для каждого штата.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/ddQ0>.

Дополнительные упражнения

1. Примените метод `describe` к расчету минимальной и максимальной температуры для каждой комбинации города и штата.
2. Метод `describe` сработал, но мы видим лишь первые и последние несколько строк результата. С помощью функции `pd.set_option` измените значение параметра `display_max_rows` таким образом, чтобы можно было в Jupyter видеть все результаты. Затем верните параметру значение 10.
3. Какова средняя разница температур (т. е. максимальная – минимальная) для каждого города в нашем наборе данных?

Оконные функции

Давайте представим, что у нас есть датафрейм, содержащий информацию о продажах за последний год:

```
df = DataFrame({'sales':[100, 150, 200, 250,
                        200, 150, 300, 400,
                        500, 100, 300, 200],
               'quarters': 'Q1 Q2 Q3 Q4'.split()*3})
```

Мы уже знаем, какими способами можно анализировать подобные данные. Например:

- мы можем вычислить среднее значение и другие агрегаты для всех представленных кварталов, применив метод `mean` к столбцу `sales`;
- мы можем воспользоваться методом `groupby` применительно к столбцу `quarters` и затем вызвать метод `mean` у полученного в результате объекта `DataFrameGroupBy`, чтобы узнать, какие средние показатели по продажам наблюдались у нас по кварталам.

Это очень полезные и важные разрезы аналитики. А что, если нам понадобится узнать накопительные показатели продаж по кварталам? То есть для первого квартала это данные по Q1, для второго – Q1+Q2, для третьего – Q1+Q2+Q3 и т. д. до результата `df['sales'].sum()`.

Для выполнения подобного рода операций в `pandas` предусмотрены *оконные функции* (*window function*). Существуют разные типы оконных функций, но их смысл сводится к тому, чтобы выполнять агрегатные функции, такие как `mean`, применительно к неким подмножествам строк в датафрейме.

Пример с накопительными продажами по кварталам, который мы описали выше, – это классический случай применения оконных функций. Здесь мы имеем дело с так называемым *расширяющимся окном* (*expanding window*), поскольку каждый раз вычисление производится применительно к увеличивающемуся набору строк: сначала к одной, затем к двум, трем и т. д. до целого датафрейма.

Мы можем воспользоваться следующим выражением:

```
df['sales'].expanding().sum()
```

В результате мы получим объект `Series`, в котором в качестве значений будут выступать накопительные суммы по столбцу `sales` с начала набора данных и до текущей точки. Поскольку первые четыре значения в столбце у нас 100, 150, 200 и 250, в результате применения метода `expanding` мы получим накопительные суммы 100, 250, 450 и 700, как показано на рис. 7.2.

Нам также может понадобиться рассчитать не накопительную сумму, а накопительное среднее по продажам. Мы вольны применять метод `mean` и любые другие функции агрегации:

```
df['sales'].expanding().mean()
```

В результате мы получим цифры 100, 125, 150 и 175, характеризующие динамику изменения среднего значения продаж с течением кварталов.

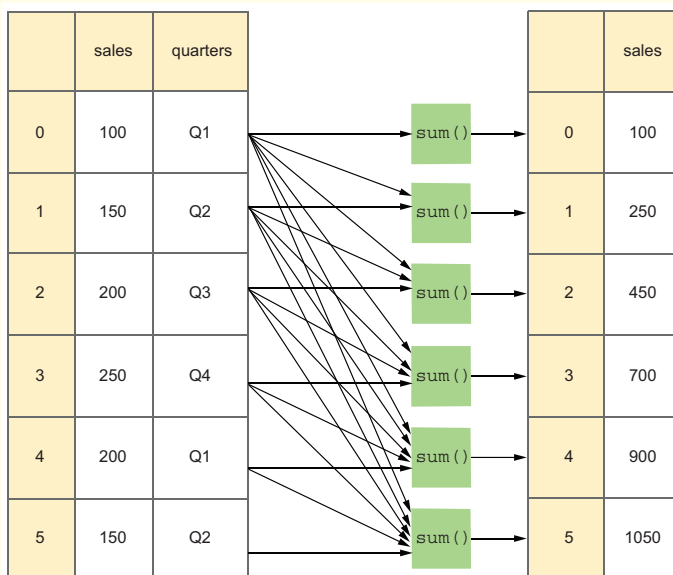


Рис. 7.2. Схематическое изображение применения оконной функции `expanding` совместно с методом `sum`

Кроме того, мы можем использовать *скользящую оконную функцию* (rolling window function). В этом случае мы должны определить заранее, сколько строк будут входить в так называемое окно. К примеру, если мы определим окно из трех строк, функция агрегации будет последовательно запускаться для строк с индексами 0–2, 1–3, 2–4 и т. д. вплоть до достижения конца датафрейма. Допустим, если мы хотим определить средние значения для близко расположенных строк, мы можем сделать это так (см. рис. 7.3):

```
df['sales'].rolling(3).mean()
```

Здесь с помощью функции `rolling` мы задаем параметры скольжения окна по датафрейму, а значение параметра 3 указывает на то, что в окно должны входить три соседние строки. Таким образом, мы последовательно вызываем метод `mean` для диапазонов строк 0–2, 1–3, 2–4, 3–5 и т. д. В итоговом объекте `Series` скользящие значения будут располагаться в третьей и последней ячейке каждого окна. Именно поэтому в строках с индексами 0 и 1 будет стоять пропущенное значение, что видно на рис. 7.3.

Третий тип оконных функций представлен функцией `pct_change`. При ее вызове мы получим новый объект `Series` со значением `NaN` в строке с индексом 0. В остальных строках будет указано процентное изменение значения по сравнению с предыдущей строкой:

```
df['sales'].pct_change()
```

На выходе мы увидим следующий объект `Series`:

```
0    NaN
1    0.500000
2    0.333333
3    0.250000
```

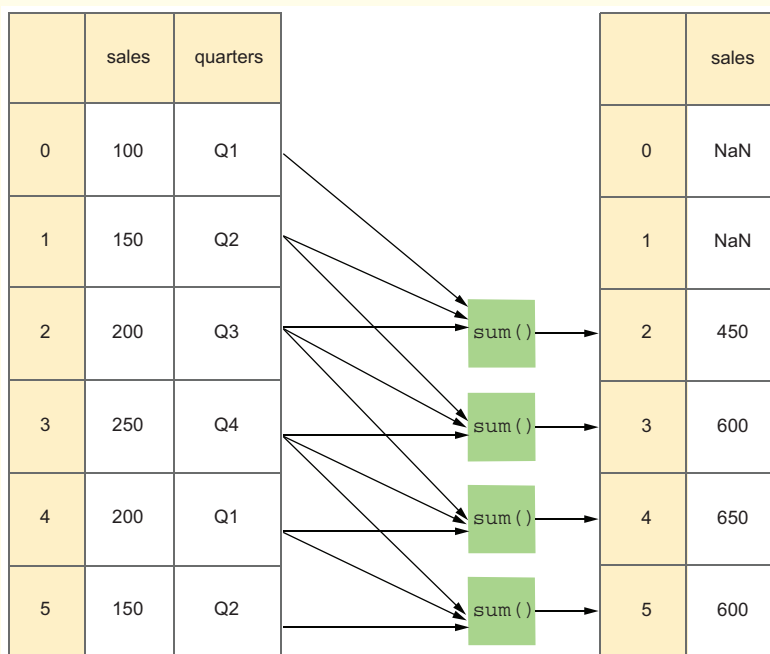


Рис. 7.3. Схематическое изображение применения скользящей оконной функции rolling совместно с методом mean

Эти результаты рассчитываются по формуле

(текущая_строка - предыдущая_строка) / предыдущая_строка:

- в строке с индексом 0 всегда будет располагаться значение NaN;
- в строке с индексом 1 будет результат вычисления $(150 - 100) / 100$;
- в строке с индексом 2 будет результат вычисления $(200 - 150) / 150$;
- в строке с индексом 3 будет результат вычисления $(250 - 200) / 200$.

Функция pct_change бывает полезна при определении относительных изменений значений от строки к строке. Если вам нужны абсолютные показатели, а не относительные, вы можете воспользоваться методом diff.

Ответы на дополнительные упражнения

Упражнение 32.1

```
# Группируя данные по городу и штату, извлекаем значения min_temp и max_temp
# Затем применяем метод describe, возвращающий датафрейм
df.groupby(['state', 'city'])[['min_temp', 'max_temp']].apply(DataFrame.describe)
```

Вывод:

```

                                min_temp    max_temp
state city
CA    Los Angeles count    728.000000    728.000000
```

```

          mean    10.637363    17.054945
          std      2.705200      2.708640
          min      4.000000     12.000000
          25%      9.000000     15.000000
          ...
NY      New York    min    -14.000000   -12.000000
          25%      -4.000000      2.000000
          50%       0.000000      4.000000
          75%       2.000000      7.000000
          max      12.000000     15.000000

```

[64 rows x 2 columns]

Упражнение 32.2

```

pd.set_option('display.max_rows', 1000)
df.groupby(['state', 'city'])[['min_temp', 'max_temp']].apply(DataFrame.describe)

```

Вывод не приводится для экономии места.

```
pd.set_option('display.max_rows', 10)
```

Упражнение 32.3

```

# Воспользуемся lambda для вычисления разницы для каждого значения
# в группе, а затем усредним результаты
df.groupby(['state', 'city'])[['min_temp', 'max_temp']].apply(lambda g:
np.mean(g.max() - g.min()) )

```

Вывод:

```

state  city
CA      Los Angeles    12.0
        San Francisco    8.0
IL      Chicago       34.0
        Springfield    35.5
MA      Boston        26.0
        Springfield    28.5
NY      Albany        26.5
        New York       26.5
dtype: float64

```

УПРАЖНЕНИЕ 33. Оценки за вступительные тесты, часть 2

В упражнении 22 мы рассмотрели набор данных, посвященный оценкам за стандартизированные вступительные тесты (SAT) в университет. Что касается этих тестов, далеко не все верят в их объективность, и многие считают, что студенты из более богатых семей чаще получают высокие баллы. У нас есть все исходные данные, чтобы подтвердить или опровергнуть эту гипотезу. Мы исследуем оценки по математике на предмет каких-то зависимостей.

Вот что вам нужно будет сделать.

1. Прочитайте в датафрейм файл с данными (sat-scores.csv). На этот раз нам понадобятся следующие колонки: Year, State.Code, Total.Math, Family Income. Less than 20k.Math, Family Income.Between 20-40k.Math, Family Income.Between 40-60k.Math, Family Income.Between 60-80k.Math, Family Income.Between 80-100k.Math и Family Income.More than 100k.Math.
2. Переименуйте столбцы с доходами семей, чтобы имена стали более емкими. Мой вариант: income<20k, 20k<income<40k, 40k<income<60k, 60k<income<80k, 80k<income 100k и income>100k.
3. Рассчитайте средние оценки по математике для каждой группы доходов и отсортируйте результаты по годам.
4. Для каждого года в наборе данных определите, насколько в среднем изменяются оценки в группах при повышении уровня дохода семьи. Можете ли вы определить на глаз, были ли в каком-нибудь году случаи с более низкими оценками в группах с более высокими доходами относительно следующих за ними по уровню дохода семей?
5. Какая группа дохода продемонстрировала наибольший прирост в оценках по сравнению со следующей группой?
6. Можно ли рассчитать, студенты из каких групп дохода постоянно (т. е. в каждом году) получали в среднем худшие баллы в сравнении со следующей группой?

Подробный разбор

В этом упражнении мы постараемся ответить на интересующие нас и общественность вопросы на примере реальных данных. Что мы будем делать с полученными выводами – это уже следующий вопрос. Для начала нам необходимо загрузить данные из файла CSV в датафрейм. Нас интересуют только оценки по математике. Загрузим данные следующим образом:

```
filename = '../data/sat-scores.csv'

df = pd.read_csv(filename,
                  usecols=['Year', 'State.Code', 'Total.Math',
                           'Family Income.Less than 20k.Math',
                           'Family Income.Between 20-40k.Math',
                           'Family Income.Between 40-60k.Math',
                           'Family Income.Between 60-80k.Math',
                           'Family Income.Between 80-100k.Math',
                           'Family Income.More than 100k.Math'])
```

Обратите внимание, что мы не задали индекс при чтении данных. Хотя зачастую бывает полезно указать столбцы, которые будут использоваться в качестве индекса, в момент чтения данных из файла, в нашем случае анализ будет почти полностью строиться на группировке данных. Да, можно группировать данные по столбцам, заданным в виде индекса, но в этом нет никакой дополнительной пользы. Именно поэтому мы решили оставить в датафрейме индекс по умолчанию – в виде числовых значений, начинающихся с нуля.

Теперь дадим столбцам с уровнями дохода более короткие имена, чтобы наши выражения были более лаконичными. Мы могли бы для указания имен столбцов

воспользоваться параметром `names`, но в этом случае нам пришлось бы использовать целочисленные значения для определения того, какие столбцы загружать из файла CSV. Мне такой подход всегда казался менее логичным и более трудным для отладки и поддержки.

Так что сначала мы загрузили столбцы с их исходными именами, а затем уже можем воспользоваться атрибутом `df.columns` для установки новых имен:

```
df.columns = ['Year', 'State.Code', 'Total.Math',
              'income<20k',
              '20k<income<40k',
              '40k<income<60k',
              '60k<income<80k',
              '80k<income<100k',
              'income>100k',
              ]
```

В более старых версиях `pandas` присвоение столбцам имен с помощью атрибута `df.columns` было связано в риском получения неправильного порядка столбцов. В этом случае лучше использовать функцию `df.rename` с передачей ей в качестве параметра `columns` словаря, в котором в виде ключей перечислены старые имена столбцов, а в виде значений – новые:

```
df = df.rename(
    columns={
        'Family Income.Less than 20k.Math':'income<20k',
        'Family Income.Between 20-40k.Math':'20k<income<40k',
        'Family Income.Between 40-60k.Math':'40k<income<60k',
        'Family Income.Between 60-80k.Math':'60k<income<80k',
        'Family Income.Between 80-100k.Math':'80k<income<100k',
        'Family Income.More than 100k.Math':'income>100k'
    })
```

Теперь, когда мы выбрали из файла только то, что нам нужно, и дали столбцам короткие и понятные имена, мы можем приступить к анализу. Сначала нас попросили рассчитать средние оценки по математике для каждой группы доходов и отсортировать результаты по годам. Сделаем это следующим образом:

```
df.groupby('Year').mean(numeric_only=True).sort_index()
```

Этот запрос похож на те, что мы использовали раньше. Наша цель состоит в том, чтобы вызвать метод `mean` для всех столбцов в датафрейме `df`, сгруппировав результаты по годам. Это поможет ответить на вопрос о том, какие средние баллы за тест по математике наблюдались в каждом году по разным группам дохода.

Поскольку мы сгруппировали данные по колонке `Year`, она не была включена в итоговый вывод. Но почему в список столбцов не попал столбец с именем `State.Code`? Потому что мы передали методу `mean` параметр `numeric_only=True`, тем самым исключив все нечисловые столбцы. В более ранних версиях `pandas` такие столбцы игнорировались по умолчанию, но теперь нам нужно либо явным образом выбирать только числовые столбцы, либо передавать параметр `numeric_only`.

Более того, поскольку мы в качестве группирующего поля выбрали столбец `Year`, именно он оказался представлен в новом датафрейме в качестве индекса.

Для надежности мы также вызвали метод `sort_index`, чтобы гарантировать вывод данных от более ранних годов к более поздним.

Далее нас попросили для каждого года в наборе данных определить, насколько в среднем изменяются оценки в группах при повышении уровня дохода семьи. Это значит, что нам, например, необходимо сначала определить средний балл за тест по математике у студентов из семей с самым низким уровнем дохода (`income<20k`), а затем сравнить его со средним баллом для следующего уровня (`20k<income<40k`). Возможно, мы увидим между этими показателями ничтожно малую разницу, что с определенной долей уверенности позволит нам сказать, что баллы за тесты по математике не коррелируют с уровнем дохода семей.

Как мы можем выполнить подобное сравнение? Мы воспользуемся функцией `pct_change`, с которой познакомились во врезке с оконными функциями.

Итак, нам нужно сравнить показатели по годам и уровням дохода. Но мы знаем, что функция `pct_change` работает со строками, а не со столбцами, тогда как в нашем наборе данных информация об уровнях дохода семей располагается именно в столбцах. Значит, нам нужно перевернуть наш датафрейм таким образом, чтобы годы переместились в столбцы, а уровни дохода – в строки. Для этого можно воспользоваться методом `transpose`, краткой аббревиатурой которого является единственная заглавная буква `T`. В результате вызова этого метода мы получим новый датафрейм, в котором строки и столбцы будут поменаны местами, как показано на рис. 7.4:

```
df.groupby('Year')[['income<20k',
                    '20k<income<40k',
                    '40k<income<60k',
                    '60k<income<80k',
                    '80k<income<100k',
                    'income>100k']].mean().T
```

Свойство `T` как сокращение от метода `transpose`

Метод `transpose` вызывается, как любой другой метод в `pandas`, с использованием круглых скобок, как показано ниже:

```
df.transpose()
```

У этого метода есть короткий псевдоним в виде свойства `T`, которое не является методом, а значит, и обращение к нему осуществляется без скобок:

```
df.T
```

В обоих случаях мы получаем новый датафрейм, тогда как исходный датафрейм остается без изменений.

Теперь мы можем вызвать метод `pct_change` применительно к этому новому датафрейму:

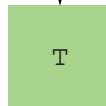
```
(
    df
    .groupby('Year')
```

```

[['income<20k',
 '20k<income<40k',
 '40k<income<60k',
 '60k<income<80k',
 '80k<income<100k',
 'income>100k']]
.mean()
.T
.pct_change()
)

```

	Year	State.Code	Total.Math	income<20k	20k<income<40k	40k<income<60k	60k<income<80k	80k<income<100k	income>100k
162	2008	CO	572	555	567	571	578	533	583
428	2013	GA	488	451	470	484	496	426	530
263	2010	AZ	527	491	506	523	531	472	553
490	2014	ME	471	494	520	535	555	468	569
334	2011	MI	605	549	565	590	609	528	626



	162	428	263	490	334
Year	2008	2013	2010	2014	2011
State.Code	CO	GA	AZ	ME	MI
Total.Math	572	488	527	471	605
income<20k	555	451	491	494	549
20k<income<40k	567	470	506	520	565
40k<income<60k	571	484	523	535	590
60k<income<80k	578	496	531	555	609
80k<income<100k	533	426	472	468	528
income>100k	583	530	553	569	626

Рис. 7.4. Пример использования свойства T для транспонирования датафрейма

В результате мы получим датафрейм, в котором в качестве столбцов будут присутствовать годы (с 2005-го по 2015-й), а в качестве строк – уровни дохода. Значения в этом датафрейме представляются в виде дробных чисел, выражающих долю текущего значения от предыдущего, т. е. изменение среднего балла для текущей группы по уровню дохода от группы, стоящей на одну ступеньку ниже, в указанном году. Для группы с минимальным уровнем дохода в ячейках будут

содержаться значения NaN, что объясняется отсутствием базы для сравнения. Это видно ниже:

Year	2005	2006	2007	...	2013	2014	2015
income<20k	NaN	NaN	NaN	...	NaN	NaN	NaN
20k<income<40k	0.069618	0.041450	0.049796	...	0.043346	0.034768	0.045059
40k<income<60k	0.025645	0.021259	0.026368	...	0.017489	0.023743	0.026038
60k<income<80k	0.023999	0.029085	0.023462	...	0.032817	0.030279	0.028277
80k<income<100k	-0.221054	-0.162486	-0.160846	...	-0.126606	-0.154137	-0.174429
income>100k	0.338116	0.241855	0.234199	...	0.185319	0.209002	0.259097

[6 rows x 11 columns]

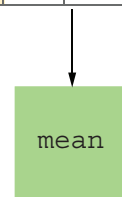
Чисто визуально заметно, что почти во всех случаях наблюдается рост среднего балла с увеличением уровня дохода семьи. К примеру, студенты из семей с уровнем дохода от 20 000 до 40 000 долл. в год в среднем получают баллы на 3–7 % выше, чем студенты из семей с доходом 20 000 долл. и меньше. И так с каждым уровнем, кроме одного.

Интересно отметить, что студенты из семей с уровнем дохода от 80 000 до 100 000 долл. во все годы получали гораздо худший средний балл в сравнении с предыдущим уровнем дохода, что видно по соответствующей строке с отрицательными значениями. Я не знаю точно, в чем причина такой закономерности, но она есть и наблюдается во всем наборе данных.

Далее нас спросили, какая группа дохода продемонстрировала наибольший прирост в оценках по сравнению со следующей группой. Начнем мы с нашего вызова функции `pct_change`. Но в этот раз мы хотим узнать, насколько в среднем каждая группа превосходит предыдущую. Для этого воспользуемся методом `mean`, но применим его не к датафрейму, полученному после вызова функции `pct_change`. Вместо этого мы еще раз транспонируем датафрейм, чтобы уровни дохода расположились в столбцах, а годы – в строках:

```
(
    df
    .groupby('Year')
    [['income<20k',
     '20k<income<40k',
     '40k<income<60k',
     '60k<income<80k',
```

	162	428	263	490	334
Year	2008	2013	2010	2014	2011
State.Code	CO	GA	AZ	ME	MI
Total.Math	572	488	527	471	605
income<20k	555	451	491	494	549
20k<income<40k	567	470	506	520	565
40k<income<60k	571	484	523	535	590
60k<income<80k	578	496	531	555	609
80k<income<100k	533	426	472	468	528
income>100k	583	530	553	569	626



162	428	263	490	334
583	530	553	569	626

Рис. 7.5. Получение средних значений после выполнения транспонирования

```

        '80k<income<100k',
        'income>100k']]
    .mean()
    .T
    .pct_change()
    .T
    .mean()
)

```

Изменение оси

Еще один способ вычисления заключается в передаче методу `mean` дополнительного ключевого аргумента `axis`, как показано ниже:

```
df.mean(axis='columns')
```

Значение по умолчанию для аргумента `axis` – `'rows'`. В этом случае мы получаем по одной строке со средними значениями для каждого столбца. Если передать функции `mean` аргумент `axis='columns'`, в результате мы получим новый столбец с тем же индексом, что и в датафрейме.

Если датафрейм не слишком велик, можно смело выполнять двойное транспонирование, что позволяет прийти в исходное состояние. Но если в вашем датафрейме достаточно много данных, более приемлемым в плане экономии памяти может оказаться способ с ключевым аргументом `axis`.

Теперь мы знаем, насколько в среднем повышается балл за тест по математике в зависимости от уровня дохода абитуриента. А между какими двумя уровнями наблюдается наибольший скачок? Это можно выяснить, отсортировав массив данных с помощью метода `sort_values` по убыванию и вызвав метод `head()` для получения первых строк:

```

(
    df
    .groupby('Year')
    [[ 'income<20k',
        '20k<income<40k',
        '40k<income<60k',
        '60k<income<80k',
        '80k<income<100k',
        'income>100k']]
    .mean()
    .T
    .pct_change()
    .T
    .mean()
    .sort_values(ascending=False)
    .head()
)

```

Все это хорошо, но полагаться на визуальный анализ данных не лучшая идея. Можно легко автоматизировать процесс нахождения случаев, когда увеличение


```

        'Family Income.Between 80-100k.Math',
        'Family Income.More than 100k.Math']) ❶

df = df.rename(
    columns={
        'Family Income.Less than 20k.Math':'income<20k',
        'Family Income.Between 20-40k.Math':'20k<income<40k',
        'Family Income.Between 40-60k.Math':'40k<income<60k',
        'Family Income.Between 60-80k.Math':'60k<income<80k',
        'Family Income.Between 80-100k.Math':'80k<income<100k',
        'Family Income.More than 100k.Math':'income>100k' ❷
    })

df.groupby('Year').mean(
    numeric_only=True).sort_index() ❸

(
    df
    .groupby('Year')
    [['income<20k',
      '20k<income<40k',
      '40k<income<60k',
      '60k<income<80k',
      '80k<income<100k',
      'income>100k']]
    .mean()
    .T
    .pct_change()
) ❹

(
    df
    .groupby('Year')
    [['income<20k',
      '20k<income<40k',
      '40k<income<60k',
      '60k<income<80k',
      '80k<income<100k',
      'income>100k']]
    .mean()
    .T
    .pct_change()
    .T
    .mean()
    .sort_values(ascending=False)
    .head()
) ❺

change = (
    df
    .groupby('Year')

```

```

[[ 'income<20k',
   '20k<income<40k',
   '40k<income<60k',
   '60k<income<80k',
   '80k<income<100k',
   'income>100k']]
.mean()
.T
.pct_change()
)
change[change <= 0].dropna()

```

- ❶ Читаем данные из файла CSV.
- ❷ Переименовываем столбцы с помощью словаря, в котором ключи – старые имена, а значения – новые.
- ❸ Рассчитываем средние значения каждого столбца для каждого года и сортируем результат по годам.
- ❹ Транспонируем результат группировки и рассчитываем среднее, затем вызываем функцию `pct_change`, чтобы узнать, насколько в процентах уровень дохода семьи влияет на успеваемость студентов в тестах.
- ❺ На каком уровне дохода отмечается наибольший скачок среднего балла?
- ❻ Присваиваем результат расчета переменной `change`.
- ❼ Ищем все строки в переменной `change`, в которых значения во всех столбцах снижаются по сравнению с предыдущей строкой.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/rj9D>.

Дополнительные упражнения

1. Рассчитайте описательную статистику для всех изменений среднего балла для уровней дохода. Где наблюдается наибольшая разница?
2. В каких пяти штатах отмечены наибольшие различия между средним баллом за тест по математике между самыми богатыми и самыми бедными студентами?
3. Мы проанализировали оценки за тест по математике. Наблюдается ли такая же тенденция в отношении оценок за тест по чтению? Есть ли в этих оценках случаи, когда студенты с более высоким уровнем дохода получали в среднем более низкие оценки в сравнении с предыдущим уровнем?

Фильтрация и трансформация

Мы уже видели, как можно использовать метод `groupby` для агрегирования информации по заданным группам, чтобы можно было собирать и сравнивать статистику в разных разрезах. В предыдущих главах мы также узнали, как можно применять булевы индексы для фильтрации строк, отвечающих заданным критериям.

Давайте рассмотрим в качестве примера датафрейм, в котором содержатся итоговые оценки по математике для всех студентов. Строки датафрейма представляют условных студентов, а в столбцах содержится имя, год и сама оценка. Сгенерируем наш датафрейм следующим образом:

```
import numpy as np
np.random.seed(0)

df = DataFrame({'name': list('ABCDEFGHJI'),
               'year': [2018, 2019, 2020]*3 + [2021],
               'score': np.random.randint(80, 100, 10)})
```

Наш датафрейм выглядит так:

	name	year	score
0	A	2018	92
1	B	2019	95
2	C	2020	80
3	D	2018	83
4	E	2019	83
5	F	2020	87
6	G	2018	89
7	H	2019	99
8	I	2020	98
9	J	2021	84

На основе этого датафрейма мы можем произвести следующие вычисления:

- рассчитать среднюю оценку с помощью выражения `df['score'].mean()`. В нашем импровизированном случае средний балл равен 89.0;
- получить имена студентов, набравших больше 90 баллов: `df.loc[df['score'] > 90]`. В результате мы получим датафрейм, аналогичный исходному, но только с отличниками. Это студенты с индексами 0, 1, 7 и 8;
- рассчитать среднюю оценку по годам с помощью выражения `df.groupby('year')['score'].mean()`. Так мы получим объект `Series` с годами в качестве индексов и средними оценками в значениях. В данном случае наш объект будет состоять из четырех элементов.

Все отлично. Но что, если нам понадобится определить, в какие годы средний балл в нашем заведении превышал 90, и посмотреть студентов этих лет? Таким образом, нам необходимо отфильтровать наши группы на основании агрегированного вычисления по годам. Как это можно сделать?

Ответ состоит в применении метода `filter` к нашему объекту `DataFrameGroupBy`. Все что нам нужно, – это передать методу `filter` функцию, которая при получении группы строк будет возвращать значение `True` или `False`, определяющее, должен ли этот набор строк входить в итоговый результат.

Формально последовательность действий выглядит так:

- нам нужно определить, включать строки в вывод или нет, на основании поля `year`, так что мы группируем данные следующим образом: `df.groupby('year')`;
- у полученного объекта `DataFrameGroupBy` вызываем метод `filter`;
- передаем на вход методу `filter` функцию в качестве аргумента;
- переданная функция вызывается для каждой группы строк. В качестве аргумента она получает датафрейм, представляющий подмножество строк из исходного датафрейма;

- функция должна вернуть значение True или False в зависимости от того, должны ли строки из этой группы включаться в результирующий набор;
- функция может представлять собой как полноценную функцию Python, определяемую при помощи ключевого слова `def`, так и анонимную функцию, создаваемую посредством инструкции `lambda`.

Ниже приведен пример такой функции и ее вызова:

```
def year_average_is_at_least_90(df):
    return df['score'].mean() > 90

df.groupby('year').filter(year_average_is_at_least_90)
```

В результате запуска этого кода мы получим датафрейм, являющийся подмножеством исходного датафрейма `df`, в котором останутся строки только для тех лет, в которых средняя оценка студентов превышала 90 баллов. Как мы знаем, такой год был только один – 2019-й, так что в нашем наборе останутся только строки с индексами 1, 4 и 7.

Вот лишь несколько примеров, показывающих, для чего можно использовать метод `filter` в реальных наборах данных:

- чтобы показать все товары заводов, выручка которых в прошлом году перевалила за 1 млн долл.;
- извлечь из базы сотрудников, работающих в отделах с зарплатой ниже средней по организации;
- найти сети из сегментов, в которых за последнюю неделю было зафиксировано более десяти перебоев.

Еще один полезный метод, который можно вызывать у объекта `DataFrameGroupBy`, – это метод `transform`. Цель его применения состоит не в исключении строк из исходного набора данных по определенному критерию, а в их трансформации, или преобразовании, с использованием агрегированных данных о группе, которой принадлежит эта строка. К примеру, если вам необходимо узнать для каждого студента разницу между его оценкой и средней оценкой за год, вы можете сделать это следующим образом:

```
df.groupby('year')['score'].transform(lambda x: x-x.mean())
```

Здесь наша анонимная функция вызывается для каждой группы, а количество элементов в итоговом объекте `Series` будет совпадать с числом строк в исходном датафрейме. Вы также можете присвоить результат вычисления существующему или новому столбцу и посмотреть на результат:

```
df['score_diff'] = df.groupby('year')['score'].transform(lambda x: x-x.mean())
df
```

Вывод:

	name	year	score	score_diff
0	A	2018	92	4.000000
1	B	2019	95	2.666667
2	C	2020	80	-8.333333
3	D	2018	83	-5.000000
4	E	2019	83	-9.333333
5	F	2020	87	-1.333333

6	G	2018	89	1.000000
7	H	2019	99	6.666667
8	I	2020	98	9.666667
9	J	2021	84	0.000000

Также мы можем передать на вход методу `transform` функцию, которая будет применяться ко всем группам. Например, вы можете получить для студентов максимальные оценки для их года следующим образом:

```
df.groupby('year')['score'].transform(np.max)
```

В результате функция `np.max` применится к каждой группе, представляющей год, а на вход функции будет передан наш столбец `score` со строками, разделенными по годам. Итог будет таким:

0	92
1	99
2	98
3	92
4	99
5	98
6	92
7	99
8	98
9	84

Name: score, dtype: int64

В результирующем объекте `Series` будут присутствовать максимальные оценки в разрезе годов. Иными словами, мы заменили каждую конкретную оценку на максимальный годовой балл. Возможно, это не лучший способ оценивать студентов.

Как видите, методы `transform` и `filter` могут быть чрезвычайно полезны при работе с группами строк, так как позволяют выстраивать логику в зависимости от агрегированных значений.

Ниже приведены несколько примеров применения метода `transform`:

- нахождение долей текущих значений от суммы всех значений в группе;
- нахождение отклонений текущих значений от средних значений по группам;
- вычисление z-оценки (т. е. количества стандартных отклонений от среднего) для текущих значений.

ПРИМЕЧАНИЕ. В случае с методами `filter` и `transform` к параметру `df` добавляется атрибут `name` с именем текущей группы.

ПРИМЕЧАНИЕ. Метод `filter` объекта `DataFrameGroupBy` очень похож на встроенную в Python функцию `filter`, а метод `transform` очень напоминает стандартную функцию `map`. Отличаются эти функции тем, что работают с датафреймами, а не с обычными итерируемыми объектами, но принцип в них используется тот же.

Ответы на дополнительные упражнения

Упражнение 33.1

```
change = df.groupby('Year')[['income<20k',
                              '20k<income<40k',
                              '40k<income<60k',
                              '60k<income<80k',
                              '80k<income<100k',
                              'income>100k']].mean().T.pct_change()

# Самые высокие оценки получают студенты из самых богатых семей,
# их средний балл превосходит средний балл для всех остальных групп дохода

change.T.describe()
```

Вывод:

	income<20k	20k<income<40k	40k<income<60k	60k<income<80k	80k<income<100k	income>100k
count	0.0	11.000000	11.000000	11.000000	11.000000	11.000000
mean	NaN	0.083929	0.045260	0.020744	0.025247	0.034399
std	NaN	0.026723	0.009055	0.008745	0.004821	0.008947
min	NaN	0.044260	0.034768	0.003136	0.015391	0.012418
25%	NaN	0.073665	0.041450	0.019374	0.023645	0.035291
50%	NaN	0.083207	0.043872	0.023743	0.024947	0.036175
75%	NaN	0.094456	0.045532	0.026016	0.028681	0.039508
max	NaN	0.142793	0.069618	0.029694	0.032817	0.042319

Упражнение 33.2

```
df['rich_poor_diff'] = df['income>100k'] - df['income<20k']
df.groupby('State.Code')['rich_poor_diff'].mean().sort_values(ascending=False).head()
```

Вывод:

```
State.Code
ND      341.909091
WY      246.454545
DC      208.818182
SD      157.000000
MS      140.000000
Name: rich_poor_diff, dtype: float64
```

Упражнение 33.3

```
filename = '../data/sat-scores.csv'

df = pd.read_csv(filename,
                  usecols=['Year', 'State.Code', 'Total.Verbal',
                           'Family Income.Less than 20k.Verbal',
                           'Family Income.Between 20-40k.Verbal',
                           'Family Income.Between 40-60k.Verbal',
```

```

        'Family Income.Between 60-80k.Verbal',
        'Family Income.Between 80-100k.Verbal',
        'Family Income.More than 100k.Verbal'])
df.columns = ['Year', 'State.Code', 'Total.Verbal',
              'income<20k',
              '20k<income<40k',
              '40k<income<60k',
              '60k<income<80k',
              '80k<income<100k',
              'income>100k',
              ]

```

```
df.head()
```

Вывод:

	Year	State.Code	Total.Verbal	...	60k<income<80k	80k<income<100k	income>100k
0	2005	AL	567	...	577	474	590
1	2005	AK	523	...	534	467	544
2	2005	AZ	526	...	533	474	546
3	2005	AR	563	...	580	486	589
4	2005	CA	504	...	525	421	551

[5 rows x 9 columns]

```

change = df.groupby('Year')[['income<20k',
                              '20k<income<40k',
                              '40k<income<60k',
                              '60k<income<80k',
                              '80k<income<100k',
                              'income>100k']].mean().T.pct_change()

```

```
change[change <= 0].dropna()
```

Вывод:

Year	2005	2006	2007	...	2013	2014	2015
80k<income<100k	-0.165165	-0.16959	-0.166568	...	-0.13885	-0.162701	-0.18397

[1 rows x 11 columns]

УПРАЖНЕНИЕ 34. Снежные и дождливые города

Где бы я ни жил, люди везде жалуются на погоду. В тропическом климате они сетуют на невыносимую жару, а в арктическом – на страшные морозы. В полосе, где очень жаркое лето и холодная зима, люди жалуются и на то, и на другое. И, конечно, всем приезжим они рассказывают, что именно их погода худшая на земле. С этой особенностью людей ничего нельзя сделать. Может быть, мы сможем с помощью данных определить, где в действительности наблюдаются экстремальные погодные условия? Как бы хотели постоянно жалующиеся люди получить подтверждение своего недовольства в виде официальных цифр!

В этом упражнении мы поработаем с методами `filter` и `transform` объекта `DataFrameGroupBy`. С помощью этих методов мы можем сохранять (`filter`) и изменять (`transform`) строки в датафрейме с применением условий на основе вычислений для целых групп строк.

ПРИМЕЧАНИЕ. Версии методов `filter` и `transform` для объекта `DataFrameGroupBy` – это одна из наиболее сложных тем в `pandas`, и зачастую людям требуется время, чтобы научиться применять их на практике.

В этом упражнении вы должны сделать следующее.

1. Прочитать в датафрейм данные о погоде из файла, с которым мы работали в упражнении 32. На этот раз нам понадобятся столбцы `max_temp`, `min_temp` и `precipMM`.
2. Определить, в каких городах уровень осадков (поле `precipMM`) достигал 15 мм и более как минимум трижды.
3. Найти города, в которых как минимум три раза уровень осадков достигал отметки 10 мм и более при температуре 0 или ниже.
4. Для каждого измерения уровня осадков рассчитать долю от общего объема осадков для этого города.
5. Для каждого города определить наибольшую долю осадков от общего объема для этого города за заданный период.

Подробный разбор

В этом упражнении мы воспользуемся методами `filter` и `transform` объекта `DataFrameGroupBy` для работы со строками с учетом агрегированных значений по группам. Начнем мы с загрузки данных о погоде в восьми разных городах, так же как в упражнении 32. Нам понадобятся три столбца: `max_temp`, `min_temp` и `precipMM` (уровень осадков в мм). Поскольку мы уже делали это ранее, я приведу этот блок кода без комментариев:

```
import glob

all_dfs = []

for one_filename in glob.glob('../data/*/*.csv'):
    print(f'Loading {one_filename}...')
    city, state = (
        one_filename
        .removeprefix('../data/')
        .removesuffix('.csv')
        .split(',')
    )
    one_df = (
        pd
        .read_csv(one_filename,
                  usecols=[1, 2, 3],
                  names=['max_temp',
```

```

        'min_temp',
        'precipMM'],
        header=0)
    .assign(city=city.replace('+', ' ').title(),
           state=state.upper())
)

all_dfs.append(one_df)

df = pd.concat(all_dfs)

```

Теперь мы можем начать анализировать собранный датафрейм. Сначала нас попросили определить, в каких городах уровень осадков достигал 15 мм и более как минимум трижды. Что нужно сделать, чтобы ответить на поставленный вопрос:

- рассчитать показатель уровня осадков по городам с помощью метода `groupby`;
- оставить только те города, в которых значение уровня осадков достигло 15 и более как минимум три раза (метод `filter`).

Начнем с группировки. Поскольку нас интересуют сведения в разрезе городов, операция группировки могла бы выглядеть так:

```
df.groupby('city')
```

Но мы не можем так сделать, поскольку, если вы помните, в нашем наборе данных есть город, который встречается в двух разных штатах, – это Спрингфилд. По этой причине нам необходимо выполнить группировку данных сразу по двум столбцам: по штату и городу. С этой целью передадим методу `groupby` список имен группируемых столбцов:

```
df.groupby(['city', 'state'])
```

В результате мы получим объект `DataFrameGroupBy`, который ранее применяли для вычисления агрегированных значений по отдельным группам. Здесь же мы будем использовать этот объект иначе, а именно для включения или исключения строк в исходном датафрейме на основе обобщенных вычислений для города или штата. Таким образом, мы хотим фильтровать строки, но делать это по группам, чтобы в набор данных включались или исключались строки целыми наборами (вы можете воспринимать это как инструмент коллективного наказания в `pandas`).

Воспользуемся для этого методом `filter` объекта `DataFrameGroupBy`. Этот метод работает с учетом образованных групп. В качестве аргумента метод `filter` принимает функцию, которая в свою очередь будет принимать подмножество нашего исходного датафрейма. Эта функция будет вызываться для каждой группы в нашем наборе данных, т. е. для подмножества строк, принадлежащих одной группе по заранее заданному критерию.

Функция, переданная методу `filter`, должна возвращать значение `True` или `False`. Если она возвращает `True`, строки из текущей группы сохраняют свое место в новом датафрейме. В противном случае они в него не включаются. Поскольку метод `filter` принимает на вход строки одной группы в виде датафрейма, он мо-

жет выполнять любые допустимые действия для датафреймов при определении того, включать строки в итоговый набор данных или нет.

В данном случае мы хотим сохранить строки для городов, в которых уровень осадков достигал 15 мм и более как минимум трижды. Таким образом, наша фильтрующая функция должна подсчитывать заданные нами события и возвращать логическое значение. Она может выглядеть так:

```
def has_multiple_readings_at_least(mini_df):
    return mini_df.loc[
        mini_df['precipMM'] >= 15,
        'precipMM'
    ].count() >= 3
```

Если бы мы передали этой функции весь наш датафрейм, она вернула бы значение True или False в зависимости от того, поднимался ли уровень осадков до 15 мм и выше как минимум три раза во всех городах. Использование же этой функции внутри метода `filter` позволяет определить, в каких именно городах возникали подобные аномалии:

```
(
    df
    .groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least)
)
```

В результате мы получим датафрейм, представляющий подмножество нашего исходного датафрейма. Но в задаче не спрашивалось, какие строки пройдут через фильтр. Вместо этого нам нужно узнать, в каких городах наблюдались такие осадки. Таким образом, мы могли бы получить из итогового датафрейма только столбцы `city` и `state`, как показано ниже:

```
(
    df.groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least)
    [['city', 'state']]
)
```

Но в этом случае мы получим города и штаты для всех строк. Это больше, чем нам требуется. Вызовем метод `drop_duplicates`, чтобы избавиться от дубликатов:

```
(
    df
    .groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least)
    [['city', 'state']]
    .drop_duplicates()
)
```

В итоге мы получим правильный ответ на поставленный вопрос, заключающийся в том, что такие осадки наблюдались только в Бостоне, Спрингфилде (Иллинойс) и Олбани. Если у вас есть хотя бы небольшой опыт программирования, функция `has_multiple_readings_at_least` в таком виде вас явно не устроит. Вы уверены, что хотите статически зафиксировать в теле функции значения 15 и 3?

Правильно, лучше написать более универсальную функцию, которая будет принимать нужные нам значения в качестве параметров.

А как можно это сделать? Мы ведь не вызываем нашу функцию `has_multiple_readings_at_least` напрямую, а просто передаем ее методу `filter`, который вызывает ее самостоятельно. Это делается с помощью стандартных конструкций Python `*args` и `**kwargs`, позволяющих функциям принимать произвольное число позиционных и ключевых параметров. За подробностями по использованию этого механизма вы можете обратиться к моей статье по адресу <https://lerner.co.il/2021/06/07/python-parameters-primer>.

Таким образом, мы можем переписать нашу функцию так:

```
def has_multiple_readings_at_least(mini_df, min_mm, times):
    return mini_df.loc[
        mini_df['precipMM'] >= min_mm,
        'precipMM'
    ].count() >= times
```

Теперь она больше похожа на обычную функцию в Python, принимающую три аргумента. Первый остался прежним – это часть нашего исходного датафрейма, содержащая строки, принадлежащие текущей группе. Второй и третий аргументы передаются в функцию косвенно при помощи метода `filter`, когда он ее вызывает. Это выглядит так:

```
(
    df
    .groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least,
            min_mm=10,
            times=3)
    [['city', 'state', 'precipMM']]
    .drop_duplicates()
)
```

Здесь мы вызываем метод `filter` и передаем ему нашу функцию `has_multiple_readings_at_least`. В теории мы могли бы передать значения для аргументов `min_mm` и `times` по позиции. Но в этом случае нам придется передавать методу `filter` и второй позиционный аргумент – `dropna`. Таким образом, вместо `filter(func, True, 10, 3)` мы предпочли вызвать метод так: `filter(func, min_mm=10, times=3)`. Этот выбор носит чисто эстетический характер, но в данном случае так действительно будет более наглядно.

Далее нас попросили найти города, в которых как минимум три раза уровень осадков достигал отметки 10 мм и более при температуре 0 или ниже.

Для решения этой задачи мы снова воспользуемся методами `groupby` и `filter`, но изменим нашу функцию для отбора строк следующим образом:

```
def has_multiple_readings_at_least(mini_df, min_mm, times):
    return mini_df.loc[
        ((mini_df['precipMM'] >= min_mm) &
         (mini_df['min_temp'] <= 0)),
        'precipMM'
    ].count() >= times
```

Теперь мы можем выполнить группировку данных и отфильтровать их с помощью следующего выражения:

```
(
    df
    .groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least, min_mm=10, times=3)
    [['city', 'state']]
    .drop_duplicates()
)
```

Далее нам необходимо для каждого измерения уровня осадков рассчитать долю от общего объема осадков для этого города. Иными словами, если в одном городе было произведено два измерения уровня осадков (в первый день значение составило 3 мм, а во второй – 7 мм), то мы должны получить значения 30 и 70 % соответственно.

Для этого нам нужно рассчитывать определенное значение для каждой строки в датафрейме, но использовать при этом агрегированные значения, вычисленные для группы, в которую входит текущая строка. Именно для таких случаев в pandas предусмотрен метод `transform`. Подобно тому как мы делали с методом `filter`, первым аргументом в метод `transform` мы отправим функцию. Эта функция будет вызываться для каждой группы и принимать на вход объект `Series`, т. е. столбец, значения в котором нам нужно преобразовать. Функция должна возвращать объект `Series` той же длины и с тем же индексом, что и поступивший на вход аргумент.

Предположим, у нас есть объект `Series`, состоящий из числовых значений, каждое из которых представляет одно измерение уровня осадков. Какую функцию можно написать, чтобы она возвращала новый объект `Series` той же длины и с тем же индексом, что у оригинала, но со значениями, представляющими долю каждого исходного значения от суммы всех значений? Эта функция может выглядеть так:

```
def proportion_of_city_precip(s):
    return s / s.sum()
```

Функция принимает на вход объект `Series` `s` и возвращает результат поэлементного деления значений на сумму всех значений в последовательности. Именно так мы бы сделали, если бы все измерения относились к одному городу. А что, если измерения проводятся в разных городах? Здесь-то нам и пригодится метод `transform`, используемый совместно с операцией группировки. В функцию `proportion_of_city_precip` будут передаваться строки, принадлежащие одной группе, а на выходе мы будем получать аналогичный переданному объект `Series` с новыми значениями в соответствующих элементах. Мы можем присвоить полученный результат существующему столбцу, создать на его основе новый столбец или просто сохранить в переменную для дальнейшего использования.

Разница между обычным методом `transform` и его версией для объекта с группировкой состоит в том, что во втором случае у нас есть доступ ко всему объекту `Series`, и мы можем в процессе расчетов пользоваться агрегациями:

```
df['precip_pct'] = df.groupby('city')[
    'precipMM'].transform(proportion_of_city_precip)
```

Обратите внимание, что в данном случае мы присвоили полученный результат новому столбцу в датафрейме. С помощью этого столбца мы сможем ответить на заключительный вопрос этого упражнения, который состоит в определении наибольшей доли осадков от общего объема для каждого города за заданный период. Иными словами, нам нужно найти измерение с максимальной долей осадков для каждого города.

Для ответа на этот вопрос мы можем воспользоваться классической группировкой с применением функции агрегации `max`. Разумеется, поскольку у нас есть города с одинаковыми названиями в разных штатах, мы включим штат в состав группировки. В результате получим простое выражение:

```
df.groupby(['city', 'state'])['precip_pct'].max()
```

Решение

```
import glob

all_dfs = []

for one_filename in glob.glob('../data/*/*.csv'):
    print(f'Loading {one_filename}...')
    city, state = (
        one_filename
        .removeprefix('../data/')
        .removesuffix('.csv')
        .split(',')
    )
    one_df = (
        pd
        .read_csv(one_filename,
                  usecols=[1, 2, 3],
                  names=['max_temp',
                        'min_temp',
                        'precipMM'],
                  header=0)
        .assign(city=city.replace('+', ' ').title(),
                state=state.upper())
    )

    all_dfs.append(one_df) ❶

df = pd.concat(all_dfs) ❷

def has_multiple_readings_at_least(mini_df):
    return mini_df.loc[
        mini_df['precipMM'] >= 15,
        'precipMM'
    ].count() >= 3 ❸

(
```



```

df
.groupby(['city', 'state'])
.filter(has_multiple_readings_at_least)
[['city', 'state']]
.drop_duplicates()
)

def has_multiple_readings_at_least(mini_df, min_mm, times):
    return mini_df.loc[
        ((mini_df['precipMM'] >= min_mm) &
         (mini_df['min_temp'] <= 0)),
        'precipMM'
    ].count() >= times

(
    df
    .groupby(['city', 'state'])
    .filter(has_multiple_readings_at_least, min_mm=10, times=3)
    [['city', 'state']]
    .drop_duplicates()
)

def proportion_of_city_precip(s):
    return s / s.sum()

df['precip_pct'] = df.groupby('city')['
    'precipMM'].transform(proportion_of_city_precip)

df.groupby(['city', 'state'])['precip_pct'].max()

```

- ❶ Добавляем датафреймы в список.
- ❷ Создаем единый датафрейм на основе полученного списка.
- ❸ Эта функция возвращает True, если в ней содержится как минимум три строки с `precipMM >= 15`.
- ❹ Группировка по городу и штату с сохранением только самых дождливых городов.
- ❺ Получение уникальных комбинаций штата и города.
- ❻ Эта функция возвращает True, если в ней содержится как минимум `times` строк с `precipMM >= min_mm` и нулевой или отрицательной температурой.
- ❼ Используем обновленную функцию `has_multiple_readings_at_least` для поиска самых дождливых городов.
- ❽ Получение уникальных комбинаций штата и города.
- ❾ Функция возвращает долю одного измерения во всем наборе данных.
- ❿ Добавляем новый столбец `precip_pct` с долями осадков по каждому городу.
- ⓫ Находим измерения с максимальной долей осадков для данного города.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/VRA0>.

Дополнительные упражнения

1. Реализуйте первую версию функции `has_multiple_readings_at_least`, принимающую один аргумент (`df`), в виде анонимной функции (`lambda`).
2. Реализуйте вторую версию функции `has_multiple_readings_at_least`, принимающую три аргумента (`df`, `min_mm` и `times`), в виде анонимной функции (`lambda`).
3. Реализуйте наше преобразование, заменив функцию `proportion_of_city_precip` на анонимную функцию (`lambda`), и найдите измерения с максимальной долей осадков для каждого города.

Ответы на дополнительные упражнения

Упражнение 34.1

```
(
    df
    .groupby(['city', 'state'])
    .filter(lambda df_: df_.loc[df_['precipMM'] >= 15, 'precipMM'].count() >= 3)
    [['city', 'state']]
    .drop_duplicates()
)
```

Вывод:

```
      city state
0    Boston  MA
0 Springfield IL
0    Albany  NY
```

Упражнение 34.2

При позиционной передаче аргументов нужно перед аргументами `min_mm` и
`times` явно передать значение `False`

```
(
    df
    .groupby(['city', 'state'])
    .filter(lambda df_, min_mm, times: df_.loc[df_['precipMM'] >= min_mm,
'precipMM'].count() >= times, min_mm=15, times=3)
    [['city', 'state']]
    .drop_duplicates()
)
```

Вывод:

```
      city state
0    Boston  MA
0 Springfield IL
0    Albany  NY
```

Упражнение 34.3

```
df['precip_pct'] = (df
                    .groupby('city')['precipMM']
                    .transform(lambda s: s/s.sum())
                    )

df.groupby(['city', 'state'])['precip_pct'].max()
```

УПРАЖНЕНИЕ 35. Вино и туризм...

Ранее в этой главе мы применяли метод `join` для объединения двух датафреймов в один. В этом упражнении мы погрузимся в изучение этого важного метода глубже и узнаем, как можно объединять более двух таблиц одновременно, комбинировать объединение с группировкой и использовать другие типы объединения. Также мы попробуем поискать корреляцию между объединенными наборами данных.

На этот раз будем объединять разные наборы данных, чтобы ответить на вопрос о том, производится ли в странах, которые много тратят на туризм, хорошее вино. Мы будем работать не только с набором данных от ОЭСР, с которым уже знакомы, но и с файлом, содержащим более 150 тыс. отзывов о винах разных стран.

Для выполнения этого упражнения вы должны сделать следующее.

1. Создать датафрейм с именем `oecd_df` на основе данных из файла `oecd_locations.csv`, в котором содержится информация о нескольких странах из состава ОЭСР. Нам будет достаточно единственного столбца `country`. В качестве индекса должны выступать сокращенные названия стран.
2. Создать второй датафрейм с именем `oecd_tourism_df` на основе файла `oecd_tourism.csv`. Из этого файла нам понадобятся столбцы `LOCATION` (будет выступать в качестве индекса), `TIME` (год показателя), `SUBJECT` (тип операции) и `Value` (сумма, выраженная в тысячах долларов). Кроме того, нас интересуют только строки, в которых в столбце `SUBJECT` стоит значение `'INT-EXP'`, что означает расходы на туризм. После выполнения фильтрации по столбцу `SUBJECT` от него можно избавиться.
3. Создать объект `Series` с именем `tourism_spending` на основе двух этих датафреймов, в котором в качестве индекса должны выступать названия (не сокращения) стран, а в качестве значений – средние расходы на туризм для этих стран.
4. Создать третий датафрейм с именем `wine_df` на основе файла `winemag-150k-reviews.csv`. Из этого файла нам понадобятся столбцы `country` и `points`.
5. Рассчитать средние оценки за вино (`points`) для каждой страны на основе всех имеющихся обзоров и отсортировать результат в порядке убывания.
6. Выполнить стандартное объединение данных со средними оценками вин по странам с датафреймом со средними затратами на туризм по странам. Где появились пропущенные значения и что они означают?

7. Выполнить внешнее объединение данных со средними оценками вин по странам с датафреймом со средними затратами на туризм по странам. Где появились пропущенные значения и что они означают на этот раз?
8. Найти корреляции между средними оценками вин и средними затратами на туризм по странам. Какие выводы вы можете сделать? Была ли обнаружена корреляция?

Подробный разбор

В этом упражнении мы попытались совместить разные идеи, озвученные в данной главе, а именно объединение нескольких датафреймов, переход между объектом Series и датафреймом и нахождение корреляции между наборами данных. Первое, что нас попросили сделать, это создать датафрейм с именем `oecd_df`, в котором будет содержаться информация о нескольких странах из состава ОЭСР. В исходном файле CSV, как мы видели в упражнении 31, находится всего два столбца и нет никаких заголовков, в связи с чем мы сами назначим колонкам имена `abbrev` и `country`. В качестве индекса нам нужно установить столбец с сокращенными названиями стран. Получим следующий код:

```
oecd_df = pd.read_csv('../data/oecd_locations.csv',
                      header=None,
                      names=['abbrev', 'country'],
                      index_col='abbrev')
```

Взглянем на первые строки в нашем датафрейме с помощью инструкции `oecd_df.head()`:

	country
abbrev	
AUS	Australia
AUT	Austria
BEL	Belgium
CAN	Canada
DNK	Denmark

Сам по себе этот датафрейм не очень полезен. Цель загрузки его состоит в установке соответствий между сокращенными (`abbrev`) и полными (`country`) названиями стран. Полные названия мы будем использовать для связки с оценками вин, а сокращенные – для работы с данными о расходах на туризм. Подобные справочники часто бывают нужны при работе сразу с несколькими источниками.

Теперь мы можем создать второй датафрейм, с именем `oecd_tourism_df`. Он будет загружен из файла CSV с заголовками, так что нам не придется вручную именовать столбцы. В то же время нам понадобятся только четыре столбца из файла, так что мы применим параметр `usecols`. Значениями в столбце `SUBJECT` мы воспользуемся, чтобы оставить только строки с расходами на туризм. После этого с данным столбцом можно расстаться. Также нас попросили установить в качестве индекса столбец `LOCATION` (с сокращенными названиями стран).

Сделаем то, что попросили:

```
oecd_tourism_df = (
    pd
```

```

.read_csv('../data/oecd_tourism.csv',
           usecols=['LOCATION', 'TIME',
                    'Value', 'SUBJECT'],
           index_col='LOCATION')
.loc[lambda df_: df_['SUBJECT'] == 'INT-EXP'] ❶
.drop('SUBJECT', axis='columns') ❷
)

```

- ❶ Оставляем только строки с типом операции 'INT-EXP'.
- ❷ Удаляем столбец SUBJECT.

Обратите внимание, как мы использовали анонимную функцию (`lambda`) в качестве аргумента атрибута `loc`. В результате мы оставляем в наборе данных только строки, для которых наша анонимная функция вернет значение `True`. Заметьте, что в функции мы назвали аргумент `df_`, чтобы подчеркнуть его временный характер и не путать с переменной `df`, часто используемой для создания и хранения датафреймов.

После выполнения фильтрации по столбцу `SUBJECT` мы можем избавиться от него с помощью метода `drop`, поскольку дальше он нам не понадобится. Не забывайте, что по умолчанию этот метод работает с индексом, а если вам нужно удалить один или несколько столбцов, нужно передать ему параметр `axis='columns'`.

Ниже показан результат вызова метода `oecd_tourism_df.head()`:

	TIME	Value
LOCATION		
AUS	2008	27620.0
AUS	2009	25629.6
AUS	2010	31916.5
AUS	2011	39381.5
AUS	2012	41632.8

Теперь у нас есть два датафрейма, в которых в качестве индекса используются сокращенные названия стран. Не беспокойтесь о том, что в датафрейме `oecd_tourism_df` индекс содержит дубликаты, а в датафрейме `oecd_df` в нем располагаются уникальные значения. Метод объединения данных в `pandas` прекрасно справляется с подобными ситуациями. Ключевым фактором является то, что в обоих индексах содержатся одни и те же элементы. А что будет, если в одном или обоих датафреймах будут присутствовать значения, которых не будет во втором датафрейме? Мы рассмотрим и такую ситуацию в этом упражнении.

Далее нам нужно рассчитать средние расходы на туризм по странам из нашего подмножества стран. У нас есть данные о расходах в разных странах из состава ОЭСР по нескольким годам. И мы должны определить, сколько в среднем тратили страны на туризм за все учтенные годы. При этом нам необходимо, чтобы в результате присутствовали полные названия стран, а не сокращенные.

Подобные расчеты легко выполняются при помощи группировки, например так:

```
oecd_tourism_df.groupby('LOCATION')['Value'].mean()
```

Таким образом, мы получим средние значения по столбцу `Value` для каждого уникального значения из столбца `LOCATION`. Обратите внимание, что мы можем

использовать столбец `LOCATION` для группировки, несмотря на то что он выступает в качестве индекса. Но нам не нужны сокращенные названия стран в результирующем наборе. Вместо этого нам нужны полные названия, которые хранятся в датафрейме `oecd_df`.

Следовательно, необходимо объединить два этих датафрейма. В обоих в качестве индекса выступают сокращенные названия стран, так что с этим не возникнет проблем. При этом неважно, как называются столбцы, представленные в индексе, – операция объединения по умолчанию работает с индексами. При объединении датафреймов мы обычно получаем новый, более широкий датафрейм, в котором содержатся все столбцы из обоих датафреймов и индекс с пересечениями. В нашем случае мы получим датафрейм, состоящий из четырех столбцов: сокращенные названия будут собраны в индекс, столбец `country` поступит из датафрейма `oecd_df`, а столбцы `TIME` и `Value` – из датафрейма `oecd_tourism_df`. Строки в новом датафрейме образуются в результате совпадения значений в индексах двух датафреймов, так что дублирование значений в индексе в одном или обоих датафреймах не является проблемой.

Итак, объединим наши датафреймы:

```
oecd_df.join(oecd_tourism_df)
```

Мы вызвали метод `join` у датафрейма `oecd_df`, который в этой связке называется левым датафреймом, и передали ему в качестве параметра датафрейм `oecd_tourism_df`, который стал правым датафреймом. В результате объединения мы получили новый датафрейм. Затем мы вызовем у этого датафрейма метод `groupby` для группировки данных по столбцу `country`, в котором содержатся полные названия стран. После этого извлечем столбец `Value` и применим к нему агрегацию с усреднением значений, как показано ниже:

```
(
    oecd_df
    .join(oecd_tourism_df)
    .groupby('country')['Value'].mean()
)
```

Вывод:

```
country
Australia    36727.966667
Austria      11934.563636
Belgium      20859.883455
Brazil       21564.351833
Canada       40984.633333
...
Italy        34148.908455
Japan        32197.925000
Korea        25573.509091
United Kingdom 75262.227273
United States 142080.666667
Name: Value, Length: 16, dtype: float64
```

Как видите, это позволило нам получить средние значения расходов на туризм по странам за все годы в наборе данных. Но в качестве названий здесь используются полные названия стран, а не сокращенные. К тому же, поскольку мы получили один столбец с индексом и один – со значениями, результат был нам возвращен в виде объекта Series, а не в виде датафрейма. Давайте присвоим полученный результат переменной `tourism_spending`, чтобы с ним было легче работать в дальнейшем:

```
tourism_spending = (
    oecd_df
    .join(oecd_tourism_df)
    .groupby('country')['Value'].mean()
)
```

Ниже показаны первые строки вывода метода `tourism_spending.head()`:

```
country
Australia    36727.966667
Austria       11934.563636
Belgium       20859.883455
Brazil        21564.351833
Canada        40984.633333
Name: Value, dtype: float64
```

Теперь пришло время загрузить данные об оценках вин разных стран в третий датафрейм. На этот раз нас интересуют только столбцы `country` и `points`:

```
wine_df = pd.read_csv(
    '../data/winemag-150k-reviews.csv',
    usecols=['country', 'points'])
```

Ниже приведены первые несколько строк датафрейма:

```
country  points
0      US      96
1  Spain      96
2      US      96
3      US      96
4  France      95
```

Согласитесь, не терпится посмотреть, какие средние оценки (`points`) получили вина разных стран. Применим для этого группировку:

```
country_points = (
    wine_df
    .groupby('country')['points'].mean()
)
```

Первые несколько строк результата выглядят так:

```
country
Albania      88.000000
Argentina     85.996093
Australia     87.892475
Austria       89.276742
```

```
Bosnia and Herzegovina    84.750000
Name: points, dtype: float64
```

Как видите, мы получили объект Series, в котором в качестве индекса выступают полные названия стран, а в качестве значений – средние оценки качества вин из этих стран. Мы присвоили результат переменной `country_points`, так что теперь можем выполнять с ней нужные нам действия.

Первое, что хочется сделать, – это отсортировать оценки в порядке их убывания. Это можно реализовать с помощью метода `sort` с параметром `ascending=False`, как показано ниже:

```
country_points.sort_values(ascending=False)
```

В результате мы получили упорядоченный список оценок вин по странам, первые несколько строк которого приведены ниже:

```
country
England    92.888889
Austria     89.276742
France      88.925870
Germany     88.626427
Italy       88.413664
Name: points, dtype: float64
```

Теперь перейдем к кульминации этого упражнения и объединим оценки качества вин с расходами стран на туризм. Как это можно сделать?

Хочется снова воспользоваться методом `join`, расположив переменную `country_points` слева, а `tourism_spending` – справа. Но есть одна проблема, заключающаяся в том, что переменная `country_points` представлена объектом Series, а метод `join` мы можем вызывать только у датафреймов (объект Series можно передать в качестве параметра методу `join`, что сделает его правой таблицей в объединении).

К счастью, в нашем распоряжении есть метод `to_frame`, который объект Series превращает в датафрейм с единственным столбцом и тем же индексом:

```
country_points.to_frame()
```

Теперь можно использовать метод `join` для объединения образовавшегося датафрейма с датафреймом `tourism_spending`:

```
country_points.to_frame().join(tourism_spending)
```

Опять же, важно помнить, что метод `join` объединяет левый датафрейм с правым на основании значений в индексах. В данном случае мы получим датафрейм с тремя столбцами: `country` (общая колонка с индексом в обоих источниках), `points` (из левого датафрейма) и `Value` (из правого).

Вот как выглядят первые пять строк объединенного датафрейма:

```
country
Albania          88.000000      NaN
Argentina        85.996093      NaN
Australia        87.892475  36727.966667
Austria          89.276742  11934.563636
Bosnia and Herzegovina 84.750000      NaN
```


Дублирование имен столбцов

Что произойдет, если в объединяемых датафреймах будут присутствовать столбцы с одинаковыми именами? Хотя pandas допускает повторение значений в индексе, имена столбцов в датафрейме должны быть уникальными. Если попытаться создать датафрейм с повторяющимися именами столбцов, то вы получите ошибку `ValueError`, указывающую на то, что в именах столбцов не используются суффиксы. И действительно, pandas позволяет указать, какой суффикс должен быть использован для левого датафрейма (`lsuffix`), а какой – для правого (`rsuffix`) при вызове метода `join`. К примеру, мы могли бы объединить датафрейм `oecd_df` сам с собой (довольно дикая идея, называемая самообъединением (`self join`)), но имеющая практическое применение в определенных ситуациях), как показано ниже:

```
oecd_df.join(oecd_df, lsuffix='_l', rsuffix='_r')
```

В результате мы получим датафрейм, в котором в качестве индекса будет выступать столбец `abbrev`, а также будут присутствовать два идентичных столбца с именами `country_l` и `country_r`.

Наше объединение сработало, но вы видите, что в итоговом датафрейме в столбце `Value` присутствует приличное количество значений `NaN`. Причина в том, что именно индекс из левой таблицы (в нашем случае это `country_points.to_frame()`) определяет состав индекса в результирующем наборе данных. А при левом объединении, как в этом случае, недостающие элементы в правой таблице заменяются на значения `NaN`.

В выполненном нами объединении значения `NaN` в столбце `Value` присутствуют, например, для Албании (Albania), Болгарии (Bulgaria) и Чили (Chile), как и для многих других стран. Причина в том, что оценки качества вин из этих стран у нас есть, а информации о расходах туризм – нет.

Существуют и другие виды объединения. Если вы хотите, чтобы содержимое индекса итогового датафрейма определялось на основе индекса из правого датафрейма, вы можете реализовать *правое объединение* (`right join`). Для этого необходимо передать параметр `how='right'` при вызове метода `join` (по умолчанию используется параметр `how='left'`). В этом случае значения `NaN` могут появиться уже в столбцах из левого датафрейма при отсутствии соответствий с правым.

Можно пойти еще дальше и реализовать *внешнее объединение* (`outer join`), в результате чего индексы из левого и правого датафреймов будут объединены. В этом случае пропущенные значения могут оказаться как в столбцах из левого датафрейма, так и в столбцах из правого, в зависимости от того, какое значение в индексе будет отсутствовать. В упражнении нас как раз и попросили выполнить внешнее объединение таблиц, что делается следующим образом:

```
country_points.to_frame().join(tourism_spending,
                              how='outer')
```

В результирующем датафрейме у нас оказалось 54 строки вместо 48, что объясняется объединением индексов из обоих датафреймов. Теперь значения `NaN` у нас есть как в столбце `points` из левой таблицы (например, для Бельгии (Belgium) и Дании (Denmark)), так и в столбце `Value` из правой (Албания (Albania), Аргентина

(Argentina) и т. д.). При использовании внешних объединений вы гарантированно не потеряете данные из источников, но при этом можете получить множество пропущенных значений, которые затем придется очищать (приемы для этого были показаны в главе 5).

Ниже показаны первые несколько строк из объединенного датафрейма:

	points	Value
country		
Albania	88.000000	NaN
Argentina	85.996093	NaN
Australia	87.892475	36727.966667
Austria	89.276742	11934.563636
Belgium	NaN	20859.883455

Наконец, нас попросили определить, есть ли корреляция между средними оценками качества вин из разных стран и средними затратами на туризм в этих странах. Для этого мы воспользуемся методом `corr`, как показано ниже:

```
country_points.to_frame().join(
    tourism_spending, how='outer').corr()
```

Вывод:

	points	Value
points	1.000000	0.288231
Value	0.288231	1.000000

Корреляция помогает определить связь между разными столбцами в датафрейме. Значение 1 означает полную положительную корреляцию, а это значит, что при увеличении значений в первой колонке значения во второй увеличиваются ровно в такой же степени. Значение -1 означает полную отрицательную корреляцию – это значит, что при увеличении значений в первой колонке значения во второй *уменьшаются* ровно в такой же степени. Значение 0 показывает полное отсутствие корреляции между столбцами. Иначе говоря, чем ближе значение корреляции к 1 или -1 и дальше от 0, тем сильнее взаимосвязаны данные в столбцах. По умолчанию при использовании метода `corr` применяется *корреляция Пирсона* (Pearson correlation), но вы можете передать другой метод с помощью параметра `method`.

На выходе мы получили датафрейм с идентичными значениями индекса и именами столбцов. Корреляция между столбцами определяется по значениям, стоящим на пересечении имени одного столбца в индексе, а второго – в столбцах, или наоборот. На главной диагонали датафрейма будут располагаться единицы, означающие полную положительную корреляцию любого столбца с самим собой.

Мы видим, что в нашем случае корреляция столбцов `points` и `Value` составляет 0.288, что свидетельствует о слабой положительной зависимости между значениями в них. Таким образом, мы можем сделать вывод, что страны с большими расходами на туризм чаще будут славиться хорошими винами. Но зависимость не самая сильная, так что не стоит выбирать вина на основе данных о расходах стран-производителей на туризм.

Решение

```

oecd_df = pd.read_csv('../data/oecd_locations.csv',
                      header=None,
                      names=['abbrev', 'country'],
                      index_col='abbrev')

oecd_tourism_df = pd.read_csv(
    '../data/oecd_tourism.csv',
    usecols=['LOCATION', 'TIME', 'Value'],
    index_col='LOCATION')

tourism_spending = (
    oecd_df
    .join(oecd_tourism_df)
    .groupby('country')['Value'].mean()
)

wine_df = pd.read_csv(
    '../data/winemag-150k-reviews.csv',
    usecols=['country', 'points'])

country_points = (
    wine_df
    .groupby('country')['points'].mean()
)

country_points.sort_values(ascending=False)
country_points.to_frame().join(tourism_spending)
country_points.to_frame().join(tourism_spending,
                               how='outer')
country_points.to_frame().join(tourism_spending,
                               how='outer').corr()

```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/A8eK>.

Дополнительные упражнения

1. Прочитайте все три датафрейма, но без установки индексов. Убедитесь, что в датафрейме `oecd_tourism_df` столбцы называются `abbrev`, `TIME` и `Value`, а тип данных в столбце `Value` - `np.int64`.
2. Выполните все те же объединения, но с использованием метода `merge` вместо `join`.
3. Чем отличаются датафреймы, полученные при помощи методов `merge` и `join`, в отношении значений `NaN`?

Ответы на дополнительные упражнения

Упражнение 35.1

```

oecd_df = pd.read_csv('../data/oecd_locations.csv', header=None,
                      names=['abbrev', 'country'])

oecd_tourism_df = pd.read_csv('../data/oecd_tourism.csv',
                              usecols=[0, 5, 6],
                              header=0,
                              names=['abbrev', 'TIME', 'Value'])

wine_df = pd.read_csv('../data/winemag-150k-reviews.csv',
                      usecols=['country', 'points'])

```

Упражнение 35.2

```

tourism_spending = (oecd_df
                    .merge(oecd_tourism_df, on='abbrev')
                    .groupby('country')['Value'].mean()
                    )
tourism_spending

```

Вывод:

```

country
Australia    37634.433333
Austria      16673.886364
Belgium       16525.237545
Brazil        13942.913958
Canada        32593.612500
...
Italy         39539.560000
Japan         28606.891667
Korea         21677.131818
United Kingdom 63507.159091
United States 171847.083333
Name: Value, Length: 16, dtype: float64

```

```

country_points = (
    wine_df
    .groupby('country')['points'].mean()
)

```

country_points

Вывод:

```

country
Albania      88.000000
Argentina    85.996093

```

```

Australia      87.892475
Austria        89.276742
Bosnia and Herzegovina  84.750000
...
Turkey        88.096154
US             87.818789
US-France     88.000000
Ukraine       84.600000
Uruguay        84.478261
Name: points, Length: 48, dtype: float64

```

```

(
    country_points.to_frame()
    .merge(tourism_spending, on='country')
)

```

Вывод:

```

country
Australia  87.892475  37634.433333
Austria    89.276742  16673.886364
Brazil     83.240000  13942.913958
Canada     88.239796  32593.612500
France     88.925870  58228.804000
Germany    88.626427  75011.823091
Hungary    87.329004   5108.871591
Israel     87.176190   6634.454042
Italy      88.413664  39539.560000
Japan      85.000000  28606.891667

```

```

(
    country_points.to_frame()
    .merge(tourism_spending, on='country', how='outer')
)

```

Вывод:

```

              points      Value
country
Albania      88.000000      NaN
Argentina    85.996093      NaN
Australia    87.892475  37634.433333
Austria      89.276742  16673.886364
Belgium      NaN      16525.237545
...
US-France    88.000000      NaN
Ukraine      84.600000      NaN
United Kingdom  NaN  63507.159091
United States  NaN  171847.083333
Uruguay      84.478261      NaN

```

[54 rows x 2 columns]

Упражнение 35.3

По умолчанию метод `join` выполняет левое объединение, а значит, значения NaN могут появиться только в столбцах из правой таблицы. Метод `merge`, напротив, по умолчанию производит *внутреннее объединение* (`inner join`), подразумевающее пересечение индексов из левой и правой таблиц. В этом случае значения NaN не могут появиться в выводе по причине объединения таблиц, но могут оказаться там вследствие их наличия в исходных таблицах.

Заключение

В этой главе мы глубже погрузились в операции *разделить-применить-объединить*, взглянув на группировку, объединение и сортировку данных под совершенно новым углом. Редкий проект на `pandas` может обойтись без применения описанных здесь приемов и техник, так что я надеюсь, что вы потратили немного времени и решили все упражнения самостоятельно, впоследствии сверив свои ответы с моими.

Глава 8

Промежуточный проект

Поздравляю, вы дошли до половины книги! Если вы действительно самостоятельно выполняли все упражнения, включая дополнительные, то, убежден, почувствовали немалую уверенность при работе с библиотекой `pandas`. Если же вы просто просмотрели книгу до этого места, не пытаясь выполнять упражнения, что ж, это ваш выбор!

Вы могли заметить, что даже в процессе чтения книги вы можете забывать имена тех или иных методов, функций и параметров и делать глупые, казалось бы, ошибки при попытках решить поставленные задачи. Не расстраивайтесь, это вполне естественно, и, уверяю вас, это происходит со всеми вне зависимости от опыта использования `pandas` или любой иной большой библиотеки. Со временем, когда вы будете постоянно использовать `pandas` на практике, его синтаксис уляжется у вас в голове, и вы будете гораздо легче извлекать из памяти названия нужных вам функций и методов.

Вся эта книга построена на том, что беглость и гибкость владения тем или иным программным инструментом может быть достигнута только посредством постоянной практики. Вы можете не ощущать своего прогресса, но поверьте, что он есть.

В этой главе мы отдохнем от выполнения упражнений, посвященных какой-то конкретной теме. Вместо этого мы реализуем небольшой проект, в процессе работы над которым вам придется применять многие из описанных в предыдущих главах техники и приемы. Надеюсь, этот проект позволит вам почувствовать тягу к объединению накопленных знаний и их совместному плодотворному использованию.

В проекте мы рассмотрим набор данных, посвященный исследованию среди разработчиков на Python от 2020 года, и набор, вобравший в себя информацию об исследовании в рамках Stack Overflow за 2021 год. Первое исследование было проведено компанией JetBrains, разработавшей PyCharm и множество других инструментов для Python и прочих языков программирования, и является лучшим информационным срезом в отношении того, кем являются разработчики на Python и чем они занимаются. Второе исследование прошло в рамках ежегодного опроса программистов всех мастей, включая разработчиков на Python, на всемирно известном сайте Stack Overflow.

В табл. 8.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 8.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.MultiIndex.from_tuples</code>	Возвращает объект множественного индекса на основе списка кортежей	<code>pd.MultiIndex.from_tuples(a_list)</code>	http://mng.bz/ZqnZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.MultiIndex.from_tuples.html)
<code>str.split</code>	Разбивает строку на части слева направо, возвращая список и размещающая в нем оставшиеся дополнительные элементы справа	<code>'abc def ghi'.split(None, 1) # возвращает ['abc', 'def ghi']</code>	http://mng.bz/aR4z (https://docs.python.org/3/library/stdtypes.html#str.split)
<code>str.rsplit</code>	Разбивает строку на части справа налево, возвращая список и размещающая в нем оставшиеся дополнительные элементы слева	<code>'abc def ghi'.rsplit(None, 1) # возвращает ['abc def', 'ghi']</code>	http://mng.bz/aR4z (https://docs.python.org/3/library/stdtypes.html#str.rsplit)

Задача

Вот что вам нужно будет сделать в рамках промежуточного проекта.

1. Загрузите данные из файла CSV (2020_sharing_data_outside.csv), посвященного исследованию деятельности разработчиков на Python, в датафрейм с именем `py_df`.
2. Преобразуйте столбцы в множественный индекс. Как это сделать – зависит от столбца:
 - имена большинства столбцов в наборе данных имеют формат `first.second.third` с двумя и более слов, разделенных точками. Разделите имя столбца на две части: до последней точки в имени и после нее. Множественный индекс для приведенного выше имени столбца должен выглядеть так: `('first.second', 'third')`. Если слов в имени столбца всего два (`first.second`), то множественный индекс должен быть таким: `('first', 'second')`;
 - в наборе данных есть порядка 20 столбцов, для которых первым уровнем в множественном индексе должно быть слово `general` (даже если в имени есть точки), а вторым – все имя столбца целиком. Имена этих столбцов:
 - `age`;
 - `are.you.datascientist`;
 - `is.python.main`;
 - `company.size`;
 - `country.live`;

- `employment.status;`
- `first.learn.about.main.id;`
- `how.often.use.main.id;`
- `is.python.main;`
- `main.purposes;`
- `missing.features.main.id;`
- `nps.main.id;`
- `python.version.most;`
- `python.years;`
- `python2.version.most;`
- `python3.version.most;`
- `several.projects;`
- `team.size;`
- `use.python.most;`
- `years.of.coding;`

- воспользуйтесь функцией `pd.MultiIndex.from_tuples` для создания множественного индекса, после чего назначьте полученный результат обратно атрибуту `df.columns` (подсказка: вы можете написать собственную функцию и воспользоваться циклом или генератором списков).
3. Отсортируйте столбцы в алфавитном порядке. Это не обязательное требование, но упорядоченные данные легче просматривать и понимать.
 4. Ответьте на следующие вопросы:
 - какие десять IDE для Python пользуются наибольшей популярностью?
 - какие десять языков программирования (`other.lang`), помимо профильного, чаще используют в своей работе разработчики на Python?
 - какие десять стран представлены в исследовании наиболее полно?
 - как разработчики на Python распределяются в отношении имеющегося опыта работы в процентном отношении?
 - из какой страны наибольшее количество разработчиков на Python с более чем 10-летним (11+) опытом работы?
 - в какой стране наблюдается наибольшая доля разработчиков на Python с более чем 10-летним (11+) опытом работы?
 5. Загрузите данные, представляющие исследование на Stack Overflow (`so_2021_survey_results.csv`), в датафрейм с именем `so_df`.
 6. Выведите среднюю зарплату для разных типов сотрудников. Подрядчики и фрилансеры любят говорить, что они зарабатывают больше штатных сотрудников. Подтвердите или опровергните это заявление.
 7. Создайте сводную таблицу, в которой в качестве индекса будут представлены страны, в качестве столбцов – уровни образования, а в ячейках – средняя зарплата по этим пересечениям.
 8. Создайте аналогичную сводную таблицу, но только по странам, входящим в ОЭСР (файл `oecd_locations.csv`). В какой из этих стран сотрудники с ди-

пломом младшего специалиста ('Associate degree (A.A., A.S., etc.)') зарабатывают больше, чем в других странах? А в какой стране сотрудники с докторской степенью ('Other doctoral degree (Ph.D., Ed.D., etc.)') зарабатывают больше?

9. Удалите из датафрейма `so_df` строки, в которых в столбце `LanguageHaveWorkedWith` стоит значение `NaN`.
10. Удалите из датафрейма `so_df` строки, в которых язык Python не включен в список наиболее часто используемых языков программирования (`LanguageHaveWorkedWith`). Сколько строк осталось?
11. Удалите из датафрейма `so_df` строки, в которых в столбце `YearsCode` стоит значение `NaN`. Сколько строк осталось?
12. Замените строковое значение `Less than 1 year` в столбце `YearsCode` на 0, а значение `More than 50 years` – на 51.
13. Приведите столбец `YearsCode` к целочисленному типу данных.
14. Создайте новый столбец с именем `experience` в датафрейме `so_df` с категориями опыта работы на основе значений в столбце `YearsCode` следующим образом:
 - `Less than 1 year` (менее 1 года);
 - `1-2 years` (1-2 года);
 - `3-5 years` (3-5 лет);
 - `6-10 years` (6-10 лет);
 - `11+ years` (11 лет и более).
15. Согласно исследованию сайта Stack Overflow, как распределяются разработчики Python в процентном отношении по опыту работы?

Решение

В этом проекте мы погрузимся в мир разработчиков на языке Python с помощью двух проведенных исследований. На основе имеющихся данных можно было бы составить еще сотню, если не тысячу, вопросов. Если вам понравится работать с этими данными, можете продолжить их изучение самостоятельно, что будет для вас очень полезно.

Загрузка данных исследования в датафрейм

Начнем с загрузки данных, собранных в результате исследования, в датафрейм. Казалось бы, что может быть проще:

```
py_filename = '../data/2020_sharing_data_outside.csv'
py_df = pd.read_csv(py_filename)
```

Но если загрузить данные таким способом, вы, вероятно, получите предупреждение от `pandas` о том, что некоторые столбцы в наборе данных обладают смешанным типом. Мы уже сталкивались с подобным предупреждением ранее. `Pandas` обычно неплохо справляется с определением значений атрибута `dtype` для столбцов, но это бывает сопряжено с большим расходом памяти. Вы можете либо явно задать типы данных при вызове функции `pd.read_csv`, либо, если распола-

гаете достаточным объемом памяти, позволить pandas определить типы самостоятельно.

В этом проекте мы будем использовать не так много столбцов, и указать их можно с помощью параметра `usecols`. Но мы хотим потренироваться в создании множественных индексов и оставить данные в датафрейме для дальнейшего исследования. В связи с этим мы прочитаем все данные целиком и позволим pandas использовать всю доступную память для корректного определения типов данных:

```
py_filename = '../data/2020_sharing_data_outside.csv'
py_df = pd.read_csv(py_filename, low_memory=False)
```

ПРИМЕЧАНИЕ. Я надеюсь, что в вашем компьютере достаточно памяти для загрузки всех столбцов в наборе данных. В противном случае вам лучше будет воспользоваться параметром `usecols` при вызове функции `read_csv` для указания только нужных вам столбцов для выполнения этого упражнения. Это позволит pandas снизить нагрузку на память и успешно определить все типы данных в наборе.

Нет ничего плохого в том, чтобы работать со всем набором данных целиком. Но в нашем опросе присутствует 264 столбца, что весьма много для визуального анализа. Кроме того, поскольку формат CSV не поддерживает иерархии, некоторые столбцы были названы так, чтобы дать нам полное ощущение присутствия иерархии в данных. К примеру, у нас есть столбцы с именами `other.lang.Java`, `other.lang.JavaScript`, `other.lang.C/C++` и т. д., которые можно условно отнести к одной группе `other.lang`.

Можно ли взять плоский список колонок и превратить его в множественный индекс, чтобы с данными было удобно работать? Да, но для этого нужно кое-что сделать. Обратите внимание, что наши столбцы именуются по следующему шаблону: `first.second.third`. Если разбить этот шаблон по последней точке в имени, то можно создать множественный индекс, состоящий из первичной колонки с именем `first.second` и вторичной с именем `third`. В нашем случае колонкой верхнего уровня будет `other.lang`, а под ней будут располагаться столбцы `Java`, `JavaScript`, `C/C++` и т. д., как показано на рис. 8.1.

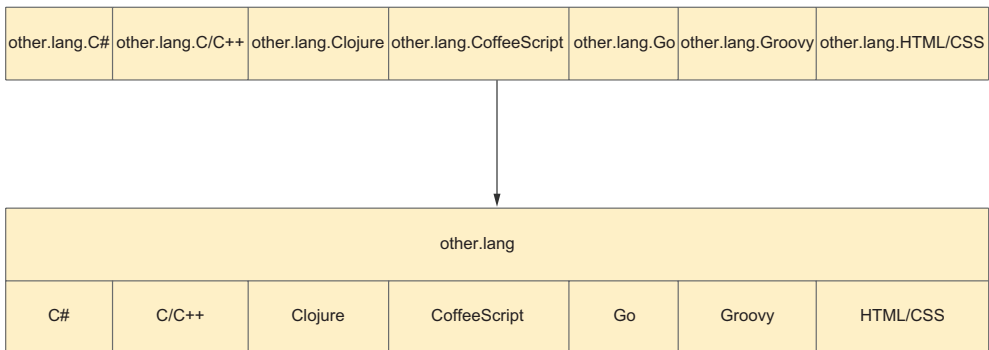


Рис. 8.1. Преобразование столбцов в множественный индекс

Как можно создать такой индекс и применить его к нашим данным? Мы можем воспользоваться удобной функцией `pd.MultiIndex.from_tuples`, которая су-

существует именно для таких нужд. Если передать этой функции список кортежей, она вернет объект множественного индекса, который мы можем впоследствии присвоить атрибуту `index` или `columns`. В нашем случае мы можем присвоить его атрибуту `columns`, что позволит заменить существующий индекс, используемый в столбцах.

Для начала создадим список кортежей. Первым элементом в наших кортежах будет составляющая имени столбца до последней точки, а вторым – после нее. Для этого мы можем воспользоваться стандартным методом `str.rsplit`, который работает аналогично методу `str.split`, но начинает разбиение строки справа, а не слева. Чтобы добиться желаемого результата, нам необходимо передать методу `str.rsplit` второй параметр с именем `maxsplit`, указывающий на то, сколько разбиений нужно произвести. Передавая этому параметру единицу, мы получим список из двух элементов с нужными нам частями имени столбца:

```
s = 'abcd.efgh.ijkl'
s.rsplit('.', 1)
```

Этот код вернет значение `['abcd.efgh', 'ijkl']`, что нам подходит. За исключением того, что это не кортеж, а список.

В то же время у нас есть порядка 20 столбцов, которые не попадают под эту категорию. Для них мы хотим в качестве столбца первого уровня в индексе использовать слово `general`, а в качестве второго – полное имя столбца. Давайте перечислим имена таких столбцов в виде списка:

```
general_columns = ['age',
                   'are.you.datascientist',
                   'is.python.main',
                   'company.size',
                   'country.live',
                   'employment.status',
                   'first.learn.about.main.ide',
                   'how.often.use.main.ide',
                   'is.python.main',
                   'main.purposes',
                   'missing.features.main.ide',
                   'nps.main.ide',
                   'python.version.most',
                   'python.years',
                   'python2.version.most',
                   'python3.version.most',
                   'several.projects',
                   'team.size',
                   'use.python.most',
                   'years.of.coding'
                  ]
```

Теперь напомним функцию с именем `column_multi_name`, которая будет принимать на вход единственный параметр в виде имени столбца. Далее, если этот столбец входит в список `general_columns`, мы будем возвращать кортеж, в котором первым элементом будет идти строка `'general'`, а вторым – полное имя столбца. В противном

случае мы будем разбивать имя столбца надвое при помощи метода `str.rsplit` и возвращать кортеж из двух его составляющих по описанной выше схеме:

```
def column_multi_name(column_name):
    if column_name in general_columns:           ❶
        return ('general', column_name)         ❷
    else:
        first, rest = column_name.rsplit('.', 1)  ❸
        return (first, rest)                     ❹
```

- ❶ Должен ли переданный столбец иметь префикс `general`?
- ❷ Возвращаем кортеж, в котором первым элементом идет строка `'general'`.
- ❸ Разбиваем строку с именем столбца надвое.
- ❹ Возвращаем составляющие в виде кортежа.

Вызовем эту функцию для всех столбцов в датафрейме `py_df` и передадим результат в виде списка функции `pd.MultiIndex.from_tuples`. Для этого воспользуемся генератором списков (list comprehension):

```
(
    pd
    .MultiIndex.from_tuples([
        column_multi_name(one_column_name)  ❶
    for one_column_name in py_df.columns])  ❷
)
```

- ❶ Вызываем функцию `column_multi_name` для текущего имени столбца.
- ❷ Проходим по всем именам столбцов в датафрейме `py_df`.

После этого присвоим результирующий список атрибуту `py_df.columns`, тем самым заменив исходные имена столбцов на множественный индекс:

```
py_df.columns = (
    pd
    .MultiIndex.from_tuples([
        column_multi_name(one_column_name)
    for one_column_name in py_df.columns ])
)
```

Сортировка имен столбцов по алфавиту

В большинстве случаев не имеет значения, отсортированы ли столбцы в датафрейме или нет. Но мне кажется, что при работе с множественным индексом бывает удобно упорядочивать столбцы – это облегчает анализ. Воспользуемся преимуществом того, что мы можем передать датафрейму отсортированный список колонок, и он вернется в упорядоченном виде по столбцам. Присваивание результата обратно переменной `py_df` позволит получить новый датафрейм в желаемом для нас виде:

```
py_df = py_df[sorted(py_df.columns)]
```

Десять самых популярных IDE среди разработчиков на Python

Информацию о наиболее популярных в среде разработчиков на языке Python инструментах IDE можно получить из множественного индекса со столбцом первого уровня `ide` и столбцом второго уровня `main`, передав их в виде кортежа, как показано ниже:

```
py_df[('ide', 'main')]
```

Посчитать, сколько раз встречается в списке та или иная среда, можно с помощью метода `value_counts`, а вывести первые десять элементов – посредством метода `head`:

```
(
    py_df[('ide', 'main')]
        .value_counts()
        .head(10)
)
```

Вывод:

```
(ide, main)
VS Code                8010
PyCharm Professional Edition  5144
PyCharm Community Edition   3815
Vim                    2176
Sublime Text           1201
Jupyter Notebook       1167
Atom                   784
Other                  711
Emacs                  636
Spyder                 580
Name: count, dtype: int64
```

Десять языков программирования (`other.lang`), помимо профильного, которые чаще используют в своей работе разработчики на Python

Здесь нам предлагается определить, какие еще языки программирования, помимо Python, используют разработчики в своей профессиональной деятельности. В нашем множественном индексе эти сведения располагаются в элементе первого уровня с именем `other.lang`, а вложенные уровни называются в соответствии с используемыми языками. Таким образом, если мы запросим из нашего датафрейма `py_df['other.lang']`, то получим датафрейм с 24 столбцами, соответствующими прочим языкам программирования, из группы `other.lang`. Количество строк у нас будет то же, что и в исходном датафрейме – 54462. В каждой ячейке этого датафрейма будет стоять либо имя соответствующего языка программирования, либо значение NaN, говорящее о том, что данный респондент не использует этот язык в своей работе. Как мы на основе этих данных можем посчитать, сколько разработчиков используют тот или иной язык?

Ответ на этот вопрос проще, чем может показаться, поскольку, как мы помним, метод `count` возвращает количество непропущенных значений в объекте

Series. При применении к датафрейму этот метод возвращает объект Series с индексами в виде имен столбцов и значениями, соответствующими количеству непустых вхождений в каждом столбце. Таким образом, мы можем вызвать метод следующим образом:

```
py_df['other.lang'].count()
```

На выходе получим результат в виде объекта Series. Осталось отсортировать полученный список по убыванию:

```
(
    py_df['other.lang']
    .count()
    .sort_values(ascending=False)
)
```

И финальный штрих в виде получения первых десяти значений:

```
(
    py_df['other.lang']
    .count()
    .sort_values(ascending=False)
    .head(10)
)
```

Полученный объект Series будет содержать всю нужную нам информацию:

```
JavaScript      16662
HTML/CSS        15469
Bash / Shell    13793
SQL             13391
C/C++           11623
Java            8109
C#              4460
PHP             4060
TypeScript      3717
Other           3592
dtype: int64
```

Десять наиболее полно представленных стран в исследовании

Эту информацию можно извлечь из нашего множественного индекса, обратившись к первому уровню `general` и второму – `country.live`, как показано ниже:

```
py_df[('general', 'country.live')]
```

В результате мы получим список стран респондентов. Для подсчета количества вхождений каждой из стран снова воспользуемся методом `value_counts`:

```
py_df[('general', 'country.live')].value_counts()
```

Поскольку метод `value_counts` сортирует результаты в порядке убывания, мы можем сразу применить метод `head` для получения десяти самых популярных стран, в которых обитают разработчики на Python:

```
py_df[('general', 'country.live')].value_counts().head(10)
```

Вывод:

```
(general, country.live)
United States      3975
India              2800
Germany            1807
China              1155
United Kingdom     1110
France             1078
Russian Federation  935
Other country      880
Brazil             812
Canada             644
Name: count, dtype: int64
```

Опыт разработчиков на Python в процентном отношении

На этот раз мы в наш любимый метод `value_counts` передадим параметр `normalize=True`, позволяющий извлечь информацию в виде долей каждого значения:

```
py_df[
    ('general', 'python.years')
].value_counts(normalize=True)
```

Вывод:

```
(general, python.years)
3-5 years      0.284272
Less than 1 year 0.239542
1-2 years      0.224834
6-10 years     0.154939
11+ years      0.096413
Name: proportion, dtype: float64
```

Большая часть разработчиков обладает опытом работы от 3 до 5 лет, но также очень много людей программируют на Python меньше года. В целом около 75% разработчиков имеют опыт менее 5 лет, а половина из них – менее 2 лет.

Страна с наибольшим количеством разработчиков на Python с более чем 10-летним (11+) опытом работы

А как насчет опытных разработчиков на Python? Давайте узнаем, где проживает наибольшее количество программистов как минимум с 10-летним стажем. Чтобы ответить на этот вопрос, сначала нужно оставить только строки с опытными разработчиками:

```
py_df[py_df[
    ('general', 'python.years')] == '11+ years']
```

Но постойте, нам ведь нужно сгруппировать данные по столбцу `country.live`, родителем которого является столбец `general`, как и у `python.years`. А значит, мы можем ограничить наш запрос, применив булев индекс только к столбцам, находящимся внутри группы `general`, как показано ниже:


```
py_df['general'][py_df[
    ('general', 'python.years')] == '11+ years']
```

Теперь, когда у нас остались только столбцы из группы `general`, можно сгруппировать данные по странам следующим образом:

```
py_df['general'][py_df[
    ('general', 'python.years')] == '11+ years'
].groupby('country.live')
```

Мы получили объект `DataFrameGroupBy`, из которого теперь можно извлекать нужную нам информацию. Давайте узнаем, сколько непустых значений присутствует в каждом столбце для всех стран:

```
py_df['general'][
    py_df[('general', 'python.years')] == '11+ years'
].groupby('country.live').count()
```

В полученном датафрейме в строках будут располагаться названия стран, а в столбцах – все колонки из группы `general`. Целочисленные значения в ячейках показывают, сколько непустых значений для этой страны присутствует в том или ином столбце. Если помните, нам необходимо посчитать сверхопытных разработчиков на Python в каждой стране, для чего мы можем ограничиться только столбцом `python.years`, как показано ниже:

```
(
    py_df['general']
    [py_df[(('general', 'python.years')] == '11+ years'
    ['python.years']
    .groupby('country.live')
    .count()
    )
```

В итоговом датафрейме в строках будут располагаться названия стран, а в столбцах – количество разработчиков на Python с опытом более 10 лет.

Чтобы найти страну с наибольшим представительством таких «зубров», нужно применить метод `sort_values` с упорядочиванием по убыванию. А для получения лидера среди этих стран достаточно вызвать метод `head(1)`, как показано ниже:

```
(
    py_df['general']
    [py_df[(('general', 'python.years')] == '11+ years'
    .groupby('country.live')
    ['python.years']
    .count()
    .sort_values(ascending=False)
    .head(1)
    )
```

Вывод:

```
country.live
United States    691
Name: python.years, dtype: int64
```

Страна с наибольшей долей разработчиков на Python с более чем 10-летним (11+) опытом работы

Неудивительно, что США лидируют по количеству наиболее опытных разработчиков на языке Python. Но нам интереснее узнать, в какой стране наблюдается наибольший процент таких опытных разработчиков.

Для этого нам потребуется узнать общее количество разработчиков по странам. Создадим переменную `country_experience`, в которой соберем столбцы `country.live` и `python.years` из датафрейма `py_df['general']`, и извлечем интересующую нас информацию, как показано ниже:

```
country_experience = (
    py_df['general']
    [['country.live', 'python.years']]
)

all_per_country = (
    country_experience
    ['country.live']
    .value_counts()
)
```

Теперь узнаем, сколько опытных разработчиков проживают в этих странах. Мы уже делали это, отвечая на предыдущий вопрос, но с переменной `country_experience` можно получить ответ иначе:

```
expert_per_country = (
    country_experience
    .loc[country_experience['python.years'] == '11+ years',
         'country.live']
    .value_counts()
)
```

Итак, мы получили два объекта `Series` (`expert_per_country` и `all_per_country`) с совпадающими индексами (название страны). Воспользуемся тем, что `pandas` будет применять индекс для сопоставления данных при делении одного объекта `Series` на другой:

```
(expert_per_country / all_per_country
 ).sort_values(ascending=False).dropna().head(10)
```

Вывод:

country.live	
Norway	0.265432
Ireland	0.225490
Australia	0.225420
Belgium	0.225108
Slovenia	0.224490
New Zealand	0.197917
Sweden	0.194030
Finland	0.190141

```
United Kingdom    0.186486
Austria           0.186170
Name: count, dtype: float64
```

Здесь мы сначала делим количество экспертов по Python на общее число разработчиков, после чего сортируем значения в убывающем порядке, чтобы найти страны с наибольшей долей опытных программистов. Во избежание появления пропущенных значений мы применяем метод `dropna`.

Как это ни удивительно, но США, где, как мы уже выяснили, проживает больше всего опытных программистов на Python, по соотношению экспертов с другими группами разработчиков не входят даже в первую десятку.

Но отражают ли полученные сведения реальную картину мира? Иными словами, действительно ли в Норвегии на каждых троих неопытных разработчиков приходится один сверхопытный? Лично я отношусь к этому весьма скептически. Возможно, речь идет о завышении своего реального опыта работы респондентами.

Загрузка данных об исследовании на сайте Stack Overflow

Теперь переключим скорости и добавим в анализ данные на основе опроса на сайте Stack Overflow. Похожим образом загрузим данные из файла CSV:

```
so_filename = '../data/so_2021_survey_results.csv'
so_df = pd.read_csv(so_filename, low_memory=False)
```

Мы снова передали функции `pd.read_csv` параметр `low_memory=False`, дав pandas понять, что можно не стесняться и использовать всю доступную память при определении типов данных столбцов.

Средняя зарплата для разных типов сотрудников

Опрос, проведенный на сайте Stack Overflow, включает большое количество информации о занятости и зарплатах программистов. Нас попросили на основе этих данных подтвердить или опровергнуть распространенные высказывания о том, что подрядчики и фрилансеры зарабатывают больше штатных сотрудников. Для этого мы выполним группировку по столбцу `Employment`, в котором представлены типы сотрудников:

```
so_df.groupby('Employment')
```

Нас интересуют средние зарплаты в долларах за год по типам сотрудников, которые можно получить следующим образом:

```
so_df.groupby('Employment')['ConvertedCompYearly'].mean()
```

Осталось отсортировать значения по убыванию, что позволит сравнить зарплаты:

```
(
    so_df
    .groupby('Employment')['ConvertedCompYearly'].mean()
    .sort_values(ascending=False)
)
```

Вывод:

Employed full-time	129913.094086
Retired	120252.500000
Independent contractor, freelancer, or self-employed	111160.260190
I prefer not to say	44589.437500
Employed part-time	43344.532974
Not employed, and not looking for work	NaN
Not employed, but looking for work	NaN
Student, full-time	NaN
Student, part-time	NaN
Name: ConvertedCompYearly, dtype: float64	

Трудно поверить в то, что средняя зарплата сотрудников, вышедших на пенсию, может лишь немного уступать зарплате штатных сотрудников.

Что касается самого вывода, его довольно трудно читать. Можно избавиться от пропущенных значений и обрезать суммы до двух знаков после запятой плюс разделить разряды запятыми.

Удалить значения NaN можно очень просто – при помощи метода `dropna`. А как должным образом отформатировать числа с плавающей точкой? Я бы для этого воспользовался f-строками:

```
x = 12345.6789
print(f'{x:,.2f}') ❶
```

❶ Выведет 12,345.68.

Мы не можем применить f-строки ко всем значениям в объекте `Series`, но можем воспользоваться для этого анонимной функцией с вызовом метода `apply`, как показано ниже:

```
(
    so_df
    .groupby('Employment')['ConvertedCompYearly'].mean()
    .sort_values(ascending=False)
    .dropna()
    .apply(lambda n: f'{n:,.2f}')
)
```

Если вы не любите `lambda`, можете воспользоваться методом `apply` совместно с методом `str.format` следующим образом:

```
(
    so_df
    .groupby('Employment')['ConvertedCompYearly'].mean()
    .sort_values(ascending=False)
    .dropna()
    .apply('{:,.2f}'.format)
)
```

Обратите внимание, что мы не вызываем этот метод, а передаем его в метод `apply`, где он вызывается для каждого значения.

Чтобы все числа с плавающей точкой отображались подобным образом, можно воспользоваться глобальной опцией `pd.options.display.float_format`:

```
pd.options.display.float_format = '{:,.2f}'.format
```

С установленной таким образом опцией pandas будет вызывать этот метод при работе с любыми числами с плавающей точкой.

Теперь посмотрим на зарплаты сотрудников не по типам занятости, а по уровню образования. Дополнительно мы разобьем данные по странам. Вы уже, наверное, поняли, что мы построим сводную таблицу, в индексе которой будут располагаться страны, в столбцах – уникальные значения из поля `EdLevel`, а в ячейках – средние значения по столбцу `ConvertedCompYearly`:

```
so_df.pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
```

Из-за длинных имен столбцов мы не будем приводить вывод этой инструкции.

Эта же сводная таблица, но для стран, входящих в набор данных ОЭСР

Далее загрузим данные о странах из файла `oecd_locations.csv` следующим образом:

```
oecd_filename = '../data/oecd_locations.csv'
oecd_df = pd.read_csv(oecd_filename,
                     header=None, index_col=1,
                     names=['abbrev', 'Country'])
```

В загруженном датафрейме в качестве индекса будет выступать столбец с полными названиями стран. Причина в том, что сейчас мы будем объединять эти данные с датафреймом `so_df` по этому полю.

Итак, объединим наш свежий датафрейм со странами с данными опроса с сайта Stack Overflow и воссоздадим сводную таблицу. В результате количество строк в ней сильно сократится, поскольку датафрейм со списком стран у нас располагается слева в объединении, а значит, именно он будет диктовать состав индекса в результирующем наборе данных, где окажется всего 13 строк:

```
(
    oecd_df
    .join(so_df
          .set_index('Country'))
    .pivot_table(index='Country',
                 columns='EdLevel',
                 values='ConvertedCompYearly')
)
```

Обратите внимание на вызов `so_df.set_index('Country')`, в котором мы временно установили в качестве индекса в датафрейме `so_df` столбец `Country`. Это позволило нам легко объединить два датафрейма по индексу и создать сводную таблицу.

Теперь, когда мы знаем все средние зарплаты по странам и уровням образования, можно ответить на поставленные вопросы. К примеру, нас попросили узнать, в какой из стран сотрудники с дипломом младшего специалиста ('Associate degree (A.A., A.S., etc.)') могут рассчитывать на большую зарплату. Можно было бы сохранить сводную таблицу в переменную, но мы реализуем наше решение в виде цепочки методов:

```
(
    oecd_df
    .join(so_df.set_index('Country'))
    .pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
    ['Associate degree (A.A., A.S., etc.)']
    .sort_values(ascending=False)
)
```

После создания сводной таблицы мы извлекли колонку с нужным нам именем и отсортировали значения от большего к меньшему.

Вывод:

Country	
Finland	282353.666667
Israel	146420.900000
Japan	143196.833333
Australia	117049.640000
Germany	98530.516854
	...
Hungary	51041.000000
Austria	43623.384615
Italy	36427.941176
Belgium	35664.000000
Brazil	25347.424242

Name: Associate degree (A.A., A.S., etc.), Length: 13, dtype: float64

Похоже, лучше всего сотрудникам с дипломом младшего специалиста платят в Финляндии, Израиле и Японии.

А как насчет сотрудников с докторской степенью ('Other doctoral degree (Ph.D., Ed.D., etc.)')? Получим ли мы те же страны? Давайте проверим:

```
(
    oecd_df
    .join(so_df.set_index('Country'))
    .pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
    ['Other doctoral degree (Ph.D., Ed.D., etc.)']
    .sort_values(ascending=False)
)
```

Вывод:

```
Country
Japan      157239.400000
Australia  150234.957447
France     140402.856000
Israel     131812.625000
Germany    108718.459893
...
Belgium    80832.444444
Austria    74783.166667
Finland    61508.250000
Hungary    52833.600000
Brazil     43123.214286
Name: Other doctoral degree (Ph.D., Ed.D., etc.), Length: 13, dtype: float64
```

На этот раз первенство захватили Япония, Австралия и Франция.

К этим результатам также есть определенные вопросы в плане правильности. К примеру, трудно представить, что в Израиле младшим специалистам платят в среднем 146 420 долл., а сотрудникам с докторской степенью – 131 812 долл.

Я всегда говорю, что нужно уметь не только оперировать данными на уровне технических приемов, но и развивать критическое мышление в отношении полученных результатов и ставить под сомнение правильность расчетов и исходных цифр по причине небольшой или нерелевантной выборки и других факторов.

Удаление строк из датафрейма `so_df`, в которых в поле `LanguageHaveWorkedWith` стоит значение `NaN`

Теперь нам нужно проанализировать работу программистов на Python на основе исследования сайта Stack Overflow. Столбец `LanguageHaveWorkedWith` позволяет понять, с какими языками программирования работает разработчик, и в нем данные хранятся в виде текста с точками с запятой (;) в качестве разделителей. Таким образом, если человек работает с Python и JavaScript, значение в этом столбце у него будет `Python;JavaScript`. Если мы хотим отобразить только разработчиков на Python, мы должны найти всех, у кого этот язык есть в составе значения в этом поле. Для выяснения этого мы можем воспользоваться стандартным методом `str.contains`. Но есть проблема – не все разработчики заполняли это поле, так что в некоторых строках здесь стоит значение `NaN`. А использование в методе `str.contains` значений, содержащих `NaN`, приведет к возникновению ошибки.

Таким образом, сначала нам нужно избавиться от строк с пропущенными значениями в этом столбце. Это можно сделать обычным образом, с помощью метода `dropna`:

```
so_df = (
    so_df
    .dropna(subset=[ 'LanguageHaveWorkedWith' ])
)
```

Удаление строк, в которых в поле `LanguageHaveWorkedWith` отсутствует Python

После того как мы избавились от пропущенных значений в столбце, мы можем быть уверены, что все значения в нем представляют собой строки. Теперь найдем строки, в которых в этом столбце содержится среди прочих языков программирования Python:

```
so_df = (
    so_df
    .loc
    [so_df['LanguageHaveWorkedWith'].str.contains('Python')]
)
```

В результате у нас осталось порядка 40 тыс. разработчиков на Python. Это чуть меньше, чем в первом исследовании, в котором приняли участие более 54 тыс. программистов, но тоже достаточно, чтобы делать какие-то выводы. Кроме того, в опросе спрашивалось о том, какие языки разработчики использовали в последний год, и мы не знаем точно, делали ли они это постоянно или открыли лишь раз.

Теперь нам бы хотелось сравнить результаты этого исследования с итогами предыдущего в отношении опыта разработчиков. Но сделать это напрямую нам не удастся, поскольку эти данные хранятся по-разному: если в опросе от JetBrains пользователи должны были выбирать опыт работы из предложенных вариантов (например, *Less than 1 year*, *1-2 years* и т. д.), то в исследовании от Stack Overflow они вводили конкретные цифры, соответствующие количеству лет в профессии.

Удаление строк из датафрейма `so_df`, в которых в поле `YEARS_CODE` стоит значение `NaN`

Сначала избавимся от пропущенных значений в столбце `YearsCode`, как показано ниже:

```
so_df = so_df.dropna(subset=['YearsCode'])
```

Замена значений в столбце `YearsCode`

Нас попросили создать новый столбец `experience` в датафрейме с опросом от Stack Overflow, в котором категории с опытом будут выражаться в виде чисел. Мы знаем, что для этого было бы удобно воспользоваться функцией `pd.cut`, но она умеет работать только с числовыми диапазонами, а в нашем столбце есть два строковых значения: *Less than 1 year* и *More than 50 years*. И сначала мы преобразуем их в числа:

```
so_df.loc[so_df['YearsCode'] ==
    'Less than 1 year', 'YearsCode'] = 0
so_df.loc[so_df['YearsCode'] ==
    'More than 50 years', 'YearsCode'] = 51
```

После этого мы можем спокойно приводить этот столбец к числовому типу, как показано ниже:

```
so_df['YearsCode'] = so_df['YearsCode'].astype(int)
```


Создание столбца experience с категориальными значениями

Теперь мы можем воспользоваться функцией `pd.cut` для воссоздания тех же категорий, связанных с опытом работы, что и в первом исследовании:

```
so_df['experience'] = pd.cut(so_df['YearsCode'],
                             bins=[-1, 1, 2, 5, 10, 100],
                             labels=['Less than 1 year',
                                     '1-2 years',
                                     '3-5 years',
                                     '6-10 years',
                                     '11+ years'])
```

Помните, что в функции `pd.cut` числа, переданные параметру `bins`, обозначают границы наших отрезков, а значит, если мы хотим назначить первой метке значение 0, то должны начинать отсчет от `-1`. Да, можно задать специальные параметры для включения левой и правой границы, но мне больше нравится такой подход.

Распределение разработчиков по опыту в опросе от Stack Overflow

После этого мы можем взглянуть на распределение программистов на основе опыта работы в исследовании, проведенном Stack Overflow. Воспользуемся для этого методом `value_counts`:

```
so_df['experience'].value_counts(normalize = True)
```

Вывод:

```
experience
11+ years      0.430323
6-10 years     0.297583
3-5 years      0.192158
1-2 years      0.040935
Less than 1 year 0.039000
Name: proportion, dtype: float64
```

Напомним результаты распределения разработчиков по опыту работы из первого исследования:

```
(general, python.years)
3-5 years      0.284272
Less than 1 year 0.239542
1-2 years      0.224834
6-10 years     0.154939
11+ years      0.096413
Name: proportion, dtype: float64
```

По этим результатам мы можем сделать вывод, что на сайте Stack Overflow опытных разработчиков гораздо больше, чем в опросе, проведенном JetBrains. К примеру, программистов с опытом работы меньше пяти лет в общем опросе было около 75 %, а на Stack Overflow их оказалось всего 25 %. При этом опытом работы меньше двух лет в общем опросе обладает едва ли не половина респондентов, тогда как в исследовании от Stack Overflow их процент не превышает десяти!

Стоит отметить, что эти два опроса нельзя сравнивать напрямую. В конце концов, что касается опыта, в опросе на Stack Overflow люди отвечали на вопрос о том, сколько лет они в целом занимаются программированием, тогда как в исследовании от JetBrains речь шла об опыте работы программистом на Python. Таким образом, человек, посвятивший программированию на Java 20 лет жизни, а в последние два года перешедший на Python, мог ответить на соответствующие вопросы в двух исследованиях совершенно по-разному. Соответственно, делать выводы, основываясь на данных, взятых из разных источников, можно только после тщательного анализа и проверки на сопоставимость. Просто проанализировать и сравнить данные из двух источников недостаточно. И все же интересно отметить, насколько общий опрос среди разработчиков смещен в сторону новичков, а исследование на Stack Overflow – в сторону опытных разработчиков. В будущие опросы от JetBrains можно было бы включить отдельный вопрос об общем опыте программирования, что помогло бы лучше понять общую картину.

Решение

```
py_filename = '../data/2020_sharing_data_outside.csv'
py_df = pd.read_csv(py_filename, low_memory=False)

general_columns = ['age', 'are.you.datascientist',
                   'is.python.main', 'company.size',
                   'country.live', 'employment.status',
                   'first.learn.about.main.id',
                   'how.often.use.main.id',
                   'is.python.main', 'main.purposes',
                   'missing.features.main.id',
                   'nps.main.id',
                   'python.version.most',
                   'python.years',
                   'python2.version.most',
                   'python3.version.most',
                   'several.projects',
                   'team.size',
                   'use.python.most',
                   'years.of.coding'
                  ]

def column_multi_name(column_name):
    if column_name in general_columns:
        return ('general', column_name)
    else:
        first, rest = column_name.rsplit('.', 1)
        return (first, rest)

py_df.columns = pd.MultiIndex.from_tuples(
    [column_multi_name(one_column_name)
     for one_column_name in py_df.columns])

py_df = py_df[sorted(py_df.columns)]
```

```

py_df[('ide', 'main')].value_counts().head(10)
py_df['ide'].value_counts().head(10)
py_df['other.lang'].count().sort_values(ascending=False).head(10)
py_df['general', 'country.live'].value_counts().head(10)
py_df[('general', 'python.years')].value_counts(normalize=True)

(
    py_df['general']
    [py_df[('general', 'python.years')] == '11+ years']
    .groupby('country.live')['python.years'].count()
    .sort_values(ascending=False)
    .head(1)
)

country_experience = (
    py_df['general']
    [['country.live', 'python.years']]
)

all_per_country = (
    country_experience['country.live']
    .value_counts()
)

expert_per_country = (
    country_experience
    .loc[
        country_experience['python.years'] == '11+ years',
        'country.live']
    .value_counts()
)

(expert_per_country / all_per_country).sort_values(
    ascending=False).dropna().head(10)

so_filename = '../data/so_2021_survey_results.csv'
so_df = pd.read_csv(so_filename, low_memory=False)

so_df.pivot_table(index='Country', columns='EdLevel',
    values='ConvertedCompYearly')

oecd_filename = '../data/oecd_locations.csv'
oecd_df = pd.read_csv(oecd_filename, header=None,
    index_col=1, names=['abbrev', 'Country'])

(
    oecd_df
    .join(so_df

```

```

        .set_index('Country'))
    .pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
)

(
    oecd_df
    .join(so_df.set_index('Country'))
    .pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
    ['Associate degree (A.A., A.S., etc.)']
    .sort_values(ascending=False)
)

(
    oecd_df
    .join(so_df.set_index('Country'))
    .pivot_table(index='Country',
                  columns='EdLevel',
                  values='ConvertedCompYearly')
    ['Other doctoral degree (Ph.D., Ed.D., etc.)']
    .sort_values(ascending=False)
)

so_df = so_df.dropna(subset=['LanguageHaveWorkedWith'])
so_df = so_df[so_df['LanguageHaveWorkedWith'].str.contains('Python')]

so_df.shape

so_df = so_df.dropna(subset=['YearsCode'])

so_df.shape

so_df.loc[so_df['YearsCode'] ==
          'Less than 1 year', 'YearsCode'] = 0
so_df.loc[so_df['YearsCode'] ==
          'More than 50 years', 'YearsCode'] = 51

so_df['YearsCode'] = so_df['YearsCode'].astype(int)

so_df['experience'] = pd.cut(so_df['YearsCode'],
                             bins=[-1, 1, 2, 5, 10, 100],
                             labels=['Less than 1 year',
                                     '1-2 years',
                                     '3-5 years',
                                     '6-10 years',
                                     '11+ years'])

so_df['experience'].value_counts(normalize=True)

```

Заключение

Фух! Это было довольно длинное упражнение, в котором мы постарались по максимуму объединить техники и приемы, показанные в предыдущих главах. Конечно, мы не воспользовались всеми инструментами и возможностями, представленными в `pandas`, но, честно говоря, я не знаю ни одного проекта, в котором это было бы возможно. Но сделали мы здесь, согласитесь, немало: загрузили и очистили исходные данные, объединили датафреймы, проанализировали их и даже попробовали сравнить. Если вам при выполнении этого проекта было довольно комфортно, значит `pandas` успешно проникает в вашу ДНК, и вы можете начинать использовать его в реальных проектах.

Глава 9

Строки

Размышляя о библиотеке `pandas` как об аналитическом инструменте, мы зачастую подразумеваем цифры и расчеты. И действительно, в большинстве случаев мы работаем в `pandas` с числовыми данными, разворачивая и рассматривая их под разными углами. Это неудивительно, поскольку ноги у этой библиотеки растут из `NumPy` – пакета для эффективной работы с числами, в основе которого лежит язык низкого уровня `C`. И именно поэтому большинство упражнений в этой книге посвящены работе с числовыми вычислениями.

В то же время при анализе данных нам зачастую приходится иметь дело с текстовыми данными, такими как имена пользователей, наименования товаров, регионы продаж, структурные подразделения организации, биржевые коды (тикеры), названия компаний и др. Иногда текст составляет основу вашего анализа, например при подготовке данных для модели машинного обучения на основе текста. Да и при работе с числами вам как-то нужно озаглавливать категориальные данные, так что без строк в `pandas` вам просто не обойтись.

На самом деле в `pandas` достаточно инструментов для работы с текстом. При этом данные хранятся не в объектах `NumPy`, а в специализированных строковых объектах – либо в тех, которые используются в `Python` по умолчанию, либо (в последнее время все чаще) в особых строковых классах `pandas`, меньше подверженных ошибкам и неоднозначности данных. Подробнее об этих объектах мы поговорим далее в этой главе. Но в том и ином случае мы можем применять все богатство механизмов для работы со строками, встроенных в `Python` и `pandas`.

Обычно работа со строками ведется при помощи атрибута доступа `str`, доступного для каждого объекта `Series`, содержащего строки. При вызове методов посредством атрибута `str` мы получаем на выходе новый объект `Series`, который можно присвоить исходной переменной, новой переменной или одному из столбцов в датафрейме.

В этой главе мы поработаем с упражнениями, которые помогут вам понять, как можно эффективно применять на практике атрибут `str` и его методы в `pandas`, а также использовать собственные функции при работе с текстовыми данными в столбцах.

Текстовые типы данных

На протяжении долгих лет библиотека `pandas` использовала в своей работе со строками встроенный текстовый тип данных, заимствованный у Python. Он отличается большей эффективностью в сравнении с библиотекой `NumPy`, которая хранит строки в массивах `C`, предоставляющих гораздо меньший функционал. Таким образом, при работе со строками `pandas` присваивает атрибуту `dtype` значение `object`. Преимуществом этого способа является достаточно богатый функционал для работы со строками, а недостатком – тот факт, что в этом случае в объекте `Series` могут храниться не только текстовые данные, но и любые другие объекты Python. Это нередко приводит к ошибкам, когда мы пытаемся хранить в таких столбцах списки, словари или значения `None`. В конце концов, все это объекты Python, так что у `pandas` нет никаких причин, чтобы отказать нам в их хранении в данном столбце.

В `pandas` версии 1.0.0 добавился тип данных `pd.StringDtype`, призванный решить эти проблемы. Как ясно из его названия, его предполагается использовать в качестве значения атрибута `dtype` в объектах `Series`. А поскольку он предназначен для работы исключительно со строками, в таком столбце нельзя будет хранить объекты других типов. В документации к `pandas` сказано, что этот тип данных однажды станет стандартом для хранения строковых данных.

Но постойте, значение атрибута `dtype object` позволяет безболезненно хранить значения `NaN`. А как дело будет обстоять с новым типом? Ведь `NaN` представляет собой тип `float`, а не `pd.StringDtype`. Ответ такой – если вы собираетесь применять этот новый тип данных, вам придется воспользоваться значениями `pd.NA` вместо `NaN`. Вы можете думать о `pd.NA` как о более гибкой версии значения `NaN`, совместимой со всеми типами данных в `pandas`.

Нужно ли начинать использовать тип `pd.StringDtype` уже сейчас? На момент написания книги документация отвечала на этот вопрос весьма уклончиво. С одной стороны, в ней заявляется масса преимуществ этого нового типа, а с другой – написано, что он является экспериментальным, и его реализация и API могут меняться без предупреждений.

В упражнениях из этой главы мы будем предполагать, что вы используете «старомодный», но стабильный и проверенный тип `object`. Хотя вы также можете перейти на тип `pd.StringDtype` и попробовать решить упражнения с ним. В принципе, в этом случае вам не придется что-то сильно менять, достаточно будет лучше следить за консистентностью данных в столбцах. А взамен вы можете получить большую производительность.

В табл. 9.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 9.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>s.explode</code>	Возвращает новый объект <code>Series</code> , в котором каждый элемент списка располагается в отдельном элементе	<code>s.explode()</code>	http://mng.bz/RxDP (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.explode.html)

Таблица 9.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>str.contains</code>	Возвращает объект Series с булевыми значениями, указывающими на то, какие элементы исходного объекта Series содержат искомую строку	<code>s.str.contains('a')</code>	http://mng.bz/2D2X (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.contains.html)
<code>str.get_dummies</code>	Возвращает датафрейм, заполненный нулями и единицами на основе объекта Series с категориальными данными	<code>s['country'].get_dummies(sep=';')</code>	http://mng.bz/1q2g (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.get_dummies.html)
<code>str.index</code>	Возвращает объект Series, состоящий из целочисленных значений, указывающих позицию искомой строки в соответствующих элементах в столбце	<code>s.str.index('a')</code>	http://mng.bz/PzEP (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.index.html)
<code>str.len</code>	Возвращает объект Series, состоящий из целочисленных значений, указывающих длины соответствующих элементов в столбце	<code>s.str.len()</code>	http://mng.bz/Jg0v (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.len.html)
<code>str.replace</code>	Возвращает объект Series на основе исходного объекта, в котором строковые вхождения из первого аргумента заменены на вхождения из второго	<code>s.str.replace('a', 'e')</code>	http://mng.bz/wv6Q (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.replace.html)
<code>str.split</code>	Возвращает объект Series, в котором каждый элемент представляет собой список строк. В качестве аргумента указывается разделитель, на основе которого выполняется разбивка	<code>s.str.split(';')</code>	http://mng.bz/qR2D (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.split.html)
<code>str.strip</code>	Возвращает объект Series с элементами, у которых с обеих сторон отсечены заданные символы	<code>s.str.strip('!?!')</code>	http://mng.bz/7D2y (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.str.strip.html)

Таблица 9.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>s.isin</code>	Возвращает объект Series с булевыми значениями, указывающими на то, входит ли текущий элемент в переданный в виде списка аргумент	<code>s.isin(['A', 'B', 'C'])</code>	http://mng.bz/mVW2 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isin.html)
<code>i.intersection</code>	Возвращает новый объект с индексом, содержащий пересечения элементов в двух существующих индексах	<code>i.intersection(i2)</code>	http://mng.bz/5w21 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Index.intersection.html)

Атрибут доступа `str`

Обычные строки в Python поддерживают огромное количество специальных методов и операторов от поиска (`str.index`) и замены (`str.replace`) до выделения подстрок (срезы) и проверки содержимого (`str.isdigit` и `str.isspace`). Но как применить все это богатство при работе с объектами Series, содержащими строки?

Для этого можно воспользоваться циклами `for` или генераторами списков. Но в pandas, как вы помните, не приветствуются циклы из-за их низкой эффективности. Таким образом, мы могли бы применить метод `apply` для вызова нужного нам метода с каждым элементом объекта Series. Также `apply` можно использовать для вызова применительно ко всем элементам собственной функции.

Но зачастую лучше будет отдать предпочтение атрибуту доступа `str`. С его помощью можно получить доступ к великому множеству методов для работы со строками, включая, но не ограничиваясь стандартными методами Python. Метод, применяемый с помощью атрибута `str`, вызывается для каждого элемента в последовательности. Возвращает он новый объект Series той же длины и с тем же индексом, что у оригинала, но со значениями, «пропущенными» через переданный метод обработки. К примеру, для определения длины каждого слова в строке мы можем вызвать метод `len` у атрибута `str` следующим образом:

```
s = Series('this is a test 123 456'.split())
s.str.len()
```

В результате мы получим новый объект Series с длинами элементов в исходном Series, как показано ниже и на рис. 9.1:

```
0    4
1    2
2    1
3    4
```

```
4    3
5    3
dtype: int64
```

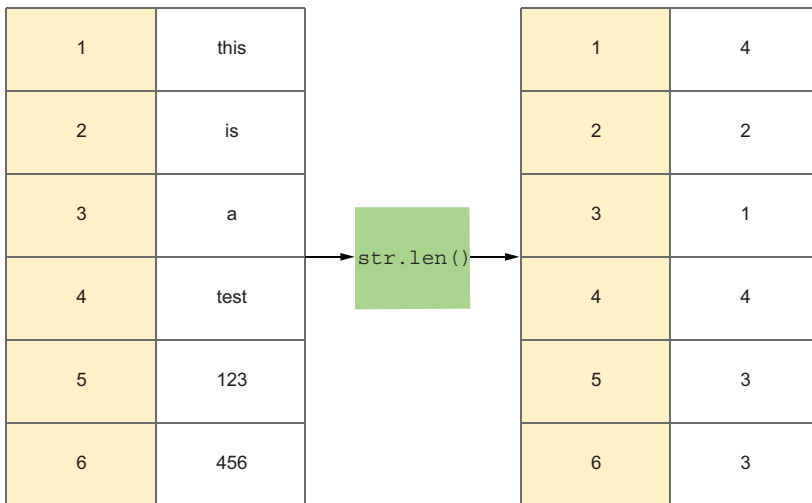


Рис. 9.1. Получение длин всех элементов в объекте Series посредством конструкции `.str.len()`

А что, если нам нужно найти все элементы в объекте Series, которые могут быть приведены к целочисленному типу данных? Воспользуемся для этого следующим выражением (см. рис. 9.2):

```
s.str.isdigit()
```

В результате мы получим объект Series, состоящий из логических значений, указывающих на то, какие элементы содержат только цифры от 0 до 9:

```
0    False
1    False
2    False
3    False
4     True
5     True
dtype: bool
```

Поскольку полученный объект содержит только логические значения, а его индекс совпадает с индексом исходного объекта, мы можем использовать его в качестве булевой маски для объекта `s` с целью нахождения числовых значений, как показано ниже:

```
s.loc[s.str.isdigit()]
```

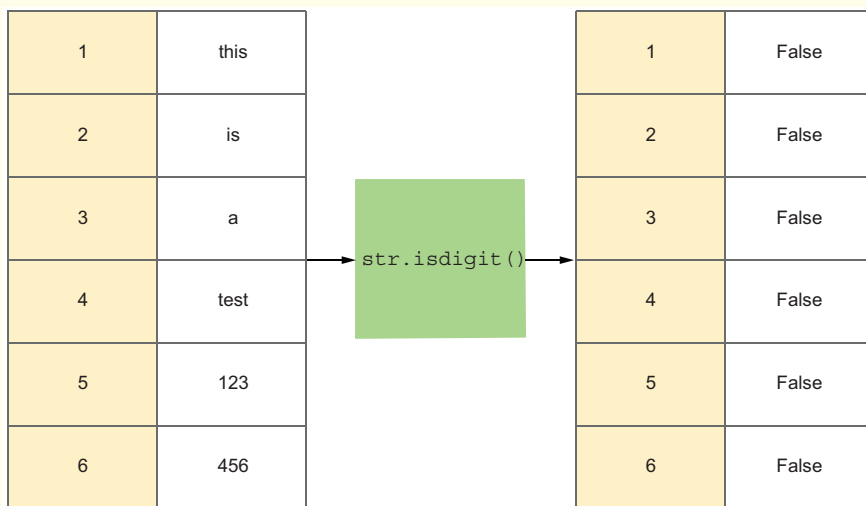


Рис. 9.2. Определяем, какие элементы исходного объекта содержат числовые значения

Кроме того, атрибут доступа `str` поддерживает методы, выходящие за рамки стандартных методов, характерных для класса `str` в Python. К примеру, мы можем воспользоваться методом `contains` для поиска подстроки в строке. В то же время метод `contains` позволяет осуществлять поиск в столбцах датафреймов с использованием регулярных выражений. Таким образом, мы можем легко найти в столбце слова, содержащие букву `a` или `e`, как показано ниже и на рис. 9.3:

```
s.str.contains('[ae]')
```

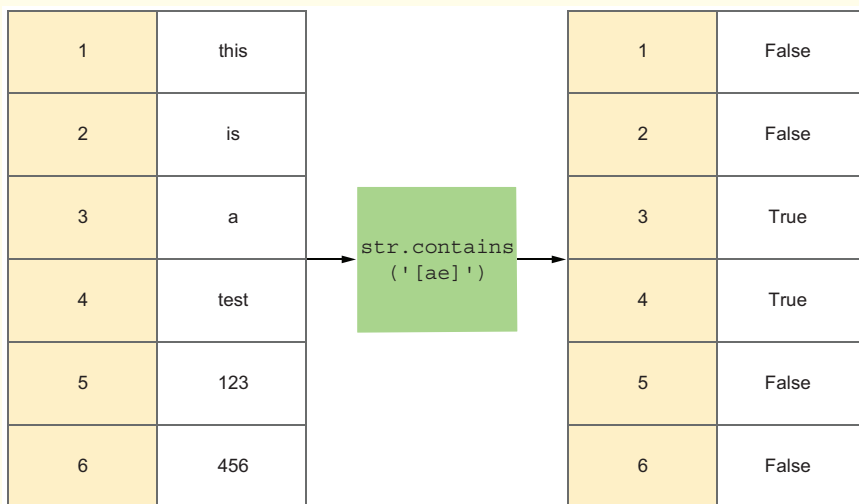


Рис. 9.3. Ищем элементы, содержащие букву `a` или `e`

На выходе мы получим следующий объект Series:

```
0    False
1    False
2     True
3     True
4    False
5    False
```

```
dtype: bool
```

Применив результат к нашему исходному объекту в качестве маски, мы оставим только строки с нужными нам словами:

```
s.loc[s.str.contains('[ae]')]
```

Результат будет следующим:

```
2      a
3    test
dtype: object
```

Обратите внимание, что, хотя на момент написания книги метод `str.contains` по умолчанию ожидает поступления на вход регулярного выражения, в будущем разработчики библиотеки планируют изменить это поведение. Так что всегда лучше явно указывать свои намерения работать с регулярными выражениями при помощи параметра `regex=True`, как показано ниже:

```
s[s.str.contains('[ae]', regex=True)]
```

Атрибут доступа `str` значительно облегчает работу со строковыми значениями в `pandas`. Но вы должны выделить время и прочитать документацию по всем доступным методам, чтобы досконально понять, как именно они работают.

УПРАЖНЕНИЕ 36. Анализируем Алису

В этом упражнении мы вместе прочитаем замечательную книжку «Алиса в стране чудес», текст которой можно бесплатно загрузить с сайта *Project Gutenberg*. Он также содержится в сопроводительных файлах к этой книге. Вот что вам нужно сделать.

1. Загрузить текст книги из файла `alice-in-wonderland.txt` в объект Series, чтобы каждое слово было представлено отдельным элементом. Если вам больше по душе датафрейм, ничего страшного. Я буду оперировать при разборе задачи объектом Series, или столбцом.
2. Ответить на следующие вопросы:
 - какие десять слов встречаются в книге чаще остальных?
 - изменится ли результат, если при подсчете слов использовать регистр?
 - изменится ли результат, если удалить знаки препинания (как они определены в `string.punctuation`) в начале и конце каждого слова?

- сколько слов, начинающихся с заглавной буквы, содержится в книге?
- а если игнорировать знаки препинания и кавычки перед началом слова, сколько останется слов, начинающихся с заглавной буквы?
- подсчитайте количество гласных букв (а, е, и, о и у) в каждом слове. Каково среднее количество гласных букв в каждом слове?

Подробный разбор

Как вы уже догадались, в этом упражнении мы отработаем на практике применение различных методов для работы со строками, присутствующих в `pandas`. Для начала загрузим книгу «Алиса в стране чудес» в объект `Series`. Обычно мы не загружаем длинные тексты в `pandas`, хотя применение этой библиотеки настолько обширно, что я не удивлюсь, если кто-то делает это на регулярной основе. Если передать на вход объекту `Series` выражение `open(filename)`, мы получим последовательность из строк книги, а не из слов, как сказано в условии задачи.

Чтобы загрузить текст по словам, необходимо сделать следующее.

1. Прочитать весь текст в Python в виде строки с помощью метода `read`.
2. Превратить строку в список посредством вызова метода `str.split`.
3. Преобразовать полученный список в объект `Series`.
4. Ниже показано, как это можно сделать:

```
filename = '../data/alice-in-wonderland.txt'
s = Series(open(filename).read().split())
```

ПРИМЕЧАНИЕ. Метод `read` возвращает строку с содержимым файла. А что, если в файле содержится несколько терабайтов данных? В этом случае, если вы работаете не в богатейшей компании в IT-индустрии, у вас возникнут проблемы с нехваткой памяти. Обычно я советую не считывать файлы целиком, а обрабатывать их последовательно, по строкам. В данном случае я знаю, что файл у нас небольшой, а значит, никаких проблем не возникнет.

Итак, загрузив текст книги в объект `Series`, мы можем начать его анализировать. Сначала нас попросили узнать, какие десять слов встречаются в книге чаще остальных. Здесь, как и во множестве других случаев, нам поможет метод `value_counts`. Его вызов приведет к появлению нового объекта `Series`, в котором в качестве индекса будут присутствовать слова из книги (т. е. значения из объекта `s`), а в качестве значений – частота появления этих слов в книге в порядке убывания:

```
s.value_counts()
```

Неудивительно, что наиболее часто в книге встречаются слова `the`, `and`, `a` и `to`. А что, если слова стоят в начале предложений и пишутся с заглавной буквы? Тогда они будут восприниматься как отдельные уникальные слова. Как можно преобразовать все слова в нижний регистр, а затем подсчитать их количество в тексте? Для этого можно воспользоваться атрибутом доступа `str` и его методом `lower`. В результате мы получим объект `Series`, состоящий из слов в нижнем регистре, к которому можно применить метод `value_counts`, как показано ниже:

```
s.str.lower().value_counts().head(10)
```

Но постойте, мы ведь разбили текст по пробельным символам, а значит, к нашим словам могут примыкать разные знаки препинания, что также нарушает правильность подсчета. Таким образом, мы должны сначала избавиться от символов пунктуации и только после этого считать количество слов. Это можно легко сделать с помощью метода `str.strip`, который может удалять заданные символы с левой и правой стороны слова. Обычно он применяется для избавления от ненужных множественных пробелов следующим образом:

```
s = ' a b c '
s.strip() ❶

❶ Вернет 'abc'.
```

Но мы также можем передать методу `str.strip` строковый аргумент с указанием символов, которые необходимо удалить с начала и конца слова:

```
s = ';;;abc;;;'
s.strip(';;') ❶

❶ Удалит все двоеточия и точки с запятыми в начале и конце слова и вернет 'abc'.
```

В специальном модуле `string` представлено множество predefined строк, среди которых есть `string.punctuation`, которой очень удобно пользоваться в подобных случаях, как показано ниже:

```
import string
s = ';;;abc;;;'
s.strip(string.punctuation) ❶

❶ Вернет 'abc'.
```

Таким образом, мы можем избавиться все значения в объекте `Series` от символов пунктуации с обеих сторон, вызвав метод `strip` атрибута `str`:

```
s.str.strip(string.punctuation)
```

Получается, что подсчитать количество уникальных слов в тексте, не обремененных знаками препинания, можно следующим образом:

```
(
    s
    .str
    .strip(string.punctuation)
    .value_counts()
    .head(10)
)
```

И хотя нас об этом не просили, мы можем совместить оба предыдущих требования и узнать количество слов после приведения их к нижнему регистру и избавления от символов пунктуации:

```
(
    s
    .str
    .lower()
    .str
```

```

        .strip(string.punctuation)
        .value_counts()
        .head(10)
    )

```

Обратите внимание, что здесь мы дважды воспользовались атрибутом доступа `str`: один раз для применения метода `lower`, второй – для `strip`, который вызывается у объекта `Series`, возвращенного методом `str.lower`. Мы еще не раз увидим подобный подход при решении этого и следующих упражнений.

Далее нас попросили узнать, сколько в тексте слов, начинающихся с заглавной буквы. Это означает, что нам нужно подсчитать количество слов, первым символом в которых является буква в диапазоне от А до Z. Существует масса способов сделать это, но я люблю в подобных ситуациях применять регулярные выражения. Метод `str.contains` поддерживает регулярные выражения, а значит, мы можем написать следующее выражение:

```

s.str.contains('^ [A-Z] \w*$',    ❶
                regex=True)

```

❶ Слова, начинающиеся с заглавной буквы, следом за которой идут ноль или больше буквенных символов.

В результате мы получим объект `Series` с логическими значениями и таким же индексом, как у `s`. Значения `True` будут соответствовать словам, начинающимся (начало строки определяется при помощи символа `^`) с заглавной буквы, следом за которой идут ноль или более буквенных символов (`\w*`) вплоть до конца строки (конец строки указывается символом `$`). Мы ищем *ноль* и более символов следом за заглавной буквой, поскольку хотим отловить слова, состоящие из одной заглавной буквы, такие как `A` или `I`.

Теперь можно применить результат к исходному объекту в качестве маски, как показано ниже:

```

(
    s
    .loc[s.str.contains('^ [A-Z] \w*$', regex=True)]
)

```

Осталось подсчитать количество вхождений с помощью метода `count`:

```

(
    s
    .loc[s.str.contains('^ [A-Z] \w*$', regex=True)]
    .count()
)

```

А что, если перед заглавной буквой будет стоять символ пунктуации, например кавычка? Для точного подсчета нам необходимо заранее избавиться от таких символов в начале и конце слов, что мы сделаем следующим образом:

```

(
    s
    .loc[s.str.strip(string.punctuation)
         .str
         .contains('^ [A-Z] \w*$', regex=True)]
)

```

```
.count()
)
```

Здесь мы сначала удалили знаки препинания в начале и конце слов, а затем результат отправили в метод `str.contains` для поиска слов, начинающихся с заглавной буквы. Далее мы полученные логические значения в виде объекта `Series` применили в виде маски к исходному объекту, что позволило подсчитать количество нужных нам вхождений.

После этого нас попросили подсчитать среднее количество гласных букв в каждом слове. Для этого нам сначала надо узнать, сколько гласных в каждом слове, а затем вычислить среднее. Проще всего можно сделать это с помощью метода `apply`, позволяющего вызывать функцию применительно ко всем элементам в объекте `Series`. Начнем с написания простой итерационной функции для подсчета количества гласных в слове, без изысков:

```
def count_vowels(one_word):
    total = 0
    for one_letter in one_word.lower():
        if one_letter in 'aeiou':
            total += 1
    return total
```

Эта функция принимает в качестве аргумента строку, считает в ней гласные буквы и возвращает целочисленный результат. Теперь мы можем вызвать ее для всех слов в нашей книге, как показано ниже:

```
s.apply(count_vowels)
```

Найти среднее количество гласных после этого не составит труда:

```
s.apply(count_vowels).mean()
```

Решение

```
filename = '../data/alice-in-wonderland.txt'
s = Series(open(filename).read().split()) ❶

s.value_counts().head(10) ❷
s.str.lower().value_counts().head(10) ❸
s.str.strip(string.punctuation).value_counts().head(10) ❹

(
    s
    .loc[s.str.contains('^[A-Z]\w*$', regex=True)]
) ❺

(
    s
    .loc[s.str.contains('^[A-Z]\w*$', regex=True)]
    .count()
) ❻
```



```
def count_vowels(one_word):
    total = 0
    for one_letter in one_word.lower():
        if one_letter in 'aeiou':
            total += 1
    return total
```

7

```
s.apply(count_vowels).mean()
```

8

- ❶ Создаем объект Series на основе слов в файле.
- ❷ Какие 10 слов встречаются чаще остальных?
- ❸ А если без учета регистра?
- ❹ А без знаков пунктуации?
- ❺ Сколько в книге слов, начинающихся с заглавной буквы?
- ❻ А без знаков пунктуации?
- ❼ Функция, определяющая количество гласных в слове.
- ❽ Какое среднее количество гласных в слове?

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/y82d>.

Дополнительные упражнения

- Каково среднее всех целочисленных значений, встречающихся в книге?
- Какие слова из книги не встречаются в словаре? Какие пять из них употребляются наиболее часто? Для этого упражнения я воспользовался версией словаря Linux, который можно загрузить по адресу <http://mng.bz/MZWB> (<https://gist.github.com/reuven/9ea704169a2b633d8afd27fc340ad8c5>). В сопроводительных материалах этот словарь есть в виде файла по пути `data/words.txt`.
- Какое минимальное и максимальное число слов в одном абзаце встречается в книге (абзацы разделяются двойным символом переноса строк)?

Ответы на дополнительные упражнения

Упражнение 36.1

```
import string

(
    s
    .str
    .strip(string.punctuation)
    .loc[lambda s_: s_.str.isdigit()]
    .astype(int)
    .mean()
)
```

Вывод:

```
8030.851851851852
```

Упражнение 36.2

```
words = {one_word.strip() for one_word in open('../data/words.txt')}

(
    s
    .str.strip(string.punctuation)      # Обрезаем знаки пунктуации
    .loc[lambda s_: s_.str.isalpha()]   # Оставляем только буквенные значения
    .loc[lambda s_: ~s_.isin(words)]    # Оставляем слова не из словаря
                                        # и считаем количество
    .value_counts()
)
```

Вывод:

```
Project      83
She          36
Rabbit       28
Queen        27
Gutenberg    27
..
reasons      1
knocked      1
curls        1
From          1
includes     1
Name: count, Length: 758, dtype: int64
```

Упражнение 36.3

```
# Читаем файл в объект Series по абзацам
s = Series(open(filename).read().split('\n\n'))

# Воспользуемся методом describe для получения минимума, максимума
# и всего остального
(
    s
    .str
    .split()
    .str
    .len()
    .describe()
)
```

Вывод:

```
count      393.000000
mean        32.475827
std         32.428415
min          0.000000
25%          7.000000
50%         22.000000
75%         48.000000
```

```
max      169.000000  
dtype: float64
```

УПРАЖНЕНИЕ 37. *Винные слова*

Я, как и многие, не против бокала вина за ужином. Зачастую на винной этикетке с обратной стороны содержится описание сорта вина, в котором производитель использует самые изысканные слова для демонстрации того, какие ощущения вы можете испытать, попробовав его продукцию. Уверен, не у меня одного поднимаются брови при виде употребляемых в этом жанре литературы слов. Я решил воспользоваться `pandas` для анализа слов, используемых при описании вин. Быть может, мы найдем в этой области нечто интересное...

В упражнении 35 мы уже обращались к базе данных с отзывами о винах разных стран. Здесь мы попробуем проанализировать частоту появления слов в этих отзывах и постараемся отследить закономерности для вин определенных сортов и из определенных регионов. Попутно мы познакомимся с некоторыми новыми техниками для исследования текста с помощью `pandas`. Вот что вам нужно будет сделать.

1. Загрузить данные из файла `winemag-150k-reviews.csv` в датафрейм. Вам понадобятся только столбцы `country` (страна), `province` (регион), `description` (описание) и `variety` (сорт).
2. Ответить на следующие вопросы:
 - какие десять слов, содержащих пять и более букв, используются в описаниях разных сортов вин чаще остальных? Для более правильного сравнения приведите все слова к нижнему регистру и избавьтесь от знаков препинания в начале и конце слов. Также исключите из поиска распространенные слова `flavors`, `aromas`, `finish`, `palate` и `drink`;
 - какие десять слов используются чаще остальных в описаниях французских (France) вин?
 - какие десять слов используются чаще остальных в описаниях вин не из Калифорнии (`province` не California)?
 - какие десять слов используются чаще остальных при описании белых вин? Здесь вы должны ориентироваться на сорт винограда и отобрать только описания для сортов Шардоне (Chardonnay), Совиньон-блан (Sauvignon Blanc) и Рислинг (Riesling);
 - какие десять слов используются чаще остальных при описании красных вин? Для этого будем отбирать вина сортов Пино-нуар (Pinot Noir), Каберне-совиньон (Cabernet Sauvignon), Сира (Syrah), Мерло (Merlot) и Зинфандель (Zinfandel);
 - какие десять слов используются чаще остальных при описании розовых вин (Rosé)?
3. Вывести десять самых популярных слов, используемых для описания пяти наиболее часто упоминаемых сортов вин.

Подробный разбор

Для начала нам нужно загрузить данные в датафрейм с теми столбцами, которые нам указали:

```
filename = '../data/winemag-150k-reviews.csv'

df = pd.read_csv(filename,
                  usecols=['country', 'province',
                           'description', 'variety'])
```

Теперь начнем анализировать слова, используемые в описаниях. Однако, поскольку мы видим, что нам придется отвечать на несколько похожих вопросов, лучше будет сразу написать некую функцию, к которой мы сможем повторно обращаться при решении похожих задач. Вот что должна делать наша функция:

- принять объект Series, состоящий из текстовых описаний вин;
- перевести слова в нижний регистр для более корректного сравнения;
- преобразовать входящий объект в объект Series, состоящий из отдельных слов;
- удалить символы пунктуации в начале и конце слов;
- удалить слова, состоящие менее чем из пяти букв;
- избавиться от слов, часто используемых при описании вин;
- вернуть десять самых часто встречающихся слов в наборе.

К счастью, в написании подобных функций нет ничего сложного. Мы назовем нашу функцию `top_10_words`. На вход она будет принимать единственный аргумент `s` в виде объекта Series, содержащего строки. В каждой строке будет содержаться некий текст, разделенный пробелами.

Первое, что нужно сделать согласно нашему плану, – это привести содержимое массива строк к нижнему регистру. Мы уже умеем делать это при помощи атрибута доступа `str` и его метода `lower`:

```
words = (
    s
    .str
    .lower()
)
```

Далее преобразуем принятый аргумент в объект Series, состоящий из слов. Таким образом, мы хотим, чтобы каждый элемент перечисления содержал ровно одно слово, а не их последовательность. И новый объект Series, как вы понимаете, будет более объемным (гораздо более объемным) в сравнении с исходным.

Если вы знакомы со стандартными методами Python для работы со строками, вас не удивит, что мы здесь воспользуемся методом `split` атрибута доступа `str`. Это потребует от нас второго использования атрибута `str` в одном выражении, чтобы метод `split` применялся ко всем элементам объекта, возвращенного из вызова `str.lower()`. Метод `split` также может принимать дополнительный параметр с разделителями, по которым нужно разбивать строки, но в нашем случае подойдут и стандартные пробельные символы, включающие в себя пробел, табуляцию и перенос строки:

```
words = (
    s
    .str
    .lower()
    .str
    .split()
)
```

Хорошая новость состоит в том, что нам удалось отделить слова друг от друга, а плохая – в том, что в нашем объекте `Series` осталось прежнее количество элементов. Просто внутри каждый элемент теперь содержит не строку, а список слов.

К счастью, как вы помните, в нашем распоряжении есть метод `explode`, принимающий на вход объект `Series`, содержащий итерируемые объекты, и возвращающий новый объект `Series`, в котором каждый из элементов этих объектов располагается на отдельной строке. Этот метод поможет нам собрать новую последовательность, которая будет состоять из отдельных слов:

```
words = (
    s
    .str
    .lower()
    .str
    .split()
    .explode()
)
```

Мы могли бы на этом остановиться, но нужно еще избавиться от знаков препинания в начале и конце слов. Это позволит, например, удалить артефакты в виде символов, стоящих в начале или конце предложений. Самый простой способ сделать это – воспользоваться методом `str.strip`. Обычно о методе `strip` мы думаем только в контексте обрезания лишних пробельных символов в обеих сторонах от слова, но это лишь поведение метода по умолчанию. В общем случае мы можем передать ему в виде параметра последовательность символов, от которых мы хотим избавиться. В результате мы получим новый объект `Series`, в котором не будет этих паразитных символов:

```
words = (
    s
    .str
    .lower()
    .str
    .split()
    .explode()
    .str
    .strip(' ,$.?!$%')
)
```

Теперь в нашей переменной `words` находится объект `Series`, состоящий из отдельных слов в нижнем регистре без знаков препинания в начале и конце. Далее избавимся от слов длиной менее пяти символов. Это можно сделать с

помощью применения индекса на основе метода `len`, вызванного у атрибута доступа `str`:

```
words.loc[(words.str.len())>=5]
```

Но это не единственный фильтр, который нужно применить к переменной `words`. Мы также хотим исключить из поиска слова, присутствующие почти в каждом описании вин. Мы можем воспользоваться методом `isin` объекта `Series`, передав ему список строк в качестве аргумента. Это позволит определить вхождение элементов в этот контрольный список:

```
common_wine_words = ['flavors', 'aromas', 'finish', 'drink', 'palate']
~words.isin(common_wine_words)
```

Мы можем скомбинировать эти две маски для получения только тех слов, которые содержат не менее пяти символов и не находятся в списке популярных слов:

```
(
    words
    .loc[(words.str.len())>=5) &
        (~words.isin(common_wine_words))]
)
```

Обратите внимание, что мы воспользовались оператором `~` (тильда), который в `pandas` означает оператор отрицания, позволяющий перевернуть логику на противоположную.

Наша функция называется `top_10_words`, поскольку она должна возвращать десять наиболее популярных слов в списке. С учетом того что в переменной `words` у нас хранится объект `Series` со словами, мы можем вызвать у него метод `value_counts` и применить фильтр `head(10)`, как показано ниже:

```
return (
    words
    .loc[(words.str.len())>=5) &
        (~words.isin(common_wine_words))]
    .value_counts()
    .head(10)
)
```

Теперь мы можем собрать нашу функцию `top_10_words`, которую можно применять к любому объекту `Series` со словами:

```
def top_10_words(s):
    common_wine_words = ['flavors', 'aromas',
                        'finish', 'drink', 'palate']

    words = s.str.lower().str.split(
        ).explode().str.strip(',$?!$%')

    return (
        words
        .loc[(words.str.len())>=5) &
```

```

        (~words.isin(common_wine_words))]]
    .value_counts()
    .head(10)
)

```

Давайте попробуем применить нашу функцию ко всем описаниям вин:

```
top_10_words(df['description'])
```

Вывод:

```

description
fruit      56327
acidity    32536
tannins     32098
cherry     30639
black      24568
spice      22601
sweet      21243
notes      19581
fresh      17641
berry      17083
Name: count, dtype: int64

```

Далее нас попросили узнать, какие десять слов используются чаще остальных в описаниях французских (France) вин.

Для этого мы отберем строки, в которых в столбце `country` указано название страны France, как показано ниже:

```
df.loc[df['country'] == 'France', 'description']
```

Полученный в результате объект `Series` мы можем затем отправить в нашу функцию `top_10_words`, чтобы найти десять самых популярных слов:

```

top_10_words(
    df
    .loc[df['country'] == 'France',
         'description']
)

```

Вывод:

```

description
fruit      8688
acidity    8632
tannins     6491
fruits     5449
fresh      4213
character   3494
black      3119
texture    3069
years      2880
crisp      2875
Name: count, dtype: int64

```

Теперь мы узнаем самые популярные слова в описаниях вин не из Калифорнии. Для этого нужно отфильтровать столбец `province` с помощью оператора `!=` следующим образом:

```
top_10_words(
    df
    .loc[df['province'] != 'California',
         'description']
)
```

Вывод:

```
description
fruit      46371
acidity    22270
tannins    21929
cherry     19440
spice      18522
black      17758
notes      16569
fresh      16200
berry      15478
sweet      12708
Name: count, dtype: int64
```

Обратите внимание, что здесь мы указали именно столбец `province`, а не `country`. Этот столбец позволяет проанализировать вина, произведенные не только в конкретной стране, но и в конкретном регионе страны, что бывает очень полезно.

Далее нас попросили узнать, какие десять слов используются чаще остальных при описании белых, красных и розовых вин по отдельности. Я привел список сортов, которые относятся к каждому виду вина, и ниже показано, как можно отфильтровать только нужные нам сорта:

```
top_10_words(
    df.loc[df['variety']
           .isin(['Chardonnay',
                  'Sauvignon Blanc',
                  'Riesling']),
         'description'])
top_10_words(
    df.loc[df['variety']
           .isin(['Pinot Noir',
                  'Cabernet Sauvignon',
                  'Syrah', 'Merlot',
                  'Zinfandel']),
         'description'])
top_10_words(
    df.loc[df['variety'] == 'Rosé',
         'description'])
```


Вывод приведем только для розовых вин:

```
acidity      1135
fruit        696
crisp        669
fresh        622
strawberry   534
light        514
raspberry    509
cherry       469
fruity       428
fruits       419
Name: count, dtype: int64
```

Обратите внимание, как метод `isin` позволил нам реализовать логику логического ИЛИ. Мы могли реализовать ее с помощью логических операторов в `pandas` и наложения маски, но вариант с методом `isin` является более лаконичным и понятным.

Наконец, нас попросили вывести десять самых популярных слов, используемых для описания пяти наиболее часто упоминаемых сортов вин. Для этого нам сначала понадобится определить эти пять сортов вин.

Сделаем мы это так:

```
(
    df['variety']
    .value_counts()
    .head(5)
    .index
)
```

Здесь мы воспользовались методом `value_counts` применительно к столбцу `variety`, отвечающему за сорта вин, для определения того, как часто встречается тот или иной сорт. С помощью вызова метода `head(5)` мы добыли пять самых популярных сортов вин, которые приведены ниже, и взяли индекс:

Вывод:

```
variety
Chardonnay      14482
Pinot Noir      14291
Cabernet Sauvignon 12800
Red Blend       10062
Bordeaux-style Red Blend 7347
Name: count, dtype: int64
```

Теперь мы можем отобрать описания для этих сортов, как показано ниже:

```
(
    df
    .loc[df['variety']
         .isin(df['variety']
              .value_counts()
              .head(5))
    ]
)
```

```
        .index),
        'description']
    )
```

Обратите внимание, что мы не можем применить метод `isin` к значениям, полученным в результате вызова метода `value_counts`, поскольку это будут числа. Вместо этого мы обратились к индексу объекта, в котором и хранятся наименования сортов вин.

В заключение мы обратимся к написанной нами ранее функции `top_10_words`, чтобы извлечь десять самых популярных слов в описаниях этих вин:

```
top_10_words(
    df
    .loc[df['variety']
         .isin(df['variety']
               .value_counts()
               .head(5)
               .index),
         'description']
)
```

Решение

```
filename = '../data/winemag-150k-reviews.csv'
df = pd.read_csv(filename,
                  usecols=['country', 'province',
                           'description', 'variety'])

def top_10_words(s):
    common_wine_words = ['flavors', 'aromas',
                         'finish', 'drink', 'palate']

    words = (
        s
        .str.lower()
        .str.split()
        .explode()
        .str.strip(',$?!$%')
    )

    return (
        words
        .loc[(words.str.len() >= 5) &
              (~words.isin(common_wine_words))]
        .value_counts()
        .head(10)
    )

top_10_words(df['description'])
top_10_words(df.loc[df['country'] ==
                     'France', 'description'])
```

```

top_10_words(df.loc[df['province'] !=
    'California', 'description'])

top_10_words(
    df.loc[df['variety']
        .isin(['Chardonnay',
            'Sauvignon Blanc',
            'Riesling']),
        'description'])

top_10_words(
    df.loc[df['variety']
        .isin(['Pinot Noir',
            'Cabernet Sauvignon',
            'Syrah', 'Merlot',
            'Zinfandel']),
        'description'])

top_10_words(
    df.loc[df['variety'] == 'Rosé',
        'description'])

top_10_words(
    df
    .loc[df['variety']
        .isin(df['variety']
            .value_counts()
            .head(5)
            .index),
        'description']
)

```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/aE4m>.

Дополнительные упражнения

1. Вина из каких десяти стран получили наивысшие средние оценки (добавьте столбец `points` из исходного файла)?
2. Создайте сводную таблицу, в которой в качестве индекса будут присутствовать страны, в качестве столбцов – сорта вин, а в ячейках будут показаны средние оценки. Включите в таблицу только десять самых популярных сортов вин.
3. Как соотносится разнообразие вин, представленных страной, со средней оценкой, выставленной этой стране? Иными словами, коррелирует ли средняя оценка страны с количеством вин, представленных от ее имени в опросе?

Ответы на дополнительные упражнения

Упражнение 37.1

```
(
    df
    .groupby('country')['points'].mean()
    .sort_values(ascending=False)
    .head(10)
)
```

Вывод:

```
country
England      92.888889
Austria      89.276742
France       88.925870
Germany      88.626427
Italy         88.413664
Canada       88.239796
Slovenia     88.234043
Morocco      88.166667
Turkey       88.096154
Portugal     88.057685
Name: points, dtype: float64
```

Упражнение 37.2

```
(
    df.loc[df['variety']
           .isin(df['variety']
                 .value_counts()
                 .head(10)
                 .index)]
    .pivot_table(index='country',
                  columns='variety',
                  values='points')
)
```

Частичный вывод:

variety	Bordeaux-style Red Blend	Cabernet Sauvignon	...	Syrah	Zinfandel
country					
Argentina	89.575472	85.527745	...	85.232394	NaN
Australia	88.841463	88.115502	...	91.952381	88.200000
Austria	91.625000	87.750000	...	87.000000	NaN
Brazil	86.000000	81.000000	...	NaN	NaN
Bulgaria	NaN	84.812500	...	90.000000	NaN
...	...	...	...	...	...
Spain	86.333333	85.689189	...	87.548387	NaN
Switzerland	NaN	NaN	...	NaN	NaN
Turkey	86.666667	91.000000	...	89.000000	NaN

US	89.311681	88.555786	...	88.359850	86.664819
Uruguay	NaN	83.500000	...	NaN	NaN

[37 rows x 10 columns]

Упражнение 37.3

```
(
    df
    .groupby('country')['points']
    .agg(['count', 'mean'])
    .corr()
)
```

Вывод:

	count	mean
count	1.000000	0.236117
mean	0.236117	1.000000

УПРАЖНЕНИЕ 38. Зарплата программистов

В опросе на сайте Stack Overflow, данные из которого мы исследовали в упражнении 8, разработчики среди прочего указывали языки программирования, используемые в работе. К сожалению, в наборе данных эта информация собрана в одном столбце с разделителями в виде точки с запятой. В этом упражнении мы поработаем с этими данными и рассмотрим их под разными углами.

Что вам нужно будет сделать.

1. Загрузить данные из файла `so_2021_survey_results.csv` в датафрейм. Вам понадобятся только столбцы `LanguageHaveWorkedWith`, `LanguageWantToWorkWith`, `Country` и `CompTotal`.
2. Ответить на приведенные ниже вопросы:
 - какие вообще языки программирования используют в своей работе разработчики?
 - какие десять из этих языков программирования пользуются наибольшей популярностью?
 - какие десять из этих языков программирования большинство разработчиков хотели бы использовать в своей работе?
 - какие языки входят в первую десятку в обеих группах, указанных выше?
 - с какими языками из топ-10 программисты работают, но не хотели бы работать в будущем?
 - выведите самые популярные языки программирования по странам;
 - какое в среднем количество языков программирования разработчики применяли за прошедший год?
 - какое самое большое количество используемых языков было указано разработчиком за прошедший год?

- сколько разработчиков выбрали наибольшее количество используемых языков?
 - сколько программистов, принявших участие в опросе, заявили о зарплате в 2 и более млн долл. в год?
3. Удалить строки с зарплатой свыше 2 млн долл. в год.
 4. Превратить столбец `LanguageHaveWorkedWith` в `dummy`-столбцы, чтобы каждый язык оказался в отдельной колонке.
 5. Найти наилучшую комбинацию языков, если вы хотите увеличить свою зарплату и вынуждены выбрать два языка из трех представленных: Python, JavaScript и Java.

Подробный разбор

В этом упражнении мы поработаем с одним из самых любопытных разделов опроса, проводимого на сайте Stack Overflow, который касается разнообразия языков программирования, используемых в работе. Хорошая новость состоит в том, что у нас на руках достаточно сведений, на основе которых можно делать серьезные аналитические выводы. Неудобство же состоит в том, что все используемые разработчиками языки программирования перечислены в исходном файле CSV в одном столбце через точку с запятой, что затрудняет работу с ними. В этом упражнении мы опробуем разные техники для обработки подобных данных.

Для начала загрузим нужные нам столбцы из файла с данными в датафрейм:

```
filename = '../data/so_2021_survey_results.csv'
df = pd.read_csv(filename,
                  usecols=['LanguageHaveWorkedWith',
                          'LanguageWantedToWorkWith',
                          'Country', 'CompTotal'])
```

Столбцы мы указали для того, чтобы pandas не использовал лишнюю память и смог более корректно определить типы данных в нашем наборе.

Первый вопрос, который нам задали, касается того, какие вообще языки программирования перечислили разработчики. Ответы содержатся в текстовом столбце `LanguageHaveWorkedWith`. При этом разработчики, разумеется, могли выбрать более одного языка, что обусловило такой неудобный формат хранения данных, с использованием точки с запятой в качестве разделителя, как показано ниже:

```
0      C++;HTML/CSS;JavaScript;Objective-C;PHP;Swift
9                                C++;Python
11  Bash/Shell;HTML/CSS;JavaScript;Node.js;SQL;Typ...
12                                C;C++;Java;Perl;Ruby
16      C#;HTML/CSS;Java;JavaScript;Node.js
```

Обратите внимание на третью строку. Респондент перечислил так много языков программирования, что pandas даже отказался выводить их все, добив вывод многоточием.

Опции отображения в pandas

При работе с pandas вы можете по своему усмотрению настроить максимальную ширину столбцов при отображении с помощью опции `display.max_colwidth`. Например, так:

```
pd.set_option('display.max_colwidth', 100)
```

Чтобы вернуть значение по умолчанию, можно воспользоваться функцией `pd.reset_option`, как показано ниже:

```
pd.reset_option('display.max_colwidth')
```

С подробностями опций отображения в pandas вы можете ознакомиться по адресу <http://mng.bz/gvYv> (https://pandas.pydata.org/pandas-docs/stable/user_guide/options.html).

Чтобы строить запросы к датафрейму с используемыми разработчиками языками программирования, нужно отделить языки один от другого и собрать в одном объекте `Series`. Как мы видели в упражнении 37, легче всего можно сделать это, сначала применив метод `split` к столбцу со строками и получив в итоге объект `Series` со списками, а затем вызвав у него метод `explode`, как показано на рис. 9.4:

```
(
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
)
```

В результате мы получили объект `Series` со всеми вхождениями языков из столбца `LanguageHaveWorkedWith` в качестве отдельных элементов. Это позволяет провести подсчет частоты встречаемости языков с помощью метода `value_counts`:

```
(
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
    .value_counts()
)
```

Вывод:

```
LanguageHaveWorkedWith
JavaScript      53587
HTML/CSS       46259
Python         39792
SQL            38835
Java           29162
...
F#              804
Erlang          651
APL             536
Crystal         466
COBOL           437
Name: count, Length: 38, dtype: int64
```

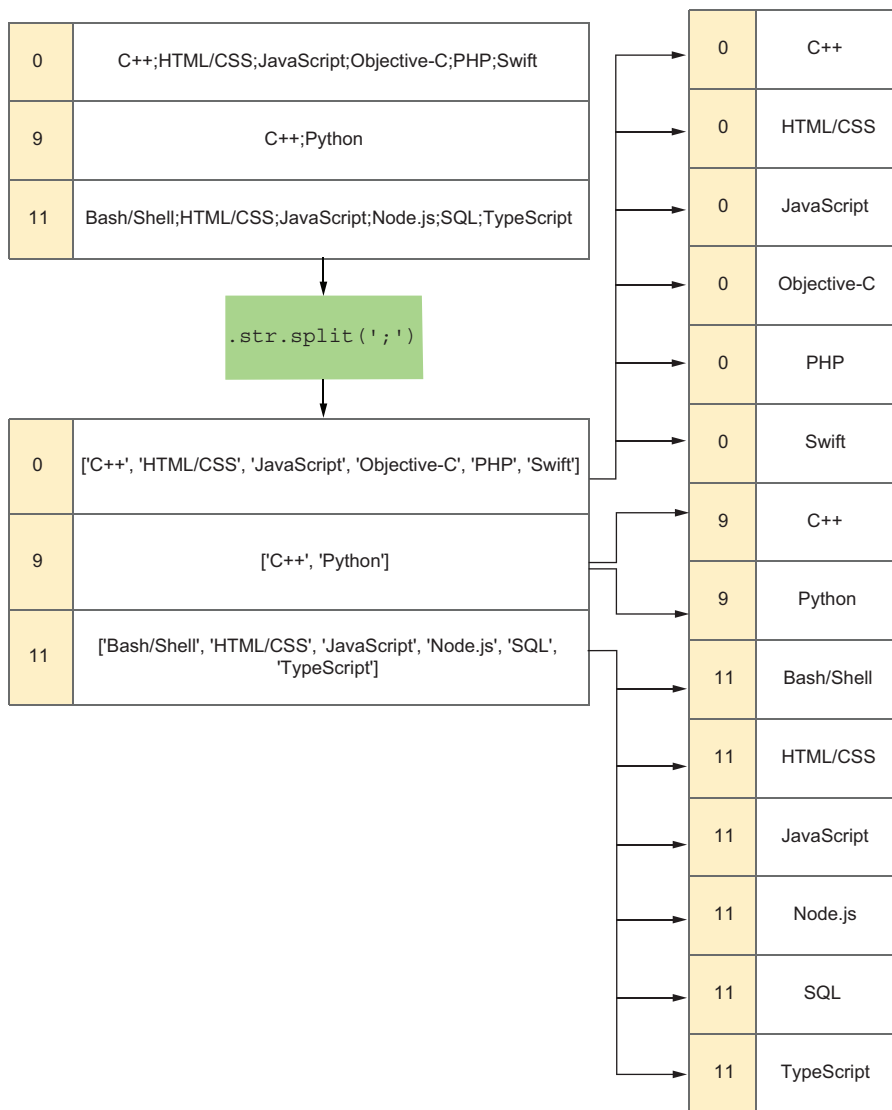


Рис. 9.4. Использование методов `split` и `explode` для преобразования Series с текстом в Series с отдельными словами

Таким образом, мы видим частоту упоминания каждого языка программирования в обзоре с сортировкой от самого популярного языка (JavaScript) к самому непопулярному (COBOL). Нас интересуют только первые десять языков по популярности:

```
(
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
    .value_counts()
```



```

        .head(10)
    )

```

При этом нас не столько интересуют цифры, сколько сами названия языков. Так что мы возьмем только содержимое индекса. Кроме того, желательно сохранить результаты в переменную, с которой нам будет удобно работать впоследствии:

```

have_worked_with = (
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
    .value_counts()
    .head(10)
    .index
)

```

Теперь в переменной `have_worked_with` у нас будет находиться следующий объект:

```

Index(['JavaScript', 'HTML/CSS', 'Python', 'SQL', 'Java', 'Node.js',
      'TypeScript', 'C#', 'Bash/Shell', 'C++'], dtype='object',
      name='LanguageHaveWorkedWith')

```

Далее мы аналогичным образом соберем информацию о языках, с которыми разработчики хотели бы работать в следующем году. Именно так был поставлен вопрос, ответы на который собраны в таком же формате в столбце `LanguageWantToWorkWith`. Извлечем эти языки:

```

want_to_work_with = (
    df['LanguageWantToWorkWith']
    .str.split(';')
    .explode()
    .value_counts()
    .head(10)
    .index
)

```

Получим следующий набор языков в виде индекса:

```

Index(['JavaScript', 'Python', 'HTML/CSS', 'TypeScript', 'SQL', 'Node.js',
      'C#', 'Java', 'Rust', 'Go'], dtype='object', name='LanguageWantToWorkWith')

```

Далее нас попросили найти языки программирования, присутствующие в обоих списках. Поскольку объекты индекса в `pandas` аналогичны `Series`, мы можем воспользоваться методом `isin`, чтобы узнать, какие элементы из переменной `want_to_work_with` встречаются в переменной `have_worked_with`, и применить полученный булев индекс к переменной `want_to_work_with`, как показано ниже:

```

(
    want_to_work_with
    .loc[want_to_work_with.isin(have_worked_with)]
)

```

Но в pandas можно сделать это проще – при помощи метода `intersection`. Обратите внимание, что этот метод работает именно с индексами, а не с объектами `Series`:

```
want_to_work_with.intersection(have_worked_with)
```

Вывод:

```
Index(['JavaScript', 'Python', 'HTML/CSS', 'TypeScript', 'SQL', 'Node.js',  
'C#', 'Java'], dtype='object')
```

Далее нас попросили узнать, с какими языками из топ-10 программисты работают, но *не* хотели бы работать в будущем? Мы снова можем воспользоваться методом `isin`, чтобы посмотреть, какие элементы из переменной `have_worked_with` присутствуют в переменной `want_to_work_with`:

```
have_worked_with.isin(want_to_work_with)
```

В результате мы получим булев индекс. Мы можем инвертировать значения в нем с помощью уже известного вам оператора `~` (тильда), чтобы определить, какие элементы из переменной `have_worked_with` *отсутствуют* в переменной `want_to_work_with`:

```
~have_worked_with.isin(want_to_work_with)
```

Осталось применить результирующий индекс в качестве маски к переменной `have_worked_with`, как показано ниже:

```
(  
    have_worked_with  
    [~have_worked_with.isin(want_to_work_with)]  
)
```

Вывод:

```
Index(['Bash/Shell', 'C++'], dtype='object', name='LanguageHaveWorkedWith')
```

Как видим, несмотря на популярность скриптов в Linux и языка C++, большинству программистов не хотелось бы работать с ними в будущем. И я с ними солидарен.

Затем нас попросили вывести самые популярные языки программирования по странам. Мы уже выяснили, что самым популярным языком в среде разработчиков является JavaScript. Но насколько объемлющей является эта тенденция? В нашем датафрейме есть столбец `Country`, так что кажется, что мы можем воспользоваться методом `groupby`, чтобы найти самые популярные языки по странам. Но есть проблема. Все языки у нас сейчас собраны в столбце `LanguageHaveWorkedWith` в виде строк с разделителями, и если применить метод `explode`, то полученный в результате объект `Series` будет иметь больше строк, чем в исходном датафрейме, а значит, мы не сможем добавить его в датафрейм в виде нового столбца.

В то же время мы знаем, что объект `Series`, возвращенный методом `explode`, будет содержать тот же индекс, что и в оригинале. Иными словами, если в элементе с индексом 0 в исходном объекте присутствовала строка с упоминанием языков Python и JavaScript, в результирующем объекте `Series` будет две строки с одина-

ковым индексом 0, в одной из которых будет Python, а в другой – JavaScript. Это означает, что, хоть мы и не можем объект Series, полученный из метода `explode`, добавить в датафрейм в качестве отдельного столбца, мы можем использовать метод `join` для объединения Series с датафреймом.

Для начала давайте создадим новый объект Series с именем `all_languages`, где будут содержаться все языки программирования, с которыми работали разработчики. Это делать совсем не обязательно, но данный шаг облегчит понимание операции объединения:

```
all_languages = (
    df
    ['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
)
```

Затем выполним объединение. Обратите внимание, что, хотя метод `join` и предназначен для объединения датафреймов, а не объектов Series, мы можем в качестве аргумента передать ему любой из этих объектов. Иными словами, мы могли бы написать следующую инструкцию:

```
df.join(all_languages)
```

На самом деле это не сработает, поскольку результирующий датафрейм должен был бы содержать два столбца с одинаковыми именами (`LanguageHaveWorkedWith`). Решать эту проблему можно по-разному. Например, можно воспользоваться параметрами `lsuffix` и/или `rsuffix` для добавления суффиксов к именам столбцов. Но мне кажется, гораздо проще будет понять, что нас на самом деле интересует только столбец `Country` в нашем датафрейме, а значит, мы можем выполнить объединение с сокращенным датафреймом, содержащим только один этот столбец. Сделать это можно так:

```
df[['Country']].join(all_languages)
```

Обратите внимание на двойные квадратные скобки, которые мы использовали, чтобы гарантировать получение на выходе датафрейма, а не объекта Series. Теперь, когда у нас есть объединенный датафрейм, можно применить к нему метод `groupby`, как показано ниже:

```
(
    df[['Country']]
    .join(all_languages)
    .groupby('Country')
)
```

В результате мы получим объект `DataFrameGroupBy`, к которому нужно применить какой-то агрегирующий метод. А какой метод нам выбрать? Обычные методы вроде `mean`, `count` или `std` здесь не подойдут, поскольку нам нужно получить наиболее часто встречающееся значение, которое именуется *модой* (`mode`). Но у нас нет агрегирующего метода `mode`. Однако мы можем позаимствовать этот метод у объекта Series, передав его параметру `agg` при вызове метода `groupby`, как показано ниже:

```
(
    df[['Country']]
    .join(all_languages)
    .groupby('Country')
    .agg(pd.Series.mode)
)
```

В результате мы получим датафрейм, состоящий из единственного столбца и индекса со странами. Значения в этом столбце будут соответствовать самому популярному языку программирования в каждой стране. Мы можем даже посмотреть сводные данные по количеству стран, в которых те или иные языки обладают наибольшей популярностью:

```
(
    df[['Country']]
    .join(all_languages)
    .groupby('Country')
    .agg(pd.Series.mode)['LanguageHaveWorkedWith']
    .value_counts()
    .head(5)
)
```

Вывод:

```
LanguageHaveWorkedWith
JavaScript      124
HTML/CSS        14
Python           4
SQL              4
C                 1
Name: count, dtype: int64
```

Далее нас попросили узнать, какое в среднем количество языков программирования разработчики применяли за прошедший год? Для решения этой задачи можно разбить значения в столбце `LanguageHaveWorkedWith` на списки и воспользоваться методом `len`. Это даст нам объект `Series`, состоящий из целочисленных значений, к которому мы можем применить метод `mean`, как показано ниже:

```
(
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .str.len()
    .mean()
)
```

В итоге мы получили число 5.37. Обратите внимание, что мы вынуждены были дважды использовать в этом выражении атрибут доступа `str`: первый раз для вызова метода `split`, а второй – для вызова метода `len`. Да, мы применили атрибут `str` для вызова метода `len` для списков. Атрибут будет пытаться применить этот метод к любым данным, а поскольку списки поддерживают этот метод, никаких проблем не возникнет.

Следующий вопрос: какое самое большое количество используемых языков было указано разработчиком за прошедший год? Это можно узнать, применив метод `max`, как показано ниже:

```
(
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .str.len()
    .max()
)
```

Как минимум один программист указал, что работает одновременно со всеми 38 языками. Похоже, он просто проставил все галочки без разбора. А может, не он один? В этом и состоит следующий вопрос в упражнении. Мы решим его следующим образом:

```
(
    df
    .loc[df['LanguageHaveWorkedWith']
    .str.split(';')
    .str.len() == 38,
    'LanguageHaveWorkedWith']
    .count()
)
```

Оказалось, что примеру этого «полиглота» последовал еще 31 разработчик.

Здесь мы воспользовались булевым индексом, который применили в качестве маски к столбцу `LanguageHaveWorkedWith` и вызвали метод `count` для подсчета итоговых строк.

Далее нас попросили узнать, сколько программистов, принявших участие в опросе, заявили о зарплате в 2 и более млн долл. в год. Сделаем это следующим образом:

```
(
    df
    .loc[df['CompTotal'] >= 2_000_000]
    ['CompTotal']
    .count()
)
```

Ничего себе, сразу 2369 разработчиков указали, что зарабатывают такие деньги! Давайте-ка удалим их из нашей выборки, чтобы они не вносили перекося в данные:

```
df = (
    df
    .loc[df['CompTotal'] < 2_000_000]
)
```

Скоро мы вернемся к разговору о зарплатах. А пока возьмем столбец `LanguageHaveWorkedWith` и преобразуем его в несколько столбцов для удобства анализа данных по отдельным языкам программирования. Таким образом, мы создадим так называемые фиктивные столбцы, или *dummy*-столбцы (*dummy column*), имена которых будут строиться на основании значений формата

'JavaScript;Python'. В результате мы получим столбцы, соответствующие каждому языку, значения в которых будут указывать на то, выбрал человек этот язык (1) или нет (0).

Это можно сделать на основе столбца `LanguageHaveWorkedWith`, воспользовавшись методом `str.get_dummies`, как показано ниже:

```
(
    df['LanguageHaveWorkedWith']
    .str.get_dummies(sep=';')
)
```

Результат будет следующий:

	APL	Assembly	Bash/Shell	C	C#	...	SQL	Scala	Swift	TypeScript	VBA
0	0	0	0	0	0	...	0	0	1	0	0
1	0	0	0	0	0	...	0	0	0	0	0
2	0	1	0	1	0	...	0	0	0	0	0
3	0	0	0	0	0	...	0	0	0	1	0
4	0	0	1	0	0	...	1	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...
83434	0	0	0	0	0	...	1	0	0	0	0
83435	0	0	0	0	0	...	0	0	0	0	0
83436	0	0	0	0	0	...	0	0	0	0	0
83437	0	0	1	0	0	...	0	0	0	0	0
83438	0	0	0	0	0	...	0	0	0	0	0

[83439 rows x 38 columns]

Но как полученные данные интегрировать в существующий датафрейм? В этом нам поможет функция `pd.concat`, которую мы уже использовали ранее. Разница будет состоять в том, что на этот раз мы будем объединять датафреймы горизонтально, т. е. присоединяя один к другому сбоку, а не снизу. Для этого достаточно передать функции `pd.concat` параметр `axis='columns'` аналогично тому, как мы делали с другими методами, такими как `df.drop`. Результат конкатенации мы можем присвоить обратно переменной `df`, как показано ниже:

```
df = (
    pd.concat([df,
               df['LanguageHaveWorkedWith']
               .str.get_dummies(sep=';')],
              axis='columns')
)
```

В итоге исходные колонки будут добавлены к датафрейму с `dummys`-столбцами, показанному выше. Теперь мы можем задавать вопросы по зарплате в корреляции с разными языками программирования. Сначала узнаем среднюю зарплату разработчиков, в арсенале которых есть языки Python и JavaScript, но нет Java:

```
df['CompTotal'][(df['Python'] == 1) &
                 (df['JavaScript'] == 1) &
                 (df['Java'] == 0)].mean()
```

Полученный результат составляет 126 817 долл.

А как насчет средней зарплаты разработчиков, использующих языки Python и Java, но не применяющих в работе JavaScript?

```
df['CompTotal'][(df['Python'] == 1) &
                 (df['JavaScript'] == 0) &
                 (df['Java'] == 1)].mean()
```

На этот раз средняя зарплата составила 162 737 долл.

А если разработчик знает Java и JavaScript, но не знает Python?

```
df['CompTotal'][(df['Python'] == 0) &
                 (df['JavaScript'] == 1) &
                 (df['Java'] == 1)].mean()
```

Результат составил 140 867 долл.

Решение

```
filename = '../data/so_2021_survey_results.csv'
df = pd.read_csv(filename,
                  usecols=['LanguageHaveWorkedWith',
                           'LanguageWantToWorkWith',
                           'Country', 'CompTotal'])

(
    df['LanguageHaveWorkedWith'].str.split(';')
    .explode()
    .value_counts()
    .index
)

have_worked_with = (
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
    .value_counts()
    .head(10)
    .index
)

want_to_work_with = (
    df['LanguageWantToWorkWith']
    .str.split(';')
    .explode()
    .value_counts()
    .head(10)
    .index
)

want_to_work_with.intersection(have_worked_with)
have_worked_with[~have_worked_with.isin(want_to_work_with)]
```

```

all_languages = (
    df['LanguageHaveWorkedWith']
    .str.split(';')
    .explode()
)

df[['Country']].join(all_languages).groupby('Country').agg(pd.Series.mode)

df['LanguageHaveWorkedWith'].str.split(';').str.len().mean()
df['LanguageHaveWorkedWith'].str.split(';').str.len().max()

(
    df['LanguageHaveWorkedWith'][df['LanguageHaveWorkedWith']
    .str.count(';') == 38]
    .count()
)

(
    df
    .loc[df['CompTotal'] >= 2_000_000]
    ['CompTotal']
    .count()
)

df = (
    df
    .loc[df['CompTotal'] < 2_000_000]
)

df = pd.concat(
    [df,
    df['LanguageHaveWorkedWith']
    .str.get_dummies(
        sep=';'), axis='columns']

df['CompTotal'][(df['Python'] == 1) &
    (df['JavaScript'] == 1) &
    (df['Java'] == 1)].mean()

df['CompTotal'][(df['Python'] == 1) &
    (df['JavaScript'] == 0) &
    (df['Java'] == 1)].mean()

df['CompTotal'][(df['Python'] == 0) &
    (df['JavaScript'] == 1) &
    (df['Java'] == 1)].mean()

```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/4JvR>.

Дополнительные упражнения

1. Какие три действия являются наиболее популярными для разработчиков, попавших в тупик (столбец NEWStuck)?
2. Какая доля респондентов выбрала в качестве пола мужской (Male) в столбце Gender? Соотносится ли результат с вашим представлением о соотношении полов среди разработчиков?
3. Сколько лет в процентном отношении от своего стажа (столбец YearsCode) программисты занимаются разработкой профессионально (столбец YearsCodePro) в среднем?

Ответы на дополнительные упражнения

Упражнение 38.1

```
(
    df['NEWStuck']
      .str.split(';')
      .explode()
      .value_counts()
      .head(3)
)
```

Вывод:

```
NEWStuck
Google it                74491
Visit Stack Overflow     66410
Do other work and come back later  39871
Name: count, dtype: int64
```

Упражнение 38.2

90% мужчин соответствует тому, что я обычно вижу в профессиональной среде

```
(
    df[['Gender']]
      .value_counts(normalize=True)
      .head(3)
)
```

Вывод:

```
Gender
Man                0.909231
Woman              0.050069
Prefer not to say  0.017524
Name: proportion, dtype: float64
```

Упражнение 38.3

```
df.loc[df['YearsCode'] == 'Less than 1 year', 'YearsCode'] = 0
df.loc[df['YearsCode'] == 'More than 50 years', 'YearsCode'] = 51
```

```

df.loc[df['YearsCodePro'] == 'Less than 1 year', 'YearsCodePro'] = 0
df.loc[df['YearsCodePro'] == 'More than 50 years', 'YearsCodePro'] = 51

# Переведем в целые числа
df = df[['YearsCode', 'YearsCodePro']].dropna()
df['YearsCode'] = df['YearsCode'].astype(np.int16)
df['YearsCodePro'] = df['YearsCodePro'].astype(np.int16)

# Избавимся от строк с нулями в столбце YearsCode
df = df[df['YearsCode'] != 0]

(df['YearsCodePro'] / df['YearsCode']).mean()

```

Вывод:

```
0.5923711657118932
```

Заклучение

В этой главе мы рассмотрели разные способы работы в pandas с текстовыми данными, в частности с использованием атрибута доступа `str`. Богатейшие возможности для работы с текстом в Python вкупе с разнообразными механизмами для манипуляции объектами `Series` и датафреймами, присутствующими в pandas, обеспечивают достаточную гибкость при ответе на самые сложные вопросы, не касающиеся чисел напрямую. В большинстве наборов данных (и один из них мы рассмотрели в этой главе) присутствуют как текстовые, так и числовые данные, и уметь работать с текстом столь же искусно, как с числами, крайне важно для любого аналитика данных.

Глава 10

Даты и время

Встроенные структуры данных в языках программирования всегда отражают специфику работы и используемые на постоянной основе типы данных. Очевидно, что мы часто работаем с числовыми данными, так что нам нужны структуры для хранения таких данных. Также мы часто работаем с текстом. Ну и без стандартных коллекций разработчику тоже никак не обойтись, в связи с чем во многих языках программирования предусмотрены соответствующие объекты (в Python это списки, кортежи, словари и множества).

Современное программирование предъявляет высокие требования к обработке еще одного типа данных, которому на заре моей карьеры не уделялось так много внимания. Речь идет о датах и времени. По прошествии времени кажется, что такие важные составляющие, как дата и время, просто не могут не быть досконально описаны и формализованы в языках программирования. Но дело в том, что работать с датами не так просто, и с ними связано множество проблем, начиная от определения високосного года и заканчивая расчетом часовых поясов. Кроме того, сами структуры данных для хранения дат довольно объемные, и изначально они не закладывались в архитектуру компьютера.

В Python и pandas информация о датах и времени хранится и обрабатывается при помощи двух разных структур данных. Первая – это *timestamp*, или метка времени, также известная в других языках программирования как *datetime*. Timestamp отражает конкретную точку во времени, которую можно использовать при работе с календарем. Каждая метка времени уникальна по своей природе и может отражать момент вашего рождения, время взлета самолета, дату и время свидания или момент его окончания. Любую метку времени можно охарактеризовать конкретным годом, месяцем, числом, часом, минутой и секундой.

Вторая структура именуется *timedelta*, или временные отрезки, и в некоторых других системах она именуется *interval*. Timedelta описывает протяженность времени, или расстояние от одной метки времени до другой. Таким образом, если момент начала и окончания свидания – это метки времени, то время, на протяжении которого это свидание продолжалось, – это *timedelta* (см. рис. 10.1).

Неудивительно, что большинство наборов данных, с которыми приходится работать на постоянной основе, содержат информацию о датах и времени. В связи с этим хорошая новость заключается в том, что в pandas достаточно много средств для работы с датами и временем. Мы можем легко загружать данные из файлов и преобразовывать их в даты на лету. Также можно преобразовывать существующие данные в метки времени, причем как отдельные значения, так и столбцы в

целом. В Python легко можно выполнять вычисления с временными отрезками и сравнивать их друг с другом.

Но в pandas пошли дальше и позволили использовать информацию о датах и времени в индексах. Это значительно облегчает поиск событий, произошедших в определенные промежутки времени. Кроме того, в pandas можно легко производить передискретизацию временных рядов, подразумевающую под собой группировку данных по временным интервалам.

В упражнениях из этой главы мы тщательно проработаем все возможности pandas по взаимодействию с датами и временем. Вы научитесь свободно оперировать типами данных, характерными для работы с датами, и строить отчеты на основе этих типов.

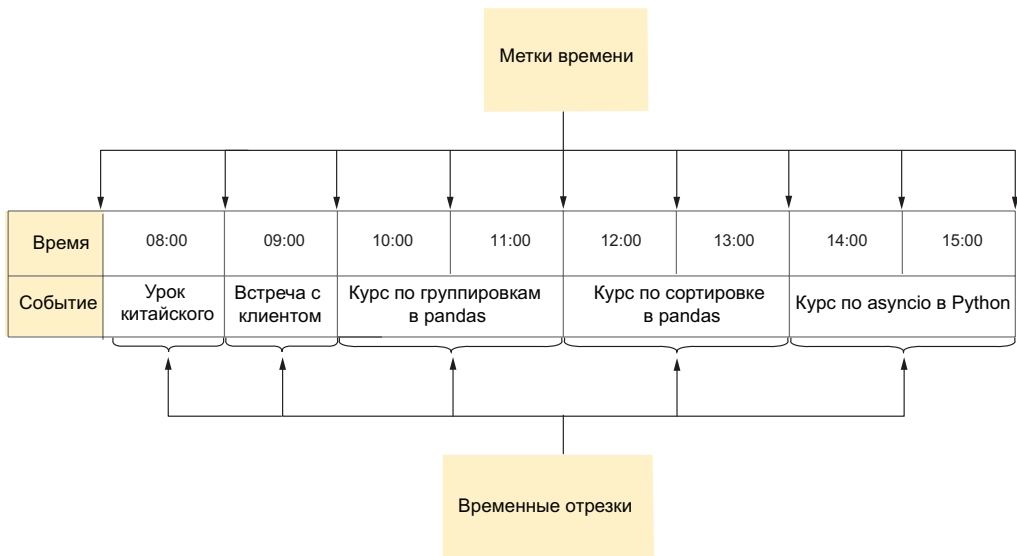


Рис. 10.1. Мое расписание в метках времени и временных отрезках

В табл. 10.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 10.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>pd.to_datetime</code>	При передаче в качестве аргумента объекта Series, состоящего из строк, возвращает новый объект Series с объектами типа <code>Timestamp</code>	<code>pd.to_datetime(s['when'])</code>	http://mng.bz/6D2D (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.to_datetime.html)

Таблица 10.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
pd.to_timedelta	При передаче в качестве аргумента объекта Series, состоящего из строк, возвращает новый объект Series с объектами типа Timedelta	pd.to_timedelta(s['how_long'])	http://mng.bz/o1Er (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.to_timedelta.html)
pd.read_csv	Возвращает новый датафрейм с данными на основе файла CSV	df = pd.read_csv('myfile.csv')	http://mng.bz/nW8g (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
time.strptime	Преобразовывает значение даты и времени в строку с определенным форматом	time.strptime(a_time, a_format)	http://mng.bz/vn5J (https://docs.python.org/3/library/time.html#time.strptime)
time.strptime	Преобразовывает строку с определенным форматом в значение даты и времени	time.strptime(time_string)	http://mng.bz/4D2a (https://docs.python.org/3/library/time.html#time.strptime)
df.to_csv	Записывает файл CSV на основе данных из датафрейма	df.to_csv('mydata.csv')	http://mng.bz/QPNw (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to_csv.html)
df.resample	Выполняет передискретизацию временных рядов, или группировку данных по временным интервалам	df.resample('1M')	http://mng.bz/XN6G (https://pandas.pydata.org/pandas-docs/stable/user_guide/timeseries.html#resampling)
s.diff	Возвращает новый объект Series на основе переменной s, в элементах которого содержится разница между текущим значением и предыдущим	s.diff()	http://mng.bz/yQnG (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.diff.html)
s.pct_change	Возвращает новый объект Series на основе переменной s, в элементах которого содержится разница в процентах между текущим значением и предыдущим	s.pct_change()	http://mng.bz/MBj7 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.pct_change.html)

Создание объектов `datetime` и `timedelta`

Как мы уже не раз видели, `pandas` зачастую отстраняется от структур данных, используемых в Python, в пользу собственных типов или типов, применяемых в NumPy. Это касается и области работы с датами и временем. К примеру, для представления конкретной точки во времени здесь используется класс `Timestamp`, а не стандартный для Python класс `datetime.datetime` или поставляемый с NumPy класс `np.datetime64`.

Обычно объекты типа `Timestamp` создаются при помощи функции `to_datetime`, которая может принимать на вход аргументы разных типов. При передаче функции одного аргумента она возвращает один объект типа `Timestamp`. Например, мы можем получить текущую дату и время, передав функции строку `'now'`, как показано ниже:

```
pd.to_datetime('now')
```

Но гораздо чаще функция `pd.to_datetime` вызывается применительно к существующему объекту `Series` со строками, представляющими информацию о дате и времени.

Пример:

```
s = Series(['1970-07-14', '1972-03-01', '2000-12-16',
            '2002-12-17', '2005-10-31'])
pd.to_datetime(s)
```

В результате мы получим показанный ниже объект `Series`:

```
0    1970-07-14
1    1972-03-01
2    2000-12-16
3    2002-12-17
4    2005-10-31
dtype: datetime64[ns]
```

Пусть вас не смущает показанный внизу тип данных `datetime64`, представляющий собой тип из NumPy. Все значения на самом деле обладают типом данных `Timestamp` из `pandas`. В представленном выше примере мы передали функции `to_datetime` недвусмысленные значения, которые легко разбирать. А что, если передать ей значения в другом формате, например с месяцами, указанными текстом?

```
s = Series(['1970-Jul-14', '1972-Mar-01', '2000-Dec-16',
            '2002-Dec-17', '2005-Oct-31'])
pd.to_datetime(s)
```

И в этом случае все сработало правильно. Причина в достаточной гибкости функции `pd.to_datetime`, которая умеет читать и преобразовывать различные форматы дат. Так что здесь никаких проблем не возникло, как не возникнет и в следующем примере:

```
s = Series(['14-Jul-1970', '01-Mar-1972', '16-Dec-2000',
            '17-Dec-2002', '31-Oct-2005'])
pd.to_datetime(s)
```

А что, если передать функции даты в более двусмысленном виде? К примеру, с месяцами в виде чисел, как показано ниже:

```
s = Series(['14-07-1970', '01-03-1972', '16-12-2000',
           '17-12-2002', '31-10-2005'])
pd.to_datetime(s)
```

И снова никаких вопросов. Но иногда формат дат может быть связан с некоторыми региональными и культурными особенностями. Возьмем, к примеру, следующие примеры дат:

```
s = Series(['01/03/1972', '05/12/1995'])
pd.to_datetime(s)
```

Как pandas воспримет эти даты? Как 1 марта или 3 января? 5 декабря или 12 мая? По умолчанию в pandas принят формат следования сначала номера месяца, а затем – дня, как в США. Но вы можете переопределить это поведение, передав функции `pd.to_datetime` параметр `dayfirst=False`, как показано ниже:

```
s = Series(['01/03/1972', '05/12/1995'])
pd.to_datetime(s, dayfirst=False)
```

До этого мы передавали функции `pd.to_datetime` только даты. Но мы также можем передавать ей и время, например так:

```
s = Series(['1970-07-14 8:00:00', '1972-03-01 10:00:00',
           '2000-12-16 12:15:28', '2002-12-17 18:17:00', '2005-10-31 23:51:00'])
print(pd.to_datetime(s))
```

Вывод:

```
0    1970-07-14 08:00:00
1    1972-03-01 10:00:00
2    2000-12-16 12:15:28
3    2002-12-17 18:17:00
4    2005-10-31 23:51:00
dtype: datetime64[ns]
```

Если вы хотите для каких-то меток времени не указывать секунды или добавить специальные указатели типа АМ или РМ, вы можете воспользоваться параметром `format='mixed'` в функции, как показано ниже:

```
s = Series(['1970-07-14 8:00', '1972-03-01 10:00 am',
           '2000-12-16 12:15:28', '2002-12-17 18:17', '2005-10-31 23:51'])
print(pd.to_datetime(s, format='mixed'))
```

Вывод:

```
0    1970-07-14 08:00:00
1    1972-03-01 10:00:00
2    2000-12-16 12:15:28
3    2002-12-17 18:17:00
4    2005-10-31 23:51:00
dtype: datetime64[ns]
```

А что, если годы, месяцы и дни у нас представлены в разных объектах Series? Мы можем воспользоваться функцией `pd.to_datetime` для получения нового объекта `Timestamp`

на основе этих данных. Это бывает очень полезно при необходимости добавить в датафрейм столбец с данными в формате Timestamp на основе других столбцов:

```
df = DataFrame([s.split('-')
                 for s in ['14-07-1970', '01-03-1971',
                           '16-12-2000', '17-12-2002',
                           '31-10-2005']],
               columns='day month year'.split())
df['our_date'] = pd.to_datetime(df[['year', 'month', 'day']])
```

В результате мы получим такой датафрейм:

	day	month	year	our_date
0	14	07	1970	1970-07-14
1	01	03	1971	1971-03-01
2	16	12	2000	2000-12-16
3	17	12	2002	2002-12-17
4	31	10	2005	2005-10-31

Все это хорошо, но нам как-то нужно при загрузке данных из файла CSV сразу идентифицировать столбцы с датами и временем, чтобы для них устанавливался тип Timestamp. Это можно легко сделать с помощью параметра `parse_dates` в функции `read_csv`. При этом мы можем передать этому параметру как имена столбцов, так и целочисленные индексы. Пример:

```
pd.read_csv(filename,
             parse_dates=['birthday', 'anniversary'])
```

Существует несколько параметров, с помощью которых можно влиять на процесс разбора дат. Один из таких параметров – это `dayfirst`, который позволяет указать, что первым компонентом идет день, а затем – месяц, как принято в Европе.

После приведения объекта Series к типу данных Timestamp мы можем свободно пользоваться атрибутом доступа `dt` для извлечения различных составляющих даты и времени. Примеры показаны ниже:

<code>s.dt.month</code>	# номер месяца
<code>s.dt.month_name</code>	# название месяца
<code>s.dt.hour</code>	# час
<code>s.dt.day_of_week</code>	# день недели
<code>s.dt.is_leap_year</code>	# признак високосного года

Некоторые из этих атрибутов возвращают целочисленные значения, другие – логические. Документация атрибута `dt` располагается по адресу <http://mng.bz/j1wa> (<https://pandas.pydata.org/docs/reference/api/pandas.Series.dt.date.html>).

В начале главы я упомянул, что для работы с датами в pandas предназначены два типа данных. Выше мы уже познакомились с типом Timestamp, а теперь узнаем, что из себя представляют временные интервалы. В общем смысле можно пользоваться следующими формулами:

```
datetime - datetime = interval
datetime + interval = datetime
datetime - interval = datetime
```


Иными словами, если у вас есть два объекта с датой и временем, вы можете вычесть один из другого и получить временной интервал, т. е. количество времени между этими двумя метками. К примеру, зная день рождения человека и день его смерти, мы можем рассчитать продолжительность его жизни. С другой стороны, если у нас есть метка времени и заданный интервал, мы можем получить новую метку времени, прибавив или отняв этот интервал от первой даты. Например, зная время встречи и ее продолжительность, мы можем легко узнать время окончания встречи. И наоборот, зная время окончания встречи и ее продолжительность, можно вычислить время ее начала.

Pandas позволяет выполнять подобные вычисления. Если у вас есть два объекта Series с датами и временем, вычитание одного из другого даст вам объект Series с типом `timedelta`. Пример:

```
s = Series(['1970-07-14 8:00', '1972-03-01 10:00 pm',
            '2000-12-16 12:15:28', '2002-12-17 18:17',
            '2005-10-31 23:51'])
s = pd.to_datetime(s, format='mixed')
pd.to_datetime('2021-July-01') - s
```

Вывод будет следующим, а принцип выполнения этой операции показан на рис. 10.2:

```
0    18614 days 16:00:00
1    18018 days 02:00:00
2     7501 days 11:44:32
3     6770 days 05:43:00
4     5721 days 00:09:00
dtype: timedelta64[ns]
```

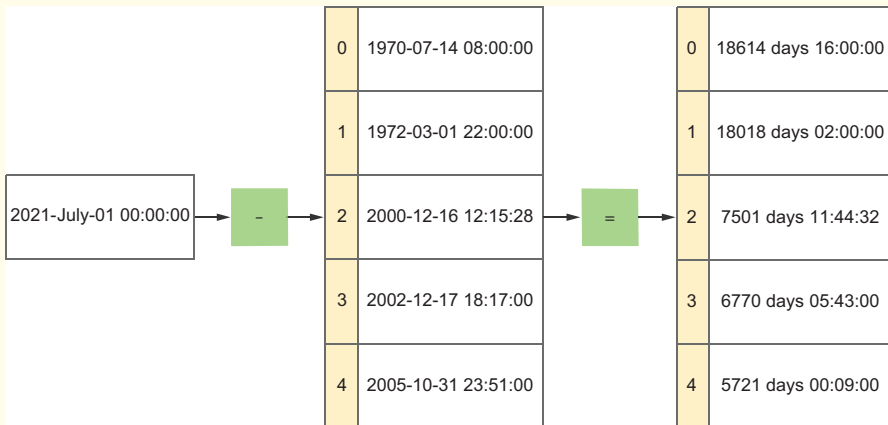


Рис. 10.2. Вычитание одной даты из другой дает временной интервал

Для создания объекта `timedelta` мы можем также воспользоваться функцией `pd.to_timedelta` подобно тому, как пользовались ранее функцией `pd.to_timestamp`. Обычно на вход этой функции передается временной интервал в виде строки, описывающей этот интервал, например `'1 hour'` или `'2 days'`.

Компоненты объекта `timedelta` можно извлечь с помощью атрибута `components`, как показано ниже:

```
pd.to_timedelta('2 days 3:20:10').components
```

Вывод:

```
Components(days=2, hours=3, minutes=20, seconds=10, milliseconds=0,
microseconds=0, nanoseconds=0)
```

Также при наличии объекта `timedelta` мы можем извлекать его атрибуты, такие как `days`, `seconds`, `microseconds` и `nanoseconds`, напрямую.

Теперь, когда вы имеете общее представление о том, как создавать объекты `timestamp` и `timedelta` и извлекать из них информацию, пришло время закрепить полученные знания на практике, превратив их в навыки.

УПРАЖНЕНИЕ 39. Короткие, средние и длинные поездки на такси

Мы уже не раз обращались к набору данных о поездках на нью-йоркских такси, а в упражнении 30 даже рассмотрели тему, связанную с короткими, средними и продолжительными поездками. Однако в том упражнении мы оперировали лишь преодоленной водителем дистанцией. В этом упражнении мы взглянем на эту картину с другой стороны, а именно со стороны времени, проведенного пассажирами в такси. Вы знаете, что в больших городах можно провести немало времени в машине, так и не сдвинувшись с места.

Итак, что вам нужно будет сделать.

1. Загрузить данные из файла `nyc_taxi_2019-07.csv` в датафрейм с использованием только столбцов `trip_pickup_datetime`, `trip_dropoff_datetime`, `passenger_count`, `trip_distance` и `total_amount`. Укажите, что столбцы `trip_pickup_datetime` и `trip_dropoff_datetime` должны быть загружены с типом `datetime`.
2. Создать новый столбец с именем `trip_time` с типом `timedelta`, в котором будет содержаться продолжительность поездки.
3. Определить количество и долю поездок, которые длились меньше одной минуты.
4. Рассчитать среднюю стоимость таких коротких поездок.
5. Определить количество и долю поездок, которые длились больше 10 ч.
6. Создать новый столбец с именем `trip_time_group` с категориальными значениями `short` (< 10 мин), `medium` (от 10 мин до 1 ч) и `long` (> 1 ч).
7. Определить долю поездок из каждой группы.
8. Для каждой группы из столбца `trip_time_group` рассчитать среднее количество пассажиров.

Подробный разбор

Это упражнение начинается так же, как и все предыдущие, которые были связаны с набором данных с поездками на такси. Но если раньше мы полагались

на интеллект библиотеки pandas при определении значений атрибута dtype для столбцов, то теперь нам необходимо явным образом задать для двух столбцов тип данных Timestamp. Конечно, мы могли бы загрузить их изначально в виде текста, а затем преобразовать в даты и время с помощью функции `pd.to_timestamp`, но это можно сделать проще, как показано ниже:

```
filename = '../data/nyc_taxi_2019-07.csv'
df = (
    pd
    .read_csv(filename,
               usecols=['tpep_pickup_datetime',
                       'tpep_dropoff_datetime',
                       'trip_distance',
                       'passenger_count',
                       'total_amount'],
               parse_dates=['tpep_pickup_datetime',
                           'tpep_dropoff_datetime'])
)
```

Обратите внимание, что имена двух столбцов (`tpep_pickup_datetime` и `tpep_dropoff_datetime`) указаны в обоих параметрах: `usecols` и `parse_dates`. Данные будут успешно загружены, поскольку даты и время в этом наборе данных указаны в недвусмысленном формате YYYY-MM-DD.

Мы можем убедиться, что столбцы были загружены корректно, воспользовавшись свойством `dtypes`. Результат, показанный ниже, полностью соответствует нашим ожиданиям:

```
tpep_pickup_datetime    datetime64[ns]
tpep_dropoff_datetime   datetime64[ns]
passenger_count         float64
trip_distance           float64
total_amount            float64
dtype: object
```

Если бы мы не перечислили две эти колонки в параметре `parse_dates`, они получили бы тип `object`, которым в Python в основном обозначаются строковые данные.

Загрузив столбцы в датафрейм в виде даты и времени, мы можем создать новый столбец с именем `trip_time` и типом данных `timedelta` с помощью вычитания времени посадки пассажиров из времени высадки, как показано ниже:

```
df['trip_time'] = (
    df['tpep_dropoff_datetime'] -
    df['tpep_pickup_datetime']
)
```

Теперь мы можем спокойно начать отвечать на поставленные вопросы. Сначала нас попросили определить количество и долю поездок, которые длились меньше одной минуты.

Для ответа на этот вопрос нам придется выполнить операцию сравнения со столбцом `trip_time`. Мы могли бы явно создать объект `timedelta`, воспользовавшись функцией `pd.to_timedelta`, но оказывается, что разработчики pandas обо

всем позаботились заранее и позволили нам производить операции прямо со строками, выполняя все необходимые преобразования под капотом:

```
df['trip_time'] < '1 minute'
```

В результате мы получим объект Series, заполненный булевыми значениями, где значения True будут соответствовать поездкам, которые длились меньше одной минуты. Получить эти поездки можно, как и всегда, при помощи атрибута `df.loc`. А заодно можно и подсчитать их количество:

```
df.loc[
    df['trip_time'] < '1 minute',
    'trip_time'
].count()
```

Мы получили 70 212 поездок короче одной минуты. Странно, что их так много. Может, Нью-Йорк не такое уж и «большое яблоко»? Давайте узнаем, какую долю от всех поездок составили эти микропрогулки на такси. Для этого разделим полученное количество на общее количество поездок и умножим результат на 100:

```
df.loc[
    df['trip_time'] < '1 minute',
    'trip_time'
].count() / df['trip_time'].count() * 100
```

Мы получили 1.11%. Мне кажется, многовато. Но, быть может, в Нью-Йорке есть традиция проезжать на такси один-два квартала, я не знаю.

Интересно, а сколько в среднем стоят такие короткие поездки? Чтобы посчитать это, необходимо применить нашу маску, выбрать столбец `total_amount` и вызвать метод `mean`, как показано ниже и на рис. 10.3:

```
df.loc[
    df['trip_time'] < '1 minute',
    'total_amount'
].mean()
```

Получилось чуть больше 30 долл.! Проехать так мало и заплатить 30 долл.? Это какой-то неизвестный мне вид роскоши!

Далее нас попросили в подобном же ключе проанализировать поездки, которые длились более 10 ч. Я не могу представить себе, что можно столько времени делать в такси в Нью-Йорке, да и в любом другом городе, но мне и правда интересно, много ли было совершено таких поездок в июле 2019 года.

Снова сравним столбец `trip_time` со строкой, характеризующей временной интервал:

```
df['trip_time'] > '10 hours'
```

Применим нашу маску и подсчитаем общее количество поездок:

```
df.loc[df['trip_time'] > '10 hours', 'trip_time'].count()
```

Таких длинных поездок нашлось в базе вчетверо меньше, чем экстремально коротких, а именно 16 698. Какую долю они составили в общей массе?

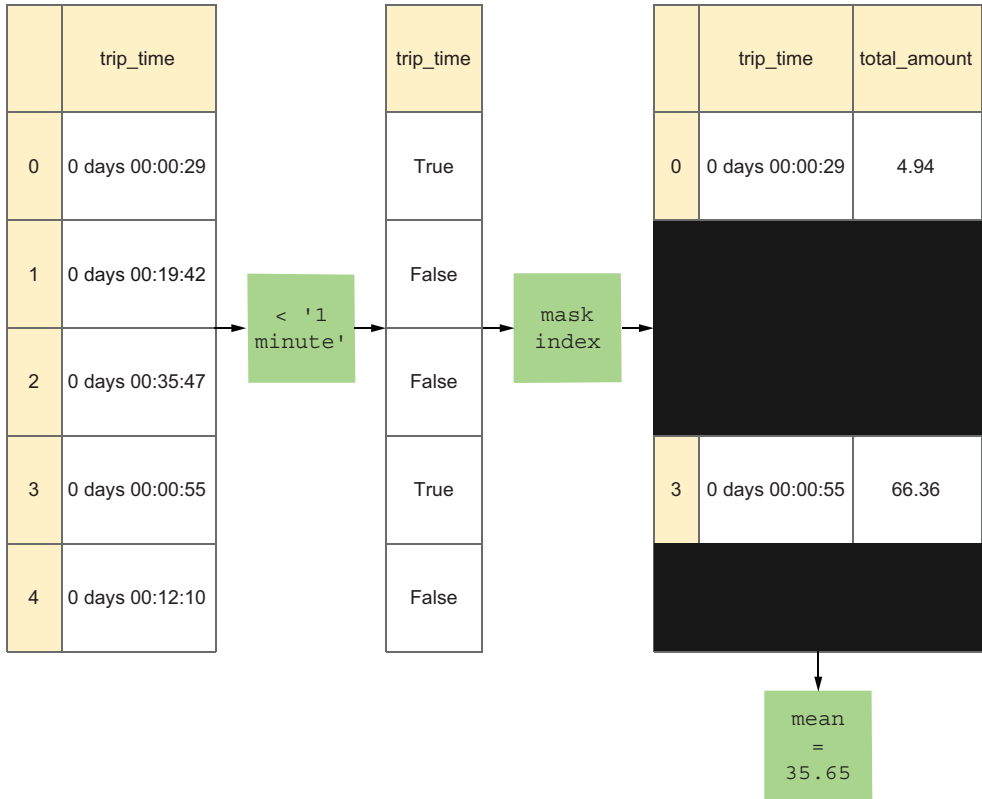


Рис. 10.3. Получение средней стоимости поездок, длившихся менее одной минуты

```
df.loc[df[
    'trip_time'] > '10 hours',
    'trip_time'].count() / df['trip_time'].count() * 100
```

Получилось около 0.2 %. Но все равно интересно, куда же ездят два человека из тысячи по десять часов...

Далее нас попросили сгруппировать поездки в три категории: `short`, `medium` и `long`. Для этого мы воспользуемся уже знакомой нам функцией `pd.cut` и передадим ей параметр `bins` со значениями, сравнимыми с содержимым нашей колонки с продолжительностью поездок.

Наши точки разрывов для категорий – это 10 мин и 1 ч. Таким образом, к категории `short` мы будем относить поездки продолжительностью менее 10 мин, к категории `medium` – от 10 мин до 1 ч, а к категории `long` – более 1 ч. Однако функция `pd.cut` не столь либеральна и не позволит нам использовать для сравнения строки. Так что нам придется создать список (или объект `Series`) со значениями типа `timedelta`. Сделаем это с помощью генератора списков, как показано ниже:

```
[pd.to_timedelta(arg)
 for arg in ['0 seconds', '10 minutes',
            '1 hour', '100 hours']]
```

Если говорить коротко, этот генератор списков делает следующее.

1. Проходит по переданному ему списку строк.
2. Приводит каждый элемент к типу данных `timedelta`.
3. Возвращает список из четырех значений типа `timedelta` на основе исходного списка.

Полученный список можно передать функции `pd.cut` следующим образом:

```
df['trip_time_group'] = (
    pd.cut(
        df['trip_time'],
        bins=[pd.to_timedelta(arg)
              for arg in ['0 seconds',
                          '10 minutes',
                          '1 hour',
                          '100 hours']],
        labels=['short', 'medium', 'long'])
)
```

Обратите внимание, что для получения трех меток нам необходимо передать четыре отсечки в параметр `bins`. И хотя имена меток можно и не передавать, лучше это сделать. В результате вызова функции `pd.cut` мы получим новый объект `Series`, который впоследствии присвоим столбцу `df['trip_time_group']`.

Теперь, когда у нас есть столбец с категориями поездок, мы можем применить метод `groupby`, чтобы определить, есть ли существенные различия между разными категориями в отношении среднего количества пассажиров:

```
df.groupby('trip_time_group')['passenger_count'].mean()
```

Вывод:

```
trip_time_group
short      1.552411
medium     1.585806
long       1.700859
Name: passenger_count, dtype: float64
```

Как видите, для коротких и средних поездок среднее количество пассажиров приблизительно равно 1.5, тогда как для продолжительных средний показатель увеличился до 1.7 пассажиров. Это не такая большая разница, хотя определенная тенденция прослеживается: 1.55, 1.58, 1.70.

Решение

```
filename = '../data/nyc_taxi_2019-07.csv'
```

```
df = pd.read_csv(filename,
                  usecols=['tpep_pickup_datetime',
                           'tpep_dropoff_datetime',
                           'trip_distance', 'passenger_count',
                           'total_amount'],
                  parse_dates=['tpep_pickup_datetime',
```

```

        'tpep_dropoff_datetime'])

df['trip_time'] = df['tpep_dropoff_datetime']
    - df['tpep_pickup_datetime'] ❷

df.loc[df['trip_time'] < '1 minute', 'trip_time'].count() ❸
df.loc[df['trip_time'] < '1 minute', 'trip_time'
    ].count() / df['trip_time'].count() * 100 ❹
df.loc[df['trip_time'] < '1 minute', 'total_amount'].mean() ❺

df.loc[df['trip_time'] > '10 hours', 'trip_time'].count() ❻
df.loc[df['trip_time'] > '10 hours', 'trip_time'
    ].count() / df['trip_time'].count() * 100 ❼

df['trip_time_group'] = (
    pd.cut(
        df['trip_time'],
        bins=[pd.to_timedelta(arg)
            for arg in ['0 seconds',
                '10 minutes',
                '1 hour',
                '100 hours']], ❸
        labels=['short', 'medium', 'long']) ❹
)

df.groupby('trip_time_group')['passenger_count'].mean() ❺

```

- ❶ Во время загрузки определяем два столбца как дату и время.
- ❷ Вычитая одну метку времени из другой, мы получаем временной отрезок.
- ❸ Считаем поездки продолжительностью менее 1 мин.
- ❹ Считаем долю поездок продолжительностью менее 1 мин.
- ❺ Применяем булев индекс, извлекаем стоимость и берем среднее.
- ❻ Считаем поездки продолжительностью более 10 ч.
- ❼ Считаем долю поездок продолжительностью более 10 ч.
- ❸ Используем генератор списков совместно с функцией `pd.to_timedelta` для создания временных отрезков.
- ❹ Присваиваем три метки нашим отрезкам
- ❺ Считаем среднее количество пассажиров для разных категорий поездок

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/W1og>.

Дополнительные упражнения

- В этом упражнении мы загрузили данные о поездках в такси за июль 2019 года. А сколько поездок в этом наборе не принадлежит этому временному отрезку? Иными словами, сколько в наборе данных ошибочных дат?
- Вычислите среднее время поездки для каждого числа пассажиров.
- Загрузите данные за июль 2019 и 2020 годов и объедините их. Найдите средние стоимости поездок с группировкой по годам, а затем по количеству пассажиров.

Ответы на дополнительные упражнения

Упражнение 39.1

Помните, что без указания времени считается, что вы подразумевали полночь

```
len(df[(df['tpep_pickup_datetime'] < '2019-07-01') |
      (df['tpep_pickup_datetime'] > '2019-07-31 23:59')].index)
```

Вывод:

387

Упражнение 39.2

```
df['trip_time'] = df['tpep_dropoff_datetime'] - df['tpep_pickup_datetime']
df.groupby('passenger_count')['trip_time'].mean()
```

Вывод:

```
passenger_count
0.0    0 days 00:14:18.929810752
1.0    0 days 00:17:46.148103924
2.0    0 days 00:18:34.024342704
3.0    0 days 00:19:02.079604271
4.0    0 days 00:20:10.057290100
5.0    0 days 00:22:29.870464324
6.0    0 days 00:20:54.109564300
7.0    0 days 00:16:38.206896551
8.0    0 days 00:11:00.500000
9.0    0 days 00:49:16.125000
Name: trip_time, dtype: timedelta64[ns]
```

Упражнение 39.3

```
all_filenames = ['./data/nyc_taxi_2019-07.csv',
                 './data/nyc_taxi_2020-07.csv']

all_dfs = [pd.read_csv(one_filename,
                       usecols=['tpep_pickup_datetime',
                                'tpep_dropoff_datetime',
                                'trip_distance', 'passenger_count', 'total_amount'],
                       parse_dates=['tpep_pickup_datetime', 'tpep_dropoff_datetime'])
            for one_filename in all_filenames]

df = pd.concat(all_dfs)
df.head()
```

Вывод:

	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	total_amount
0	2019-07-01 00:51:04	2019-07-01 00:51:33	1.0	0.00	4.94
1	2019-07-01 00:46:04	2019-07-01 01:05:46	1.0	4.16	20.30
2	2019-07-01 00:25:09	2019-07-01 01:00:56	1.0	18.80	70.67

3	2019-07-01 00:33:32	2019-07-01 01:15:27	1.0	18.46	66.36
4	2019-07-01 00:00:55	2019-07-01 00:13:05	0.0	1.70	15.30

```
df.groupby([df['tpep_pickup_datetime'].dt.year, 'passenger_count'])['total_amount'].mean()
```

Вывод:

tpep_pickup_datetime	passenger_count	
2002	1.0	18.002500
	2.0	18.800000
	5.0	11.966667
2008	1.0	18.340000
	2.0	42.860000
	5.0	11.966667
2020	5.0	16.725836
	6.0	16.812911
	7.0	22.456000
	8.0	10.300000
	9.0	11.760000
	...	

Name: total_amount, Length: 28, dtype: float64

УПРАЖНЕНИЕ 40. Пишем и читаем даты

В предыдущем упражнении мы увидели, как легко можно читать данные из файлов CSV в pandas, даже если в них содержатся колонки с датой и временем. В общем случае нам просто необходимо перечислить в параметре `parse_dates` имена колонок, которые мы хотели бы пропустить через функцию `pd.to_datetime` при создании датафрейма, и больше ни о чем можно не думать. Но время от времени приходится сталкиваться с нестандартными форматами дат и времени. При этом нам может понадобиться как записывать даты в необычном формате, так и (гораздо чаще) читать даты в форматах, не распознаваемых pandas.

К счастью, у нас есть ряд инструментов для контроля за чтением и записью дат на диск. В этом упражнении мы попрактикуемся в этой области.

Что вам нужно сделать.

1. Загрузить данные из файла `nyc_taxi_2019-07.csv` в датафрейм с использованием только столбцов `tpep_pickup_datetime`, `passenger_count`, `trip_distance` и `total_amount`. Укажите, что столбец `tpep_pickup_datetime` должен быть загружен с типом `datetime`.
2. Сохранить загруженный датафрейм в файл CSV с символами табуляции в качестве разделителей. При этом даты должны быть сохранены в формате `day/month/year HHh:MMm:SSs`. Это означает, что:
 - день должен быть представлен двумя цифрами;
 - месяц должен быть представлен двумя цифрами;
 - год должен быть представлен четырьмя цифрами;
 - часы должны быть представлены двумя цифрами в формате 24 ч, следом за которыми должна идти буква `h`;

- минуты должны быть представлены двумя цифрами, следом за которыми должна идти буква m;
- секунды должны быть представлены двумя цифрами, следом за которыми должна идти буква s;
- прочитайте данные из записанного вами файла CSV и убедитесь, что столбец с датами и временем загрузился и отображается правильно.

В таком необычном формате дата 3 февраля 2023 года, 10:11:12 должна выглядеть так:

```
03/02/23 10h:11m:12s
```

Подробный разбор

В этом упражнении вы потренируетесь читать и записывать файлы CSV с использованием альтернативных форматов даты. В большинстве случаев аналитики данных работают с источниками, в которых даты представлены в формате, удобном для чтения в pandas. Но изредка приходится загружать данные из разных логов, создаваемых сторонними программами, обладающими собственным представлением о форматах даты и времени. Хорошая новость заключается в том, что мы довольно легко можем использовать пользовательские форматы при работе с файлами CSV.

Мне как-то пришлось воспользоваться pandas для перевода дат из одного формата в другой. Иными словами, я применил его не для анализа данных, а для преобразования файлов. Да, может, я стрельнул из пушки по воробьям, но задача была выполнена довольно быстро и качественно.

Начнем упражнение, как обычно, с загрузки данных из файла. Для преобразования столбца `tpep_pickup_datetime` в формат `datetime` укажем его в параметре `parse_dates` функции `read_csv`:

```
filename = '../data/nyc_taxi_2019-07.csv'
df = pd.read_csv(
    filename,
    usecols=['tpep_pickup_datetime',
            'trip_distance',
            'passenger_count',
            'total_amount'],
    parse_dates=['tpep_pickup_datetime'])
```

Теперь можно попробовать экспортировать наши данные в другой файл CSV, но с измененным форматом дат. Мы могли бы создать новый столбец с нужным нам форматом и затем выгрузить его в новый файл. Но разработчики pandas в очередной раз подумали за нас и предусмотрели в методе `to_csv`, служащем для сохранения данных из датафрейма в файл, специальный параметр с именем `date_format`.

Формат в этом случае указывается при помощи символов % и соответствует формату, применяемому в методе `time.strftime`. Полное описание всех возможных компонентов можно найти в документации по адресу <http://mng.bz/84DK> (<https://docs.python.org/3/library/time.html#time.strftime>). Вывод может содержать любую комбинацию элементов, представляющих часы, минуты, месяцы, дни, часовые пояса и пр. Тре-

буемый формат для этого упражнения предполагает наличие дня и месяца в формате двух знаков, года в формате четырех знаков, а также времени со следующими за отдельными компонентами суффиксами. Мы можем записать этот формат следующим образом:

```
'%d/%m/%Y %Hh:%Mm:%Ss'
```

Тогда код для записи данных в новый файл CSV с использованием измененного формата дат может выглядеть так:

```
df.to_csv('ex40_taxi_07_2019.csv',
         sep='\t',
         columns=['tpep_pickup_datetime', 'passenger_count',
                 'trip_distance', 'total_amount'],
         date_format='%d/%m/%Y %Hh:%Mm:%Ss')
```

Здесь мы указали имя нового файла `ex40_taxi_07_2019.csv` и задали в качестве разделителя (параметр `sep`) символ табуляции. Параметр `date_format` применяется для того, чтобы можно было задать определенный формат записи колонок с датами и временем (в нашем случае это одна колонка с именем `tpep_pickup_datetime`).

Если мы собираемся использовать этот формат сразу в нескольких местах в программе, можно сохранить его в переменной `dt_format`. Впоследствии мы сможем обращаться к этой переменной как в методе `df.to_csv`, так и в наших собственных функциях. С использованием переменной код будет выглядеть так:

```
dt_format='%d/%m/%Y %Hh:%Mm:%Ss'

df.to_csv('ex40_taxi_07_2019.csv',
         sep='\t',
         columns=['tpep_pickup_datetime',
                 'passenger_count',
                 'trip_distance',
                 'total_amount'],
         date_format=dt_format)
```

В заключение нас попросили прочитать данные из записанного файла и убедиться, что столбец `tpep_pickup_datetime` имеет тип `datetime`, несмотря на хранение в странном формате.

ПРИМЕЧАНИЕ. В версиях `pandas` до 2.0 при чтении файлов CSV с необычными форматами дат необходимо было при вызове функции `pd.read_csv` передавать специальную функцию преобразования с помощью параметра `date_parser`. Но в последних версиях `pandas` достаточно указать формат в параметре `date_format` аналогично методу `to_csv`.

Итак, воспользуемся функцией `df.read_csv` и передадим ей созданную ранее переменную `dt_format` в качестве параметра `date_format`, как показано ниже:

```
df = pd.read_csv('ex40_taxi_07_2019.csv',
                sep='\t',
                usecols=['tpep_pickup_datetime',
                        'passenger_count',
```

```
        'trip_distance',
        'total_amount'],
    parse_dates=['tpep_pickup_datetime'],
    date_format=dt_format)
```

Решение

```
filename = '../data/nyc_taxi_2019-07.csv'
df = pd.read_csv(
    filename,
    usecols=['tpep_pickup_datetime',
             'trip_distance',
             'passenger_count',
             'total_amount'],
    parse_dates=['tpep_pickup_datetime'])

dt_format='%d/%m/%Y %Hh:%Mm:%Ss'

df.to_csv('ex40_taxi_07_2019.csv',
          sep='\t',
          columns=['tpep_pickup_datetime',
                  'passenger_count',
                  'trip_distance',
                  'total_amount'],
          date_format=dt_format)

import time
def parse_weird_format(s):
    return pd.to_datetime(s, format=dt_format)

df = pd.read_csv('ex40_taxi_07_2019.csv',
                  sep='\t',
                  usecols=['tpep_pickup_datetime',
                           'passenger_count',
                           'trip_distance',
                           'total_amount'],
                  parse_dates=['tpep_pickup_datetime'],
                  date_format=dt_format)
```

- ❶ Читаем данные из файла CSV, включая столбец с датами.
- ❷ Указываем формат для чтения и записи.
- ❸ Пишем файл CSV.
- ❹ Функция принимает строку и возвращает объект datetime.
- ❺ Читаем сохраненный файл CSV, используя наш формат.
- ❻ Этот параметр ожидает строку форматирования.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/E9Mq>.

Дополнительные упражнения

1. Выгрузите столбец `trp_pickup_datetime` в формате времени Unix, т. е. в виде количества полных секунд, прошедших с 1 января 1970 года.
2. Прочитайте в датафрейм набор данных из этого упражнения, но на этот раз столбец `trp_pickup_datetime` пусть будет текстовым. Затем воспользуйтесь функцией `pd.to_datetime` для преобразования его в даты и время.
3. Сравните скорость преобразования столбца в формат даты в функциях `read_csv` и `to_datetime`.

Временные ряды

Мы уже видели, что индекс в датафрейме может быть представлен целочисленными значениями или строками. Но все становится куда интереснее, когда в качестве индекса используется столбец с датами и временем. В `pandas` это называется *временными рядами* (time series). В работе с временными рядами мы можем воспользоваться целым рядом продвинутых техник и приемов.

Для начала давайте создадим временной ряд. Посвятим его миссиям космической программы NASA Аполлон (Apollo), информацию о которых возьмем из «Википедии»:

```
all_dfs = pd.read_html('https://en.wikipedia.org/wiki/Apollo_program')
df = all_dfs[2].copy()[['Date', 'Designation']]
```

Вывод:

	Date	Designation
0	Feb 26, 1966	AS-201
1	Jul 5, 1966	AS-203
2	Aug 25, 1966	AS-202
3	Feb 21, 1967	Apollo 1
4	Nov 9, 1967	Apollo 4
..	...	...
13	Apr 11-17, 1970	Apollo 13
14	Jan 31 - Feb 9, 1971	Apollo 14
15	Jul 26 - Aug 7, 1971	Apollo 15
16	Apr 16-27, 1972	Apollo 16
17	Dec 7-19, 1972	Apollo 17

[18 rows x 2 columns]

На рис. 10.4 показано, как выглядит страница в «Википедии» с таблицей, из которой мы хотим извлекать данные.

Некоторые даты в таблице указывают на конкретный день (например, Jul 5, 1966), другие захватывают целые интервалы (Jan 22-23, 1968). Мы будем удалять даты окончания интервалов, где они встречаются, чтобы остались только даты старта миссии:

```
df['Date'] = pd.to_datetime(df['Date'].str.replace(
    '(-.+)?, ', ' ', regex=True))
```

Contents [hide] (Top) > Background > NASA expansion - Choosing a mission mode > Spacecraft > Launch vehicles - Astronauts > Lunar mission profile > Development history - Mission summary - Samples returned - Costs - Apollo Applications Program - Recent observations > Legacy > Depictions on film - See also > References - Further reading > External links					
Main article: Canceled Apollo missions Several missions were planned for but were canceled before details were finalized. Mission summary [edit] For a more comprehensive list, see List of Apollo missions.					
Designation	Date	Launch vehicle	CSM	LM	Crew
AS-201	Feb 26, 1966	AS-201	CSM-009	None	None
AS-203	Jul 5, 1966	AS-203	None	None	None
AS-202	Aug 25, 1966	AS-202	CSM-011	None	None
AS-204 (Apollo 1)	Feb 21, 1967	AS-204	CSM-012	None	Gus Grissom Ed White Roger B. Chaffee
Apollo 4	Nov 9, 1967	AS-501	CSM-017	LTA-10R	None
Apollo 5	Jan 22–23, 1968	AS-204	None	LM-1	None
Apollo 6	Apr 4, 1968	AS-502	CM-020 SM-014	LTA-2R	None
Apollo 7	Oct 11–22, 1968	AS-205	CSM-101	None	Wally Schirra Walt Cunningham Donn Eisele
Apollo 8	Dec 21–27, 1968	AS-503	CSM-103	LTA-B	Frank Borman James Lovell William Anders
Apollo 9	Mar 3–13, 1969	AS-504	CSM-104 <i>Gumdrop</i>	LM-3 <i>Spider</i>	James McDivitt David Scott Russell Schweickart
Apollo 10	May 18–26, 1969	AS-505	CSM-106 <i>Charlie Brown</i>	LM-4 <i>Snoopy</i>	Thomas Stafford John Young Eugene Cernan
Apollo 11	Jul 16–24, 1969	AS-506	CSM-107 <i>Columbia</i>	LM-5 <i>Eagle</i>	Neil Armstrong Michael Collins Buzz Aldrin
Apollo 12	Nov 14–24, 1969	AS-507	CSM-108 <i>Yankee Clipper</i>	LM-6 <i>Intrepid</i>	C. "Pete" Conrad Richard Gordon Alan Bean
Apollo 13	Apr 11–17, 1970	AS-508	CSM-109	LM-7	James Lovell Jack Swigert

Рис. 10.4. Страница в «Википедии», посвященная миссиям космической программы Аполлон

Вывод:

	Date	Designation
0	1966-02-26	AS-201
1	1966-07-05	AS-203
2	1966-08-25	AS-202
3	1967-02-21	Apollo 1
4	1967-11-09	Apollo 4
..	...	...
13	1970-04-11	Apollo 13
14	1971-01-31	Apollo 14
15	1971-07-26	Apollo 15
16	1972-04-16	Apollo 16
17	1972-12-07	Apollo 17

[18 rows x 2 columns]

После этого установим столбец с датами в качестве индекса:

```
df = df.set_index('Date')
```

Начиная с этого момента наш датафрейм можно смело относить к категории временных рядов. Формальным признаком этого может быть вид индекса, показанный ниже:

```
DatetimeIndex(['1966-02-26', '1966-07-05', '1966-08-25',
                '1967-02-21', '1967-11-09', '1968-01-22',
                '1968-04-04', '1968-10-11', '1968-12-21',
                '1969-03-03', '1969-05-18', '1969-07-16',
                '1969-11-14', '1970-04-11', '1971-01-31',
                '1971-07-26', '1972-04-16', '1972-12-07'],
              dtype='datetime64[ns]', name='Date', freq=None)
```

В нашем индексе содержатся объекты `datetime`. Теперь мы можем извлечь строку за определенную дату, как в случае с обычным индексом:

```
df.loc['1970-04-11']
```

Но гораздо интереснее то, что мы можем спокойно указывать только определенные составляющие дат, и все будет прекрасно работать. К примеру, мы можем убрать день из нашей даты, что позволит извлечь строки за один конкретный месяц, как показано ниже:

```
df.loc['1970-07']
```

Или можно оставить только год, как показано на рис. 10.5:

```
df.loc['1971']
```

Также мы можем извлечь набор строк с помощью среза, указав начальную и конечную даты, как показано ниже:

```
df.loc['1968-07-01': '1972-08-31']
```

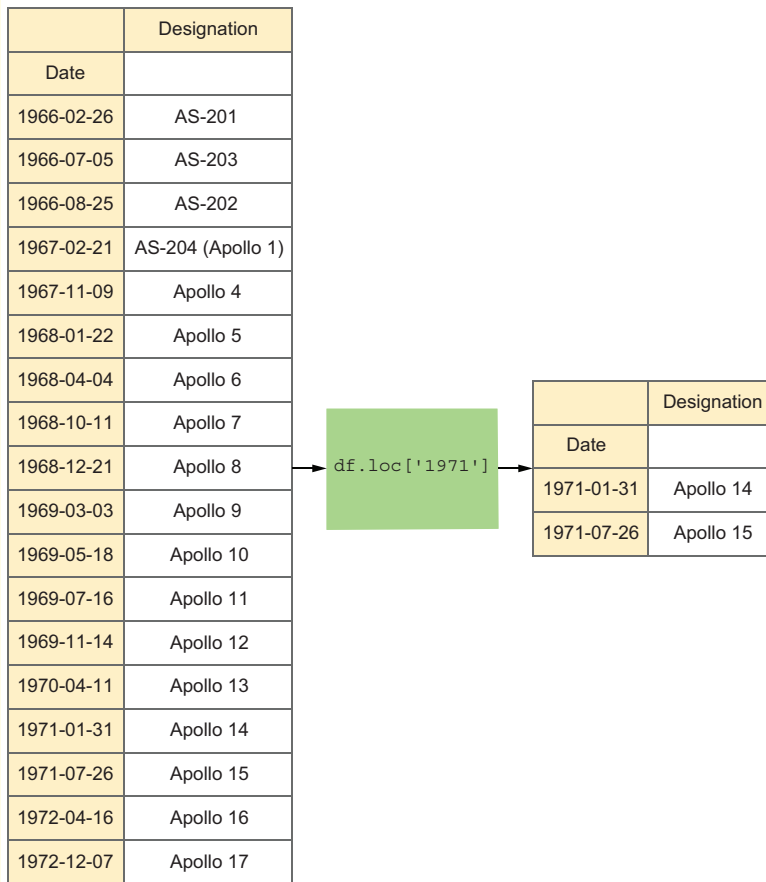


Рис. 10.5. Извлечение строк из временного ряда по году

Но, вероятно, одной из самых полезных возможностей, присущих временным рядам, является их *передискретизация* (*resampling*), или группировка данных по временным интервалам. Операция передискретизации аналогична применению метода `groupby`, за исключением того, что вместо получения агрегированных показателей для каждого уникального значения в столбце группировки мы можем вычислять агрегаты для заданных временных интервалов. Например, с помощью передискретизации можно рассчитать средние значения не только по дням, но также по произвольным отрезкам – двухнедельным периодам, кварталам, полугодиям, годам и т. д. Допустим, мы можем легко узнать, сколько миссий стартовало каждые шесть месяцев, следующим образом:

```
df.resample('6M').count()
```

Вывод:

```

      Designation
Date
1966-02-28      1

```


1966-08-31	2
1967-02-28	1
1967-08-31	0
1968-02-29	2
...	...
1971-02-28	1
1971-08-31	1
1972-02-29	0
1972-08-31	1
1973-02-28	1

[15 rows x 1 columns]

При работе с числовыми данными мы можем применять любые другие агрегирующие функции, такие как `mean` и `std`.

Ответы на дополнительные упражнения

Упражнение 40.1

```
df['tpep_pickup_datetime'] = df['tpep_pickup_datetime'].view(np.int64) / 10**9

df.to_csv('ex40b1_taxi_07_2019.csv',
         sep='\t',
         columns=['tpep_pickup_datetime', 'passenger_count', 'trip_distance',
                  'total_amount'])
```

Упражнение 40.2

```
df = pd.read_csv('ex40b1_taxi_07_2019.csv',
                sep='\t',
                usecols=['tpep_pickup_datetime', 'passenger_count', 'trip_distance',
                          'total_amount'])

df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'],
unit='s', origin='unix')
df.head()
```

Упражнение 40.3

```
import datetime

filename = '../data/nyc_taxi_2019-07.csv'

start = datetime.datetime.now()
print('Время скрапа (parse_dates): ' + str(start))

df = pd.read_csv(
    filename,
    usecols=['tpep_pickup_datetime',
```

```

        'trip_distance',
        'passenger_count',
        'total_amount'],
    parse_dates=['tpep_pickup_datetime'])

finish = datetime.datetime.now()
print('Время окончания: ' + str(finish))
print('Время работы: ' + str(finish - start), end='\n\n')

start = datetime.datetime.now()
print('Время старта (pd.to_datetime): ' + str(start))

df = pd.read_csv(
    filename,
    usecols=['tpep_pickup_datetime',
             'trip_distance',
             'passenger_count',
             'total_amount'])
df['tpep_pickup_datetime'] = pd.to_datetime(df['tpep_pickup_datetime'])

finish = datetime.datetime.now()
print('Время окончания: ' + str(finish))
print('Время работы: ' + str(finish - start))

```

Вывод:

```

Время старта (parse_dates): 2024-10-18 12:46:23.147446
Время окончания: 2024-10-18 12:46:27.247081
Время работы: 0:00:04.099635

```

```

Время старта (pd.to_datetime): 2024-10-18 12:46:27.247110
Время окончания: 2024-10-18 12:46:31.363784
Время работы: 0:00:04.116674

```

На моем компьютере время выполнения операций оказалось приблизительно равным.

УПРАЖНЕНИЕ 41. Цены на нефть

В этом упражнении мы поработаем с файлом CSV, содержащим цены на нефть марки West Texas Intermediate. Эти цены обновляются и публикуются ежедневно, и в нашем наборе собраны данные за период со 2 января 1986 года по 20 декабря 2021 года. Файл CSV я создал с помощью программы на Python, загруженной по адресу <https://github.com/datasets/oil-prices>, которая извлекает данные о ценах на нефть с сайта Правительства США. В этом упражнении мы проанализируем исторические данные о ценах на нефть с использованием функционала для работы с временными рядами, имеющегося в pandas. В частности, вам нужно будет сделать следующее.

1. Импортировать данные из файла wti-daily.csv в датафрейм, в котором столбец Date должен быть приведен к типу datetime и установлен в качестве индекса.

2. Ответить на приведенные ниже вопросы:
 - какова была средняя цена за баррель нефти в июне 1992 года?
 - какова была средняя цена за баррель нефти в 1987 году?
 - какова была средняя цена за баррель нефти в период с сентября 2003 года по июль 2014 года?
3. Вывести цену на нефть на конец каждого квартала в наборе данных.
4. Вывести среднюю цену на нефть за каждый год в наборе данных.
5. Узнать даты наибольшей и наименьшей цены на нефть в наборе данных.

Подробный разбор

Мы уже видели, как с помощью атрибута доступа `dt` можно извлекать различные компоненты из столбцов с датой и временем. С этим инструментом в арсенале мы можем анализировать данные, привязанные к датам, под самыми разными углами. Но мы также видели, что подобные запросы удобнее писать и читать, когда в датафрейме задан нужный индекс. Это особенно верно тогда, когда мы имеем дело с календарными данными. И в этом упражнении мы в полной мере воспользуемся этим преимуществом при анализе цен на нефть.

Сначала, как и всегда, нам необходимо загрузить данные в датафрейм из файла CSV. Наш файл содержит только два столбца с именами `Date` и `Price`. При загрузке данных мы укажем `pandas` на необходимость приведения столбца `Date` к типу `datetime`. Мы также сделаем этот столбец индексом, как показано ниже:

```
filename = '../data/wti-daily.csv'

df = pd.read_csv(filename,
                  parse_dates=['Date'],
                  index_col=['Date'])
```

Теперь мы можем формулировать запросы к нашему датафрейму. Для начала нас попросили вычислить среднюю цену за баррель нефти в июне 1992 года. Как и всегда, мы можем обратиться к нашему датафрейму при помощи атрибута доступа `loc` с указанием интересующего нас значения индекса. Но поскольку мы имеем дело с временным рядом, то можем комбинировать разные компоненты даты и времени, чтобы запрос удовлетворял нашим требованиям. К примеру, мы можем сформулировать наш запрос так:

```
df.loc['1992-06-15']
```

Это позволит получить строку, соответствующую 15 июня 1992 года. Если бы в нашем наборе данных присутствовало больше одной строки для этой даты, мы бы получили их все. Но в данном наборе все даты уникальны. Для получения всех строк за июнь 1992 года мы можем написать следующее выражение:

```
df.loc['1992-06']
```

Опустив компоненту, относящуюся к дню, мы тем самым попросили `pandas` отобрать все даты, относящиеся к нужному нам месяцу и году. Теперь получить среднюю цену не составит труда:

```
df.loc['1992-06'].mean()
```

Полученный результат: 22.38 долл. Далее нас попросили узнать среднюю цену на нефть в 1987 году. Подобно тому как мы можем опустить день, мы можем исключить из строки с датой и месяц. В этом случае мы получим все строки, соответствующие 1987 году, для которых можем вычислить среднее, как показано ниже:

```
df.loc['1987'].mean()
```

Результат оказался равным 19.20 долл.

После этого нас попросили определить среднюю цену за баррель нефти в период с сентября 2003 года по июль 2014 года. Простейшим способом фильтрации интервалов при работе с временными рядами является использование срезов. Обычно при применении срезов в Python мы указываем первый элемент и элемент, идущий за последним в нужном интервале. К примеру, если у нас есть структура вроде списка, кортежа или строки с именем `s`, то выражение `s[10:20]` позволит извлечь значения, начиная с индекса 10 и заканчивая индексом 20, но не включая его самого.

С временными рядами срезы применяются точно так же, но, как и в случае с другими нечисловыми значениями, последний элемент в срезе включается в итоговую выборку. Таким образом, мы можем указать дату начала и дату окончания требуемого интервала, как показано ниже:

```
df.loc['2003-09':'2014-07']
```

Это позволит извлечь все строки из датафрейма, входящие в интервал с 1 сентября 2003 года по 31 июля 2014 года. После этого можно применить агрегирующую функцию `mean`, как показано ниже:

```
df.loc['2003-09':'2014-07'].mean()
```

Мы получили результат 76.45 долл.

Также нам была поставлена задача вывести цену на нефть на конец каждого квартала в наборе данных. Pandas значительно облегчает эту задачу, предоставляя атрибут `is_quarter_end` для свойства `dt`, которое присуще любому объекту Series, содержащему значения в формате `datetime`. Но в нашем случае эти значения располагаются не в объекте Series, а в индексе. Как можно вызвать атрибут `is_quarter_end` у индекса?

Оказывается, этот атрибут доступен непосредственно для свойства `index` нашего датафрейма, представляющего собой временной ряд. В результате обращения к нему мы получим объект Series, заполненный логическими значениями:

```
df.index.is_quarter_end
```

Далее мы можем применить полученный объект в качестве маски к объекту `df`, как показано ниже:

```
df.loc[df.index.is_quarter_end]
```

В результате мы получим датафрейм, состоящий из единственного столбца и индекса с последними днями кварталов, которые могут выпадать как на 30-е, так и на 31-е число.

Также нас попросили вывести среднюю цену на нефть за каждый год. Мы сделаем это с помощью метода `resample`, который действует аналогично мето-

ду `groupby`, но для временных рядов, позволяя применять агрегирующие методы вроде `mean` к любым временным отрезкам. Если какой-то период времени отсутствует в датафрейме или заполнен не полностью, он все равно будет показан, чтобы в итоговом представлении не было пропусков.

При использовании метода `resample` мы должны указать гранулярность временных отрезков при помощи букв и цифр. К примеру, мы можем выполнить еженедельный анализ, передав методу строку `'1w'`, или агрегировать показатели за каждые два месяца, передав строку `'2M'`. В этом задании нам нужно вывести среднюю цену на нефть за каждый год, так что мы воспользуемся значением параметра `'1Y'`, как показано ниже:

```
df.resample('1Y').mean()
```

Результатом применения метода `resample` всегда является датафрейм, в котором в качестве индекса указаны последние даты выбранных отрезков. В нашем случае значения в индексе будут простирались от 1986-12-31 до 2021-12-31. Обратите внимание, что, даже если за какой-то год у вас в наборе данных заполнены не все значения, мы получим усреднение данных за год.

В заключение нас попросили узнать даты наибольшей и наименьшей цены на нефть в наборе данных. Как вы понимаете, решить эту задачу можно разными способами, но простейшим, с моей точки зрения, является извлечение первого и последнего значений после выполнения сортировки по столбцу `Price`. Помните, что мы можем извлекать больше одного значения, передавая список индексов атрибуту `loc` или `iloc`, а в случае с `iloc`, извлекающим значения по позициям, можно легко затребовать первый (индекс 0) и последний (индекс -1) элементы, чем мы и воспользуемся:

```
df['Price'].sort_values().iloc[[0, -1]]
```

Вас может удивить минимальная цена за баррель нефти, равная -36.98 долл., как если бы вам доплачивали за покупку этого ценного сырья. Такое резкое падение спроса на нефть возникло в первые дни начала пандемии Covid-19. В результате образовался огромный дефицит в отношении мест хранения уже выкачанной из земли нефти, что привело к такой странной рыночной ситуации. Это было одним из экономических последствий начала пандемии.

Заметьте, что если нам нужно получить только даты, в которые была зафиксирована минимальная и максимальная цена на нефть, мы можем воспользоваться агрегирующими функциями `idxmin` и `idxmax`, как показано ниже:

```
df['Price'].agg(['idxmin', 'idxmax'])
```

Эти функции возвращают индексы, соответствующие наименьшему и наибольшему значениям соответственно. Метод `agg`, как вы помните, позволяет применить сразу несколько агрегирующих методов к набору данных. В результате мы получим значения индекса, что в нашем случае равносильно извлечению дат.

Решение

```
filename = '../data/wti-daily.csv'
```

```
df = pd.read_csv(filename,
```

```

parse_dates=['Date'],
index_col=['Date'])

df.loc['1992-06'].mean()
df.loc['1987'].mean()
df.loc['2003-09':'2014-07'].mean()
df.loc[df.index.is_quarter_end]
df.resample('1Y').mean()
df['Price'].sort_values().iloc[[0, -1]]

```

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/NVIE>.

Дополнительные упражнения

1. Воспользуйтесь методом `resample` для нахождения среднего значения и стандартного отклонения цены на нефть по кварталам.
2. В каком квартале наблюдался наибольший рост цены на нефть в сравнении с предшествующим кварталом?
3. В каком квартале был зафиксирован наибольший рост цены на нефть в процентах в сравнении с предшествующим кварталом?

Ответы на дополнительные упражнения

Упражнение 41.1

```
df.resample('1Q').agg(['mean', 'std'])
```

Вывод:

	Price mean	std
Date		
1986-03-31	17.217213	4.856866
1986-06-30	13.866094	1.346364
1986-09-30	13.813906	1.809548
1986-12-31	15.406452	0.824353
1987-03-31	18.250328	0.661624
...	...	...
2020-12-31	42.524921	3.844217
2021-03-31	58.093443	4.931396
2021-06-30	66.186667	4.403607
2021-09-30	70.575469	3.078296
2021-12-31	77.701273	5.712271

[144 rows x 2 columns]

Упражнение 41.2

```
df.resample('1Q').mean().diff().sort_values('Price', ascending=False).iloc[0]
```

Вывод:

```
Price      25.91959
Name: 2008-06-30 00:00:00, dtype: float64
```

Упражнение 41.3

```
(
    df
    .resample('1Q')
    .mean()
    .pct_change()
    .sort_values('Price', ascending=False)
    .iloc[0]
)
```

Вывод:

```
Price      0.475456
Name: 1990-09-30 00:00:00, dtype: float64
```

УПРАЖНЕНИЕ 42. *Чаевые за поездки на такси*

Мы уже не раз обращались в наших упражнениях к набору данных о поездках на такси в Нью-Йорке. Но мы еще не все выяснили. На этот раз нас заинтересовал вопрос о том, насколько щедрые чаевые оставляют пассажиры такси. Если вы не из США, вы, возможно, не знаете о традиции давать порядка 15–20 % чаевых водителю сверх того, что показывает счетчик. В некоторых странах такая практика не приветствуется и даже может быть наказуема. Итак, что вам нужно будет сделать.

1. Загрузить и объединить данные о поездках на такси за январь и июль 2019 года. Вам понадобятся столбцы `trip_pickup_datetime`, `passenger_count`, `trip_distance`, `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount` и `congestion_surcharge`.
2. Создать столбец с именем `pre_tip_amount`, в котором необходимо суммировать значения из всех числовых колонок, за исключением `total_amount` и `tip_amount`. (Обратите внимание, что в столбце `total_amount` хранится сумма значений из всех столбцов, включая `tip_amount`. Таким образом, в столбце `pre_tip_amount` должно находиться значение, соответствующее разнице между значениями в столбцах `total_amount` и `tip_amount`).
3. Создать новый столбец с именем `tip_percentage`, показывающий долю чаевых от исходной суммы без чаевых.
4. Ответить на следующие вопросы:
 - какой процент чаевых в среднем оставляли пассажиры в такси?
 - сколько раз размер чаевых превышал сумму без чаевых?
 - в какой день недели пассажиры в среднем оставляли больше всего чаевых?

- в какое время (час) пассажиры давали в среднем больше всего чаевых?
- в каком месяце люди в среднем больше давали чаевых – в январе или в июле?
- найдите дату, в которую пассажиры оставили в такси наибольший процент чаевых.

Подробный разбор

В этом упражнении мы рассмотрим вопрос, связанный с чаевыми в нью-йоркских такси, под разными углами. И все виды анализа так или иначе будут ориентированы на даты или время.

Для начала загрузим данные, относящиеся к январю и июлю 2019 года, с помощью функции `pd.read_csv`, после чего объединим их посредством функции `pd.concat`, воспользовавшись в процессе генератором списков:

```
filenames = ['../data/nyc_taxi_2019-01.csv',
             '../data/nyc_taxi_2019-07.csv']

all_dfs = [pd.read_csv(one_filename,
                      usecols=['tpep_pickup_datetime',
                              'passenger_count',
                              'trip_distance',
                              'fare_amount', 'extra', 'mta_tax',
                              'tip_amount', 'tolls_amount',
                              'improvement_surcharge',
                              'total_amount', 'congestion_surcharge'],
                      parse_dates=['tpep_pickup_datetime'])
            for one_filename in filenames]

df = pd.concat(all_dfs)
```

Здесь мы загрузили довольно много столбцов, чтобы можно было более точно оперировать суммами чаевых. Я мог бы попросить вас загрузить все столбцы, не указывая параметр `usecols`, но предостерег себя от этого, чтобы вы не обретали дурные привычки. Вы всегда должны указывать только те столбцы, которые нужны вам для конкретного вида анализа. В противном случае вам может не хватить ресурсов для обработки больших наборов данных.

Итак, первым делом мы рассчитаем сумму поездки без учета чаевых, т. е. ту сумму, от которой рассчитываются чаевые. Не совсем понятно, что именно стоит включать в чаевые. К примеру, стоит ли включать в расчет оплату за проезд по платным мостам и туннелям? А как насчет наценки на загруженность дорог в центре города, которая иногда взимается? Мы в этом упражнении будем включать в базу для расчета чаевых все платежи.

Итак, нас попросили создать новый столбец с именем `pre_tip_amount`, значения в котором должны собираться из шести столбцов. Как это лучше сделать?

Один из способов состоит в перечислении всех слагаемых столбцов, как показано ниже:

```
df['pre_tip_amount'] = (df['fare_amount'] +
                      df['extra'] +
```



```
df['mta_tax'] +
df['tolls_amount'] +
df['improvement_surcharge'] +
df['congestion_surcharge']])
```

Это сработает, но получилось довольно многословно. Может, есть способ перечислить имена столбцов, значения из которых нужно просуммировать? Кажется, можно воспользоваться методом `sum`, но ведь он складывает значения в строках, а не в столбцах. Хотя постойте! Мы ведь во многих методах `pandas` можем указать ось для применения вычисления, и метод `sum` должен входить в их число. И действительно, чтобы сложить значения в указанных столбцах, нам достаточно передать методу `sum` параметр `axis='columns'`, как показано ниже:

```
df['pre_tip_amount'] = df[['fare_amount',
                           'extra',
                           'mta_tax',
                           'tolls_amount',
                           'improvement_surcharge',
                           'congestion_surcharge']]
                        ].sum(axis='columns')
```

Обратите внимание, что для столбцов мы перечислили в списке их имена в виде строк. В результате вызова метода `sum` мы получим объект `Series`, который присвоим новому столбцу `df['pre_tip_amount']`.

Теперь, когда у нас есть суммы без учета чаевых, мы можем рассчитать процент уплаченных чаевых следующим образом:

```
df['tip_percentage'] = df['tip_amount'] / df['pre_tip_amount']
```

Ну вот, вся предварительная работа сделана, и у нас есть все необходимые данные для ответа на поставленные вопросы. И первым делом нас попросили узнать, какой процент чаевых в среднем оставляли пассажиры в такси. Мы можем посчитать это, применив метод `mean` к полю `tip_percentage`, как показано ниже:

```
df['tip_percentage'].mean()
```

Как видите, при правильной предварительной подготовке данных отвечать на аналитические вопросы легко и приятно.

В результате мы получили 13 %. Мне кажется, это маловато. Возможно, мы определили базу для расчета чаевых не так, как предполагалось. А может, набор данных достаточно сложный для применения простых расчетов процентов. Допустим, в базе могут быть случаи, когда размер чаевых превышает базу для их расчета. Давайте найдем их:

```
(df['tip_percentage'] > 1).value_counts()
```

Вывод:

```
tip_percentage
False    13970379
True      7832
Name: count, dtype: int64
```

Применив метод `value_counts` к объекту с логическими значениями, мы узнали, как часто встречаются значения `True`, т. е. как часто выполняется наше условие. Мы видим, что 7832 раза пассажиры давали водителю чаевые, превышающие сумму без чаевых. Это не так много с учетом количества поездок, но все же не ноль, что меня удивляет. К тому же такие выбросы будут тянуть общий процент чаевых вверх. Но он может выравниваться за счет тех, кто не платит чаевые вовсе. Давайте узнаем, много ли таких:

```
(df['tip_percentage'] == 0).value_counts(normalize=True)
```

Вывод:

```
tip_percentage
False    0.67923
True     0.32077
Name: proportion, dtype: float64
```

Мы снова воспользовались методом `value_counts`, но на этот раз с параметром `normalize=True` для получения результата в процентах. Надо сказать, полученный результат немало меня удивил. Около 32 % пассажиров в Нью-Йорке не платят чаевые совсем! Очевидно, что это сильно отразилось на среднем проценте чаевых.

Далее нас попросили узнать, в какой день недели пассажиры в среднем оставляли больше всего чаевых. Для ответа на этот вопрос мы объединим метод `groupby` с атрибутом `day_of_week`, присутствующим в свойстве `dt`, который возвращает порядковый номер дня недели, где 0 – это понедельник, а 6 – воскресенье. Вы думаете, что нам обязательно нужно создать новый столбец с номером дня недели для выполнения этого расчета? Отнюдь. Разработчики `pandas` позаботились о нас и позволили применять группировку не только к столбцу, но и к результатам, полученным при обращении к атрибуту `dt.day_of_week`, что показано ниже:

```
df.groupby(df['tpep_pickup_datetime'].dt.day_of_week)
```

Далее для каждого дня недели нам необходимо рассчитать средние проценты чаевых. Сделаем это так:

```
df.groupby(df['tpep_pickup_datetime'].dt.day_of_week)
    ['tip_percentage'].mean()
```

Вывод:

```
tpep_pickup_datetime
0    0.128723
1    0.131424
2    0.132221
3    0.133970
4    0.129136
5    0.125801
6    0.126634
Name: tip_percentage, dtype: float64
```

Мы получили средние проценты чаевых по дням недели. Отсортируем полученные результаты в порядке убывания, чтобы они были более наглядными:

```
df.groupby(df['tpep_pickup_datetime'].dt.day_of_week
           )['tip_percentage'].mean().sort_values(ascending=False)
```

Вывод:

```
tpep_pickup_datetime
3    0.133970
2    0.132221
1    0.131424
4    0.129136
0    0.128723
6    0.126634
5    0.125801
Name: tip_percentage, dtype: float64
```

Для меня довольно удивительно, что между днями недели нет практически никакой разницы в отношении процента чаевых. Перед проведением анализа я полагал, что люди склонны платить больше чаевых во время уик-энда, но данные мою гипотезу не подтвердили. Более того, оказалось, что люди дают меньше чаевых как раз в выходные, а больше всего платят в разгар рабочей недели – в среду и четверг. Но разница не так велика, так что мы не можем делать какие-то выводы. Как следствие, если бы мы были водителями такси, нам бы не стоило планировать свои смены исходя из дня недели. К тому же, как мы помним, добрая треть пассажиров не дает чаевых вовсе...

Может, время суток имеет значение? Быть может, люди склонны давать больше чаевых утром или днем? Нас как раз попросили узнать, в какое время (час) пассажиры давали в среднем больше всего чаевых. Давайте воспользуемся предыдущим примером:

```
df.groupby(df['tpep_pickup_datetime'].dt.hour
           )['tip_percentage'].mean().sort_values(ascending=False)
```

Вывод:

```
tpep_pickup_datetime
22    0.138816
20    0.138160
21    0.137685
8     0.137116
19    0.135174
23    0.134978
18    0.133292
9     0.133017
7     0.132134
0     0.131490
2     0.130914
1     0.130710
17    0.128640
10    0.127200
11    0.125022
16    0.124655
13    0.124567
```

```

12    0.124376
14    0.123727
15    0.123547
3     0.121053
6     0.119915
4     0.118987
5     0.112028
Name: tip_percentage, dtype: float64

```

Запрос в этом случае получился очень похожим на предыдущий. Но результаты оказались более интересными. Люди в среднем оставляют порядка 11 % чаевых рано утром (с 3 до 6 ч утра) и почти 14 % – поздно вечером (с 20 до 23 ч). Немного ниже процент чаевых для интервала с 7 до 9 утра. Так что, если вы, как водитель такси, выбираете время для работы между ранним и поздним утром, выгоднее будет выйти на смену попозже.

Теперь давайте ответим еще на один вопрос с помощью наших данных. Нас интересует, в каком месяце люди в среднем больше давали чаевых – в январе или в июле? Есть вообще разница? Сгруппируем и усредним данные по месяцам:

```

df.groupby(df['tpep_pickup_datetime'].dt.month
           )['tip_percentage'].mean().sort_values(ascending=False)

```

Вывод:

```

tpep_pickup_datetime
5     0.200000
8     0.158099
3     0.148046
9     0.141431
1     0.137011
2     0.132224
7     0.121570
12    0.109367
6     0.107354
10    0.100000
4     0.074877
11    0.046026
Name: tip_percentage, dtype: float64

```

Как видим, самые большие чаевые в среднем приходятся на май, следом за которым идут август, март и сентябрь.

Но постойте, мы же загружали данные только за январь и июль! Как сюда попали остальные месяцы? Ответ кроется в ошибках в данных. Водители могли неправильно записать дату, сдать информацию задним числом, да что угодно могло произойти. Если сравнить только январь и июль, мы увидим, что в зимний месяц чаевые в среднем составляли 13.7 %, а в летний – 12.1 %. Возможно, дело в туристах летом, которые могут (я просто предполагаю) давать меньше чаевых, а может, зимой у людей меньше трат и больше свободных денег.

В заключение нас попросили найти дату, когда пассажиры оставили в такси наибольший процент чаевых в среднем. Такой тип задач легче всего решается с

помощью временных рядов, а значит, нам нужно использовать в качестве индекса столбец с типом `datetime`:

```
df = df.set_index('tpep_pickup_datetime')
```

Теперь можно воспользоваться методом `resample` с аргументом `1D` (т. е. один день) для нахождения дня, в который пассажиры оставили в среднем больше всего чаевых. Для начала найдем средние значения чаевых по всем дням, как показано ниже:

```
df.resample('1D')['tip_percentage'].mean()
```

Это работает, но нам хотелось бы отсортировать значения, чтобы легче было найти самый удачный день для таксистов. Применим для этого метод `sort_values` к результатам и оставим только первые десять дат:

```
df.resample('1D')['tip_percentage']
    ].mean().sort_values(ascending=False).head(10)
```

Вывод:

```
tpep_pickup_datetime
2019-02-13    0.358127
2019-02-25    0.250000
2019-08-20    0.241865
2019-11-27    0.200000
2019-08-15    0.200000
2019-05-20    0.200000
2019-08-10    0.200000
2019-09-22    0.200000
2019-09-24    0.200000
2019-09-25    0.200000
Name: tip_percentage, dtype: float64
```

Но нам нужно оставить только данные для января и июля. Давайте попробуем еще раз, но сначала отсортируем индекс и избавимся от паразитных записей:

```
df = df.sort_index()
df = pd.concat([df['2019-01-01':'2019-01-31'],
df['2019-07-01':'2019-07-31']])
df.resample('1D')['tip_percentage'].mean().sort_values(
    ascending=False).head(10)
```

Вывод:

```
tpep_pickup_datetime
2019-01-31    0.144351
2019-01-30    0.143530
2019-01-24    0.143434
2019-01-22    0.142769
2019-01-15    0.142329
2019-01-29    0.141330
2019-01-10    0.141291
2019-01-16    0.141147
2019-01-17    0.140356
```

```
2019-01-23    0.140309
Name: tip_percentage, dtype: float64
```

Теперь после очистки данных от ненужных записей мы видим, что все десять дней с наибольшим средним процентом чаевых принадлежат январю. Мы можем проверить свою догадку о выгодности работы таксистом зимой, перегруппировав данные по месяцам:

```
df.resample('1M')['tip_percentage'].mean().dropna()
```

Вывод:

```
tpep_pickup_datetime
2019-01-31    0.137012
2019-07-31    0.121570
Name: tip_percentage, dtype: float64
```

Поскольку у нас в данных теперь только два месяца (январь и июль), в остальных месяцах мы получили значения NaN и отфильтровали их с помощью метода `dropna`. Как видим, в январе средний процент чаевых составил 13.7 %, а в июле – 2.1 %.

Решение

```
filenames = ['../data/nyc_taxi_2019-01.csv',
              '../data/nyc_taxi_2019-07.csv']

all_dfs = [pd.read_csv(one_filename,
                       usecols=['tpep_pickup_datetime',
                                'passenger_count',
                                'trip_distance',
                                'fare_amount', 'extra',
                                'mta_tax', 'tip_amount',
                                'tolls_amount',
                                'improvement_surcharge',
                                'total_amount', 'congestion_surcharge'],
                       parse_dates=['tpep_pickup_datetime'])
            for one_filename in filenames]

df = pd.concat(all_dfs)

df['pre_tip_amount'] = df[['fare_amount', 'extra',
                           'mta_tax', 'tolls_amount',
                           'improvement_surcharge',
                           'congestion_surcharge']].sum(
    axis='columns')

df['tip_percentage'] = df[
    'tip_amount'] / df['pre_tip_amount']

df['tip_percentage'].mean()
```

```

(df['tip_percentage'] > 1).value_counts() 7

(df['tip_percentage'] == 0).value_counts(normalize=True) 8

df.groupby(df[
    'tpep_pickup_datetime'].dt.day_of_week)[
    'tip_percentage'].mean().sort_values(ascending=False) 9

df.groupby(df[
    'tpep_pickup_datetime'].dt.hour)[
    'tip_percentage'].mean().sort_values(
        ascending=False).head(5) 10

df.groupby(df[
    'tpep_pickup_datetime'].dt.month)[
    'tip_percentage'].mean().sort_values(
        ascending=False) 11

df = df.set_index('tpep_pickup_datetime') 12

df.resample('1D')[
    'tip_percentage'].mean().sort_values(
        ascending=False).head(10) 13

df = df.sort_index()
df = pd.concat([df['2019-01-01': '2019-01-31'],
                df['2019-07-01': '2019-07-31']]) 14

df.resample('1D')[
    'tip_percentage'].mean().sort_values(
        ascending=False).head(10)

```

- ❶ Вызов метода `read_csv` для всех файлов.
- ❷ Генератор списков возвращает список датафреймов.
- ❸ Объединяем список датафреймов в один датафрейм.
- ❹ Создаем столбец `pre_tip_amount`.
- ❺ Рассчитываем процент чаевых.
- ❻ Средний процент чаевых по всем поездкам.
- ❼ Количество поездок с суммой чаевых, превышающей сумму без чаевых.
- ❽ Какой процент пассажиров не платит чаевые?
- ❾ Группируем по дням недели с расчетом средних чаевых и сортировкой.
- ❿ Группируем по часам с расчетом средних чаевых и сортировкой.
- ⓫ Группируем по месяцам с расчетом средних чаевых и сортировкой.
- ⓫ Устанавливаем индекс в датафрейме, превращая его во временной ряд.
- ⓫ Находим средние чаевые по отдельным датам.
- ⓫ Искключаем даты, не входящие в нужный нам интервал.

Вы можете посмотреть это решение на сайте Pandas Tutor по адресу <http://mng.bz/tW42>.

Дополнительные упражнения

1. Мы увидели, что около 32 % пассажиров нью-йоркских такси не платит чаевые вовсе. Рассчитайте средний процент чаевых только среди тех, кто платит.
2. Сколько в нашем наборе данных, предположительно ограниченном январем и июлем 2019 года, присутствует поездок с ошибочным периодом?
3. В наборе данных, ограниченном январем и июлем 2019 года, найдите неделю, в которую пассажиры оставили максимальный процент чаевых в среднем.

Ответы на дополнительные упражнения

Упражнение 42.1

```
df.loc[df['tip_amount'] > 0, 'tip_percentage'].mean()
```

Вывод:

```
0.19146519965282618
```

Упражнение 42.2

```
df[(df['tpep_pickup_datetime'] < '2019-01-01 00:00:00') |
    ((df['tpep_pickup_datetime'] > '2019-01-31 23:59:59') &
     (df['tpep_pickup_datetime'] < '2019-07-01 00:00:00')) |
    (df['tpep_pickup_datetime'] > '2019-07-31 23:59:59')].shape
```

Вывод:

```
(816, 13)
```

Упражнение 42.3

```
df = df.set_index('tpep_pickup_datetime')
df = df.sort_index()
```

```
df = pd.concat([df['2019-01-01':'2019-01-31'],
                df['2019-07-01':'2019-07-31']])
```

```
df.resample('1W')['tip_percentage'].mean().sort_values(ascending=False).dropna()
```

Вывод:

```
tpep_pickup_datetime
2019-02-03    0.141979
2019-01-27    0.138930
2019-01-20    0.138536
2019-01-13    0.137901
2019-01-06    0.126983
2019-08-04    0.124910
```



```
2019-07-14    0.123459
2019-07-21    0.123341
2019-07-28    0.123036
2019-07-07    0.112952
Name: tip_percentage, dtype: float64
```

Заключение

В этой главе мы рассмотрели разные способы работы в pandas с данными, включающими в себя даты и время. Мы узнали, как правильно загружать такие данные в датафрейм, извлекать компоненты, связанные с датой и временем, из существующего столбца, разбивать такие столбцы на несколько и работать со странными форматами дат и времени. Также мы научились создавать временные ряды, т. е. датафреймы, в которых в качестве индекса выступает столбец с датой и временем. Мы узнали, как строить запросы, специфичные для временных рядов, и выполнять их передискретизацию, или группировку данных по различным временным интервалам.

Глава 11

Визуализация

Анализ данных, как вы уже смогли убедиться, читая эту книгу, в основном связан с числовыми данными. Типичный датафрейм в *pandas* в основном состоит именно из числовых данных, к которым применяются разные математические и статистические методы и техники.

Это все прекрасно, за исключением того, что люди по большей части не способны визуально воспринимать и анализировать большие массивы чисел. Они гораздо лучше воспринимают графики, чем таблицы, особенно когда речь касается сравнения показателей или определения связи между ними. И хотя визуализация часто представляется как красивое графическое оформление для сухих идей, предназначенное для людей, далеких от технических специальностей, на самом деле графики могут изрядно помогать и самим аналитикам при обнаружении тенденций и шаблонов в данных. Вывод данных на график зачастую позволяет взглянуть на цифры под иным углом и приблизиться к пониманию сути задачи.

В мире Python настоящим мастодонтом визуализации данных является *Matplotlib*. Эта библиотека отличается почти бескрайними возможностями в плане формирования графиков и диаграмм, но вместе с тем она достаточно сложна для понимания и не всегда необходима. К счастью, *pandas* располагает собственным API визуализации, позволяющим строить достаточно сложные графики на основе данных, не обращаясь напрямую к услугам *Matplotlib*. В результате мы получаем лучшее от двух подходов: имеем возможность формировать диаграммы, не обременяя себя необходимостью разбираться в тонкостях библиотеки *Matplotlib*. При этом всякий раз, когда вам не хватает возможностей графического движка *pandas*, вы можете обратиться к *Matplotlib*.

В этой главе мы посмотрим, как можно визуализировать данные при помощи обертки *Matplotlib*, присутствующей в *pandas*. Мы рассмотрим на примерах самые разные графики, которые позволят вашим сухим цифрам ожить и заиграть новыми красками.

Кроме того, мы немного поработаем с библиотекой *Seaborn*, которая часто используется вместо *Matplotlib*. Но *Seaborn* не единственная альтернатива *Matplotlib*, есть и другие, которые, в отличие от нее, созданы с нуля, а не на основе *Matplotlib*. Но чтобы сделать выбор в пользу того или иного пакета, необходимо изучить имеющиеся варианты. Лично мне больше по душе API *Seaborn*, поскольку меня привлекает идея построения разнообразных диаграмм с минимумом внешнего вмешательства в настройку.

Также в этой главе вы познакомитесь с уникальной возможностью Jupyter выводить графики непосредственно в окне с кодом, как на рис. 11.1. Возможность

наблюдать данные, код и элементы визуализации в одном и том же общем пространстве позволяет исследователям данных делиться своими наработками и принимать обратную связь от менее подкованных в техническом плане коллег.

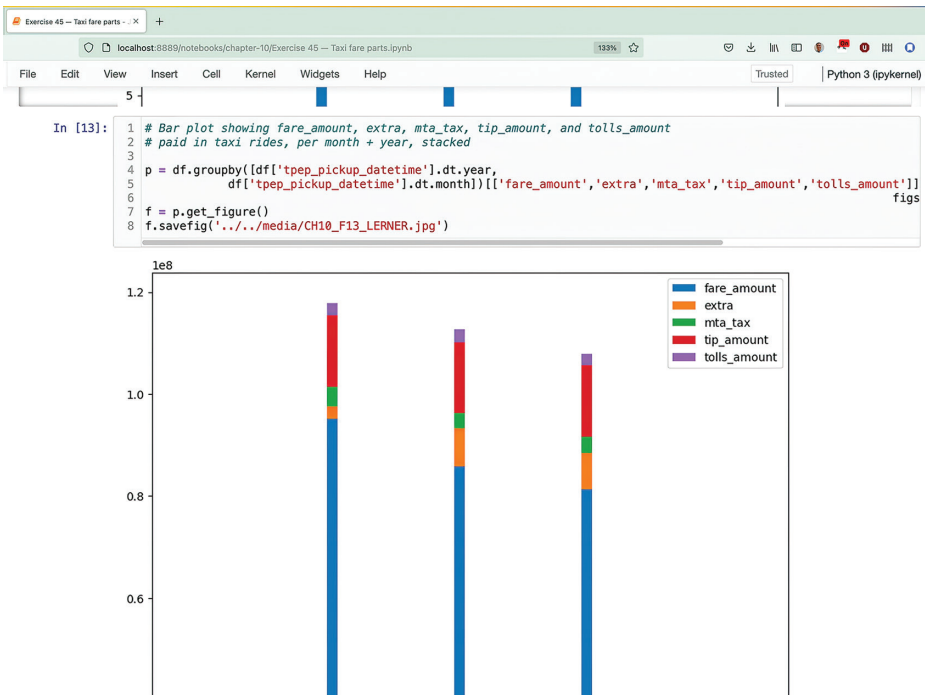


Рис. 11.1. Код и график в одном окне Jupyter notebook

В табл. 11.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 11.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
pd.read_csv	Позволяет прочитать содержимое файла CSV в виде датафрейма	df = df.read_csv('myfile.csv')	http://mng.bz/a1az (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
df.groupby	Позволяет применить один или несколько методов агрегации для каждого значения в конкретном столбце	df.groupby('year')	http://mng.bz/gBG1 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.groupby.html)
df.loc	Извлекает заданные строки и столбцы	df.loc[:, 'passenger_count'] = df['passenger_count']	http://mng.bz/pPNG (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.loc.html)

Таблица 11.1. Предметы изучения (*продолжение*)

Предмет	Описание	Пример	Ссылки для изучения
<code>df.plot</code>	Объект построения графиков для датафрейма	<code>df.plot.box()</code>	http://mng.bz/0x8n (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.plot.html)
<code>df.corr</code>	Показывает корреляцию между числовыми столбцами в датафрейме	<code>df.corr()</code>	http://mng.bz/Y1oN (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.corr.html)
<code>s.quantile</code>	Позволяет получить элемент, соответствующий определенному процентилу	<code>s.quantile(0.25)</code>	http://mng.bz/GyYq (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.quantile.html)
<code>df.join</code>	Объединяет два датафрейма на основе индексов	<code>df.join(other_df)</code>	http://mng.bz/zXva (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.join.html)
<code>pandas.plotting.scatter_matrix</code>	Создает диаграмму рассеяния для сравнения всех пар числовых столбцов	<code>pandas.plotting.scatter_matrix</code>	http://mng.bz/0Kqx (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.plotting.scatter_matrix.html)
Matplotlib	Библиотека Python для создания графиков и диаграмм	<code>import matplotlib.pyplot as plt</code>	http://mng.bz/9D6l (https://matplotlib.org)
Seaborn	Библиотека Python для создания графиков и диаграмм	<code>import seaborn as sns</code>	http://mng.bz/jPKx (http://seaborn.pydata.org)
<code>df.reset_index</code>	Возвращает новый датафрейм с индексом по умолчанию (числовым, порядковым)	<code>df.reset_index(drop=True)</code>	http://mng.bz/Wz50 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.reset_index.html)
<code>pd.concat</code>	Позволяет объединить вместе два датафрейма	<code>df = pd.concat(df1, df2)</code>	http://mng.bz/8r5P (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.concat.html)

УПРАЖНЕНИЕ 43. Города

В упражнении 20 мы работали с файлом JSON, содержащим информацию о 1000 крупнейших городов в США. В этом упражнении мы взглянем на тот же файл, но выведем информацию не в виде безмолвных цифр и таблиц, а в виде красивых и понятных графиков. Итак, что вам нужно будет сделать.

1. Загрузить данные из файла `cities.json` в датафрейм.
2. Отобразить при помощи столбчатой диаграммы количество городов с разнесением по штатам. Каждому штату или неинкорпорированной организованной территории США (Пуэрто-Рико, Виргинские острова и Вашингтон, округ Колумбия) должен соответствовать один вертикальный столбик. Столбики на графике должны быть упорядочены от штата с самым низким количеством городов к штату с самым высоким слева направо.
3. Отобразить при помощи столбчатой диаграммы изменение численности населения в городах с 2000 по 2013 год (столбец `growth_from_2000_to_2013`) в штате Пенсильвания (Pennsylvania). Каждому городу должен соответствовать один столбик. Столбики на графике должны быть упорядочены от города с самым низким изменением численности населения к городу с самым высоким слева направо.
4. Отобразить при помощи круговой диаграммы долю каждого города в штате Массачусетс (Massachusetts) в общей численности населения штата (конечно, не все жители штата обязательно живут в крупных городах). На диаграмме каждому городу должен соответствовать один сегмент, а его размер должен отражать долю этого города.
5. С помощью диаграммы рассеяния отобразить местоположение городов, разместив на оси *x* широту, а на оси *y* – долготу. Как будет выглядеть итоговая диаграмма?

Подробный разбор

Библиотека Matplotlib предлагает на выбор большое разнообразие графиков и диаграмм, и в этом упражнении мы воспользуемся некоторыми из них и опробуем разные техники, чтобы лучше понять наши данные. Визуализация не ограничивается одним лишь выбором типа графика – часто перед выводом данных на диаграмме нам требуется очистить их, преобразовать и изменить их внешний вид.

Сначала нас попросили отобразить при помощи *столбчатой диаграммы* количество городов с разнесением по штатам. В нашем датафрейме, который мы наполнили из файла JSON, присутствует несколько столбцов, и один из них – `state`. Мы воспользуемся им и применим к нему метод `groupby`, чтобы узнать количество городов в каждом штате, как показано ниже:

```
df.groupby('state').count()
```

Это приведет к агрегированию данных во всех столбцах в датафрейме. Поскольку нас интересует только информация о количестве городов, выберем только один столбец `city`:

```
df.groupby('state')['city'].count()
```

Теперь можно приступать к построению столбчатой диаграммы. Но постойте! В задании говорится, что столбики на графике должны быть упорядочены слева направо – от штата с самым низким количеством городов к штату с самым высоким. Это означает, что перед выводом данных на диаграмму нам нужно сначала отсортировать результат, полученный из метода `groupby`. Это можно легко сделать с помощью метода `sort_values`, как показано ниже:

```
(
    df
    .groupby('state')['city'].count()
    .sort_values()
)
```

Вывод:

state	
Alaska	1
Vermont	1
District of Columbia	1
Maine	1
Hawaii	1
	...
Massachusetts	36
Illinois	52
Florida	73
Texas	83
California	212

Name: city, Length: 51, dtype: int64

Теперь можно легко и просто построить столбчатую диаграмму, воспользовавшись атрибутом `plot` и его методом `bar`:

```
(
    df
    .groupby('state')['city'].count()
    .sort_values()
    .plot.bar()
)
```

ПРИМЕЧАНИЕ. Еще один способ построения этого графика заключается в вызове функции `plot` и передаче ей ключевого аргумента `kind='bar'`. Оба синтаксиса равноправны и приемлемы.

Это сработало, но при 50 штатах диаграмма оказалась довольно маленькой. Можно передать методу `bar` аргумент с именем `figsize`, который будет отправлен в движок Matplotlib. Передавая этому аргументу кортеж (10, 10), мы тем самым говорим, что размер графика должен быть 10 на 10 дюймов:

```
(
    df
    .groupby('state')['city'].count()
    .sort_values()
    .plot.bar(figsize=(10,10))
)
```

Неудивительно, что Калифорния отмечена самым большим количеством крупных городов, что видно на рис. 11.2, но отрыв этого штата от остальных меня, если честно, просто поразил!

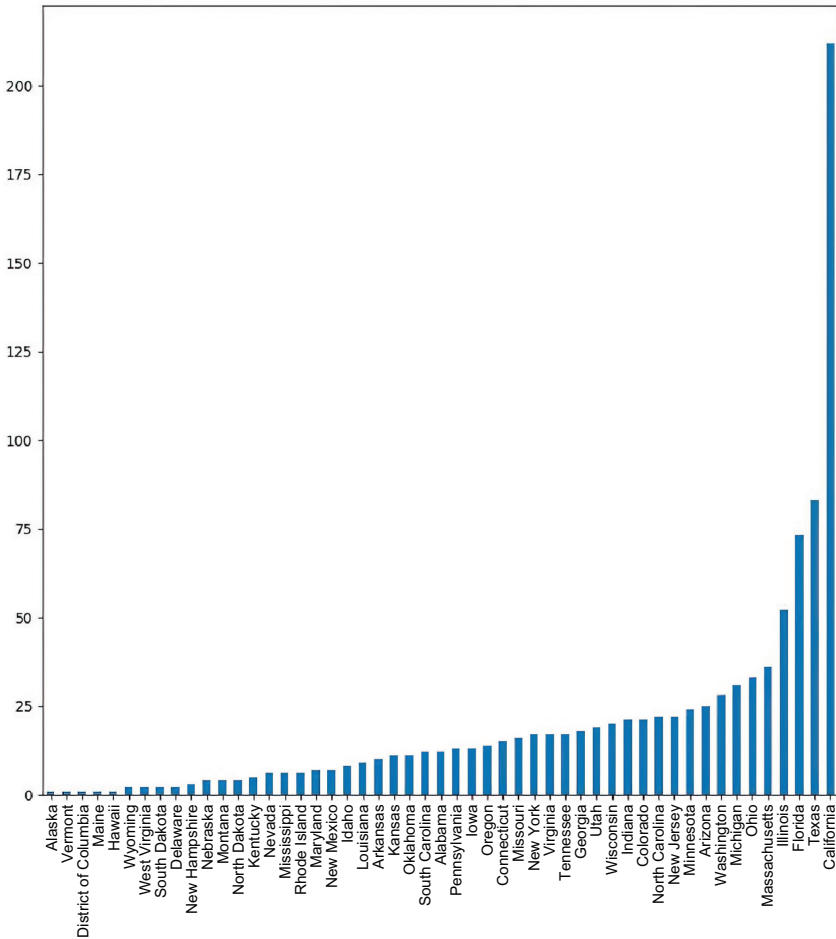


Рис. 11.2. Столбчатая диаграмма с распределением крупных городов по штатам США

Далее нас попросили отобразить при помощи столбчатой диаграммы изменение численности населения в городах с 2000 по 2013 год в штате Пенсильвания. Для этого мы воспользуемся столбцами `city` и `growth_from_2000_to_2013` и отберем только те строки, в которых в столбце `state` содержится значение `'Pennsylvania'`. При этом будет проще построить из нужных нам столбцов и строк отдельный датафрейм, воспользовавшись атрибутом доступа `loc`, как показано ниже:

```
df.loc[df['state']=='Pennsylvania',          ❶
      ['city','growth_from_2000_to_2013']]  ❷
]
```

- ❶ Селектор строк: только строки со значением `'Pennsylvania'` в столбце `state`.
- ❷ Селектор столбцов: только два нужных нам столбца.

Здесь мы выбираем только те строки, у которых в столбце `state` указано значение `'Pennsylvania'`, а затем ограничиваем выбор только нужными нам столбцами. Также, поскольку мы хотим на нашем графике видеть на оси `x` названия городов, мы установим этот столбец в качестве индекса, как показано ниже:

```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city', 'growth_from_2000_to_2013']]
    .set_index('city')
)
```

❶ Устанавливаем индекс на названия городов.

Кажется, сейчас уже можно строить диаграмму. Но у нас есть проблема – дело в том, что данные в столбце `growth_from_2000_to_2013` представлены в виде строк, заканчивающихся символом `%`. Если мы хотим выводить на графике агрегированные значения, нам необходимо привести этот столбец к числовому виду. Как это можно сделать?

Для этого нам сначала надо избавиться от символа `%`, что можно сделать с помощью метода `replace`, имеющегося у атрибута доступа `str`. Но перед этим необходимо преобразовать датафрейм в `Series`, поскольку атрибут `str` есть только у этого объекта. К счастью, при извлечении из датафрейма колонки в виде объекта `Series` индекс сохраняется:

```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city', 'growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013']
)
```

❶ Извлекаем столбец `growth_from_2000_to_2013`.

Теперь мы можем избавиться от символа процента разными способами. Легче всего можно сделать это с помощью метода `str.replace`. Также можно было бы воспользоваться срезом, оставив все символы в строке, кроме последнего:

```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city', 'growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013']
    .str.replace('%', '')
)
```

❶ Удаляем символы `%` из всех элементов в объекте `Series`.

В результате мы также получим объект `Series` и сможем преобразовать строковые значения в числа с плавающей точкой с помощью метода `astype`, как показано ниже:


```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city','growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013']
    .str.replace('%', '')
    .astype(np.float16)
)
```

❶ Приводим столбец `growth_from_2000_to_2013` к типу `np.float16`.

Мы получили объект `Series` со всеми городами из штата Пенсильвания и их темпами роста. Теперь можно вывести эту информацию на графике, но прежде мы отсортируем данные в порядке возрастания значений с помощью метода `sort_values`:

```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city','growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013']
    .str.replace('%', '')
    .astype(np.float16)
    .sort_values()
)
```

Вывод:

```
city
Pittsburgh      -8.296875
Altoona         -7.300781
Wilkes-Barre    -4.300781
Erie            -2.800781
Scranton         0.000000
...
Bethlehem       5.199219
York            6.398438
Reading         8.000000
State College   8.703125
Allentown       11.203125
Name: growth_from_2000_to_2013, Length: 13, dtype: float16
```

В результате мы получили упорядоченный объект `Series`, на основе которого можно построить столбчатую диаграмму размером (10, 10), показанную на рис. 11.3:

```
(
    df.loc[
        df['state']=='Pennsylvania',
        ['city','growth_from_2000_to_2013']]
    .set_index('city')
```

```
['growth_from_2000_to_2013']
.str.replace('%', '')
.astype(np.float16)
.sort_values()
.plot.bar(figsize=(10,10))
)
```

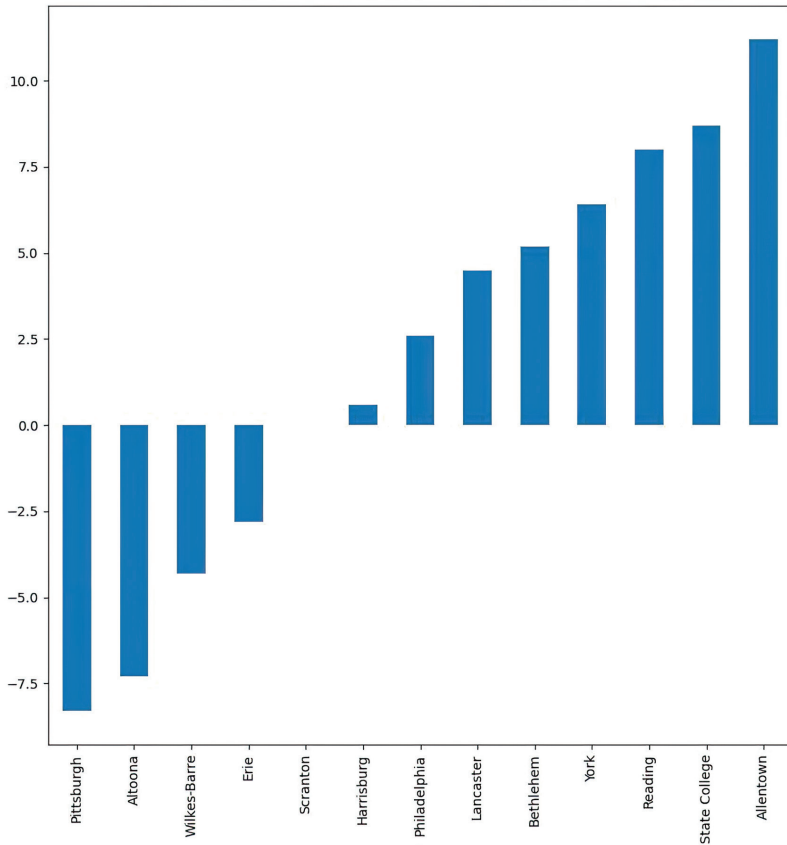


Рис. 11.3. Темпы роста городов в штате Пенсильвания

Далее нас попросили отобразить при помощи *круговой диаграммы* долю каждого города в штате Массачусетс (Massachusetts) в общей численности населения штата. Помните, что круговая диаграмма получает все значения, суммирует их и вычисляет доли, на основе которых заполняет график сегментами.

Для начала получим названия городов и показатели численности населения в штате Массачусетс. Это можно сделать следующим образом:

```
(
df
.loc[
    df['state'] == 'Massachusetts', ❶
    ['city', 'population']] ❷
.set_index('city') ❸
)
```

```
['population']
)
```

- ❶ Селектор строк: только города из штата Массачусетс.
- ❷ Селектор столбцов: только два нужных нам столбца.
- ❸ Устанавливаем названия городов в качестве индекса.
- ❹ Извлекаем только столбец с численностью населения.

Вывод:

```
city
Boston      645966
Worcester   182544
Springfield 153703
Lowell      108861
Cambridge   107289
...
Fitchburg   40383
Holyoke     40249
Marlborough 39414
Woburn      39083
Chelsea     37670
Name: population, Length: 36, dtype: int64
```

Этот запрос очень похож на предыдущий. Мы взяли нужные нам столбцы (city и population) из датафрейма и ограничили выбор штатом Массачусетс. Далее мы установили в качестве индекса столбец city и извлекли только один столбец population в виде объекта Series. Теперь построим круговую диаграмму на основе этих данных, как показано ниже:

```
(
    df
    .loc[
        df['state'] == 'Massachusetts',
        ['city', 'population']]
    .set_index('city')
    ['population']
    .plot.pie(figsize=(10,10))
)
```

Как видно на рис. 11.4, в штате Массачусетс довольно много крупных городов, но явным лидером по численности населения является Бостон (Boston), следом за которым идут Вустер (Worcester) и Спрингфилд (Springfield).

Наконец, нас попросили с помощью диаграммы рассеяния отобразить местоположение городов, разместив на оси x широту, а на оси y – долготу. Это можно сделать с помощью метода plot.scatter у датафрейма, передав ему столбцы для расположения на осях, как показано ниже:

```
df.plot.scatter(x='longitude', y='latitude')
```

Как будет выглядеть наша диаграмма? Мы задали для осей столбцы с широтой и долготой городов, а значит, диаграмма будет походить на географическую карту США с отображением густонаселенных городов, что видно на рис. 11.5.

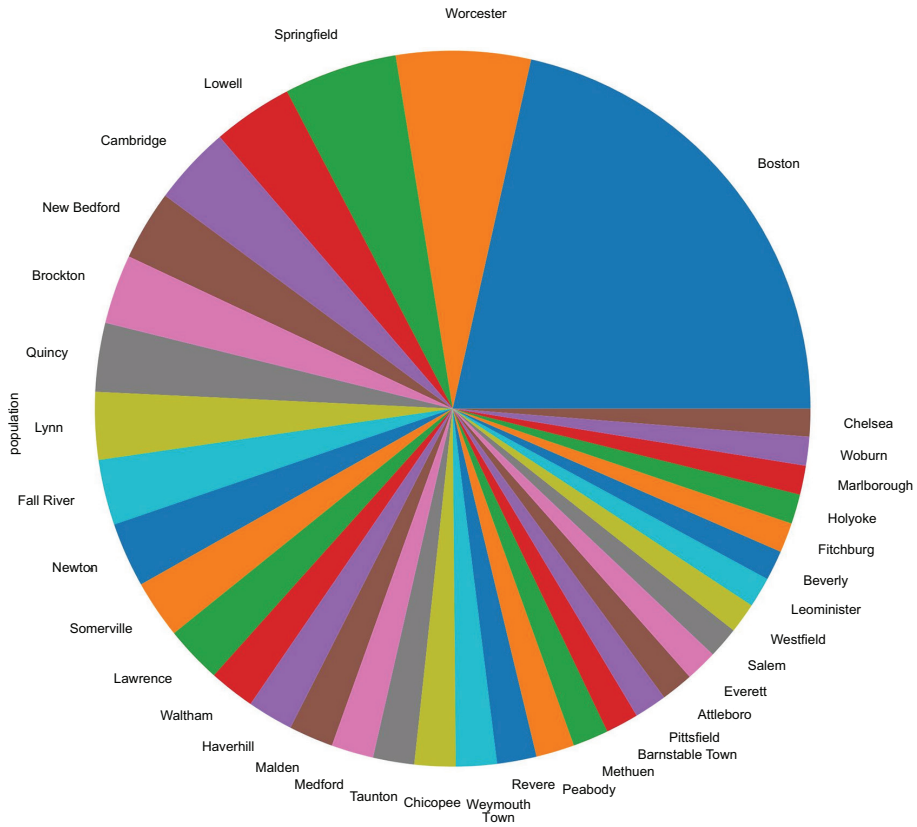


Рис. 11.4. Круговая диаграмма, показывающая численность населения штата Массачусетс в разрезе городов

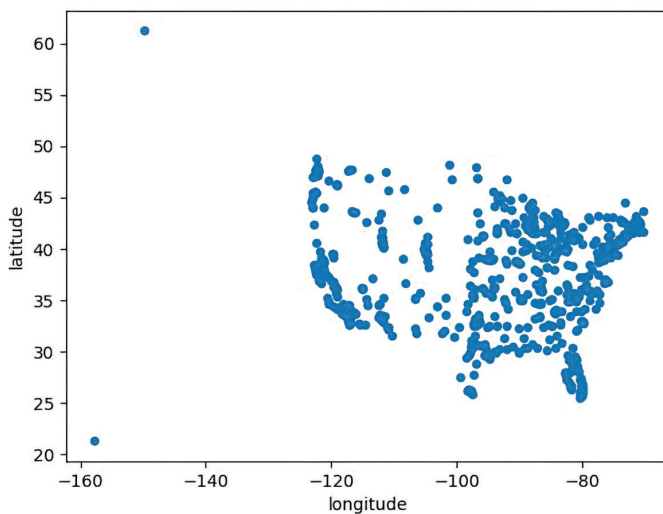


Рис. 11.5. Диаграмма рассеяния с самыми населенными городами США

Решение

```

filename = '../data/cities.json'
df = pd.read_json(filename) ❶

(
    df
    .groupby('state')['city'].count()
    .sort_values()
    .plot.bar(figsize=(10,10)) ❷
)

(
    df.loc[
        df['state']=='Pennsylvania', ❸
        ['city','growth_from_2000_to_2013']] ❹
    .set_index('city')
    ['growth_from_2000_to_2013'] ❺
    .str.replace('%', '')
    .astype(np.float16)
    .sort_values()
    .plot.bar(figsize=(10,10)) ❻
)

(
    df
    .loc[
        df['state'] == 'Massachusetts',
        ['city','population']] ❼
    .set_index('city') ❽
    ['population']
    .plot.pie(figsize=(10,10))
)

df.plot.scatter(x='longitude', y='latitude') ❾

```

- ❶ Читаем данные из файла JSON в датафрейм.
- ❷ Получаем количество городов с группировкой по штатам, сортируем значения и строим столбчатую диаграмму.
- ❸ Получаем все города из штата Пенсильвания.
- ❹ Только столбцы city и growth_from_2000_to_2013.
- ❺ Избавляемся от символа %.
- ❻ Приводим столбец к числовому виду, сортируем значения и строим столбчатую диаграмму.
- ❼ Получаем все города из штата Массачусетс, только столбцы city и population.
- ❽ Строим круговую диаграмму на основе данных о численности населения.
- ❾ Строим диаграмму рассеяния на основе координат крупнейших городов США.

Дополнительные упражнения

1. Постройте объединенную *гистограмму* по темпам роста городов в штатах Техас (Texas) и Мичиган (Michigan).

2. Постройте гистограмму по темпам роста городов в штатах Техас (Texas) и Калифорния (California).
3. Постройте столбчатую диаграмму, показывающую средний рост городов в разрезе штатов.

Диаграмма размаха (ящик с усами)

В старших классах, когда мы начали проходить графики и диаграммы, учитель посвящал их объяснению очень много времени, я даже не понимал, зачем так распространяться на тему графиков, которые мы и так видим ежедневно: линейчатые, столбчатые и даже круговые. Но когда он перешел к описанию диаграммы размаха, формально называемой ящиком с усами, я оказался заинтригован.

Если помните, мы не раз использовали метод `describe` для описания данных, содержащихся в объекте `Series`. Этот метод позволяет увидеть так называемую *сводку пяти чисел Тьюки* (Tukey five-number summary), содержащую выборочный минимум, первый квартиль, медиану, третий квартиль и выборочный максимум, а также среднее значение и стандартное отклонение. Эти показатели помогают нам сделать выводы о наших данных.

Упоминание Тьюки здесь относится к имени Джона Тьюки (John Tukey) – известного математика и статистика, который разработал не только сводку пяти чисел, но и ее удобное графическое отображение, получившее название *диаграмма размаха, коробчатая диаграмма* (boxplot), или попросту *ящик с усами*. Кстати, этот плодовитый математик также первым ввел слова *bit* (*bit*, от binary digit) и *software*.

Давайте для примера создадим простой объект `Series`:

```
s = Series([10, 15, 17, 20, 25])
```

Мы можем легко получить всю описательную статистику по этому набору данных, вызвав метод `s.describe()`:

```
count    5.000000
mean     17.400000
std       5.59464
min      10.000000
25%      15.000000
50%      17.000000
75%      20.000000
max      25.000000
dtype: float64
```

С помощью диаграммы размаха можно увидеть все важнейшие характеристики набора данных в графическом представлении. В `pandas` для вывода диаграммы размаха предусмотрен метод `plot.box`, который можно вызвать как для объекта `Series` (получим один ящик с усами), так и для датафрейма (получим много ящиков). На рис. 11.6 показан вывод для следующего выражения:

```
s.plot.box()
```

Сам ящик характеризуется тремя следующими свойствами:

- верхняя его граница находится на уровне третьего квартиля (75 %);
- линия, делящая ящик на две части, находится на уровне медианы (50 %);
- нижняя его граница находится на уровне первого квартиля (25 %).

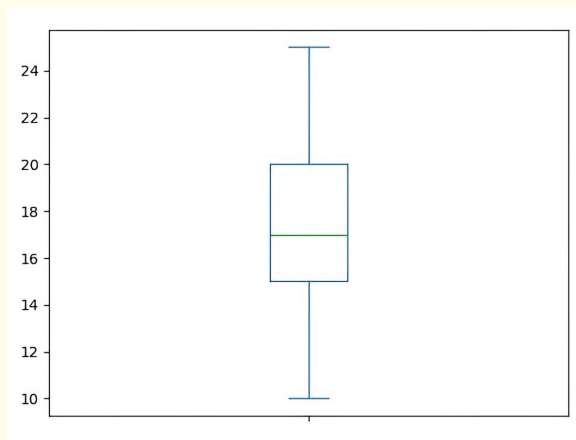


Рис. 11.6. Диаграмма размаха для объекта s

Далее мы видим, что из ящика выходят две линии – одна вверх, вторая вниз. Именно эти линии в просторечии именуются усами. Верхний ус заканчивается на уровне максимального значения, но не поднимается выше полутора межквартильных размахов, а нижний – на уровне минимального значения, но не опускается ниже полутора межквартильных размахов. Таким образом, при одном взгляде на диаграмму размаха мы можем визуально определить все пять основных статистик.

Также над верхним и под нижним усами часто рисуют точки, символизирующие выбросы. *Выбросами* (outlier) обычно называются наблюдения, располагающиеся ниже, чем 25-й процентиль, из которого вычли полтора межквартильного размаха ($25\% - 1.5 * IQR$), и выше, чем 75-й процентиль, к которому прибавили полтора межквартильного размаха ($75\% + 1.5 * IQR$).

Давайте рассмотрим еще один объект Series, в котором присутствуют выбросы:

```
s = Series([-20, 10, 15, 17, 20, 25, 40])
```

Описательная статистика в этом случае будет выглядеть так:

```
count    7.000000
mean     15.285714
std      18.273061
min      -20.000000
25%      12.500000
50%      17.000000
75%      22.500000
max       40.000000
dtype: float64
```

А как будет выглядеть диаграмма размаха, можно увидеть на рис. 11.7.

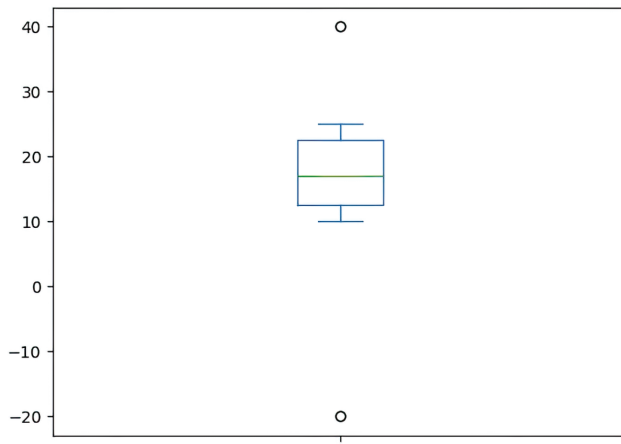


Рис. 11.7. Диаграмма размаха для объекта `s` с выбросами

Диаграммы размаха помогают аналитикам быстрее делать выводы о данных, с которыми приходится работать. Но также они могут быть полезны при сравнении разных наборов данных. В этом случае мы легко можем увидеть, как ящики располагаются относительно друг друга и где они пересекаются. Одновременный вывод информации о нескольких столбцах в таком виде часто применяется в машинном обучении, поскольку одинаковый масштаб переменных позволяет строить более точные модели. Обратите внимание, что при выводе на диаграмме размаха сразу нескольких переменных они делят общую ось `y`, что и позволяет сравнивать диапазоны их значений. Но при необходимости вы можете разнести оси `y` для наборов данных, передав методу `box` параметр `subplots=True`, как показано ниже:

```
df.plot.box(subplots=True)
```

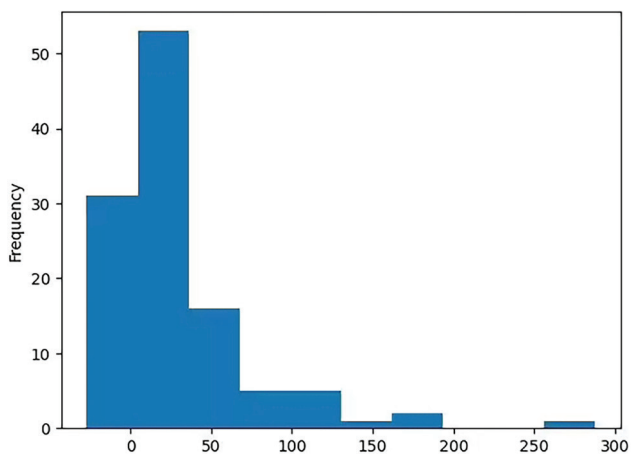
Заметьте, что на диаграмме размаха нигде не присутствует метрика среднего значения. Лично мне это кажется упущением, поскольку среднее значение зачастую бывает важно.

Ответы на дополнительные упражнения

Упражнение 43.1

```
(
    df
    .loc[
        df['state'].isin(['Texas', 'Michigan']),
        ['city', 'growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013'].str.rstrip('%')
    .astype(np.float16)
    .plot.hist()
)
```

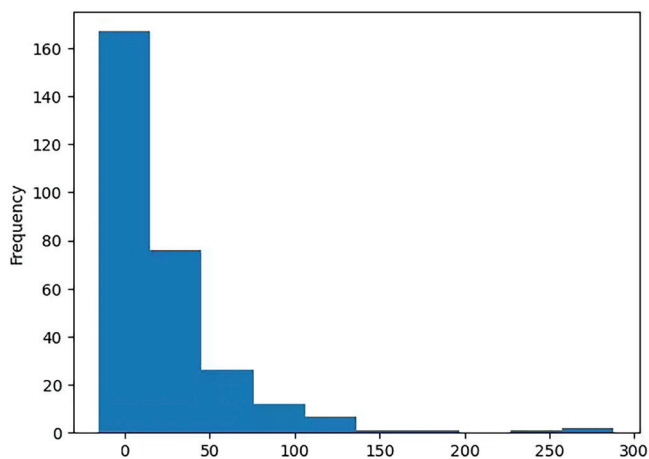

Вывод:



Упражнение 43.2

```
(
    df.loc[
        df['state'].isin(['Texas', 'California']),
        ['city', 'growth_from_2000_to_2013']]
    .set_index('city')
    ['growth_from_2000_to_2013'].str.rstrip('%')
    .replace('', np.nan)
    .dropna()
    .astype(np.float16)
    .plot.hist()
)
```

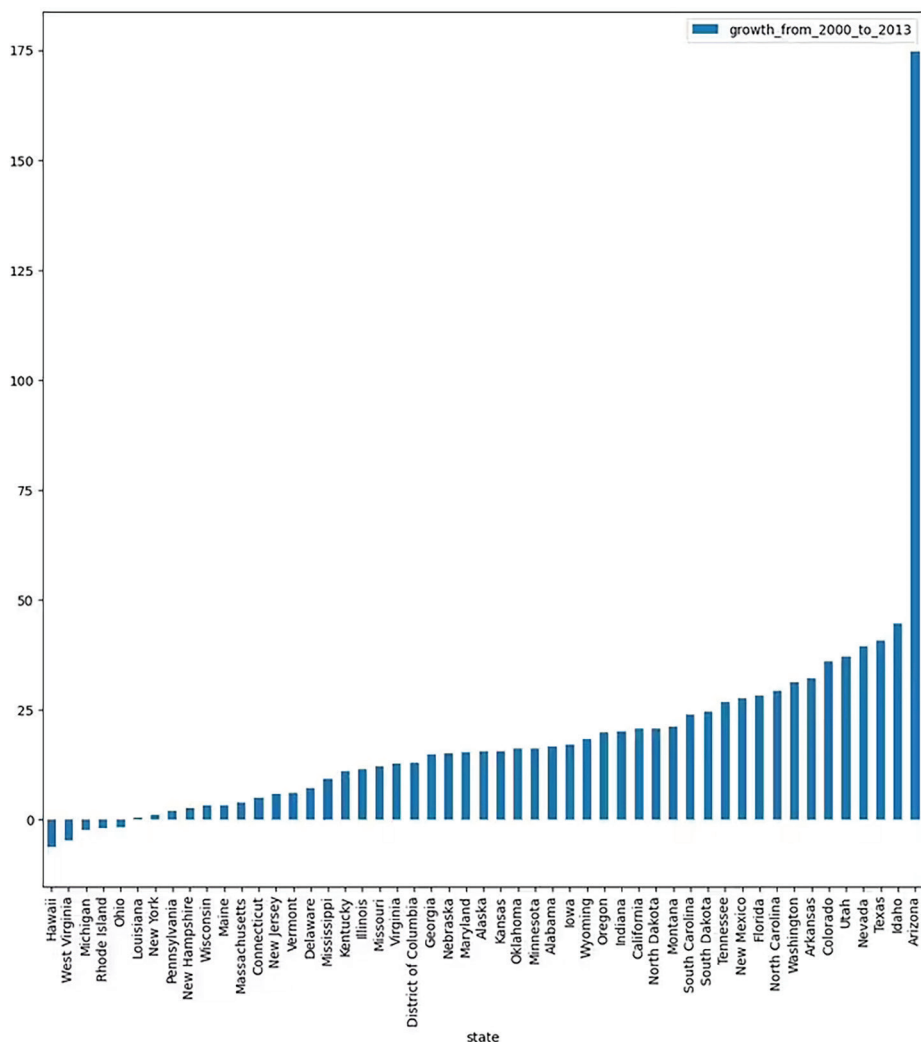
Вывод:



Упражнение 43.3

```
(
df[['state', 'growth_from_2000_to_2013']]
.set_index('state')
['growth_from_2000_to_2013'].str.rstrip('%').replace('', np.nan)
.dropna()
.astype(np.float16)
.reset_index()
.groupby('state').mean()
.sort_values(by='growth_from_2000_to_2013')
.plot.bar(figsize=(12,12))
)
```

Вывод:



УПРАЖНЕНИЕ 44. Погода в ящиках с усами

Я постоянно повторяю своим студентам, и в этой книге не раз писал об этом, что аналитик должен знать свои данные. И один из лучших способов изучить их – построить диаграмму размаха. В этом упражнении мы с помощью этой диаграммы попробуем проникнуть глубже в набор данных, посвященный погоде зимой в сезоне 2018–2019 годов в трех городах США. Мы проанализируем погодные условия в Чикаго, Лос-Анджелесе и Бостоне и увидим, насколько эти города отличаются в плане климатических условий (спойлер: в Чикаго действительно очень холодно зимой).

Итак, вам нужно будет сделать следующее.

1. Загрузите в датафрейм данные о погоде в Чикаго (файл `chicago.il.csv`). Нам понадобятся только столбцы `date_time`, `mintemp` и `maxtemp`. Установите в качестве индекса столбец `date_time` и дайте столбцам с минимальной и максимальной температурами имена `mintemp` и `maxtemp` соответственно.
2. Постройте диаграмму размаха на основе значений минимальной температуры в Чикаго за выбранный период.
3. Найдите значения, представленные на диаграмме размаха отдельными точками, характеризующими выбросы.
4. Постройте диаграмму размаха на основе значений минимальной температуры в Чикаго в феврале 2019 года.
5. Разместите на диаграмме размаха два ящика с усами, соответствующих минимальной и максимальной температурам в Чикаго с начала февраля по конец марта.
6. Загрузите данные о погоде в Лос-Анджелесе и Бостоне (файлы `los+angeles, ca.csv` и `boston, ma.csv` соответственно). Создайте единый датафрейм, содержащий данные о всех трех городах, а также новый столбец с именем `city`, в котором будет указано название города.
7. Соберите описательную статистику по столбцам `mintemp` и `maxtemp` с группировкой по городам.
8. Выведите на двух диаграммах размаха ящики с усами для минимальной и максимальной температур по всем трем городам. На одной диаграмме – данные о минимальных температурах, на второй – о максимальных.

Подробный разбор

В этом упражнении мы объединим вместе техники, которые использовали ранее. В частности, мы загрузим данные с помощью функции `read_csv` с применением большого числа параметров, объединим в один датафрейм информацию из разных файлов CSV и установим в качестве индекса столбец с датой и временем. Но в основном мы сделаем упор на создание диаграмм размаха, которые позволят вам лучше понять свои данные.

Сначала нас попросили загрузить в датафрейм данные о погоде в Чикаго (файл `chicago.il.csv`), установить в качестве индекса столбец `date_time` и дать столбцам с минимальной и максимальной температурами имена `mintemp` и `maxtemp` соответственно. Сделаем это:

```
filename = '../data/chicago,il.csv'
df = pd.read_csv(filename,
                  usecols=[0, 1, 2],
                  header=0,
                  names=['date_time', 'maxtemp', 'mintemp'],
                  parse_dates=['date_time'],
                  index_col=['date_time'])
```

Все эти параметры функции `read_csv` мы уже использовали в разных комбинациях, а здесь объединили вместе. Сперва мы указали, что нас интересуют только первые три колонки в файле, при этом обратились мы к ним по порядковым номерам. Имена мы им дадим сами при помощи параметра `names`. Также мы воспользовались параметром `parse_dates` для преобразования столбца `date_time` в тип с датой и временем, а затем установили его в качестве индекса. Кроме того, с помощью параметра `header=0` мы указали, что в первой строке файла содержатся заголовки, которые не нужно воспринимать как данные.

В результате мы получили датафрейм, состоящий из 728 строк и двух столбцов. Период наблюдений – с полночи 12 декабря 2018 года до 9 ч вечера 11 марта 2019 года с интервалом в три часа.

Затем нас попросили построить диаграмму размаха на основе значений минимальной температуры в Чикаго за выбранный период. Это можно сделать очень просто (см. рис. 11.8):

```
df['mintemp'].plot.box()
```

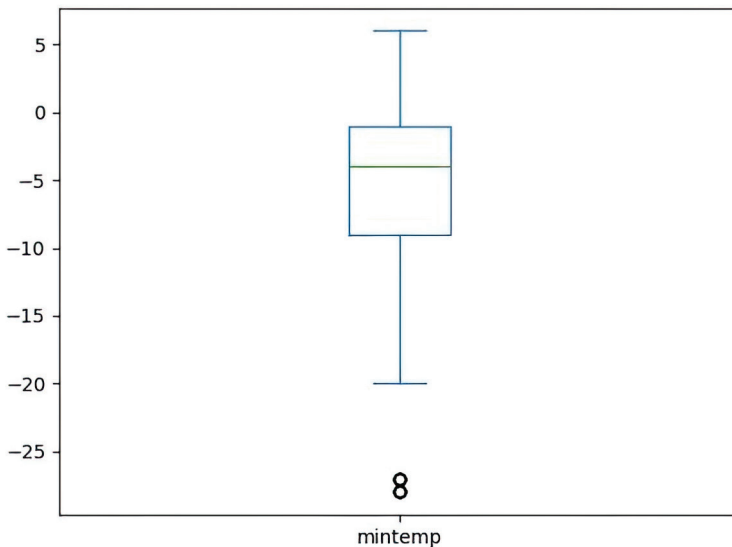


Рис. 11.8. Диаграмма размаха по минимальным температурам в Чикаго

На представленном графике мы видим всю необходимую описательную статистику по нашим данным. Таким образом, мы можем заключить, что нижняя планка температуры в Чикаго колеблется в основном в диапазоне от -20 до $+5$ °C. В то

же время под нижним усом мы видим пару выбросов. В реализации диаграммы размаха в pandas выбросами (outlier) считаются наблюдения, располагающиеся ниже, чем 25-й процентиль, из которого вычли полтора межквартильного размаха ($25\% - 1.5 * IQR$), и выше, чем 75-й процентиль, к которому прибавили полтора межквартильного размаха ($75\% + 1.5 * IQR$). Чтобы убедиться, что на графике эти точки отображаются корректно, нас попросили найти эти экстремальные значения температур. Сделаем это:

```
iqr = df['mintemp'].quantile(0.75) - df['mintemp'].quantile(0.25)

(
    df.loc[
        df['mintemp'] < df['mintemp'].quantile(0.25) - (iqr * 1.5),
        'mintemp'
    ]
)
```

Вывод:

```
date_time
2019-01-30 00:00:00    -28
2019-01-30 03:00:00    -28
2019-01-30 06:00:00    -28
2019-01-30 09:00:00    -28
2019-01-30 12:00:00    -28
..
2019-01-31 09:00:00    -27
2019-01-31 12:00:00    -27
2019-01-31 15:00:00    -27
2019-01-31 18:00:00    -27
2019-01-31 21:00:00    -27
Name: mintemp, Length: 16, dtype: int64
```

Мы видим информацию о двух днях (30 и 31 января), когда температура воздуха опускалась до отметок -28 и -27° соответственно – не просто холодно, а очень холодно, даже для Чикаго. Таким образом, на диаграмме мы видели в качестве выбросов именно эти наблюдения.

После этого нас попросили построить диаграмму размаха на основе значений минимальной температуры в Чикаго в феврале 2019 года. Ограничим строки при помощи среза и построим график:

```
(
    df
    .loc[
        '01-Feb-2019': '28-Feb-2019', ❶
        'mintemp' ❷
    ]
    .plot.box()
)
```

❶ Селектор строк: февраль 2019 года.

❷ Селектор столбцов: столбец mintemp.

В селекторе строк мы ограничили выбор диапазоном с 1 по 28 февраля. Здесь мы воспользовались тем, что в нашем датафрейме в качестве индекса установлен столбец с датой и временем, а это облегчает процесс извлечения строк при помощи срезов. Далее мы выбрали только столбец `mintemp` и вызвали для полученного в результате датафрейма из одного столбца метод `plot.box`. Диаграмма показана на рис. 11.9. Как видите, медианная февральская температура составила -5° , что соответствует климату в Чикаго.

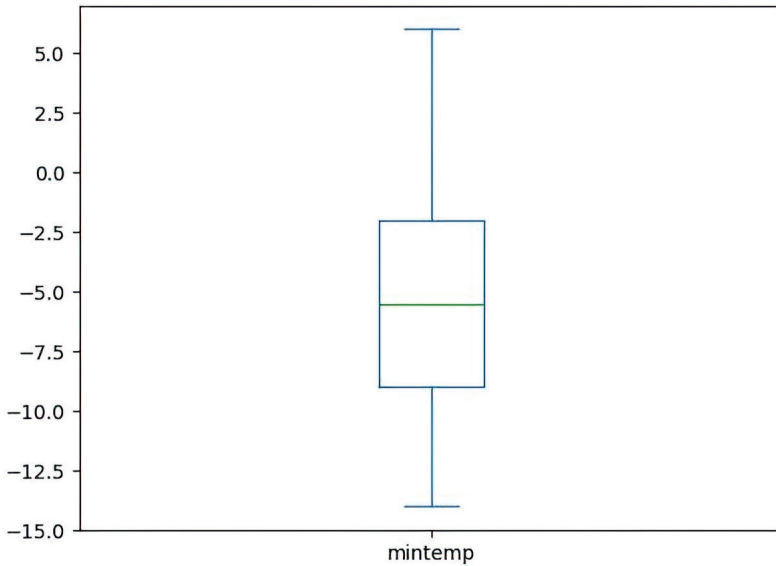


Рис. 11.9. Диаграмма размаха по минимальным температурам в Чикаго за февраль 2019 года

Далее нас попросили разместить на диаграмме размаха два ящика с усами, соответствующих минимальной и максимальной температурам в Чикаго с начала февраля по конец марта. Мы снова воспользуемся срезом для выбора временного интервала и извлечем два столбца, как показано ниже:

```
(
    df
    .loc['01-Feb-2019': '30-Mar-2019',
         ['mintemp', 'maxtemp']]
    .plot.box()
)
```

- ❶ Селектор строк: с февраля по март 2019 года.
- ❷ Селектор столбцов: столбцы `mintemp` и `maxtemp`.

Мы снова применили срез для фильтрации строк. В то же время в селекторе столбцов нам пришлось обернуть имена нужных столбцов в список. Передав новый датафрейм методу `plot.box`, мы получили два ящика с усами на одной диаграмме, как показано на рис. 11.10.

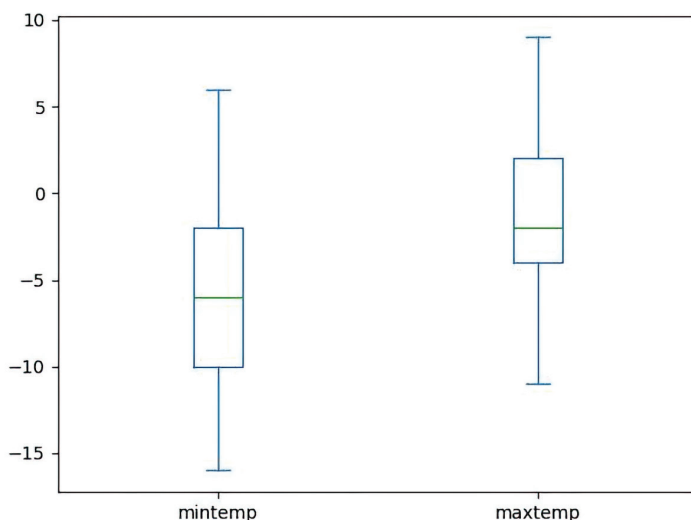


Рис. 11.10. Диаграмма размаха по минимальным и максимальным температурам в Чикаго за период с февраля по март 2019 года

Теперь пришло время добавить в наш набор данных информацию о погодных условиях в других городах. Нас как раз попросили загрузить данные о погоде в Лос-Анджелесе и Бостоне и создать единый датафрейм на основе информации о трех городах. Чтобы мы могли легко отличать города в нашем объединенном наборе, добавим в него столбец `city` с названиями городов. Поскольку у нас уже загружены данные по Чикаго, добавим столбец и инициализируем его начальными значениями:

```
df['city'] = 'Chicago'
```

Для загрузки данных о других двух городах мы воспользуемся циклом, что типично для Python, но не слишком распространено при работе с `pandas`. Пройдем по названиям файлов и загрузим из них данные:

```
for city_stem in ['los+angeles,ca', 'boston,ma']:
    new_df = pd.read_csv(f'../data/{city_stem}.csv',
                        usecols=[0, 1, 2],
                        header=0,
                        names=['date_time', 'maxtemp', 'mintemp'],
                        parse_dates=['date_time'],
                        index_col=['date_time'])
    new_df['city'] = city_stem.split(',')[0].replace('+', ' ').title()
    df = pd.concat([df, new_df])
```

Поясним, что мы сделали.

1. Собрали в список имена нужных нам файлов без расширения CSV.
2. Воспользовались циклом `for` для перебора файлов.
3. Для каждого файла вызвали функцию `read_csv`, передав ей текущее имя файла и дополнив его расширением.

4. Снова выбрали нужные столбцы, присвоили полю `date_time` тип с датами и временем и установили его в качестве индекса.
5. Добавили столбец с именем `city` и заполнили его именами городов, для чего применили целый ряд строковых функций к итерационной переменной `city_stem`. А именно:
 - вызвали метод `str.split`, взяв только первую составляющую до запятой, отделяющей название города от названия штата;
 - заменили символы `+` на пробел для правильного преобразования названий городов, состоящих из двух и более слов;
 - применили метод `str.title` для написания названий городов с большой буквы.
6. В конце мы воспользовались функцией `pd.concat` для добавления нового датафрейма к существующему. В результате мы получили объединенный датафрейм с информацией о погоде во всех трех городах и дополнительным столбцом `city` с названием города.

Седьмым пунктом нас попросили собрать описательную статистику по столбцам `mintemp` и `maxtemp` с группировкой по городам в полученном датафрейме. Сделаем это:

```
df.groupby('city')[['mintemp', 'maxtemp']].describe()
```

Вывод:

	mintemp					...	maxtemp					
	count	mean	std	min	25%	...	min	25%	50%	75%	max	
city						...						
Boston	728.0	-3.142857	4.957195	-14.0	-6.0	...	-12.0	0.0	2.0	6.0	17.0	
Chicago	728.0	-5.076923	6.255857	-28.0	-9.0	...	-25.0	-3.0	0.0	3.0	9.0	
Los Angeles	728.0	10.637363	2.705200	4.0	9.0	...	12.0	15.0	16.0	19.0	23.0	

[3 rows x 16 columns]

В результате мы получили датафрейм, состоящий из трех строк – по одной на каждый город. Столбцы собраны в множественный индекс так, что сначала показываются все характеристики для поля `mintemp`, а затем – для поля `maxtemp`. И хотя эта информация очень полезна, она не столь наглядна, как диаграмма размаха. В заключение нас попросили вывести на двух диаграммах размаха ящики с усами для минимальной и максимальной температур по всем трем городам. На одной диаграмме – данные о минимальных температурах, на второй – о максимальных. Это звучит сложно, а делается очень легко:

```
(
    df
    .plot.box(column=['mintemp', 'maxtemp'],
              by='city')
)
```


В итоге мы получили две диаграммы с тремя ящиками с усами на каждой, что видно на рис. 11.11. Неудивительно, что в Бостоне погода оказалась чуть получше, чем в Чикаго. Но Лос-Анджелес опередил оба города с большим отрывом.

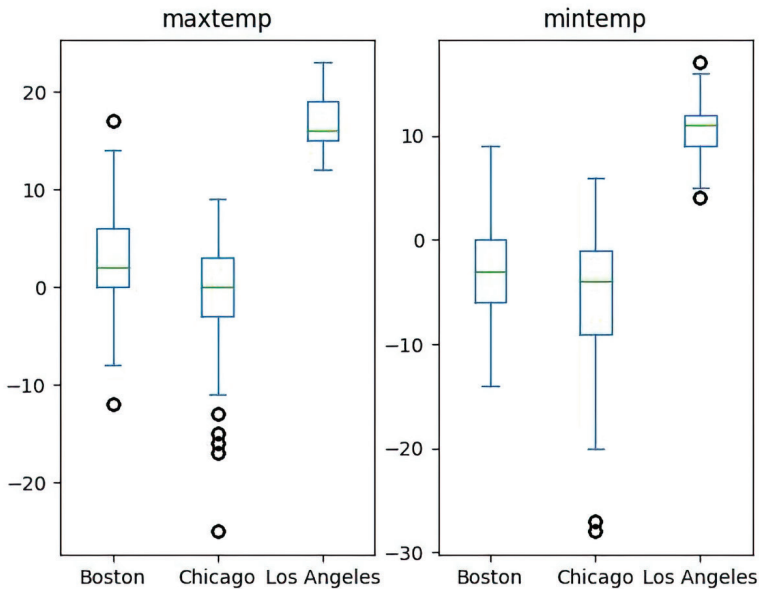


Рис. 11.11. Диаграмма размаха по минимальным и максимальным температурам для трех разных городов

Решение

```
filename = '../data/chicago,il.csv'
df = pd.read_csv(filename,
                  usecols=[0, 1, 2],
                  header=0,
                  names=['date_time', 'maxtemp', 'mintemp'],
                  parse_dates=['date_time'],
                  index_col=['date_time'])

df['mintemp'].plot.box()

iqr = df['mintemp'].quantile(0.75) - df['mintemp'].quantile(0.25)

(
    df.loc[
        df['mintemp'] < df['mintemp'].quantile(0.25) - (iqr * 1.5),
        'mintemp'
    ]
)

(
```

1

2

3

4

5

6

7

```

df
.loc['01-Feb-2019':'28-Feb-2019',
     'mintemp']
.plot.box()
)
(
df
.loc['01-Feb-2019':'30-Mar-2019',
     ['mintemp', 'maxtemp']]
.plot.box()
)
df['city'] = 'Chicago'
for city_stem in ['los+angeles,ca', 'boston,ma']:
    new_df = pd.read_csv(f'../data/{city_stem}.csv',
                        usecols=[0, 1, 2],
                        header=0,
                        names=['date_time', 'maxtemp', 'mintemp'],
                        parse_dates=['date_time'],
                        index_col=['date_time'])
    new_df['city'] = city_stem.split(',')[0].replace('+', ' ').title()
    df = pd.concat([df, new_df])
df.groupby('city')[['mintemp', 'maxtemp']].describe()
(
df
.plot.box(column=['mintemp', 'maxtemp'],
          by='city')
)

```

- ❶ Загружаем только три столбца.
- ❷ Явно говорим pandas, что в первой строке содержатся заголовки.
- ❸ Даем имена загруженным колонкам.
- ❹ Столбец date_time должен быть приведен к типу datetime.
- ❺ Устанавливаем в качестве индекса столбец date_time.
- ❻ Строим диаграмму размаха по минимальным температурам в Чикаго.
- ❼ Находим выбросы.
- ❽ Строим диаграмму размаха по минимальным температурам в Чикаго за февраль.
- ❾ Строим диаграмму размаха по минимальным и максимальным температурам в Чикаго за февраль и март.
- ❿ Заполняем новый столбец city значениями 'Chicago'.
- ⓫ Добавляем данные еще по двум городам.
- ⓫ Загружаем данные из файлов CSV.
- ⓫ Используем строковые методы Python для получения названия города из имени файла.
- ⓫ Добавляем новый датафрейм к существующему.
- ⓫ Извлекаем описательные статистики по температурам в разрезе городов.
- ⓫ Строим диаграмму размаха по минимальным и максимальным температурам в разрезе городов.

Дополнительные упражнения

1. Мы начинали работу с загрузки данных по температурам в Чикаго. А теперь начните с пустого датафрейма и загрузите в цикле данные по всем трем городам.
2. Для каждого города рассчитайте среднее и медианное значения для столбцов `mintemp` и `maxtemp`. Насколько эти значения близки? И если они отличаются, то в какую сторону?
3. Постройте линейчатую диаграмму, показывающую минимальные температуры во всех трех городах. На оси *x* расположите даты, а на оси *y* – температуру. Каждая линия на графике должна соответствовать определенному городу.

Ответы на дополнительные упражнения

Упражнение 44.1

```
df = DataFrame()

all_dfs = []

for city_stem in ['chicago,il', 'los+angeles,ca', 'boston,ma']:
    new_df = pd.read_csv(f'../data/{city_stem}.csv',
                        usecols=[0, 1, 2],
                        header=0,
                        names=['date_time', 'maxtemp', 'mintemp'],
                        parse_dates=['date_time'],
                        index_col=['date_time'])
    new_df['city'] = city_stem.split(',')[0].replace('+', ' ').title()
    all_dfs.append(new_df)

df = pd.concat(all_dfs)
df
```

Вывод:

	maxtemp	mintemp	city
date_time			
2018-12-11 00:00:00	1	-2	Chicago
2018-12-11 03:00:00	1	-2	Chicago
2018-12-11 06:00:00	1	-2	Chicago
2018-12-11 09:00:00	1	-2	Chicago
2018-12-11 12:00:00	1	-2	Chicago
...	...	...	...
2019-03-11 09:00:00	8	2	Boston
2019-03-11 12:00:00	8	2	Boston
2019-03-11 15:00:00	8	2	Boston
2019-03-11 18:00:00	8	2	Boston
2019-03-11 21:00:00	8	2	Boston

[2184 rows x 3 columns]

Упражнение 44.2

```
df.groupby('city')[['mintemp', 'maxtemp']].agg(['mean', 'median'])
```

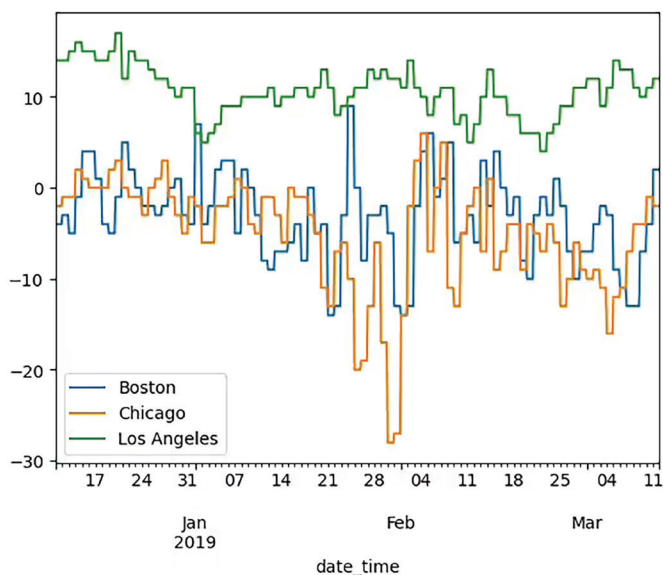
Вывод:

	mintemp		maxtemp	
city	mean	median	mean	median
Boston	-3.142857	-3.0	2.868132	2.0
Chicago	-5.076923	-4.0	-0.736264	0.0
Los Angeles	10.637363	11.0	17.054945	16.0

Упражнение 44.3

```
df.groupby('city')['mintemp'].plot.line(legend=True)
```

Вывод:



УПРАЖНЕНИЕ 45. Анализируем стоимость поездок на такси с помощью графиков

Мы уже не раз в этой книге обращались к набору данных, посвященному поездкам в нью-йоркских такси. В этом упражнении мы проанализируем данные из этого любопытного набора визуально, построив несколько полезных графиков и диаграмм. Визуализация данных позволяет не только представить информацию в удобном для понимания виде сторонним наблюдателям, но и лучше разобраться в ней самим аналитикам. При взгляде на графики вы можете обнаружить неочи-

данные для себя зависимости внутри данных, а ответы на поставленные вопросы могут приходить сами собой, да еще и приносить с собой новые уточняющие вопросы.

Итак, что вам нужно сделать.

1. Загрузить данные из всех четырех файлов с информацией о поездках на такси в один датафрейм. Вам понадобятся следующие столбцы: `tpep_pickup_datetime`, `passenger_count`, `trip_distance`, `fare_amount`, `extra`, `mta_tax`, `tip_amount`, `tolls_amount`, `improvement_surcharge`, `total_amount` и `congestion_surcharge`.
2. Построить столбчатую диаграмму, показывающую общую стоимость поездок на такси в каждом месяце каждого года. Пропуски на диаграмме – это нормальное явление.
3. Построить столбчатую диаграмму, показывающую общее количество поездок на такси в каждом месяце каждого года.
4. Построить столбчатую диаграмму с накоплением, показывающую значения из столбцов `fare_amount`, `extra`, `mta_tax`, `tip_amount` и `tolls_amount` в каждом месяце каждого года.
5. Отобразить на столбчатой диаграмме с накоплением значения столбцов `fare_amount`, `extra`, `mta_tax`, `tip_amount` и `tolls_amount` для разной численности пассажиров.
6. Построить гистограмму, показывающую распределение процента оставляемых чаевых от 0 до 50 % включительно.

Подробный разбор

В этом упражнении мы будем визуализировать уже знакомые нам данные, касающиеся поездок на такси в Нью-Йорке. Для большего разнообразия и объема анализируемых данных нас попросили загрузить в датафрейм данные из всех четырех файлов CSV. Загрузим их в список при помощи генератора списков, как показано ниже:

```
filenames = ['../data/nyc_taxi_2019-01.csv',
              '../data/nyc_taxi_2019-07.csv',
              '../data/nyc_taxi_2020-01.csv',
              '../data/nyc_taxi_2020-07.csv']

all_dfs = [pd.read_csv(one_filename,
                        usecols=['tpep_pickup_datetime',
                                'passenger_count',
                                'trip_distance',
                                'fare_amount',
                                'extra',
                                'mta_tax',
                                'tip_amount',
                                'tolls_amount',
                                'improvement_surcharge',
                                'total_amount',
```

```
        'congestion_surcharge'],
        parse_dates=['tpep_pickup_datetime'])
for one_filename in filenames]
```

Здесь мы с помощью атрибута `usecols` загрузили только нужные колонки и привели к типу `datetime` столбец `tpep_pickup_datetime` (в этом упражнении нам не понадобится время высадки пассажиров). Мы получили список датафреймов, который объединим вместе с помощью функции `pd.concat`, как показано ниже:

```
df = pd.concat(all_dfs)
```

Теперь можно приступить к анализу. Сначала нас попросили построить столбчатую диаграмму, показывающую общую стоимость поездок на такси в каждом месяце каждого года. При подготовке данных для этой диаграммы нам нужно воспользоваться методом `groupby` применительно к двум столбцам, отражающим год и месяц поездки. Сделаем это так:

```
(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
)
```

В результате мы получили объект с группировкой, к которому можно применять нужные нам агрегации. Нам нужно посмотреть информацию об общей стоимости поездок, и это можно сделать следующим образом:

```
(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
    ['total_amount'].sum()
)
```

Вывод:

```
tpep_pickup_datetime  tpep_pickup_datetime
2001                  2                      3.80
2002                  2                    162.82
2003                  1                      0.00
2008                 12                     848.05
2009                  1                   2074.56
...
2020                  6                     142.10
                7                   14912738.33
                8                      42.76
2021                  1                      47.76
2088                  1                      20.60
Name: total_amount, Length: 30, dtype: float64
```

Как видите, мы получили данные о суммарной стоимости поездок в такси для каждой комбинации из года и месяца в нашем наборе данных. Вроде мы загрузи-

ли данные всего из четырех файлов, в каждом из которых должна присутствовать информация лишь об одном месяце. Откуда же у нас в выводе столько разных годов и месяцев? Это может быть сопряжено с различного рода ошибками как со стороны операторов, так и со стороны автоматизированной системы. Так что вас никогда не должны удивлять такие паразитные данные в ваших наборах.

Итак, нам требуется построить на основе полученных данных столбчатую диаграмму. Сделать это можно очень просто:

```
(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
    ['total_amount'].sum()
    .plot.bar(figsize=(10,10))
)
```

На рис. 11.12 показан вывод в виде диаграммы. Данные из индекса датафрейма расположились на оси *x*, а значения – на оси *y*.

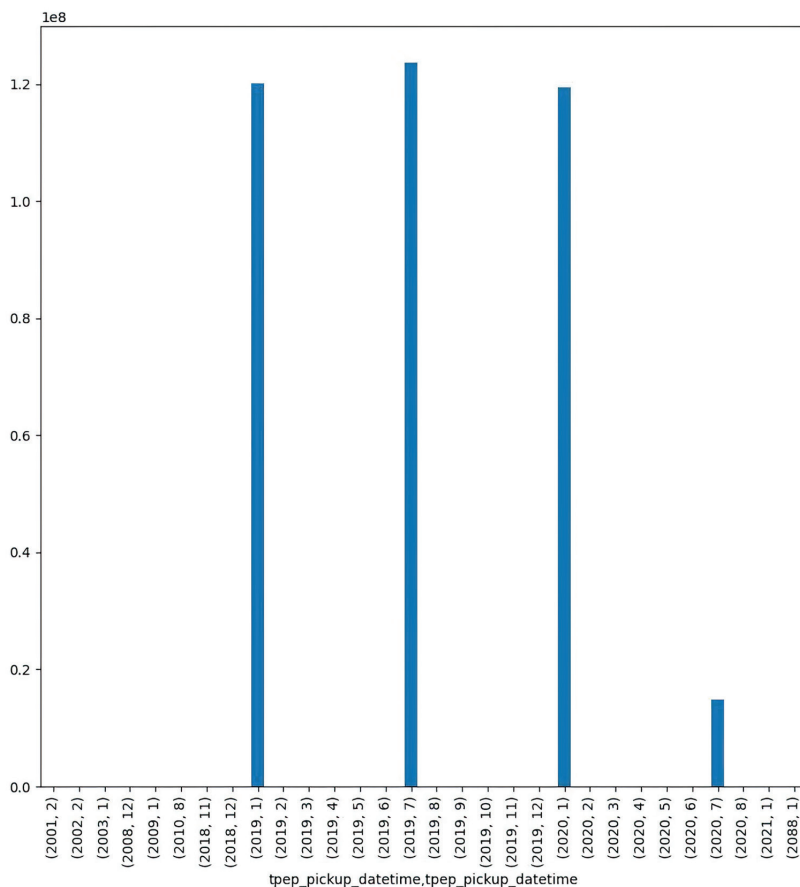


Рис. 11.12. Столбчатая диаграмма, показывающая общую стоимость поездок на такси в каждом месяце каждого года

Далее нас попросили построить столбчатую диаграмму, показывающую общее количество поездок на такси в каждом месяце каждого года. Снова начнем с группировки данных, а завершим вызовом метода `plot.bar`, как показано ниже:

```
(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
    ['passenger_count'].count()
    .plot.bar(figsize=(10,10))
)
```

Здесь нам необходимо подсчитать количество строк. Мы могли бы для этого воспользоваться методом `count`, но применительно к датафрейму он вернет нам количество элементов для каждого столбца. Таким образом, если в датафрейме будет десять столбцов, вызов метода `df.count()` вернет десять результатов, по одному для каждого столбца.

ПРИМЕЧАНИЕ. Поскольку метод `count` возвращает количество заполненных значений, игнорируя значения `NaN`, его бывает удобно использовать для сравнения столбцов на предмет количества пропущенных значений.

Нам не нужно несколько результатов, в связи с чем мы и выбрали единственный столбец `passenger_count` для подсчета количества строк. Могли выбрать и другой. Итоговая диаграмма показана на рис. 11.13.

Хотя на оси *x* у нас расположились те же годы и месяцы, что и на предыдущей диаграмме, сами значения в столбиках отличаются. К примеру, хотя в июле 2019 года пассажиры в такси заплатили максимальную общую сумму за весь исследуемый период, в плане количества поездок этот месяц занял лишь третье место. Также мы видим, что вследствие начала пандемии годом позже существенно снизились показатели количества поездок и затраченных сумм на такси.

Обычно, когда мы говорим о стоимости поездок на такси, мы анализируем значения в столбце `total_amount`. Однако в этом столбце содержится уже итоговая сумма, которую пассажир должен заплатить за услугу. И хотя пассажиры редко задумываются об этом, их плата за проезд включает в себя сразу несколько составляющих. И в следующем задании нас попросили разбить столбики на диаграмме на сегменты, соответствующие каждой составляющей в общей сумме.

Мы снова воспользуемся группировкой, чтобы правильно заложить основу расчетов:

```
(
    df.groupby([df['tpep_pickup_datetime'].dt.year,
               df['tpep_pickup_datetime'].dt.month])
)
```

Поскольку нам нужно выделить пять составляющих в общей стоимости поездки, мы оставим в нашем датафрейме только нужные столбцы (`fare_amount`, `extra`, `mta_tax`, `tip_amount` и `tolls_amount`) следующим образом:

```
(
    df.groupby([df['tpep_pickup_datetime'].dt.year,
```



```

        df['tpep_pickup_datetime'].dt.month])
[['fare_amount', 'extra', 'mta_tax',
  'tip_amount', 'tolls_amount']]
)

```

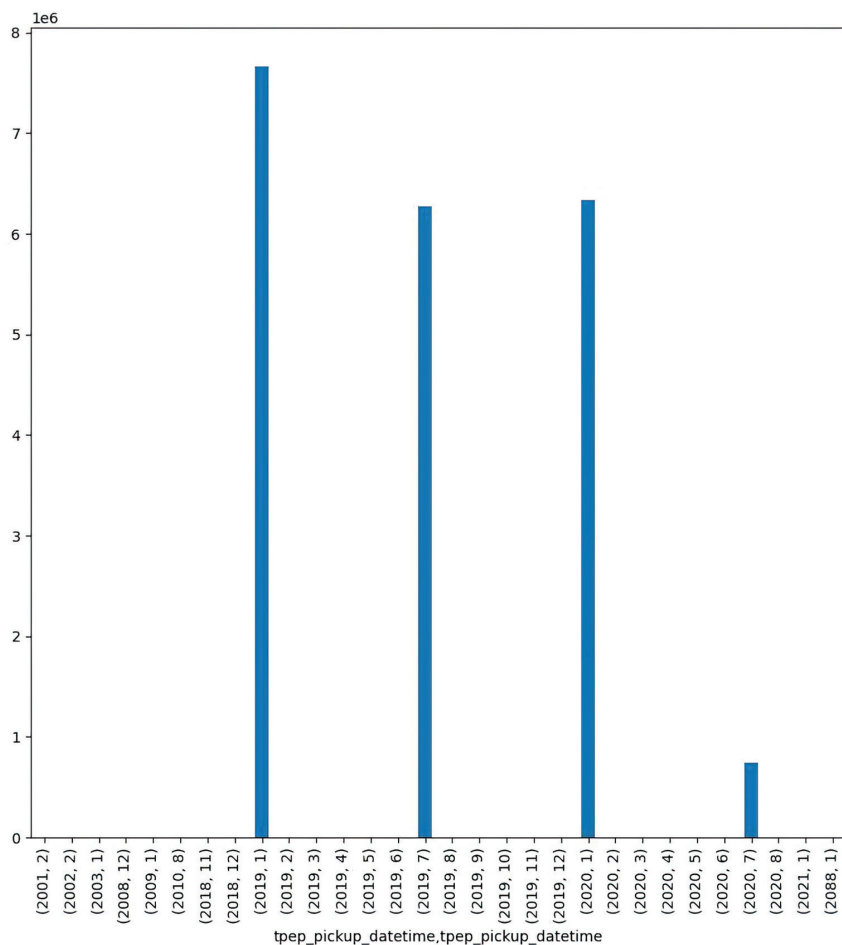


Рис. 11.13. Столбчатая диаграмма, показывающая количество поездок на такси в каждом месяце каждого года

После этого воспользуемся методом `sum`, который рассчитает итоги по всем пяти колонкам для каждой комбинации года и месяца, и применим метод `plot.bar`:

```

(
    df.groupby([df['tpep_pickup_datetime'].dt.year,
                df['tpep_pickup_datetime'].dt.month])
    [['fare_amount', 'extra', 'mta_tax',
      'tip_amount', 'tolls_amount']].sum()
    .plot.bar(stacked=True, figsize=(10,10))
)

```

Единственное отличие в вызове метода `plot.bar` здесь состоит в передаче параметра `stacked=True`, с помощью которого столбики обретают накопление, т. е. складываются из составляющих. Каждая составляющая выделяется своим цветом, и `pandas` выводит легенду для диаграммы с перечислением этих составляющих. Таким образом, мы можем наблюдать не только общую стоимость поездок, но и источники, из которых она складывается, что видно на рис. 11.14. Рассмотрите эту диаграмму. Хотя основную часть стоимости поездки на такси составляет ее номинал (`fare_amount`), чаевые (`tip_amount`) тоже имеют немалый вес. Также мы видим, что в общую сумму включены разные налоги и сборы.

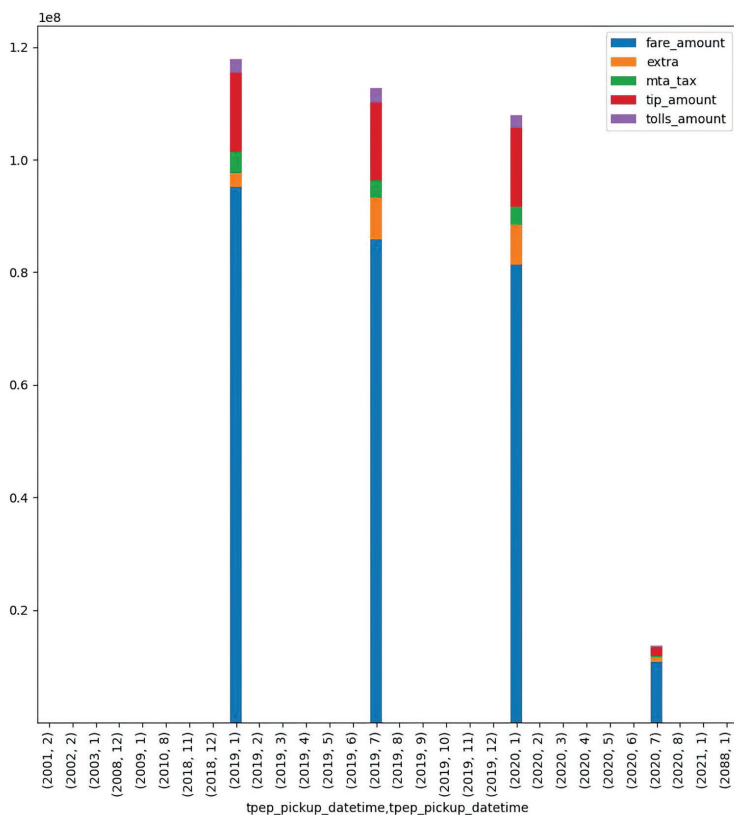


Рис. 11.14. Столбчатая диаграмма с накоплением, показывающая составляющие общей стоимости поездок на такси по годам и месяцам

Далее нас попросили отобразить на столбчатой диаграмме с накоплением значения тех же столбцов, но не для временных отрезков, а для разной численности пассажиров в такси. Иными словами, нам нужно выполнить группировку по столбцу `passenger_count`. Это можно сделать похожим образом, как показано ниже:

```
(
    df
    .groupby(df['passenger_count'])
    [['fare_amount', 'extra', 'mta_tax',
```

```

    'tip_amount', 'tolls_amount']]).sum()
).plot.bar(stacked=True, figsize=(10,10))
)

```

Результат показан на рис. 11.15.

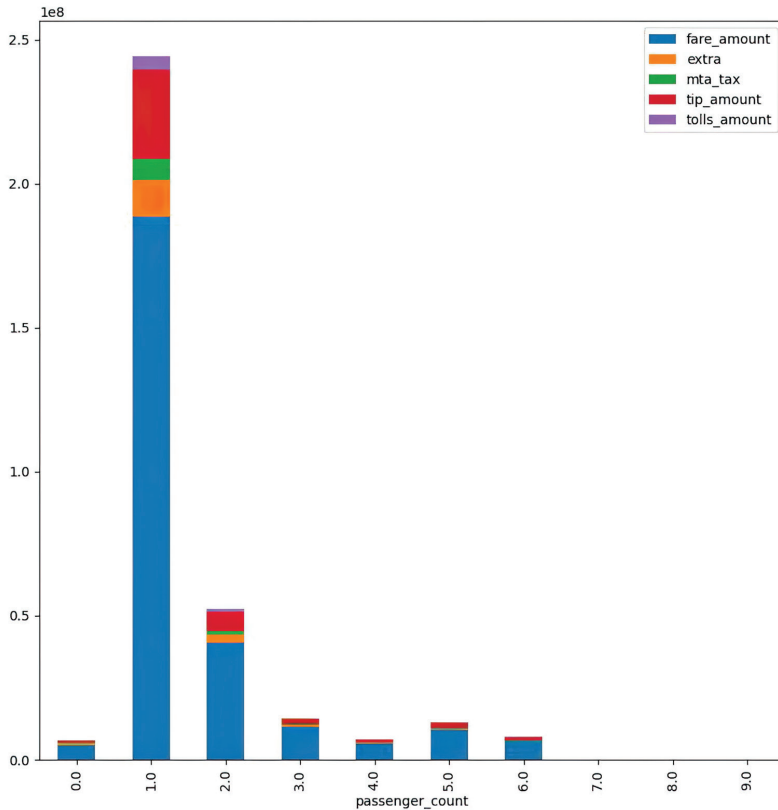


Рис. 11.15. Столбчатая диаграмма с накоплением, показывающая составляющие общей стоимости поездок на такси по численности пассажиров

В заключение нас попросили построить гистограмму, показывающую распределение процента оставляемых чаевых от 0 до 50 % включительно. Для этого нам необходимо рассчитать процент чаевых для каждой поездки и оставить только строки, в которых этот процент укладывается в указанный диапазон. Легче всего можно сделать это при помощи нового столбца с именем `tip_percentage`, в котором мы поделим значение из столбца `tip_amount` на значение из столбца `fare_amount`. Но в реальности нас может ждать немало сюрпризов – от наличия в полях пропущенных значений и до нулевых значений в столбце `fare_amount`, в результате чего мы будем получать значения `np.inf`. Во избежание таких ситуаций сначала избавимся от поездок, в которых стоимость меньше или равна нулю:

```
df = df[df['fare_amount'] > 0]
```

После этого создадим новый столбец с именем `tip_percentage`, зная, что не получим в нем значения `np.inf`:

```
df['tip_percentage'] = df['tip_amount'] / df['fare_amount']
```

Наконец, выведем на гистограмму с помощью метода `plot.hist` только поездки с долей чаевых, не превышающей 50 %:

```
(
    df
    .loc[
        df['tip_percentage'] <= .50, ❶
        'tip_percentage'              ❷
    ]
    .plot.hist()
)
```

❶ Селектор строк.

❷ Селектор столбцов.

На полученной в результате гистограмме, показанной на рис. 11.16, мы видим, что самый высокий столбик соответствует нулевой отметке, что говорит о большой доле пассажиров, не дающих чаевые вовсе. Если не считать этого столбца, остальные значения расположились примерно в соответствии с нормальным распределением вокруг значений 20 и 25 %.

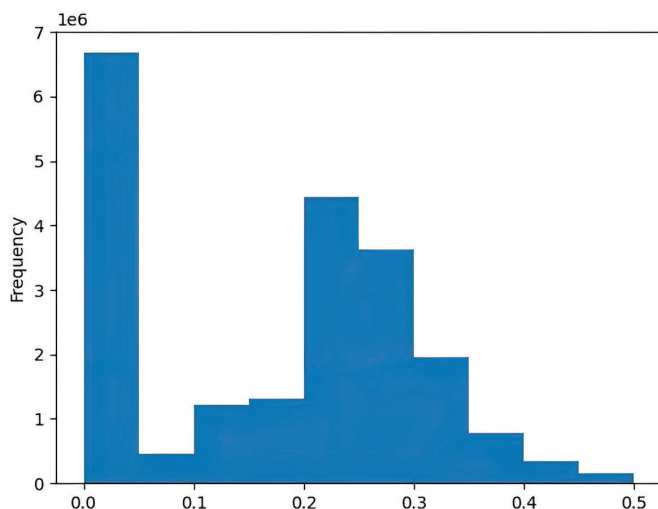


Рис. 11.16. Гистограмма, показывающая распределение долей чаевых в диапазоне от 0 до 50 %

Мы можем построить эту гистограмму и иначе, воспользовавшись атрибутом доступа `loc`, а также методом `assign` и анонимной функцией `lambda`:

```
(
    df
    .loc[lambda df_: df['fare_amount'] > 0]
```

```

        .assign(tip_percentage =
            lambda df_: df_['tip_amount'] / df_['fare_amount'])
        .loc[lambda df_: df_['tip_percentage'] <= 0.5,
            'tip_percentage']
        .plot.hist()
    )

```

Этот фрагмент кода легче понять, если прочитать его последовательно, строку за строкой.

1. Сначала мы обратились к атрибуту `loc` для нахождения всех строк в датафрейме `df`, в которых значение в столбце `fare_amount` больше нуля. При этом мы воспользовались анонимной функцией (`lambda`) и временной переменной `df_` внутри этой функции.
2. Далее мы прибегли к помощи метода `assign` с целью создания нового столбца с именем `tip_percentage`. Значения в этом столбце рассчитываются при помощи анонимной лямбда-функции, принимающей в качестве параметра всю текущую строку и возвращающей результат деления значения в столбце `tip_amount` на значение в столбце `fare_amount`. Столбец, созданный при помощи метода `assign`, в действительности не добавляется в датафрейм `df`, а остается в построенной нами цепочке методов для построения диаграммы.
3. После этого мы снова обращаемся к атрибуту `loc`, чтобы сохранить только те строки, в которых значение в столбце `tip_percentage` меньше или равно 0.5. Но в этот раз мы воспользовались и вторым аргументом атрибута `loc`, выбрав нужный нам столбец `tip_percentage`.
4. Наконец, мы вызываем метод `plot.hist`, который выводит гистограмму.

Решение

```

filenames = ['../data/nyc_taxi_2019-01.csv',
              '../data/nyc_taxi_2019-07.csv',
              '../data/nyc_taxi_2020-01.csv',
              '../data/nyc_taxi_2020-07.csv']

all_dfs = [pd.read_csv(one_filename,
                       usecols=['tpep_pickup_datetime',
                                'passenger_count',
                                'trip_distance',
                                'fare_amount',
                                'extra',
                                'mta_tax',
                                'tip_amount',
                                'tolls_amount',
                                'improvement_surcharge',
                                'total_amount',
                                'congestion_surcharge'],
                       parse_dates=['tpep_pickup_datetime'])

```

```
for one_filename in filenames]

df = pd.concat(all_dfs)

(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
    ['total_amount'].sum()
    .plot.bar(figsize=(10,10))
)

(
    df
    .groupby([df['tpep_pickup_datetime'].dt.year,
              df['tpep_pickup_datetime'].dt.month])
    ['passenger_count'].count()
    .plot.bar(figsize=(10,10))
)

(
    df.groupby([df['tpep_pickup_datetime'].dt.year,
                df['tpep_pickup_datetime'].dt.month])
    [['fare_amount', 'extra', 'mta_tax', 'tip_amount', 'tolls_amount']].sum()
    .plot.bar(stacked=True, figsize=(10,10))
)

(
    df
    .groupby(df['passenger_count'])
    [['fare_amount', 'extra', 'mta_tax',
      'tip_amount', 'tolls_amount']].sum()
    .plot.bar(stacked=True, figsize=(10,10))
)

df = df[df['fare_amount'] > 0]
df['tip_percentage'] = df['tip_amount'] / df['fare_amount']

df.loc[df['tip_percentage'] <= .50,
       'tip_percentage'].plot.hist()

(
    df
    .loc[lambda df_: df_['fare_amount'] > 0]
    .assign(tip_percentage = lambda df_: df_['tip_amount'] / df_['fare_
amount'])
    .loc[lambda df_: df_['tip_percentage'] <= 0.5,
         'tip_percentage']
    .plot.hist()
)
```

Дополнительные упражнения

1. Постройте столбчатую диаграмму, показывающую среднюю преодоленную такси дистанцию в разрезе дней недели за июль 2020 года. На оси *x* должны быть выведены названия дней недели.
2. Постройте диаграмму рассеяния на основе данных о поездках на такси в июле 2020 года для сравнения столбцов `trip_distance` и `total_amount`. В задании не нужно учитывать поездки со значениями в каком-либо из этих столбцов, меньшими или равными нулю или превышающими 500.
3. Постройте диаграмму рассеяния на основе данных о поездках на такси в июле 2020 года для сравнения столбцов `trip_distance` и `passenger_count`. В задании не нужно учитывать поездки со значениями в столбце `trip_distance`, меньшими или равными нулю или превышающими 500.

Корреляция и причинно-следственные связи

Вне зависимости от опыта работы с данными вы должны были хоть раз слышать фразу о том, что *корреляция* (correlation) не означает наличие причинно-следственных связей. Что это значит? И что вообще такое корреляция?

Если говорить общими словами, то две меры можно считать коррелирующими, если изменение одной сопровождается изменением другой. При этом если эти меры одновременно увеличиваются или уменьшаются, говорят, что между ними наблюдается положительная корреляция, а если подъем одной ведет к спаду другой и наоборот, то отрицательная.

В дополнение к типу корреляции она также может характеризоваться величиной, или силой. Иными словами, корреляция бывает сильной и слабой. Допустим, между уровнем дохода человека и площадью его жилища чаще наблюдается сильная корреляция, а между уровнем дохода и размером обуви – слабая.

Давайте рассмотрим простой пример. Чем больше электричества вы используете, тем больше в конце месяца вам придет счет. И если вы еще увеличите потребление, сумма в счете непременно вырастет. Таким образом, мы можем сказать, что между уровнем потребления электричества и суммой счета за него наблюдается сильная положительная корреляция.

А вот другой пример. Чем выше ваши доходы, тем выше вероятность наличия у вас в собственности частного самолета. Если вы мультимиллионер, в вашем ангаре с высокой степенью вероятности будет стоять личный самолет или даже несколько (ну, я в сериале «Наследники» это видел!). Таким образом, можно сказать, что с ростом доходов увеличивается и количество частных самолетов в собственности человека. И наоборот.

Здесь возникает опасный соблазн провести четкую причинно-следственную связь между двумя коррелирующими наблюдениями. Иногда такая связь действительно существует. Например, повышение объема расходуемой энергии является причиной увеличения суммы в счете за электричество.

Но в общем случае наличие корреляции между двумя наблюдениями не говорит напрямую о наличии между ними причинно-следственных связей. А даже если они есть, стоит с большим вниманием отнестись к направленности таких связей. К примеру, если бы я считал, что между наличием личного самолета и уровнем дохода есть четкая связь, быть может, мне следовало бы приобрести себе самолет? Это ведь повысило бы мои шансы стать миллиардером?

Существует немало примеров наличия сильной корреляции между событиями при полном отсутствии причинно-следственных связей. Самые любопытные из них собраны на сайте Тайлера Виджена (Tyler Vigen) по адресу <https://www.tylervigen.com/spurious-correlations>.

Факт необязательного наличия причинно-следственных связей для коррелирующих событий долгое время успешно использовался в табачной индустрии. Да, уже давно говорили, что курящие люди чаще заболевают раком. Но долгое время наличие причины и следствия медики установить не могли. Лишь после длительных многолетних экспериментов ученые доказали наличие связи между курением и онкологическими заболеваниями, и теперь мы можем с уверенностью связать эти факты.

Определение наличия причинно-следственных связей – дело весьма сложное и зачастую требующее большого количества времени и средств. Вы набираете большую группу людей, делите ее на две категории, одним даете лекарство, а другим – пустышку (плацебо), после чего долго и терпеливо проводите экспериментальные измерения нужных вам показателей.

К счастью, в статистике нас чаще интересуют не причинно-следственные связи, а корреляция как таковая. Если я знаю, что в онлайн-магазине пик продаж приходится на интервал с 12 до 13 ч, меня не будет интересовать, что именно к этому привело. Мне просто нужно знать об этом и научиться извлекать из этого выгоду.

Здесь возникает вопрос: а что вообще имеется в виду, когда говорится, что два числовых набора данных коррелируют друг с другом? Давайте рассмотрим два числовых набора, характеризующих максимальные и минимальные температуры в городе Модии на предстоящую неделю:

```
df = DataFrame(
    {'high': [19, 21, 24, 17, 14, 16, 16, 19, 16, 16, 15, 16, 18, 18],
     'low': [12, 9, 11, 12, 11, 11, 10, 8, 10, 8, 8, 6, 6, 7]})
```

Что будет означать корреляция между ними?

- если столбцы характеризуются положительной корреляцией, то в дни с высокой максимальной температурой будут наблюдаться и высокие минимальные температуры, и наоборот;
- если столбцы характеризуются отрицательной корреляцией, то в дни с высокой максимальной температурой будут наблюдаться низкие минимальные температуры, и наоборот.

Что касается величины корреляции, то при сильной корреляции большие изменения в первом наборе будут соответствовать большим изменениям во втором, а при слабой – большие изменения в одном будут соответствовать небольшим изменениям во втором.

Наиболее распространенной мерой корреляции, которую мы используем в этой книге, является *коэффициент корреляции Пирсона* (Pearson's correlation coefficient), который часто обозначается буквой *r*. Значение этого показателя может варьироваться от -1 (наиболее выраженная отрицательная корреляция) до 1 (наиболее выраженная положительная корреляция), а 0 означает полное отсутствие корреляции. Корреляция всегда вычисляется для двух наборов данных, что в случае с *pandas* означает два столбца в датафрейме.

Мы можем найти корреляцию для ожидаемых максимальных и минимальных температур в нашем датафрейме с помощью метода `corr`, как показано ниже:

```
df.corr()
```

Результатом будет новый датафрейм, в котором исходные имена столбцов присутствуют как в столбцах, так и в строках. На главной диагонали датафрейма, т. е. в ячейках, где имена в столбце и строке совпадают, располагаются единицы, что говорит (без особой пользы) о том, что любой столбец обладает предельной положительной корреляцией с самим собой. Больше нас интересуют другие пересечения, в которых можно увидеть коэффициенты корреляции между разными столбцами. В данном случае наш исходный датафрейм содержал лишь два столбца, так что результат оказался не самым впечатляющим:

```
      high      low
high  1.000000  0.105603
low   0.105603  1.000000
```

Как видим, коэффициент корреляции для максимальных и минимальных температур в нашем случае оказался равен 0.105603, что означает наличие очень слабой положительной корреляции. Если бы у нас было больше данных, возможно, мы смогли бы обнаружить более сильную корреляцию. Можно попробовать загрузить набор данных по Нью-Йорку, состоящий из 728 строк, чтобы убедиться в этом:

```
filename = '../data/new+york,ny.csv'

df = pd.read_csv(filename, usecols=[1, 2],
                  header=0,
                  names=['high', 'low'])
```

При вызове метода `df.corr()` для этого датафрейма мы получим иной результат:

```
      high      low
high  1.000000  0.874205
low   0.874205  1.000000
```

В данном случае мы видим очень сильную положительную корреляцию. Как может так оказаться, что в одном наборе данных корреляция слабая, а в другом – сильная?

Причин может быть множество. Возможно, в городе Модиин труднее предсказывать погоду. Или исходные данные были взяты за период, когда происходили какие-то погодные катаклизмы. Но мне кажется, что дело в недостаточном объеме данных, ведь у нас есть всего 13 наблюдений. Трудно делать выводы о корреляции по столь малому объему.

Почему нас интересует корреляция? Во-первых, она может побудить нас к каким-то действиям. К примеру, если мы знаем, что пик продаж в нашем магазине приходится на определенный временной интервал, возможно, в это время нужно подключать к обработке данных дополнительные серверы. Или делать скидки в другое время, чтобы повысить продажи.

Также информацию о корреляции можно использовать для идентификации сходства или наличия зависимостей между нашими наборами данных. Если два набора данных коррелируют, возможно, есть что-то, что объясняет их связь. Если же эта связь не очевидна, быть может, стоит провести дополнительные изыскания на этот счет.

Хотя наличие корреляции легко определяется математически, ее можно увидеть и на диаграмме рассеяния. При формировании такого графика мы выбираем столбцы, которые будут располагаться на оси x и на оси y . После этого мы выводим все наблюдения в виде меток на пересечении этих двух осей. Мы вряд ли можем надеяться, что итоговая линия будет идти строго по диагонали (это возможно только для идеально скоррелированных столбцов), но некий шаблон движения точек из нижнего левого угла (координата 0, 0) к верхнему правому мы заметить можем (для положительной корреляции). Или из верхнего левого в нижний правый (для отрицательной корреляции). Таким образом, диаграммы рассеяния помогают нам лучше понять наши данные. В pandas мы можем построить такой график на основе датафрейма с помощью метода `plot.scatter`, как показано ниже:

```
df.plot.scatter(x='high', y='low')
```

Результат показан на рис. 11.17. Здесь мы видим достаточно сильную положительную корреляцию между столбцами `high` и `low`.

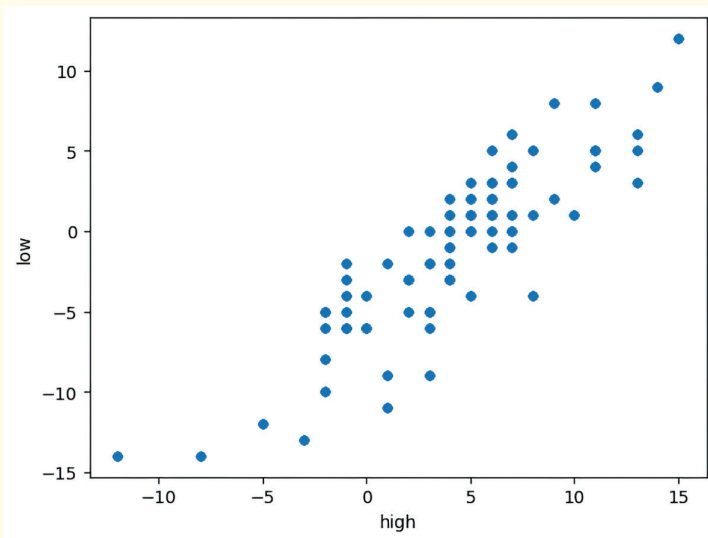


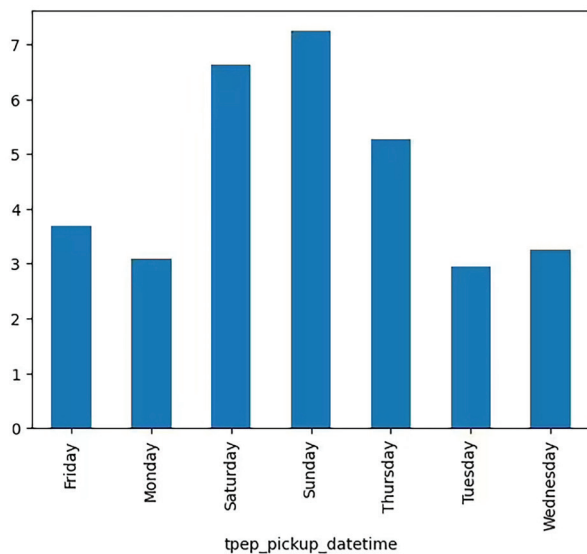
Рис. 11.17. Диаграмма рассеяния с сильной положительной корреляцией

Ответы на дополнительные упражнения

Упражнение 45.1

```
(
    df
    .groupby(df['tpep_pickup_datetime'].dt.day_name())
    ['trip_distance'].mean()
    .plot.bar()
)
```

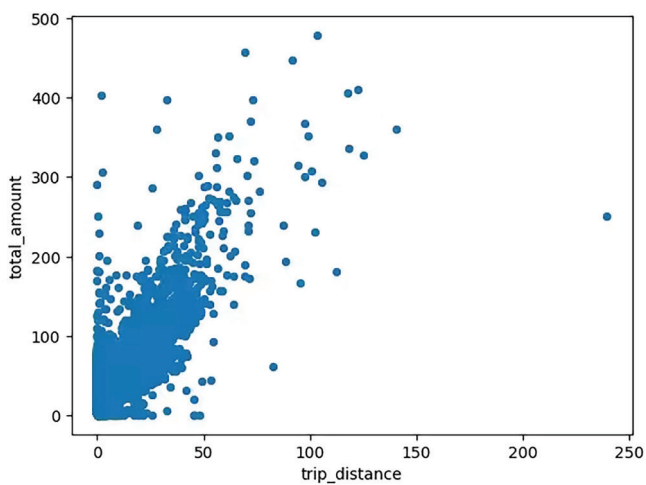
Вывод:



Упражнение 45.2

```
(
    df
    .loc[
        ((df['trip_distance'] > 0) &
         (df['total_amount'] > 0) &
         (df['trip_distance'] < 500) &
         (df['total_amount'] < 500))
    ]
    .plot.scatter(x='trip_distance', y='total_amount')
)
```

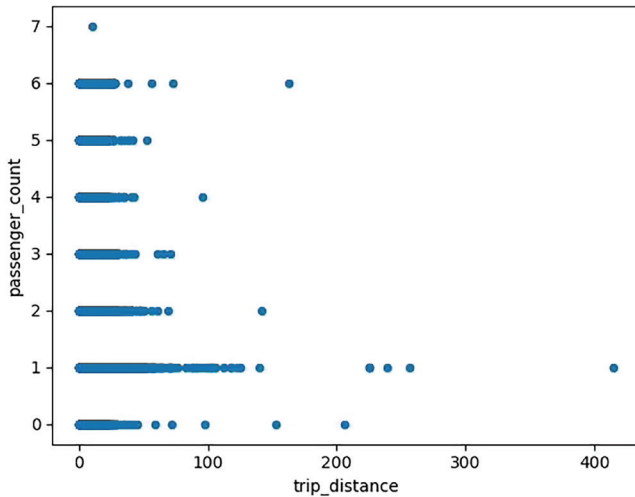
Вывод:



Упражнение 45.3

```
(
df
.loc[((df['trip_distance'] > 0) &
      (df['trip_distance'] < 500))]
.plot.scatter(x='trip_distance', y='passenger_count')
)
```

Вывод:



УПРАЖНЕНИЕ 46. Машины, нефть и мороженое

В этом упражнении мы попробуем ответить на вопрос, который, возможно, приходил в голову и вам: связаны ли цены на нефть и бензин с желанием людей ездить на своих личных авто? А заодно зададимся не самым логичным, на первый взгляд, вопросом о том, связаны ли цены на мороженое с ценами на нефть.

При решении задач мы будем не только определять наличие корреляции между наборами данных, но и воспользуемся разными техниками и приемами, изученными в предыдущих главах, включая обработку дат, выбор нужных строк и столбцов, очистку данных и объединение датафреймов. Вот что вам нужно будет сделать.

1. Загрузите набор данных с ценами на нефть (`wti-daily.csv`), которым мы пользовались в упражнении 41, в датафрейм. Установите для столбцов имена `date` и `oil`, приведите первый из них в формат даты и времени и сделайте его индексом.
2. Загрузите набор данных с ценами на мороженое в США (за половину галлона, т. е. 1.9 л) в отдельный датафрейм из файла `ice-cream.csv`. Установите для столбцов имена `date` и `iccream`, также приведите первый из них в формат даты и времени и сделайте его индексом.

3. Задайте для поля `icescream` тип с плавающей точкой, предварительно удалив строки, которые мешают сделать это.
4. Загрузите исторические данные о ежемесячном пробеге автомобилей в США из файла `miles-traveled.csv` в отдельный датафрейм. Установите для столбцов имена `date` и `miles`, приведите первый из них в формат даты и времени и сделайте его индексом.
5. Соберите один датафрейм на основе имеющихся трех. В качестве индекса в нем будут представлены даты, а столбцы будут следующие: `oil`, `icescream` и `miles`. Мы оставим в датафрейме только те даты, для которых заполнены все показатели.
6. Рассчитайте корреляцию для столбцов в датафрейме.
7. Постройте диаграмму рассеяния на основе столбцов `oil` и `icescream`.
8. Постройте диаграмму рассеяния на основе столбцов `oil` и `miles`.
9. Постройте диаграмму рассеяния на основе всех трех столбцов.

Подробный разбор

В этом упражнении мы возьмем три разных набора данных, объединим их в единый датафрейм и исследуем его на предмет наличия корреляций. Кстати, результаты нас немало удивят.

Но сначала необходимо загрузить данные. Я всегда предпочитаю загружать информацию в отдельные датафреймы, а затем объединять их. Это позволяет легко повторять процесс по шагам и исправлять возможные недочеты.

Первым мы загрузим набор данных с ценами на нефть, к которому уже обращались ранее. Для простоты объединения датафреймов нас попросили соблюсти некоторые требования к именованию столбцов. К примеру, во всех трех датафреймах мы приведем столбец `date` к календарному типу и установим его в качестве индекса. В первом датафрейме мы также переименуем столбцы, назвав их `date` и `oil`.

В большинстве случаев, особенно при работе с файлами CSV, в данных содержатся заголовки с именами исходных столбцов, которые можно использовать в функции `read_csv`. Это значительно облегчает отладку в дальнейшем. Но когда мы хотим переименовать столбцы с помощью параметра `names`, мы идентифицируем их по порядковым номерам. А чтобы строка с заголовком не была воспринята как данные, мы передаем параметр `header=0`, как показано ниже:

```
oil_filename = '../data/wti-daily.csv'
oil_df = pd.read_csv(oil_filename,
                    parse_dates=[0],
                    header=0,
                    index_col=0,
                    names=['date', 'oil'])
```

Быстрая проверка при помощи `oil_df.head()` и `oil_df.dtypes` позволит вам понять, что данные загрузились и имеют корректные типы данных. Теперь пришло время наполнить данными о ценах на мороженое в США второй датафрейм.

Формат файла с этой информацией очень похож на предыдущий – в нем также содержится два столбца. В первом столбце также располагаются даты, а именно

последние даты отчетных месяцев. Загрузим данные в датафрейм следующим образом:

```
ice_cream_filename = '../data/ice-cream.csv'
ice_cream_df = pd.read_csv(ice_cream_filename,
                           parse_dates=[0],
                           index_col=0,
                           header=0,
                           names=['date', 'icecream'])
```

Обратившись к атрибуту `ice_cream_df.dtypes`, мы можем понять, что столбец `icecream` не был приведен к типу с плавающей точкой. Вместо этого он обладает типом `object`. Обычно это означает, что какие-то значения в столбце помешали pandas выполнить ожидаемое приведение типов самостоятельно. Узнать, в чем именно проблема, можно, попытавшись выполнить приведение типов вручную с помощью метода `astype`, как показано ниже:

```
ice_cream_df['icecream'].astype(np.float64)
```

Конечно, у нас ничего не получится, и мы видим причину – наличие в столбце значения с одной только точкой вместо числа.

Давайте оставим только значения, которые содержат хотя бы одну цифру, в надежде на то, что сможем привести их к формату с плавающей точкой. Для этого создадим объект `Series` с логическими значениями на основе данных из столбца `ice_cream_df['icecream']`, в котором значениям `True` будут соответствовать вхождению как минимум с одной цифрой. Воспользуемся для этого методом `str.contains` с регулярными выражениями, для чего передадим параметр `regex=True`. После этого приведем значения к типу `np.float64` с помощью метода `astype`:

```
ice_cream_df = (
    ice_cream_df
    .loc[ice_cream_df['icecream'].str.contains(r'\d', regex=True)]
    .astype(np.float64)
)
```

Обратите внимание, что в качестве шаблона регулярной строки мы использовали так называемую сырую строку (`raw string`), которая задается с помощью буквы `r` перед открывающей кавычкой. В сырых строках не производится обработка специальных символов, таких как обратный слеш, что позволяет писать регулярные выражения намного проще, без дополнительного экранирования.

Далее мы создадим датафрейм с данными о ежемесячном пробеге автомобилей в США из файла `miles-traveled.csv`. Моя гипотеза строится на том, что при повышении цен на нефть люди, вероятно, менее склонны проводить за рулем много времени. Создадим третий датафрейм по образцу первых двух:

```
miles_filename = '../data/miles-traveled.csv'
miles_df = pd.read_csv(miles_filename,
                       parse_dates=[0],
                       index_col=0,
                       header=0,
                       names=['date', 'miles'])
```

Теперь пришло время объединить их. Ранее мы уже пользовались методом `join` для объединения двух датафреймов. Здесь же нам нужно собрать вместе сразу три датафрейма. Как это можно сделать?

Ответ очевиден – мы просто объединяем два датафрейма, а результат объединения с третьим. Если во всех трех датафреймах индексы совпадают, можно обойтись простейшим выражением, показанным ниже:

```
df = oil_df.join(ice_cream_df).join(miles_df)
```

При таком объединении мы заметим одну особенность, связанную с тем, что данные в датафрейме с ценами на нефть содержатся за каждый день, тогда как в двух других датафреймах – за каждый месяц. Таким образом, в результате объединения мы сохраним все значения из датафрейма с ценами на нефть, а в столбцах, измеряющих цены на мороженое и пробег автомобилей, в основном будут содержаться значения NaN, за исключением последних дней месяцев.

Существует несколько способов решения этой проблемы. Один из них, не самый элегантный, состоит в объединении датафреймов, как мы делали это раньше, с последующим удалением строк, в которых присутствуют пропущенные значения, как показано ниже:

```
df = oil_df.join(ice_cream_df).join(miles_df).dropna()
```

Второй способ предполагает указание в качестве первого датафрейма в цепочке объединений датафрейм с именем `ice_cream_df`, чтобы соответствующим образом ограничить индексы в итоговом наборе данных:

```
df = ice_cream_df.join(oil_df).join(miles_df)
```

Но лучше в таких случаях пользоваться операцией *внутреннего объединения* (`inner join`), в результате которой индекс будет содержать только те значения, которые присутствуют во всех трех датафреймах. Это можно сделать путем передачи параметра `how='inner'` при каждом вызове метода `join`, как показано ниже:

```
df = (
    oil_df
    .join(ice_cream_df, how='inner')
    .join(miles_df, how='inner')
)
```

Вывод:

	oil	icecream	miles
date			
1986-04-01	11.13	2.382	150277.0
1986-05-01	13.80	2.368	160459.0
1986-07-01	12.39	2.369	171114.0
1986-08-01	11.56	2.319	173977.0
1986-10-01	15.23	2.377	159434.0
...	...	...	...
2021-07-01	75.33	4.943	296475.0
2021-09-01	68.63	4.900	277979.0
2021-10-01	76.01	4.952	285760.0
2021-11-01	84.08	4.770	267647.0

```
2021-12-01  65.44      4.766  268398.0
```

```
[275 rows x 3 columns]
```

В результате мы получили датафрейм, в индексе которого содержится 275 уникальных значений с апреля 1986 года по декабрь 2021 года. Теперь, когда мы объединили наши данные, можно приступить к выявлению наличия корреляций в них. Для начала вызовем метод `corr` применительно ко всему датафрейму и посмотрим на результат:

```
df.corr()
```

Вывод:

	oil	icecream	miles
oil	1.000000	0.777347	0.645250
icecream	0.777347	1.000000	0.818383
miles	0.645250	0.818383	1.000000

В результирующем датафрейме содержится три столбца (`oil`, `icecream` и `miles`) и столько же строк. На пересечении столбцов и строк располагаются коэффициенты корреляции, значения которых находятся в диапазоне от -1 до 1 . Как видим, цены на нефть довольно сильно положительно коррелируют с пробегом автомобилей – коэффициент составляет 0.64 . В то же время корреляция между ценами на нефть и ценами на мороженое коррелирует еще сильнее – коэффициент равен 0.77 .

Но самый большой коэффициент корреляции наблюдается между ценами на мороженое и пробегом автомобилей – 0.818 . Это довольно много и позволяет сделать вывод о том, что при падении цен на мороженое люди меньше ездят на машинах.

Можем ли мы с уверенностью сказать, что в наших данных есть причинно-следственные связи? Я очень сомневаюсь в этом. Вряд ли цены на мороженое могут как-то влиять на то, насколько часто вы будете ездить на машине. Единственное объяснение, которое я нахожу, заключается в том, что летом, когда жарко, вы чаще покупаете мороженое, находясь за рулем, а цены на него растут при увеличении спроса.

Далее нас попросили построить две диаграммы рассеяния. Первая, связывающая столбцы `oil` и `icecream`, показана на рис. 11.18:

```
df.plot.scatter(x='oil', y='icecream')
```

Вторая диаграмма, показанная на рис. 11.19, сопоставляет поля `oil` и `miles`:

```
df.plot.scatter(x='oil', y='miles')
```

Хотя по этим графикам можно определить факт наличия положительной корреляции на глаз, думаю, вызов метода `corr` всегда может сказать нам больше.

Наконец, нас попросили построить диаграмму рассеяния на основе всех трех числовых столбцов. Результат показан на рис. 11.20:

```
from pandas.plotting import scatter_matrix
scatter_matrix(df)
```

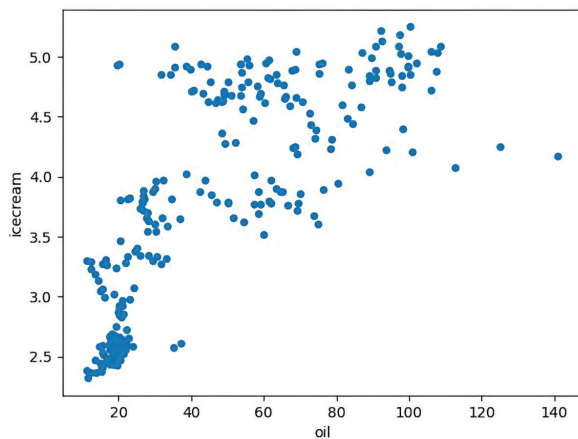



Рис. 11.18. Диаграмма рассеяния, сравнивающая цены на нефть с ценами на мороженое

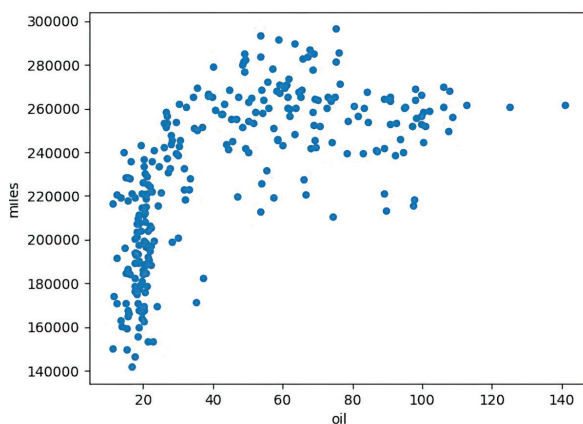


Рис. 11.19. Диаграмма рассеяния, сравнивающая цены на нефть с пробегом автомобилей

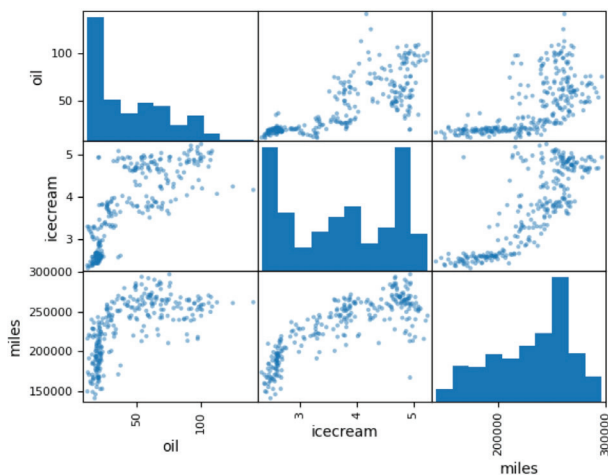


Рис. 11.20. Общая диаграмма рассеяния

Такая общая диаграмма рассеяния позволяет быстро определить взаимосвязи между столбцами в наборе данных. На главной диагонали, где обычно стоят единицы, мы видим гистограммы, показывающие распределение значений в каждом столбце.

Решение

```
oil_filename = '../data/wti-daily.csv'
oil_df = pd.read_csv(oil_filename,
                     parse_dates=[0],
                     header=0,
                     index_col=0,
                     names=['date', 'oil'])

ice_cream_filename = '../data/ice-cream.csv'
ice_cream_df = pd.read_csv(ice_cream_filename,
                           parse_dates=[0],
                           index_col=0,
                           header=0,
                           names=['date', 'icecream'])

ice_cream_df = (
    ice_cream_df
    .loc[ice_cream_df['icecream']]
    .str.contains(r'\d', regex=True)
    .astype(np.float64)
)

miles_filename = '../data/miles-traveled.csv'
miles_df = pd.read_csv(miles_filename,
                       parse_dates=[0],
                       index_col=0,
                       header=0,
                       names=['date', 'miles'])

df = (
    oil_df
    .join(ice_cream_df, how='inner')
    .join(miles_df, how='inner')
)

df.corr()

df.plot.scatter(x='oil', y='icecream')
df.plot.scatter(x='oil', y='miles')

from pandas.plotting import scatter_matrix
scatter_matrix(df)
```

- ❶ Игнорируем заголовки, поскольку мы сами даем колонкам имена.
- ❷ Устанавливаем индекс на первый столбец.

- ③ Именуем столбцы.
- ④ Используем регулярные выражения для исключения строк, в которых в столбце `icescream` нет цифр.
- ⑤ Приводим тип к `np.float64`.
- ⑥ Выполняем два внутренних объединения, создавая единый датафрейм с тремя столбцами.
- ⑦ Получаем корреляционную матрицу, сравнивающую все столбцы.
- ⑧ Строим диаграмму рассеяния по столбцам `oil` и `icescream`.
- ⑨ Строим диаграмму рассеяния по всем возможным комбинациям столбцов.

Дополнительные упражнения

1. Коррелирует ли значение месяца с каким-либо из этих трех столбцов?
2. Постройте диаграмму рассеяния для столбцов `icescream` и `miles`, добавив к ней цветовую карту с помощью параметра `Spectral`.
3. Вместо использования внутреннего объединения вы могли бы пойти по пути удаления из датафрейма `oil_df` всех строк, проходящихся не на последний день месяца. Как бы вы могли это сделать?

Библиотека Seaborn

Как мы уже говорили, библиотека `Matplotlib` является безусловным лидером среди библиотек визуализации в экосфере `Python`. При этом многим она кажется настолько сложной, что постичь ее в полной мере просто невозможно. В связи с этим стали появляться другие библиотеки, предназначенные для визуализации данных. Одной из лучших является библиотека *Seaborn* (<http://seaborn.pydata.org>), реализованная специалистом по работе с данными Майклом Васкомом (Michael Waskom) на базе API пакета `Matplotlib`.

До сих пор в этой книге мы пользовались инструментами визуализации, присутствующими в самом API `pandas`, который, как и *Seaborn*, использует под капотом библиотеку `Matplotlib`. Этот интерфейс упрощает многие механизмы и процедуры, принятые в `Matplotlib`, но в остальном соблюдает те же подходы и пользуется API этой библиотеки. В *Seaborn*, напротив, подошли к построению графиков иначе, заменив исходные вызовы `Matplotlib` и `pandas` на собственный набор функций и параметров.

Подобно тому как мы импортируем библиотеки `Numpy` и `pandas`, мы должны выполнить импорт *Seaborn* перед началом работы следующим образом:

```
import seaborn as sns
```

Если в `pandas` визуализация осуществляется при помощи атрибута `plot`, у которого мы вызываем соответствующие методы для того или иного вида графика, то в *Seaborn* вывод диаграмм организован более концептуально – с учетом аналитических выводов, которые мы хотим сделать на основе диаграмм. В этой связи у нас есть выбор из следующих четырех функций, определенных в `sns`:

- для визуализации на основе числовых столбцов мы можем воспользоваться функцией `sns.relplot`;
- визуализация отношений, включающих в себя категориальные столбцы, выполняется при помощи функции `sns.catplot`;

- визуализация отношений, включающих в себя категориальные столбцы, выполняется при помощи функции `sns.catplot`;
- распределение данных можно визуализировать посредством функции `sns.displot`;
- визуализация регрессионных моделей осуществляется с помощью функции `sns.regplot`.

Для более глубокого погружения в тему давайте загрузим набор данных с температурой воздуха и осадками из файлов CSV, как показано ниже:

```
import glob

all_dfs = []
all_filenames = glob.glob('../data/*/*.csv')

for one_filename in all_filenames:
    print(f'Loading {one_filename}...')
    city, state = one_filename.removeprefix('../data/').removesuffix('.csv').
split(',')
    one_df = pd.read_csv(one_filename,
                        usecols=[1, 2, 19],
                        names=['max_temp', 'min_temp', 'precipMM'],
                        header=0)
    one_df['city'] = city.replace('+', ' ').title()
    one_df['state'] = state.upper()
    all_dfs.append(one_df)

df = pd.concat(all_dfs)
```

Мы уже видели, какие выводы о взаимосвязях между столбцами можно делать на основе линейчатых диаграмм и диаграмм рассеяния. В Seaborn эти типы диаграмм объединены в общую функцию `relplot`. Давайте посмотрим, как можно построить диаграмму рассеяния для минимальных и максимальных температур в Чикаго:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df.loc[df['city'] == 'Chicago'])
```

Здесь мы передали функции `sns.relplot` три обязательных ключевых аргумента:

- аргумент `x` относится к столбцу в нашем датафрейме, который должен быть выведен на оси `x`;
- аргумент `y` – к столбцу, который должен быть выведен на оси `y`;
- аргумент `data` – это датафрейм, включающий в себя оба эти столбца.

Результат вызова этой функции показан на рис. 11.21.

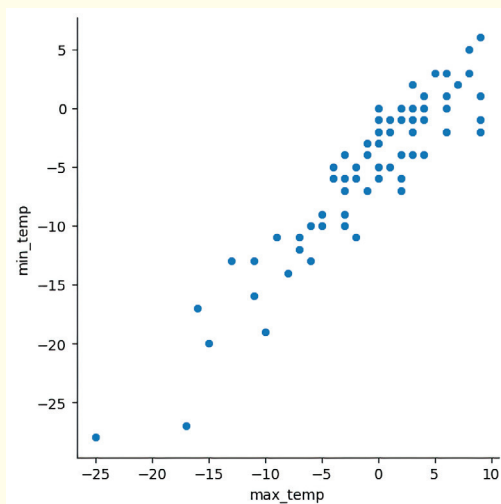


Рис. 11.21. Диаграмма рассеяния с погодой в Чикаго

Как видите, мы передали функции поднабор данных, содержащий лишь информацию о погоде в Чикаго. А что, если нам нужно посмотреть график по всем городам вместе? Можно передать функции весь датафрейм:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df)
```

Написать-то это легко, но для анализа данных полученная диаграмма, показанная на рис. 11.22, не представляет никакого интереса. Мы просто смешали данные о всех городах. К счастью, в Seaborn есть масса механизмов, позволяющих «очеловечить» графики, сделав их пригодными для визуального анализа.

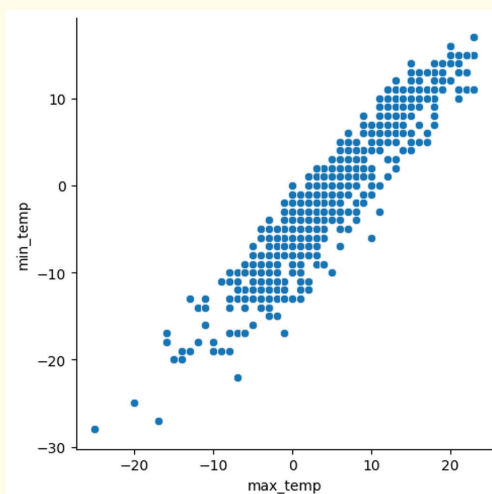


Рис. 11.22. Диаграмма рассеяния с погодой во всех городах

К примеру, мы можем попросить Seaborn выводить точки данных для городов с разделением по цветам при помощи ключевого аргумента `hue`, как показано ниже:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='city')
```

Результат показан на рис. 11.23.

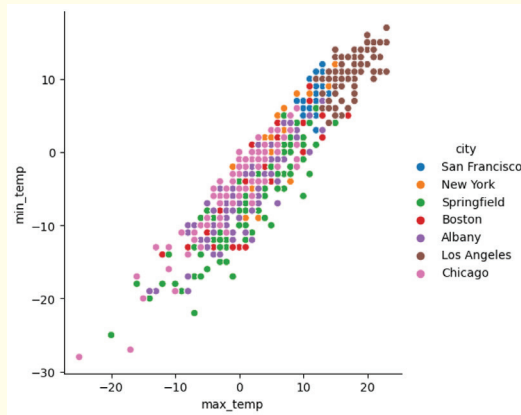


Рис. 11.23. Диаграмма рассеяния с погодой во всех городах с разделением по цветам

Кроме того, мы можем для каждого города задать разные типы меток, чтобы визуально они не смешивались. Для этого в Seaborn существует параметр `style`:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='city', style='city')
```

Результат показан на рис. 11.24.

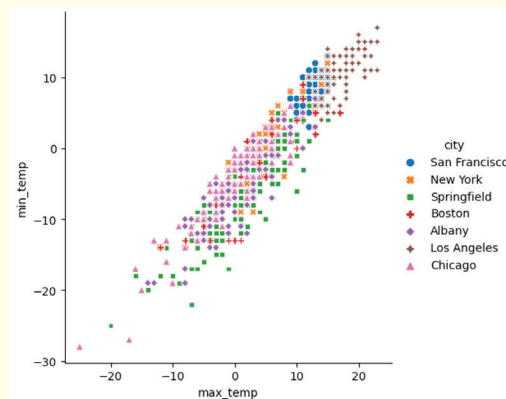


Рис. 11.24. Диаграмма рассеяния с погодой во всех городах с разделением по цветам и типам меток

При этом для разделения по цветам и меткам мы можем использовать разные столбцы, что позволяет произвести довольно глубокий визуальный анализ. Например, цветом мы можем выделить разные штаты, а типом меток – города, как показано ниже:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='state', style='city')
```

Результат показан на рис. 11.25.

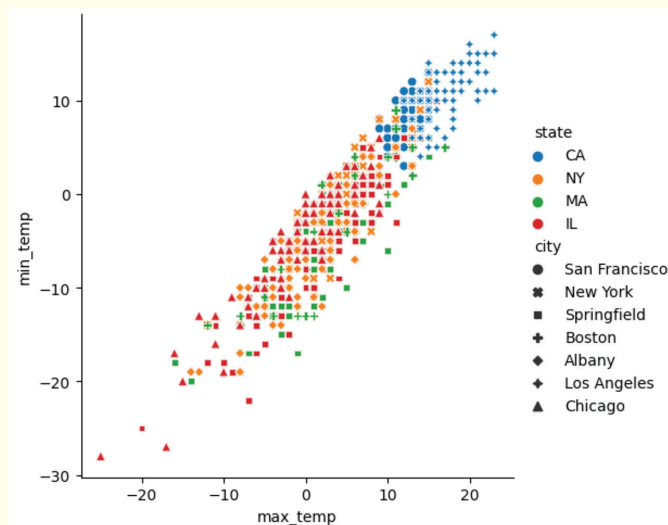


Рис. 11.25. Диаграмма рассеяния с погодой во всех городах с разделением по цветам (штаты) и типам меток (города)

И все-таки согласитесь – как ни разделяй по цветам и меткам наши наблюдения, все равно данные визуально смешиваются. Чтобы вывести их на отдельных графиках, выполнив внутреннюю группировку по городам, можно воспользоваться двумя способами: передать имя группирующего столбца параметру `row` (чтобы данные обо всех уникальных значениях в этом столбце выводились на новой строке) или параметру `col` (в этом случае графики будут разбиты по столбцам). Пример:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='state',
            row='city')
```

Результат показан на рис. 11.26.

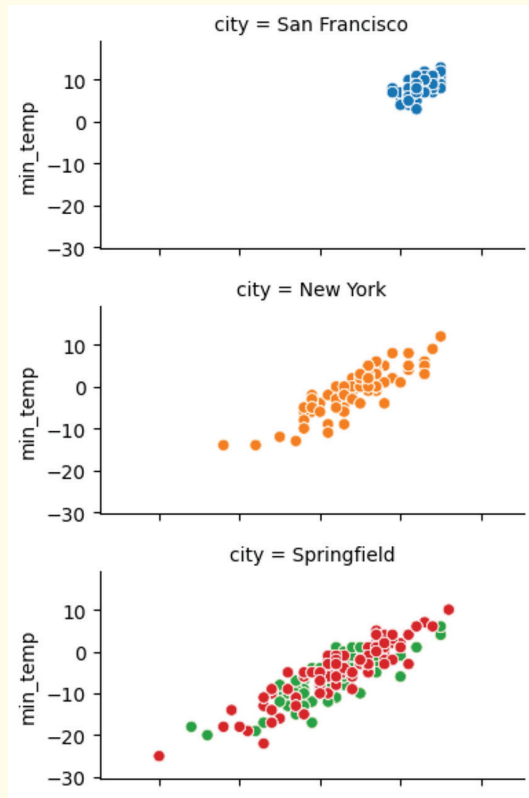


Рис. 11.26. Диаграмма рассеяния с погодой во всех городах с разделением на строки

Хотя диаграммы рассеяния представляют собой довольно мощный и визуально понятный графический инструмент, взаимосвязь между столбцами можно показать и с помощью линейчатых диаграмм (параметр `kind='line'`). В этом случае Seaborn будет соединять точки, выведенные на графике, линиями. Пример:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='state', kind='line')
```

Вроде все нормально, но это не сработает. При этом я собрал комбо: получил одновременно и предупреждение от pandas, и ошибку от Seaborn. В обоих случаях мне указали на то, что в индексе в моем датафрейме содержатся неunikальные значения. Это можно поправить при помощи метода `reset_index`, как показано ниже:

```
df = df.reset_index(drop=True)
```

Обратите внимание, что мы передали методу параметр `drop=True`, чтобы прежний индекс не был оставлен в датафрейме в виде столбца. Нам достаточно будет выбросить старый индекс и заменить его новым.

Теперь с новым индексом можно снова попытаться построить линейчатую диаграмму:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='state', kind='line')
```

Результат показан на рис. 11.27.

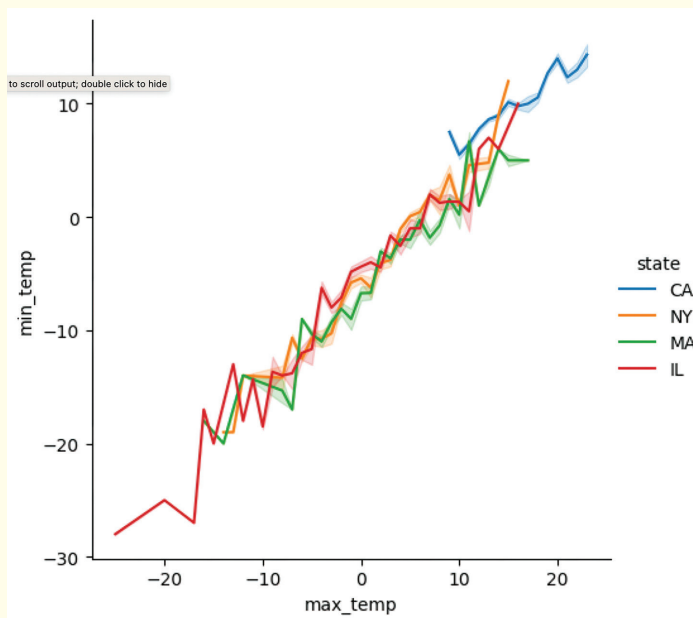


Рис. 11.27. Линейчатая диаграмма с погодой во всех городах с разделением по цветам

На графике мы видим данные сразу по всем штатам и городам, при этом мы разделили штаты по цветам с помощью параметра `hue`. Проблема в том, что два города из нашего набора данных могут находиться в одном штате. К тому же читать этот график не очень удобно из-за перемешивания линий.

Давайте снова попросим Seaborn разнести города на разные строки:

```
sns.relplot(x='max_temp',
            y='min_temp',
            data=df,
            hue='state',
            kind='line',
            row='city')
```

Результат показан на рис. 11.28.

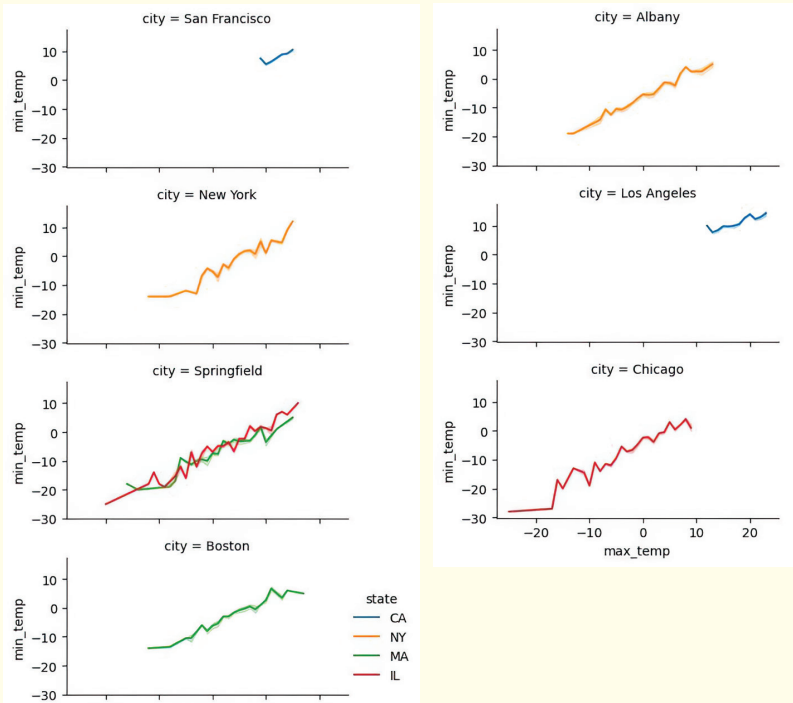


Рис. 11.28. Линейчатая диаграмма с погодой с разделением городов на строки

Библиотека Seaborn поддерживает большое количество видов диаграмм. К примеру, что делать, если вы хотите вывести все значения из столбца `max_temp` по конкретному городу? Вы можете представить это как набор одномерных диаграмм рассеяния:

```
sns.catplot(x='city', y='max_temp', data=df)
```

Результат показан на рис. 11.29.

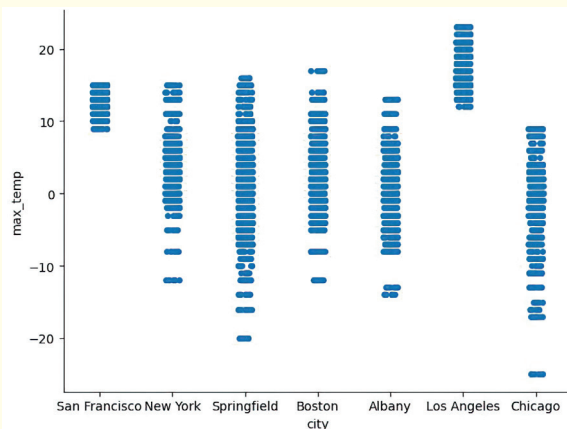


Рис. 11.29. Вывод всех значений температуры по городам

Обратите внимание, что на оси x у нас выводятся категории, т. е. города, а на оси y – значения. В итоге мы видим все значения в наборе данных.

Если нам нужно агрегировать все данные и оценить их распределение, мы можем воспользоваться диаграммой размаха, передав функции `sns.catplot` параметр `kind='box'`, как показано ниже:

```
sns.catplot(x='city', y='max_temp', data=df, kind='box')
```

Результат показан на рис. 11.30.

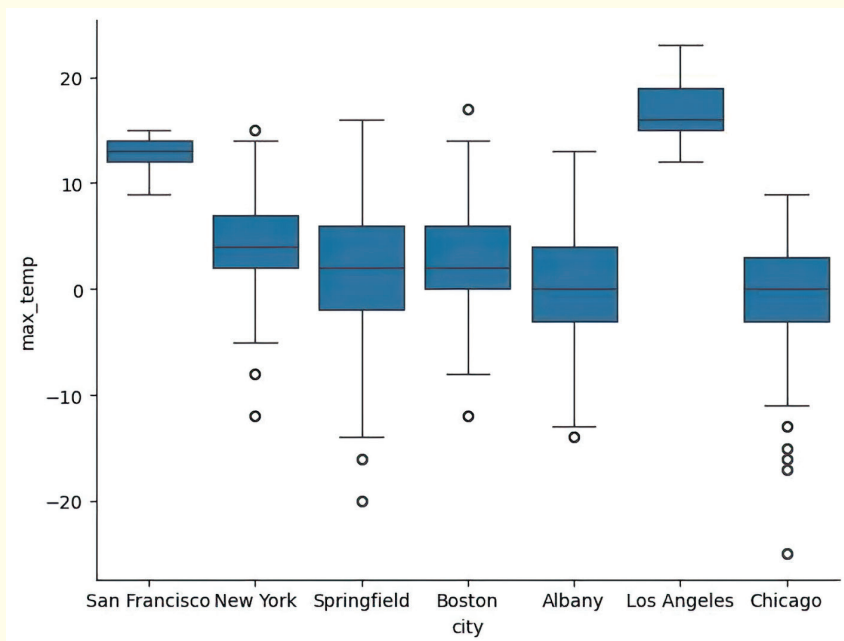


Рис. 11.30. Диаграмма размаха по температуре во всех городах

Здесь мы видим отдельные ящики с усами по всем семи городам в нашем наборе данных.

С помощью библиотеки Seaborn также можно легко строить гистограммы. Поскольку гистограммы отвечают за распределение данных, мы воспользуемся подходящей для этого функцией `sns.displot`, как показано ниже:

```
sns.displot(x='max_temp', data=df)
```

Результат показан на рис. 11.31.

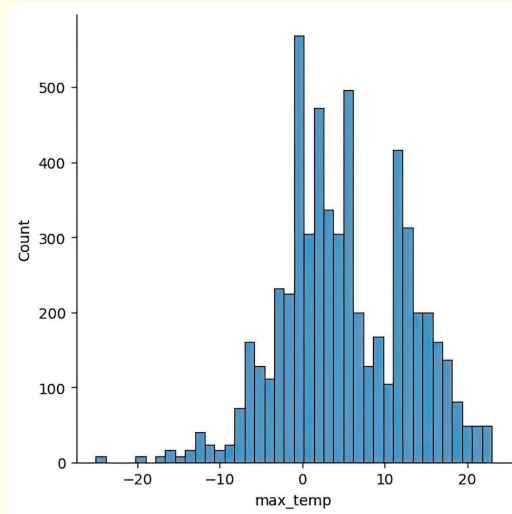


Рис. 11.31. Гистограмма для столбца `max_temp` по всем городам

Здесь также можно разделить города по цветам, передав функции параметр `hue`:

```
sns.displot(x='max_temp', data=df, hue='city')
```

Результат показан на рис. 11.32.

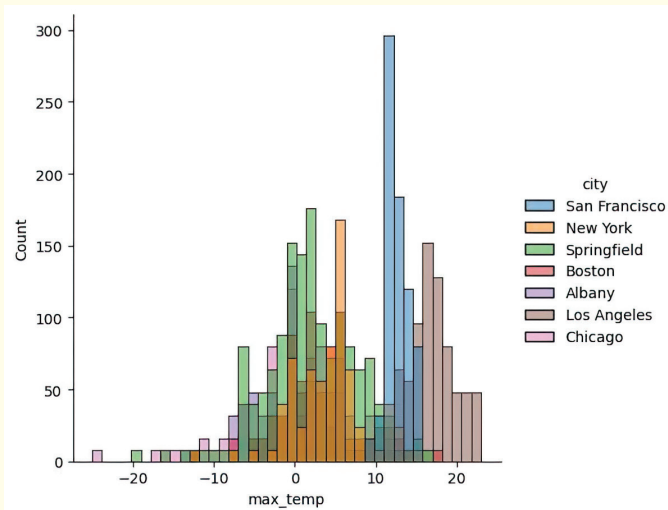


Рис. 11.32. Гистограмма для столбца `max_temp` по всем городам с разделением по цветам

При необходимости мы можем вывести и отдельные гистограммы по городам, воспользовавшись параметром `row`, как показано ниже:

```
sns.displot(x='max_temp', data=df, hue='city', row='city')
```

Результат показан на рис. 11.33.

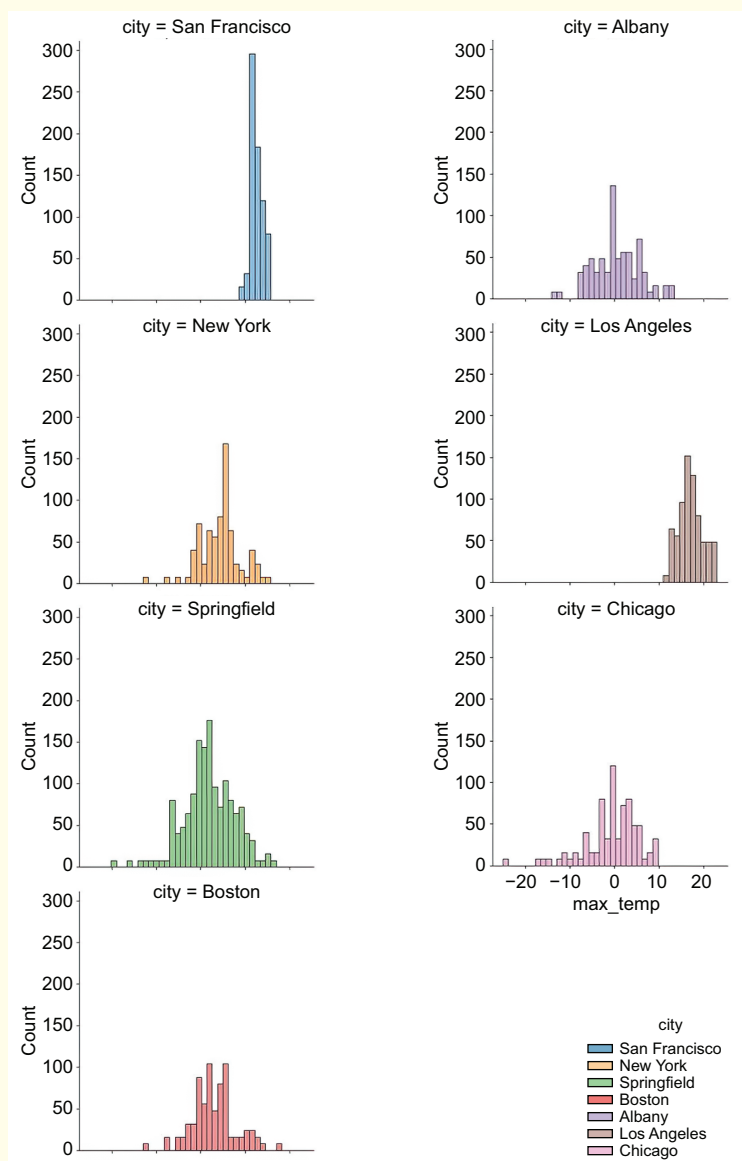


Рис. 11.33. Гистограмма для столбца `max_temp` по всем городам с разделением по цветам и разнесением по строкам

Здесь мы перечислили лишь малую часть возможностей библиотеки Seaborn. Если вам интересно глубоко вникнуть в содержание библиотеки, я рекомендую вам ознакомиться с ее документацией по адресу <https://seaborn.pydata.org>. Лично мне очень по душе подход Seaborn к визуализации данных. С помощью него можно строить очень понятные и красивые диаграммы, да и для понимания API этой библиотеки гораздо легче в сравнении с другими инструментами.

Ответы на дополнительные упражнения

Упражнение 46.1

```
df = df.reset_index()
df['month'] = df['date'].dt.month
df = df.set_index('date')
df.corr()
```

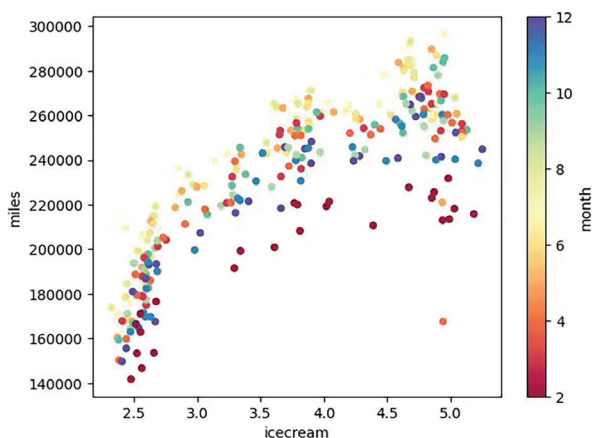
Вывод:

	oil	icecream	miles	month
oil	1.000000	0.777347	0.645250	0.006616
icecream	0.777347	1.000000	0.818383	-0.003985
miles	0.645250	0.818383	1.000000	0.079290
month	0.006616	-0.003985	0.079290	1.000000

Упражнение 46.2

```
df.plot.scatter(x='icecream', y='miles', c='month', colormap='Spectral')
```

Вывод:



Упражнение 46.3

```
oil_df = oil_df.reset_index()
oil_df = oil_df[oil_df['date'].dt.is_month_start]
oil_df = oil_df.set_index('date')

df = oil_df.join(ice_cream_df).join(miles_df)
df.head()
```

Вывод:

	oil	icecream	miles
date			
1986-04-01	11.13	2.382	150277.0

1986-05-01	13.80	2.368	160459.0
1986-07-01	12.39	2.369	171114.0
1986-08-01	11.56	2.319	173977.0
1986-10-01	15.23	2.377	159434.0

УПРАЖНЕНИЕ 47. Такси и визуализация в Seaborn

В этом упражнении мы снова вернемся к набору данных с поездками на такси в Нью-Йорке в 2020 году и построим полезные графики с помощью библиотеки Seaborn. Что вам нужно будет сделать.

1. Загрузите в единый датафрейм данные о поездках на такси за январь и июль 2020 года (файлы `nyc_taxi_2020-01.csv` и `nyc_taxi_2020-07.csv`). Нам будут интересовать только столбцы `trip_pickup_datetime`, `passenger_count`, `trip_distance` и `total_amount`.
2. Добавьте в датафрейм столбцы `day`, `month` и `year` на основе столбца `trip_pickup_datetime`. Оставьте в датафрейме только данные за 2020 год и месяцы январь и июль.
3. Установите для набора данных числовой индекс, начинающийся с нуля.
4. Оставьте в датафрейме случайную выборку данных в размере 1 % от оригинала.
5. С помощью библиотеки Seaborn постройте диаграмму рассеяния, в которой на оси *x* будет выведено поле `trip_distance`, а на оси *y* – поле `total_amount` с разделением по цветам по столбцу `passenger_count`. Используйте для графика полученную выборку с 1 % исходных данных.
6. Постройте линейчатую диаграмму, показывающую преодоленную дистанцию по дням в январе и июле. На оси *x* должны располагаться дни месяца, а на оси *y* – средняя дистанция. Для января и июля должны быть две разные линии.
7. Используя библиотеку Seaborn, покажите количество поездок, совершенных в каждый из дней месяца (1–31) в январе и июле. На оси *x* должны располагаться дни месяца, а на оси *y* – количество поездок.
8. С помощью Seaborn постройте диаграмму размаха по данным из столбца `total_amount` с двумя ящиками с усами – по одному для каждого месяца.

Подробный разбор

В этом упражнении мы загрузим данные о поездках на нью-йоркском такси за январь и июль 2020 года и воспользуемся библиотекой Seaborn для их визуализации. Для начала объединим данные из двух наборов в один датафрейм, при этом мы будем работать с четырьмя нужными нам столбцами:

```
filenames = ['../data/nyc_taxi_2020-01.csv',
              '../data/nyc_taxi_2020-07.csv']

all_dfs = [pd.read_csv(one_filename,
                       usecols=['trip_pickup_datetime',
                                'passenger_count',
```

```

        'trip_distance',
        'total_amount'],
        parse_dates=['tpep_pickup_datetime'])
    for one_filename in filenames]

```

```
df = pd.concat(all_dfs)
```

Обратите внимание, что мы воспользовались параметром `parse_dates` для приведения столбца `tpep_pickup_datetime` к типу `datetime`. Остальные столбцы будут загружены в виде чисел с плавающей точкой. Как и раньше, мы создаем список датафреймов с помощью генератора списков, после чего объединяем их методом `pd.concat`.

Для начала нас попросили добавить в датафрейм три столбца с компонентами даты и времени:

```

df['year'] = df['tpep_pickup_datetime'].dt.year
df['month'] = df['tpep_pickup_datetime'].dt.month
df['day'] = df['tpep_pickup_datetime'].dt.day

```

Также мы должны сохранить в датафрейме только данные за 2020 год и месяцы январь и июль. Как мы уже видели, набор данных с поездками на такси весьма «загрязнен» паразитными данными за другие годы и месяцы. И чтобы наши графики не выглядели сумбурно, давайте подчистим данные при помощи комбинации масок, как показано ниже:

```

df = df.loc[(df['month'].isin([1, 7])) &
            (df['year'] == 2020)]

```

Теперь необходимо убедиться, что новый набор данных содержит уникальные индексы. Это нужно делать всякий раз после объединения нескольких наборов данных. Проверить индекс на уникальность можно при помощи атрибута `is_unique` следующим образом:

```
df.index.is_unique
```

Если этот атрибут вернет `True`, значит в индексе содержатся уникальные значения. В противном случае при построении графиков в Seaborn вы получите ошибку. Мы можем перенумеровать индекс самостоятельно, но зачем это делать, если pandas может все сделать за нас?

```
df = df.reset_index(drop=True)
```

Да, это тот же самый метод `reset_index`, который мы использовали ранее, чтобы избавиться от индекса, созданного из столбца. С помощью параметра `drop=True` мы указываем методу `reset_index` не создавать колонку с данными на основе предыдущего индекса, а вместо этого безжалостно удалить его.

Теперь можно было бы приступить к визуализации. Но наш набор данных достаточно велик, в нем содержится несколько миллионов наблюдений. Для простоты мы оставим в наборе данных лишь 1% исходных данных, пожертвовав при этом точностью, на основе которых и будем строить графики. Сделаем это при помощи метода `sample`:

```
df = df.sample(frac=0.01)
```


Итак, мы готовы визуализировать наши усеченные данные. Сначала построим диаграмму рассеяния, сравнивающую данные в столбцах `trip_distance` (ось x) и `total_amount` (ось y), как показано ниже:

```
sns.relplot(x='trip_distance',
            y='total_amount',
            data=df,
            hue='passenger_count')
```

Функция `relplot` позволяет строить диаграммы на основе числовых столбцов, и по умолчанию она выводит диаграмму рассеяния. Мы передали функции следующие параметры:

- в качестве наполнения для оси x мы передали столбец `trip_distance`;
- на ось y мы поместили данные из столбца `total_amount`;
- в качестве источника данных мы передали датафрейм `df`;
- с помощью параметра `passenger_count` мы задали цветовое разделение для меток.

Результат показан на рис. 11.34.

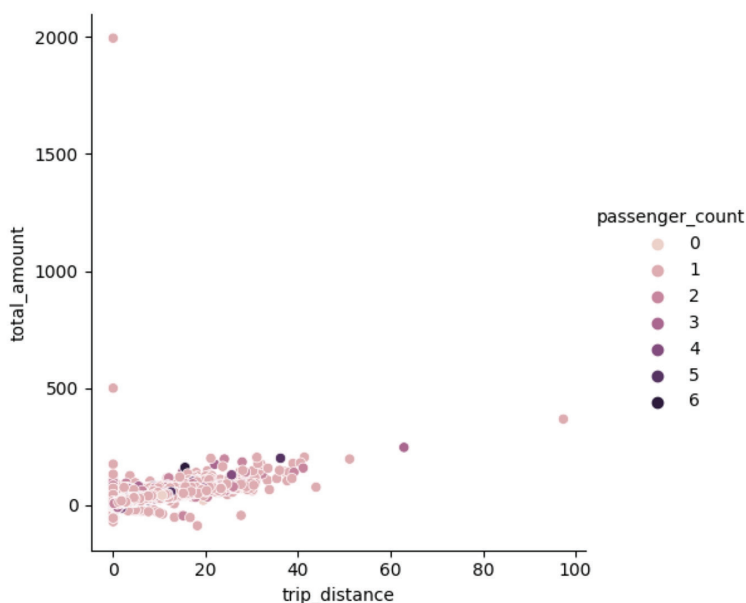


Рис. 11.34. Диаграмма рассеяния, сравнивающая поля `trip_distance` и `total_amount`

Далее нас попросили построить линейчатую диаграмму, показывающую преодоленную дистанцию по дням в январе и июле. Для этого мы также воспользуемся функцией `relplot` для работы с числовыми полями:

- на ось x мы отправим содержимое созданного столбца `day`;
- на оси y разместим столбец `trip_distance`;
- для получения линейчатой диаграммы передадим функции `relplot` параметр `kind='line'`;

- исходные данные мы будем брать из датафрейма `df`;
- для распределения цветов воспользуемся столбцом `month`.

```
sns.relplot(x='day', y='trip_distance', kind='line',
            data=df, hue='month')
```

Разделив месяцы по цветам, мы можем вывести две линии на одном графике для удобства сравнения. Результат показан на рис. 11.35.

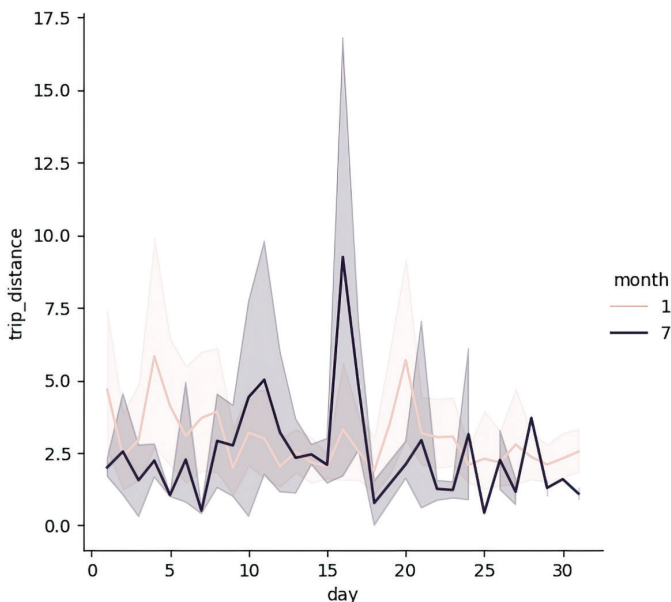


Рис. 11.35. Линейчатая диаграмма по данным из столбца `trip_distance` с разделением на месяцы

Обратите внимание на серые зоны вокруг каждой линии. Таким образом отображаются доверительные интервалы для каждого набора данных. Доверительные интервалы служат для оценки вероятности вхождения значения в определенный диапазон. Мы можем отключить опцию вывода доверительных интервалов, передав параметр `ci='None'` функции `relplot`, как показано ниже:

```
sns.relplot(x='day', y='trip_distance', kind='line',
            data=df, hue='month', ci=None)
```

После этого нас попросили с помощью библиотеки Seaborn показать количество поездок, совершенных в каждый из дней месяца (1–31) в январе и июле. Для этого мы воспользуемся агрегирующим методом `count`. В качестве набора данных мы должны взять не весь датафрейм, а результат применения к нему метода `groupby`. Если сгруппировать данные по месяцу и дню и вычислить количество значений в столбце с годом, мы как раз получим количество поездок за каждый день. Использование столбца с годом выглядит странно, но нам просто нужен столбец для агрегации. После выполнения операции группировки мы перестраиваем индекс, преобразуя поля `month` и `day` обратно в обычные столбцы:

```
pl = sns.relplot(x='day', y='year', hue='month', kind='line',
                data=df.groupby(['month', 'day'])['year'].count()
                .reset_index(), ci=None)
pl.set(ylabel = "count")
```

Это довольно сложный запрос с не самым тривиальным выводом, так что разберем его по шагам.

1. Нам нужно узнать, сколько поездок на такси было совершено в каждый день каждого месяца. Таким образом, запрашивается следующая группировка: `groupby(['month', 'day'])`.
2. Далее применяем агрегирующий метод `count` к объекту, полученному в результате группировки.
3. Это даст нам агрегацию по каждому столбцу, а нам нужно получить одну цифру. Выберем для этого столбец `year`.
4. Далее необходимо воспользоваться методом `reset_index`, чтобы вернуть полям из индекса `month` и `day` (в объекте с группировкой) облик обычных столбцов.
5. Передаем результат метода `reset_index` в качестве аргумента `data` в метод `relplot`.
6. Указываем методу `relplot`, что на оси `x` должен располагаться столбец `day`, а на оси `y` – столбец `year`, в нашем случае отражающий количество поездок.
7. С помощью параметра `hue` говорим методу `relplot`, что хотим разделить месяцы по цветам.
8. Задаем линейчатый тип диаграммы с помощью параметра `kind='line'`.
9. Передаем параметр `ci='None'`, чтобы не выводились доверительные интервалы.
10. Присваиваем результат функции `sns.relplot` переменной `pl` и устанавливаем для нее подпись для оси `y`, чтобы там не выводилось слово `year`, с помощью универсального метода `set`.

Результат показан на рис. 11.36.

Как видим, в июле, после начала пандемии, количество поездок на такси резко сократилось по сравнению с январем.

Наконец, нас попросили с помощью Seaborn построить диаграмму размаха по данным из столбца `total_amount` с двумя ящиками с усами – по одному для каждого месяца. В терминологии Seaborn диаграмма размаха носит категориальный характер, поскольку с ее помощью можно сравнивать распределение значений по категориям. В связи с этим нам придется воспользоваться функцией `catplot` следующим образом:

- на ось `x` мы вынесем категории, по которым будем выполнять сравнение, т. е. столбец `month`;
- на оси `y` разместим столбец со значениями для анализа: `total_amount`;
- в качестве источника у нас будет датафрейм `df`;
- чтобы построить диаграмму размаха, необходимо передать функции `catplot` параметр `kind='box'`.

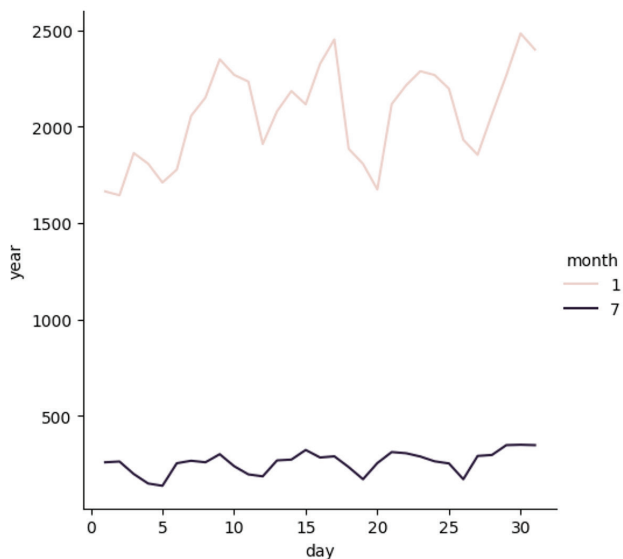


Рис. 11.36. Линейчатая диаграмма по количеству поездок на такси за день с разделением на месяцы

Получим такой вызов функции:

```
sns.catplot(x='month', y='total_amount', data=df, kind='box')
```

Результат показан на рис. 11.37.

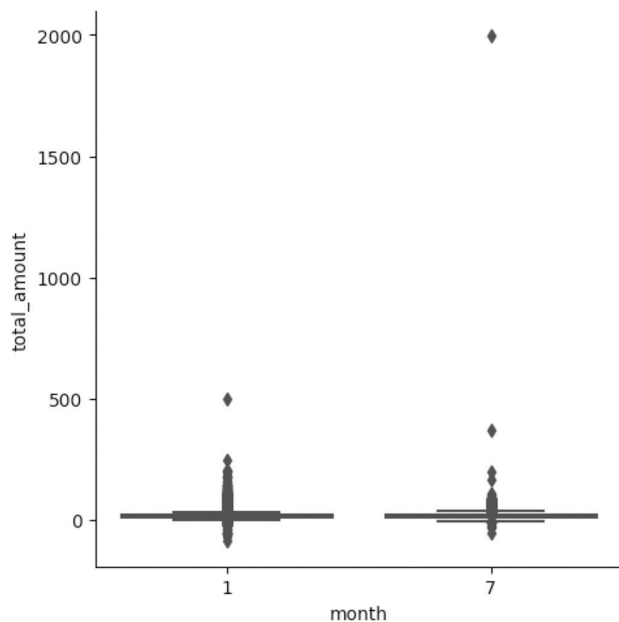


Рис. 11.37. Диаграмма размаха по столбцу total_amount с разделением на месяцы

Медианные значения стоимости поездок на такси с января по июль сильно не изменились, что можно видеть и в числовом отображении:

```
df.groupby('month')['total_amount'].median()
```

Решение

```
filenames = ['../data/nyc_taxi_2020-01.csv', '../data/nyc_taxi_2020-07.csv']

all_dfs = [pd.read_csv(one_filename,
                      usecols=['tpep_pickup_datetime', 'passenger_count',
                              'trip_distance', 'total_amount'],
                      parse_dates=['tpep_pickup_datetime'])
           for one_filename in filenames]

df = pd.concat(all_dfs)

df['year'] = df['tpep_pickup_datetime'].dt.year
df['month'] = df['tpep_pickup_datetime'].dt.month
df['day'] = df['tpep_pickup_datetime'].dt.day

df = df.loc[(df['month'].isin([1, 7])) & (df['year'] == 2020)]

df = df.reset_index(drop=True)

df = df.sample(frac=0.01)

sns.relplot(x='trip_distance', y='total_amount', data=df,
            hue='passenger_count')

sns.relplot(x='day', y='trip_distance', kind='line',
            data=df, hue='month', ci=None)

pl = sns.relplot(x='day', y='year', hue='month', kind='line',
                data=df.groupby(['month', 'day'])['year'].count()
                .reset_index(), ci=None)
pl.set(ylabel = "count")

sns.catplot(x='month', y='total_amount', data=df, kind='box')
```

Дополнительные упражнения

1. Загрузите данные о поездках на такси за 2019 и 2020 годы (январь и июль). Удалите лишние данные, не входящие в эти периоды. Выведите стоимость поездок в каждый день месяца на четырех графиках: верхний ряд графиков должен соответствовать 2019 году, а нижний – 2020-му, слева – январь, справа – июль.
2. Добавьте в датафрейм столбец с именем `trip_length` с указанием коротких, средних и продолжительных поездок, как мы делали в упражнении 7. Выведите на трех графиках распределение преодоленной дистанции (`trip_distance`) по этим трем категориям.

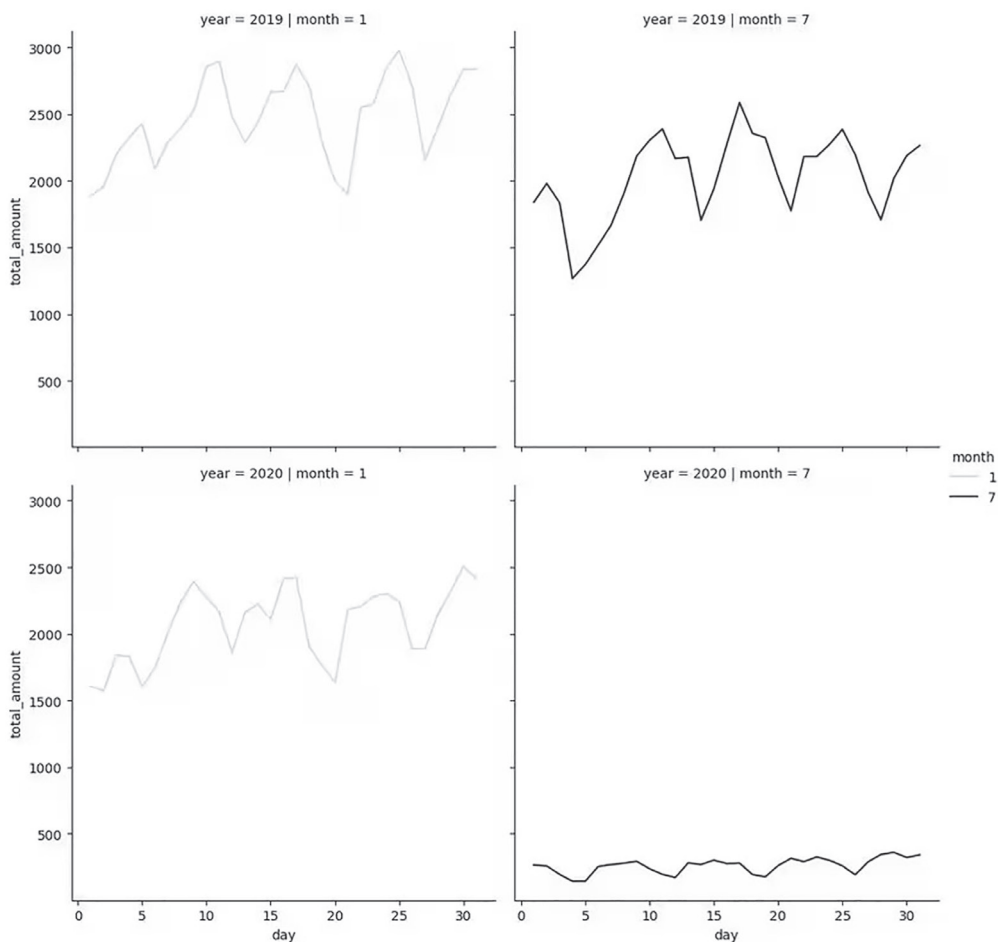
3. Постройте столбчатую диаграмму по количеству поездок в каждый из 24 ч в сутках в январе и июле. Каждый месяц должен быть выделен своим цветом. Слева должен быть график по январю, а справа – по июлю.

Ответы на дополнительные упражнения

Упражнение 47.1

```
sns.relplot(x='day', y='total_amount', hue='month', kind='line',
            row='year', col='month',
            data=df.groupby(['year', 'month', 'day'])['total_amount'].
count().reset_index(),
            errorbar=None)
```

Вывод:

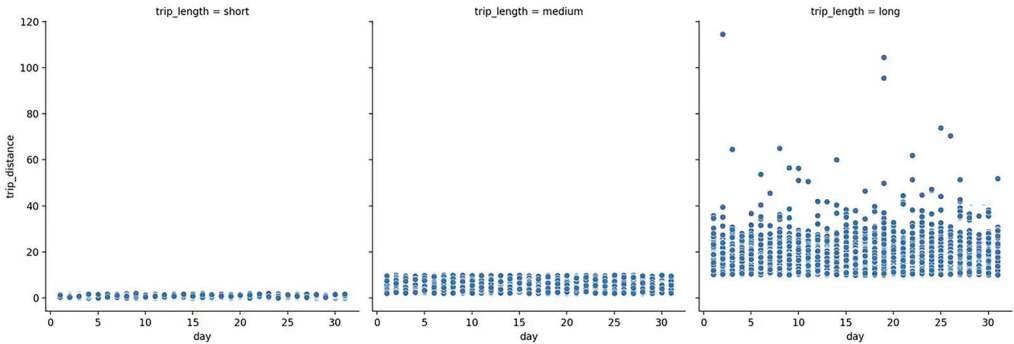


Упражнение 47.2

```
df['trip_length'] = pd.cut(df['trip_distance'],
                           bins=[0, 2, 10, df['trip_distance'].max()],
                           include_lowest=True,
                           labels=['short', 'medium', 'long'])
```

```
sns.relplot(x='day', y='trip_distance', data=df, col='trip_length')
```

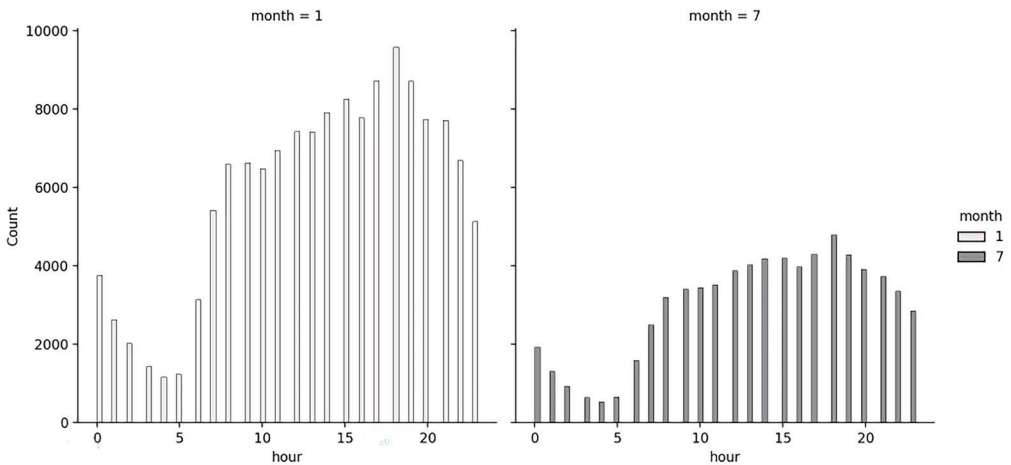
Вывод:



Упражнение 47.3

```
df['hour'] = df['tpep_pickup_datetime'].dt.hour
sns.displot(data=df, x='hour', hue='month', col='month')
```

Вывод:



Заклучение

Визуализация представляет собой один из ключевых аспектов науки о данных. Мы часто представляем ее как промежуточное звено между аналитиком данных и их потребителем в лице обычного пользователя, не знакомого с техническими тонкостями анализа. Однако графики способны дать полезную информацию и самому специалисту по работе с данными, поскольку не все можно увидеть в сухих цифрах. В этой главе мы узнали, как с помощью `pandas` и его упрощенного API для `Matplotlib` можно строить разнообразные графики и диаграммы. Также мы познакомились с библиотекой `Seaborn`, которая позволяет выводить сложные диаграммы, пользуясь при этом собственным интерфейсом, реализованным поверх `Matplotlib`.

Глава 12

Оптимизация

Современные компьютеры обладают огромным быстродействием. Они с легкостью выполняют миллиарды операций в секунду, позволяя нам общаться по видеосвязи из любой точки мира, предсказывать погоду с невероятной точностью и осуществлять поиск в огромных базах данных в мгновение ока. Офисному работнику из прошлого столетия и в голову не могло прийти, сколько информации и с какой скоростью можно обрабатывать.

Но давайте будем честны. Когда в последний раз вы были довольны быстродействием своего компьютера? Если мы с вами хоть чуточку похожи, вы, как и я, не особо восторгаетесь тем, как все летает, а ищите поводы погрузиться над тем, как все ужасно тормозит.

Я люблю говорить, что язык программирования Python отлично подходит для нашего времени, когда компьютеры достаточно дешевы, а человеческий ресурс достаточно дорог. Под этим я подразумеваю, что Python не требует больших усилий в процессе написания программ, зачастую принося в жертву их быстродействие. Но все не так плохо, как может показаться. Тот факт, что pandas использует в своей основе библиотеку NumPy, частично написанную на C, позволяет оперировать большими объемами данных гораздо эффективнее по сравнению с использованием стандартных объектов Python. И чем плотнее ваша программа будет интегрирована с NumPy, тем быстрее она будет выполняться.

Но, помимо этого золотого правила, существует целый ряд техник, позволяющих ускорить работу pandas и сократить физический объем датафреймов, с которыми вам приходится работать. Многие из этих техник так или иначе связаны с необходимостью обрабатывать только те данные, которые действительно нужно. Поскольку pandas оперирует данными в памяти, сокращение объема обрабатываемых ячеек напрямую влияет на быстродействие.

В этой главе мы познакомимся с некоторыми приемами, позволяющими повысить эффективность работы библиотеки pandas. Мы научимся правильно измерять объем занимаемой датафреймами памяти и засекают время выполнения операций при помощи модуля `timeit`. Также мы узнаем, как можно сэкономить память с помощью специального типа с категориями. Кроме того, мы увидим, как можно повысить быстродействие с помощью `PuArrow`, представляющего собой интерфейс для работы с библиотекой Arrow, как при загрузке данных из файлов CSV, так и используя его в качестве альтернативы NumPy.

К концу главы вы будете знать все основные подводные камни, связанные с pandas, и научитесь их обходить.

В табл. 12.1 собраны полезные ссылки по объектам и методам для самостоятельного изучения.

Таблица 12.1. Предметы изучения

Предмет	Описание	Пример	Ссылки для изучения
<code>df.info</code>	Позволяет извлечь информацию о датафрейме, включая данные о занимаемой памяти	<code>df.info()</code>	http://mng.bz/D4eE (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.info.html)
<code>df.memory_usage</code>	Позволяет извлечь информацию о занимаемой датафреймом памяти	<code>df.memory_usage(deep=True)</code>	http://mng.bz/LWjy (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.memory_usage.html)
Категориальные данные	Использование категориальных данных в pandas	<code>df['a'].astype('categorical')</code>	http://mng.bz/BmaJ (https://pandas.pydata.org/pandas-docs/stable/user_guide/categorical.html)
<code>df.to_feather</code>	Записывает датафрейм в формате feather	<code>df.to_feather('mydata.feather')</code>	http://mng.bz/d1pQ (https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.to_feather.html)
<code>pd.read_feather</code>	Создает датафрейм на основе данных в формате feather	<code>df = pd.read_feather('mydata.feather')</code>	http://mng.bz/rWLX (https://pandas.pydata.org/docs/reference/api/pandas.read_feather.html)
<code>pd.read_csv</code>	Создает датафрейм на основе данных в формате CSV	<code>df = pd.read_csv('myfile.csv')</code>	http://mng.bz/V1r5 (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html)
<code>pd.read_json</code>	Возвращает новый датафрейм на основе данных в формате JSON	<code>df = pd.read_json('myfile.json')</code>	http://mng.bz/x4qB (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_json.html)
<code>time.perf_counter</code>	Возвращает значение (в долях секунды) счетчика производительности, что помогает при замере быстродействия операций	<code>time.perf_counter()</code>	http://mng.bz/AonW (https://docs.python.org/3/library/time.html#time.perf_counter)

Таблица 12.1. Предметы изучения (продолжение)

Предмет	Описание	Пример	Ссылки для изучения
<code>df.query</code>	Позволяет писать запросы к датафреймам в формате, близком к SQL	<code>df.query('v > 300')</code>	http://mng.bz/ZqBZ (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.query.html)
<code>df.eval</code>	Выполняет действия и запросы применительно к датафрейму	<code>df.eval('v + 300')</code>	http://mng.bz/Rx9P (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.eval.html)
<code>pd.eval</code>	Позволяет выполнять различные операции в pandas в виде строки вычисления	<code>pd.eval('df.v > 300')</code>	http://mng.bz/2D9X (https://pandas.pydata.org/docs/reference/api/pandas.eval.html#pandas.eval)
Timeit	Модуль Python для оценки скорости выполнения кода и магическая команда в Jupyter для вызова этого модуля	<code>%timeit 3+2</code>	http://mng.bz/1qKg (https://docs.python.org/3/library/timeit.html)
Isin	Проверяет, находится ли значение в последовательности Python	<code>df['a'].isin([10, 20, 30])</code>	http://mng.bz/Pz0P (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.Series.isin.html)
<code>pd.CategoricalDtype</code>	Возвращает новый категориальный тип dtype	<code>pd.CategoricalDtype(['a', 'b', 'c', 'd'])</code>	http://mng.bz/Jgev (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.CategoricalDtype.html)

Экономия памяти с помощью категорий

Скажем, мы хотим поработать с данными из файла `olympic_athlete_events.csv`:

```
filename = '../data/olympic_athlete_events.csv'
df = pd.read_csv(filename)
```

Сколько памяти потребуется для этого датафрейма? Это очень важный вопрос при работе с pandas, поскольку все ваши данные должны уместиться в памяти компьютера. Мы можем воспользоваться методом `memory_usage`, как показано ниже:

```
df.memory_usage()
```

В результате мы получим объект Series с данными о количестве байт, занимаемом каждым столбцом в отдельности. В качестве индекса будут присутствовать имена столбцов в исходном датафрейме. Общий объем занимаемой памяти можно узнать, просуммировав все значения:

```
df.memory_usage().sum()
```

У меня получилось 32 534 048 байт, или чуть больше 31 Мб.

Как вы можете догадаться, это число не может соответствовать истинному объему данных в датафрейме. Причина в том, что pandas по умолчанию игнорирует размер любых объектов Python, содержащихся в датафрейме. А с учетом того, что эти объекты по большей части представляют собой строки, которые могут быть довольно длинными, разница между отображаемым значением и задействованной памятью может оказаться весьма большой.

Мы можем попросить pandas учитывать истинные размеры объектов Python при подсчете, передав методу `memory_usage` параметр `deep=True`, как показано ниже:

```
df.memory_usage(deep=True).sum()
```

Теперь размер датафрейма составил 186 408 012 байт, или 182 Мб, что в пять раз больше первоначального измерения.

Но постойте, неужели так много памяти требуется для такого небольшого датафрейма?! А если мы будем работать с датафреймами побольше, нам может вовсе не хватить памяти? Как можно снизить объем потребляемой памяти и при этом иметь возможность выполнять все требуемые операции? Ранее мы уже говорили о двух способах:

- ограничить количество загружаемых столбцов с помощью параметра `usecols`;
- явным образом задать параметр `dtype` для каждого столбца, что позволит сократить объем потребляемой памяти за счет выбора более экономичных типов данных.

Однако больше всего места обычно занимают строковые данные. В нашем случае мы можем убедиться в этом, выполнив следующую инструкцию:

```
df.memory_usage(deep=True).sort_values()
```

Вывод:

```
Index      128
ID         2168928
Age        2168928
Height     2168928
Weight     2168928
...
Team       17734961
Sport      18031019
Games      18435888
Name       20697535
Event      24146495
Length: 16, dtype: int64
```

Как видите, самыми требовательными в плане объема задействованной памяти являются столбцы со строками, а не числами. Таким образом, нам нужно как-то ограничить количество строк, которые будут в действительности храниться в датафрейме.

Один из способов состоит в использовании специального типа данных в `pandas`, называемого `category`. Этот тип позволяет все уникальные строки хранить лишь один раз, а в датафрейме обращаться к ним по ссылке. При этом для пользователя и разработчика все будет довольно прозрачно – мы можем продолжать делать вид, что в датафрейме у нас хранятся строки, и применять атрибут доступа `str` для выполнения нужных нам преобразований.

Мы уже не раз использовали метод `astype` для создания нового объекта `Series` на основе существующего. Этот же метод вполне применим и для создания нового категориального столбца на базе существующего столбца со строками.

Давайте продемонстрируем это на примере столбца `Games`, в котором содержатся названия Олимпийских игр:

```
df['Games'].value_counts().head(10)
```

Вывод:

```
Games
2000 Summer    13821
1996 Summer    13780
2016 Summer    13688
2008 Summer    13602
2004 Summer    13443
1992 Summer    12977
2012 Summer    12920
1988 Summer    12037
1972 Summer    10304
1984 Summer     9454
Name: count, dtype: int64
```

Как видите, строка `2000 Summer` встречается в столбце `Games` 13 821 раз. Если на основе этого столбца построить категории, то это значение будет храниться лишь раз, а в датафрейме физически будут содержаться целочисленные указатели на него, для хранения которых нужно гораздо меньше места. Это показано на рис. 12.1.

Создать столбец с категориальным типом можно следующим образом:

```
df['Games'].astype('category')
```

Но пока мы ничего полезного не сделали, поскольку не сохранили столбец в датафрейме. Проще всего заменить старый столбец на новый, изменив при этом его тип, можно так:

```
df['Games'] = df['Games'].astype('category')
```

Сколько памяти мы сэкономили, выполнив это действие? Узнаем это, снова воспользовавшись методом `memory_usage`:

```
df.memory_usage(deep=True).sum()
```

Объем памяти снизился примерно на 15 Мб, и при этом мы не утратили никакой важной информации.

Какие столбцы в первую очередь необходимо приводить к категориальному типу? Очевидно, те, в которых не так много уникальных значений. Определить это можно с помощью следующего несложного расчета:

```
(df.count() / df.nunique()).sort_values(ascending=False)
```

id	Games
0	1992 Summer
1	2012 Summer
2	1920 Summer
3	1900 Summer
4	1988 Winter
5	1988 Winter
6	1992 Winter
7	1992 Winter
8	1994 Winter
9	1994 Winter

id	Games
0	0
1	1
2	2
3	3
4	4
5	4
6	5
7	5
8	6
9	6

id	Games
0	1992 Summer
1	2012 Summer
2	1920 Summer
3	1900 Summer
4	1988 Winter
5	1992 Winter
6	1994 Winter

Рис. 12.1. Десять строк из столбца Games до и после приведения его к категориальному типу

Вывод:

```
Sex      135558.000000
Season   135558.000000
Medal    13261.000000
Year     7746.171429
City     6455.142857
```

```
...
Weight   946.550000
Event    354.400000
Team     228.983108
Name     2.012261
ID       1.999808
Length: 15, dtype: float64
```

Здесь мы поделили общее количество непропущенных элементов в столбце на количество уникальных значений в нем. Чем выше это число, тем чаще в столбце повторяются одни и те же значения и тем пригоднее этот столбец может оказаться для приведения к категориальному типу. Методом `sort_values` мы воспользовались для сортировки значений в выводе.

Я решил выбрать для преобразования все столбцы с типом `object` и числом повторений больше 100. В результате получим следующий код:

```
for column_name in ['Sex', 'Season', 'Medal', 'City', 'Games',
                    'Sport', 'NOC', 'Event', 'Team']:
    print(column_name)
    df[column_name] = df[column_name].astype('category')
```

В итоге наш датафрейм стал занимать в памяти всего около 33 Мб. Таким образом, всего несколько строк кода позволили нам в пять раз снизить объем датафрейма. Это достаточно большая экономия.

Но постойте, мы ведь создали столбец на основе категорий, которые присутствуют в нем на данный момент. А что, если нам нужно будет в будущем добавить новую категорию? Рассмотрим следующий пример:

```
s = Series(['a', 'b', 'c', 'a', 'b', 'c', 'c', 'c']).astype('category')
```

Теперь попытаемся присвоить одному из элементов объекта Series значение 'd', которого не было в перечислении на момент его создания:

```
s.loc[7] = 'd'
```

В результате мы получим ошибку `TypeError`, говорящую о том, что мы не можем присвоить элементу несуществующую категорию.

Мы можем решить эту проблему, создав категорию перед созданием объекта Series (или столбца в датафрейме) и включив в нее все возможные значения. Далее мы присваиваем объекту не абстрактный тип `category`, а конкретный созданный с помощью функции `pd.CategoricalDtype` тип. Посмотрим, как это будет выглядеть на примере с нашим объектом Series:

```
abcd_category = pd.CategoricalDtype(['a', 'b', 'c', 'd'])
s = Series(['a', 'b', 'c', 'a', 'b',
           'c', 'c', 'c']).astype(abcd_category)
s.loc[7] = 'd' # Успешно!
```

Здесь мы заранее создали категорию с помощью функции `pd.CategoricalDtype`. После этого в методе `astype` мы указали созданную ранее категорию, а не анонимную, как раньше. То же самое мы можем сделать применительно к нашему датафрейму с олимпийскими достижениями, как показано ниже:

```
medals_category = pd.CategoricalDtype(['Gold', 'Bronze', 'Silver'])
df['Medal'] = df['Medal'].astype(medals_category)
```

УПРАЖНЕНИЕ 48. Категории

Ранее в этой книге мы уже работали с набором данных, в котором хранится информация о выданных штрафных талонах за неправильную парковку, и при этом мы не могли не думать о том, сколько памяти потребует хранение всего датафрейма. И правда, при загрузке всех строк в датафрейм он занимает порядка 18 Гб. Нам бы хотелось существенно сократить этот объем за счет приведения столбцов к категориальному типу.

ПРИМЕЧАНИЕ. Поскольку я осознаю, что не у всех читателей этой книги есть в компьютере десяток гигабайт свободной памяти, мы будем ограничивать количество загружаемых столбцов. Если вам повезло и у вас нет недостатка памяти, можете загрузить весь набор данных целиком и попробовать поработать с ним. Вы удивитесь, сколько места поможет сэкономить приведение столбцов к категориальному типу. Если же у вас недостаточно памяти и для столбцов, перечисленных в этом упражнении, вы можете смело ограничить загружаемый датафрейм еще больше.

Итак, что вам нужно сделать.

1. Загрузите набор данных в память. Нам потребуются следующие столбцы: Plate ID, Registration State, Vehicle Make, Vehicle Color, Vehicle Body Type, Violation Time, Street Name и Violation Legal Code.
2. Определите объем используемой датафреймом памяти.
3. Приведите все столбцы к категориальному типу.
4. Ответьте на следующие вопросы:
 - какого типа стали ваши столбцы?
 - сколько памяти датафрейм занимает сейчас?
 - сколько памяти вы сэкономили благодаря использованию категорий?

Подробный разбор

В этом упражнении заданий гораздо меньше, чем в предыдущих, и на то есть две причины. Во-первых, я хотел показать вам, как легко можно работать с категориями в pandas. А во-вторых, при обработке больших объемов данных даже самым производительным компьютерам требуется немало времени.

Давайте загрузим наши данные, ограничившись только указанными столбцами:

```
filename = '../data/nyc-parking-violations-2020.csv'

df = pd.read_csv(filename,
                  usecols=['Plate ID',
                           'Registration State',
                           'Vehicle Make',
                           'Vehicle Color',
                           'Vehicle Body Type',
                           'Violation Time',
                           'Street Name',
                           'Violation Legal Code'])
```

Вероятно, вы получите предупреждения DtypeWarning от pandas, свидетельствующие о наличии смешанных типов в столбцах. Мы уже видели такие предупреждения ранее. Мы собираемся приводить столбцы к категориальному типу, так что не будем обращать на это внимания.

Узнаем, сколько памяти потребовалось для загрузки нашего датафрейма. Это можно сделать двумя способами. Первый состоит в использовании уже знакомого нам метода `memory_usage` с параметром `deep=True`. В результате мы получим объект Series, в котором в качестве индекса будут присутствовать имена столбцов, а в качестве значений – объем занимаемой ими памяти:

```
df.memory_usage(deep=True)
```

Вывод:

Index	128
Plate ID	798282162
Registration State	737248306
Vehicle Body Type	758166224


```
Vehicle Make          768611575
Violation Time        774726961
Street Name           879156216
Violation Legal Code  515644296
Vehicle Color         735089399
dtype: int64
```

Согласно этому отчету для хранения каждой колонки нам требуется более чем по 0.5 Гб памяти. Даже для современных компьютеров это немалые объемы, к тому же нам не хотелось бы использовать всю доступную память.

Я хочу лишний раз напомнить вам о важности параметра `deep=True`, если вы хотите знать истинный объем памяти, занимаемой датафреймом. Без этого параметра мы получили бы такие результаты:

```
Index 128
Plate ID 99965872
Registration State 99965872
Vehicle Body Type 99965872
Vehicle Make 99965872
Violation Time 99965872
Street Name 99965872
Violation Legal Code 99965872
Vehicle Color 99965872
dtype: int64
```

Обратите внимание, что в этом случае все столбцы, за исключением индекса, будут занимать одинаковый объем памяти, а именно 99 965 872 байт, или около 100 Мб. Это не так мало, но все же гораздо меньше, чем актуальный объем данных.

Почему `pandas` по умолчанию не использует параметр `deep=True`? Поскольку, вместо того чтобы просто запросить информацию об используемой памяти у движка `NumPy`, `pandas` вынужден проходить по всем объектам `Python` и собирать информацию отдельно. Именно поэтому требование глубокого анализа необходимо передавать явно.

Общий объем получим следующим образом:

```
df.memory_usage(deep=True).sum()
```

На моем компьютере объем составил 5 966 925 267 байт, или около 6 Гб.

Теперь приведем столбцы к категориальному типу данных. Как вы помните, если колонка называется `colname`, привести ее к категориальному типу можно так:

```
df['colname'] = df['colname'].astype('category')
```

В результате `pandas` удаляет из столбца значения `NaN`, извлекает уникальные значения, создает объект категории на основе этих значений и использует его для присваивания значений. И хотя после этого фактически все привязки к данным останутся, `pandas` заменит строковые значения на гораздо более экономичные с точки зрения памяти целочисленные ссылки.

Перед приведением типов запомним в переменной, сколько места занимает наш датафрейм:

```
orig_mem = df.memory_usage(deep=True).sum()
```

Далее мы выполним преобразование столбцов в цикле. Вы можете удивиться, поскольку я обычно не советую использовать традиционные циклы при работе с `pandas`. Но это касается ситуаций, когда вы выполняете строчные преобразования, для которых в `pandas` есть много механизмов, гораздо более быстрых по сравнению с циклами. Использование движка NumPy позволяет эффективно использовать векторизацию, которая связана с меньшими затратами памяти в сравнении с применением традиционных структур данных в Python.

Но у нас другой случай. Нам нужно выполнять по одной векторизованной операции в рамках каждого столбца. По столбцам нам векторизация не нужна. Поэтому традиционный цикл здесь вполне применим. Объект индекса, который мы получим в результате обращения к атрибуту `df.columns`, является итерируемым, что позволяет получить подряд все имена столбцов. Напишем следующий цикл:

```
for one_colname in df.columns:
    print(f'Categorizing {one_colname}...')
    df[one_colname] = df[one_colname].astype('category')
    print('\tDone.')
```

Обратите внимание, что мы два раза обращаемся к функции `print` в рамках одной итерации: один раз до преобразования столбца, а второй – после. Причина в том, что операция приведения столбца к категориальному типу может занимать некоторое время, и нам бы хотелось знать, когда она началась и когда закончилась. К тому же это позволит легче отловить ошибку, если с каким-то столбцом что-то пойдет не так.

После выполнения всех преобразований нам нужно убедиться, что все прошло хорошо. Обратившись к атрибуту `dtypes` нашего датафрейма, мы можем легко узнать типы данных для всех столбцов в нем:

```
df.dtypes
```

Вывод:

```
Plate ID           category
Registration State  category
Vehicle Body Type  category
Vehicle Make       category
Violation Time     category
Street Name        category
Violation Legal Code category
Vehicle Color      category
dtype: object
```

Как видим, все столбцы в нашем датафрейме приобрели тип `category`. А как это повлияло на объем памяти? Проверим:

```
new_mem = df.memory_usage(deep=True).sum()
```

Результат на моем компьютере получился таким: 574 455 678 байт. При первоначальном объеме в 6 Гб мы можем сказать, что добились 10-кратного выигрыша:

```
new_mem / orig_mem
```

И это при том, что мы не потеряли ни толики данных и не лишились функционала по работе с ними.

Сколько памяти?

Метод `df.info` возвращает сводные данные по датафрейму, включая объем задействованной памяти. По умолчанию выполняется неглубокий анализ, в результате чего к показателям объема в выводе добавляются символы `+`. Если вам необходимо выполнить точный подсчет, воспользуйтесь параметром `memory_usage='deep'`, как показано ниже:

```
df.info(memory_usage='deep')
```

Это позволит получить полные и достоверные сведения о датафрейме.

Решение

```
filename = '../data/nyc-parking-violations-2020.csv'

df = pd.read_csv(filename,
                  usecols=['Plate ID', 'Registration State',
                           'Vehicle Make', 'Vehicle Color',
                           'Vehicle Body Type', 'Violation Time',
                           'Street Name', 'Violation Legal Code'])

orig_mem = df.memory_usage(deep=True).sum() ❶

for one_colname in df.columns: ❷
    print(f'Categorizing {one_colname}...')
    df[one_colname] = df[one_colname].astype('category') ❸
    print('\tDone.')

df.dtypes ❹

new_mem = df.memory_usage(deep=True).sum() ❺

print(new_mem / orig_mem) ❻
```

- ❶ Получаем объем памяти для хранения каждого столбца, суммируем значения и сохраняем в переменную.
- ❷ Проходим по всем столбцам в датафрейме.
- ❸ Приводим каждый столбец к категориальному типу.
- ❹ Получаем информацию о типах данных столбцов.
- ❺ Объем памяти после приведения типов сохраняем в другую переменную.
- ❻ Выводим информацию об экономии используемой памяти.

Дополнительные упражнения

1. Без расчетов: какие столбцы из загруженных вами не стоит приводить к категориальному типу? Мысленно ответив на этот вопрос, посчитайте количество уникальных значений во всех столбцах и определите (весьма формально), какие из столбцов больше всего выиграют от приведения к категориальному типу.

2. В упражнении 25 мы видели, что названия производителей машин и их цвета могут быть записаны в базе как угодно, с опечатками и разными подходами к сокращению слов. Если бы вы привели все столбцы к надлежащему виду перед преобразованием их в категории, дало бы это преимущество в плане расхода памяти? И почему?
3. Прочитайте только первые 10 000 строк из файла CSV и все столбцы. Определите десять столбцов, которые с наибольшей вероятностью выиграли бы от приведения к категориальному типу.

Apache Arrow

На протяжении книги я не раз упоминал, что *pandas* представляет собой своеобразную обертку на основе *NumPy*. Да, *pandas* предлагает очень богатый функционал, но в действительности данные хранятся в формате *NumPy*, а значит, и атрибут *dtype* находится в ведении этой библиотеки. Исключения составляют лишь строки, которые обычно хранятся в столбцах с типом данных *object*, и категории, о которых мы говорили ранее.

Но есть один проект с открытым исходным кодом под названием *Apache Arrow*, который способен разорвать эту тесную связь. *Arrow* разрабатывался как высокоэффективный механизм хранения данных в памяти. Ниже приведены некоторые его особенности и характеристики:

- технология *Arrow* работает не только с *pandas*, но и с другими языками программирования и системами, такими как *R* и *Apache Spark*;
- в *Python* привязка к *Arrow* реализована в виде библиотеки *PyArrow*;
- в *Arrow* реализованы собственные типы данных, похожие на аналоги в *NumPy*, но имеющие свои особенности и отличия;
- типы данных в *Arrow* допускают присутствие пропущенных значений. Это означает, что при наличии одного значения *NaN* в столбце целочисленного типа вам не придется преобразовывать столбец к типу с плавающей точкой;
- *Arrow* поддерживает два формата файлов: *feather* и *parquet*, которые являются двоичными, а значит, не занимают много места на диске и не требуют много времени для чтения и записи;
- *Arrow* позволяет читать и записывать файлы CSV.

Начнем с конца. По умолчанию *pandas* читает файлы CSV с использованием своего внутреннего движка. Но мы можем ускорить процесс загрузки данных, если явно попросим *pandas* использовать для загрузки движок *PyArrow*, как показано ниже:

```
df = pd.read_csv(filename, engine='pyarrow')
```

При задействовании этой библиотеки время загрузки данных из файла CSV на моем компьютере снизилось в 20 раз. Если только вы не работаете с очень маленькими наборами данных, пожалуй, стоит всегда пользоваться этой опцией при обращении к функции *read_csv*.

А что с двоичными форматами, поддерживаемыми *Arrow*? Формат *feather*, упомянутый мной ранее, сочетает в себе сжатие исходных данных и хранение их в двоичном виде, что позволяет уменьшить размер файлов и ускорить их чтение и запись в сравнении с форматами CSV и JSON.

Для сохранения датафрейма pandas в формате feather можно воспользоваться методом `to_feather`, работающим аналогично методам `to_csv` и `to_json`, что видно ниже:

```
df.to_feather('mydata.feather')
```

Считать данные из файла в формате feather в датафрейм можно при помощи функции `pd.from_feather`, которая действует похожим образом с функциями `from_csv` и `from_json`:

```
df = pd.from_feather('mydata.feather')
```

В pandas версии 2.0 была также в экспериментальном режиме реализована поддержка типов хранения PyArrow, которые можно использовать вместо NumPy. Таким образом, при чтении данных из файла CSV в pandas мы можем явно указать на необходимость использования типов PyArrow с помощью параметра `dtype_backend='pyarrow'`, как показано ниже:

```
df = pd.read_csv(filename, dtype_backend='pyarrow')
```

Теперь если вы проверите атрибут `dtypes` в своем датафрейме, то увидите, что типы данных в нем взяты из PyArrow, а не из NumPy.

Обратите внимание, что параметр `dtype_backend` никак не связан с движком, используемым при чтении файла CSV. Таким образом, вы можете использовать параметры `engine` и `dtype_backend` как по отдельности, так и вместе.

При этом, как я уже сказал, реализация PyArrow в качестве типов данных в pandas на момент написания книги обладает экспериментальным статусом, и нет никаких гарантий, что ее использование непременно будет приводить к ускорению процесса. Сразу после выхода pandas 2.0 я провел ряд опытов, в процессе которых выяснилось, что при использовании простых операций сравнения типы данных PyArrow показывают себя лучше, но на обработку сложных запросов с применением группировок и объединений меньше времени потребовалось старому доброму NumPy. Не сомневаюсь, что с течением времени ситуация улучшится, но на данный момент не стоит думать, что использование PyArrow непременно будет приводить к ускорению процессов. Вместо этого лучше провести несколько экспериментов и выяснить, какой выбор в вашем случае будет более эффективным.

Ответы на дополнительные упражнения

Упражнение 48.1

```
# Выводим имена столбцов в порядке убывания пользы от приведения
# к категориальному типу
(df.count() / df.nunique()).sort_values(ascending=False) * 100
```

Вывод:

Violation Legal Code	2.226554e+08
Registration State	1.837608e+07
Vehicle Body Type	7.619201e+05
Violation Time	6.835589e+05
Vehicle Color	6.383835e+05

```
Vehicle Make      2.386433e+05
Street Name      2.163256e+04
Plate ID         3.850149e+02
dtype: float64
```

Упражнение 48.2

Максимальное преимущество мы получили бы, если бы сначала нормализовали данные, а затем выполнили приведение типов. Чем больше повторений в столбце, тем больше пользы от категоризации строк. При выполнении нормализации данных мы снижаем количество уникальных значений в столбце и увеличиваем число повторений, тем самым делая столбец более пригодным для приведения к категориальному типу.

Упражнение 48.3

```
filename = '../data/nyc-parking-violations-2020.csv'
```

```
df = pd.read_csv(filename, nrows=10_000)
(df.count() / df.nunique()).sort_values(ascending=False).head(10)
```

Вывод:

```
Violation Description      5615.000000
Violation Legal Code       5615.000000
Law Section                3333.333333
Unregistered Vehicle?      2169.000000
Violation County           1086.333333
Issuing Agency             909.090909
Feet From Curb              833.333333
Violation In Front Of Or Opposite 796.800000
Date First Observed        400.000000
Plate Type                 344.827586
dtype: float64
```

УПРАЖНЕНИЕ 49. Быстрое чтение, быстрая запись

У каждого формата файлов есть свои преимущества и недостатки, и одной из характеристик форматов является скорость чтения и записи. Если у вас есть датафрейм, в каком формате его быстрее будет сохранить в файл: CSV, JSON или feather? Если вы прочитали врезку, посвященную Apache Arrow и feather, вы без труда сможете ответить на этот вопрос. А как понять, насколько быстрее тот или иной формат окажется в конкретном случае? И будет ли существенная разница в скорости при чтении данных из файлов CSV, JSON и feather в датафрейм?

Чтобы выяснить это, вам необходимо будет сделать следующее.

1. Загрузить данные о штрафах за парковку в Нью-Йорке из файла `nyc-parking-violations-2020.csv` в датафрейм.
2. Сохранить датафрейм в каждом из форматов: CSV, JSON и feather. Засеки-те время, которое потребовалось на сохранение файлов, и выведите его на экран с указанием типа файла.

3. Проверьте размер созданных файлов.
4. Прочитайте данные в датафреймах из созданных вами файлов. Снова засеки-те время и выведите его.

ПРИМЕЧАНИЕ. Набор с данными о парковках в Нью-Йорке довольно велик, и с ним может быть затруднительно работать, если на вашем компьютере нет 32 Гб свободной памяти. Если это ваш случай, можете ограничить количество используемых столбцов с помощью параметра `usecols`. Да, разница между форматами файлов в этом случае может оказаться не столь показательной, но вы по крайней мере сможете выполнить упражнение.

Подробный разбор

Первое, что нам нужно сделать, – это загрузить данные о штрафах за парковку в Нью-Йорке в датафрейм. Предположим, что у вас достаточно свободной памяти на компьютере. Тогда вы можете сделать это так, как показано ниже:

```
filename = '../data/nyc-parking-violations-2020.csv'
df = pd.read_csv(filename, low_memory=False)
```

Обратите внимание, что при загрузке данных мы передали параметр `low_memory=False`. Тем самым мы указали `pandas` на то, что у нас достаточно оперативной памяти для того, чтобы определение подходящих значений атрибута `dtype` осуществлялось на основе всего набора.

Теперь можем записать наши данные на диск в разных форматах и замерить потребовавшееся на это время. Для этого нам надо научиться замерять это самое время. На этот случай в стандартной библиотеке Python присутствует модуль `time`, в котором есть множество функций для работы с временем. Но самой полезной для замера времени операций является функция `time.perf_counter`. Эта функция задействует высокочувствительные часы и возвращает результат в виде количества секунд в формате числа с плавающей точкой. Возвращаемое этой функцией значение не стоит использовать для определения текущей даты и времени, но в рамках одной программы ее можно использовать для точного измерения прошедшего времени.

ПРИМЕЧАНИЕ. В стандартной библиотеке Python имеется также модуль `timeit` (<https://docs.python.org/3/library/timeit.html#module-timeit>), содержащий множество функций для выполнения замеров. Я являюсь большим поклонником этого модуля, и в упражнении 50 мы с ним немного поработаем. Но он обычно используется, когда нужно выполнить действие многократно и рассчитать среднее время. Мы же имеем дело с длительными операциями, так что ограничимся их однократным запуском и замером времени, для чего идеально подходит функция `perf_counter`.

Для записи датафрейма в форматах CSV, JSON и feather нам достаточно вызвать следующие три метода:

```
import time

df.to_json('parking-violations.json')
df.to_csv('parking-violations.csv')
df.to_feather('parking-violations.feather')
```

Но мы ведь хотим засечь время, необходимое для выполнения этих операций, так что дополним наш код нужными вызовами, как показано ниже:

```

start_time = time.perf_counter()           ❶
df.to_json('parking-violations.json')
end_time = time.perf_counter()             ❷
total_time = end_time - start_time         ❸
print(f'\tWriting JSON: {total_time=}')

start_time = time.perf_counter()           ❹
df.to_csv('parking-violations.csv')
end_time = time.perf_counter()
total_time = end_time - start_time
print(f'\tWriting CSV: {total_time=}')

start_time = time.perf_counter()           ❺
df.to_feather('parking-violations.feather')
end_time = time.perf_counter()
total_time = end_time - start_time
print(f'\tWriting feather: {total_time=}')

```

- ❶ Засекаем время перед сохранением данных в JSON.
- ❷ Фиксируем время после окончания операции.
- ❸ Вычисляем разницу.
- ❹ Повторяем для формата CSV.
- ❺ Повторяем для формата feather.

Этот код работает и выполняет возложенные на него функции. Но также он нарушает одно из основных правил программирования о недопустимости повторений. Ведь мы, по сути, три раза сделали одно и то же. Если бы можно было написать цикл, все было бы намного удобнее. Но как это сделать, если на каждой итерации нам нужно вызывать собственный метод? Хорошая новость состоит в том, что в Python мы можем любой объект, включая функции, сохранить в структуре данных, такой как словарь. Взгляните на приведенный ниже код:

```

root = 'parking-violations'
write_methods = {'JSON': df.to_json,
                 'CSV': df.to_csv,
                 'feather': df.to_feather}

for one_format, method in write_methods.items():
    print(f'Saving in {one_format}')
    start_time = time.perf_counter()
    method(f'parking-violations.{one_format.lower()}')
    end_time = time.perf_counter()

    total_time = end_time - start_time
    print(f'\tWriting {one_format}: {total_time=}')

```


Здесь мы с помощью цикла `for` проходим по словарю, сохраняя название формата во внутренней переменной `one_format`, а нужный нам метод – в переменной `method`. Далее мы выводим на экран текущий формат и засекаем время с помощью функции `time.perf_counter()`, получая текущее время, выраженное в секундах. После этого вызываем нужный метод при помощи `f`-строки. Записав файл на диск, мы снова вызываем функцию `time.perf_counter()` и сохраняем разницу в переменную `total_time` и выводим ее на экран.

На моем компьютере результат получился такой:

```
Saving in JSON
    Writing JSON: total_time=46.29149689315818
Saving in CSV
    Writing CSV: total_time=114.35314526595175
Saving in feather
    Writing feather: total_time=7.929971480043605
```

Как видите, для записи датафрейма в файл CSV нам потребовалось почти 2 мин (114 с). Запись в файл JSON заняла 46 с. А в формате feather данные записались всего за 8 (!!!) с, что в 14 раз быстрее в сравнении с CSV. Если и это не убедило вас переключиться на работу с feather, то я не знаю, что может убедить.

Обратите внимание, что перед определением словаря нам необходимо объявить переменную `df`. Также мы воспользовались методом `one_format.lower()` для приведения формата к нижнему регистру.

Насколько объемными получились созданные в результате файлы? Снова воспользуемся средствами из стандартной библиотеки Python. Мы уже встречались с функцией `glob.glob` в предыдущих упражнениях. Здесь мы используем ее для поиска всех файлов, начинающихся со строки, хранящейся в переменной `root`. Размер файла мы можем извлечь с помощью функции `os.stat`. Эта функция возвращает особую структуру данных по образу команды `stat` в Unix. В Python мы можем получить размер файла в байтах из атрибута `st_size` в возвращенной функцией `os.stat` структуре, как показано ниже:

```
for one_filename in glob.glob(f'{root}*'):
    print(f'{one_filename:27}: {os.stat(one_filename).st_size:,}')
```

Внутри `f`-строк мы здесь воспользовались двумя трюками, связанными с форматированием вывода:

- при выводе переменной `one_filename` мы добились имени файлов пробелами справа до достижения 27 символов. Это позволило выравнивать результаты;
- мы разделили разряды в числах запятыми, что сделало вывод более читаемым.

На моем компьютере результат оказался следующим:

```
parking-violations.json      : 8,820,247,015
parking-violations.csv       : 2,440,860,181
parking-violations.feather   : 1,466,535,674
```

Как видим, размер файла CSV оказался чуть больше 2 Гб, JSON занял аж 8 Гб (!) на диске, а feather – чуть больше 1 Гб. Это не единственная причина быстрой записи файлов feather на диск, а лишь одна из них.

Теперь измерим скорость чтения данных из разных форматов файлов. Мы воспользуемся той же техникой, что и выше: создадим словарь (на этот раз с именем `read_methods`), содержащий расширения файлов и вызываемые методы. Код получится следующим:

```
read_methods = {'JSON': pd.read_json,
                'CSV': pd.read_csv,
                'feather': pd.read_feather
}

for one_format, method in read_methods.items():
    print(f'Reading from {one_format}')
    start_time = time.perf_counter()
    df = read_methods[one_format](
        f'parking-violations.{one_format.lower()}')
    end_time = time.perf_counter()

    total_time = end_time - start_time
    print(f'\tReading {one_format}: {total_time=}')
```

Как и прежде, мы проходим в цикле по словарю, считывая расширение в переменную `one_format`, а метод – в переменную `method`. Выводим текущий формат на экран и вызываем функцию `time.perf_counter()`. С помощью конструкции `read_methods[one_format]` мы извлекаем нужный метод и вызываем его с соответствующим именем файла, переданным в качестве параметра. По завершении чтения данных мы снова вызываем функцию `time.perf_counter()`, сохраняем разницу в переменной `total_time` и выводим ее на экран.

Возможно, у вас, как и у меня, при запуске этого кода появится предупреждение `DtypeWarning`, о котором мы ранее уже говорили. Напомню, что оно появляется тогда, когда `pandas` пытается определить тип данных в столбце на основании определенной части строк, а не на всем наборе данных, с целью экономии памяти. В данном случае мы можем просто проигнорировать это предупреждение. На моем компьютере я получил следующие результаты:

```
Reading from JSON
    Reading JSON: total_time=469.92014819500037
Reading from CSV
    Reading CSV: total_time=35.20077076088637
Reading from feather
    Reading feather: total_time=9.132312984904274
```

Как видите, чтение из формата `JSON` заняло очень много времени – целых 469 с, или около 8 мин. На втором месте формат `CSV`, на чтение которого потребовалось 35 с. Ну а чемпионом, как и ожидалось, стал формат `feather`, чтение из которого заняло чуть больше 9 с.

Эта демонстрация явно дает понять, что формат файлов `feather` от `Apache Arrow` значительно превосходит форматы `CSV` и `JSON` как по скорости чтения, так и по скорости записи. Это не значит, что вам нужно бежать и переделывать все свои проекты на `feather`, но у этого формата действительно есть ряд преимуществ

как в отношении скорости обработки данных, так и в плане объема итоговых файлов на диске.

Решение

```
import glob
import os

filename = '../data/nyc-parking-violations-2020.csv'
df = pd.read_csv(filename, low_memory=False)

root = 'parking-violations'
write_methods = { 'JSON': df.to_json,           ❶
                  'CSV': df.to_csv,
                  'feather': df.to_feather }

for one_format, method in write_methods.items(): ❷
    print(f'Saving in {one_format}')
    start_time = time.perf_counter()
    method(f'parking-violations.{one_format.lower()}') ❸
    end_time = time.perf_counter()

    total_time = end_time - start_time
    print(f'\tWriting {one_format}: {total_time=}')

for one_filename in glob.glob(f'{root}*'):       ❹
    print(f'{one_filename:27}: {os.stat(one_filename).st_size:,}') ❺

read_methods = { 'JSON': pd.read_json,
                 'CSV': pd.read_csv,
                 'feather': pd.read_feather }

for one_format, method in read_methods.items(): ❻
    print(f'Reading from {one_format}')
    start_time = time.perf_counter()
    df = read_methods[one_format](
        f'parking-violations.{one_format.lower()}') ❼
    end_time = time.perf_counter()

    total_time = end_time - start_time
    print(f'\tReading {one_format}: {total_time=}')

```

- ❶ Словарь с форматами и методами.
- ❷ Проходим по форматам и методам записи.
- ❸ Вызываем метод с файлом.
- ❹ Проходим по созданным файлам.
- ❺ Используем функцию `os.stat` для получения размера файла.
- ❻ Проходим по форматам и методам чтения.
- ❼ Вызываем подходящий метод чтения для формата.

Дополнительные упражнения

1. Будет ли достигнуто ускорение, если читать файл CSV с использованием движка `pyarrow`? Иными словами, можно ли прочитать данные из файла в память быстрее при использовании разных движков?
2. Если при вызове функции `read_csv` явно указать параметр `dtypes`, это сократит или увеличит время чтения?
3. Сколько места в памяти занимает ваш датафрейм? А сколько места он будет занимать в виде таблицы Arrow?

Ускорение при помощи методов `eval` и `query`

На протяжении этой книги мы обсудили большое количество приемов для повышения эффективности обработки данных в `pandas`, в числе которых следующие:

- никогда не используйте традиционные циклы в Python (циклы `for` и генераторы списков) применительно к объектам `Series` и датафреймам;
- всегда пользуйтесь преимуществами транслирования, или бродкастинга;
- прибегайте к помощи атрибута доступа `str` всякий раз, когда работаете со строками;
- используйте минимально допустимые типы данных, которые позволят не потерять точность;
- избегайте использования двойных квадратных скобок при присваивании и извлечении значений;
- загружайте только те столбцы из исходных данных, с которыми в действительности будете работать;
- столбцы с большим количеством повторяющихся строковых значений должны быть приведены к категориальному типу;
- воспользуйтесь двоичными форматами, такими как `feather`, при многократном сохранении и извлечении данных.

Но даже при соблюдении всех этих правил мы можем обнаружить, что наши запросы выполняются недостаточно быстро и потребляют довольно много памяти. Часто это бывает заметно при выполнении арифметических операций над столбцами, в которых содержится большое количество строк. Похожие проблемы возникают при выполнении транслирования применительно к скалярному значению и объекту `Series`. Хотя `pandas` пытается использовать все преимущества высокоэффективных вычислений, заложенных в NumPy, большая часть работы по-прежнему выполняется интерпретатором Python, который серьезно уступает в эффективности языку C.

Еще одна проблема подстерегает нас при использовании объектов `Series` с логическими значениями в качестве масок на основе нескольких условий. Конечно, бывает удобно воспользоваться логическими операторами `&` и `|` для объединения условий, но за кулисами `pandas` вынужден создавать несколько объектов `Series`, которые затем нужно объединять. Если в исходном столбце у нас будет 1 млн значений, то при объединении трех условий будет создано как минимум 3 млн строк во временных объектах, которые затем должны быть объединены и применены к нашим данным.

Этой проблемы можно избежать, заодно сделав ваши запросы более понятными, если воспользоваться методом `query`, который мы упоминали в главе 2, и двумя разновидностями более общего метода `eval`. Это поможет избежать излишнего расходования памяти при использовании операторов `&` и `|` и зачастую позволит выполнять вычисления в библиотеке `numexpr`. Вместе экономия памяти и ускорение обработки могут приводить к существенному повышению эффективности операций при меньшем расходе ресурсов. В то же время стоит понимать, что применение этих методов не позволит вам решить все проблемы с производительностью. В частности:

- использование этих приемов при работе с небольшими наборами данных (объемом менее 10 000 строк) может привести к замедлению обработки, а не к ускорению;
- зачастую узким местом в программе является не область вычислений, а операции присваивания и извлечения значений. В таких случаях вы также не получите прироста производительности;
- вам понадобится устанавливать библиотеку `numexpr` из PyPI и затем явно указывать ее при работе с `pandas`. Если этого не делать, `pandas` будет по умолчанию использовать встроенный движок при разборе строк с запросами, что не приведет к повышению быстродействия;
- любые операции, не включающие в себя вычисления, сравнения и булевы операторы, будут либо порождать исключения, либо выполняться с обычной скоростью.

Давайте начнем с вызова метода `query` у датафрейма. После этого рассмотрим использование двух разновидностей метода `eval`, принадлежащих одному семейству.

Итак, если у нас есть датафрейм `df`, вызов метода `df.query` позволит выбрать строки из него согласно указанному фильтру. При этом сам запрос передается в формате, напоминающем язык SQL, с обращением к столбцам так, как если бы они представляли собой переменные. Результатом вызова метода будет датафрейм, представляющий собой подмножество из всех исходных столбцов и только тех строк, для которых выражение вернуло значение `True`. К примеру, если у вас есть датафрейм `df` с числовыми столбцами `a` и `b` и вы хотите оставить в нем только строки, в которых в столбце `a` будут значения больше 100, а в столбце `b` – меньше 700, то вы можете написать следующий фрагмент кода:

```
np.random.seed(0)
df = DataFrame(np.random.randint(0, 1000, [5,5]),
               index=list('vwxyz'),
               columns=list('abcde'))

df.loc[((df['a'] > 100) &
        (df['b'] < 700))]
```

Если воспользоваться методом `df.query`, ваш запрос существенно упростится и примет следующий вид:

```
df.query('a > 100 & b < 700')
```

Иногда версия с методом `query` будет выполняться быстрее, и уж точно всегда она будет задействовать меньше памяти. Причина в том, что в этом случае не будут создаваться два отдельных временных объекта `Series` – один с условием `a > 100`, а второй – с условием `b < 700`.

При использовании стандартных запросов эти временные объекты создаются невидимо для нас и нещадно расходуют свободную память. Я могу отметить, что многие разработчики предпочитают всегда использовать в своей работе метод `query` из-за его внешней простоты и экономии памяти.

Похожим образом вы можете воспользоваться методом `eval`, присутствующим у датафреймов в `pandas`. С помощью него можно строить запросы к данным подобно тому, как это происходит с методом `query`, а также выполнять некоторые другие действия, включая транслирование и присваивание. Пример:

```
df.eval('(a + b)* 3')
```

Это выражение позволит сложить содержимое столбцов `a` и `b` и умножить результат на 3 посредством транслирования. В результате мы получим объект `Series`. А что, если передать этому методу нашу предыдущую строку запроса?

```
df.eval('a > 100 & b < 700')
```

Мы снова получим объект `Series`. Но если метод `df.query` применит маску к датафрейму и вернет результат, то метод `df.eval` вернет саму маску в виде объекта `Series`, которую мы можем использовать по своему усмотрению. Мы можем даже создать новый столбец (или обновить содержимое существующего), присвоив в строке запроса выражение имени столбца, как показано ниже:

```
df.eval('f = d + e - c')
```

А при помощи тройных кавычек можно выполнять множественные присваивания следующим образом:

```
df.eval('''
f = d + e - c
g = a * 2
h = a * b
''')
```

В общем случае метод `df.eval` может быть использован как для наложения условий, так и для присваивания. Но применение тройных кавычек обязывает нас ограничиться только операциями присваивания – указание условий в этом случае недопустимо.

Третий и последний метод, который позволяет расходовать меньше памяти, ускорять вычисления и писать более легкий для чтения код, – это метод `pd.eval`. Обратите внимание, что на самом деле это не метод, а функция верхнего уровня из модуля `pandas`. Функцию `pd.eval` можно использовать вместо метода `df.eval`, но при этом необходимо явно указать датафрейм, с которым мы работаем. К примеру, мы можем написать:

```
pd.eval('df[df.a > 100 & df.b < 700]')
```

При вызове функции `pd.eval` вы можете воспользоваться точечной нотацией для обращения к столбцам в датафрейме вместо нотации с квадратными скобками, которую мы использовали на протяжении всей книги во избежание синтаксических сложностей. Таким образом, для обращения к столбцу `a` в датафрейме `df` вы можете написать `df.a`, а не `df['a']`. Но это также означает, что в названиях столбцов не должны присутствовать пробелы.

Этот код вернет все строки из датафрейма, удовлетворяющие условию. При этом мы написали запрос в виде строки, которая может быть передана пакету `numexpr`.

Этот код вернет все строки из датафрейма, удовлетворяющие условию. При этом мы написали запрос в виде строки, которая может быть передана пакету `pymexrt`.

При обработке данных этот пакет, как мы уже видели, задействует меньше памяти и (обычно) дает лучшую производительность. Обратите внимание, что вызов метода `df.eval` транслируется в вызов функции `pd.eval`, а значит, вы можете получить более высокую производительность, непосредственно прибегнув к помощи функции `pd.eval`. Но все же удобнее в использовании будет метод `df.eval`.

Как и в случае с методом `df.eval`, мы можем выполнять операции присваивания одному или нескольким столбцам в строке, передаваемой функции `pd.eval`. Но поскольку это не метод, а обобщенная функция, нам необходимо явно передать ссылку на наш датафрейм посредством параметра `target`. Присваивание будет выполнено в датафрейме, который мы получим на выходе:

```
pd.eval('f = df.d + df.e - df.c', target=df)
```

Так когда же стоит использовать каждый из описанных инструментов? В целом любой из них может быть эффективно использован при необходимости указания нескольких условий применительно к большим датафреймам. Чем объемнее датафрейм и чем более сложный у вас запрос, тем больший прирост скорости вы получите. Но даже если скорость останется прежней, при выполнении запроса в этом случае будет задействовано гораздо меньше памяти.

Краткое описание трех описанных выше инструментов:

- для извлечения выбранных строк из датафрейма можно воспользоваться методом `df.query`;
- для присваивания значений нескольким столбцам или выполнения запроса к датафрейму можно предпочесть метод `df.eval`;
- для работы с несколькими датафреймами лучше подойдет функция `pd.eval`. Но эта функция не поддерживает множественные присваивания, и ее синтаксис не так удобен.

Ответы на дополнительные упражнения

Упражнение 49.1

```
filename = '../data/nyc-parking-violations-2020.csv'
start_time = time.perf_counter()
df = pd.read_csv(filename, engine='pyarrow')
end_time = time.perf_counter()
total_time = end_time - start_time
print(f'Reading via pyarrow engine, {total_time=}')
```

Вывод:

Reading via pyarrow engine, total_time=9.923564148019068

Упражнение 49.2

```
start_time = time.perf_counter()
df = pd.read_csv(filename, low_memory=False,
```

```
dtype=dict(df.dtypes))
end_time = time.perf_counter()

total_time = end_time - start_time
total_time
```

Вывод:

63.521172957960516

Упражнение 49.3

```
# Таблица pandas
n = df.memory_usage(deep=True).sum()
print(f'{n:,}')
```

Вывод:

16,789,335,057

```
# Таблица Arrow
import pyarrow.feather as feather
read_arrow = feather.read_table('parking-violations.feather')

n = read_arrow.nbytes
print(f'{n:,}')
```

Вывод:

4,309,680,899

УПРАЖНЕНИЕ 50. *query и eval*

В этой главе мы в последний раз поработаем с набором данных, посвященным штрафным талонам за неправильную парковку в Нью-Йорке, при этом мы будем использовать как атрибут `df.loc`, так и методы `df.query` и `df.eval`. Для замера времени мы будем пользоваться модулем `timeit`. Итак, что вам нужно будет сделать.

1. Загрузите данные из файла `nyc-parking-violations-2020.csv` в датафрейм. Нам понадобятся только столбцы `Plate ID`, `Registration State`, `Plate Type`, `Feet From Curb`, `Vehicle Make` и `Vehicle Color`.
2. Переименуйте столбцы, дав им следующие имена: `pid`, `state`, `ptype`, `make`, `color` и `feet`. Это позволит писать более лаконичные запросы в дальнейшем.
3. Найдите все машины, зарегистрированные в штатах Нью-Йорк (NY), Нью-Джерси (NJ) или Коннектикут (CT), с помощью атрибута `.loc`.
4. Найдите эти же машины с использованием метода `df.query`.
5. Насколько быстрее выполнился запрос с `df.query`?
6. Воспользуйтесь для поиска по штатам методом `isin`. Как он показал себя в сравнении с логическими операторами?

7. Выполните все запросы, перечисленные ниже, с использованием атрибута `df.loc` и методов `df.query` и `df.eval` с замером времени. Какой способ оказался наиболее быстрым в каждом из случаев?
 - найдите все машины из штата Нью-Йорк;
 - найдите все частные легковые машины (`p_type == "PAS"`) из штата Нью-Йорк;
 - найдите все белые частные легковые машины из штата Нью-Йорк;
 - найдите все белые частные легковые машины из штата Нью-Йорк, которые были припаркованы дальше чем в одном футе от поребрика (`feet > 1`);
 - найдите все белые частные легковые машины марки Toyota из штата Нью-Йорк, которые были припаркованы дальше чем в одном футе от поребрика (`feet > 1`).
8. Какие запросы работают быстрее остальных?

Подробный разбор

В этом упражнении вы научитесь следующим вещам.

1. Как формулировать один и тот же запрос с использованием атрибута `loc` и методов `df.query` и `df.eval`.
2. Как использовать модуль `timeit` для оценки времени выполнения запросов.
3. Узнаете, что может приводить к замедлению запросов.
4. Узнаете о некоторых синтаксических проблемах, связанных с альтернативными механизмами выполнения запросов.

Первое, что нужно сделать, – это загрузить набор данных в датафрейм, как мы делали на протяжении всей книги:

```
df = pd.read_csv(filename,
                  usecols=['Plate ID', 'Registration State',
                           'Plate Type',
                           'Vehicle Make', 'Vehicle Color', 'Feet From Curb'])
```

Нет ничего плохого в том, чтобы так загружать данные. Но при использовании методов `df.query` и `df.eval` бывает неудобно формулировать запросы к столбцам, в названии которых есть пробелы. Да, можно в таких случаях воспользоваться обратными апострофами, но гораздо удобнее будет переименовать столбцы так, чтобы их имена напоминали обычные переменные. Это можно сделать, присвоив список строк атрибуту `df.columns`, как показано ниже:

```
df.columns = ['pid', 'state', 'ptype', 'make', 'color', 'feet']
```

Вы могли подумать, что более эффективно было бы указать новые имена столбцов прямо в функции `read_csv`. В конце концов, у этой функции есть параметр `names`, принимающий список строк для установки новых имен в загруженном датафрейме. Но все становится чуть сложнее, когда мы хотим и задать имена для столбцов в новом датафрейме (с помощью параметра `names`), и загрузить лишь определенные столбцы из источника (параметр `usecols`). Причина в том, что, передавая список строк параметру `names`, мы должны использовать эти имена, а не

оригинальные при выборе столбцов посредством параметра `usecols`. А это можно сделать только при переименовании всех столбцов в датафрейме, что нужно далеко не всегда.

На самом деле есть один способ обойти это ограничение, заключающийся в передаче списка целочисленных значений выбираемых столбцов параметру `usecols`, а не списка имен. В этом случае `pandas` выберет колонки по индексам. Далее мы можем присвоить столбцам новые имена с помощью параметра `names`. Вот что получится в результате:

```
df = pd.read_csv(filename,
                  usecols=[1, 2, 3, 7, 33, 37],
                  names=['pid', 'state', 'ptype',
                        'make', 'color', 'feet'])
```

Это сработает, и зачастую такой подход бывает удобен. Но у меня к нему есть пара претензий. Во-первых, бывает затруднительно перечислять все номера столбцов, которые мы хотим выбрать. К тому же при запуске этого кода на своем компьютере я получил предупреждение о недостатке памяти, которое мы уже видели ранее. В общем, мне кажется, что удобнее переименовывать столбцы в два приема – сначала загрузить нужные столбцы, а затем присвоить атрибуту `columns` список, состоящий из новых имен.

Теперь, когда мы загрузили датафрейм в память, можно начать выполнять запросы. В этом упражнении мы будем активно замерять время выполнения запросов и сравнивать результаты. Python предлагает удобный модуль `timeit`, который можно использовать в обычных программах, но в Jupyter присутствует особая магическая команда `%timeit`, которая может располагаться непосредственно в ячейках ноутбука. Таким образом, мы можем написать следующее выражение:

```
%timeit myfunc(2, 3, 4)
```

В результате функция `myfunc` будет вызвана несколько раз, и мы увидим среднее время ее выполнения с обнаруженными отклонениями. Количество запусков зависит от продолжительности выполнения функции. Если она выполняется доли секунды, Jupyter может запустить ее сотни или даже тысячи раз. А если функция у нас большая, запуск может быть произведен всего несколько раз.

ПРИМЕЧАНИЕ. При использовании магической команды `%timeit` в Jupyter необходимо не забывать, что тестируемый код должен быть однострочным и располагаться сразу следом за командой. Если вам нужно проверить быстродействие многострочного кода, заключите его в функцию и вызывайте ее. Или воспользуйтесь разновидностью команды `%timeit`, которая работает во всей ячейке, а не в одной строке. Также не забывайте при вызове функций ставить после имени функции круглые скобки.

В первом задании нас попросили найти все машины, зарегистрированные в штатах Нью-Йорк (NY), Нью-Джерси (NJ) или Коннектикут (CT), с помощью атрибута `.loc` и метода `query`. Также нам нужно сравнить быстродействие этих способов. Начнем с обращения к атрибуту `loc` с объединением трех отдельных запросов:

```
%%timeit
```

```
df.loc[(df['state'] == 'NY') |
```

```
(df['state'] == 'NJ') |
(df['state'] == 'CT']]
```

На моем компьютере этот запрос выполнен в среднем за 1.84 с (как обычно, время выполнения запроса может варьироваться от запуска к запуску).

Давайте подумаем, какие операции pandas вынужден был выполнить для обработки этого запроса:

- сравнить каждое значение в столбце df['state'] со значением 'NY';
- сравнить каждое значение в столбце df['state'] со значением 'NJ';
- сравнить каждое значение в столбце df['state'] со значением 'CT';
- выполнить операцию логического ИЛИ для первых двух объектов Series (для Нью-Йорка и Нью-Джерси);
- выполнить операцию логического ИЛИ для полученного результата и объекта Series для штата Коннектикут;
- применить итоговый объект Series в качестве маски к нашему датафрейму с помощью атрибута loc.

Неудивительно, что при таком большом количестве строк на выполнение операций сравнения требуется определенное время. Кроме того, и применение логического ИЛИ к двум объектам – операция не из дешевых. Использование метода query не поможет с первой проблемой – от сравнений мы никуда не денемся. Но мы можем существенно сэкономить на выполнении операций логического ИЛИ. Причина в том, что метод query использует в своей работе библиотеку numexpr, которая справляется с подобными задачами весьма эффективно. Насколько эффективно? Давайте проверим:

```
%timeit df.query('state == 'NY' or state == 'NJ' or state == 'CT')])
```

На моем компьютере это заняло 1.03 с в среднем, что на 0.8 с (или 45 %) быстрее в сравнении с исходным запросом. Это очень существенное ускорение, свидетельствующее об уместности применения метода query в подобных запросах.

Однако сравнение значений в столбце с тремя аббревиатурами штатов тоже занимает немало времени. Можем ли мы уменьшить количество операций сравнения? Да, если воспользуемся методом isin совместно со списком, как показано ниже:

```
%timeit df.loc[df['state'].isin(['NY', 'NJ', 'CT'])]
```

Выполнение этого запроса на моем компьютере заняло даже меньше времени, чем предыдущий запрос, – всего 0.77 с. Это составляет прирост быстродействия 58 % в сравнении с исходным запросом.

А может, мы сможем еще больше ускорить процесс, если одновременно воспользуемся методами query и isin? Давайте попробуем:

```
%timeit df.query('state.isin(["NY", "NJ", "CT"])')
```

К сожалению, в этом случае улучшения мы не получили – среднее время составило 0.8 с. Это лучше, чем исходный запрос, но ускорения, по сравнению с использованием только метода isin, не произошло.

Как видите, при оптимизации запросов бывает недостаточно просто выбрать используемую технику. Необходимо думать о том, что именно мы делаем и какие

действия для достижения результата будет выполнять pandas за кулисами. Также очень важно тестировать и сравнивать разные решения. По нашим запросам мы можем сказать две вещи. Во-первых, если объединять условия при помощи операторов `|` или `&`, то мы получим явную прибавку в скорости при использовании метода `query` вместо атрибута `loc` благодаря обращению к библиотеке `numexpr`. Во-вторых, использование метода `isin` практически всегда будет более оптимальным вариантом в сравнении с объединением нескольких условий, поскольку в этом случае движку нужно будет выполнить одну операцию сравнения вместо трех.

Далее нас попросили выполнить более сложные запросы с применением атрибута доступа `loc` и методов `df.query` и `df.eval`. При этом мы особо отметим, что эффект от использования методов `df.query` и `df.eval` растет по мере усложнения запросов.

Для начала нас попросили найти все машины из штата Нью-Йорк. Ниже показаны все три запроса вместе:

```
%timeit df.loc[(df['state'] == 'NY')]
%timeit df.query('state == "NY"')
%timeit df[df.eval('state == "NY"')]
```

На моем компьютере время их выполнения составило 903, 733 (на 19 % быстрее) и 758 мс (на 17 % быстрее) соответственно. Таким образом, обращение к атрибуту `loc` показало худший результат из трех вариантов, а вызовы методов `df.query` и `df.eval` оказались примерно равны по скорости.

Но ведь в результате вызова метода `df.eval` мы получаем объект `Series` с логическими значениями, который затем применяем к датафрейму. Возможно, вместо применения маски к `df` нам следует воспользоваться атрибутом `df.loc`. Давайте замерим время:

```
%timeit df.loc[df.eval('state == "NY"')]
```

Мы добились ускорения, но совсем небольшого, всего до 729 мс. В оставшихся решениях мы будем везде использовать атрибут `loc` для более точного сравнения.

Далее нам необходимо найти все частные легковые машины (`ptype == "PAS"`) из штата Нью-Йорк. Ниже приведены три решения:

```
%timeit df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS'))]
%timeit df.query('state == "NY" & ptype == "PAS"')
%timeit df.loc[df.eval('state == "NY" & ptype == "PAS"')]
```

Полученные времена: 1.27 с для атрибута `df.loc`, 965 мс для метода `df.query` (на 24% быстрее) и 924 мс для метода `df.eval` (на 27% быстрее). Здесь мы воспользовались оператором `&` для объединения двух логических объектов `Series`, полученных на основе сравнений. Если операцию логического ИЛИ можно оптимизировать с помощью метода `isin`, то для логического И подобного метода не существует.

Далее нас попросили еще больше сузить поиск, ограничившись только машинами белого цвета. Снова сравним запросы:

```
%%timeit
df.loc[((df['state'] == 'NY') &
        (df['ptype'] == 'PAS') &
```

```

(df['color'] == 'WHITE'))]

%%timeit
df.query(
    'state == "NY" & ptype == "PAS" & color == "WHITE"')

%%timeit
df.loc[df.eval(
    'state == "NY" & ptype == "PAS" & color == "WHITE"')]

```

На этот раз времена оказались такими: 1.34 с, 728 мс для метода `df.query` (на 45 % быстрее) и 727 мс для метода `df.eval` (также на 45 % быстрее). Как видим, добавление еще одного условия немного замедляет исходный запрос, зато запросы, использующие библиотеку `numexpr`, в результате ускорились. Я не уверен, почему именно так происходит. Возможно, движок `numexpr` активируется только при достижении определенного порога.

Добавим еще одно условие на дальность парковки от поребрика. Ниже показаны обновленные запросы:

```

%%timeit
df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS') &
        (df['color'] == 'WHITE') & (df['feet'] > 1)))]

%%timeit
df.query('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1')

%%timeit
df.loc[df.eval('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1')]

```

Обновленные времена: 1.31 с, 712 мс для `df.query` (на 45 % быстрее) и 706 мс для `df.eval` (на 46 % быстрее). И снова мы видим, что при усложнении запроса использование библиотеки `numexpr` позволяет сократить время выполнения.

Наконец, добавим условие на марку автомобиля в наши запросы, в результате чего получим следующее:

```

%%timeit
df.loc[((df['state'] == 'NY') &
        (df['ptype'] == 'PAS') &
        (df['color'] == 'WHITE') &
        (df['feet'] > 1) &
        (df['make'] == 'TOYOTA')))]

%%timeit
df.query('state == "NY" & ptype == "PAS" & color == "WHITE" &
        feet>1 & make == "TOYOTA"')

%%timeit
df.loc[df.eval('state == "NY" & ptype == "PAS" & color == "WHITE" &
        feet>1 & make == "TOYOTA"')]

```

Новые результаты сравнения: 1.75 с, 896 мс для `df.query` (на 49 % быстрее) и 899 мс для `df.eval` (на 48 % быстрее). Добавление этого условия увеличило время

выполнения всех трех запросов, но методы `df.query` и `df.eval` продолжили удерживать лидерство.

Означает ли это, что вам всегда стоит использовать методы `df.query` или `df.eval` при построении запросов? Многие разработчики на pandas ответят на этот вопрос утвердительно с учетом того, что даже простые запросы в этом случае работают быстрее. А при усложнении запросов рост производительности становится очевиден.

Однако не стоит делать акцент на скорости выполнения запросов до определения их узких мест. Помните, что метод `df.query` возвращает все столбцы в датафрейме, а если нам нужны лишь некоторые из них, то мы будем впустую расходовать драгоценную память. В то же время атрибут доступа `loc` позволяет задать не только селектор строк, но и селектор столбцов, что дает нам больше гибкости. Таким образом, обычно при написании запросов я использую атрибут `loc`, а уже на этапе оптимизации проверяю, можно ли добиться лучшей производительности и экономии памяти за счет использования методов `df.query` и `df.eval`.

Решение

```
filename = '../data/nyc-parking-violations-2020.csv'
df = pd.read_csv(filename,
                  usecols=['Plate ID',
                           'Registration State',
                           'Plate Type',
                           'Feet From Curb',
                           'Vehicle Make',
                           'Vehicle Color'])
df.columns = ['pid', 'state', 'ptype', 'make', 'color', 'feet']

%timeit df.loc[(df['state'] == 'NY') | (df['state'] == 'NJ') | (df['state'] == 'CT')]
%timeit df.query("state == 'NY' or state == 'NJ' or state == 'CT'")

%timeit df.loc[df['state'].isin(['NY', 'NJ', 'CT'])]
%timeit df.query('state.isin(["NY", "NJ", "CT"])')

%timeit df.loc[(df['state'] == 'NY')]
%timeit df.query('state == "NY"')
%timeit df[df.eval('state == "NY"')]

%timeit df.loc[df.eval('state == "NY"')]

%timeit df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS'))]
%timeit df.query('state == "NY" & ptype == "PAS"')
%timeit df.loc[df.eval('state == "NY" & ptype == "PAS"')]

%timeit df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS') & (df['color'] == 'WHITE'))]
%timeit df.query('state == "NY" & ptype == "PAS" & color == "WHITE"')
%timeit df.loc[df.eval('state == "NY" & ptype == "PAS" & color == "WHITE"')]

%timeit df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS') & (df['color'] ==
'WHITE') & (df['feet'] > 1))]
%timeit df.query('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1')
%timeit df.loc[df.eval('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1')]
```

```
%timeit df.loc[((df['state'] == 'NY') & (df['ptype'] == 'PAS') & (df['color'] == 'WHITE') &
(df['feet'] > 1) & (df['make'] == 'TOYOTA'))]
%timeit df.query('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1 &
make == "TOYOTA"')
%timeit df.loc[df.eval('state == "NY" & ptype == "PAS" & color == "WHITE" & feet > 1 &
make == "TOYOTA"')]
```

Дополнительные упражнения

В этом упражнении мы вместе построили несколько запросов с использованием атрибута `loc` и методов `df.query` и `df.eval`, сравнив их быстродействие. А теперь попробуйте сделать нечто похожее самостоятельно.

1. При использовании метода `df.query` вы можете задействовать в запросах операторы `and` и `or` вместо `&` и `|` благодаря библиотеке `numexpr`. Перепишите последний запрос с использованием этих операторов. Это как-то скажется на скорости выполнения запроса?
2. Лично я предпочитаю измерять расстояние в метрах, а не в футах. В связи с этим мне хотелось бы найти в нашем датафрейме информацию о всех машинах, которые были припаркованы более чем в одном метре от поребрика (1 м равен 3.28 фута). Выполните такой запрос с помощью атрибута `loc` и метода `df.query`. Какая из реализаций будет быстрее?
3. А что, если к нашему условию из предыдущего задания добавить ограничение на штат Нью-Йорк? Какой запрос будет быстрее и насколько?

Ответы на дополнительные упражнения

Упражнение 50.1

```
%timeit df.query('state == "NY" and ptype == "PAS" and color == "WHITE" and
feet > 1 and make == "TOYOTA"')
```

Вывод:

914 ms ± 7.43 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)

Упражнение 50.2

```
%timeit df.loc[(df['feet'] * 0.3048) > 1]
```

Вывод:

63.2 ms ± 2.4 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

```
%timeit df.query('(feet * 0.3048) > 1')
```

Вывод:

84.4 ms ± 1.51 ms per loop (mean ± std. dev. of 7 runs, 10 loops each)

Упражнение 50.3

```
%timeit df.loc[((df['feet'] * 0.3048) > 1) & (df['state'] == 'NY')]
```

Вывод:

```
507 ms ± 1.65 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

```
%timeit df.query('(feet * 0.3048) > 1 and state == "NY"')
```

Вывод:

```
314 ms ± 4.27 ms per loop (mean ± std. dev. of 7 runs, 1 loop each)
```

Заключение

Вычисления и анализ данных в pandas обычно выполняются намного быстрее, чем силами одного Python. Но даже с учетом этого при работе с действительно большими наборами данных то и дело возникает желание ускорить выполнение запросов и минимизировать расход памяти. В этой главе мы обсудили несколько техник и приемов, способных положительно влиять на эти аспекты. Как итог, лишь несколько советов:

- тщательно выбирайте столбцы при загрузке данных, чтобы не задействовать лишние колонки;
- всегда, когда это возможно, используйте категориальный тип данных для столбцов с большим количеством повторяющихся значений;
- отдавайте предпочтение формату feather в сравнении с традиционным CSV;
- всегда рассматривайте варианты оптимизации запросов с использованием методов `df.query` и `df.eval`.

Если вы дочитали книгу до этого места – что ж, поздравляю! Надеюсь, вы успешно выполнили все 50 основных упражнений и 150 дополнительных. Если это действительно так, значит вы в полной мере постигли тонкости работы с библиотекой pandas и сможете применять ее на практике в самых разных ситуациях.

А в качестве закрепления усвоенного материала в заключительной главе книги мы реализуем итоговый проект, в котором вы сможете использовать все полученные знания. Я очень надеюсь, что вы сможете выполнить этот проект самостоятельно. Это поможет вам обрести уверенность в себе и начать активно применять в своих задачах библиотеку pandas.

Глава 13

Итоговый проект

Поздравляю! Вы прошли все упражнения из этой книги! Если вы выполнили их самостоятельно, да еще и с дополнительными упражнениями, можете не сомневаться в своих способностях применять библиотеку `pandas` на практике.

Но, перед тем как расходиться, давайте выполним итоговый проект для закрепления полученных знаний. В этом проекте мы поработаем с набором данных, собранным Министерством просвещения США, который касается программ высшего и среднего специального образования. В этом наборе данных собрана информация о существующих образовательных программах, численности студентов, стоимости обучения, количестве выпускников и ожидаемой зарплате после трудоустройства. Анализ этого набора данных позволяет лучше понять устройство системы образования в США. А я могу добавить, что подобные наборы можно с пользой применять для оттачивания навыков анализа данных. После реализации этого проекта я очень рекомендую вам самостоятельно исследовать эти данные и попытаться с помощью библиотеки `pandas` ответить на интересующие лично вас вопросы.

Задача

Вот что вам необходимо сделать в рамках итогового проекта.

1. Загрузите в переменную `institutions_df` данные из файла `Most-Recent-Cohorts-Institution.csv.gz`. Нам понадобятся только столбцы `OPEID6`, `INSTNM`, `CITY`, `STABBR`, `FTFTRCTPELL`, `TUITIONFEE_IN`, `TUITIONFEE_OUT`, `ADM_RATE`, `NPT4_PUB`, `NPT4_PRIV`, `NPT41_PUB`, `NPT41_PRIV`, `NPT45_PUB`, `NPT45_PRIV`, `MD_EARN_WNE_P10` и `C100_4` (описание столбцов приведено в табл. 13.1).
2. Загрузите из файла `FieldOfStudyData1718_1819_PP.csv.gz` в датафрейм с именем `fields_of_study_df` данные о специальностях. Здесь нам понадобятся столбцы `OPEID6`, `INSTNM`, `CREDESC`, `CIPDESC` и `CONTROL`.
3. Ответьте на поставленные вопросы:
 - в каком штате располагается большинство заведений из этого набора данных?
 - в каком городе какого штата располагается больше всего учреждений?
 - сколько оперативной памяти вы сможете сэкономить, если выполните приведение столбцов `CITY` и `STABBR` в датафрейме `institutions_df` к категориальному типу?

- постройте гистограмму, показывающую, сколько бакалаврских программ предлагают заведения;
 - какое учебное заведение предлагает самое большое количество бакалаврских программ?
 - постройте гистограмму, показывающую, сколько программ высшего образования (магистратура и докторантура) предлагают заведения;
 - какое заведение предлагает самое большое количество программ высшего образования (магистратура + докторантура)?
4. Ответьте на следующие вопросы:
- сколько учреждений предлагают бакалаврские программы, но при этом не предлагают программы высшего образования (магистратура или докторантура)?
 - сколько заведений предлагают программы высшего образования (магистратура или докторантура), но при этом не предлагают бакалаврские программы?
 - сколько заведений предлагают бакалаврские программы, в названии которых есть словосочетание *Computer Science* (компьютерные науки)?
 - в столбце CONTROL содержится информация о типе заведений. Сколько бакалаврских программ, связанных с компьютерными науками, предлагает каждый тип заведения?
5. Постройте круговую диаграмму, показывающую количество предлагаемых бакалаврских программ, связанных с компьютерными науками, с разделением по типам заведений.
6. Определите минимальную, медианную, среднюю и максимальную сумму оплаты (поле TUITIONFEE_OUT) за бакалаврские программы, связанные с компьютерными науками.
7. Соберите те же описательные статистики, но с группировкой по типам заведений (CONTROL).
8. Определите наличие корреляции между нормой поступления (ADM_RATE) и стоимостью обучения (TUITIONFEE_OUT). Как бы вы прокомментировали результаты?
9. Постройте диаграмму рассеяния, в которой на оси *x* будет располагаться стоимость обучения, на оси *y* – норма поступления, а цвет будет соответствовать медианной зарплате выпускников через 10 лет после выпуска (MD_EARN_WNE_P10). Воспользуйтесь цветовой картой 'Spectral'. Где на графике располагаются выпускники с минимальным уровнем зарплаты?
10. Определите, какие заведения входят в первые 25 % по стоимости обучения и в первые 25 % по доле студентов с грантами (т. е. получивших помощь правительства). Выведите только название заведения, город и штат с сортировкой по названию заведения.
11. В поле NPT4_PUB хранится средняя чистая стоимость для государственных учреждений (стоимость для студентов из этого же штата), а в поле

NPT4_PRIV – для частных учреждений. Столбцы NPT41_PUB и NPT45_PUB показывают среднюю стоимость, выплачиваемую студентами из нижней категории дохода (категории 1) и высшей (категории 5) соответственно в государственных учреждениях. В столбцах NPT41_PRIV и NPT45_PRIV хранятся аналогичные показатели, но для частных заведений. В скольких учреждениях нижний квартиль получает деньги (т. е. соответствующее значение в полях NPT41_PUB или NPT41_PRIV ниже нуля)?

12. Вычислите среднюю долю суммы оплаты студентами из нижней категории дохода от суммы оплаты студентами из высшей категории дохода в государственных учреждениях.
13. Вычислите среднюю долю суммы оплаты студентами из нижней категории дохода от суммы оплаты студентами из высшей категории дохода в частных учреждениях.
14. Теперь попытаемся определить, какие заведения предлагают наилучшую *окупаемость инвестиций* (return on investment – ROI) по всем дисциплинам:
 - в каких государственных заведениях со средней стоимостью оплаты (поле NPT4_PUB), входящей в нижний квартиль, ожидаемая зарплата через 10 лет после выпуска (поле MD_EARN_WNE_P10) входит в верхний квартиль?
 - а как насчет частных заведений?
 - наблюдается ли корреляция между нормой поступления (поле ADM_RATE) и нормой выпуска (поле C100_4)? Иными словами, повышаются ли шансы на выпуск в заведениях с жестким отбором студентов?
 - в заведениях какого типа (поле CONTROL) студенты могут надеяться на самую большую зарплату через 10 лет после выпуска?
 - зарабатывают ли студенты заведений, входящих в расширенную Лигу плюща (традиционная восьмерка плюс Массачусетский Технологический институт (MIT), Стэнфорд (Stanford) и Чикагский университет (University of Chicago)), больше, чем средний выпускник частного заведения? И если да, то насколько? Список названий заведений, входящих в расширенную Лигу плюща: 'Harvard University', 'Massachusetts Institute of Technology', 'Yale University', 'Columbia University in the City of New York', 'Brown University', 'Stanford University', 'University of Chicago', 'Dartmouth College', 'University of Pennsylvania', 'Cornell University', 'Princeton University';
 - сколько в среднем зарабатывают студенты заведений из разных штатов через 10 лет после выпуска?
15. Постройте столбчатую диаграмму, показывающую среднюю зарплату студентов через 10 лет после выпуска по штатам, с сортировкой по возрастанию суммы.
16. Постройте диаграмму размаха по тем же данным.

Столбцы и их описание

Описание столбцов из двух файлов CSV, с которыми мы будем работать в этом проекте, показано в табл. 13.1 и 13.2.

Таблица 13.1. Описание столбцов в файле Most-Recent-Cohorts-Institution.csv.gz

Имя столбца	Описание	Пример значения
OPEID6	Уникальный идентификатор (целочисленный) учебного заведения	1002
INSTNM	Название учебного заведения	"Alabama A & M University"
CITY	Город учебного заведения	"Chicago"
STABBR	Штат (сокращенно) учебного заведения	"AL"
FTFTRCTPELL	Процент студентов с грантами (государственной поддержкой)	0.6925
TUITIONFEE_IN	Стоимость обучения для студентов из этого же штата	10024.0
TUITIONFEE_OUT	Стоимость обучения для студентов из других штатов	18634.0
ADM_RATE	Норма поступления	0.8965
NPT4_PUB	Чистая стоимость обучения (для государственных учебных заведений). Значение NaN, если заведение частное	15529.0
NPT4_PRIV	Чистая стоимость обучения (для частных учебных заведений). Значение NaN, если заведение государственное	NaN
NPT41_PUB	Средняя стоимость, выплачиваемая студентами из нижней категории дохода (категории 1) в государственных учреждениях. Значение NaN, если заведение частное	14694.0
NPT41_PRIV	Средняя стоимость, выплачиваемая студентами из нижней категории дохода (категории 1) в частных учреждениях. Значение NaN, если заведение государственное	NaN
NPT45_PUB	Средняя стоимость, выплачиваемая студентами из верхней категории дохода (категории 5) в государственных учреждениях. Значение NaN, если заведение частное	20483.0
NPT45_PRIV	Средняя стоимость, выплачиваемая студентами из верхней категории дохода (категории 5) в частных учреждениях. Значение NaN, если заведение государственное	NaN


```
'NPT41_PUB', 'NPT41_PRIV',
'NPT45_PUB', 'NPT45_PRIV',
'MD_EARN_WNE_P10', 'C100_4']])
```

Загрузка информации о специальностях, предлагаемых учебными заведениями

Теперь загрузим в переменную `fields_of_study_df` данные о специальностях, в последние несколько лет предлагаемых к изучению в этих заведениях:

```
fields_filename = '../data/FieldOfStudyData1718_1819_PP.csv.gz'
fields_of_study_df = pd.read_csv(fields_filename,
                                usecols=['OPEID6', 'INSTNM',
                                         'CREDESC', 'CIPDESC',
                                         'CONTROL'])
```

Теперь, когда оба набора данных загружены в память, мы можем начать отвечать на поставленные вопросы.

В каком штате располагается большинство заведений из этого набора данных?

Это классический пример для применения операции группировки. Мы можем сгруппировать данные по столбцу `STABBR` (сокращенное название штата) и воспользоваться агрегирующим методом `count`. Это позволит нам узнать, как часто тот или иной штат встречается в нашем наборе данных. Но для применения метода подсчета значений нам нужно передать какой-то столбец. Мы выберем столбец `OPEID6`, хранящий уникальные идентификаторы учебных заведений. В итоге получим простой запрос:

```
(
    institutions_df
    .groupby('STABBR')['OPEID6'].count()
)
```

Таким образом мы узнаем, как часто каждый штат встречается в наших данных. Но нас попросили узнать, в каком штате располагается наибольшее количество заведений. Для этого нужно отсортировать полученный результат по количеству заведений в порядке убывания. После этого узнать штат с наибольшим числом учреждений не составит труда:

```
(
    institutions_df
    .groupby('STABBR')['OPEID6'].count()
    .sort_values(ascending=False)
    .head(1)
)
```

Вывод:

```
STABBR
CA      705
Name: OPEID6, dtype: int64
```

Как видим, лидером по количеству учебных заведений (705) является штат Калифорния.

В каком городе какого штата располагается больше всего учреждений?

Теперь узнаем, в каком городе находится больше всего учебных заведений. На первый взгляд ответ на этот вопрос кажется тривиальным – подставь город вместо штата в предыдущем запросе, и получишь информацию по городам. Однако мы знаем, что в нашем наборе данных есть город Спрингфилд (Springfield) из штата Иллинойс и город с таким же названием из штата Массачусетс. Таким образом, для правильного анализа нам необходимо выполнить группировку сразу по двум столбцам: по штату и по городу. Это позволит нам получить информацию по уникальным городам с принадлежностью к штатам. Запрос показан ниже:

```
(
    institutions_df
    .groupby(['STABBR', 'CITY'])['OPEID6'].count()
    .sort_values(ascending=False)
    .head(1)
)
```

Вывод:

```
STABBR  CITY
NY      New York      81
Name: OPEID6, dtype: int64
```

Здесь мы снова применили метод агрегации `count` к полю `OPEID6`, поскольку нам нужно провести подсчет по столбцу, не входящему в группировку. Затем мы опять отсортировали данные и извлекли первую строку. Как видим, больше всего учебных заведений (81) располагается в Нью-Йорке из одноименного штата.

Сколько памяти можно сэкономить, если привести столбцы CITY и STABBR в датафрейме institutions_df к категориальному типу?

Если учесть, что названия городов и аббревиатуры штатов представлены текстовыми данными с большим количеством повторений, имеет смысл задуматься о преобразовании этих столбцов в категории. Однако, как в случае с любой оптимизацией, мы должны тщательно измерить возможный прирост производительности и расход ресурсов до и после этого действия.

Нас попросили узнать о потенциальном снижении расхода памяти в результате приведения этих полей к категориальному виду. Легче всего подобный анализ произвести с помощью метода `memory_usage`. Не забывайте использовать при этом параметр `deep=True`, который позволит получить данные обо всех столбцах, включая объекты, на которые они ссылаются. В главе 12 мы видели, что использование этого параметра существенно сказывается на результатах. Ниже показано, как можно узнать полный размер датафрейма:

```
pre_category_memory = (
    institutions_df
```

```
.memory_usage(deep=True)
.sum()

print(f'{pre_category_memory:,}')
```

Вывод:

2,105,659

Мы рассчитали общий объем задействованной памяти и сохранили результат в переменной `pre_category_memory`. Затем мы вывели его на экран с запятыми в виде разделителей разрядов. Формат мы задали с помощью f-строки, указав после переменной символ `:` (двоеточие).

Теперь давайте приведем наши столбцы к категориальному типу:

```
institutions_df['CITY'] = (
    institutions_df['CITY']
    .astype('category')
)

institutions_df['STABBR'] = (
    institutions_df['STABBR']
    .astype('category')
)
```

Сохраним новые данные по расходу памяти в переменную `post_category_memory`, как показано ниже:

```
post_category_memory = (
    institutions_df
    .memory_usage(deep=True)
    .sum()
)

savings = pre_category_memory - post_category_memory
print(f'{savings:,}')
```

Вывод:

579,371

Как видите, нам удалось сэкономить почти треть занимаемой памяти. Очень неплохо для пары лишних потраченных секунд.

Постройте гистограмму, показывающую, сколько бакалаврских программ предлагают заведения

Здесь мы хотим узнать, сколько учебных заведений предлагают до 10, 20, 30 и т. д. бакалаврских программ своим студентам.

Для построения подобной диаграммы нам сначала нужно узнать, какие бакалаврские программы предлагают наши учебные заведения, т. е. извлечь из да-

тафрейма `fields_of_study_df` все строки, в которых в столбце `CREDESC` находится значение `'Bachelors Degree'`:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC'] == 'Bachelors Degree']
)
```

Теперь можно сгруппировать данные по столбцу с названием учебного заведения. При этом мы рискуем, что наш метод агрегации `count` применится ко всем столбцам. Во избежание этого выберем один столбец `CIPDESC`, как показано ниже:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC'] == 'Bachelors Degree']
    .groupby('INSTNM')['CIPDESC'].count()
)
```

Вывод:

```
INSTNM
AI Miami International University of Art and Design    8
ASA College                                           5
ATA College                                           1
ATI College-Norwalk                                  1
Aarhus University                                    1
..
York College of Pennsylvania                         50
York St John University                              4
York University                                       8
Young Harris College                                24
Youngstown State University                         83
Name: CIPDESC, Length: 3006, dtype: int64
```

Мы получили объект `Series`, в котором в качестве индекса присутствуют названия заведений, а в качестве значений – количество предлагаемых ими бакалаврских программ. Осталось построить диаграмму на основе полученных данных, которая показана на рис. 13.1.

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC'] == 'Bachelors Degree']
    .groupby('INSTNM')['CIPDESC'].count()
    .plot.hist()
)
```

Как видим, очень много учебных заведений (более 1400!) предлагают своим студентам менее 20 бакалаврских программ. Меньше 600 заведений предлагают от 20 до 50 программ, и менее чем в 200 заведениях студенты могут выбирать из полусотни и более бакалаврских программ.

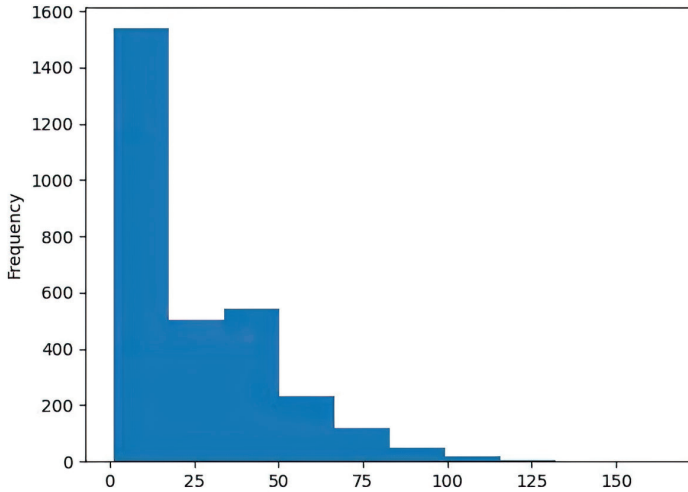


Рис. 13.1. Гистограмма, показывающая, сколько бакалаврских программ предлагают разные учебные заведения

Какое учебное заведение предлагает самое большое количество бакалаврских программ?

Теперь, когда мы узнали распределение количества предлагаемых институтами бакалаврских программ, можно задаться вопросом о том, в каком из них студентам предлагается сделать выбор из наибольшего количества доступных программ.

К счастью, у нас уже все есть для этого подсчета благодаря выполненной ранее группировке. Таким образом, нам нужно просто отсортировать полученные результаты в нисходящем порядке и оставить первые 10 строк. Сделаем это:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDDESC'] == 'Bachelors Degree']
    .groupby('INSTNM')['CIPDESC'].count()
    .sort_values(ascending=False)
    .head(10)
)
```

Вывод:

INSTNM	
Westminster College	165
Pennsylvania State University-Main Campus	141
University of Washington-Seattle Campus	137
Ohio State University-Main Campus	126
Bethel University	125
University of Minnesota-Twin Cities	116
Arizona State University Campus Immersion	116
University of Arizona	116

```
Anderson University                114
Purdue University-Main Campus     114
Name: CIPDESC, dtype: int64
```

Как видно по результатам, больше всех бакалаврских программ (165) своим студентам предоставляет Колледж Вестминстер, следом за которым идут основной кампус Университета штата Пенсильвания (141) и кампус в Сиэтле Университета Вашингтона (137).

Постройте гистограмму, показывающую, сколько программ высшего образования (магистратура и докторантура) предлагают заведения

Теперь, когда мы посчитали количество предлагаемых учебными заведениями бакалаврских программ, пришло время обратить взор на программы высшего образования, включая магистратуру и докторантуру. Нюанс здесь заключается в том, что мы не можем сравнивать столбец CREDESC с одной строкой. Вместо этого мы должны учитывать как магистратуру ("Master's Degree"), так и докторантуру ("Doctoral Degree"). Мы уже умеем делать это с помощью метода `isin`, возвращающего `True` в случае совпадения с одним из вхождений в перечислении.

Для начала отберем все учебные заведения, предлагающие такого рода программы:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC']
         .isin(["Master's Degree", "Doctoral Degree"])]
)
```

Теперь можно выполнить группировку и применить метод агрегации `count`, как мы уже делали ранее:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC']
         .isin(["Master's Degree", "Doctoral Degree"])]
    .groupby('INSTNM')['CIPDESC'].count()
)
```

Вывод:

```
INSTNM
A T Still University of Health Sciences    13
AI Miami International University of Art and Design    2
AOMA Graduate School of Integrative Medicine    2
Aarhus University                            14
Abertay University                           1
..
York College                                2
York College of Pennsylvania                 4
York St John University                      4
York University                             7
Youngstown State University                 42
Name: CIPDESC, Length: 2487, dtype: int64
```

Наконец, построим гистограмму на основе этих данных. Итоговая гистограмма показана на рис. 13.2:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC']
        .isin(["Master's Degree", "Doctoral Degree"])]
    .groupby('INSTNM')['CIPDESC'].count()
    .plot.hist()
)
```

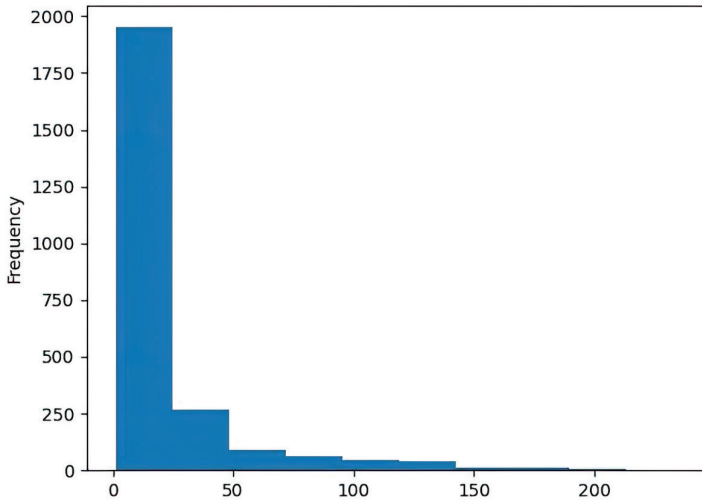


Рис. 13.2. Гистограмма, показывающая, сколько программ высшего образования предлагают заведения

Как видим, подавляющее число заведений предлагает своим студентам на выбор менее 25 программ высшего образования. А больше 50 программ предлагают совсем немногие заведения. И уж совсем единицы имеют в распоряжении более 200 программ магистратуры и докторантуры.

Какое заведение предлагает самое большое количество программ высшего образования (магистратура + докторантура)?

В связи с проведенным выше исследованием очень интересно было бы узнать, какие учебные заведения предлагают своим студентам больше всех программ высшего образования. Нам не составит труда это узнать:

```
(
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC']
        .isin(["Master's Degree", "Doctoral Degree"])]
    .groupby('INSTNM')['CIPDESC'].count()
    .sort_values(ascending=False)
    .head(10)
)
```

Вывод:

INSTNM	
University of Washington-Seattle Campus	237
Pennsylvania State University-Main Campus	230
New York University	226
University of Minnesota-Twin Cities	205
Ohio State University-Main Campus	200
University of Southern California	199
Arizona State University Campus Immersion	199
University of Arizona	195
University of Florida	194
University of Illinois Urbana-Champaign	190
Name: CIPDESC, dtype: int64	

Лидером здесь стал кампус в Сиэтле Университета Вашингтона (237), а следом за ним идут основной кампус Университета штата Пенсильвания (230) и Университет Нью-Йорка (226).

ПРИМЕЧАНИЕ. Не стоит по количеству предлагаемых учебным заведением программ обучения делать выводы о качестве образования в нем. Решающим фактором, особенно в отношении выбора программы высшего образования, должно быть то, насколько подходит вам конкретная программа и кто именно посоветовал вам конкретное образовательное учреждение. Так что не рассматривайте полученные результаты в качестве мотива для выбора стратегии обучения. Кроме того, я в действительности не думаю, что чем больше программ предлагает учебное заведение, тем лучше.

Сколько учреждений предлагают бакалаврские программы, но при этом не предлагают программы высшего образования (магистратура или докторантура)?

Хотя в большинстве случаев учебные заведения предлагают как бакалаврские программы обучения, так и программы высшего образования, бывают случаи, когда институт специализируется только на одном типе программ. На этот раз нам нужно найти количество заведений, которые предлагают бакалаврские программы, но при этом не предлагают программы магистратуры или докторантуры.

Для ответа на этот вопрос мы для начала отберем заведения, предлагающие бакалаврские программы, и заведения, располагающие магистратурой или докторантурой. Подобные запросы мы уже писали выше. На этот раз мы сохраним результаты в переменных в виде объектов Series, чтобы в дальнейшем можно было производить вычисления на их основе:

```
ug_schools = (
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC'] == 'Bachelors Degree',
         'INSTNM']
)

grad_schools = (
    fields_of_study_df
    .loc[fields_of_study_df['CREDESC']
```

```
.isin(["Master's Degree", "Doctoral Degree"]),
'INSTNM']
)
```

Значения этих объектов Series содержат названия учебных заведений. Однако, поскольку мы извлекали эти названия из датафрейма `fields_of_study_df`, среди них будет большое количество дубликатов, а в строках будут находиться конкретные предлагаемые институтами программы. Но пока мы не будем применять метод `unique` с помощью метода `apply`, ведь в результате мы получим массив NumPy, а нам еще нужно будет воспользоваться функционалом `pandas`.

Теперь, когда у нас есть эти два объекта Series, как можно определить, какие учебные заведения предлагают бакалаврские программы, но при этом не предлагают программы высшего образования? Мы можем снова положиться на метод `isin`. Если нам нужно найти заведения, которые есть в обоих перечислениях, можно воспользоваться следующим выражением:

```
ug_schools.isin(grad_schools)
```

В результате мы получим объект Series с логическими значениями. Но нам нужно сделать обратное – найти элементы из первого перечисления, которые отсутствуют во втором. Обратим логику с помощью логического оператора `~` (тильда):

```
~ug_schools.isin(grad_schools)
```

В результате получим объект Series с логическими значениями. Применив его в качестве маски к объекту `ug_schools`, мы получим строки, соответствующие заведениям с бакалавриатом, в которых отсутствуют программы высшего образования:

```
ug_schools[~ug_schools.isin(grad_schools)]
```

Но мы помним про нашу маленькую проблему с дубликатами. На этом этапе можно избавиться от них, воспользовавшись методом `drop_duplicates`, как показано ниже:

```
ug_schools[~ug_schools.isin(grad_schools)].drop_duplicates()
```

Для получения количества таких заведений обратимся к атрибуту `size`:

```
ug_schools[~ug_schools.isin(grad_schools)].drop_duplicates().size
```

Вывод:

```
923
```

Получается, что в 923 учебных заведениях, предлагающих бакалаврские программы, отсутствует магистратура и докторантура.

Сколько заведений предлагают программы высшего образования (магистратура или докторантура), но при этом не предлагают бакалаврские программы?

Для ответа на этот вопрос нам достаточно развернуть логику предыдущего запроса, как показано ниже:

```
grad_schools[~grad_schools.isin(ug_schools)].drop_duplicates().size
```

Вывод:

404

Как видите, 404 заведения с программами высшего образования не располагают программами для бакалавров.

Сколько заведений предлагают бакалаврские программы, в названии которых есть словосочетание Computer Science (компьютерные науки)?

Теперь нас заинтересовал вопрос о количестве учебных заведений, в которых студентам предлагаются бакалаврские программы с упоминанием компьютерных наук (Computer Science) в названии. Каждое заведение именует свои программы обучения по-своему, так что велика вероятность, что мы обнаружим не все программы, которые нас в действительности могли бы заинтересовать. Мы ограничимся поиском программ, содержащих в названии словосочетание 'Computer Science'.

Для начала найдем все строки, в которых в столбце CIPDESC содержится подстрока 'Computer Science':

```
fields_of_study_df['CIPDESC'].str.contains('Computer Science')
```

Также нам необходимо ограничить выбор только бакалаврскими программами, что мы и сделаем следующим образом:

```
fields_of_study_df['CIPDESC'].str.contains('Computer Science') &
fields_of_study_df['CREDESC'] == 'Bachelors Degree'
```

Этот объединенный запрос вернет объект Series с логическими значениями. Далее мы можем применить его в качестве маски к объекту fields_of_study_df с помощью атрибута loc:

```
(
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
          .str.contains('Computer Science')) &
          (fields_of_study_df['CREDESC'] == 'Bachelors Degree')]
)
```

Но нам не нужны все столбцы – нужны только названия заведений, чтобы мы могли посчитать их. Это можно реализовать посредством добавления селектора столбцов в атрибут loc для отбора колонки INSTNM, как показано ниже:

```
(
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
          .str.contains('Computer Science')) &
          (fields_of_study_df['CREDESC'] == 'Bachelors Degree'),
          'INSTNM']
)
```

Вывод:

```
360          Alabama State University
671    Auburn University at Montgomery
```

```

1110          Faulkner University
1936          Oakwood University
2040          Samford University
...
223531        University of Wollongong
223619          La Trobe University
223971        Anglia Ruskin University
224154          University of Brighton
224709          CETYS Universidad
Name: INSTNM, Length: 824, dtype: object

```

В результате мы получим объект Series, содержащий 824 названия заведений. Но опять же имена в этом списке могут оказаться неуникальными, поскольку один институт может предлагать несколько бакалаврских программ по компьютерным наукам. Для избавления от дубликатов воспользуемся методом `unique` и получим количество уникальных названий заведений с помощью атрибута `size`, как показано ниже:

```

(
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
        .str.contains('Computer Science')) &
        (fields_of_study_df['CREDESC'] == 'Bachelors Degree'),
        'INSTNM']
    .unique()
    .size
)

```

Вывод:

762

Сколько бакалаврских программ, связанных с компьютерными науками, предлагает каждый тип заведения?

В нашем наборе данных все учебные заведения разделены на четыре типа (поле `CONTROL`): государственные (`Public`), частные некоммерческие (`Private, nonprofit`), частные коммерческие (`Private, for-profit`) и иностранные (`Foreign`). Давайте узнаем, сколько заведений каждого типа предлагают интересующие нас бакалаврские программы по компьютерным наукам.

Начнем с нашего предыдущего запроса до применения метода `unique`:

```

(
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
        .str.contains('Computer Science')) &
        (fields_of_study_df['CREDESC'] == 'Bachelors Degree'),
        'INSTNM']
)

```

Далее необходимо выполнить группировку по столбцу `CONTROL`, поскольку нужно узнать, сколько бакалаврских программ, связанных с компьютерными наука-

ми, предлагает каждый тип заведения. Для этого в качестве селектора столбцов мы укажем столбцы INSTNM и CONTROL и сгруппируем данные по столбцу CONTROL, как показано ниже:

```
fields_of_study_df.loc[(fields_of_study_df[
    'CIPDESC'].str.contains('Computer Science')) &
    (fields_of_study_df['CREDESC'] ==
     'Bachelors Degree'), ['CONTROL',
     'INSTNM']].groupby('CONTROL')
```

В результате получим объект DataFrameGroupBy, к которому теперь применим метод count:

```
fields_of_study_df.loc[(fields_of_study_df[
    'CIPDESC'].str.contains('Computer Science')) &
    (fields_of_study_df['CREDESC'] ==
     'Bachelors Degree'), ['CONTROL',
     'INSTNM']].groupby('CONTROL').count()
```

Вывод:

	INSTNM
CONTROL	
Foreign	32
Private, for-profit	18
Private, nonprofit	501
Public	273

Как видим, больше остальных (501) такие программы предлагают частные некоммерческие учебные заведения, следом за которыми идут государственные (273).

Постройте круговую диаграмму, показывающую количество предлагаемых бакалаврских программ, связанных с компьютерными науками, с разделением по типам заведений

Представлять результаты предыдущего упражнения в виде таблицы можно, но это не совсем наглядно. Гораздо лучше такие данные с небольшим количеством элементов группировки выводить на круговой диаграмме. Давайте возьмем наш предыдущий запрос и извлечем из него только столбец INSTNM, как показано ниже:

```
fields_of_study_df.loc[(fields_of_study_df[
    'CIPDESC'].str.contains('Computer Science')) &
    (fields_of_study_df['CREDESC'] ==
     'Bachelors Degree'), ['CONTROL',
     'INSTNM']].groupby('CONTROL').count()['INSTNM']
```

Вывод:

CONTROL	
Foreign	32
Private, for-profit	18
Private, nonprofit	501

```
Public                273
Name: INSTNM, dtype: int64
```

В результате мы получим объект Series, в индексе которого будут располагаться типы учебных заведений. Подготовить этот запрос для вывода в виде круговой диаграммы можно очень просто – достаточно добавить к нему вызов метода `.plot.pie()`, как показано ниже. Сама диаграмма продемонстрирована на рис. 13.3:

```
(
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
        .str.contains('Computer Science')) &
        (fields_of_study_df['CREDESC'] == 'Bachelors Degree'),
    ['CONTROL', 'INSTNM']]
    .groupby('CONTROL').count()['INSTNM']
    .plot.pie()
)
```

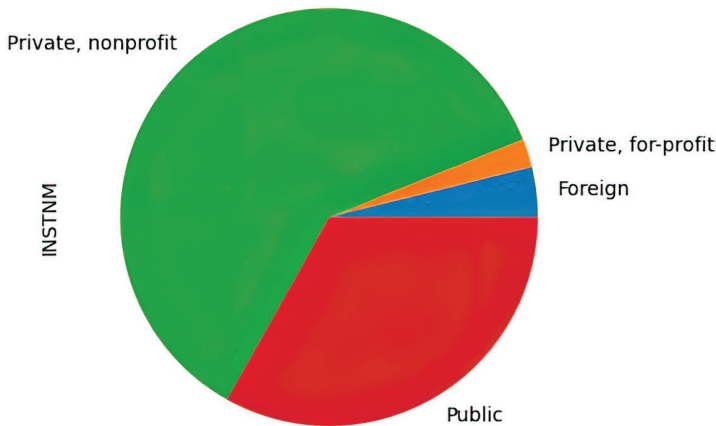


Рис. 13.3. Круговая диаграмма, показывающая количество предлагаемых бакалаврских программ по компьютерным наукам с разделением по типам заведений

Далее мы будем говорить о стоимости прохождения бакалаврских программ по компьютерным наукам в учебных заведениях США. А для этого нужно подготовиться. Сначала найдем все заведения, в которых такие программы имеются. Этот запрос будет похож на один из предыдущих, за исключением того, что мы выберем три столбца: `OPEID6` (уникальный идентификатор заведения), `CONTROL` (тип заведения) и `INSTNM` (название заведения):

```
comp_sci_universities = (
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
        .str.contains('Computer Science')) &
        (fields_of_study_df['CREDESC'] == 'Bachelors Degree'),
    ['OPEID6', 'CONTROL', 'INSTNM']]
)
```

Мы собрали все нужные нам строки в одном датафрейме, но в индексе у нас сейчас те же значения, что и в датафрейме `fields_of_study_df`. Это не так плохо, но для ответов на поставленные вопросы нам нужно будет объединять этот датафрейм с датафреймом `institutions_df`, а объединение требует соответствия индексов. Поэтому немного поправим наш запрос, установив правильный индекс:

```
comp_sci_universities = (
    fields_of_study_df
    .loc[(fields_of_study_df['CIPDESC']
          .str.contains('Computer Science')) &
          (fields_of_study_df['CREDDESC'] == 'Bachelors Degree'),
          ['OPEID6', 'CONTROL', 'INSTNM']]
    .set_index('OPEID6')
)
comp_sci_universities
```

Вывод:

	CONTROL	INSTNM
OPEID6		
1005	Public	Alabama State University
8310	Public	Auburn University at Montgomery
1003	Private, nonprofit	Faulkner University
1033	Private, nonprofit	Oakwood University
1036	Private, nonprofit	Samford University
...	...	...
30914	Foreign	University of Wollongong
30961	Foreign	La Trobe University
34353	Foreign	Anglia Ruskin University
35173	Foreign	University of Brighton
41839	Foreign	CETYS Universidad

[824 rows x 2 columns]

Убедимся, что датафрейм `institutions_df` также подготовлен для объединения в плане индекса:

```
institutions_df[['OPEID6', 'TUITIONFEE_OUT']].set_index('OPEID6')
```

Обратите внимание, что мы здесь не изменяем датафрейм `institutions_df`, а возвращаем новый датафрейм с индексом `OPEID6`.

Теперь, когда два датафрейма имеют одинаковые индексы, мы можем объединить их следующим образом:

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']]
          .set_index('OPEID6'))
)
```

Но нам нужен не весь датафрейм, а только столбец `TUITIONFEE_OUT`. Получим его так:

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']]
        .set_index('OPEID6'))
    ['TUITIONFEE_OUT']
)
```

Вывод:

```
OPEID6
1005      19396.0
8310      18820.0
1003      22990.0
1033      19990.0
1036      34198.0
...
30914      NaN
30961      NaN
34353      NaN
35173      NaN
41839      NaN
Name: TUITIONFEE_OUT, Length: 1241, dtype: float64
```

Здесь мы получили стоимости всех бакалаврских программ по компьютерным наукам в учебных заведениях из нашего набора данных.

Определите минимальную, медианную, среднюю и максимальную сумму оплаты (поле `TUITIONFEE_OUT`) за бакалаврские программы, связанные с компьютерными науками

Мы могли бы вычислить все нужные нам агрегаты по отдельности, но зачем, когда у нас есть метод `describe`, позволяющий рассчитать их все вместе?

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']]
        .set_index('OPEID6'))
    ['TUITIONFEE_OUT']
    .describe()
)
```

Вывод:

```
count      1139.000000
mean       26996.482002
std        14903.734488
min         3154.000000
25%        13202.500000
50%        24320.000000
75%        37836.000000
max         61671.000000
Name: TUITIONFEE_OUT, dtype: float64
```

Соберите те же описательные статистики, но с группировкой по типам заведений (CONTROL)

Далее нас попросили сгруппировать наши описательные статистики по типам заведений. Это можно сделать, применив метод `groupby('CONTROL')` к результату объединения, выбрав поле `TUITIONFEE_OUT` и вызвав для него метод `describe`, как показано ниже:

```
comp_sci_universities.join(institutions_df[
    ['OPEID6', 'TUITIONFEE_OUT']].set_index('OPEID6')).groupby(
    'CONTROL')['TUITIONFEE_OUT'].describe()
```

Вывод:

	count	mean	std	...	50%	75%	max
CONTROL				...			
Foreign	0.0	NaN	NaN	...	NaN	NaN	NaN
Private, for-profit	136.0	12359.161765	1954.582965	...	12233.0	12233.0	25820.0
Private, nonprofit	582.0	33789.982818	15973.754351	...	34245.0	47128.5	61671.0
Public	421.0	22333.437055	9618.584458	...	21312.0	27540.0	47220.0

[4 rows x 8 columns]

Но у этого подхода есть две проблемы. Во-первых, как мы видим, для иностранных заведений (Foreign) в таблице выводятся только нули и значения NaN. Во-вторых, довольно странно видеть типы заведений в индексе, а статистические показатели – в столбцах. Оба замечания носят чисто эстетический характер, но раз уж мы работаем с данными, давайте их немного почистим. Сначала воспользуемся методом `dropna`, чтобы удалить строку Foreign, в которой присутствуют пропущенные значения:

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']])
    .set_index('OPEID6')
    .groupby('CONTROL')['TUITIONFEE_OUT'].describe()
    .dropna()
)
```

Хорошо, а как насчет моего желания вывести статистические показатели в строках, а не в столбцах? В этом нам поможет метод `transpose`, предназначенный для транспонирования данных в датафрейме:

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']])
    .set_index('OPEID6')
    .groupby('CONTROL')['TUITIONFEE_OUT'].describe()
    .dropna()
    .transpose()
)
```

Из-за частого использования этого метода для него придумали более короткий атрибут с именем `T`, которым можно воспользоваться следующим образом:

```
(
    comp_sci_universities
    .join(institutions_df[['OPEID6', 'TUITIONFEE_OUT']]
        .set_index('OPEID6'))
    .groupby('CONTROL')['TUITIONFEE_OUT'].describe()
    .dropna()
    .T
)
```

Вывод:

CONTROL	Private, for-profit	Private, nonprofit	Public
count	136.000000	582.000000	421.000000
mean	12359.161765	33789.982818	22333.437055
std	1954.582965	15973.754351	9618.584458
min	8280.000000	4300.000000	3154.000000
25%	12054.000000	20260.000000	15636.000000
50%	12233.000000	34245.000000	21312.000000
75%	12233.000000	47128.500000	27540.000000
max	25820.000000	61671.000000	47220.000000

ПРИМЕЧАНИЕ. Поскольку `transpose` – это метод, его необходимо вызывать со скобками. В свою очередь `T` представляет собой свойство, обращение к которому не требует постановки скобок. Таким образом, вызов `T()` приведет к ошибке. В остальном они идентичны.

Определите наличие корреляции между нормой поступления (ADM_RATE) и стоимостью обучения (TUITIONFEE_OUT).

Как бы вы прокомментировали результаты?

Мы часто слышим, что в дорогие учебные заведения бывает очень трудно поступить. Правда ли это? Давайте попробуем найти корреляцию между нормой поступления в тот или иной институт (ADM_RATE) и стоимостью обучения в нем (TUITIONFEE_OUT). Воспользуемся для этого методом `corr`, как показано ниже:

```
institutions_df[['ADM_RATE', 'TUITIONFEE_OUT']].corr()
```

Вывод:

	ADM_RATE	TUITIONFEE_OUT
ADM_RATE	1.000000	-0.309658
TUITIONFEE_OUT	-0.309658	1.000000

Как мы уже говорили ранее, коэффициент корреляции, равный нулю, означает отсутствие взаимосвязи между показателями. Значение 1 говорит о полной положительной линейной связи, а -1 – о полной отрицательной. В данном случае мы получили коэффициент корреляции, равный -0.3 , который показывает, что между нормой поступления и стоимостью обучения в учебных заведениях есть слабая отрицательная корреляция. Иными словами, это означает, что при увеличении стоимости обучения незначительно снижается норма поступления, что действительно характерно для многих американских институтов. Таким образом, мы можем сказать, что в учебных заведениях, в которые трудно поступить, обычно выше стоимость обучения.

Постройте диаграмму рассеяния, в которой на оси x будет располагаться стоимость обучения, на оси y – норма поступления, а цвет будет соответствовать медианной зарплате выпускников через 10 лет после выпуска (MD_EARN_WNE_P10). Воспользуйтесь цветовой картой 'Spectral'. Где на графике располагаются выпускники с минимальным уровнем зарплаты?

Начнем с простой диаграммы рассеяния, которую построим с помощью следующей инструкции:

```
institutions_df.plot.scatter(x='TUITIONFEE_OUT', y='ADM_RATE')
```

На полученном графике мы видим небольшой нисходящий тренд из левого верхнего угла к правому нижнему, что соответствует определенному нами ранее коэффициенту корреляции между этими двумя столбцами.

Но мы хотим также проанализировать на этом графике любопытный столбец с именем MD_EARN_WNE_P10, который говорит о том, сколько в среднем зарабатывают выпускники учебного заведения через 10 лет после выпуска. Это позволит нам ответить сразу на несколько вопросов. К примеру, мы уже выяснили, что в более дорогие заведения труднее поступить. Но есть ли существенная выгода от увеличенной платы за обучение? В частности, может ли студент в будущем рассчитывать на более высокую зарплату, отучившись в элитном университете? Для этого нас попросили вывести цветом на диаграмме поле MD_EARN_WNE_P10. Результат показан на рис. 13.4:

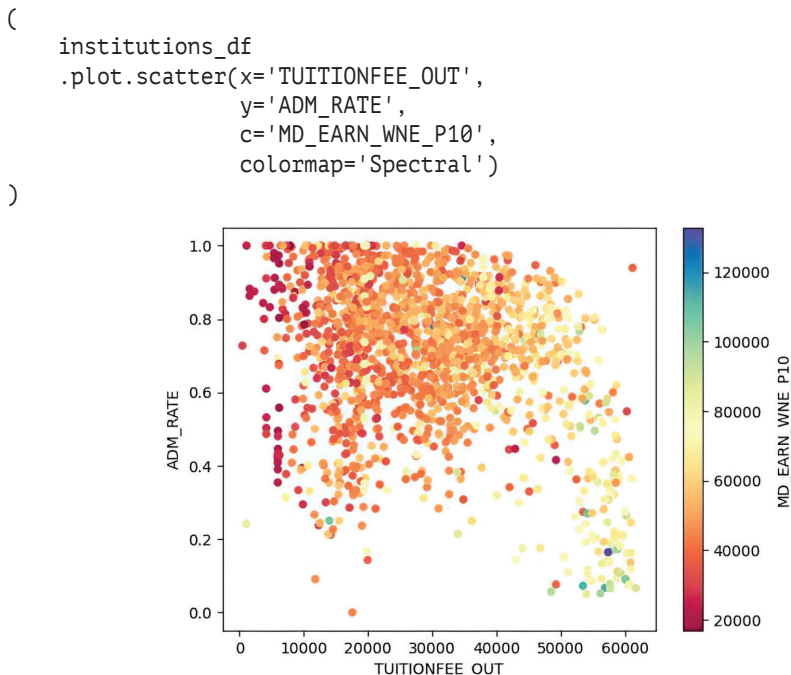


Рис. 13.4. Диаграмма рассеяния, сравнивающая стоимость обучения и норму поступления

На нашей диаграмме точки красного цвета, как понятно из легенды, соответствуют низким средним зарплатам в районе 20 000 долл. в год, а синие – высоким, около 120 000 долл. Неудивительно, что мы видим много красных точек ближе к левому верхнему углу (более дешевые заведения с высокой нормой поступления), тогда как желтые, зеленые и синие точки больше сосредоточены в правом нижнем углу диаграммы (более дорогие заведения с низкой нормой поступления). Таким образом, можно сделать вывод, что выпускники более элитных институтов в среднем действительно зарабатывают больше.

Определите, какие заведения входят в первые 25 % по стоимости обучения и в первые 25 % по доле студентов с грантами (т. е. получивших помощь правительства). Выведите только название заведения, город и штат с сортировкой по названию заведения

Теперь исследуем элитные учебные заведения. Сначала нас попросили вывести заведения, входящие в первые 25 % по стоимости обучения (самые дорогие) и в первые 25 % по доле студентов с грантами (т. е. получивших помощь правительства). Гранты от правительства позволяют одаренным детям из малоимущих семей получать достойное образование. Таким образом, в этом упражнении мы попробуем найти элитные учебные заведения, охотно принимающие в свои ряды абитуриентов с финансовыми трудностями.

Для этого воспользуемся методом `quantile(0.75)` применительно к столбцу `TUITIONFEE_OUT`, чтобы определить верхний квартиль по стоимости обучения. Этот же вызов мы применим и к столбцу `FTFTPCTPELL`, что позволит определить отсечку по учебным заведениям с большим количеством студентов с государственной поддержкой. Далее мы оставим в датафрейме только строки, в которых значения в соответствующих столбцах выше установленных порогов, как показано ниже:

```
(
    institutions_df
    .loc[(institutions_df['TUITIONFEE_OUT'] >
          institutions_df['TUITIONFEE_OUT'].quantile(0.75)) &
         (institutions_df['FTFTPCTPELL'] >
          institutions_df['FTFTPCTPELL'].quantile(0.75))]
)
```

В результате мы получим все строки из датафрейма `institutions_df`, в которых значения в столбцах `TUITIONFEE_OUT` и `FTFTPCTPELL` входят в верхний квартиль. Но опять же нам не нужны все столбцы – достаточно вывести название заведения, город и штат. Для этого добавим селектор столбцов в атрибут `loc`:

```
(
    institutions_df
    .loc[(institutions_df['TUITIONFEE_OUT'] >
          institutions_df['TUITIONFEE_OUT'].quantile(0.75)) &
         (institutions_df['FTFTPCTPELL'] >
          institutions_df['FTFTPCTPELL'].quantile(0.75)),
         ['INSTNM', 'CITY', 'STABBR']]
)
```


Наконец, нас попросили отсортировать результат по названию учебного заведения. Сделаем это при помощи метода `sort_values` следующим образом:

```
(
    institutions_df
    .loc[(institutions_df['TUITIONFEE_OUT'] >
          institutions_df['TUITIONFEE_OUT'].quantile(0.75)) &
          (institutions_df['FTFTPCTPELL'] >
           institutions_df['FTFTPCTPELL'].quantile(0.75)),
          ['INSTNM', 'CITY', 'STABBR']]
    .sort_values(by='INSTNM')
)
```

Вывод:

	INSTNM	CITY	STABBR
5491	Antioch College	Yellow Springs	OH
1206	Berea College	Berea	KY
1930	Berkeley College-Woodland Park	Woodland Park	NJ
1932	Bloomfield College	Bloomfield	NJ
2334	Chowan University	Murfreesboro	NC
...	...	...	...
5895	Institute of Medical Ultrasound	Atlanta	GA
3579	Mount Mary University	Milwaukee	WI
1487	Pine Manor College	Chestnut Hill	MA
4647	SAE Institute of Technology-Nashville	Nashville	TN
2945	Williamson College of the Trades	Media	PA

[18 rows x 3 columns]

Всего в нашем наборе данных оказалось 18 таких учебных заведений. Честь им и хвала.

В поле `NPT4_PUB` хранится средняя чистая стоимость для государственных учреждений (стоимость для студентов из этого же штата), а в поле `NPT4_PRIV` – для частных учреждений. Столбцы `NPT41_PUB` и `NPT45_PUB` показывают среднюю стоимость, выплачиваемую студентами из нижней категории дохода (категории 1) и высшей (категории 5) соответственно в государственных учреждениях. В столбцах `NPT41_PRIV` и `NPT45_PRIV` хранятся аналогичные показатели, но для частных заведений. В скольких учреждениях нижний квартиль получает деньги (т. е. соответствующее значение в полях `NPT41_PUB` или `NPT41_PRIV` ниже нуля)?

Теперь взглянем на стоимость обучения под несколько иным углом. По каждому институту у нас есть информация о средней чистой стоимости обучения для государственных учреждений (поле `NPT4_PUB`) и для частных (поле `NPT4_PRIV`). В то же время эти показатели разбиваются на среднюю стоимость, выплачиваемую

студентами из нижней категории дохода (поля NPT41_PUB и NPT41_PRIV соответственно) и из высшей категории (поля NPT45_PUB и NPT45_PRIV).

Давайте узнаем, в скольких институтах студенты из нижней категории дохода по итогу не тратят деньги, а получают их. Если бы нас интересовали только государственные заведения, мы могли бы узнать это, написав следующее выражение:

```
institutions_df.loc[institutions_df['NPT41_PUB'] < 0,
                    'INSTNM'].count()
```

Для частных выражение выглядело бы так:

```
institutions_df.loc[institutions_df['NPT41_PRIV'] < 0,
                    'INSTNM'].count()
```

Узнать суммарное количество таких заведений можно при помощи логического оператора |, как показано ниже:

```
institutions_df.loc[((institutions_df['NPT41_PUB'] < 0) |
                    (institutions_df['NPT41_PRIV'] < 0)),
                    'INSTNM'].count()
```

Вывод:

12

Как видим, в сумме таких учебных заведений 12. Но мы можем получить этот результат и иначе, а именно путем сложения значений в двух столбцах посредством метода add с параметром fill_value=0. Далее мы выберем только отрицательные значения, как показано ниже:

```
institutions_df.loc[institutions_df['NPT41_PUB'].add(
    institutions_df['NPT41_PRIV'], fill_value=0) < 0,
                    'INSTNM'].count()
```

Так мы получим тот же самый результат. Это не значит, что один или другой способ лучше, а лишний раз свидетельствует о возможности решать задачи в pandas самыми разными способами.

Вычислите среднюю долю суммы оплаты студентами из нижней категории дохода от суммы оплаты студентами из высшей категории дохода в государственных учреждениях

Для вычисления запрошенной средней доли необходимо разделить значения в столбце NPT41_PUB (нижний квартиль) на значения из столбца NPT45_PUB (верхний квартиль) и взять среднее, как показано ниже:

```
(institutions_df['NPT41_PUB'] / institutions_df['NPT45_PUB']).mean()
```

Результат составил 0.5233221766529079, или 52 %.

Вычислите среднюю долю суммы оплаты студентами из нижней категории дохода от суммы оплаты студентами из высшей категории дохода в частных учреждениях

Повторим это вычисление для частных заведений:

```
(institutions_df['NPT41_PRIV'] / institutions_df['NPT45_PRIV']).mean()
```

Получили 71 %. Таким образом, студентам не только приходится платить больше за обучение в частных заведениях, но в таких заведениях студенты из нижней категории дохода в среднем платят больше.

Теперь перейдем к вопросам, касающимся наилучшей окупаемости инвестиций для студентов.

В каких государственных заведениях со средней стоимостью оплаты (поле NPT4_PUB), входящей в нижний квартиль, ожидаемая зарплата через 10 лет после выпуска (поле MD_EARN_WNE_P10) входит в верхний квартиль?

Для ответа на вопрос о том, какие институты предлагают наилучшую окупаемость инвестиций, нас попросили найти государственные учебные заведения со средней стоимостью оплаты, входящей в нижний квартиль, и ожидаемой зарплатой через 10 лет после выпуска, входящей в верхний квартиль. Это не составит труда:

```
(
    institutions_df
    .loc[(institutions_df['NPT4_PUB']
          <= institutions_df['NPT4_PUB'].quantile(0.25)) &
         (institutions_df['MD_EARN_WNE_P10']
          >= institutions_df['MD_EARN_WNE_P10'].quantile(0.75)),
         ['INSTNM', 'STABBR', 'CITY']]
    .sort_values(by=['STABBR', 'CITY'])
)
```

Вывод:

	INSTNM	STABBR	CITY
203	California State University-Dominguez Hills	CA	Carson
267	De Anza College	CA	Cupertino
208	California State University-Los Angeles	CA	Los Angeles
363	Moorpark College	CA	Moorpark
228	Canada College	CA	Redwood City
...	...	...	...
2101	CUNY Hunter College	NY	New York
2102	CUNY John Jay College of Criminal Justice	NY	New York
2108	CUNY Queens College	NY	Queens
2097	College of Staten Island CUNY	NY	Staten Island
3218	Texas A & M International University	TX	Laredo

[22 rows x 3 columns]

Этот запрос представляет собой разновидность запроса, который мы уже писали ранее. В селектор столбцов мы включили только название заведения, штат и город. Это позволило нам отсортировать результаты сначала по штату, а затем по городу. В итоге мы получили 22 заведения из штатов Калифорния, Флорида, Нью-Йорк, Техас и Нью-Мексико.

А как насчет частных заведений?

Посмотрим, как дело с окупаемостью инвестиций обстоит в частных учебных заведениях:

```
(
institutions_df
.loc[(institutions_df['NPT4_PRIV']
      <= institutions_df['NPT4_PRIV'].quantile(0.25)) &
      (institutions_df['MD_EARN_WNE_P10']
      >= institutions_df['MD_EARN_WNE_P10'].quantile(0.75)),
['INSTNM', 'STABBR', 'CITY']]
.sort_values(by=['STABBR', 'CITY'])
)
```

Вывод:

	INSTNM	STABBR	CITY
4795	Columbia Southern University	AL	Orange Beach
3695	Stanford University	CA	Stanford
4208	Mercy Hospital School of Practical Nursing-Pla...	FL	Miami
842	Brigham Young University-Idaho	ID	Rexburg
895	Graham Hospital School of Nursing	IL	Canton
...	...	...	...
3962	Center for Advanced Legal Studies	TX	Houston
3319	Brigham Young University	UT	Provo
4322	Western Governors University	UT	Salt Lake City
3556	Beloit College	WI	Beloit
4732	American Public University System	WV	Charles Town

[30 rows x 3 columns]

Здесь мы видим уже 30 заведений из разных штатов. Среди наиболее известных можно выделить Гарвард, Стэнфорд и Принстон.

Наблюдается ли корреляция между нормой поступления (поле ADM_RATE) и нормой выпуска (поле C100_4)?

Теперь узнаем, существует ли зависимость между нормой поступления и нормой выпуска. Иными словами, повышаются ли шансы на выпуск в заведениях с жестким отбором студентов. Сделать это несложно:

```
institutions_df[['C100_4', 'ADM_RATE']].corr()
```

Вывод:

	C100_4	ADM_RATE
C100_4	1.000000	-0.336871
ADM_RATE	-0.336871	1.000000

Здесь мы наблюдаем слабую отрицательную корреляцию. Таким образом, можно сделать вывод, что в заведениях с жестким отбором процент выпуска больше. Это может показаться странным, но на самом деле ничего удивительного тут нет, ведь заведения, охотно принимающие студентов, с большей вероятностью рискуют набрать и тех, кто не закончит обучение.

В заведениях какого типа (поле CONTROL) студенты могут надеяться на самую большую зарплату через 10 лет после выпуска?

Теперь попробуем узнать, при выпуске из заведений какого типа студенты могут надеяться на большую зарплату. Таким образом, мы должны сгруппировать наши данные по столбцу CONTROL. К тому же мы снова объединим два датафрейма, но только после выполнения группировки в присоединяемом наборе данных:

```
(
    institutions_df[['OPEID6', 'MD_EARN_WNE_P10']]
    .set_index('OPEID6')
    .join(fields_of_study_df
          .groupby('OPEID6')['CONTROL'].min())
    .groupby('CONTROL')
    .mean()
)
```

Вывод:

	MD_EARN_WNE_P10
CONTROL	
Private, for-profit	30474.754943
Private, nonprofit	48530.408744
Public	40314.442820

Результаты получились вполне ожидаемыми. Абитуриенты, поступающие в частные коммерческие заведения, по прошествии времени зарабатывают меньше всех, а больше всех зарабатывают бывшие студенты некоммерческих частных заведений. Государственные учреждения занимают среднюю позицию. Но, конечно, это только усредненные показатели. Я знаю немало примеров, не подтверждающих полученные результаты. Однако аналитика данных зачастую связана с обобщениями, при которых частные случаи приносятся в жертву общим тенденциям.

Зарабатывают ли студенты заведений, входящих в расширенную Лигу плюща, больше, чем средний выпускник частного заведения? И если да, то насколько?

Давайте еще немного поговорим о частных учебных заведениях. Школьники часто стремятся попасть в самые известные университеты и колледжи в надежде получить лучшее образование и зарабатывать больше. Но как связаны факт окончания престижного заведения и успешность будущей карьеры? В этом задании мы сравним среднюю зарплату выпускников заведений, входящих в расширенную Лигу плюща, с зарплатой среднего выпускника частного заведения.

Для этого воспользуемся методом `isin` в селекторе столбцов, как показано ниже:

```
ivy_plus = ['Harvard University',
            'Massachusetts Institute of Technology',
            'Yale University',
            'Columbia University in the City of New York',
            'Brown University',
            'Stanford University',
            'University of Chicago',
            'Dartmouth College',
            'University of Pennsylvania',
            'Cornell University',
            'Princeton University']

(
    institutions_df
    .loc[institutions_df['INSTNM'].isin(ivy_plus),
         'MD_EARN_WNE_P10']
    .mean()
)
```

Вывод:

91806.81818181818

Таким образом, мы можем сделать вывод, что выпускники заведений, входящих в расширенную Лигу плюща, в среднем примерно вдвое больше зарабатывают по прошествии 10 лет после выпуска в сравнении со средним студентом частного института.

Сколько в среднем зарабатывают студенты заведений из разных штатов через 10 лет после выпуска?

Наконец, мы можем сравнить зарплаты выпускников учебных заведений по штатам. Для этого выполним группировку по полю `STABBR` и рассчитаем средние значения, как показано ниже:

```
institutions_df.groupby('STABBR')['MD_EARN_WNE_P10'].mean()
```

Разумеется, анализировать подобные данные удобнее, когда они упорядочены. Так что добавим сортировку:

```
(
    institutions_df
    .groupby('STABBR', observed=True)
    ['MD_EARN_WNE_P10'].mean()
    .sort_values(ascending=False)
)
```

Вывод:

```
STABBR
MA      53234.396226
RI      50432.789474
```

```

DC      49081.470588
CT      48662.017857
VT      48383.857143
...
GU      26182.333333
MP      24972.000000
FM      22919.000000
PR      21613.220339
PW      NaN
Name: MD_EARN_WNE_P10, Length: 59, dtype: float64

```

Постройте столбчатую диаграмму, показывающую среднюю зарплату студентов через 10 лет после выпуска по штатам, с сортировкой по возрастанию суммы

Теперь давайте визуализируем данные, полученные на предыдущем шаге, в виде столбчатой диаграммы, показанной на рис. 13.5:

```

(
    institutions_df
    .groupby('STABBR', observed=True)
    ['MD_EARN_WNE_P10'].mean()
    .sort_values()
    .plot.bar(figsize=(20,10))
)

```

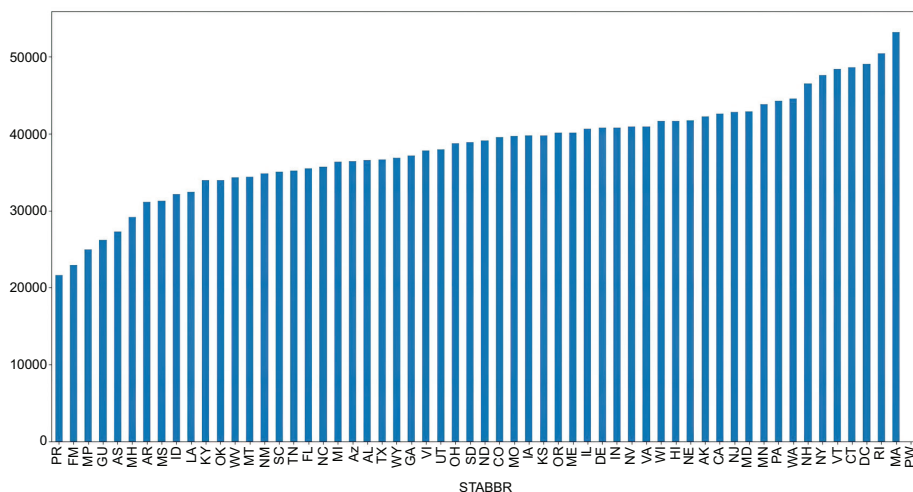


Рис. 13.5. Столбчатая диаграмма со средними зарплатами выпускников по штатам

Сортировка по возрастанию в этом случае более применима. Как видим, выпускники учебных заведений в Массачусетсе и Род-Айленде в среднем зарабатывают намного больше выпускников из штатов Арканзас и Миссисипи. Но чтобы делать серьезные выводы о качестве заведений, сначала нужно понять, какая доля выпускников остается жить в том штате, в котором выпускалась. В конце концов, стоимость проживания в регионе Новая Англия гораздо выше, чем в Ар-

канзасе или Миссисипи. Так что вполне логично, что бывшие студенты, поселившиеся в этом регионе, будут зарабатывать больше вне зависимости от того, какой институт они закончили.

Постройте диаграмму размаха по тем же данным

В завершение нас попросили построить диаграмму размаха на основе полученных выше данных, чтобы мы могли легко проанализировать их распределение. Результат показан на рис. 13.6:

```
(
    institutions_df
    .groupby('STABBR', observed=True)
    ['MD_EARN_WNE_P10'].mean()
    .plot.box()
)
```

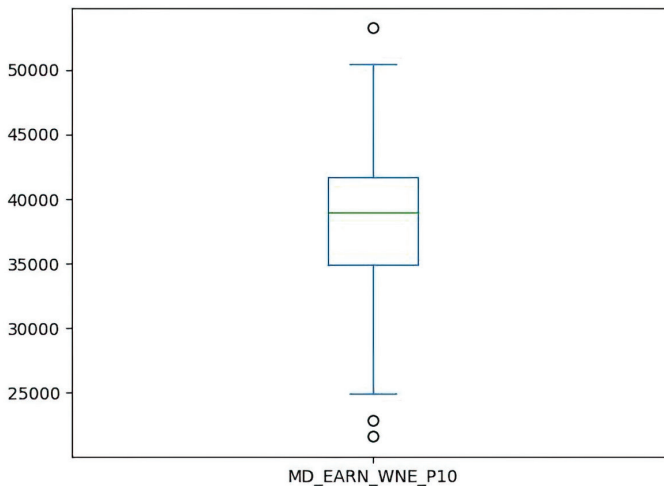


Рис. 13.6. Диаграмма размаха по средней зарплате бывших студентов

На этой диаграмме видно, что подавляющее большинство годовых зарплат находится в интервале от 25 000 до 50 000 долл., а медианное значение располагается чуть ниже отметки в 40 000 долл.

Заключение

Ну вот вы и добрались до последней страницы книги. Спасибо за то, что вместе со мной проделали этот долгий путь. Я очень надеюсь, что упражнения из этой книги, включая дополнительные, позволили вам приобрести необходимые навыки в области использования библиотеки pandas, и вы сможете применять их на практике при решении самых разных задач.

Помимо методов, способов и приемов, применяемых в pandas, я постарался сделать так, чтобы вы впитали концепции, применяемые в этой библиотеке, чтобы они стали частью вас. Pandas – это огромная и постоянно расширяющаяся библиотека, и вы не встретите никого, кто разобрался бы в ней досконально. Но вы должны стремиться к ощущению того, что, столкнувшись с задачей, вы точно сможете найти ответы при помощи pandas и даже точно знаете, в какой области искать.

Я желаю вам огромного успеха в анализе данных с помощью pandas и надеюсь, что эта книга действительно помогла вам лучше понять внутреннее устройство этой богатой библиотеки. Если это так и есть, можете черкнуть мне пару строк на адрес reuven@lerner.co.il, я с радостью прочитаю!

Предметный указатель

Символы

// 43
& 120
| 186
~ 197
*args 297
**kwargs 297
%%timeit 509
%timeit 509

А

Агрегирующий метод 33

Б

Бродкастинг 38
Булев индекс 47

В

Векторизация 37
Внешнее объединение 254, 308
Внутреннее объединение 313
Временной ряд 392
Выборочное стандартное отклонение 33
Выброс 98, 426

Г

Генератор списков 320
Гистограмма 424
Группировка данных 228

Д

Датафрейм 19, 69
Диаграмма размаха 425
Дисперсия 32

И

Индекс 20, 152
Индекс-маска 47
Интерполяция 104

К

Коробчатая диаграмма 425
Корреляция 450
Корреляция Пирсона 309
Кортеж 163
Коэффициент корреляции Пирсона 451
Круговая диаграмма 421

М

Медиана 31
Межквартильный размах 98
Множественный индекс 152, 161
Мода 366

Н

Нормализация 254
Нормальное распределение 33

О

Объединение данных 228, 249
Объединение слева 254
Оконная функция 276
Описательная статистика 52

П

Передискретизация 395
Правое объединение 308
Причудливая индексация 49

Р

Расширяющееся окно 276

С

Самообъединение 308
Сводная таблица 152, 182
Скользящая оконная функция 277
Сортировка данных 228
Среднее значение 31
Срез 27
Стандартное отклонение 32
Столбчатая диаграмма 416

Т

Точечная нотация 72
Транслирование 38

Я

Ящик с усами 425

А

add 541
agg 145, 241
Apache Arrow 495
apply 327, 347
assign 78, 270

astype 44

C

category 488
copy 102, 135
corr 452

D

DataFrameGroupBy 241
def 290
describe 53
df.columns 281
df.dropna() 135
df.info 494
df.rename 281
display.max_colwidth 362
drop 304
drop_duplicates 296, 529
dt 379
dtype 34
dtypes 131, 382

E

eval 505
expanding 276
explode 352

F

f-строка 27
feather 495
fillna 44, 102
filter 289
float_format 79

G

get 44
glob.glob 270
groupby 240

H

head 29

I

idxmax 144, 400
idxmin 54, 144, 400
iloc 28
include_lowest 64
index 26
index_col 140
IndexSlice 181

info 131
inplace 157
integers 24
intersection 365
isdigit 208
isin 105
is_lexsorted 171
is_monotonic_decreasing 171
isna 197
isnan 102
isnull 196

J

JSON 148

K

KeyError 72

L

lambda 290
len 199, 353
loc 28
low_memory 132
lsuffix 308

M

Matplotlib 413
mean 19, 31
memory_usage 486
mode 219

N

NaN 31, 102
notnull 197
np.default_rng 24
np.isnan 197
np.nan 102
np.NaN 102
np.random.seed 25
np.size 199
numexpr 504
NumPy 24

O

object 44, 338
os.stat 500

P

pandas 19
pct_change 277

pd.CategoricalDtype 490
 pd.concat 89, 125, 369
 pd.cut 63, 247
 pd.eval 505
 pd.from_feather 496
 pd.MultiIndex.from_tuples 318
 pd.NA 102, 338
 pd.read_csv 60
 pd.read_html 146
 pd.read_json 148
 pd.StringDtype 338
 pd.StringDtype 46
 pd.to_numeric 210
 pd.to_timedelta 380
 pip 13
 pivot 184
 plot 417
 plot.box 425
 plot.hist 447
 plot.scatter 422, 453
 PyArrow 495
 PyPI 12

Q

quantile 53, 99
 query 92, 504

R

randint 25
 random 25
 range 88
 read 344
 replace 223
 requests 142
 resample 399
 resampling 395
 rolling 277
 round 30, 58
 rsuffix 308

S

sample 475
 scipy.stats.trimboth 101
 scipy.stats.zscore 101
 Seaborn 413, 462
 Series 19
 set_index 155
 SettingWithCopyWarning 102
 slice 165, 206, 208
 sns.catplot 462, 463
 sns.displot 463

sns.regplot 463
 sns.relplot 462
 sort_index 171
 sort_values 237
 squeeze 60
 std 31
 str 44, 206, 340
 str.contains 330
 str.get_dummies 369
 str.index 340
 string 345
 StringIO 142
 string.punctuation 345
 str.isdigit 340
 str.isspace 340
 str.replace 340
 str.rsplit 319
 str.split 269, 319
 str.strip 345

T

T 282, 536
 tail 29
 time 498
 timedelta 374, 380
 timeit 498
 time.perf_counter 498
 timestamp 374
 Timestamp 377
 to_csv 389
 to_datetime 377
 to_feather 496
 to_frame 307
 transform 290
 transpose 282

V

ValueError 35

X

xs 180

Реувен Лернер

Python: Pandas на практике

200 упражнений по анализу данных с решениями и пояснениями

Главный редактор *Мовчан Д. А.*
Зам. главного редактора *Яценков В. С.*
editor@dmkpress.com

Перевод *Гинько А. Ю.*
Корректор *Абросимова Л. А.*
Верстка *Паранская Н. В.*
Дизайн обложки *Мовчан А. Г.*

Формат 70×100¹/₁₆.
Печать цифровая. Усл. печ. л. 44.85.
Тираж 100 экз.

Веб-сайт издательства: **www.dmkpress.com**

Книга рассказывает о библиотеке `pandas`, которая существенно облегчает и ускоряет процесс анализа, очистки, исследования и манипулирования данными в Python. Единственным способом освоить `pandas` и уверенно применять ее в своей работе, являются регулярные практические занятия. Опытный знаток Python Реувен Лернер поделится с вами опытом и поможет осилить этот непростой путь.

Прочитав эту книгу и самостоятельно выполнив 200 упражнений, вы сможете достичь небывалого уровня владения `pandas`. Все упражнения основаны на реальных наборах данных. Автор уделяет особое внимание процедурам загрузки данных, их очистки, визуализации и оптимизации.

Рассматриваемые темы:

- очистка данных с минимумом ручного труда;
- извлечение и присваивание данных;
- обработка и манипулирование строковыми данными;
- вычисления, применяемые к выборочным наборам данных.

Издание адресовано программистам на Python и аналитикам данных.

Реувен Лернер - практикующий преподаватель Python и науки о данных и автором еженедельной рассылки `Bamboo Weekly` с задачами по `pandas`. Автор книги `Python Workout` (2020).



«Тщательно выстроенный путеводитель в мир `pandas`, наполненный новыми знаниями и идеями, которые помогут вам более грамотно работать с этой библиотекой и анализировать данные».

*Кевин Маркэм, основатель
онлайн-школы Data School,
создатель курса
«Pandas in 30 days»*

«Книга поможет вам применять `pandas` для решения ежедневных задач и выведет вас на новый уровень».

*Майкл Дрисколл,
RFA Engineering, автор курса
«Teach me Python»*

«Умение Реувена объяснять сложные вещи значительно облегчает обучение. Больше точно не ошибусь!» »

Родриго Жиран Серран

«Лучший источник знаний о `pandas`!»

Киран Ананта, Amazon

ISBN 978-5-93700-227-3



9 785937 002273 >